PDF Digital Signatures — A Technical Reference

PDFalyzer Studio Contributors

2026-03-06

Contents

0.1 Preface . . . 1
0.1.1 Who This Book Is For . . . 1
0.1.2 What This Book Is Not . . . 1
0.1.3 How This Book Is Organized . . . 1
0.1.4 Conventions Used in This Book . . . 2
0.1.5 A Note on the PDF Specification . . . 3
0.1.6 Three Decades of PDF: A Timeline . . . 3
0.1.7 Acknowledgments . . . 10

1 Cryptographic Foundations 13

2 Asymmetric Cryptography Primer 15
2.1 The Key Distribution Problem . . . 15
2.2 Key Pairs: The Fundamental Asymmetry . . . 16
2.3 RSA: The Workhorse . . . 16
2.3.1 The Mathematical Basis . . . 16
2.3.2 Why Large Primes Matter . . . 17
2.3.3 RSA Key Sizes vs Security Levels . . . 17
2.3.4 Performance Impact of Key Size . . . 18
2.3.5 Why RSA-1024 Persists in the Wild . . . 19
2.4 Elliptic Curve Cryptography: Smaller Keys, Same Security . . . 19
2.4.1 Elliptic Curve Basics . . . 19
2.4.2 The NIST Curves: P-256, P-384, P-521 . . . 20
2.4.3 Why ECDSA Signatures Are Smaller . . . 20
2.4.4 ECDSA Adoption in PDF Signatures . . . 21
2.5 EdDSA and Ed25519: The Future Direction . . . 21
2.5.1 Bernstein's Work and Design Philosophy . . . 22
2.5.2 Ed25519 and Ed448 . . . 22
2.5.3 EdDSA and PDF: Adoption Status . . . 22
2.6 Algorithm Comparison . . . 23
2.7 Connecting to PDF: OIDs and SignerInfo . . . 24
2.8 Java's Cryptographic Provider Model and BouncyCastle . . . 25
2.8.1 JCA, JCE, and the Provider Architecture . . . 25
2.8.2 Why BouncyCastle Exists . . . 25
2.8.3 Provider Registration . . . 26
2.9 Practical Code: Generating Key Pairs . . . 26
2.9.1 RSA Key Pair Generation . . . 26
2.9.2 ECDSA Key Pair Generation . . . 27
2.9.3 Performance Comparison . . . 28
2.10 Choosing an Algorithm for PDF Signatures . . . 29

2.11 Summary . . . 29

3 Hash Functions & Digital Signing **31**
3.1 What Hash Functions Do . . . 31
3.2 Why We Hash Before Signing . . . 32
3.3 The SHA Family . . . 33
3.3.1 SHA-1: The Algorithm That Refused to Die . . . 33
3.3.2 SHA-256, SHA-384, SHA-512: The Modern Workhorses . . . 35
3.4 MD5: The Ghost in the Machine . . . 36
3.4.1 The Wang-Yu Attacks (2004) . . . 36
3.4.2 Why MD5 Still Appears in Legacy PDFs . . . 36
3.5 The Signing Process Step by Step . . . 37
3.5.1 Step 1: Hash the Document . . . 37
3.5.2 Step 2: Pad the Hash . . . 37
3.5.3 Step 3: Encrypt with Private Key . . . 38
3.5.4 Step 4: Package as CMS SignedData . . . 38
3.6 The Verification Process Step by Step . . . 38
3.6.1 Step 1: Extract the CMS SignedData . . . 38
3.6.2 Step 2: Re-Hash the Byte Range . . . 38
3.6.3 Step 3: Compare Hashes . . . 39
3.6.4 Step 4: Verify the Cryptographic Signature . . . 39
3.7 DigestMethod in PDF Signatures . . . 39
3.8 Hash Algorithm OIDs . . . 40
3.9 Signature Algorithm OIDs . . . 41
3.10 When Hash and Signature Algorithms Collide . . . 42
3.11 Java: Computing Hashes . . . 42
3.12 Java: Signing with BouncyCastle . . . 44
3.13 Debugging Hash Mismatches . . . 45
3.14 Summary . . . 47
3.15 X.509 Certificates . . . 47
3.15.1 What a Certificate Is . . . 47
3.15.2 ASN.1 and DER Encoding . . . 48
3.15.3 Certificate Structure . . . 50
3.15.4 Certificate Extensions . . . 52
3.15.5 Certificate Profiles . . . 55
3.15.6 Common Certificate Problems in the Wild . . . 57
3.15.7 Java: Loading and Inspecting Certificates . . . 58
3.15.8 Dissecting a Real Signing Certificate . . . 61
3.15.9 Summary . . . 63

4 Certificate Chains & Trust **65**
4.1 Why Chains Exist . . . 65
4.2 Chain Structure in Detail . . . 66
4.3 Building the Chain . . . 67
4.3.1 AuthorityKeyIdentifier and SubjectKeyIdentifier . . . 67
4.3.2 Fallback: Issuer Distinguished Name Matching . . . 69
4.4 Path Validation: RFC 5280 Section 6 . . . 69
4.4.1 The Six Steps . . . 69
4.4.2 Validity Period Checking and Signing Time . . . 71

4.5 Revocation Checking . . . 72
4.5.1 CRL Deep Dive . . . 72
4.5.2 OCSP Deep Dive . . . 73
4.5.3 CRL vs OCSP: When to Use Which . . . 75
4.6 Trust Stores . . . 77
4.6.1 Adobe Approved Trust List (AATL) . . . 77
4.6.2 EU Trusted List (EUTL) . . . 78
4.6.3 Operating System Trust Stores . . . 79
4.6.4 Java cacerts . . . 80
4.6.5 Custom Trust Stores . . . 81
4.7 Trust Store Comparison . . . 82
4.8 Debugging Chain Failures . . . 82
4.8.1 Step 1: Extract the Certificates . . . 82
4.8.2 Step 2: Build the Chain Manually . . . 82
4.8.3 Step 3: Check the Trust Anchor . . . 83
4.8.4 Step 4: Check Validity Periods . . . 83
4.8.5 Step 5: Check Revocation . . . 83
4.8.6 Step 6: Check Key Usage . . . 83
4.9 Cross-Certification and Bridge CAs . . . 83
4.10 Summary . . . 84

5 Key Material Formats **85**
5.1 PKCS#12 / PFX: The Universal Container . . . 85
5.1.1 History . . . 85
5.1.2 Internal Structure . . . 85
5.1.3 Loading PKCS#12 in Java . . . 86
5.2 PEM: The Text Format . . . 87
5.2.1 History . . . 87
5.2.2 Multiple Objects in One File . . . 88
5.2.3 Loading PEM with BouncyCastle . . . 88
5.3 DER: The Binary Encoding . . . 90
5.3.1 Relationship to PEM . . . 90
5.3.2 When DER Matters . . . 90
5.4 JKS: Java KeyStore (Legacy) . . . 91
5.4.1 Why JKS Is Being Replaced . . . 91
5.5 PKCS#8: Private Key Format . . . 91
5.5.1 Unencrypted vs Encrypted . . . 91
5.6 PKCS#1: RSA-Specific Key Format . . . 92
5.7 Format Conversion Reference . . . 93
5.8 HSM Integration: When Keys Cannot Leave . . . 94
5.8.1 PKCS#11 . . . 94
5.8.2 Why This Matters for PDF Signing . . . 95
5.9 Why Format Conversion Goes Wrong . . . 95
5.9.1 Password Encoding Mismatch . . . 95
5.9.2 OpenSSL Legacy Format vs PKCS#8 . . . 95
5.9.3 Certificate Chain Order in PKCS#12 . . . 95
5.9.4 DER vs PEM Confusion . . . 96
5.10 Summary . . . 96

6 CMS/PKCS#7 Signed Data **99**
6.1 From PKCS#7 to CMS: A History . . . 99
6.1.1 PKCS#7: RSA Laboratories, 1993 . . . 99
6.1.2 S/MIME and RFC 2315, 1998 . . . 100
6.1.3 CMS: RFC 2630 (1999) and RFC 3852 (2004) . . . 100
6.1.4 RFC 5652: The Current Standard (2009) . . . 100
6.2 Why PDF Uses CMS . . . 101
6.3 The ContentInfo Wrapper . . . 102
6.4 The SignedData Structure . . . 102
6.4.1 Version Number Rules . . . 103
6.4.2 The digestAlgorithms SET . . . 104
6.4.3 EncapsulatedContentInfo: Detached vs Encapsulated . . . 104
6.4.4 The certificates Field . . . 105
6.5 SignerInfo: The Core of the Signature . . . 105
6.5.1 SignerIdentifier: Finding the Signer's Certificate . . . 105
6.5.2 Signed Attributes: What Gets Signed . . . 106
6.5.3 The ESS-signing-certificate-v2 Attribute . . . 107
6.6 The Signed Attributes DER Encoding Trap . . . 107
6.6.1 The Problem . . . 107
6.6.2 Why This Exists . . . 108
6.7 Unsigned Attributes: Post-Signature Additions . . . 109
6.8 CAdES: CMS Advanced Electronic Signatures . . . 109
6.9 Old PKCS#7 vs Modern CMS: What Changed . . . 110
6.10 Java: Parsing CMS SignedData . . . 111
6.11 Java: Building CMS SignedData . . . 113
6.12 Annotated ASN.1 Dump of a Real PDF Signature . . . 115
6.13 Summary . . . 116

7 PDF Internals **117**

8 PDF File Structure **119**
8.1 The Four Sections . . . 119
8.1.1 The Header . . . 119
8.1.2 The Body . . . 120
8.1.3 The Cross-Reference Table . . . 121
8.1.4 The Trailer . . . 122
8.2 PDF Object Types . . . 123
8.2.1 Boolean, Integer, Real, Null . . . 123
8.2.2 String (Literal and Hexadecimal) . . . 123
8.2.3 Name . . . 123
8.2.4 Array . . . 123
8.2.5 Dictionary . . . 124
8.2.6 Stream . . . 124
8.2.7 Indirect Object and Indirect Reference . . . 124
8.3 Cross-Reference Streams . . . 124
8.3.1 Why They Exist . . . 124
8.3.2 Structure . . . 125
8.4 Object Streams . . . 126
8.5 Hex Dump Walkthrough: A Minimal Valid PDF . . . 126

8.6 How Different Generators Produce Different Structures . . . 127
8.7 Why File Structure Matters for Signatures . . . 128
8.8 PDFBox COS Layer: Mapping PDF Types to Java . . . 129
8.9 Common Structural Problems and Debugging . . . 130
8.9.1 Incorrect /Length Values . . . 130
8.9.2 Missing /ID Array . . . 131
8.9.3 Multiple %%EOF Markers . . . 131
8.9.4 Trailing Bytes After Final %%EOF . . . 131
8.9.5 Cross-Reference Table Corruption . . . 131
8.10 What Comes Next . . . 131

9 Incremental Updates 133
9.1 What Incremental Updates Are . . . 133
9.2 Why Incremental Updates Exist . . . 133
9.2.1 Problem 1: Performance on Large Documents . . . 133
9.2.2 Problem 2: Preserving Previous State . . . 134
9.2.3 Problem 3: Digital Signatures . . . 134
9.3 Anatomy of an Incremental Update . . . 134
9.3.1 The Original File . . . 134
9.3.2 Appending an Incremental Update . . . 135
9.3.3 The `/Prev` Chain . . . 136
9.4 How Objects Are Overridden . . . 137
9.5 Cross-Reference Streams and Incremental Updates . . . 137
9.6 Why Incremental Updates Are Critical for Signatures . . . 138
9.6.1 The Fundamental Constraint . . . 138
9.6.2 Signing Is an Incremental Update . . . 139
9.6.3 Multiple Signatures . . . 139
9.6.4 Form Filling After Signing . . . 139
9.7 What Goes Wrong: Flattening and Full Rewrites . . . 140
9.7.1 The Flattening Problem . . . 140
9.7.2 The ”Save” vs ”Save As” Trap . . . 140
9.8 Extracting a Specific Revision . . . 141
9.8.1 Extracting with PDFBox . . . 141
9.8.2 Extracting with QPDF . . . 142
9.9 How PDFBox Handles Incremental Saves . . . 142
9.9.1 `doc.save()` — Full Rewrite . . . 143
9.9.2 `doc.saveIncremental()` — Append Mode . . . 143
9.9.3 The `saveIncremental()` and Signing Workflow . . . 143
9.10 Multiple `%%EOF` Markers: What They Mean . . . 144
9.10.1 What Revision Counts Tell You About a Signed PDF . . . 144
9.11 Tracing the Revision History . . . 145
9.12 Linearized PDFs and Incremental Updates . . . 145
9.13 The Incremental Saving Attack . . . 146
9.14 Debugging Incremental Updates . . . 147
9.14.1 QPDF: Your Best Friend . . . 147
9.14.2 Comparing Revisions . . . 147
9.14.3 Hex-Level Inspection . . . 148
9.15 Real-World Scenarios . . . 148

9.15.1 Scenario 1: Single Signature . . . 148
9.15.2 Scenario 2: Two Approval Signatures . . . 148
9.15.3 Scenario 3: Certification + Approval . . . 149
9.15.4 Scenario 4: Annotations After Signing . . . 149
9.15.5 Scenario 5: The Flattened Disaster . . . 149
9.15.6 Scenario 6: The Double-Linearization Trap . . . 149
9.16 Summary . . . 150

10 AcroForm & Interactive Forms **151**
10.1 The AcroForm Dictionary . . . 151
10.1.1 /Fields — The Root of the Field Tree . . . 151
10.1.2 /SigFlags — The Two Bits That Protect Everything . . . 152
10.1.3 /NeedAppearances — The Flag That Breaks Signatures . . . 153
10.1.4 /DA — Default Appearance String . . . 153
10.1.5 /DR — Default Resources . . . 153
10.2 Field Tree Hierarchy . . . 154
10.2.1 Parent-Child Relationships and Fully Qualified Names . . . 154
10.2.2 Terminal vs. Non-Terminal Fields . . . 154
10.3 Field Types . . . 154
10.3.1 /Tx — Text Fields . . . 154
10.3.2 /Btn — Button Fields . . . 154
10.3.3 /Ch — Choice Fields . . . 155
10.3.4 /Sig — Signature Fields . . . 155
10.4 Widget Annotations . . . 156
10.4.1 The Connection Between Fields and Pages . . . 156
10.4.2 Page Association . . . 156
10.5 Appearance Streams . . . 156
10.5.1 Normal, Rollover, and Down . . . 156
10.5.2 Why Appearance Streams Matter for Signatures . . . 157
10.5.3 Generating Appearances with PDFBox . . . 157
10.6 XFA Forms: The Deprecated Complication . . . 157
10.6.1 What XFA Was and Why It Was Deprecated . . . 157
10.6.2 Why XFA Still Causes Problems . . . 158
10.7 Form Flattening and Signatures . . . 158
10.8 PDFBox AcroForm API . . . 158
10.8.1 PDAcroForm: The Entry Point . . . 158
10.8.2 Field Tree Traversal . . . 159
10.8.3 PDSignatureField and PDSignature . . . 159
10.8.4 Creating Signature Fields Programmatically . . . 159
10.9 Common Form/Signature Interaction Bugs . . . 160
10.9.1 Missing Appearance Streams . . . 160
10.9.2 Wrong Field Hierarchy . . . 160
10.9.3 SigFlags Not Set or Wrong . . . 161
10.9.4 Orphaned Signature Fields . . . 161
10.9.5 Acrobat vs. PDFBox Rendering Differences . . . 161
10.9.6 Real-World Scenario: The Government Tax Form . . . 161

11 PDF Coordinate System **163**
11.1 User Space: The Default Coordinate System . . . 163

11.2 Page Boxes: The Five Rectangles . . . 164
11.2.1 MediaBox (Required) . . . 164
11.2.2 CropBox . . . 164
11.2.3 BleedBox . . . 164
11.2.4 TrimBox . . . 165
11.2.5 ArtBox . . . 165
11.2.6 Which Box Matters for Signatures? . . . 165
11.3 Annotation Rectangles . . . 165
11.4 Why Signature Positioning Goes Wrong . . . 166
11.5 Page Rotation . . . 166
11.5.1 Rotation and Signature Placement . . . 167
11.6 Transformation Matrices and the CTM . . . 168
11.7 Signature Appearance Rectangles and Page Coordinates . . . 169
11.8 PDFBox Coordinate Handling . . . 169
11.8.1 Practical Example: Placing a Visible Signature . . . 170
11.9 Common Coordinate Bugs . . . 171
11.9.1 Flipped Signatures (Y-axis Inversion) . . . 171
11.9.2 Signatures on the Wrong Page Position (Rotated Pages) . . . 171
11.9.3 Zero-Area Annotation (Invisible Signature Becomes Truly Invisible) 171
11.9.4 Signature Appears Off-Page . . . 172
11.9.5 Distorted Signature Appearance . . . 172
11.10 Debugging Coordinates with QPDF and RUPS . . . 172
11.10.1 QPDF . . . 172
11.10.2 RUPS (Reading and Updating PDF Syntax) . . . 172
11.10.3 PDFBox Programmatic Inspection . . . 173
11.11 Summary . . . 173

12 Font Embedding in PDF **175**
12.1 Why Fonts Matter for Digital Signatures . . . 175
12.2 Font Types in PDF . . . 176
12.2.1 Type 1 Fonts (PostScript Era) . . . 176
12.2.2 The Standard 14 Fonts . . . 176
12.2.3 TrueType Fonts . . . 178
12.2.4 Type 0 / CIDFont (Composite Fonts) . . . 178
12.3 Font Embedding vs. Referencing . . . 179
12.3.1 Referenced Fonts . . . 179
12.3.2 Embedded Fonts . . . 180
12.4 Font Subsetting . . . 180
12.4.1 The Subset Prefix Convention . . . 180
12.4.2 How PDFBox Handles Subsetting . . . 181
12.5 Encoding . . . 181
12.5.1 Simple Font Encoding (Single-Byte) . . . 182
12.5.2 The 256-Glyph Wall . . . 182
12.5.3 Composite Font Encoding . . . 182
12.6 The ToUnicode CMap . . . 183
12.7 Font Descriptors . . . 184
12.8 PDFBox Font Handling in 3.x . . . 184
12.8.1 Loading Fonts . . . 185

12.8.2 The getEncoding() Removal . . . 185
12.8.3 Inspecting Font Embedding Programmatically . . . 185
12.9 Why Signature Text Shows as Squares or Blanks . . . 186
12.9.1 Missing Font . . . 186
12.9.2 Wrong Encoding . . . 186
12.9.3 Unembedded Font with Non-Standard Glyphs . . . 186
12.9.4 Subset Missing Glyphs . . . 186
12.10 Font Licensing and the fsType Flag . . . 187
12.11 Practical Advice for Signature Appearances . . . 188
12.11.1 For ASCII-Only Signatures . . . 188
12.11.2 For International Signatures . . . 188
12.11.3 Font Size Guidelines . . . 188
12.12 Debugging Font Issues . . . 189
12.12.1 Step 1: Identify the Font . . . 189
12.12.2 Step 2: Check Embedding . . . 189
12.12.3 Step 3: Check Encoding . . . 189
12.12.4 Step 4: Check the ToUnicode CMap . . . 189
12.12.5 Step 5: veraPDF Validation . . . 189
12.12.6 Step 6: PDFBox Font Inspection . . . 190
12.13 Summary . . . 190

13 PDF Digital Signatures 193
13.1 Signature Architecture . . . 193
13.1.1 Overview: What a PDF Signature Actually Is . . . 193
13.1.2 Signature Fields in the AcroForm Tree . . . 194
13.1.3 The Signature Dictionary (`/Type /Sig`) . . . 198
13.1.4 The `/Contents` Value: CMS SignedData . . . 203
13.1.5 The Signing Process: Step by Step . . . 207
13.1.6 Incremental Save: The Foundation of Multi-Signature . . . 209
13.1.7 Widget Annotations and Page Placement . . . 210
13.1.8 AcroForm `/SigFlags`: Document-Level Signature Metadata . . . 211
13.1.9 PDFBox API Mapping . . . 211
13.1.10 Common Mistakes and How to Avoid Them . . . 213
13.1.11 Key Points . . . 214
13.1.12 Specification References . . . 215

14 Byte Range Deep Dive 217
14.1 The Byte Range Model . . . 217
14.1.1 What the Spec Says . . . 217
14.1.2 Visualizing the Byte Range . . . 218
14.1.3 Why the Gap Exists: The Chicken-and-Egg Problem . . . 218
14.2 The Gap in Detail . . . 219
14.2.1 What's Actually in the Gap . . . 219
14.2.2 The Zero-Padding Question . . . 220
14.3 Incremental Saves and the Byte Range . . . 220
14.3.1 How Signing Creates an Incremental Update . . . 220
14.3.2 What Happens After Signing: The Trailing Bytes Problem . . . 221
14.3.3 Multi-Signature Scenarios . . . 222
14.3.4 Permission-Based Restrictions: What Can Change After Signing? . . . 223

14.4 Size Estimation: How Much Space to Reserve . . . 224
14.4.1 The Problem . . . 224
14.4.2 Estimation Strategies . . . 225
14.4.3 What Happens When the Estimate Is Wrong . . . 225
14.5 Byte Range Verification: How to Check It . . . 226
14.5.1 The Verification Algorithm . . . 226
14.5.2 Implementation in Java . . . 226
14.6 Hex Dump Walkthrough: A Real Signed PDF . . . 228
14.6.1 Finding the Signature Dictionary . . . 228
14.6.2 Examining the Gap Boundaries . . . 228
14.6.3 Verifying No Trailing Bytes . . . 229
14.7 Common Byte Range Problems . . . 229
14.7.1 Problem 1: Trailing Bytes After Signed Region . . . 229
14.7.2 Problem 2: `/Contents` Size Exceeded . . . 229
14.7.3 Problem 3: ByteRange Doesn't Start at Zero . . . 230
14.7.4 Problem 4: Gap Contains More Than Just `/Contents` . . . 230
14.7.5 Problem 5: Hash Algorithm Mismatch . . . 230
14.8 Key Takeaways . . . 230

15 Approval vs. Certification Signatures 233
15.1 The Two Signature Types . . . 233
15.1.1 Approval Signatures . . . 233
15.1.2 Certification Signatures . . . 234
15.2 The DocMDP Permission Model . . . 235
15.2.1 Permission Level 1: No Changes Allowed . . . 235
15.2.2 Permission Level 2: Form Filling and Signing Only . . . 235
15.2.3 Permission Level 3: Form Filling, Signing, and Annotations . . . 235
15.2.4 What "Invalidates the Certification" Actually Means . . . 236
15.3 Lock Dictionaries: Field-Level Protection . . . 236
15.3.1 What Lock Dictionaries Do . . . 236
15.3.2 Lock Actions . . . 236
15.3.3 The `/Fields` Array . . . 237
15.4 FieldMDP: Fine-Grained Modification Detection . . . 237
15.4.1 FieldMDP vs. DocMDP . . . 237
15.5 Interaction Rules: When Multiple Signatures Coexist . . . 238
15.5.1 The Certification + Approval Pattern . . . 238
15.5.2 What Happens When Permissions Conflict . . . 239
15.6 Implementation: Certification Signatures with PDFBox . . . 239
15.6.1 Creating a Certification Signature . . . 239
15.6.2 Adding Lock Dictionaries . . . 240
15.7 Seed Value Dictionaries: Constraining How a Signature Field Is Signed . . . 240
15.7.1 Purpose of Seed Values . . . 240
15.7.2 Key Seed Value Entries . . . 241
15.7.3 Timestamp Seed Values . . . 242
15.8 Real-World Patterns and Mistakes . . . 242
15.8.1 Mistake 1: Using Certification When Approval Is Needed . . . 242
15.8.2 Mistake 2: Not Setting MDP When It's Needed . . . 243
15.8.3 Mistake 3: Lock Dictionary with Wrong Field Names . . . 243

15.8.4 Mistake 4: Assuming All Viewers Enforce MDP . . . 243
15.9 Key Takeaways . . . 243

16 Visual Appearance of Signatures **245**
16.1 Appearance Streams: The Basics . . . 245
16.1.1 What an Appearance Stream Is . . . 245
16.1.2 Visible vs. Invisible Signatures . . . 246
16.2 The Layer Model: n0, n2, and n4 . . . 247
16.2.1 Adobe's Legacy Layer System . . . 247
16.2.2 What This Means for Your Implementation . . . 247
16.3 Creating Signature Appearances with PDFBox . . . 247
16.3.1 Basic Text Appearance . . . 247
16.3.2 Appearance with Signature Image . . . 248
16.3.3 Attaching the Appearance to the Signature . . . 249
16.4 Appearance Rendering Across Viewers . . . 250
16.4.1 How Different Viewers Handle Appearances . . . 250
16.4.2 The `/NeedAppearances` Problem . . . 251
16.5 Positioning: Getting the Rectangle Right . . . 251
16.5.1 The Coordinate System . . . 251
16.5.2 Handling Page Rotation . . . 252
16.6 Security Implications of Visual Appearances . . . 252
16.6.1 The Appearance/Content Disconnect . . . 252
16.6.2 Shadow Attacks and Visual Appearances . . . 253
16.7 Common Appearance Problems . . . 253
16.7.1 Problem: Blank Signature Rectangle . . . 253
16.7.2 Problem: Text Appears Clipped or Off-Screen . . . 253
16.7.3 Problem: Font Not Rendering . . . 254
16.8 Key Takeaways . . . 254

17 Signature Validation **255**
17.1 The Five Steps of Signature Validation . . . 255
17.1.1 Step 1: Structural Integrity (Byte Range) . . . 255
17.1.2 Step 2: Hash Verification (Digest Match) . . . 256
17.1.3 Step 3: Cryptographic Verification (CMS Signature) . . . 256
17.1.4 Step 4: Certificate Chain Validation . . . 257
17.1.5 Step 5: Trust Evaluation . . . 258
17.2 Validation Differences Across Viewers . . . 259
17.3 Debugging Validation Failures . . . 259
17.3.1 The Systematic Approach . . . 259
17.3.2 Common Error Messages and Their Meaning . . . 260
17.4 Timestamp Verification . . . 261
17.4.1 Why Timestamps Matter for Validation . . . 261
17.4.2 Verifying the Timestamp . . . 261
17.4.3 Timestamp Chain of Trust . . . 262
17.5 Building a Complete Validation Pipeline . . . 262
17.5.1 Architecture . . . 262
17.5.2 When to Use EU DSS Instead . . . 262
17.6 Key Takeaways . . . 263

18 PAdES Profiles (ETSI EN 319 142) 265
18.1 The Four PAdES Baseline Profiles . . . 265
18.1.1 The Progression: Why Four Levels? . . . 266
18.2 PAdES-B-B: The Baseline Signature . . . 266
18.2.1 Requirements . . . 266
18.2.2 The ESS-signing-certificate-v2 Attribute . . . 267
18.2.3 SubFilter: `ETSI.CAdES.detached` vs. `adbe.pkcs7.detached` . . . 267
18.3 PAdES-B-T: Adding a Timestamp . . . 268
18.3.1 What B-T Adds . . . 268
18.3.2 How Timestamping Works . . . 268
18.3.3 Choosing a TSA . . . 269
18.4 PAdES-B-LT: Long-Term Data . . . 269
18.4.1 What B-LT Adds . . . 269
18.4.2 The DSS Dictionary . . . 269
18.4.3 Why B-LT Matters . . . 270
18.5 PAdES-B-LTA: Archival Timestamps . . . 270
18.5.1 What B-LTA Adds . . . 270
18.5.2 The Archival Chain . . . 271
18.5.3 Re-Timestamping . . . 271
18.6 Profile Comparison Matrix . . . 271
18.7 Migrating Between Profiles . . . 272
18.8 Key Takeaways . . . 272

19 Timestamps & Long-Term Validation 273
19.1 RFC 3161: The Timestamp Protocol . . . 273
19.1.1 What a Timestamp Proves . . . 273
19.1.2 The Timestamp Request/Response Flow . . . 273
19.1.3 Implementation with BouncyCastle . . . 275
19.1.4 TSA Selection Criteria . . . 276
19.2 Signature Timestamps vs. Document Timestamps . . . 276
19.2.1 Signature Timestamps . . . 277
19.2.2 Document Timestamps . . . 277
19.3 The Document Security Store (DSS) . . . 278
19.3.1 Purpose . . . 278
19.3.2 Building the DSS . . . 278
19.3.3 VRI: Validation Related Information . . . 279
19.4 Fetching Revocation Data . . . 280
19.4.1 OCSP (Online Certificate Status Protocol) . . . 280
19.4.2 CRL (Certificate Revocation List) . . . 281
19.5 The Complete B-LTA Workflow . . . 282
19.6 Key Takeaways . . . 282

20 eIDAS & Qualified Signatures 285

21 The eIDAS Regulation 287
21.1 A Brief History: The Electronic Signatures Directive . . . 287
21.1.1 Directive 1999/93/EC . . . 287
21.1.2 Why Directives Fail . . . 287
21.1.3 Cross-Border Failure . . . 288

21.1.4 The Lessons Learned . . . 288
21.2 eIDAS: Architecture of the Regulation . . . 289
21.2.1 Regulation, Not Directive . . . 289
21.2.2 Scope and Structure . . . 289
21.3 The Three Tiers of Electronic Signatures . . . 290
21.3.1 Tier 1: Electronic Signature (Article 3(10)) . . . 290
21.3.2 Tier 2: Advanced Electronic Signature (Article 3(11) and Article 26) . 290
21.3.3 Tier 3: Qualified Electronic Signature (Article 3(12)) . . . 292
21.4 Key Articles in Detail . . . 292
21.4.1 Article 25: Legal Effects of Electronic Signatures . . . 292
21.4.2 Article 28: Qualified Certificates for Electronic Signatures . . . 293
21.4.3 Article 29: Requirements for Qualified Signature Creation Devices . 294
21.4.4 Article 35-40: Electronic Seals . . . 294
21.5 Mutual Recognition . . . 295
21.5.1 Within the EU . . . 295
21.5.2 Non-EU Recognition . . . 295
21.6 eIDAS 2.0: Regulation (EU) 2024/1183 . . . 296
21.6.1 What Changed . . . 296
21.6.2 Impact on Developers . . . 296
21.7 The Gap Between Regulation and Reality . . . 297
21.7.1 What the Regulation Says . . . 297
21.7.2 What Actually Happens . . . 297
21.8 Common Misconceptions . . . 297
21.8.1 ”eIDAS makes all digital signatures legal” . . . 297
21.8.2 ”If my signature uses a certificate, it’s advanced” . . . 298
21.8.3 ”PAdES = eIDAS compliance” . . . 298
21.8.4 ”My country’s signatures are recognized everywhere in the EU” . . . 298
21.8.5 ”eIDAS applies outside the EU” . . . 298
21.9 What Your Code Needs to Do . . . 298
21.9.1 For Basic Electronic Signatures . . . 298
21.9.2 For Advanced Electronic Signatures . . . 298
21.9.3 For Qualified Electronic Signatures . . . 299
21.9.4 For Validation . . . 299
21.10 Key Takeaways . . . 300

22 Qualified Certificates & Trust Services 301
22.1 Qualified Trust Service Providers (QTSPs) . . . 301
22.1.1 What a QTSP Is . . . 301
22.1.2 How to Become a QTSP . . . 302
22.1.3 The Supervision Model . . . 302
22.2 The EU Trusted List (EUTL) . . . 303
22.2.1 Architecture: The LOT Hierarchy . . . 303
22.2.2 XML Schema . . . 303
22.2.3 Key Service Types and Statuses . . . 305
22.2.4 Cryptographic Signing of the TSL . . . 306
22.3 Qualified Certificates: What Makes Them Qualified . . . 306
22.3.1 The QCStatements Extension . . . 307
22.3.2 Certificate Policy OIDs . . . 308

22.3.3 QSCD Requirements . . . 308
22.4 Real-World QTSPs . . . 309
22.5 Java Code: Parsing the EUTL . . . 309
22.5.1 Downloading and Parsing the LOTL . . . 309
22.5.2 Matching Certificates Against the EUTL . . . 311
22.6 Key Takeaways . . . 314

23 Qualified Signatures vs Seals 315
23.1 The Fundamental Distinction . . . 315
23.1.1 Signatures: Natural Persons . . . 315
23.1.2 Seals: Legal Persons . . . 315
23.2 eIDAS Articles 25 vs 35 . . . 316
23.2.1 Article 25: Signatures . . . 316
23.2.2 Article 35: Seals . . . 316
23.3 Same Code, Different Certificate . . . 317
23.3.1 What Differs in the Certificate . . . 317
23.3.2 Creating a Seal vs a Signature in Java . . . 317
23.3.3 Detecting Signature vs Seal During Validation . . . 318
23.4 Use Cases: When to Sign, When to Seal . . . 320
23.4.1 Contracts and Agreements: Signature . . . 320
23.4.2 Invoices and Automated Documents: Seal . . . 320
23.4.3 Hybrid Workflows . . . 320
23.5 Server-Side Seal Creation and Batch Sealing . . . 320
23.5.1 The Architecture . . . 320
23.5.2 Cost Implications . . . 321
23.6 Common Pitfalls . . . 321
23.6.1 Displaying Seals as Signatures . . . 321
23.6.2 Using Signature Certificates for Automation . . . 321
23.6.3 Ignoring the Certificate Type in Code . . . 321
23.7 Key Takeaways . . . 321

24 Remote Signing 323
24.1 Why Remote Signing Won . . . 323
24.1.1 The Smart Card Problem . . . 323
24.1.2 The Remote Alternative . . . 324
24.1.3 eIDAS and Remote QSCDs . . . 324
24.2 The Hash-Signing Model . . . 325
24.2.1 Why Send the Hash, Not the Document . . . 325
24.2.2 The Flow Step by Step . . . 325
24.3 The CSC API (Cloud Signature Consortium) . . . 325
24.3.1 What the CSC Is . . . 325
24.3.2 API Endpoints . . . 327
24.3.3 Authentication: OAuth 2.0 . . . 328
24.3.4 Credential Discovery . . . 329
24.3.5 Credential Authorization and SAD . . . 330
24.3.6 Hash Signing . . . 331
24.4 SCAL: Sole Control Assurance Levels . . . 333
24.4.1 The Problem SCAL Solves . . . 333
24.4.2 SCAL1: Server-Side Sole Control . . . 333

24.4.3 SCAL2: Explicit Per-Signature Authorization . . . 333
24.5 HSM and SAM Architecture . . . 334
24.5.1 The HSM (Hardware Security Module) . . . 334
24.5.2 The SAM (Signature Activation Module) . . . 335
24.5.3 PKCS#11: The HSM Interface . . . 336
24.5.4 Key Lifecycle in an HSM . . . 337
24.6 Vendor Landscape . . . 337
24.6.1 Major Remote Signing Providers . . . 337
24.6.2 Choosing a Provider . . . 339
24.7 Complete Integration Example . . . 340
24.8 Security Considerations . . . 343
24.8.1 Transport Security . . . 343
24.8.2 Hash Manipulation . . . 343
24.8.3 Key Compromise . . . 344
24.9 Key Takeaways . . . 344

25 The EU DSS Framework 345
25.1 History and Governance . . . 345
25.1.1 Origins . . . 345
25.1.2 Development . . . 345
25.1.3 Version History . . . 346
25.2 Architecture . . . 346
25.2.1 Design Principles . . . 346
25.2.2 Module Overview . . . 347
25.2.3 Maven Coordinates . . . 348
25.3 The Signing Flow: ToBeSigned / SignatureValue . . . 349
25.3.1 The Two-Phase Signing Model . . . 349
25.3.2 SignatureTokenConnection Implementations . . . 350
25.4 Complete PAdES-B-T Signing Example . . . 350
25.5 Extending Signatures . . . 352
25.6 Validation with EU DSS . . . 353
25.6.1 Basic Validation . . . 353
25.6.2 Validation Reports . . . 354
25.6.3 EU Trusted List Integration . . . 355
25.7 Spring Boot Integration . . . 356
25.7.1 Configuration Properties . . . 356
25.7.2 Auto-Configured Beans . . . 357
25.8 EU DSS vs. Rolling Your Own . . . 358
25.8.1 When to Use EU DSS . . . 358
25.8.2 When to Consider Alternatives . . . 358
25.8.3 The Hybrid Approach . . . 359
25.9 Common Pitfalls . . . 359
25.9.1 1. Forgetting to Set the Certificate Chain . . . 359
25.9.2 2. Not Configuring a Certificate Verifier . . . 359
25.9.3 3. Underestimating /Contents Size . . . 360
25.9.4 4. Blocking the EUTL Load on First Request . . . 360
25.9.5 5. Ignoring Validation Sub-Indications . . . 360
25.10 Key Takeaways . . . 360

26 Security, Incidents & Outlook **363**
26.1 PDF Signature Attacks: A Complete Technical Analysis . . . 363
26.1.1 The Researchers . . . 363
26.1.2 Before Bochum: The State of PDF Signature Validation . . . 364
26.1.3 Attack Class 1: Universal Signature Forgery (USF) . . . 365
26.1.4 Attack Class 2: Incremental Saving Attack (ISA) . . . 369
26.1.5 Attack Class 3: Signature Wrapping Attack (SWA) . . . 374
26.1.6 Attack Class 4: Shadow Attacks (2021) . . . 377
26.1.7 Pre-Bochum Vulnerabilities: Earlier PDF Signature Issues . . . 381
26.1.8 Exploits in the Wild . . . 384
26.1.9 The Patch Response: What Changed . . . 385
26.1.10 Building a Robust Validator: Lessons from the Attacks . . . 386
26.1.11 A Note on Responsible Disclosure . . . 387
26.1.12 Summary . . . 388

27 Certificate Authority Breaches **389**
27.1 Why CA Breaches Matter for PDF Signatures . . . 389
27.2 DigiNotar (2011): The Breach That Broke a Country . . . 390
27.2.1 Background: A Small CA With a Big Responsibility . . . 390
27.2.2 The Attack . . . 390
27.2.3 The Use: Surveillance of Iranian Citizens . . . 391
27.2.4 Discovery: How Certificate Pinning Saved the Day . . . 391
27.2.5 The Response: Death of a CA . . . 392
27.2.6 Impact on PDF Signatures . . . 392
27.2.7 Lessons from DigiNotar . . . 393
27.3 Comodo (2011): The Parallel Breach . . . 393
27.3.1 The Attack . . . 393
27.3.2 The Response: How Comodo Got It Right . . . 394
27.3.3 The Attacker's Manifesto . . . 394
27.4 Symantec Distrust (2017-2018): The Slow-Motion Collapse . . . 395
27.4.1 Background: The World's Largest CA . . . 395
27.4.2 The Mis-Issuance Scandal . . . 395
27.4.3 The Distrust Timeline . . . 395
27.4.4 Impact on PDF Signatures . . . 396
27.5 Let's Encrypt: The Free CA's Growing Pains . . . 397
27.5.1 The 2020 Mass Revocation . . . 397
27.5.2 The Chaos . . . 398
27.5.3 Relevance to PDF Signing . . . 398
27.6 Certificate Transparency: The Response to CA Failures . . . 399
27.6.1 What Certificate Transparency Is . . . 399
27.6.2 How CT Detects Rogue Certificates . . . 399
27.6.3 CT and PDF: Limited but Growing . . . 399
27.7 What These Incidents Taught Us . . . 401
27.7.1 Trust Is Fragile and Revocable . . . 401
27.7.2 Long-Term Validity Requires Embedded Evidence . . . 401
27.7.3 You Cannot Predict CA Longevity . . . 401
27.8 Timeline: Major CA Incidents (2001-2024) . . . 402
27.9 Impact on PDF Signature Validation: What Happens in Practice . . . 404

27.9.1 The Validator's Trust Store . . . 404
27.9.2 Whether LTV Data Is Present . . . 405
27.9.3 The Certificate's Position in the Chain . . . 405
27.10 Key Takeaways . . . 405

28 Oddities & Annoyances **407**
28.1 The Adobe Approved Trust List Monopoly . . . 407
28.1.1 What the AATL Is . . . 407
28.1.2 The Economics of Green . . . 407
28.1.3 The EUTL Alternative . . . 408
28.1.4 The Political Dimension . . . 409
28.2 Broken Validation Across Viewers . . . 409
28.2.1 The Problem . . . 409
28.2.2 The Comparison . . . 409
28.2.3 Why Viewers Disagree . . . 410
28.2.4 What You Can Do About It . . . 411
28.3 SubFilter Confusion . . . 412
28.3.1 The Five SubFilter Values . . . 412
28.3.2 Which One Should You Use? . . . 412
28.3.3 What Happens with Unrecognized SubFilter Values . . . 413
28.4 PAdES vs ISO 32000-2:2020: The Divergence . . . 413
28.4.1 Two Standards, One Problem . . . 413
28.4.2 Where They Disagree . . . 414
28.4.3 The Practical Consequence . . . 414
28.5 The Timestamp Chicken-and-Egg Problem . . . 415
28.5.1 The Problem . . . 415
28.5.2 The Standard Solution . . . 415
28.5.3 The 32 KB Problem . . . 415
28.6 Silent Signing Failures . . . 417
28.6.1 The Taxonomy of Silence . . . 417
28.6.2 Empty `/Contents` . . . 417
28.6.3 Bad Byte Range . . . 418
28.6.4 Missing Certificate Chain . . . 418
28.6.5 Expired Timestamp . . . 418
28.7 Reader-Enabled PDFs vs Signing Conflicts . . . 419
28.7.1 What Reader-Enabling Is . . . 419
28.7.2 The Conflict with Document Signing . . . 419
28.7.3 The Historical Context . . . 420
28.7.4 The Modern Alternative . . . 420
28.8 Trailing Bytes and Garbage After `%%EOF` . . . 420
28.8.1 The Spec Says . . . 420
28.8.2 The Reality . . . 420
28.9 Key Takeaways . . . 421

29 EUDI Wallet & Digital Identity **423**
29.1 eIDAS 2.0: What Changed and Why . . . 423
29.1.1 The Regulatory History . . . 423
29.1.2 Regulation (EU) 2024/1183 . . . 424
29.1.3 The Political Context . . . 424

29.2 The EUDI Wallet Concept . . . 425
29.2.1 What the Wallet Is . . . 425
29.2.2 What the Wallet Is Not . . . 425
29.3 Architecture Reference Framework (ARF) . . . 426
29.3.1 Overview . . . 426
29.3.2 The Trust Model . . . 426
29.4 PID and QEAA . . . 427
29.4.1 Person Identification Data (PID) . . . 427
29.4.2 Qualified Electronic Attestations of Attributes (QEAA) . . . 427
29.5 Selective Disclosure and SD-JWT . . . 428
29.5.1 The Privacy Problem . . . 428
29.5.2 How SD-JWT Works . . . 428
29.5.3 The ISO/IEC 18013-5 mdoc Alternative . . . 429
29.6 Signing with the Wallet . . . 429
29.6.1 The Signing Flow . . . 429
29.6.2 Remote QSCD: Why the Key Is Not on the Phone . . . 430
29.6.3 The Sole Control Assurance Problem . . . 431
29.7 Impact on PDF Signing . . . 432
29.7.1 What Changes . . . 432
29.7.2 PAdES-B-LTA from Mobile . . . 432
29.8 Pilot Programs . . . 434
29.8.1 The Large-Scale Pilots . . . 434
29.8.2 What the Pilots Are Testing . . . 434
29.9 What Developers Need to Prepare . . . 435
29.9.1 If You Build Signing Software . . . 435
29.9.2 If You Build Validation Software . . . 435
29.9.3 If You Build Enterprise Systems . . . 436
29.10 The Open Questions . . . 436
29.11 Key Takeaways . . . 437

30 Code, Tools & Implementation 439

31 The Java PDF Library Landscape 441
31.1 The iText Saga: From PhD Project to Corporate Licensing Drama . . . 441
31.1.1 Bruno Lowagie and the Birth of iText (2000–2009) . . . 441
31.1.2 The LGPL-to-AGPL Switch: The Day the Rules Changed (2009) . . . 442
31.1.3 The CMS Structure Problem: Why Old iText Signatures Look Different 443
31.1.4 iText 5.x, 7.x, and 8.x: The Commercial Evolution . . . 444
31.1.5 The IBM Connection . . . 444
31.2 OpenPDF: The Community Fork That Refuses to Die . . . 445
31.2.1 The Fork (2016) . . . 445
31.2.2 What OpenPDF Is (and Isn't) . . . 445
31.2.3 Should You Use OpenPDF? . . . 446
31.3 Apache PDFBox: The Open-Source Standard . . . 446
31.3.1 Origin Story: Ben Litchfield and the Apache Incubation . . . 446
31.3.2 PDFBox 2.x: The Workhorse (2016–2023) . . . 446
31.3.3 PDFBox 3.x: The Breaking Change (2024) . . . 447
31.3.4 The PDFBox Signing Infrastructure: What's Under the Hood . . . 448
31.4 The Legion of the Bouncy Castle . . . 449

31.4.1 Why BouncyCastle Exists . . . 449
31.4.2 The BouncyCastle Ecosystem . . . 450
31.4.3 BouncyCastle Version History and the `-jdk18on` Suffix . . . 450
31.5 EU DSS: The European Commission's Signing Framework . . . 451
31.5.1 What EU DSS Is . . . 451
31.5.2 Why EU DSS Matters for PDF Signatures . . . 451
31.5.3 EU DSS Architecture . . . 452
31.5.4 EU DSS vs. Rolling Your Own . . . 452
31.5.5 EU DSS Version History . . . 453
31.6 Analysis and Debugging Tools . . . 454
31.6.1 RUPS: The PDF Object Explorer . . . 454
31.6.2 veraPDF: The Compliance Validator . . . 454
31.6.3 QPDF: The Swiss Army Knife . . . 455
31.6.4 MuTool (from MuPDF) . . . 455
31.6.5 OpenSSL: CMS Verification Outside the PDF Context . . . 455
31.6.6 ASN.1 Dump Tools . . . 456
31.7 Choosing the Right Library for Your Project . . . 456
31.7.1 Decision Matrix . . . 456
31.7.2 Our Recommendation . . . 457
31.8 The Ecosystem at a Glance . . . 458
31.9 Key Takeaways . . . 459

32 PDFBox Signing — Complete Walkthrough 461
32.1 The PDFBox Signing API: Three Moving Parts . . . 461
32.1.1 PDSignature: The Signature Dictionary . . . 461
32.1.2 SignatureInterface: The Cryptographic Callback . . . 463
32.1.3 SignatureOptions: Container Size and Visual Appearance . . . 464
32.2 Complete Basic Signing Example . . . 465
32.3 The InputStream in sign(): What You Are Actually Signing . . . 467
32.4 Visible Signature Appearances . . . 468
32.4.1 The Form XObject Approach . . . 468
32.4.2 Image-Based Appearances . . . 470
32.5 Multi-Signature Support . . . 471
32.5.1 How Multiple Signatures Work . . . 471
32.5.2 Code: Adding a Second Signature . . . 472
32.6 External Signing: HSM and Remote Key Flows . . . 473
32.6.1 The Hash-Signing Pattern . . . 473
32.7 Certification Signatures with DocMDP . . . 475
32.7.1 The Three DocMDP Levels . . . 475
32.7.2 Code: Applying a Certification Signature . . . 475
32.8 PDFBox 2.x to 3.x Migration . . . 476
32.8.1 Document Loading . . . 476
32.8.2 Standard 14 Fonts . . . 477
32.8.3 AcroForm API Changes . . . 477
32.8.4 Content Stream Writing . . . 477
32.8.5 Complete Migration Checklist . . . 477
32.9 Common Pitfalls . . . 478
32.9.1 Pitfall 1: Not Registering the BouncyCastle Provider . . . 478

32.9.2 Pitfall 2: Using save() Instead of saveIncremental() 479
32.9.3 Pitfall 3: /Contents Container Too Small 479
32.9.4 Pitfall 4: Not Closing the PDDocument 479
32.9.5 Pitfall 5: Signing After Closing the Document 479
32.9.6 Pitfall 6: Reusing a PDSignature Object 480
32.10 Timestamp Embedding 480
32.10.1 Embedding a TSA Timestamp as an Unsigned Attribute 480
32.11 Complete Production-Ready Signing Class 481

33 BouncyCastle CMS Construction 491
33.1 The CMS SignedData Structure: A Quick Refresher 491
33.2 BouncyCastle Provider Registration 492
33.3 Building a Basic CMS Signature 493
33.3.1 Step 1: The ContentSigner 493
33.3.2 Step 2: The DigestCalculatorProvider 493
33.3.3 Step 3: The SignerInfoGenerator 493
33.3.4 Step 4: The CMSSignedDataGenerator 494
33.4 Signed Attributes: What PAdES Requires 494
33.5 Creating ESS-signing-certificate-v2 495
33.5.1 Wiring ESS-signing-certificate-v2 into the Generator 497
33.6 Algorithm Negotiation: RSA vs. ECDSA 497
33.7 Timestamp Token as Unsigned Attribute 499
33.7.1 The Timestamping Flow 499
33.7.2 Complete Implementation 499
33.8 Debugging CMS Structures 502
33.8.1 ASN1Dump: Java-Side Inspection 502
33.8.2 OpenSSL: Command-Line Inspection 504
33.9 Complete PAdES-B-B Builder 504
33.10 Complete PAdES-B-T Builder 507
33.11 Putting It All Together 510

34 Verification & Analysis with PDFBox 513
34.1 Extracting Signatures from a PDF Document 513
34.1.1 The Extraction Path: AcroForm to PDSignature 513
34.1.2 Reading Signature Metadata 514
34.2 Parsing the CMS SignedData from /Contents 515
34.2.1 Extracting the Raw CMS Bytes 515
34.2.2 Parsing with BouncyCastle 515
34.2.3 Navigating the CMS Structure 516
34.3 Byte Range Verification in Code 517
34.3.1 The Three Checks 517
34.3.2 Validating the Contents Gap 518
34.4 CMS Cryptographic Verification 519
34.4.1 Building the SignerInformationVerifier 519
34.4.2 What verify() Actually Does 520
34.5 Certificate Chain Reconstruction and Validation 521
34.5.1 The Java PKIX CertPath API 521
34.5.2 Building the Trust Anchor Set 522
34.5.3 Key Usage Validation 523

34.6 Timestamp Verification . . . 523
34.6.1 Extracting the Timestamp Token . . . 523
34.6.2 Verifying the Timestamp Covers the Signature . . . 525
34.7 The Complete Five-Step Verification Pipeline . . . 526
34.7.1 Putting It All Together . . . 526
34.7.2 The VerificationReport Model . . . 528
34.7.3 The ValidationResult Enum . . . 529
34.8 Modification Detection Between Revisions . . . 529
34.8.1 Why Multi-Signature Documents Need Revision Comparison . . . 529
34.8.2 Extracting Individual Revisions . . . 530
34.9 Edge Cases: Self-Signed, Expired, and Missing Chain . . . 531
34.9.1 Self-Signed Certificates . . . 531
34.9.2 Expired Certificates . . . 532
34.9.3 Missing Intermediate Certificates . . . 532
34.10 Algorithm OID Mapping Utility . . . 533
34.11 Key Takeaways . . . 534

35 Test Cases & Reference PDFs 535
35.1 Why Testing PDF Signatures Is Hard . . . 535
35.1.1 The Three Enemies of Deterministic Signature Tests . . . 535
35.1.2 The Testing Strategy . . . 535
35.2 Test Infrastructure: JUnit 5 and Test Keystore Generation . . . 536
35.2.1 The Test Keystore Generator . . . 536
35.2.2 JUnit 5 Test Base Class . . . 538
35.3 Test PDF Generation Scenarios . . . 539
35.3.1 Generating Signed PDFs Programmatically . . . 539
35.3.2 Scenario Generators . . . 540
35.4 Test Matrix: Scenario to Expected Result . . . 540
35.5 Mocking TSA and OCSP . . . 542
35.5.1 In-Process TSA with BouncyCastle . . . 542
35.5.2 Mocking OCSP with WireMock . . . 543
35.6 Time Travel in Tests . . . 544
35.6.1 The Clock Abstraction . . . 544
35.6.2 Testing Expired Certificates . . . 545
35.7 Complete Signing-to-Verification Round-Trip Test . . . 546
35.8 PAdES Compliance Test . . . 547
35.9 Reference PDF Corpus: Naming Convention . . . 548
35.10 Key Takeaways . . . 549
35.11 Complete Code Listings . . . 549
35.12 SigningService: Complete PDF Signing Implementation . . . 549
35.13 VerificationService: Complete Signature Verification . . . 553
35.14 Signature Appearance Builder . . . 558
35.15 Test Keystore Generator: Full Implementation . . . 560
35.16 ASN.1 Structure Reference . . . 563
35.17 OID Reference Table . . . 564
35.17.1 Digest Algorithms . . . 564
35.17.2 Signature Algorithms . . . 564
35.17.3 CMS Attribute OIDs . . . 565

35.17.4 X.509 Extension OIDs 565
35.18 CMS SignedData Structure (RFC 5652 — Cryptographic Message Syntax (CMS)) 566
35.19 X.509 v3 Certificate Structure (RFC 5280 — Internet X.509 PKI Certificate and CRL Profile) 567
35.20 OCSP Response Structure (RFC 6960 — Online Certificate Status Protocol (OCSP)) 568
35.21 Timestamp Token Structure (RFC 3161 — Internet X.509 PKI Time-Stamp Protocol (TSP)) 569
35.22 Algorithm Identifier Structure 570
35.23 Glossary & Spec References 571
35.24 Glossary 571
35.25 Specification Reference Table 574

0.1 Preface

PDF digital signatures are one of those technologies that seem straightforward until you actually have to implement them. You read the spec, write some code, run a test — and the signature validates in your tool but fails in Adobe Acrobat. Or it works in Acrobat but fails the EU's validation service. Or it works everywhere on Monday and breaks on Friday because a timestamp authority renewed its certificate.

If you've been there, this book is for you.

This is a technical reference for engineers, developers, and security researchers who need to understand PDF digital signatures at the deepest level — from the raw bytes in the file to the legal framework that gives those bytes legal standing across 27 EU member states. Every claim is backed by precise references to ISO 32000-2:2020, the ETSI PAdES standards, relevant RFCs, and the eIDAS regulation. Code examples use Apache PDFBox 3.0.0 and BouncyCastle 1.80 on Java 21, because that's what most of us actually use in production.

0.1.1 Who This Book Is For

You are a software engineer, a security researcher, or a technically curious person who needs to *understand* PDF signatures, not just use them. Maybe you're building a signing service. Maybe you're debugging why a customer's signature shows "invalid" in one viewer but "valid" in another. Maybe you're the person who got handed a government compliance checklist that says "PAdES-B-LTA" and nobody in your organization knows what that means.

We assume you know how to program, that you've encountered public-key cryptography before (even if the details are fuzzy), and that you're comfortable reading data structures. We do *not* assume you know anything about PDF internals, CMS, or the European regulatory landscape. Parts I and II build that foundation from scratch.

0.1.2 What This Book Is Not

This is not a tutorial. There are no "Hello World" signing examples in Chapter 1. If you need to sign a PDF in the next 20 minutes, copy the code from Chapter 28 and come back later to understand why it works.

This is not a product manual. We use PDFBox and BouncyCastle because they're open-source, widely deployed, and well-documented. The concepts apply equally to iText, OpenPDF, or any other library — the PDF specification doesn't care what language you write in.

This is not a legal guide. We explain the eIDAS regulation and its technical requirements in detail, but we are engineers, not lawyers. When legal precision matters — and with qualified electronic signatures, it always does — consult someone with a law degree.

0.1.3 How This Book Is Organized

The book is structured in six parts, each building on the previous:

Part I: Cryptographic Foundations (Chapters 1–6) starts from first principles. If you can explain the difference between RSA and ECDSA, between a certificate and a key pair, between CMS and PKCS#7 — skip ahead. If any of those distinctions are hazy, start here. We cover just

enough cryptography to make the rest of the book make sense, with a focus on the specific algorithms and structures you'll encounter in PDF signatures.

Part II: PDF Internals (Chapters 7–11) dissects the PDF file format itself. You'll learn how objects, cross-reference tables, and incremental updates work — because you cannot understand PDF signatures without understanding incremental saves. You'll also learn about AcroForm fields, annotations, and coordinate systems, because signature fields are form fields, signature appearances are annotations, and the coordinate system *will* trip you up.

Part III: PDF Digital Signatures (Chapters 12–18) is the heart of the book. This is where we take apart the `/Sig` dictionary key by key, trace the byte range byte by byte, and walk through the CMS structure field by field. We cover approval and certification signatures, visual appearances, validation logic, PAdES profiles, and long-term validation. If you only read one part of this book, read this one.

Part IV: eIDAS & Qualified Signatures (Chapters 19–23) enters the regulatory world. The European eIDAS regulation created a legal framework where a digital signature can have the same legal standing as a handwritten one — but only if it meets very specific technical requirements. We explain what those requirements are, how the EU Trusted List works, what a Qualified Trust Service Provider does, and how remote signing via the Cloud Signature Consortium API fits into the picture.

Part V: Security, Incidents & Outlook (Chapters 24–27) covers what can go wrong — and what has gone wrong. The Ruhr University Bochum research team published devastating attacks against PDF signature validation in 2019–2021. We analyze every attack technically, explain why it worked, and show how to defend against it. We also cover certificate authority breaches, industry oddities, and the emerging EUDI wallet.

Part VI: Code & Implementation (Chapters 28–31) is where theory meets production. Complete, tested Java code for signing, CMS construction, verification, and analysis — not snippets, but full classes you can drop into a project. We also provide test cases and reference PDFs for every scenario discussed in the book.

0.1.4 Conventions Used in This Book

PDF dictionary keys are written in monospace with a leading slash: `/SubFilter`, `/ByteRange`, `/Contents`. When we show a full dictionary, it's presented as a code listing with callout annotations explaining each key.

Spec references follow this pattern: "ISO 32000-2:2020 §12.8.1" means ISO 32000-2:2020, section 12.8.1. RFC references: "RFC 5652 — Cryptographic Message Syntax (CMS) §5" means RFC 5652, section 5. We use these abbreviations consistently throughout.

Code examples are Java 21 using PDFBox 3.0.0 and BouncyCastle 1.80. Every code listing compiles and runs — the source is in the book's repository alongside test cases that verify the examples.

Admonition blocks carry specific meaning:

Additional context, background, or alternative approaches.

Practical advice from experience — the kind of thing a senior colleague would tell you over coffee.

Something you must understand to avoid a specific class of bugs.

A common mistake or misunderstanding that leads to broken signatures.

A security-relevant issue that could lead to signature bypasses or compliance failures.

0.1.5 A Note on the PDF Specification

The PDF specification has a complicated history. Adobe published it as a proprietary format in 1993, released it as ISO 32000-1 in 2008, and then ISO 32000-2:2020 superseded it in 2020 with significant changes to the digital signature sections.

Throughout this book, we reference ISO 32000-2:2020 as the authoritative source. Where the behavior of older PDF versions matters (and for digital signatures, it often does — plenty of documents in the wild still use PDF 1.7 features), we note the differences explicitly.

The PAdES standards (ETSI EN 319 142) add an additional layer of requirements on top of ISO 32000-2. They don't replace the PDF spec — they constrain it. A PAdES-compliant signature is always a valid ISO 32000-2 signature, but not every valid ISO 32000-2 signature is PAdES-compliant. This distinction matters enormously in the EU, and increasingly, everywhere else.

0.1.6 Three Decades of PDF: A Timeline

PDF was born as a page description language for printing. Digital signatures were an afterthought — bolted on six years later in PDF 1.3, using a cryptographic framework (PKCS#7) that predated PDF itself. Understanding why PDF signatures work the way they do — and why they break the way they break — requires understanding the history that shaped them.

The following table traces every major milestone in the evolution of PDF and, specifically, the digital signature infrastructure built on top of it. Some of these are format changes. Some are regulatory events. Some are security incidents that forced the entire industry to rethink assumptions it had held for years. All of them left marks on the code you will write or debug.

Table 1: The evolution of PDF and PDF digital signatures, 1993–2026

Year	Event	What happened and why it matters
1993	**PDF 1.0** released (Acrobat 1.0)	Adobe published the Portable Document Format as a proprietary specification for reliable cross-platform document rendering. No forms, no interactivity, no signatures — just pages, text, images, and fonts. The file structure established here (header, body, cross-reference table, trailer) survives unchanged to this day. Every PDF you sign in 2026 still starts with `%PDF-1.x` or `%PDF-2.0` and ends with `%%EOF`.

Year	Event	What happened and why it matters
1996	**PDF 1.2** (Acrobat 3.0)	Introduced **AcroForm** — interactive form fields that users could fill in. This matters for signatures because signature fields *are* form fields. The `/AcroForm` dictionary, the field tree, widget annotations — all of the infrastructure that Chapter 9 dissects — was born here. PDF 1.2 also added Unicode support and Flate compression, both of which are ubiquitous in modern PDFs.
1999	**PDF 1.3** — digital signatures introduced (Acrobat 4.0)	The moment PDF gained the ability to carry cryptographic signatures. Adobe defined the `/Sig` dictionary type, the `/ByteRange` array, the `/Contents` hex string for the signature value, and the `/SubFilter` key with its initial value `adbe.pkcs7.detached`. The design decision to embed the CMS signature *inside* the PDF file (rather than as a detached sidecar) and to define the signed region as a byte range with a gap for the signature value itself — these were architectural choices made in 1999 that we still live with. Every quirk in Chapter 12 traces back to this release.
2001	**PDF 1.4** (Acrobat 5.0)	Added **transparency** (the imaging model's biggest change ever), 128-bit RC4 encryption, and **tagged PDF** for accessibility. For signatures: PDF 1.4 formalized the `adbe.pkcs7.sha1` SubFilter (where the SHA-1 hash of the entire document is embedded as the CMS content, rather than using a detached signature). This SubFilter is now deprecated but still appears in legacy documents. Acrobat 5 also introduced **certification signatures** (`/DocMDP` transform) — the distinction between "I approve this document" and "I certify this is the authoritative version" starts here.
2003	**PDF 1.5** (Acrobat 6.0)	Introduced **object streams** and **cross-reference streams** — a complete reworking of how PDF files store their internal structure. Instead of the classic ASCII cross-reference table, PDF 1.5 allows binary cross-reference data compressed inside stream objects. This matters for signatures because the byte range must cover the *entire* file structure, and cross-reference streams change where objects physically live in the file. PDF 1.5 also added **XFA forms** (Adobe's XML-based form format), which would become a persistent source of interoperability problems for signatures on XFA documents — a problem so annoying that ISO 32000-2:2020 deprecated XFA entirely.

Year	Event	What happened and why it matters
2005	**PDF 1.6** (Acrobat 7.0)	**AES-128 encryption** replaced the weak RC4-based encryption from earlier versions. Embedded file streams (attachments) and 3D content were added. For signatures: PDF 1.6 refined the interaction between encryption and signing — a signed, encrypted PDF requires careful handling to ensure the signature covers the encrypted content correctly. The `/NeedAppearances` flag behavior was also clarified, which matters because a missing appearance stream on a signature field causes some viewers to show a blank signature block.
2006	**PDF 1.7** (Acrobat 8.0)	The last version of PDF that Adobe defined unilaterally before handing the specification to ISO. PDF 1.7 solidified the **AES-256 encryption** scheme and finalized the AcroForm/XFA coexistence model. The vast majority of signed PDFs in the wild today are PDF 1.7 documents — not because of any technical limitation of PDF 2.0, but because template authors, form generators, and enterprise software have not yet migrated. When you debug a signature failure in production, you are almost certainly looking at a PDF 1.7 file.
2008	**ISO 32000-1:2008** — PDF becomes an ISO standard	Adobe donated the PDF 1.7 specification to ISO, making it an open international standard. The technical content was essentially identical to Adobe's PDF Reference 1.7, but the governance changed: future changes would go through the ISO committee process, not Adobe's product team. For signatures, this was the moment PDF digital signatures gained the credibility of an ISO-backed specification — a prerequisite for government and regulated-industry adoption.

Year	Event	What happened and why it matters
2009	**PAdES** — ETSI TS 102 778 (Parts 1–5)	The European Telecommunications Standards Institute published the first **PDF Advanced Electronic Signatures** standards. PAdES defined profiles for how CMS signatures should be embedded in PDF to meet European legal requirements: which signed attributes are mandatory (ESS signing-certificate-v2), which SubFilter values to use (`ETSI.CAdES.detached`), and how to embed timestamps, revocation data, and long-term validation information. PAdES didn't replace ISO 32000 — it constrained it. A PAdES-compliant signature is always a valid ISO 32000 signature, but the reverse is not true. This distinction — and the interoperability problems it causes when a PDF viewer validates against ISO 32000 rules but not PAdES rules — is a recurring theme in this book (Chapters 17 and 26).
2011	**DigiNotar breach** — CA infrastructure crisis	A Dutch certificate authority, DigiNotar, was completely compromised by an attacker who issued over 500 fraudulent certificates — including certificates for google.com, mozilla.org, and intelligence agencies. The certificates were used in man-in-the-middle attacks against Iranian dissidents. Every major browser vendor revoked trust in DigiNotar within weeks; the company went bankrupt within months. For PDF signatures: DigiNotar was a wake-up call about the fragility of the certificate trust model that all digital signatures depend on. If a CA can be compromised, any signature issued by that CA's certificates becomes suspect. This incident accelerated the adoption of Certificate Transparency logs and tighter CA audit requirements — both of which affected the trust infrastructure behind PDF signatures. Chapter 25 covers this and other CA breaches in detail.

Year	Event	What happened and why it matters
2012	**Adobe Approved Trust List (AATL)** formalized	Adobe formalized its **AATL** program — a curated list of root certificates that Acrobat trusts for signature validation. If your signing certificate chains up to an AATL root, Acrobat shows a green checkmark. If it doesn't — even if the certificate is perfectly valid, issued by a reputable CA, and chains to a trust anchor in your OS trust store — Acrobat shows a warning. This created a de facto monopoly on "valid" PDF signatures: to get a green checkmark in the world's dominant PDF viewer, you need an AATL-listed certificate, which costs significantly more than a standard code-signing certificate. The AATL remains one of the most contentious aspects of the PDF signature ecosystem (Chapter 26).
2014	**eIDAS Regulation** (EU 910/2014) enacted	The European Union passed the **Electronic Identification, Authentication and Trust Services** regulation, creating a legal framework where digital signatures can have the same legal standing as handwritten signatures — but only if they meet strict technical requirements. eIDAS defined three tiers: **electronic signatures** (any electronic indication of signing intent), **advanced electronic signatures** (uniquely linked to the signatory, capable of identifying them, under their sole control, linked to the data so any change is detectable), and **qualified electronic signatures** (advanced + created by a qualified device + based on a qualified certificate issued by a qualified trust service provider). Only QES has automatic legal equivalence to a handwritten signature across all EU member states. PAdES became the primary technical mechanism for creating eIDAS-compliant signatures in PDF. Chapters 19–23 cover the entire eIDAS ecosystem.

Year	Event	What happened and why it matters
2016	**eIDAS fully effective** (July 1, 2016)	The eIDAS regulation became fully effective across all EU member states, with the **EU Trusted List (EUTL)** operational. The EUTL is a machine-readable, cryptographically signed list of every Qualified Trust Service Provider in the EU — the authoritative source for determining whether a signing certificate qualifies for the highest legal tier. For developers: if your application needs to validate whether a PDF signature is a qualified electronic signature, you must check the signing certificate against the EUTL, not just against a generic trust store. The EUTL XML schema, its signature validation, and the LOT (List of Trusted Lists) hierarchy are covered in Chapter 20.
2017	**ISO 32000-2:2017** — PDF 2.0, first edition	The first major revision of the PDF specification under ISO governance. PDF 2.0 deprecated XFA forms, removed several legacy features, and made significant changes to the digital signature sections. Notably: the `adbe.pkcs7.sha1` SubFilter was deprecated (SHA-1 is cryptographically broken for collision resistance since the SHAttered attack of February 2017), and the specification added the `ETSI.RFC3161` SubFilter for document-level timestamps. PDF 2.0 also clarified the byte range coverage rules and tightened the requirements around incremental updates after signing — changes that were partly motivated by early reports of the signature validation problems that Bochum would systematically expose two years later.
2019	**"1 Trillion Dollar Refund"** — Bochum attacks (CCS 2019)	The Ruhr University Bochum team published their landmark paper at ACM CCS 2019, systematically breaking PDF signature validation in every major viewer. They identified three attack classes: **Universal Signature Forgery (USF)** — crafting a signed PDF with a garbage or empty signature value that viewers accepted as valid; **Incremental Saving Attack (ISA)** — appending unsigned changes to a signed PDF that viewers failed to flag; and **Signature Wrapping Attack (SWA)** — relocating the signature's byte range to point at attacker-controlled data. Of 22 desktop viewers tested, all were vulnerable to at least one attack. The paper title referenced the fact that signed PDFs are used for tax refund claims, insurance payouts, and financial contracts — if signatures could be forged, the financial exposure was enormous. This research is the subject of Chapter 24.

Year	Event	What happened and why it matters
2020	**ISO 32000-2:2020** — PDF 2.0, second edition	The second edition of PDF 2.0 incorporated corrections and clarifications, including tightened language around signature validation motivated by the Bochum attacks. In the same year, Adobe issued **CVE-2020-9592** and **CVE-2020-9596** and patched Acrobat against USF, ISA, and SWA. Foxit, Nitro, LibreOffice, Okular, and every other major viewer released their own patches. The PAdES standards were also updated (ETSI EN 319 142-1 V1.1.1) to align with the second edition of ISO 32000-2.
2021	**Shadow Attacks** (NDSS 2021)	The Bochum team published a follow-up: **Shadow Attacks**. Unlike the 2019 attacks (which modified a PDF *after* signing), Shadow Attacks prepared malicious content *before* the document was signed. The attacker created a PDF with hidden content (invisible text, overlapping images, replaced form fields) that the signer could not see during signing — but that became visible when the recipient opened the document. The signature validated correctly because the signed bytes were genuinely signed by the legitimate signer. The attack exploited the gap between what the signer *saw* and what the bytes *contained* — a fundamental problem in any visual-document signing model. Three variants were demonstrated: **Hide** (hiding content behind overlapping elements), **Replace** (swapping visible content via interactive form features), and **Hide-and-Replace** (combining both). Again, all major viewers were vulnerable.
2022	**ETSI EN 319 142-1 V1.2.1** — PAdES update	ETSI updated the PAdES standard to align with ISO 32000-2:2020 and to incorporate lessons from the Bochum attacks. The update clarified requirements around signature scope validation, incremental update handling, and the relationship between the byte range and the visible document content. It also formalized the PAdES-B-LTA profile's requirements for archival timestamps in the DSS (Document Security Store) — the mechanism that enables long-term validation decades after the signing certificate has expired.

Year	Event	What happened and why it matters
2024	**eIDAS 2.0** (EU 2024/1183) enacted	The EU adopted the revised eIDAS regulation, which introduced the **European Digital Identity (EUDI) Wallet** — a smartphone-based identity wallet that every EU member state must offer to its citizens by 2026. The EUDI wallet will carry government-issued identity attestations (PID — Person Identification Data) and qualified electronic attestations of attributes (QEAA), enabling citizens to sign documents with qualified electronic signatures from their phone. This represents the most ambitious attempt to make QES mainstream — moving it from specialized HSM-backed signing services to consumer smartphones. The Architecture Reference Framework (ARF) defines the technical specifications. Chapter 27 covers the EUDI wallet and its impact on PDF signing.
2025–26	**EUDI wallet pilots & deployment**	EU member states are conducting large-scale pilot programs for the EUDI wallet across sectors: banking (identity verification), travel (digital travel credentials), education (diplomas), healthcare (prescriptions), and government services. The pilots test the wallet's signing capabilities, including PAdES-B-LTA signature creation from mobile devices, remote QSCD integration, and cross-border signature validation. The Cloud Signature Consortium (CSC) API — the standard interface for remote signing — is being adapted for wallet integration. The outcome of these pilots will determine whether QES becomes a mainstream consumer technology or remains a specialized enterprise tool.

The table omits minor point releases and focuses on the events that changed what developers need to know. If you are working with PDF signatures today, every row in that table has left its fingerprint on the code you are writing — whether it's the byte range gap from 1999, the PAdES constraints from 2009, the trust model questions raised by DigiNotar in 2011, or the validation rewrites forced by Bochum in 2019–2021.

0.1.7 Acknowledgments

This book would not exist without the open-source communities that make PDF signature work accessible to everyone. The **Apache PDFBox** team — particularly Andreas Lehmkühler, Tilman Hausherr, and the many contributors who maintain the signing infrastructure — have built the most widely used open-source PDF library in the Java ecosystem. The **Legion of the Bouncy Castle** (yes, that's their actual name) gave us BouncyCastle, which provides the CMS and X.509 implementations that most Java signing code depends on, whether it knows it or not.

Special acknowledgment goes to the security researchers at **Ruhr University Bochum** — Vladislav Mladenov, Christian Mainka, Karsten Meyer zu Selhausen, Martin Grothe, and Jörg Schwenk — whose systematic analysis of PDF signature validation between 2019 and 2021 exposed fundamental flaws in how every major PDF viewer handled signatures. Their papers — *"1 Trillion Dollar Refund: How To Spoof PDF Signatures"* (CCS 2019), *"Shadow Attacks: Hiding and Replacing Content in Signed PDFs"* (NDSS 2021), and several follow-ups — demonstrated that attackers could forge signatures, hide content changes behind valid signatures, and bypass validation in Adobe Acrobat, Foxit Reader, and dozens of other viewers. The industry patched, but the research fundamentally changed how we think about PDF signature security. Their work is cited throughout this book, and Chapter 24 is dedicated to a technical analysis of their attacks.

What they found was this: PDF viewers did not actually verify what they claimed to verify. The Bochum team identified three major attack classes. **Universal Signature Forgery (USF)** showed that some viewers only checked whether a signature *existed* — not whether its cryptographic value was correct. An attacker could craft a PDF with an empty or garbage `/Contents` value, and viewers like Adobe Acrobat DC (before the patch) would display it as "signed and valid." **Incremental Saving Attack (ISA)** exploited the incremental update mechanism that PDF signatures depend on: an attacker could append a new revision to a signed PDF — changing text, adding pages, replacing content entirely — and because viewers did not properly check whether the appended changes fell outside the signed byte range, the signature still showed as valid. **Signature Wrapping Attack (SWA)** relocated the signed content within the file so that the byte range pointed at attacker-controlled data while the original signature remained structurally present. The follow-up **Shadow Attacks** paper (NDSS 2021) went further: an attacker could prepare a PDF *before* it was signed, embedding hidden content that would only become visible after signing — the signer would see one document, but the recipient would see another, and the signature would validate on both views.

The team tested 22 desktop PDF viewers and 7 online validation services. Every single one was vulnerable to at least one attack class. Adobe issued CVE-2020-9592 and CVE-2020-9596 and patched Acrobat in May 2020. Foxit, Nitro, LibreOffice, and every other major viewer followed with their own patches. The research forced a fundamental rethinking of PDF signature validation — in particular, how viewers handle incremental updates after signing, how byte range coverage is verified, and how the relationship between the CMS signature and the actual document bytes is validated. Before Bochum, most viewers treated signature validation as a checkbox. After Bochum, it became a security-critical code path that required adversarial thinking.

The **European Telecommunications Standards Institute (ETSI)** deserves credit for the PAdES standards that brought PDF signatures into the regulated world. Whatever one may think about the complexity of ETSI EN 319 142, it solved the real problem of making PDF signatures legally interoperable across borders.

And the PDF specification itself is the product of hundreds of engineers at Adobe, ISO, and ETSI who somehow managed to build a digital signature framework into a page description language originally designed for printing. That it works as well as it does is a testament to their effort. That it has the quirks it has is a testament to the fact that three decades of backward compatibility leaves marks.

Chapter 1

Cryptographic Foundations

Chapter 2

Asymmetric Cryptography Primer

This chapter builds the cryptographic foundation that everything else in this book depends on. We start with the key distribution problem that asymmetric cryptography was invented to solve, work through RSA and elliptic curve algorithms at a conceptual level, and end with practical Java code that generates the key pairs you will use to sign PDFs.

If you can already explain why RSA-2048 and ECDSA with P-256 provide roughly equivalent security, and you know why BouncyCastle exists alongside the JDK's built-in providers, skip to Chapter 2. Everyone else: this is where your journey into PDF signatures begins.

2.1 The Key Distribution Problem

Before we talk about RSA, elliptic curves, or any of the algorithms that make PDF signatures possible, we need to understand the problem they were invented to solve.

Symmetric cryptography — where the same key encrypts and decrypts — has been around for millennia. Caesar ciphers, Enigma machines, AES-256: they all share a single fundamental weakness. Both parties must possess the same secret key. If Alice wants to send an encrypted message to Bob, she and Bob must first agree on a shared secret. In person, this is trivial — write the key on a piece of paper and hand it over. Over a network, it is a nightmare.

This is the **key distribution problem**: how do you securely share a secret with someone you have never met, over a channel that an adversary controls?

For most of human history, the answer was "you don't — you use a trusted courier." Governments ran key distribution networks. Military organizations employed dedicated personnel whose sole job was transporting codebooks. Banks used armored vehicles. None of this scales to the internet, where you need to establish secure communications with millions of strangers, instantly, without any prior arrangement.

In 1976, Whitfield Diffie and Martin Hellman published "New Directions in Cryptography," describing a system where two parties could agree on a shared secret over a public channel without ever having communicated before. A year later, Ronald Rivest, Adi Shamir, and Leonard Adleman published the RSA algorithm — the first practical public-key cryptosystem. (Clifford Cocks at GCHQ had independently discovered an equivalent system in 1973, but it was classified until 1997. Cryptography has a habit of being invented multiple times.)

The insight behind all of these systems is the same: **mathematical one-way functions**. Operations that are easy to perform in one direction but computationally infeasible to reverse. Multiplying two large primes takes microseconds; factoring their product takes longer than the age of the universe. This asymmetry — easy forward, hard backward — is the foundation of everything that follows.

2.2 Key Pairs: The Fundamental Asymmetry

An asymmetric key pair consists of two mathematically related keys:

- A **private key** — kept secret, never shared, never transmitted
- A **public key** — derived from the private key, freely distributable

The relationship between them has two critical properties:

1. **It is computationally infeasible to derive the private key from the public key.** Given a public key, there is no known efficient algorithm to compute the corresponding private key. The security of the entire system rests on this.
2. **Operations performed with one key can only be reversed with the other.** Data encrypted with the public key can only be decrypted with the private key. Data signed with the private key can only be verified with the public key.

This second property is what makes digital signatures possible. When you sign a PDF, your private key produces a cryptographic value that is mathematically bound to both your key and the document content. Anyone who has your public key — which is embedded in your certificate, which is embedded in the PDF itself — can verify that the signature was produced by someone holding the corresponding private key.

They cannot forge a new signature. They cannot modify the document without invalidating the signature. They cannot extract your private key from the signature or the public key.

This is the fundamental asymmetry that gives asymmetric cryptography its name, and it is the reason PDF digital signatures work at all.

The terms ”encrypt” and ”decrypt” are technically imprecise when applied to digital signatures. Signing is not encryption in the classical sense — you are not hiding data; you are producing a proof. The distinction matters in formal cryptographic contexts and in legal frameworks like eIDAS (Regulation (EU) No 910/2014 (eIDAS)).

Throughout this book, we use ”sign” and ”verify” for signature operations, reserving ”encrypt” and ”decrypt” for actual confidentiality operations. Many older references (and some library APIs, including parts of the JCA) conflate the two. Do not let the terminology confuse you.

2.3 RSA: The Workhorse

RSA, named after Rivest, Shamir, and Adleman, remains the most widely deployed asymmetric algorithm in the world. Every PDF viewer supports it. Every certificate authority issues RSA certificates. Every signing library implements it. If you encounter a signed PDF in production, there is an overwhelming probability that it was signed with RSA.

2.3.1 The Mathematical Basis

The security of RSA rests on the **integer factorization problem**: given the product of two large primes, recover the original primes. Multiplication is trivial; factoring is believed to be computationally infeasible for sufficiently large numbers. Here is how this translates into a key pair.

1. **Choose two large primes, *p* and *q*.** These primes are generated independently using a cryptographically secure random number generator. For RSA-2048, each prime is approximately 1024 bits long. Finding primes of this size is itself nontrivial — it requires probabilistic primality tests (Miller-Rabin is the standard), and several candidates may be tested before a probable prime is found. This is why RSA key generation is noticeably slower than ECDSA key generation.
2. **Compute the modulus *n* = *p* * *q*.** This product *n* appears in both the public and private keys. For RSA-2048, *n* is a 2048-bit number. Computing *n* from *p* and *q* takes microseconds. Recovering *p* and *q* from *n* is the hard problem.
3. **Compute the totient.** Euler's totient function for *n* is *phi(n) = (p - 1)(q - 1)*. This value is used to derive the private exponent and is then discarded. It must never be stored or revealed — knowing *phi(n)* is equivalent to knowing the factorization of *n*.
4. **Choose the public exponent *e*.** In practice, *e* is almost always 65537 (0x10001). This value was chosen because it is prime, it has a low Hamming weight (only two bits set), and it makes modular exponentiation during verification fast. You will see 65537 in virtually every RSA certificate you examine. Some ancient systems used *e* = 3, which is faster but has known vulnerabilities when combined with naive padding.
5. **Compute the private exponent *d*.** *d* is the modular inverse of *e* modulo *phi(n)*: *d * e = 1 mod phi(n)*. This is computed using the extended Euclidean algorithm. Knowing *d* requires knowing *phi(n)*, which requires knowing *p* and *q*.
6. **The public key** is the pair (*n*, *e*).
7. **The private key** is the pair (*n*, *d*). In practice, the private key also stores *p*, *q*, and several precomputed values (the CRT — Chinese Remainder Theorem — parameters) that make signing four times faster.

2.3.2 Why Large Primes Matter

The security of RSA depends on the inability of an attacker to factor *n* and recover *p* and *q*. The difficulty of factoring grows super-polynomially with the size of *n*. The best known classical factoring algorithm is the General Number Field Sieve (GNFS), whose complexity is roughly:

```
exp( (1.923 + o(1)) * (ln n)^(1/3) * (ln ln n)^(2/3) )
```

This means that doubling the key size does *not* merely double the work — it increases it by many orders of magnitude. Factoring a 768-bit RSA modulus was achieved in 2009 by a team of researchers using hundreds of machines over two years. Factoring a 1024-bit modulus is estimated to be within the reach of a well-funded nation-state. Factoring a 2048-bit modulus is estimated to require more computation than humanity can currently muster.

2.3.3 RSA Key Sizes vs Security Levels

The size of an RSA key is the bit length of the modulus *n*. Larger modulus means harder factoring means more security — but also slower operations.

Table 2.1: RSA key sizes and their practical characteristics

Key Size	Security Level	Sign Time (relative)	Signature Size	Status
1024-bit	~80-bit	1x (baseline)	128 bytes	**Broken.** NIST disallowed after 2013. Do not use for any purpose. You will still encounter these in legacy PDFs — they were common before 2010.
2048-bit	~112-bit	~4x	256 bytes	**Current minimum.** NIST SP 800-57 minimum through 2030. This is what most CAs issue today. Adobe AATL requires at least 2048-bit RSA.
3072-bit	~128-bit	~9x	384 bytes	**Recommended for new deployments.** Provides 128-bit security — the level NIST considers adequate through at least 2030 and likely well beyond.
4096-bit	~140-bit	~16x	512 bytes	**Conservative choice.** Noticeable performance impact for high-volume signing. Some government requirements mandate this. The additional security margin over 3072 is modest relative to the performance cost.

RSA-1024 is not merely ”weak” — it is **factored**. In 2010, a team factored a 768-bit RSA modulus. The computational cost of factoring 1024-bit moduli is within reach of well-funded adversaries. If you encounter a signed PDF with an RSA-1024 certificate, the signature provides no meaningful security guarantee, even if the viewer shows it as ”valid.” Many viewers do not check key size during validation.

2.3.4 Performance Impact of Key Size

RSA has an unusual asymmetry in its operation costs: **verification is fast; signing is slow**.

This is because the public exponent e is small (65537), making the modular exponentiation during verification much cheaper than the exponentiation with the large private exponent d during signing. For PDF signatures, this works in our favor: documents are signed once but verified many times.

However, RSA's performance degrades sharply as key sizes increase. The signing operation scales roughly as $O(k^3)$ where k is the key size in bits. Doubling the key size makes signing approximately eight times slower. For a desktop application signing a single document, this is irrelevant. For a server processing thousands of signing requests per second, the difference between 2048-bit and 4096-bit RSA is significant.

2.3.5 Why RSA-1024 Persists in the Wild

If RSA-1024 is broken, why do you still encounter it in signed PDFs? The answer is a combination of institutional inertia, cost, and the peculiar nature of long-lived documents.

Government agencies issued RSA-1024 certificates throughout the 2000s for document signing infrastructure. Many of those signed documents still exist, still circulate, and still need to be validated. Some organizations operate signing infrastructure on hardware security modules (HSMs) that cannot be easily re-keyed — the cost of replacing an HSM fleet across dozens of offices can be enormous. Other systems operate in air-gapped environments where updating cryptographic infrastructure requires physical site visits.

The practical consequence: your validation code must *handle* RSA-1024 gracefully, even if it correctly flags it as insecure. Refusing to parse a legacy document because the key size is too small is unhelpful. Displaying it as "valid" without qualification is irresponsible. The right approach is what PDFalyzer does: show the signature, show the key size, and let the user draw the appropriate conclusion.

How to check the key size of a certificate in a signed PDF:

In PDFalyzer, open the Signatures tab and look at the "Public Key" section of any signature. The key size is displayed directly.

From the command line, extract the certificate and use OpenSSL:

```
openssl x509 -in cert.pem -text -noout | grep "Public-Key"
```

In Java with BouncyCastle, it is one line:

```
int keySize = ((RSAPublicKey) cert.getPublicKey()).getModulus().bitLength();
```

If you see 1024 or less, the certificate was issued before modern standards were enforced, and the signature's cryptographic value is nil.

2.4 Elliptic Curve Cryptography: Smaller Keys, Same Security

In the mid-1980s, Neal Koblitz and Victor Miller independently proposed using elliptic curves over finite fields as the basis for public-key cryptography. The core idea: the **elliptic curve discrete logarithm problem (ECDLP)** is believed to be harder than integer factorization for equivalent key sizes. This means we can achieve the same security level with dramatically smaller keys.

2.4.1 Elliptic Curve Basics

An elliptic curve, for cryptographic purposes, is the set of points (x, y) satisfying an equation of the form $y^2 = x^3 + ax + b$ over a finite field. The "finite field" part is important: we are not

working with real numbers and smooth curves. We are working with integers modulo a large prime p, and the curve is a discrete set of points.

The cryptographic operation on these points is **point addition**: given two points on the curve, there is a geometric rule for computing a third point. Repeated point addition (scalar multiplication) gives us the one-way function: given a point G and a scalar d, computing $Q = d * G$ is efficient. Recovering d from Q and G (the ECDLP) is computationally infeasible.

You do not need to understand the geometry to use ECDSA effectively. What matters is the practical consequence: the ECDLP is harder than integer factorization for the same key size, which is why a 256-bit elliptic curve key provides the same security as a 3072-bit RSA key.

2.4.2 The NIST Curves: P-256, P-384, P-521

The standard curves for ECDSA, defined by NIST in FIPS 186-4:

Table 2.2: NIST standard curves for ECDSA

Curve	Security Level	Signature Size	Notes
P-256 (secp256r1)	128-bit	64 bytes	The workhorse. Universally supported. Adobe Acrobat, PDFBox, iText, BouncyCastle — everyone supports P-256. If you are choosing an ECDSA curve for PDF signing, this is the default choice.
P-384 (secp384r1)	192-bit	96 bytes	Required by certain government profiles (e.g., NSA Suite B, CNSS Policy 15). Broader support than P-521 but less than P-256.
P-521 (secp521r1)	256-bit	132 bytes	Maximum security. The name is not a typo — it is 521 bits, not 512. Rarely used in practice for PDF signatures because the security margin over P-384 is excessive for most threat models, and some tools have spotty support.

The curve names are a source of endless confusion. ”P-256” is the NIST name. ”secp256r1” is the SECG (Standards for Efficient Cryptography Group) name. ”prime256v1” is the OpenSSL name. They are all the same curve. You will encounter all three names in certificate metadata, library APIs, and documentation. When PDFBox reports a signature algorithm, it uses the JCA name; when BouncyCastle constructs the CMS structure, it uses OIDs; when you read the certificate in OpenSSL, it says ”prime256v1.” They are all P-256.

2.4.3 Why ECDSA Signatures Are Smaller

The practical consequence of the ECDLP’s difficulty is striking. RSA-3072 provides the same security as ECC-256, but the RSA key is twelve times larger. This size difference cascades through every part of the system: smaller keys mean smaller certificates, smaller CMS structures in the PDF’s `/Contents` field, faster network transmission of signed documents, and less storage overhead.

Table 2.3: Equivalent security levels across algorithms (NIST SP 800-57 Part 1, Rev. 5)

Security Level	RSA Key Size	ECC Key Size	Ratio
112-bit	2048-bit	224-bit	9:1
128-bit	3072-bit	256-bit	12:1
192-bit	7680-bit	384-bit	20:1
256-bit	15360-bit	521-bit	30:1

For PDF signatures specifically, the `/Contents` hex string must be large enough to hold the entire CMS `SignedData` structure, including the signer's certificate chain. With RSA-4096 certificates and a chain of three certificates, this blob can exceed 10 KB. With ECDSA P-256, the same chain might be 3 KB. Chapter 13 covers why the size of `/Contents` matters — it determines the byte range gap, which must be estimated before signing.

2.4.4 ECDSA Adoption in PDF Signatures

ECDSA adoption in PDF has been slower than in TLS, where P-256 became the dominant algorithm years ago. The reasons are institutional rather than technical.

Certificate authorities have been issuing RSA signing certificates for two decades, and existing workflows are built around RSA. Many enterprise signing solutions — particularly those using HSMs provisioned in the 2010s — have RSA keys burned into hardware. The Adobe AATL program includes ECDSA certificates from several providers, but the majority of AATL roots remain RSA.

That said, new deployments increasingly choose ECDSA. The eIDAS 2.0 Architecture Reference Framework (Regulation (EU) 2024/1183 (eIDAS 2.0)) lists ECDSA as a recommended algorithm for the EUDI wallet. If you are starting a new signing project and you have control over the certificate procurement process, ECDSA with P-256 is the technically superior choice.

The random nonce k in ECDSA is a notorious source of catastrophic failures. If the same k is used for two different messages, the private key can be algebraically recovered. This is not a theoretical concern — it has happened in production. Sony's PlayStation 3 code-signing key was extracted in 2010 because they used a constant value for k. The Bitcoin Android wallet vulnerability in 2013 was caused by a weak random number generator producing predictable k values. Modern implementations use deterministic nonce generation (RFC 6979) to eliminate this class of bugs entirely. BouncyCastle 1.80 supports RFC 6979 deterministic signatures.

2.5 EdDSA and Ed25519: The Future Direction

Edwards-curve Digital Signature Algorithm (EdDSA) is a modern signature scheme designed by Daniel J. Bernstein and colleagues, published as RFC 8032 in 2017. It addresses several practical shortcomings of ECDSA while providing equivalent or better security.

2.5.1 Bernstein's Work and Design Philosophy

Daniel J. Bernstein — "djb" to everyone in the cryptographic community — designed Curve25519 in 2005 and the Ed25519 signature scheme in 2011 with a very specific philosophy: eliminate every source of implementation error.

The NIST curves (P-256, P-384) have been criticized for their opaque parameter generation — NIST never fully explained how the "random" seed values were chosen, which fueled suspicion (especially after the Dual_EC_DRBG backdoor scandal in 2013, where the NSA was found to have inserted a backdoor into a NIST-standardized random number generator). Bernstein designed Curve25519 with fully transparent, verifiable parameters — every constant has a documented justification.

More importantly, Ed25519 was designed to be *implementable* without the subtle pitfalls that plague ECDSA:

1. **Deterministic signatures.** EdDSA does not use a random nonce. The nonce is derived deterministically from the private key and the message. This eliminates the entire class of nonce-reuse vulnerabilities. There is no Sony PS3 scenario with EdDSA — it is structurally impossible.
2. **Constant-time by design.** The algorithm was designed from the ground up to run in constant time. There are no data-dependent branches or memory accesses that could leak information about the private key through timing, power analysis, or cache behavior.
3. **No parameter negotiation.** You use Ed25519 (128-bit security) or Ed448 (224-bit security). That is it. No curves to choose, no hash functions to configure, no padding schemes to get wrong.

2.5.2 Ed25519 and Ed448

EdDSA defines two concrete instances:

- **Ed25519**: Based on Curve25519 (a twisted Edwards curve). Provides ~128-bit security. Signatures are 64 bytes, public keys are 32 bytes. This is the one everyone uses.
- **Ed448**: Based on Curve448 (the "Goldilocks" curve, named by Mike Hamburg because 448 = 224 * 2, a "just right" security level). Provides ~224-bit security. Signatures are 114 bytes, public keys are 57 bytes.

2.5.3 EdDSA and PDF: Adoption Status

Here is where the good news ends, at least for PDF signatures.

As of 2026, **EdDSA support in the PDF signature ecosystem is minimal**. ISO 32000-2:2020 does not explicitly mention EdDSA. The PAdES standards (ETSI EN 319 142-1 V1.1.1 (2016-04)) reference ECDSA but not EdDSA. Adobe Acrobat does not support EdDSA signatures — a PDF signed with Ed25519 will show as "unknown algorithm" or "signature invalid" in Acrobat.

Do not use EdDSA for PDF signatures in production today.

While the algorithm itself is excellent, the ecosystem does not support it. A signature that cannot be validated by the dominant PDF viewer is, for all practical purposes, useless.

That said, EdDSA adoption is accelerating in the broader PKI ecosystem (TLS, SSH, code signing), and it is a matter of when — not if — PDF viewers add support. The eIDAS 2.0 Architecture Reference Framework (Regulation (EU) 2024/1183 (eIDAS 2.0)) includes EdDSA in its list of acceptable algorithms for the EUDI wallet. When EUDI wallet-based PDF signing becomes mainstream (Chapter 27), EdDSA support in PDF viewers will follow.

2.6 Algorithm Comparison

Table 2.4: Algorithm comparison for PDF digital signatures (as of 2026)

Property	RSA-3072	ECDSA P-256	ECDSA P-384	Ed25519	Notes
Security level	128-bit	128-bit	192-bit	~128-bit	All except P-384 provide equivalent security.
Public key size	~400 bytes	~90 bytes	~120 bytes	32 bytes	Encoded sizes (SubjectPublicKeyInfo).
Signature size	384 bytes	64 bytes	96 bytes	64 bytes	Raw signature value before CMS packaging.
Keygen speed	Slow (probabilistic prime search)	Fast	Fast	Fast	RSA-3072 keygen: 200-1500ms. EC: under 5ms.
Sign speed	Moderate	Fast	Fast	Fast	RSA scales poorly with key size.
Verify speed	Fast	Moderate	Moderate	Fast	RSA verification is cheap (small e).
Adobe Acrobat	Full support	Full support	Full support	No support	Acrobat drives compatibility requirements.
PDF spec support	ISO 32000-2:2020	ISO 32000-2:2020	ISO 32000-2:2020	Not mentioned	EdDSA OIDs are not in the PDF spec.
PAdES support	Yes	Yes	Yes	No	PAdES references ECDSA, not EdDSA.

Table 2.5: Algorithm support across PDF viewers and libraries (as of early 2026)

Viewer / Library	RSA	ECDSA (P-256)	ECDSA (P-384/P-521)	EdDSA
Adobe Acrobat DC	Yes	Yes	Yes (P-384); Partial (P-521)	No
Foxit Reader	Yes	Yes	Yes (P-384); No (P-521)	No
PDFBox 3.0.0	Yes	Yes	Yes	Partial (via BouncyCastle provider)
iText 8.x	Yes	Yes	Yes	No
BouncyCastle 1.80	Yes	Yes	Yes	Yes (Ed25519, Ed448)
LibreOffice Draw	Yes	Yes	Partial	No

2.7 Connecting to PDF: OIDs and SignerInfo

When a PDF signature is created, the signing algorithm is encoded as an ASN.1 Object Identifier (OID) inside the CMS `SignedData` structure (RFC 5652 — Cryptographic Message Syntax (CMS)). Specifically, it appears in the `SignerInfo.digestEncryptionAlgorithm` field (the CMS name for the signature algorithm — ”digestEncryptionAlgorithm” is a historical misnomer that has caused confusion for two decades, but we are stuck with it).

Table 2.6: Common algorithm OIDs you will encounter in PDF signatures

Algorithm	OID	Where You See It
RSA (PKCS#1 v1.5)	`1.2.840.113549.1.1.1`	Legacy signatures. The hash algorithm is specified separately in `digestAlgorithm`.
SHA-256 with RSA	`1.2.840.113549.1.1.11`	Most common in modern PDF signatures.
SHA-384 with RSA	`1.2.840.113549.1.1.12`	Higher-security RSA.
SHA-512 with RSA	`1.2.840.113549.1.1.13`	Government and defense contexts.
RSASSA-PSS	`1.2.840.113549.1.1.10`	Probabilistic padding. Hash specified in parameters.
ECDSA with SHA-256	`1.2.840.10045.4.3.2`	Growing adoption.

Algorithm	OID	Where You See It
ECDSA with SHA-384	`1.2.840.10045.4.3.3`	Common with P-384 curves.
Ed25519	`1.3.101.112`	Not yet seen in production PDF signatures.

When you encounter the OID `1.2.840.113549.1.1.1` (plain RSA) in the `digestEncryptionAlgorithm` field, do not panic. This does not mean the signature used no hash. It means the CMS was constructed in the older style where the signature algorithm OID says "RSA" and the hash algorithm is determined solely by the `digestAlgorithm` field in the `SignerInfo`. Both styles — the combined OID like `1.2.840.113549.1.1.11` (SHA-256 with RSA) and the split approach — are valid per RFC 5652 — Cryptographic Message Syntax (CMS). PDFBox and BouncyCastle handle both transparently.

2.8 Java's Cryptographic Provider Model and BouncyCastle

Before we write any code, we need to understand how Java organizes its cryptographic implementations, because this directly affects how you generate keys, create signatures, and interact with BouncyCastle.

2.8.1 JCA, JCE, and the Provider Architecture

The **Java Cryptography Architecture (JCA)** defines abstract interfaces — `KeyPairGenerator`, `Signature`, `MessageDigest`, `Cipher` — and a **provider model** where concrete implementations are plugged in at runtime. The **Java Cryptography Extension (JCE)** extends JCA with additional capabilities. In modern Java (since JDK 9), the distinction is mostly historical.

```
// This does NOT specify which implementation to use
KeyPairGenerator kpg = KeyPairGenerator.getInstance("RSA");

// The JCA searches registered providers in priority order
// and returns the first one that supports "RSA"
```

The JDK ships with built-in providers: `SUN`, `SunRsaSign`, `SunEC`, `SunJCE`. These cover the basics — RSA, ECDSA with NIST curves, AES, SHA-2 hashes. For many applications, the built-in providers are sufficient.

For PDF signatures, they are not.

2.8.2 Why BouncyCastle Exists

The **Legion of the Bouncy Castle** (yes, that is the organization's legal name, registered in Australia) provides the most comprehensive open-source cryptographic library for Java. BouncyCastle is not an alternative to the JCA — it is a **JCA provider** that plugs into the same framework.

We need BouncyCastle for PDF signatures for several reasons:

1. **CMS/PKCS#7 implementation.** The JDK does not include a CMS library. BouncyCastle's `bcpkix` module provides `CMSSignedDataGenerator`, `CMSSignedData`, and everything else needed to construct and parse the CMS `SignedData` structures that PDF signatures require (RFC 5652 — Cryptographic Message Syntax (CMS)).
2. **Algorithm breadth.** BouncyCastle supports algorithms the JDK does not, including EdDSA (Ed25519, Ed448), certain elliptic curves, and legacy algorithms that appear in older signed PDFs.
3. **ASN.1 library.** BouncyCastle includes a complete ASN.1 parsing and generation library — essential for working with X.509 certificates, CMS structures, and OCSP responses at the byte level.
4. **Certificate generation.** The `bcpkix` module includes `X509v3CertificateBuilder` for creating X.509 certificates programmatically — something you cannot do with the JDK alone.

2.8.3 Provider Registration

```
import java.security.Security;
import org.bouncycastle.jce.provider.BouncyCastleProvider;

// Register BouncyCastle as a JCA provider
// This must happen before any cryptographic operations
Security.addProvider(new BouncyCastleProvider());
```

In a Spring Boot application, register the BouncyCastle provider in a `@PostConstruct` method of a `@Configuration` class. This ensures it is available before any bean that needs cryptographic operations is initialized. PDFalyzer does this in `BouncyCastleConfig.java`. If you forget this step, you will get `NoSuchAlgorithmException` or `NoSuchProviderException` at runtime — and the error message will not tell you that a missing provider registration is the cause.

2.9 Practical Code: Generating Key Pairs

2.9.1 RSA Key Pair Generation

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.Security;
import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;

import org.bouncycastle.jce.provider.BouncyCastleProvider;

public class RsaKeyPairExample {

    public static KeyPair generateRsaKeyPair(int keySize) throws Exception {
        Security.addProvider(new BouncyCastleProvider());

        KeyPairGenerator kpg = KeyPairGenerator.getInstance("RSA", "BC");
        kpg.initialize(keySize);
        return kpg.generateKeyPair();
    }
```

```
    public static void main(String[] args) throws Exception {
        KeyPair kp = generateRsaKeyPair(3072);

        RSAPublicKey pub = (RSAPublicKey) kp.getPublic();
        RSAPrivateKey priv = (RSAPrivateKey) kp.getPrivate();

        System.out.println("Algorithm:    " + pub.getAlgorithm());
        System.out.println("Key size:     " + pub.getModulus().bitLength() + " bits");
        System.out.println("Pub exponent: " + pub.getPublicExponent());
        System.out.println("Pub format:   " + pub.getFormat());  // X.509
        System.out.println("Priv format:  " + priv.getFormat()); // PKCS#8
    }
}
```

The output will show a 3072-bit key with public exponent 65537, the public key in X.509 (SubjectPublicKeyInfo) format, and the private key in PKCS#8 format. These format names — X.509 for public keys, PKCS#8 for private keys — are the standard encodings that appear in PEM files. Chapter 5 covers key material formats in detail.

2.9.2 ECDSA Key Pair Generation

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.Security;
import java.security.interfaces.ECPrivateKey;
import java.security.interfaces.ECPublicKey;
import java.security.spec.ECGenParameterSpec;

import org.bouncycastle.jce.provider.BouncyCastleProvider;

public class EcdsaKeyPairExample {

    public static KeyPair generateEcdsaKeyPair(String curveName) throws Exception {
        Security.addProvider(new BouncyCastleProvider());

        KeyPairGenerator kpg = KeyPairGenerator.getInstance("EC", "BC");
        kpg.initialize(new ECGenParameterSpec(curveName));
        return kpg.generateKeyPair();
    }

    public static void main(String[] args) throws Exception {
        // P-256: use the SECG name, not the NIST name
        KeyPair kp = generateEcdsaKeyPair("secp256r1");

        ECPublicKey pub = (ECPublicKey) kp.getPublic();
        ECPrivateKey priv = (ECPrivateKey) kp.getPrivate();

        System.out.println("Algorithm:   " + pub.getAlgorithm());
        System.out.println("Curve order: " + pub.getParams().getOrder().bitLength() + " bits");
        System.out.println("Pub format:  " + pub.getFormat());  // X.509
        System.out.println("Priv format: " + priv.getFormat()); // PKCS#8

        // The public key is a point on the curve (x, y)
```

```
        System.out.println("Pub X:       " + pub.getW().getAffineX().bitLength() + " bits");
        System.out.println("Pub Y:       " + pub.getW().getAffineY().bitLength() + " bits");
    }
}
```

Note the curve name: `"secp256r1"`, not `"P-256"`. The JCA uses SECG names in `ECGenParameterSpec`. BouncyCastle accepts both, but for portability, use the SECG name.

2.9.3 Performance Comparison

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.Security;
import java.security.spec.ECGenParameterSpec;

import org.bouncycastle.jce.provider.BouncyCastleProvider;

public class KeyPairComparisonExample {

    public static void main(String[] args) throws Exception {
        Security.addProvider(new BouncyCastleProvider());

        // RSA-3072: ~128-bit security
        long start = System.nanoTime();
        KeyPairGenerator rsaKpg = KeyPairGenerator.getInstance("RSA", "BC");
        rsaKpg.initialize(3072);
        KeyPair rsaKp = rsaKpg.generateKeyPair();
        long rsaTime = System.nanoTime() - start;

        // ECDSA P-256: ~128-bit security
        start = System.nanoTime();
        KeyPairGenerator ecKpg = KeyPairGenerator.getInstance("EC", "BC");
        ecKpg.initialize(new ECGenParameterSpec("secp256r1"));
        KeyPair ecKp = ecKpg.generateKeyPair();
        long ecTime = System.nanoTime() - start;

        System.out.println("RSA-3072 keygen:    " + rsaTime / 1_000_000 + " ms");
        System.out.println("ECDSA P-256 keygen: " + ecTime / 1_000_000 + " ms");
        System.out.println("RSA pub key size:   " + rsaKp.getPublic().getEncoded().length + " bytes");
        System.out.println("EC pub key size:    " + ecKp.getPublic().getEncoded().length + " bytes");
        System.out.println("RSA priv key size:  " + rsaKp.getPrivate().getEncoded().length + " bytes");
        System.out.println("EC priv key size:   " + ecKp.getPrivate().getEncoded().length + " bytes");
    }
}
```

On a typical machine, RSA-3072 key generation takes 200-1500 ms (it varies because finding large primes is probabilistic), while ECDSA P-256 key generation completes in under 5 ms. The encoded key sizes will show the RSA public key at roughly 400 bytes versus 90 bytes for the EC public key.

These numbers matter in practice. If you are building a self-signed certificate generator for testing (as we do in Chapter 29), RSA key generation is the bottleneck. If you are deploying a signing service that provisions per-user keys, the hundred-fold speed advantage of ECDSA key generation is worth considering.

2.10 Choosing an Algorithm for PDF Signatures

Given all of the above, what should you actually use?

For maximum compatibility: RSA-2048 or RSA-3072 with SHA-256. Every PDF viewer, every library, every validation service supports this combination. If your signed PDFs need to be validated by the widest possible audience — and they usually do — RSA with SHA-256 is the safe choice. Use 3072-bit keys for new deployments; 2048-bit is acceptable for existing systems but provides only 112-bit security.

For smaller signatures and modern infrastructure: ECDSA with P-256 and SHA-256. If you control the validation environment (e.g., an internal document management system) or if you are targeting modern viewers (Adobe Acrobat 2020+, PDFBox 3.x, iText 8.x), ECDSA P-256 gives you 128-bit security with much smaller key material. The CMS blob in `/Contents` will be noticeably smaller, which simplifies byte range estimation.

For future-proofing: watch EdDSA, but do not deploy it yet. EdDSA is the superior algorithm by every technical measure, but the PDF ecosystem has not caught up. When Adobe Acrobat adds EdDSA support — likely driven by EUDI wallet adoption — it will become the preferred choice.

The algorithm choice affects your certificate, not your code.

The PDF signing code in PDFBox does not care whether you give it an RSA key or an ECDSA key. The `SignatureInterface.sign(InputStream content)` method receives the document bytes and returns a CMS blob. Whether that blob contains an RSA signature or an ECDSA signature depends entirely on what key material you provide.

This means you can switch algorithms by changing your certificate and keystore — the signing code remains identical. We demonstrate this in Chapter 29.

2.11 Summary

Asymmetric cryptography provides two operations that make digital signatures possible: signing with a private key, and verifying with the corresponding public key. The security of the system rests on mathematical problems — integer factorization for RSA, the elliptic curve discrete logarithm for ECDSA and EdDSA — that are easy to set up but computationally infeasible to reverse.

RSA remains the dominant algorithm for PDF signatures due to universal support across all viewers and libraries. ECDSA with P-256 offers equivalent security at 128 bits with dramatically smaller keys and faster operations. EdDSA (Ed25519, Ed448) is technically superior but lacks ecosystem support for PDF signatures as of 2026.

Java's JCA provider model allows BouncyCastle to extend the JDK's cryptographic capabilities, and BouncyCastle is essential for PDF signatures because it provides the CMS, ASN.1, and certificate generation capabilities that the JDK lacks.

In the next chapter, we examine hash functions and the mechanics of the signing operation itself — how the document's bytes are reduced to a fixed-length digest, how that digest is signed, and what happens when the hash algorithm is compromised (as SHA-1 was, with devastating consequences for the PDF ecosystem).

Chapter 3

Hash Functions & Digital Signing

This chapter covers the two primitives that every digital signature depends on: cryptographic hash functions and the sign/verify operation. We go deep on the SHA family, explain why MD5 and SHA-1 are still haunting PDF signatures decades after they were broken, and walk through the exact sequence of bytes that turns a document into a verifiable commitment.

If you already know what a hash function does, skip to The SHA Family for the forensic details of which algorithms you will actually encounter in the wild. If you want code, jump to Java: Computing Hashes.

3.1 What Hash Functions Do

A cryptographic hash function takes an input of arbitrary length and produces a fixed-size output — the *digest* or *hash value*. That is it. One operation, no keys involved, purely deterministic. Feed in the same bytes, get the same digest, every time, on every machine.

But the properties that make this useful for digital signatures are specific and non-negotiable:

Table 3.1: Properties of a cryptographic hash function

Property	What It Means
Deterministic	Same input always produces the same output. No randomness, no state. `SHA-256("hello")` is the same value on your laptop, on a server in Frankfurt, and on a Mars rover running Java 21. Always.
Fixed-size output	Regardless of whether the input is 1 byte or 1 terabyte, the output is always the same length. SHA-256 produces 32 bytes (256 bits). SHA-512 produces 64 bytes (512 bits). This fixed-size property is what makes hash functions useful as input to signing algorithms.
Avalanche effect	Change a single bit of the input, and on average 50% of the output bits change. The output looks completely unrelated to the original. There is no way to look at two digests and determine whether their inputs were similar.
Pre-image resistance	Given a hash value `h`, it is computationally infeasible to find any input `m` such that `hash(m) = h`. You cannot reverse a hash function. This is not encryption — there is no key, no decryption, no way back.

Property	What It Means
Second pre-image resistance	Given an input `m1`, it is computationally infeasible to find a different input `m2` such that `hash(m1) = hash(m2)`. You cannot find a substitute document that produces the same hash.
Collision resistance	It is computationally infeasible to find *any* two distinct inputs `m1` and `m2` such that `hash(m1) = hash(m2)`. This is stronger than second pre-image resistance — the attacker gets to choose both inputs.

The distinction between second pre-image resistance and collision resistance matters enormously for digital signatures. An attacker who breaks collision resistance can prepare two documents with the same hash *before* either is signed — get one signed, then substitute the other. An attacker who breaks second pre-image resistance can find a substitute for an *already signed* document. Collision attacks are easier (birthday paradox — they require roughly 2^(n/2) work for an n-bit hash), which is why we retire algorithms at the collision resistance boundary, not the pre-image boundary.

3.2 Why We Hash Before Signing

You might wonder: why not just sign the document directly?

Three reasons, and they all matter:

1. **Performance.** RSA signing is slow. ECDSA signing is faster but still orders of magnitude slower than hashing. A 10 MB PDF would take an unacceptable time to sign directly with RSA-2048. Hashing reduces the input to 32 bytes (for SHA-256), and signing 32 bytes is fast regardless of the original document size.
2. **Fixed-size input.** RSA can only operate on data smaller than its modulus (typically 256 bytes for RSA-2048). You cannot feed a 10 MB document directly into RSA. Hashing solves this by producing a fixed-size value that fits within the algorithm's constraints.
3. **Security proof structure.** The security of digital signature schemes is typically proven in two parts: the hash function provides collision resistance, and the signing algorithm provides unforgeability. Separating the two allows each component to be analyzed (and replaced) independently. When SHA-1 was broken, we could switch to SHA-256 without changing the RSA or ECDSA implementation.

The process is always the same:

```
Sign:   digest = hash(data)  -->  signature = sign(digest, privateKey)
Verify: digest = hash(data)  -->  result    = verify(digest, signature, publicKey)
```

The signer computes the hash of the data, then signs the hash with their private key. The verifier computes the hash of the same data independently, then checks the signature against that hash using the signer's public key. If the hashes match and the signature is valid, the data has not been modified since signing.

The verifier computes the hash independently. The hash is not sent alongside the signature — it is recomputed from the original data. If an attacker modifies even one byte of the

data, the verifier's independently computed hash will differ from the hash that was signed, and verification will fail. This is the fundamental integrity guarantee of digital signatures.

3.3 The SHA Family

The Secure Hash Algorithm family, designed by the NSA and published by NIST, is what you will encounter in virtually every PDF signature created after 2010.

3.3.1 SHA-1: The Algorithm That Refused to Die

SHA-1 produces a 160-bit (20-byte) digest. It was published in 1995 as FIPS PUB 180-1 and became the default hash algorithm for essentially everything: TLS certificates, code signing, Git commits, and yes, PDF signatures.

The theoretical weaknesses appeared early. In 2005, Xiaoyun Wang, Yiqun Lisa Yin, and Hongbo Yu published an attack reducing the collision resistance of SHA-1 from the expected 2^80 operations to approximately 2^69 — still impractical, but a clear signal that the algorithm was weakening. NIST recommended transitioning away from SHA-1 by 2010 for digital signatures.

The practical break came on February 23, 2017.

3.3.1.1 The SHAttered Attack

A team from CWI Amsterdam (Marc Stevens) and Google (Elie Bursztein, Pierre Karpman) published the first practical SHA-1 collision. They produced two distinct PDF files — `shattered-1.pdf` and `shattered-2.pdf` — that had identical SHA-1 hashes but displayed completely different content. One showed a blue background with the text "SHAttered"; the other showed a red background with different text.

The numbers:

- The attack required approximately 2^63.1 SHA-1 computations
- That translated to roughly 6,500 CPU-years and 110 GPU-years of computation
- Google estimated the cost at approximately $110,000 using cloud computing resources
- The collision was found using a technique called an *identical-prefix collision attack*, where both files share a common prefix and the collision is achieved by appending carefully crafted "near-collision blocks"

Table 3.2: How the SHAttered collision was constructed

Component	Description
Common prefix	Both PDFs share an identical header and initial structure — valid PDF objects, page definitions, and the beginning of a JPEG image stream.
Collision blocks	128 bytes of carefully computed data inserted into each file at the same offset. These blocks differ between the two files but are constructed so that the SHA-1 internal state converges to the same value after processing them.

Component	Description
Divergent suffix	After the collision blocks, the remainder of each file can be *anything*. Since the SHA-1 state is identical at that point, any shared suffix produces the same final hash. The attackers used this to embed different JPEG image data, producing visually distinct documents.

The SHAttered attack has direct consequences for PDF signatures. If a PDF signature uses SHA-1 as its digest algorithm (SubFilter `adbe.pkcs7.sha1` or a CMS SignedData with SHA-1 as the digest algorithm), an attacker who controls the document content *before* signing can prepare two versions with the same SHA-1 hash. Get one version signed, and the other is also ”signed” — the signature validates on both. This is not a theoretical concern: the SHAttered team demonstrated it with actual PDF files.

3.3.1.2 The Cost of $110,000

It is worth dwelling on the attack cost because it illustrates how cryptographic security erodes over time. When SHAttered was published in 2017, $110,000 in cloud compute seemed significant. By 2020, the cost had dropped to an estimated $45,000 due to GPU price improvements. A 2020 paper by Ga{eacute}tan Leurent and Thomas Peyrin (”SHA-1 is a Shambles”) demonstrated a *chosen-prefix* collision attack (more powerful than identical-prefix) for an estimated $45,000.

By 2026, a SHA-1 collision can likely be generated for under $20,000. This is well within the budget of a motivated individual, let alone a criminal organization or nation-state. The cost trajectory is clear: it only gets cheaper.

3.3.1.3 Consequences for PDF and the Deprecation Timeline

The SHAttered attack triggered a cascade of changes across the industry:

- ISO 32000-2:2020 deprecated the `adbe.pkcs7.sha1` SubFilter
- Adobe Acrobat began showing warnings for SHA-1 signatures (initially a yellow warning, later escalated to an error on newer Acrobat versions)
- The PAdES standards (ETSI EN 319 142-1 V1.1.1 (2016-04)) require SHA-256 or stronger for all profiles
- The CA/Browser Forum prohibited SHA-1 in TLS certificates effective January 1, 2017 — a deadline that was already set before SHAttered, but the attack validated the urgency
- Git, which used SHA-1 for commit hashes, began its multi-year migration to SHA-256 (still ongoing as of 2026)
- Major browsers dropped SHA-1 certificate support: Chrome (January 2017), Firefox (January 2017), Edge (May 2017)

If you encounter a PDF with a SHA-1 based signature, do not immediately panic. A SHA-1 signature is not *forged* — it was created legitimately at a time when SHA-1 was considered acceptable. The risk is that someone *could* have prepared a collision before signing. For documents signed before 2017 by trusted parties, the practical risk is low. For documents signed after 2017 with SHA-1, ask pointed questions about why.

3.3.2 SHA-256, SHA-384, SHA-512: The Modern Workhorses

These are the algorithms you should be using. All three are members of the SHA-2 family, published by NIST in 2001 as FIPS PUB 180-2.

Table 3.3: SHA-2 family members used in PDF signatures

Algorithm	Output (bits)	Output (bytes)	Block size	Usage in PDF signatures
SHA-256	256	32	512 bits (64 bytes)	The default for virtually all modern PDF signatures. PAdES-B-B requires SHA-256 as the minimum. If you are signing a PDF today, use SHA-256 unless you have a specific reason for a stronger algorithm.
SHA-384	384	48	1024 bits (128 bytes)	Used when organizational policy requires stronger-than-256-bit hashing. Common in government and defense contexts. SHA-384 is actually a truncated SHA-512 with different initial values — it uses the 64-bit word size of SHA-512, which makes it faster than SHA-256 on 64-bit platforms.
SHA-512	512	64	1024 bits (128 bytes)	Maximum strength in the SHA-2 family. Rare in PDF signatures because SHA-256 is considered sufficient, and SHA-512 produces larger digests that increase CMS signature size. Used in high-security environments.

SHA-384's performance characteristic is surprising and worth knowing. Because it operates on 64-bit words (like SHA-512) rather than 32-bit words (like SHA-256), SHA-384 is actually *faster* than SHA-256 on modern 64-bit processors. If you need to hash large documents and your platform is 64-bit (it is), SHA-384 gives you more security bits at equal or better performance. This is one of those facts that sounds wrong but is measurably true.

3.3.2.1 SHA-3: The Insurance Policy

SHA-3 (Keccak), published as FIPS 202 in 2015, is a completely different design from SHA-2 — it uses a sponge construction rather than the Merkle-Damgard construction of SHA-1

and SHA-2. SHA-3 was selected by NIST specifically because it has *no* structural similarity to SHA-2, meaning any attack on SHA-2 would not apply.

You will not encounter SHA-3 in PDF signatures today. No PDF specification references it, no PAdES profile requires it, and no mainstream PDF library defaults to it. It exists as insurance: if SHA-2 is ever broken, SHA-3 is ready to replace it without a multi-year standardization process.

BouncyCastle 1.80 supports SHA-3 for signing, so the migration path exists when needed.

3.4 MD5: The Ghost in the Machine

MD5 produces a 128-bit (16-byte) digest. It was designed by Ronald Rivest (the ”R” in RSA) in 1991 and published as RFC 1321. It was once the standard hash function for everything.

MD5 is *broken*. Not theoretically, not in some academic sense — practically, completely, irrecoverably broken for collision resistance.

3.4.1 The Wang-Yu Attacks (2004)

In August 2004, Xiaoyun Wang and Hongbo Yu (the same Wang who later weakened SHA-1) presented the first practical MD5 collision at CRYPTO 2004. They could find two distinct inputs with the same MD5 hash in less than an hour on a standard PC. The collision required approximately 2^39 MD5 operations — billions of times easier than the expected 2^64.

By 2006, Vlastimil Klima had reduced the attack to *one minute* on a single notebook computer. By 2008, Alexander Sotirov and Marc Stevens demonstrated a rogue CA attack using MD5 collisions, creating a fraudulent intermediate CA certificate that chained to a legitimate root. This was not an academic exercise — they obtained a real certificate from a real CA (RapidSSL, which used MD5 for certificate signing) and used MD5 collisions to create a rogue CA certificate that any browser would trust.

Today, MD5 collisions can be generated in *seconds* on commodity hardware. The algorithm is fully broken for any security-sensitive purpose.

3.4.2 Why MD5 Still Appears in Legacy PDFs

Despite being broken for collision resistance since 2004, you will encounter MD5 in:

- Old PDF signatures created before 2005 using `adbe.pkcs7.detached` with MD5 as the digest algorithm
- Legacy government and enterprise PDFs that were signed by systems deployed in the early 2000s
- Some PKCS#12 keystores that use MD5-based key derivation for the encryption of the private key (this is a different use of MD5 — key derivation, not signature hashing — and while not ideal, the risk profile is different from using MD5 for signing)
- The `/ID` array in PDF trailers, which traditionally contained MD5 hashes of file metadata (this is not security-sensitive)

If you encounter a PDF signed with MD5 as the digest algorithm, the signature provides *no* collision resistance. An attacker who controlled the document before signing could have

prepared a collision. For documents signed by trusted parties before 2004, the practical risk is limited (the attack was not known yet). For anything signed after 2004, the signature's integrity guarantee is meaningless against a determined attacker.

3.5 The Signing Process Step by Step

Now we connect the hash function to the asymmetric cryptography from Chapter 1 and trace the exact sequence of operations that produces a PDF digital signature.

3.5.1 Step 1: Hash the Document

The signing library reads the bytes of the PDF specified by the `/ByteRange` array. This is *not* the entire file — it is everything except the `/Contents` hex string, which is where the signature will be placed. The gap exists because you cannot include the signature value in the data being signed (you would need to know the signature before computing it — a circular dependency).

The library feeds these bytes through the chosen hash algorithm:

```
SHA-256( byteRange[0..offset1] || byteRange[offset2..end] ) = digest (32 bytes)
```

This 32-byte digest is the *fingerprint* of the document at the moment of signing.

3.5.2 Step 2: Pad the Hash

Before the hash can be signed with the private key, it must be padded to the correct size for the signing algorithm. For RSA, there are two padding schemes:

PKCS#1 v1.5 padding (the traditional scheme): The hash is prefixed with a DER-encoded `DigestInfo` structure that identifies the hash algorithm, then padded with `0xFF` bytes to fill the modulus size. The structure looks like:

```
DigestInfo ::= SEQUENCE {
    digestAlgorithm  AlgorithmIdentifier,  -- OID for SHA-256
    digest           OCTET STRING           -- the 32-byte hash
}
```

The padded message is: `0x00 0x01 [0xFF padding] 0x00 [DigestInfo DER bytes]`.

RSASSA-PSS (Probabilistic Signature Scheme): A more modern padding scheme that includes random salt, making signatures non-deterministic. PSS provides a tighter security proof than PKCS#1 v1.5, but it is less commonly used in PDF signatures because the entire PDF ecosystem was built around PKCS#1 v1.5. Some CAs issue PSS certificates, and you may encounter the OID `1.2.840.113549.1.1.10` (RSASSA-PSS) in the `SignerInfo.signatureAlgorithm` field.

For ECDSA, there is no padding step — the hash is used directly as input to the ECDSA signing equation.

The choice of padding scheme is not just a technical detail. PKCS#1 v1.5 has known theoretical weaknesses (Bleichenbacher's 1998 attack on encryption; related issues apply to signatures in certain implementation scenarios). RSASSA-PSS is provably secure under standard assumptions. In practice, PKCS#1 v1.5 remains safe for signatures (as opposed to encryption,

where the Bleichenbacher attack is devastating), but security-conscious deployments should prefer PSS. The ETSI EN 319 142-1 V1.1.1 (2016-04) standards allow both.

3.5.3 Step 3: Encrypt with Private Key

The padded hash (for RSA) or raw hash (for ECDSA) is processed with the private key:

For RSA: $s = paddedHash^d \ mod \ n$, where d is the private exponent and n is the modulus. The result s is the signature value — a number the same size as the modulus (256 bytes for RSA-2048, 384 bytes for RSA-3072).

For ECDSA: The algorithm generates a nonce k (either random or deterministic per RFC 6979), computes a curve point $(x1, y1) = k * G$, and derives the signature pair (r, s) from $x1$, the hash, and the private key d. The signature is the pair (r, s), each approximately the same size as the curve order (32 bytes each for P-256, totaling 64 bytes).

3.5.4 Step 4: Package as CMS SignedData

The raw signature value is not written directly into the PDF. It is wrapped in a CMS `SignedData` structure (RFC 5652 — Cryptographic Message Syntax (CMS) Section 5) that includes:

- The signature value itself
- The signer's certificate (and often the full certificate chain)
- The `SignerInfo` containing the digest algorithm, signature algorithm, and signed attributes
- The `messageDigest` signed attribute — the hash from Step 1
- The `signingTime` attribute (optional but common)
- The `contentType` attribute (always `id-data` for PDF)

This CMS blob is DER-encoded, then hex-encoded (each byte becomes two hex characters), and written into the `/Contents` entry of the signature dictionary.

3.6 The Verification Process Step by Step

Verification is the mirror image of signing, with one crucial addition: the verifier must independently recompute the hash and compare it against what the signer claims to have signed.

3.6.1 Step 1: Extract the CMS SignedData

The validator reads the `/Contents` hex string from the signature dictionary and decodes it from hex to DER to obtain the CMS `SignedData` structure.

3.6.2 Step 2: Re-Hash the Byte Range

The validator reads the `/ByteRange` array, extracts the specified byte ranges from the PDF file, and computes the hash using the algorithm specified in the `SignerInfo.digestAlgorithm` field.

This is the critical integrity check: if anyone has modified the PDF bytes after signing, this hash will differ from the original.

3.6.3 Step 3: Compare Hashes

The validator extracts the `messageDigest` attribute from the CMS `SignerInfo` signed attributes. This attribute contains the hash that the signer computed over the byte range content. The validator compares this with the independently computed hash from Step 2.

If they do not match, the document has been modified and validation fails immediately. There is no need to check the cryptographic signature.

3.6.4 Step 4: Verify the Cryptographic Signature

If the hashes match, the validator checks that the signature over the signed attributes (which include the `messageDigest` attribute) is cryptographically valid. For RSA: compute $m' = s^e \bmod n$ and verify that the result matches the padded hash of the signed attributes. For ECDSA: verify the (r, s) pair against the hash using the public key point Q.

If this check passes, the signature is **cryptographically valid** — it was produced by someone holding the private key that corresponds to the public key in the certificate.

Cryptographic validity is necessary but not sufficient. A signature can be cryptographically valid and still be *untrustworthy*:

- The certificate might be expired.
- The certificate might have been revoked.
- The certificate might not chain to a trusted root.
- The signed byte range might not cover the entire document (the Bochum attacks exploited exactly this — see Chapter 24).
- The certificate might lack the `digitalSignature` or `nonRepudiation` key usage extension (see Chapter 3).

Signature **validation** — as opposed to mere cryptographic verification — requires checking all of these conditions. Chapter 16 covers the full validation process.

3.7 DigestMethod in PDF Signatures

The hash algorithm used for a PDF signature is determined by the `/SubFilter` value and the CMS `SignerInfo.digestAlgorithm` field.

Table 3.4: How the hash algorithm is specified

SubFilter	Hash Determination	Notes
`adbe.pkcs7.detached`	CMS `digestAlgorithm` field	The hash algorithm is specified inside the CMS structure. Can be any algorithm the signer chooses (SHA-256, SHA-384, SHA-512, or legacy SHA-1/MD5).

SubFilter	Hash Determination	Notes
`adbe.pkcs7.sha1`	Always SHA-1	Deprecated by ISO 32000-2:2020. The SHA-1 hash of the entire byte range is embedded as the CMS content. Do not create new signatures with this SubFilter.
`ETSI.CAdES.detached`	CMS `digestAlgorithm` field	PAdES-compliant signatures. Must use SHA-256 or stronger per ETSI EN 319 142-1 V1.1.1 (2016-04). This is the recommended SubFilter for new signatures.
`ETSI.RFC3161`	CMS `digestAlgorithm` field	Document-level timestamp tokens (RFC 3161 — Internet X.509 PKI Time-Stamp Protocol (TSP)). The hash algorithm is specified in the timestamp request.

3.8 Hash Algorithm OIDs

When a hash algorithm appears inside a CMS SignedData structure, it is identified by an ASN.1 Object Identifier (OID). You need to know these OIDs because they appear in the raw bytes of every PDF signature, and when something goes wrong, you will be reading hex dumps.

Table 3.5: Hash algorithm OIDs used in CMS (RFC 5652 — Cryptographic Message Syntax (CMS) and NIST)

Algorithm	OID	Notes
MD5	`1.2.840.113549.2.5`	Legacy only. Do not use for new signatures.
SHA-1	`1.3.14.3.2.26`	Deprecated. Still accepted by some validators for backward compatibility.
SHA-224	`2.16.840.1.101.3.4.2.4`	Rarely used. Exists for completeness.
SHA-256	`2.16.840.1.101.3.4.2.1`	The default for modern PDF signatures. PAdES minimum.
SHA-384	`2.16.840.1.101.3.4.2.2`	Used in higher-security contexts.
SHA-512	`2.16.840.1.101.3.4.2.3`	Maximum SHA-2 strength.

Algorithm	OID	Notes
SHA3-256	`2.16.840.1.101.3.4.2.8`	Future use. Not currently seen in PDF signatures.
SHA3-384	`2.16.840.1.101.3.4.2.9`	Future use.
SHA3-512	`2.16.840.1.101.3.4.2.10`	Future use.

The OID `2.16.840.1.101.3.4.2.1` (SHA-256) is the one you will see most often. The prefix `2.16.840.1.101.3.4` is the NIST algorithm arc. The `.2` subtree is hash algorithms, and `.1` is SHA-256 specifically. Memorize this OID — you will recognize it instantly in ASN.1 dumps and hex views.

The OID structure `1.2.840.113549` is the RSA Security arc (the company, now part of EMC/Dell). MD5 (…`2.5`) and various PKCS algorithms live under this prefix. The OID `1.3.14.3.2.26` for SHA-1 is under the OIW (Open Information Warehouse) arc — a historical artifact from the 1990s standardization process.

3.9 Signature Algorithm OIDs

The signing operation itself also has OIDs, which combine the hash algorithm with the public-key algorithm:

Table 3.6: Common signature algorithm OIDs in PDF signatures

Algorithm Combination	OID	Notes
SHA-256 with RSA	`1.2.840.113549.1.1.11`	Most common combination in current PDF signatures.
SHA-384 with RSA	`1.2.840.113549.1.1.12`	Higher-security RSA.
SHA-512 with RSA	`1.2.840.113549.1.1.13`	Maximum SHA-2 + RSA.
SHA-256 with ECDSA	`1.2.840.10045.4.3.2`	Growing in usage. Smaller signatures than RSA.
SHA-384 with ECDSA	`1.2.840.10045.4.3.3`	Common with P-384 curves.
SHA-1 with RSA	`1.2.840.113549.1.1.5`	Deprecated. Legacy signatures only.

Algorithm Combination	OID	Notes
RSASSA-PSS	`1.2.840.113549.1.1.10`	Probabilistic RSA. The hash algorithm is specified in the parameters, not in the OID itself.

When parsing CMS SignedData from a PDF signature, you may encounter a mismatch between the `digestAlgorithm` in the SignerInfo (which specifies the hash algorithm) and the hash algorithm implied by the `signatureAlgorithm` OID. For example, the `digestAlgorithm` might say SHA-256, but the `signatureAlgorithm` OID might be `1.2.840.113549.1.1.1` (RSA encryption, with no hash specified). This is technically allowed by RFC 5652 — Cryptographic Message Syntax (CMS) (the signature algorithm can be "RSA" and the hash is determined by the `digestAlgorithm` field), but it confuses many parsers. Always use the `digestAlgorithm` field as the authoritative source for which hash was used.

3.10 When Hash and Signature Algorithms Collide

An interesting question arises: what happens when you pair a hash algorithm with a signature algorithm designed for a different security level? For example, SHA-256 (128-bit security) with ECDSA P-384 (192-bit security)?

The answer is straightforward: **the overall security level is the minimum of the two components**. SHA-256 with P-384 provides 128-bit security, not 192-bit. The hash function is the bottleneck.

Standard pairings:

- **SHA-256 with RSA-3072 or ECDSA P-256**: 128-bit security across the board. The default for most use cases.
- **SHA-384 with RSA-7680 or ECDSA P-384**: 192-bit security. Government/defense contexts.
- **SHA-512 with ECDSA P-521**: 256-bit security. Rarely needed.

Mismatched pairings are not *wrong* — they work correctly — but they waste resources. Using SHA-512 with RSA-2048 gives you 112-bit security (limited by RSA) while computing an unnecessarily expensive hash. Using SHA-256 with P-384 gives you 128-bit security (limited by SHA-256) while using a curve that provides more security than the hash can deliver.

In practice, almost everyone uses SHA-256 with either RSA-2048/3072 or ECDSA P-256. The matched security levels make the overall system coherent, and SHA-256 is fast, universally supported, and has no known weaknesses. Do not overthink this unless you have specific compliance requirements that mandate higher security levels.

3.11 Java: Computing Hashes

The `java.security.MessageDigest` class is Java's built-in hash computation API. For basic hashing, you do not need BouncyCastle — the JDK provides SHA-256 and SHA-512 out of the

box.

```
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

public class HashExample {

    /**
     * Compute the SHA-256 hash of the given data.
     * Returns the raw 32-byte digest.
     */
    public static byte[] sha256(byte[] data) throws NoSuchAlgorithmException {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        return md.digest(data);
    }

    /**
     * Compute a hash using a specified algorithm.
     * Algorithm names: "SHA-256", "SHA-384", "SHA-512", "SHA-1", "MD5".
     */
    public static byte[] hash(byte[] data, String algorithm)
            throws NoSuchAlgorithmException {
        MessageDigest md = MessageDigest.getInstance(algorithm);
        return md.digest(data);
    }

    /**
     * Demonstrate the avalanche effect: one-bit change produces
     * a completely different hash.
     */
    public static void demonstrateAvalanche() throws NoSuchAlgorithmException {
        byte[] original = "The quick brown fox jumps over the lazy dog".getBytes();
        byte[] modified = "The quick brown fox jumps over the lazy dog.".getBytes();

        byte[] hash1 = sha256(original);
        byte[] hash2 = sha256(modified);

        // hash1 and hash2 will be completely different despite
        // the inputs differing by a single period character
    }
}
```

- The only difference is a trailing period. The SHA-256 outputs differ in approximately 50% of their bits.

For PDF signing, you typically do not call `MessageDigest` directly. Instead, the signing library (PDFBox + BouncyCastle) computes the hash as part of the CMS signing operation. But knowing how to compute hashes independently is essential for debugging — when a signature fails validation, one of the first things to check is whether the hash of the byte range matches the `messageDigest` attribute in the CMS SignerInfo.

3.12 Java: Signing with BouncyCastle

Here is the complete signing flow using BouncyCastle 1.80. This code signs a byte array and produces a CMS SignedData structure — exactly what ends up in a PDF signature's `/Contents` value.

```
import java.security.KeyStore;
import java.security.PrivateKey;
import java.security.Security;
import java.security.cert.Certificate;
import java.security.cert.X509Certificate;
import java.util.ArrayList;
import java.util.List;

import org.bouncycastle.cert.jcajce.JcaCertStore;
import org.bouncycastle.cms.CMSProcessableByteArray;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.CMSSignedDataGenerator;
import org.bouncycastle.cms.jcajce.JcaSignerInfoGeneratorBuilder;
import org.bouncycastle.jce.provider.BouncyCastleProvider;
import org.bouncycastle.operator.ContentSigner;
import org.bouncycastle.operator.jcajce.JcaContentSignerBuilder;
import org.bouncycastle.operator.jcajce.JcaDigestCalculatorProviderBuilder;

public class SigningExample {

    static {
        Security.addProvider(new BouncyCastleProvider());
    }

    /**
     * Sign data and return DER-encoded CMS SignedData.
     *
     * @param dataToSign     the raw bytes to sign (byte range content)
     * @param keyStore       loaded PKCS#12 key store
     * @param alias          key entry alias
     * @param password       key password
     * @param hashAlgorithm  "SHA-256", "SHA-384", or "SHA-512"
     * @return               DER-encoded CMS SignedData
     */
    public static byte[] signData(byte[] dataToSign, KeyStore keyStore,
                                  String alias, char[] password,
                                  String hashAlgorithm) throws Exception {

        PrivateKey privateKey = (PrivateKey) keyStore.getKey(alias, password);
        Certificate[] chain = keyStore.getCertificateChain(alias);
        X509Certificate signingCert = (X509Certificate) chain[0];

        List<Certificate> certList = new ArrayList<>();
        for (Certificate cert : chain) {
            certList.add(cert);
        }

        String signatureAlgorithm = hashAlgorithm + "withRSA";
```

```
        ContentSigner signer = new JcaContentSignerBuilder(signatureAlgorithm)
                .setProvider("BC")
                .build(privateKey);

        CMSSignedDataGenerator generator = new CMSSignedDataGenerator();
        generator.addSignerInfoGenerator(
                new JcaSignerInfoGeneratorBuilder(
                        new JcaDigestCalculatorProviderBuilder()
                                .setProvider("BC")
                                .build()
                ).build(signer, signingCert)
        );
        generator.addCertificates(new JcaCertStore(certList));

        CMSProcessableByteArray content = new CMSProcessableByteArray(dataToSign);
        CMSSignedData signedData = generator.generate(content, false);

        return signedData.getEncoded();
    }
}
```

- Extract the private key from the key store. In production, use an HSM (see Chapter 5).
- Include the full certificate chain so validators can build the path without fetching intermediates.
- For ECDSA keys, use `hashAlgorithm + "withECDSA"` instead. The algorithm name must match the key type.
- Embedding certificates in the CMS is critical — without them, the validator has no chain to verify.
- The `false` parameter means *detached* signature: the content is not included in the CMS structure.
- The returned bytes are DER-encoded and will be hex-encoded to fill `/Contents`.

The `false` parameter in `generator.generate(content, false)` is the single most important boolean in PDF signing. It means "detached signature" — the signed content is *not* embedded in the CMS structure. PDF requires detached signatures because the content (the byte range) lives in the PDF file itself. If you pass `true`, the CMS will include the content, and some validators will reject it because the `/SubFilter` says `adbe.pkcs7.detached` but the CMS contains encapsulated content.

3.13 Debugging Hash Mismatches

When a PDF signature fails validation, the most common root cause is a hash mismatch: the hash of the byte range content does not match the `messageDigest` signed attribute in the CMS SignerInfo.

Here is how to debug this systematically:

```
import java.security.MessageDigest;

import org.bouncycastle.asn1.ASN1OctetString;
import org.bouncycastle.asn1.cms.Attribute;
```

```
import org.bouncycastle.asn1.cms.CMSAttributes;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.SignerInformation;
import org.bouncycastle.util.encoders.Hex;

public class HashDebugExample {

    /**
     * Compare the hash in the CMS messageDigest attribute against
     * a freshly computed hash of the byte range content.
     */
    public static void debugHashMismatch(byte[] cmsBytes, byte[] byteRangeContent)
            throws Exception {

        CMSSignedData signedData = new CMSSignedData(cmsBytes);

        for (SignerInformation signerInfo : signedData.getSignerInfos().getSigners()) {

            // 1. Get the digest algorithm from the SignerInfo
            String digestAlgOid = signerInfo.getDigestAlgOID();
            String digestAlgName = mapOidToName(digestAlgOid);

            // 2. Compute the hash of the byte range content independently
            MessageDigest md = MessageDigest.getInstance(digestAlgName);
            byte[] computedHash = md.digest(byteRangeContent);

            // 3. Extract the messageDigest attribute from signed attributes
            Attribute mdAttr = signerInfo.getSignedAttributes()
                    .get(CMSAttributes.messageDigest);
            byte[] embeddedHash = ASN1OctetString.getInstance(
                    mdAttr.getAttrValues().getObjectAt(0)
            ).getOctets();

            // 4. Compare
            boolean match = MessageDigest.isEqual(computedHash, embeddedHash);
            // If match is false: the byte range was extracted incorrectly,
            // or the PDF was modified after signing
        }
    }

    private static String mapOidToName(String oid) {
        return switch (oid) {
            case "2.16.840.1.101.3.4.2.1" -> "SHA-256";
            case "2.16.840.1.101.3.4.2.2" -> "SHA-384";
            case "2.16.840.1.101.3.4.2.3" -> "SHA-512";
            case "1.3.14.3.2.26"           -> "SHA-1";
            case "1.2.840.113549.2.5"      -> "MD5";
            default -> throw new IllegalArgumentException("Unknown OID: " + oid);
        };
    }
}
```

When debugging a hash mismatch, the most common causes are: (1) the byte range was extracted incorrectly — off by one byte, or not accounting for the gap correctly; (2) the PDF was modified after signing (perhaps by a tool that did a full save instead of incremental

save); (3) the byte range array values do not add up to cover the entire file (there are bytes before or after the byte range that are not part of the signed content but should be — see Chapter 13 for the gory details).

3.14 Summary

Digital signatures are built on two primitives: hash functions provide integrity (any modification is detectable), and asymmetric signing provides authentication (only the private key holder could have created the signature) and non-repudiation (the signer cannot deny having signed).

For PDF signatures:

- **Use SHA-256** as the digest algorithm. It is the PAdES minimum, universally supported, and has no known weaknesses.
- **SHA-1 is deprecated.** The SHAttered attack proved practical collision attacks are feasible. Do not create new SHA-1 signatures.
- **MD5 is broken.** Collisions can be generated in seconds. Legacy only.
- **The hash is always computed over the byte range**, not the entire PDF file. The byte range excludes the `/Contents` value (which contains the signature itself — you cannot sign your own signature).
- **CMS detached signatures** are the standard: the content is not embedded in the CMS structure.
- **PKCS#1 v1.5 padding** is dominant in PDF signatures, but **RSASSA-PSS** is gaining adoption for new deployments.

The next chapter introduces X.509 certificates — the mechanism that binds a public key to an identity, turning a raw cryptographic operation into a trust relationship.

3.15 X.509 Certificates

A digital signature proves that someone with a specific private key signed some data. But *who* holds that private key? A public key is just a number — a very large number, but a number with no name attached.

X.509 certificates solve this problem. A certificate is a signed statement that binds a public key to an identity: ”This public key belongs to Alice Smith at Acme Corp, and I, DigiCert, vouch for it.” This chapter takes apart the X.509 certificate format, field by field, extension by extension, with the level of detail you need to debug real-world certificate problems in PDF signatures.

3.15.1 What a Certificate Is

At its core, an X.509 certificate is a data structure containing three things:

1. A **subject** — the identity (person, organization, or device) that the certificate represents
2. A **public key** — the subject’s public key, which can be used to verify signatures created with the corresponding private key

3. A **digital signature** from a **Certificate Authority (CA)** — a trusted third party that vouches for the binding between the subject and the public key

The CA's signature is what gives the certificate its authority. Without it, you have a self-signed statement: "I am Alice, and here is my public key, trust me." With it, you have a verifiable claim backed by an entity that (presumably) verified Alice's identity before signing.

The X.509 standard is defined in ITU-T Recommendation X.509 (originally from 1988, extensively revised since) and profiled for Internet use in RFC 5280 — Internet X.509 PKI Certificate and CRL Profile. When we talk about X.509 certificates in the context of PDF signatures, we mean the RFC 5280 — Internet X.509 PKI Certificate and CRL Profile profile — the Internet PKI profile that every browser, every operating system, and every PDF viewer actually implements.

3.15.2 ASN.1 and DER Encoding

Before we look at the certificate structure, we need to talk about how certificates are encoded, because you *will* encounter raw certificate bytes when debugging PDF signatures, and you need to understand what you're looking at.

Certificates are defined using **ASN.1** (Abstract Syntax Notation One), a schema language for describing data structures. ASN.1 is to certificates what JSON Schema is to JSON — it defines the structure, types, and constraints. But unlike JSON, ASN.1 data is encoded in a compact binary format.

The encoding used for certificates is **DER** (Distinguished Encoding Rules), a subset of BER (Basic Encoding Rules) that produces exactly one encoding for any given value. This uniqueness property is critical: because a certificate's signature covers its DER encoding, there can be only one valid encoding — otherwise the signature wouldn't verify.

3.15.2.1 Tag-Length-Value

DER uses a Tag-Length-Value (TLV) structure. Every data element consists of:

- **Tag** (1 or more bytes): identifies the type (SEQUENCE, INTEGER, BIT STRING, OID, etc.)
- **Length** (1 or more bytes): the number of bytes in the value
- **Value** (variable): the actual data

Table 3.7: Common ASN.1 tags in certificates

Tag (hex)	ASN.1 Type	Where You'll See It
`30`	SEQUENCE (constructed)	The outermost wrapper of the certificate, and of most sub-structures (tbsCertificate, extensions, etc.)
`31`	SET (constructed)	Relative Distinguished Name components in the issuer and subject fields
`02`	INTEGER	Serial number, RSA modulus and exponent, version number

Tag (hex)	ASN.1 Type	Where You'll See It
`03`	BIT STRING	The signature value, and the subject public key value
`04`	OCTET STRING	Extension values, key identifiers
`05`	NULL	Algorithm parameters (RSA with SHA-256 has NULL parameters)
`06`	OBJECT IDENTIFIER	Algorithm OIDs, extension OIDs, policy OIDs
`0C`	UTF8String	Subject and issuer name components
`13`	PrintableString	Country codes and other restricted-character fields
`17`	UTCTime	Validity dates (for dates before 2050)
`18`	GeneralizedTime	Validity dates (for dates from 2050 onward, per RFC 5280 — Internet X.509 PKI Certificate and CRL Profile §4.1.2.5)
`A0`	Context-specific [0]	Version number (explicit tag), extensions container
`A3`	Context-specific [3]	Extensions container in tbsCertificate

When you see `30 82 03 A5` at the start of a binary blob, you're looking at a SEQUENCE with a two-byte length (the `82` means "the next 2 bytes are the length"), specifically `03 A5` = 933 bytes. This is almost certainly a certificate or a CMS structure. The `82` prefix on the length byte is the tell: `81` means one-byte length follows, `82` means two-byte length follows. Certificate structures are almost always in the `82` range (258-65535 bytes).

3.15.2.2 PEM vs. DER

You will encounter certificates in two physical formats:

- **DER**: raw binary. File extensions: `.der`, `.cer`, `.crt` (ambiguous)
- **PEM**: Base64-encoded DER wrapped in `-----BEGIN CERTIFICATE-----` / `-----END CERTIFICATE-----` headers. File extensions: `.pem`, `.crt` (ambiguous)

```
-----BEGIN CERTIFICATE-----
MIIFYjCCBEqgAwIBAgIQd70NbNs2+RrqIQ/E8FjTDTANBgkqhkiG9w0BAQsFAMDQ
... (Base64-encoded DER bytes) ...
-----END CERTIFICATE-----
```

The content between the headers is the DER encoding, Base64-encoded with line breaks every 64 characters. Nothing more, nothing less. To get the DER bytes, strip the headers and Base64-decode.

The `.crt` extension is ambiguous — it can be either DER or PEM. If you're loading a certificate and get a parsing error, try the other format. Java's `CertificateFactory` handles both, but BouncyCastle's `PEMParser` expects PEM, and raw `X509CertificateHolder(byte[])` expects DER.

3.15.3 Certificate Structure

An X.509v3 certificate has three top-level fields, defined in RFC 5280 — Internet X.509 PKI Certificate and CRL Profile Section 4.1:

```
Certificate ::= SEQUENCE {
    tbsCertificate       TBSCertificate,
    signatureAlgorithm   AlgorithmIdentifier,
    signatureValue       BIT STRING
}
```

- The ”to be signed” certificate — all the data that the CA signs
- The algorithm the CA used to create the signature (must match the algorithm in tbsCertificate)
- The CA’s digital signature over the DER encoding of tbsCertificate

The CA signs `tbsCertificate` — the entire inner structure, DER-encoded. The `signatureAlgorithm` and `signatureValue` are outside the signed region (they can’t be inside it — the signature can’t sign itself, just as a PDF’s `/Contents` can’t be part of its own byte range).

3.15.3.1 tbsCertificate

This is where all the interesting data lives:

```
TBSCertificate ::= SEQUENCE {
    version          [0] EXPLICIT INTEGER DEFAULT v1,
    serialNumber         INTEGER,
    signature            AlgorithmIdentifier,
    issuer               Name,
    validity             Validity,
    subject              Name,
    subjectPublicKeyInfo SubjectPublicKeyInfo,
    issuerUniqueID   [1] IMPLICIT BIT STRING OPTIONAL,
    subjectUniqueID  [2] IMPLICIT BIT STRING OPTIONAL,
    extensions       [3] EXPLICIT Extensions OPTIONAL
}
```

- Version: 0 = v1, 1 = v2, 2 = v3. All modern certificates are v3. If you see v1, it’s either very old or self-signed for testing.
- Serial number: unique within the issuing CA. Must be a positive integer, at most 20 bytes. CAs use random serial numbers to prevent hash-based attacks.
- Signature algorithm: must match the outer `signatureAlgorithm` field. This redundancy is specified by RFC 5280 — Internet X.509 PKI Certificate and CRL Profile to prevent algorithm substitution attacks.
- Issuer: the Distinguished Name (DN) of the CA that signed this certificate.
- Validity: `notBefore` and `notAfter` timestamps defining the certificate’s validity period.
- Subject: the Distinguished Name of the entity this certificate represents.
- Subject public key: the algorithm and the actual public key bytes.
- Issuer/subject unique IDs: deprecated, never used in practice. Ignore them.
- Extensions: the v3 extensions that make certificates actually useful. See Certificate Extensions below.

3.15.3.1.1 Distinguished Names The issuer and subject fields use X.500 Distinguished Names — a hierarchical naming scheme from the 1980s that looks like this:

```
CN=Alice Smith, O=Acme Corporation, L=San Francisco, ST=California, C=US
```

Each component is a Relative Distinguished Name (RDN) identified by an OID:

Table 3.8: Common RDN OIDs

Abbreviation	OID	Meaning
CN	`2.5.4.3`	Common Name — the certificate holder's name
O	`2.5.4.10`	Organization
OU	`2.5.4.11`	Organizational Unit (deprecated in TLS certificates since 2022, but still used in document signing certificates)
L	`2.5.4.7`	Locality (city)
ST	`2.5.4.8`	State or Province
C	`2.5.4.6`	Country (two-letter ISO 3166-1 code)
serialNumber	`2.5.4.5`	Subject serial number (not the certificate serial number — this is a personal or organizational identifier, like a tax ID or VAT number)
E / emailAddress	`1.2.840.113549.1.9.1`	Email address (legacy; modern certificates use the Subject Alternative Name extension instead)

The DN ordering in X.509 certificates is *most specific first* (CN before O before C), which is the opposite of LDAP directory convention (C before O before CN). This catches people off guard when comparing DNs between different tools. Java's `X509Certificate.getSubjectX500Principal().getName()` returns RFC 2253 format (CN first), while some tools show LDAP order (C first). When comparing issuer DNs for chain building, always compare the DER encoding, not the string representation.

3.15.3.1.2 Validity Period

```
Validity ::= SEQUENCE {
    notBefore   Time,    -- certificate becomes valid
    notAfter    Time     -- certificate expires
}
```

RFC 5280 — Internet X.509 PKI Certificate and CRL Profile §4.1.2.5 specifies that dates through 2049 use UTCTime (`YYMMDDHHMMSSZ`) and dates in 2050 and beyond use GeneralizedTime (`YYYYMMDDHHMMSSZ`). This means every certificate issued today uses UTCTime for both `notBefore` and `notAfter` — unless the certificate is valid until 2050 or later (some root CA certificates are).

For PDF signatures, the validity period has a nuanced interpretation. A signature made while the certificate was valid remains meaningful *after* the certificate expires — the certificate was valid *at the time of signing*. This is why timestamps are critical: a timestamp proves that the signature existed at a specific point in time, allowing validation even after certificate expiration. Without a timestamp, a validator must decide whether to check the certificate against the current time (strict) or the claimed signing time (lenient). Different validators make different choices, which is one of the many reasons the same signature shows ”valid” in one viewer and ”invalid” in another.

3.15.4 Certificate Extensions

Extensions are what make X.509v3 certificates useful for real-world applications. Without extensions, a certificate is just a name bound to a public key — no information about what the key can be used for, whether the certificate can issue other certificates, or how to check if it’s been revoked.

Each extension has three components:

- **OID**: identifies the extension type
- **Critical flag**: if true, a system that doesn’t understand this extension *must* reject the certificate
- **Value**: the extension-specific data (an OCTET STRING containing DER-encoded content)

3.15.4.1 KeyUsage

OID: `2.5.29.15` | Defined in: RFC 5280 — Internet X.509 PKI Certificate and CRL Profile §4.2.1.3 | Typically: **Critical**

The KeyUsage extension is a bit string that specifies what the public key in this certificate may be used for.

Table 3.9: KeyUsage bits relevant to PDF signing

Bit	Name	Relevance to PDF Signing
0	`digitalSignature`	**Required for PDF signing.** Indicates the key can be used to verify digital signatures (other than certificate and CRL signatures). This is the bit you need for signing documents.
1	`nonRepudiation` (contentCommitment)	Indicates the key is used for signatures where the signer cannot later deny having signed. Some certificate profiles for document signing set this bit instead of or in addition to `digitalSignature`. The RFC 5280 — Internet X.509 PKI Certificate and CRL Profile renamed this to `contentCommitment` but the old name persists in most tools.
5	`keyCertSign`	Indicates the key is used to verify signatures on certificates. Set on CA certificates. If this bit is set on an end-entity certificate, something is wrong.

Bit	Name	Relevance to PDF Signing
6	`cRLSign`	Indicates the key is used to verify signatures on CRLs. Set on CA certificates that issue CRLs.

Some document signing certificates have *only* `nonRepudiation` (bit 1) set, without `digitalSignature` (bit 0). This is technically valid per RFC 5280 — Internet X.509 PKI Certificate and CRL Profile, but it causes problems: some validators check for `digitalSignature` and reject certificates that only have `nonRepudiation`. The safest approach for CAs issuing document signing certificates is to set both bits. If you're validating signatures, check for *either* bit 0 or bit 1.

3.15.4.2 ExtendedKeyUsage

OID: `2.5.29.37` | Defined in: RFC 5280 — Internet X.509 PKI Certificate and CRL Profile §4.2.1.12 | Typically: **Non-critical**

While KeyUsage is a coarse-grained bitmask, ExtendedKeyUsage specifies more precise purposes using OIDs.

Table 3.10: ExtendedKeyUsage OIDs relevant to PDF signing

OID	Name	Meaning
`1.3.6.1.5.5.7.3.4`	`id-kp-emailProtection`	Historically used for document signing certificates (S/MIME). Many older PDF signing certificates have this EKU.
`1.3.6.1.4.1.311.10.3.12`	`id-kp-documentSigning` (Microsoft)	Microsoft's document signing EKU. Common on certificates from CAs that target Windows environments.
`1.2.840.113583.1.1.5`	Adobe Authentic Documents Trust	Adobe-specific EKU for AATL certificates. If a certificate has this EKU and chains to an AATL root, Acrobat shows maximum trust.
`1.3.6.1.5.5.7.3.9`	`id-kp-OCSPSigning`	For OCSP responder certificates only. Not for document signing.

When a PDF signature shows as "untrusted" in Adobe Acrobat but validates fine in other viewers, the first thing to check is the ExtendedKeyUsage. Acrobat is stricter about EKU than most other validators. A certificate without an explicit document signing EKU may be rejected by Acrobat even if it has `digitalSignature` in KeyUsage.

3.15.4.3 BasicConstraints

OID: `2.5.29.19` | Defined in: RFC 5280 — Internet X.509 PKI Certificate and CRL Profile §4.2.1.9 | Typically: **Critical**

```
BasicConstraints ::= SEQUENCE {
    cA                BOOLEAN DEFAULT FALSE,
    pathLenConstraint INTEGER (0..MAX) OPTIONAL
}
```

This extension distinguishes CA certificates from end-entity certificates:

- `cA: TRUE` — this certificate can issue other certificates
- `cA: FALSE` (or absent) — this is an end-entity certificate, it cannot issue other certificates
- `pathLenConstraint` — if present, limits how many intermediate CAs can follow this one in a chain

If you encounter a PDF signed by a certificate with `BasicConstraints: cA=TRUE`, something is deeply wrong. CA certificates should never be used for document signing — their private keys should be locked in HSMs and used only to sign other certificates. A document signed by a CA certificate suggests either a misconfiguration or a test setup that accidentally used the CA key for signing.

3.15.4.4 SubjectKeyIdentifier and AuthorityKeyIdentifier

SubjectKeyIdentifier (OID: `2.5.29.14`): a hash of the certificate's public key, used as a compact identifier. Typically the SHA-1 hash of the BIT STRING value of the subjectPublicKey (not the entire SubjectPublicKeyInfo structure — just the key bytes).

AuthorityKeyIdentifier (OID: `2.5.29.35`): identifies the CA certificate that signed this one. Contains the CA's SubjectKeyIdentifier (keyIdentifier), and optionally the CA's issuer name and serial number.

These two extensions are the primary mechanism for chain building: follow the AuthorityKeyIdentifier from a certificate to find the CA certificate whose SubjectKeyIdentifier matches.

The SubjectKeyIdentifier is typically a 20-byte SHA-1 hash. Yes, SHA-1 — even in 2026. This is not a security concern because the SKI is used as an identifier, not for integrity verification. There is no attack value in finding a collision for a SubjectKeyIdentifier.

3.15.4.5 CRLDistributionPoints

OID: `2.5.29.31` | Defined in: RFC 5280 — Internet X.509 PKI Certificate and CRL Profile §4.2.1.13

Contains one or more URLs where CRLs (Certificate Revocation Lists) can be downloaded to check if this certificate has been revoked.

```
CRLDistributionPoints ::= SEQUENCE SIZE (1..MAX) OF DistributionPoint

DistributionPoint ::= SEQUENCE {
    distributionPoint [0] DistributionPointName OPTIONAL,
    reasons           [1] ReasonFlags OPTIONAL,
    cRLIssuer         [2] GeneralNames OPTIONAL
}
```

In practice, this is almost always a single HTTP URL:

```
http://crl.example.com/issuing-ca.crl
```

3.15.4.6 AuthorityInfoAccess

OID: `1.3.6.1.5.5.7.1.1` | Defined in: RFC 5280 — Internet X.509 PKI Certificate and CRL Profile §4.2.2.1

This extension is critical for chain building and revocation checking. It contains two types of access descriptions:

Table 3.11: AuthorityInfoAccess methods

Method	OID	Purpose
OCSP	`1.3.6.1.5.5.7.48.1`	URL of the OCSP responder for checking certificate revocation in real time. The validator sends a request with the certificate's serial number and gets back a signed response saying "good," "revoked," or "unknown."
CA Issuers	`1.3.6.1.5.5.7.48.2`	URL where the issuing CA's certificate can be downloaded. This is how validators fetch missing intermediate certificates during chain building. If a PDF signature doesn't embed the full chain, the validator can use this URL to find the intermediates.

The "CA Issuers" access method is the safety net for incomplete certificate chains. When a signing tool embeds only the end-entity certificate in the CMS (not the full chain), the validator must fetch the intermediates. It uses the CA Issuers URL to download the next certificate up the chain, then follows *that* certificate's CA Issuers URL, and so on until it reaches a trusted root. This works — when you have network access. In offline validation scenarios, the chain must be complete. Always embed the full chain in your CMS signatures.

3.15.5 Certificate Profiles

Not all certificates are created equal. The purpose of a certificate is determined by its extensions, particularly KeyUsage and ExtendedKeyUsage.

3.15.5.1 End-Entity vs. CA Certificates

Table 3.12: End-entity vs. CA certificate comparison

Property	End-Entity Certificate	CA Certificate
BasicConstraints	`cA: FALSE` or absent	`cA: TRUE`, optionally with pathLenConstraint

Property	End-Entity Certificate	CA Certificate
KeyUsage	`digitalSignature`, `nonRepudiation`, `keyEncipherment`, `dataEncipherment` (varies)	`keyCertSign`, `cRLSign`
Purpose	Signs documents, authenticates TLS sessions, signs email	Signs other certificates and CRLs
Private key location	Software (PKCS#12), HSM, smart card	HSM (always, for production CAs)
Validity period	1-3 years typically	10-30 years for roots, 5-10 years for intermediates

3.15.5.2 Document Signing Certificates

A certificate profile specifically for document signing typically has:

- KeyUsage: `digitalSignature` and/or `nonRepudiation`
- ExtendedKeyUsage: document signing OID (varies by CA)
- Subject: identifies the signer (person or organization)
- Validity: 1-3 years
- CRL and OCSP: both present for revocation checking

3.15.5.3 How Different CAs Structure Their Certificates

Different CAs make different choices, and these choices affect interoperability:

Table 3.13: CA certificate profile comparison

Property	DigiCert	GlobalSign	National CAs (e.g., Austrian A-Trust)
Key algorithm	RSA 2048/4096 or ECC P-256/P-384	RSA 2048/4096 or ECC P-256/P-384	RSA 2048/4096 (ECC adoption varies)
Hash algorithm	SHA-256 or SHA-384	SHA-256 or SHA-384	SHA-256
Subject DN	CN = person/org name, O = organization	CN = person/org name, O = organization	CN = person name, serialNumber = national ID, C = country code
EKU	Adobe document signing OID + email protection	Adobe document signing OID	Varies widely — some use EU-specific OIDs for qualified certificates
AATL membership	Yes (since 2012)	Yes	Varies — many EU national CAs are on AATL

Property	DigiCert	GlobalSign	National CAs (e.g., Austrian A-Trust)
eIDAS qualified	Some certificate types	Some certificate types	Typically yes (this is their primary purpose)
OCSP	Yes, with OCSP stapling	Yes	Yes (required by eIDAS)
Certificate Transparency	Yes (required by Chrome/Apple)	Yes	Sometimes (not required for non-TLS certificates)

When debugging why a PDF signature is trusted in one viewer but not another, the CA's AATL status is usually the answer. Adobe Acrobat trusts AATL certificates. Other viewers (macOS Preview, Okular, Firefox's built-in PDF viewer) use the OS trust store, which is different. A DigiCert document signing certificate will be trusted in Acrobat (AATL member), probably trusted in Windows (DigiCert roots are in the Windows trust store), and possibly untrusted in Firefox (which uses its own trust store). Same certificate, same signature, three different trust decisions.

3.15.6 Common Certificate Problems in the Wild

Years of debugging PDF signatures have revealed a pattern of recurring certificate problems. Here are the ones you will encounter most often:

3.15.6.1 Missing Extensions

Symptom: Signature validates cryptographically but shows "untrusted" or "cannot determine if the signer is who they claim to be."

Cause: The signing certificate is missing critical extensions — typically ExtendedKeyUsage or SubjectKeyIdentifier. Some legacy CAs (particularly older European government CAs) issued certificates with minimal extensions that predate the current RFC 5280 — Internet X.509 PKI Certificate and CRL Profile profile requirements.

Fix: The certificate needs to be reissued with proper extensions. There is no workaround — you cannot add extensions to an existing certificate without re-signing it (which means the CA must do it).

3.15.6.2 Wrong Key Usage

Symptom: Signature fails validation with "key usage does not permit digital signature" or similar.

Cause: The certificate has KeyUsage set to `keyEncipherment` or `dataEncipherment` but not `digitalSignature` or `nonRepudiation`. This happens when someone uses a TLS server certificate (designed for key exchange) to sign a PDF document.

Fix: Use a certificate with the correct KeyUsage for document signing. A TLS certificate is not a signing certificate.

3.15.6.3 Expired Certificates

Symptom: ”Certificate has expired” warning, even though the document was signed years ago when the certificate was valid.

Cause: The validator is checking the certificate against the *current* time instead of the *signing* time. Without a trusted timestamp, the validator has no proof of when the signature was actually created.

Fix: Include a timestamp in the signature (see Chapter 18 on LTV). A timestamp from a trusted TSA proves the signature existed at a specific time, allowing validators to check the certificate’s validity against that time rather than the current time.

3.15.6.4 Self-Signed Certificates

Symptom: ”The signer’s identity is unknown” or ”certificate is not issued by a trusted CA.”

Cause: The signing certificate is self-signed (the issuer and subject are the same, and it signed itself). Self-signed certificates have no external trust anchor.

Fix: For production use, obtain a certificate from a recognized CA. For internal/testing use, you can add the self-signed certificate to the viewer’s trust store manually — but this defeats the purpose of the CA trust model.

Self-signed certificates are perfectly valid for testing and development. PDFalyzer’s self-signed certificate generation feature is designed exactly for this purpose. The cryptographic integrity of the signature is identical regardless of whether the certificate is self-signed or CA-issued — the difference is purely about trust.

3.15.6.5 Incomplete Certificate Chain

Symptom: ”Unable to build certificate chain” or ”issuer certificate not found.”

Cause: The CMS signature embeds only the end-entity certificate, not the intermediate CA certificates. The validator cannot build a path from the end-entity to a trusted root.

Fix: Include the full certificate chain (end-entity + all intermediates) in the CMS `certificates` field. Do NOT include the root CA — the root must already be in the validator’s trust store.

This is one of the most common PDF signature problems. Always embed the full chain (minus the root) in your CMS signatures. The signing code in Chapter 2 already does this — note how it includes `chain` (from the PKCS#12 key store) in the `CMSSignedDataGenerator`. If your PKCS#12 file doesn’t contain the intermediates, you need to add them before signing.

3.15.7 Java: Loading and Inspecting Certificates

Here is how to load certificates and extract their fields using BouncyCastle 1.80.

```
import java.io.FileInputStream;
import java.security.cert.CertificateFactory;
import java.security.cert.X509Certificate;
import java.util.Date;
import java.util.List;

import org.bouncycastle.asn1.ASN1ObjectIdentifier;
```

```
import org.bouncycastle.asn1.x509.AuthorityInformationAccess;
import org.bouncycastle.asn1.x509.BasicConstraints;
import org.bouncycastle.asn1.x509.CRLDistPoint;
import org.bouncycastle.asn1.x509.DistributionPoint;
import org.bouncycastle.asn1.x509.Extension;
import org.bouncycastle.asn1.x509.GeneralName;
import org.bouncycastle.asn1.x509.KeyUsage;
import org.bouncycastle.cert.X509CertificateHolder;
import org.bouncycastle.cert.jcajce.JcaX509CertificateHolder;

public class CertificateInspector {

    /**
     * Load a certificate from a DER or PEM file and inspect its fields.
     */
    public static void inspectCertificate(String filePath) throws Exception {
        // Load the certificate using JCA
        CertificateFactory cf = CertificateFactory.getInstance("X.509");
        X509Certificate javaCert;
        try (FileInputStream fis = new FileInputStream(filePath)) {
            javaCert = (X509Certificate) cf.generateCertificate(fis);
        }

        // Wrap in BouncyCastle holder for extension access
        X509CertificateHolder holder = new JcaX509CertificateHolder(javaCert);

        // --- Basic fields ---
        String subject = javaCert.getSubjectX500Principal().getName();
        String issuer = javaCert.getIssuerX500Principal().getName();
        String serial = javaCert.getSerialNumber().toString(16);
        int version = javaCert.getVersion();
        Date notBefore = javaCert.getNotBefore();
        Date notAfter = javaCert.getNotAfter();
        String sigAlg = javaCert.getSigAlgName();
        String pubKeyAlg = javaCert.getPublicKey().getAlgorithm();
        int keySize = getKeySize(javaCert);

        // --- KeyUsage ---
        boolean[] keyUsageBits = javaCert.getKeyUsage();
        if (keyUsageBits != null) {
            boolean digitalSignature = keyUsageBits[0];
            boolean nonRepudiation = keyUsageBits[1];
            boolean keyEncipherment = keyUsageBits[2];
            boolean keyCertSign = keyUsageBits[5];
        }

        // --- BasicConstraints ---
        Extension bcExt = holder.getExtension(Extension.basicConstraints);
        if (bcExt != null) {
            BasicConstraints bc = BasicConstraints.getInstance(
                    bcExt.getParsedValue());
            boolean isCA = bc.isCA();
            int pathLen = bc.getPathLenConstraint() != null
                    ? bc.getPathLenConstraint().intValue() : -1;
        }
```

```
        // --- ExtendedKeyUsage ---
        List<String> ekuOids = javaCert.getExtendedKeyUsage();
        if (ekuOids != null) {
            for (String oid : ekuOids) {
                // Map OID to human-readable name
                String name = mapEkuOid(oid);
            }
        }

        // --- AuthorityInfoAccess (OCSP + CA Issuers) ---
        Extension aiaExt = holder.getExtension(Extension.authorityInfoAccess);
        if (aiaExt != null) {
            AuthorityInformationAccess aia = AuthorityInformationAccess
                    .getInstance(aiaExt.getParsedValue());

            // OCSP responder URLs
            var ocspAccessDescriptions = aia.getAccessDescriptions();
            for (var ad : ocspAccessDescriptions) {
                if (ad.getAccessMethod().equals(
                        new ASN1ObjectIdentifier("1.3.6.1.5.5.7.48.1"))) {
                    String ocspUrl = ad.getAccessLocation()
                            .getName().toString();
                }
            }
        }

        // --- CRL Distribution Points ---
        Extension crlExt = holder.getExtension(Extension.cRLDistributionPoints);
        if (crlExt != null) {
            CRLDistPoint crlDistPoint = CRLDistPoint.getInstance(
                    crlExt.getParsedValue());
            for (DistributionPoint dp : crlDistPoint.getDistributionPoints()) {
                // Extract URL from the distribution point
                var dpName = dp.getDistributionPoint();
                if (dpName != null) {
                    var names = org.bouncycastle.asn1.x509.GeneralNames
                            .getInstance(dpName.getName());
                    for (GeneralName name : names.getNames()) {
                        if (name.getTagNo() == GeneralName.uniformResourceIdentifier) {
                            String crlUrl = name.getName().toString();
                        }
                    }
                }
            }
        }
    }

    private static int getKeySize(X509Certificate cert) {
        String algorithm = cert.getPublicKey().getAlgorithm();
        if ("RSA".equals(algorithm)) {
            var rsaKey = (java.security.interfaces.RSAPublicKey) cert.getPublicKey();
            return rsaKey.getModulus().bitLength();
        } else if ("EC".equals(algorithm)) {
            var ecKey = (java.security.interfaces.ECPublicKey) cert.getPublicKey();
```

```
            return ecKey.getParams().getOrder().bitLength();
        }
        return -1;
    }

    private static String mapEkuOid(String oid) {
        return switch (oid) {
            case "1.3.6.1.5.5.7.3.4" -> "emailProtection";
            case "1.3.6.1.4.1.311.10.3.12" -> "documentSigning (Microsoft)";
            case "1.2.840.113583.1.1.5" -> "adobeAuthenticDocumentsTrust";
            case "1.3.6.1.5.5.7.3.1" -> "serverAuth (TLS)";
            case "1.3.6.1.5.5.7.3.9" -> "OCSPSigning";
            default -> "unknown (" + oid + ")";
        };
    }
}
```

- Returns the subject DN in RFC 2253 format (CN first). For display, you might want to parse this into individual components.
- Serial numbers are typically displayed in hexadecimal. Some CAs use very large random serial numbers (up to 20 bytes per RFC 5280 — Internet X.509 PKI Certificate and CRL Profile).
- `getKeyUsage()` returns a boolean array where index 0 = digitalSignature, 1 = nonRepudiation, ..., 5 = keyCertSign, 6 = cRLSign. Returns null if the extension is absent.
- Java's `getExtendedKeyUsage()` returns OIDs as strings. You need to map them to human-readable names yourself.
- The OCSP URL is what you send revocation check requests to. This URL must be accessible from the validator's network.

3.15.8 Dissecting a Real Signing Certificate

Let's walk through a real document signing certificate, field by field, to see how the theory maps to practice.

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            0a:55:14:f1:28:1d:45:87:21:f1:27:3c:e5:b6:4f:2a
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=US, O=DigiCert Inc, CN=DigiCert SHA2 Assured ID CA
        Validity
            Not Before: Jan 15 00:00:00 2024 GMT
            Not After : Jan 14 23:59:59 2027 GMT
        Subject: C=US, ST=California, L=San Francisco,
                 O=Acme Corporation, CN=Acme Document Signing
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                RSA Public-Key: (2048 bit)
```

Let's annotate each field:

Table 3.14: Certificate field analysis

Field	Analysis
Version 3	Expected. All modern certificates are v3 to support extensions.
Serial Number	16 bytes (128 bits) of random data. DigiCert uses large random serials to prevent hash-based attacks on the serial number.
Signature Algorithm	`sha256WithRSAEncryption` — SHA-256 hash with RSA signature. This is the algorithm the CA (DigiCert) used to sign this certificate.
Issuer	`DigiCert SHA2 Assured ID CA` — this is an intermediate CA certificate. The "SHA2" in the name indicates it's a post-SHA-1 intermediate. You'll need to find *this* certificate to build the chain.
Validity	3-year validity (Jan 2024 to Jan 2027). Typical for document signing certificates. The signer needs to renew before Jan 14, 2027, or new signatures will be created with an expired certificate.
Subject	Identifies the organization (Acme Corporation) and the purpose (Document Signing). The CN often contains the signer's name or the certificate's purpose rather than a hostname.
Public Key	RSA 2048-bit. The minimum for document signing. Some organizations require 4096-bit, but 2048 is widely accepted and faster.

The extensions would look like:

```
X509v3 extensions:
    X509v3 Key Usage: critical
        Digital Signature
    X509v3 Extended Key Usage:
        1.2.840.113583.1.1.5
    X509v3 Basic Constraints: critical
        CA:FALSE
    X509v3 Subject Key Identifier:
        A1:B2:C3:D4:E5:F6:...
    X509v3 Authority Key Identifier:
        keyid:F8:7E:34:22:...
    Authority Information Access:
        OCSP - URI:http://ocsp.digicert.com
        CA Issuers - URI:http://cacerts.digicert.com/DigiCertSHA2AssuredIDCA.crt
    X509v3 CRL Distribution Points:
        Full Name:
          URI:http://crl3.digicert.com/sha2-assured-id-ca.crl
          URI:http://crl4.digicert.com/sha2-assured-id-ca.crl
```

Table 3.15: Extension analysis

Extension	Analysis
KeyUsage: digitalSignature	Correct for document signing. This certificate can create digital signatures. The `critical` flag means validators must enforce this.
ExtendedKeyUsage: 1.2.840.113583.1.1.5	This is the Adobe Authentic Documents Trust OID. It means this certificate is intended for PDF document signing and is likely on the AATL.
BasicConstraints: CA:FALSE	Correct. This is an end-entity certificate. It cannot issue other certificates.
SubjectKeyIdentifier	A hash of the public key. Used for chain building (other certificates reference this to identify this certificate as their issuer).
AuthorityKeyIdentifier	Points to the issuing CA's SubjectKeyIdentifier. Follow this to find `DigiCert SHA2 Assured ID CA`.
AIA: OCSP	The OCSP responder URL. Send a request here to check if this certificate has been revoked.
AIA: CA Issuers	The URL to download the issuing CA's certificate. If the CMS signature doesn't include the intermediate, a validator can fetch it here.
CRL Distribution Points	Two CRL URLs (redundancy). Download the CRL to check for revocation without OCSP.

DigiCert provides two CRL URLs (crl3 and crl4) for redundancy. If one CDN is down, the other should still work. This is a best practice — if your CA provides only one CRL URL and it becomes unreachable, validators that use CRL checking will fail to validate signatures. OCSP is the preferred revocation checking method (real-time, smaller responses), but CRL is the fallback.

3.15.9 Summary

X.509 certificates are the bridge between raw cryptography and trust. They bind a public key to an identity, and the CA's signature on that binding is what makes the trust model work — or fail, when CAs are compromised (DigiNotar), when extensions are missing, or when the chain is incomplete.

For PDF signatures:

- **Every signed PDF embeds one or more X.509 certificates** in the CMS SignedData structure.
- **The signing certificate must have `digitalSignature` or `nonRepudiation`** in KeyUsage.
- **The full chain** (end-entity + intermediates, not the root) should be embedded in the CMS.
- **AIA extensions** provide fallback URLs for chain building and OCSP revocation checking.

- **Trust is viewer-dependent**: Acrobat uses AATL, browsers use their own trust stores, OS-level trust stores vary. The same certificate can be trusted in one context and untrusted in another.

The next chapter follows these certificates up the chain — from the end-entity signing certificate, through intermediates, to the root CA that anchors the entire trust hierarchy.

Chapter 4

Certificate Chains & Trust

4.1 Why Chains Exist

In Chapter 3 we established that an X.509 certificate binds a public key to an identity. A certificate authority (CA) vouches for that binding by signing the certificate with its own private key. But who vouches for the CA?

This is the trust delegation problem. A root CA cannot practically issue certificates to every person and organization on the planet. The operational burden would be staggering: vetting applicants, maintaining infrastructure, handling revocations, passing audits — all for millions of subscribers. Even if it could, concentrating all certificate issuance in a single entity would create a catastrophic single point of failure. If that one root CA were compromised, every certificate it had ever issued would become suspect.

The solution is **hierarchical delegation**. A root CA issues certificates not (primarily) to end users, but to **intermediate CAs**. Those intermediate CAs, in turn, issue certificates to other intermediates or to end-entity subscribers. The root CA's private key stays in a vault — literally, in most cases, inside an HSM in a physically secured facility that is powered on only for scheduled signing ceremonies.

This creates a chain:

1. The **root CA** self-signs its own certificate. It is the trust anchor — the starting point that must be accepted on faith (or, more precisely, accepted because it appears in a trust store you have chosen to rely on).
2. One or more **intermediate CAs** hold certificates signed by the root CA (or by another intermediate above them). They perform the day-to-day work of issuing certificates.
3. The **end-entity** (sometimes called the "leaf") certificate is the one that belongs to the actual signer — the person or service that signs PDFs. This certificate is signed by an intermediate CA.

The term "chain" is somewhat misleading. In practice, a certificate's trust path can be a **directed acyclic graph** rather than a simple linear chain. An intermediate CA might hold multiple certificates — one signed by Root A, another signed by Root B — enabling cross-certification between different PKI hierarchies. However, for PDF signatures, you almost always deal with simple linear chains of depth two or three: root intermediate end-entity, or root intermediate intermediate end-entity. The rare cases where cross-certification matters are in government PKIs and the EU Trusted List hierarchy, which we cover later in this chapter.

4.2 Chain Structure in Detail

A typical PDF signing chain has three certificates:

Table 4.1: Certificate chain for a PDF signing certificate

Position	Role	Characteristics
1 (leaf)	End-entity (signer)	Contains the signer's public key. Has `keyUsage` with `digitalSignature` or `nonRepudiation` set. Has `extendedKeyUsage` with document signing or email protection OIDs. Issued by the intermediate CA. Short validity period (1-3 years).
2	Intermediate CA	Contains the intermediate CA's public key. Has `basicConstraints` with `cA=TRUE` and a `pathLenConstraint`. Has `keyUsage` with `keyCertSign` and `cRLSign`. Issued by the root CA. Medium validity period (5-10 years).
3 (anchor)	Root CA	Self-signed: the issuer and subject are identical. Has `basicConstraints` with `cA=TRUE`. Present in trust stores. Long validity period (20-30 years). Never transmitted in the CMS structure of a PDF signature (validators must already have it in their trust store).

The CMS `SignedData` structure in a PDF signature's `/Contents` field typically includes certificates at positions 1 and 2 — the end-entity and intermediate(s). The root CA certificate is **not** included. This is by design: the validator must already trust the root, and including it in the signature would just waste space. The validator checks the chain by building a path from the end-entity up through the intermediates until it reaches a certificate that matches one in its trust store.

Some signing implementations include the root certificate in the CMS `certificates` field. This is not wrong per se — RFC 5652 — Cryptographic Message Syntax (CMS) allows it — but it serves no purpose. A validator will not trust a root certificate just because it appears inside a signature. Trust comes from the trust store, not from the document. If anything, including the root wastes space in the `/Contents` hex string, which matters when you are estimating the size of the content gap (Chapter 13). PDFBox does not include the root by default. iText does. Adobe Acrobat tolerates both approaches.

A chain with a **missing intermediate** is the single most common cause of "certificate chain not trusted" errors in PDF signatures. The end-entity certificate is present, the root is in the trust store, but the intermediate that links them is nowhere to be found. The CMS structure does not contain it, and the validator cannot download it (or is configured not to). Always include the full chain (minus the root) in your CMS `certificates` field.

4.3 Building the Chain

Given a bag of certificates — perhaps from a CMS `SignedData` structure, perhaps from a PKCS#12 file, perhaps assembled from multiple sources — how do you construct the actual chain?

The algorithm relies on two X.509 extensions that exist specifically for this purpose:

4.3.1 AuthorityKeyIdentifier and SubjectKeyIdentifier

Every well-formed certificate contains:

- **SubjectKeyIdentifier (SKI)**: an extension on the certificate itself, identifying the public key it contains. Typically a SHA-1 hash of the public key's bit string. Defined in RFC 5280 — Internet X.509 PKI Certificate and CRL Profile section 4.2.1.2.
- **AuthorityKeyIdentifier (AKI)**: an extension identifying the certificate that signed this one. Contains the SKI of the issuer's certificate. Defined in RFC 5280 — Internet X.509 PKI Certificate and CRL Profile section 4.2.1.1.

The chain-building algorithm is conceptually simple:

1. Start with the end-entity certificate (the signer's cert from the CMS `SignerInfo`).
2. Read its AKI extension. This gives you the SKI of the issuer.
3. Search the available certificates for one whose SKI matches that AKI.
4. When you find it, that is the next link in the chain. Read *its* AKI.
5. Repeat until you find a certificate where the subject equals the issuer (self-signed) or one that matches a trust anchor.

```
import java.security.cert.X509Certificate;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.List;

import org.bouncycastle.asn1.x509.AuthorityKeyIdentifier;
import org.bouncycastle.asn1.x509.Extension;
import org.bouncycastle.asn1.x509.SubjectKeyIdentifier;
import org.bouncycastle.cert.X509CertificateHolder;
import org.bouncycastle.cert.jcajce.JcaX509CertificateHolder;

/**
 * Build a certificate chain from an unordered collection of certificates.
 * Matches AuthorityKeyIdentifier -> SubjectKeyIdentifier.
 */
public class ChainBuilder {

    public static List<X509Certificate> buildChain(
            X509Certificate endEntity,
            Collection<X509Certificate> candidates) throws Exception {

        List<X509Certificate> chain = new ArrayList<>();
        chain.add(endEntity);
```

```
        X509Certificate current = endEntity;
        int maxDepth = 10; // prevent infinite loops from malformed chains

        for (int i = 0; i < maxDepth; i++) {
            // Self-signed? We've reached a root.
            if (current.getSubjectX500Principal()
                    .equals(current.getIssuerX500Principal())) {
                break;
            }

            byte[] akiBytes = getAuthorityKeyIdentifier(current);
            if (akiBytes == null) {
                break; // no AKI -- cannot chain further
            }

            X509Certificate issuer = findBySki(candidates, akiBytes);
            if (issuer == null) {
                break; // issuer not in our candidate set
            }

            chain.add(issuer);
            current = issuer;
        }

        return chain;
    }

    private static byte[] getAuthorityKeyIdentifier(
            X509Certificate cert) throws Exception {

        X509CertificateHolder holder = new JcaX509CertificateHolder(cert);
        Extension akiExt = holder.getExtension(
                Extension.authorityKeyIdentifier);
        if (akiExt == null) {
            return null;
        }
        AuthorityKeyIdentifier aki =
                AuthorityKeyIdentifier.getInstance(akiExt.getParsedValue());
        return aki.getKeyIdentifier();
    }

    private static X509Certificate findBySki(
            Collection<X509Certificate> certs,
            byte[] targetSki) throws Exception {

        for (X509Certificate candidate : certs) {
            X509CertificateHolder holder =
                    new JcaX509CertificateHolder(candidate);
            Extension skiExt = holder.getExtension(
                    Extension.subjectKeyIdentifier);
            if (skiExt == null) {
                continue;
            }
            SubjectKeyIdentifier ski =
                    SubjectKeyIdentifier.getInstance(skiExt.getParsedValue());
```

```
            if (Arrays.equals(ski.getKeyIdentifier(), targetSki)) {
                return candidate;
            }
        }
        return null;
    }
}
```

In practice, chain building can fail for several reasons. The AKI extension might be absent (legal but rare in modern certificates). The SKI might not match because the issuing CA generated the SKI using a non-standard method (SHA-1 of the public key is the norm, but RFC 5280 — Internet X.509 PKI Certificate and CRL Profile does not mandate it). Or the intermediate certificate might simply not be available. When debugging chain failures, always dump both the AKI of the child and the SKI of the expected parent, and compare them byte by byte.

4.3.2 Fallback: Issuer Distinguished Name Matching

If the AKI/SKI matching fails — or if the certificates lack these extensions (which happens with very old CAs) — the fallback is matching the certificate's **Issuer DN** against candidate certificates' **Subject DN**. This is less reliable because distinguished name comparison is notoriously tricky: leading/trailing whitespace, different string encodings (PrintableString vs UTF8String), case sensitivity of certain components — all of these can cause a mismatch even when the certificates are part of the same chain.

RFC 5280 — Internet X.509 PKI Certificate and CRL Profile section 7.1 defines the comparison rules for distinguished names, and they are considerably more complex than string equality. BouncyCastle's `X500Name.equals()` handles this correctly; Java's `X500Principal.equals()` usually does too, but edge cases exist with certain encoding combinations.

4.4 Path Validation: RFC 5280 Section 6

Building the chain gives you an ordered list of certificates. **Path validation** verifies that this chain is actually valid — that every link is cryptographically sound and that the certificates meet all the required constraints.

RFC 5280 — Internet X.509 PKI Certificate and CRL Profile section 6 defines the algorithm in six phases. The full specification runs to several pages of pseudocode; here we distill the essential checks that matter for PDF signature validation.

4.4.1 The Six Steps

Table 4.2: Certificate path validation (RFC 5280 — Internet X.509 PKI Certificate and CRL Profile section 6)

Step	Name	What It Checks
1	Initialization	Set the initial trust anchor (root CA certificate), the expected issuer name, the expected public key, and policy constraints. These are the ”inputs” to the validation algorithm.
2	Basic processing	For each certificate in the chain (from root to leaf): verify the signature on the certificate using the issuer’s public key, check that the certificate is within its validity period (notBefore current time notAfter), check that the certificate has not been revoked.
3	Name constraints	Check that the subject name and any subject alternative names comply with any name constraints imposed by certificates higher in the chain. A CA can restrict its subordinates to only issue certificates for certain domains or email addresses.
4	Policy processing	Process the certificate policies extension and any policy constraints. This determines which policies are valid for the entire chain. Policy processing is the most complex part of path validation and is rarely used in PDF signature contexts — but some government PKIs rely on it.
5	Basic constraints	Verify that every CA certificate in the chain has `basicConstraints` with `cA=TRUE`, and that the `pathLenConstraint` is not exceeded. A `pathLenConstraint` of 0 means the CA can only issue end-entity certificates, not sub-CAs.
6	Wrap-up	After processing all certificates, check the final certificate (the end-entity) against the expected usage. For PDF signing, this means checking `keyUsage` for `digitalSignature` or `nonRepudiation`, and optionally checking `extendedKeyUsage`.

```
import java.security.cert.CertPathValidator;
import java.security.cert.CertificateFactory;
import java.security.cert.CertPath;
import java.security.cert.PKIXParameters;
import java.security.cert.PKIXCertPathValidatorResult;
import java.security.cert.TrustAnchor;
import java.security.cert.X509Certificate;
import java.util.Collections;
import java.util.List;

/**
 * Validate a certificate chain using the JCA PKIX path validator.
```

```
 * This performs the full RFC 5280 section 6 algorithm.
 */
public class PathValidationExample {

    public static PKIXCertPathValidatorResult validateChain(
            List<X509Certificate> chain,
            X509Certificate trustAnchor) throws Exception {

        // The chain list should NOT include the trust anchor
        CertificateFactory cf = CertificateFactory.getInstance("X.509");
        CertPath certPath = cf.generateCertPath(chain);

        TrustAnchor anchor = new TrustAnchor(trustAnchor, null);
        PKIXParameters params = new PKIXParameters(
                Collections.singleton(anchor));

        // Disable revocation checking here -- we handle it separately
        params.setRevocationEnabled(false);

        CertPathValidator validator =
                CertPathValidator.getInstance("PKIX");

        return (PKIXCertPathValidatorResult)
                validator.validate(certPath, params);
    }
}
```

The JCA `CertPathValidator` expects the chain in a specific order: **end-entity first, root last (or root omitted)**. If you pass the chain in the wrong order, validation will fail with a confusing error message about ”unable to find valid certification path to requested target.” This is the single most common mistake when using the JCA path validation API.

4.4.2 Validity Period Checking and Signing Time

There is a subtlety in validity period checking that matters enormously for PDF signatures. A certificate might be expired *now* but was valid at the time the document was signed. Should the signature be considered valid?

The answer depends on whether the signature includes a trusted timestamp. If the CMS `SignedData` contains a timestamp token in the unsigned attributes (see Chapter 18), and that timestamp was produced while the certificate was still valid, then the signature remains valid even after the certificate expires. This is the foundation of **long-term validation (LTV)** — the mechanism that allows a signed PDF to remain verifiable for decades.

Without a timestamp, most validators check the certificate against the current time. A signature made with a certificate that has since expired will show warnings or errors in Adobe Acrobat, even if the signature was perfectly valid when it was created.

This is why timestamps matter so much for PDF signatures. A timestamp cryptographically proves *when* the signature was created, allowing validators to check the certificate against the signing time rather than the current time. If you are building a signing service, always include a timestamp. Chapter 18 covers timestamps and LTV in exhaustive detail.

4.5 Revocation Checking

A certificate may be revoked before it expires. The private key might have been compromised, the subscriber might have left the organization, or the CA might have discovered that the certificate was issued incorrectly. Revocation checking is the mechanism by which validators discover whether a certificate has been revoked.

There are two mechanisms: **Certificate Revocation Lists (CRLs)** and the **Online Certificate Status Protocol (OCSP)**. Both have strengths and weaknesses, and a robust validator should support both.

4.5.1 CRL Deep Dive

A Certificate Revocation List is a signed data structure published by a CA that lists all certificates the CA has revoked. CRLs are defined in RFC 5280 — Internet X.509 PKI Certificate and CRL Profile section 5.

4.5.1.1 CRL Structure

A CRL contains:

Table 4.3: CRL structure (simplified from RFC 5280 — Internet X.509 PKI Certificate and CRL Profile section 5.1)

Field	Description
`tbsCertList`	The ”to-be-signed” portion, containing the issuer name, the `thisUpdate` timestamp (when the CRL was issued), the `nextUpdate` timestamp (when the next CRL will be issued), and the list of revoked certificates.
`signatureAlgorithm`	The algorithm used to sign the CRL (e.g., SHA-256 with RSA).
`signatureValue`	The CA’s signature over the `tbsCertList`, proving the CRL is authentic and unmodified.

Each entry in the `revokedCertificates` list contains:

- The **serial number** of the revoked certificate
- The **revocation date**
- Optional **CRL entry extensions**, including the `reasonCode` (keyCompromise, cACompromise, affiliationChanged, superseded, cessationOfOperation, certificateHold, removeFromCRL, privilegeWithdrawn, aACompromise)

4.5.1.2 CRL Distribution Points

How does a validator find the CRL? The certificate itself contains a **CRL Distribution Points** extension (RFC 5280 — Internet X.509 PKI Certificate and CRL Profile section 4.2.1.13) with one or more URLs where the CRL can be downloaded. Typically these are HTTP URLs, though LDAP URLs appear in older enterprise PKIs.

```
CRLDistributionPoints ::= SEQUENCE SIZE (1..MAX) OF DistributionPoint

DistributionPoint ::= SEQUENCE {
    distributionPoint  [0] DistributionPointName OPTIONAL,
    reasons            [1] ReasonFlags OPTIONAL,
    cRLIssuer          [2] GeneralNames OPTIONAL
}
```

4.5.1.3 The Size Problem

CRLs have a fundamental scalability problem: they grow monotonically. Every revoked certificate is listed until the certificate's natural expiration date passes. A busy CA that revokes hundreds or thousands of certificates per year will produce CRLs that are megabytes in size.

Let's Encrypt, for example, issues millions of certificates. A full CRL from Let's Encrypt would be enormous. This is why Let's Encrypt does not publish CRLs for individual certificates (they rely on OCSP and, more recently, CRL sharding).

For PDF signatures, the CRL size matters because long-term validation (LTV) requires embedding revocation data *inside the PDF*. A 2 MB CRL embedded in the Document Security Store (DSS) inflates the PDF by 2 MB. If the chain has three certificates and each CRL is large, the overhead becomes significant.

4.5.1.4 Delta CRLs

Delta CRLs (RFC 5280 — Internet X.509 PKI Certificate and CRL Profile section 5.2.4) were designed to mitigate the size problem. A delta CRL contains only the changes since the last "base" CRL was published. In theory, this allows validators to download a small delta rather than a large full CRL.

In practice, delta CRLs are rarely used. Most CAs do not publish them. Most validators do not request them. The overhead of tracking base/delta CRL relationships adds complexity that the industry has largely chosen to avoid, preferring OCSP for real-time status and full CRLs for archival purposes.

4.5.2 OCSP Deep Dive

The **Online Certificate Status Protocol** (RFC 6960 — Online Certificate Status Protocol (OCSP)) provides real-time, per-certificate revocation status. Instead of downloading a list of all revoked certificates, the validator sends a request asking "is this specific certificate revoked?" and gets back a signed response.

4.5.2.1 OCSP Request Structure

An OCSP request identifies the certificate by:

- The hash of the issuer's distinguished name
- The hash of the issuer's public key
- The certificate's serial number
- An optional nonce to prevent replay attacks

```
OCSPRequest ::= SEQUENCE {
    tbsRequest     TBSRequest,
    optionalSignature [0] EXPLICIT Signature OPTIONAL
}

TBSRequest ::= SEQUENCE {
    version        [0] EXPLICIT Version DEFAULT v1,
    requestorName  [1] EXPLICIT GeneralName OPTIONAL,
    requestList    SEQUENCE OF Request,
    requestExtensions [2] EXPLICIT Extensions OPTIONAL
}

Request ::= SEQUENCE {
    reqCert        CertID,
    singleRequestExtensions [0] EXPLICIT Extensions OPTIONAL
}

CertID ::= SEQUENCE {
    hashAlgorithm  AlgorithmIdentifier,
    issuerNameHash OCTET STRING,
    issuerKeyHash  OCTET STRING,
    serialNumber   CertificateSerialNumber
}
```

4.5.2.2 OCSP Response Structure

The responder returns one of: **good** (not revoked), **revoked** (with the revocation time and optional reason), or **unknown** (the responder does not know about this certificate).

The response is signed, either by the CA itself or by a specially designated **OCSP responder certificate** — a certificate issued by the CA with the `id-pkix-ocsp-nocheck` extension, indicating it is authorized to sign OCSP responses on behalf of the CA.

4.5.2.3 The Nonce Problem

OCSP supports a nonce extension: the requester includes a random value, and the responder echoes it back. This prevents an attacker from replaying an old ”good” response for a certificate that has since been revoked.

In practice, many OCSP responders **ignore the nonce**. They pre-compute responses for all active certificates and serve them from cache, which is incompatible with per-request nonces. Adobe’s OCSP responder, for example, does not support nonces. The security trade-off is real: without nonces, a replay window exists equal to the response’s `nextUpdate` time. For most PDF signature scenarios, this is acceptable — the replay window is typically hours, not days.

When embedding OCSP responses in a PDF for LTV, nonce-less responses are actually preferable. A response with a nonce is bound to a specific request and cannot be reused. A cached, nonce-less response can be embedded in the PDF and validated later by anyone, which is exactly what LTV requires. This is a case where the ”less secure” option (no nonce) is the correct choice for the specific use case.

4.5.2.4 OCSP Stapling

OCSP stapling is a mechanism where the entity presenting a certificate also presents a recent OCSP response for that certificate, pre-fetched from the OCSP responder. In TLS, the server staples the OCSP response to the TLS handshake.

For PDF signatures, the equivalent is embedding the OCSP response in the CMS `SignedData` structure or in the PDF's Document Security Store (DSS). This eliminates the need for the validator to contact the OCSP responder at all — the revocation proof travels with the document, just like the certificate chain.

4.5.3 CRL vs OCSP: When to Use Which

Table 4.4: CRL vs OCSP comparison for PDF signatures

Aspect	CRL	OCSP
Granularity	All revocations for a CA	Single certificate status
Freshness	Updated periodically (hours to days)	Real-time or near-real-time (minutes)
Size	Can be megabytes (proportional to revocations)	Typically 1-3 KB per response
Availability	Can be cached locally; works offline once downloaded	Requires network access (unless stapled/embedded)
LTV suitability	Good: can be embedded in DSS	Good: can be embedded in DSS (prefer nonce-less)
Privacy	Good: the CA does not learn which certificates you check	Poor: the OCSP responder learns exactly which certificates you are validating (which documents you are opening)

For PDF signing services, use OCSP for real-time validation during signing, and embed both the OCSP response and relevant CRLs in the DSS for long-term validation. Belt and suspenders. Adobe Acrobat's LTV process follows exactly this approach.

```
import java.io.InputStream;
import java.io.OutputStream;
import java.math.BigInteger;
import java.net.HttpURLConnection;
import java.net.URI;
import java.security.cert.X509Certificate;

import org.bouncycastle.asn1.DEROctetString;
import org.bouncycastle.asn1.ocsp.OCSPObjectIdentifiers;
import org.bouncycastle.asn1.x509.Extension;
import org.bouncycastle.asn1.x509.Extensions;
import org.bouncycastle.cert.jcajce.JcaX509CertificateHolder;
import org.bouncycastle.cert.ocsp.BasicOCSPResp;
import org.bouncycastle.cert.ocsp.CertificateID;
```

```
import org.bouncycastle.cert.ocsp.OCSPReq;
import org.bouncycastle.cert.ocsp.OCSPReqBuilder;
import org.bouncycastle.cert.ocsp.OCSPResp;
import org.bouncycastle.cert.ocsp.SingleResp;
import org.bouncycastle.operator.jcajce.JcaDigestCalculatorProviderBuilder;

/**
 * Send an OCSP request and check the response.
 * Demonstrates the full OCSP round-trip with BouncyCastle.
 */
public class OcspCheckExample {

    public static SingleResp checkOcsp(
            X509Certificate cert,
            X509Certificate issuer,
            String ocspUrl) throws Exception {

        // Build the CertificateID that identifies the cert to check
        CertificateID certId = new CertificateID(
                new JcaDigestCalculatorProviderBuilder()
                        .build()
                        .get(CertificateID.HASH_SHA1),
                new JcaX509CertificateHolder(issuer),
                cert.getSerialNumber());

        // Build the OCSP request
        OCSPReqBuilder reqBuilder = new OCSPReqBuilder();
        reqBuilder.addRequest(certId);

        // Add a nonce (optional -- many responders ignore it)
        BigInteger nonce = BigInteger.valueOf(System.currentTimeMillis());
        Extension nonceExt = new Extension(
                OCSPObjectIdentifiers.id_pkix_ocsp_nonce,
                false,
                new DEROctetString(nonce.toByteArray()));
        reqBuilder.setRequestExtensions(
                new Extensions(new Extension[]{ nonceExt }));

        OCSPReq ocspReq = reqBuilder.build();

        // Send the request via HTTP POST
        URI uri = new URI(ocspUrl);
        HttpURLConnection conn =
                (HttpURLConnection) uri.toURL().openConnection();
        conn.setRequestMethod("POST");
        conn.setRequestProperty("Content-Type",
                "application/ocsp-request");
        conn.setDoOutput(true);

        try (OutputStream os = conn.getOutputStream()) {
            os.write(ocspReq.getEncoded());
        }

        // Parse the response
        OCSPResp ocspResp;
```

```
        try (InputStream is = conn.getInputStream()) {
            ocspResp = new OCSPResp(is);
        }

        if (ocspResp.getStatus() != OCSPResp.SUCCESSFUL) {
            throw new RuntimeException(
                    "OCSP response status: " + ocspResp.getStatus());
        }

        BasicOCSPResp basicResp =
                (BasicOCSPResp) ocspResp.getResponseObject();

        // Find the response for our certificate
        for (SingleResp singleResp : basicResp.getResponses()) {
            if (singleResp.getCertID().equals(certId)) {
                return singleResp;
            }
        }

        return null; // no matching response found
    }
}
```

4.6 Trust Stores

A trust store is a collection of root CA certificates that a system has decided to trust. Without a trust store, there is no anchor for chain validation — every chain ends in a self-signed certificate, and if that certificate is not in a trust store, the chain is untrusted regardless of how cryptographically correct it is.

Different ecosystems maintain different trust stores, and understanding which one applies to your PDF signatures is critical.

4.6.1 Adobe Approved Trust List (AATL)

The **Adobe Approved Trust List** is a curated set of root certificates that Adobe Acrobat and Adobe Reader trust for signature validation. If your signing certificate chains to an AATL root, Acrobat shows a green checkmark with ”Signed and all signatures are valid.” If it does not — even if the certificate is perfectly valid, issued by a reputable CA, and trusted by your operating system — Acrobat shows a yellow warning triangle.

This is the single most important trust store for PDF signatures in practice, because Adobe Acrobat has approximately 75% of the worldwide PDF viewer market share.

4.6.1.1 How AATL Works

Adobe publishes the AATL as a signed file that Acrobat downloads automatically (the URL is hardcoded). The list is refreshed every 30 days. It contains approximately 250 root certificates from CAs around the world.

To get on the AATL, a CA must:

1. Pass a WebTrust or ETSI audit for PDF signing
2. Meet Adobe's technical requirements (minimum RSA-2048, SHA-256 or stronger)
3. Sign an agreement with Adobe
4. Pay an annual fee (reportedly in the range of $10,000-$50,000 USD, though Adobe does not publish the exact figure)

4.6.1.2 The Monopoly Problem

The AATL creates a de facto monopoly on "valid" PDF signatures in the world's dominant viewer. If you want your signed PDFs to show a green checkmark in Acrobat, you need a certificate from an AATL-member CA. Those certificates cost significantly more than standard TLS or code-signing certificates — typically $200-$500 per year for a basic document signing certificate, and considerably more for organizational certificates.

This pricing creates a barrier to entry. An individual developer who wants to sign PDFs "properly" (by Acrobat's standards) must pay for an AATL-rooted certificate, even though the cryptographic security of a self-signed certificate is identical. The trust model imposes a financial cost on trust itself.

This is not unique to Adobe — the TLS ecosystem has a similar dynamic with browser trust stores. But in TLS, Let's Encrypt disrupted the pricing model by offering free certificates. There is no Let's Encrypt for PDF signing certificates. The AATL's requirements (particularly the audit requirements) make a free document signing CA impractical.

4.6.2 EU Trusted List (EUTL)

The **European Union Trusted List** is a fundamentally different approach to trust. Instead of a single vendor curating a list, the EUTL is a hierarchy of machine-readable, cryptographically signed XML documents maintained by the 27 EU member states under the authority of the eIDAS regulation (Regulation (EU) No 910/2014 (eIDAS)).

4.6.2.1 Structure: The LOT Hierarchy

The EUTL follows a **List of Trusted Lists (LOT)** architecture:

1. The **European Commission** publishes the LOT — a signed XML document listing the URLs of every member state's national trusted list.
2. Each **member state** publishes its own trusted list — a signed XML document listing every Qualified Trust Service Provider (QTSP) in that country, along with the services they offer and the certificates associated with those services.
3. Each QTSP entry includes the **service type** (e.g., issuing qualified certificates for electronic signatures), the **service status** (granted, withdrawn, etc.), and the relevant certificates.

The LOT is defined by ETSI TS 119 612 V2.1.1 (2015-07) and is publicly accessible at a well-known URL maintained by the European Commission.

4.6.2.2 How EUTL Differs from AATL

Table 4.5: AATL vs EUTL comparison

Aspect	AATL	EUTL
Governance	Adobe (private company)	European Commission + member states (public authority)
Legal basis	Adobe's terms of service	eIDAS regulation (Regulation (EU) No 910/2014 (eIDAS)), legally binding across 27 member states
Format	Proprietary signed file	Machine-readable XML, signed per ETSI TS 119 612 V2.1.1 (2015-07)
Scope	Any CA willing to pay and pass audit	Only Qualified Trust Service Providers recognized under eIDAS
Cost to CAs	Annual fee to Adobe	No fee to EU; the QTSP status is granted by the national supervisory body
Viewer support	Adobe Acrobat (dominant), some others	Adobe Acrobat (imports EUTL), EU DSS validation framework, national validators
Legal effect	None (Acrobat showing "valid" has no legal meaning)	Qualified electronic signatures have the legal equivalent of handwritten signatures across the EU

Adobe Acrobat can import the EUTL in addition to the AATL. When configured to use both, Acrobat validates against the union of AATL and EUTL trust anchors. In European deployments, this is the standard configuration.

A signature that validates against the EUTL as a qualified electronic signature has legal standing that an AATL-only signature does not. The EUTL is not just a technical trust store — it is a legal instrument. If you are operating in the EU, the EUTL is the authoritative trust store for determining whether a signature meets the highest legal tier. Chapter 19 covers the eIDAS legal framework in detail.

4.6.3 Operating System Trust Stores

Every major operating system maintains its own root CA trust store:

- **Windows**: The Microsoft Root Certificate Program. Certificates are distributed via Windows Update. The store is managed through the `certmgr.msc` snap-in or via `certutil`. Programs using the Windows CryptoAPI (Schannel, .NET) use this store automatically.
- **macOS**: The Apple Root Certificate Program. Certificates are in the System Keychain. Managed through Keychain Access or `security` command-line tool. Programs using Security.framework (Safari, macOS apps) use this store.

- **Linux**: Varies by distribution. Debian/Ubuntu use `/etc/ssl/certs/` and the `ca-certificates` package. Red Hat/Fedora use `/etc/pki/tls/certs/` and the `ca-trust` system. Updated via package manager.

For PDF signatures, OS trust stores are relevant primarily when using system-level PDF viewers (Preview on macOS, Evince on Linux) or when a Java application is configured to use the OS trust store instead of Java's cacerts. Adobe Acrobat on Windows and macOS does **not** use the OS trust store for signature validation by default — it uses the AATL (and optionally the EUTL).

4.6.4 Java cacerts

Java ships with its own trust store: a PKCS#12 file (or JKS in older JDKs) located at `$JAVA_HOME/lib/security/cacerts`. The default password is the string `changeit` (and almost nobody changes it).

The Java cacerts store is maintained by the JDK vendor (Oracle, Eclipse Adoptium, Amazon Corretto, etc.) and typically mirrors the major browser/OS trust stores with some additions and removals.

```
import java.io.FileInputStream;
import java.security.KeyStore;
import java.security.cert.X509Certificate;
import java.util.Enumeration;

/**
 * List all trusted root certificates in the Java cacerts store.
 */
public class CacertsExample {

    public static void listTrustedRoots() throws Exception {
        String cacertsPath = System.getProperty("java.home")
                + "/lib/security/cacerts";

        KeyStore ks = KeyStore.getInstance("PKCS12");
        try (FileInputStream fis = new FileInputStream(cacertsPath)) {
            ks.load(fis, "changeit".toCharArray());
        }

        Enumeration<String> aliases = ks.aliases();
        int count = 0;
        while (aliases.hasMoreElements()) {
            String alias = aliases.nextElement();
            if (ks.isCertificateEntry(alias)) {
                X509Certificate cert = (X509Certificate)
                        ks.getCertificate(alias);
                count++;
                // cert.getSubjectX500Principal().getName()
                // gives the root CA's distinguished name
            }
        }
        // Typical count: 100-150 root certificates
    }
}
```

To add a custom root CA to the Java trust store (for testing or internal PKIs):

```
keytool -importcert \
    -alias "my-internal-ca" \
    -file my-root-ca.pem \
    -keystore $JAVA_HOME/lib/security/cacerts \
    -storepass changeit
```

Modifying the system cacerts file affects all Java applications on that machine. For production deployments, prefer creating a separate trust store and configuring your application to use it via `javax.net.ssl.trustStore` system property or programmatic `TrustManagerFactory` configuration. Do not modify the global cacerts unless you are administering the entire machine.

4.6.5 Custom Trust Stores

Sometimes none of the standard trust stores are appropriate:

- **Internal PKIs**: Your organization runs its own CA hierarchy for internal document signing. The root CA is not in AATL, EUTL, or any OS trust store, because it was never intended to be publicly trusted.
- **Restricted trust**: You want to validate PDF signatures but only trust a specific subset of CAs — perhaps only EUTL-listed QTSPs, not the entire AATL.
- **Air-gapped environments**: Systems with no network access cannot download CRL/OCSP responses or refresh trust stores. All trust material must be pre-loaded.

In Java, creating a custom trust store is straightforward:

```
import java.io.FileOutputStream;
import java.security.KeyStore;
import java.security.cert.CertificateFactory;
import java.security.cert.X509Certificate;
import java.io.FileInputStream;

/**
 * Create a custom trust store containing only your organization's
 * root CA certificate. Use this for internal PKI validation.
 */
public class CustomTrustStoreExample {

    public static void createCustomTrustStore(
            String rootCertPath,
            String trustStorePath,
            String password) throws Exception {

        CertificateFactory cf = CertificateFactory.getInstance("X.509");
        X509Certificate rootCert;
        try (FileInputStream fis = new FileInputStream(rootCertPath)) {
            rootCert = (X509Certificate) cf.generateCertificate(fis);
        }

        KeyStore trustStore = KeyStore.getInstance("PKCS12");
        trustStore.load(null, null); // initialize empty store
        trustStore.setCertificateEntry("internal-root", rootCert);
```

```
        try (FileOutputStream fos =
                new FileOutputStream(trustStorePath)) {
            trustStore.store(fos, password.toCharArray());
        }
    }
}
```

4.7 Trust Store Comparison

Table 4.6: Trust store comparison for PDF signature validation

Characteristic	AATL	EUTL	OS Store	Java cacerts
Primary consumer	Adobe Acrobat/Reader	EU DSS, Acrobat (configured)	OS-native apps, browsers	Java applications
Approximate root count	~250	Varies by member state; ~100 QTSPs total	~150-200	~100-150
Update mechanism	Auto-download (30-day cycle)	LOT refresh per ETSI TS 119 612 V2.1.1 (2015-07)	OS update (Windows Update, etc.)	JDK update
Legal standing	None	Qualified signatures per eIDAS	None	None
PDF viewer support	Adobe Acrobat (primary)	Adobe Acrobat (opt-in), EU validators	Evince, Preview, system viewers	PDFBox-based apps (depends on config)
Cost for CA inclusion	Annual fee + audit	National supervisory body approval	Vendor program requirements	JDK vendor decision

4.8 Debugging Chain Failures

When a PDF signature shows ”certificate chain not trusted,” here is the systematic approach:

4.8.1 Step 1: Extract the Certificates

Open the PDF in PDFalyzer or use PDFBox to extract the CMS `SignedData` from `/Contents`. List every certificate in the CMS `certificates` field. Note the subject DN, issuer DN, serial number, SKI, and AKI of each.

4.8.2 Step 2: Build the Chain Manually

Starting from the signing certificate, match AKI to SKI. Can you build a complete chain from the signer to a self-signed root? If a link is missing, that is your problem — the CMS is missing

an intermediate.

4.8.3 Step 3: Check the Trust Anchor

Is the root certificate in the relevant trust store? If you are validating in Acrobat, check the AATL. If you are validating in your Java application, check cacerts or your custom trust store. The root being ”well-known” is not enough — it must be in the specific trust store the validator is using.

4.8.4 Step 4: Check Validity Periods

Is every certificate in the chain currently within its validity period? Or, if the signature has a timestamp, was every certificate valid at the time the timestamp was created?

4.8.5 Step 5: Check Revocation

Is every certificate in the chain un-revoked? Can the validator reach the CRL distribution points or OCSP responders listed in the certificates? Firewall rules, proxy settings, and DNS resolution failures are common culprits in enterprise environments.

4.8.6 Step 6: Check Key Usage

Does the signing certificate have the correct `keyUsage` bits set? Some validators (including Adobe Acrobat) require `nonRepudiation` (also called `contentCommitment`) for document signatures. Others accept `digitalSignature`. If neither is set, the certificate is not suitable for signing regardless of chain validity.

When you encounter a chain failure in production, resist the urge to immediately add the root certificate to your trust store. First, understand *why* the chain is failing. Is it a missing intermediate? A revoked certificate? A validator misconfiguration? Adding a root to a trust store should be a deliberate decision, not a debugging shortcut. Every root you add is a root you trust to issue certificates for any purpose — including, potentially, certificates that could impersonate other entities.

4.9 Cross-Certification and Bridge CAs

We mentioned earlier that trust paths can be graphs, not just chains. **Cross-certification** is the mechanism that creates these more complex topologies.

When two independent PKI hierarchies need to trust each other’s certificates, they can cross-certify: Root A issues a certificate for Root B’s public key (or an intermediate in B’s hierarchy), and vice versa. This creates a trust bridge between the two hierarchies without requiring end-entity certificates to be re-issued.

The US Federal PKI uses a **Bridge CA** architecture: the Federal Bridge CA cross-certifies with individual agency CAs, enabling certificates issued by the Department of Defense to be validated by systems that trust the Department of State’s root, and vice versa.

For PDF signatures, cross-certification is relevant when a signing certificate chains to a root that is not directly in the validator’s trust store, but *is* cross-certified with a root that is. The

chain-building algorithm must consider cross-certificates as additional links, potentially finding a path through an unexpected route.

This adds complexity to chain building, and not all validators handle it correctly. Adobe Acrobat supports cross-certification for AATL and EUTL roots. Java's `CertPathValidator` supports it if you provide the cross-certificates as additional material via `PKIXBuilderParameters.addCertStore()`. PDFBox relies on BouncyCastle for chain building and validation, and BouncyCastle handles cross-certification correctly in most cases.

If you are debugging a chain failure and the certificates seem correct but the path does not reach a trust anchor, check for cross-certificates. The missing link might not be a traditional intermediate CA but a cross-certificate that bridges two PKI hierarchies. Cross-certificates are sometimes stored in LDAP directories rather than embedded in the CMS structure, making them easy to overlook.

4.10 Summary

Certificate chains exist because no single CA can practically serve everyone. The hierarchical delegation model — root CAs issuing to intermediate CAs, intermediates issuing to end-entities — distributes the operational burden while maintaining a chain of trust back to a known anchor.

Chain building matches AuthorityKeyIdentifier to SubjectKeyIdentifier, walking from the end-entity up through intermediates to the root. Path validation (RFC 5280 — Internet X.509 PKI Certificate and CRL Profile section 6) verifies that every link in the chain is cryptographically sound, temporally valid, and compliant with the constraints imposed by certificates higher in the chain.

Revocation checking — via CRLs for bulk status and OCSP for real-time, per-certificate status — ensures that compromised or withdrawn certificates are detected. For PDF signatures, embedding revocation data in the document (via the DSS or CMS unsigned attributes) enables long-term validation without network access.

The trust store anchoring the chain determines whether a signature is considered "trusted." In the PDF world, the AATL dominates because Adobe Acrobat dominates. In the EU, the EUTL provides legally binding trust anchors for qualified electronic signatures. OS trust stores and Java's cacerts serve broader ecosystems but are not the primary validation paths for most PDF viewers.

When chain validation fails, the debugging process is methodical: extract certificates, build the chain manually, check the trust anchor, verify validity periods, check revocation status, and verify key usage. Most chain failures in production are caused by missing intermediates, not by actual trust problems.

The next chapter examines the formats in which keys and certificates are stored and transported — PKCS#12, PEM, DER, JKS, and PKCS#8 — because loading the wrong format is the quickest way to turn a working certificate chain into an inscrutable exception stack trace.

Chapter 5

Key Material Formats

5.1 PKCS#12 / PFX: The Universal Container

PKCS#12 is the format you will use most often for PDF signing. It bundles a private key, the corresponding certificate, and (usually) the full certificate chain into a single, password-protected file. The standard file extensions are `.p12` and `.pfx`.

5.1.1 History

The lineage is tangled, as most things in cryptography are.

Microsoft created **PFX** (Personal Information Exchange) in the mid-1990s as a container format for exporting keys and certificates from Internet Explorer and IIS. RSA Laboratories, working on their PKCS (Public-Key Cryptography Standards) series, developed **PKCS#12** as a platform-neutral alternative. The two formats converged over time: PKCS#12 v1.0 (1999) incorporated Microsoft's PFX design, and **RFC 7292** (2014) formalized the standard.

Today, `.p12` and `.pfx` are effectively interchangeable extensions for the same format. Some tools (notably Windows) prefer `.pfx`; others (Java, OpenSSL) prefer `.p12`. The binary content is identical.

You will occasionally encounter ancient `.pfx` files from the late 1990s that use a pre-standard PFX format incompatible with PKCS#12. These are rare in 2026 but not extinct. OpenSSL will refuse to parse them, producing an error about "unknown bag type." If you encounter one, the only reliable conversion path is through a legacy Windows system or Internet Explorer's certificate export.

5.1.2 Internal Structure

A PKCS#12 file is a DER-encoded ASN.1 structure containing:

Table 5.1: PKCS#12 internal structure

Component	Description
PFX PDU	The outermost wrapper. Contains a version number (always 3), the authenticated safe, and an optional MAC (Message Authentication Code) for integrity verification.
AuthenticatedSafe	A SEQUENCE of ContentInfo values, each containing one or more SafeBag objects. The ContentInfo can be unencrypted (`data`), encrypted with a password-based cipher (`encryptedData`), or encrypted with a public key (`envelopedData` — rarely used).

Component	Description
SafeBag	The actual content container. Types include `keyBag` (unencrypted private key), `pkcs8ShroudedKeyBag` (encrypted private key — the most common), `certBag` (certificate), `crlBag` (CRL), and `secretBag` (arbitrary secret).
MAC	An HMAC computed over the AuthenticatedSafe using a key derived from the password. This verifies that the password is correct and that the file has not been tampered with. The MAC uses PBKDF2 or a similar key derivation function.

The typical PKCS#12 file for PDF signing contains:

1. One `pkcs8ShroudedKeyBag` — the private key, encrypted with a password-derived key
2. One or more `certBag` entries — the signer's certificate and the intermediate CA certificate(s)

The private key encryption within the PKCS#12 uses a password-based encryption scheme. Historically, this was **PBE-SHA1-3DES** (also known as `pbeWithSHAAnd3-KeyTripleDES-CBC`), which is the default in many older tools and in Java's PKCS#12 implementation prior to JDK 17.

PBE-SHA1-3DES is weak by modern standards. SHA-1 for key derivation and Triple DES for encryption are both deprecated. OpenSSL 3.x defaults to AES-256-CBC with PBKDF2-HMAC-SHA256 for new PKCS#12 files, which is much stronger. However, some tools — including older versions of Adobe Acrobat — cannot read PKCS#12 files encrypted with the newer algorithms. If you generate a PKCS#12 with OpenSSL 3.x and Acrobat cannot open it, try re-exporting with the legacy flag: `openssl pkcs12 -export -legacy -in cert.pem -inkey key.pem -out keystore.pfx`.

5.1.3 Loading PKCS#12 in Java

```
import java.io.FileInputStream;
import java.security.KeyStore;
import java.security.PrivateKey;
import java.security.cert.Certificate;
import java.security.cert.X509Certificate;
import java.util.Enumeration;

/**
 * Load a PKCS#12 keystore and extract the signing key and chain.
 */
public class Pkcs12LoadExample {

    public static void loadAndInspect(
            String p12Path, String password) throws Exception {

        KeyStore ks = KeyStore.getInstance("PKCS12");
        try (FileInputStream fis = new FileInputStream(p12Path)) {
            ks.load(fis, password.toCharArray());
        }
```

```
        // Enumerate all entries
        Enumeration<String> aliases = ks.aliases();
        while (aliases.hasMoreElements()) {
            String alias = aliases.nextElement();

            if (ks.isKeyEntry(alias)) {
                PrivateKey key = (PrivateKey) ks.getKey(
                        alias, password.toCharArray());
                Certificate[] chain = ks.getCertificateChain(alias);

                // key.getAlgorithm() -> "RSA" or "EC"
                // key.getFormat() -> "PKCS#8"
                // chain[0] is the end-entity certificate
                // chain[chain.length-1] is closest to the root

                X509Certificate signingCert =
                        (X509Certificate) chain[0];
                // signingCert.getSubjectX500Principal() -> signer DN
                // signingCert.getNotAfter() -> expiration date
            }
        }
    }
}
```

- The password for the key entry is typically the same as the store password for PKCS#12 files, but the API allows them to be different. Some tools (notably `keytool`) set the key password equal to the store password when creating PKCS#12 files.

When a PKCS#12 file fails to load with `IOException: keystore password was incorrect`, the error is misleading. It can mean (a) the password is actually wrong, (b) the file is corrupted, (c) the file uses an encryption algorithm not supported by your JDK, or (d) the file is not actually PKCS#12 format at all (perhaps it is PEM-encoded and was given a `.p12` extension by mistake). Always check the file format first: `openssl pkcs12 -info -in file.p12 -nokeys -passin pass:test 2>&1 | head -5` will quickly tell you whether the file is valid PKCS#12 and whether the password works.

5.2 PEM: The Text Format

PEM (Privacy-Enhanced Mail) is the most human-friendly key and certificate format — and, paradoxically, has nothing to do with email privacy anymore.

5.2.1 History

PEM was defined in RFCs 1421-1424 (1993) as a framework for email encryption. The framework itself was complex, poorly adopted, and quickly superseded by S/MIME and PGP. But the **encoding format** — Base64-encoded DER data bracketed by header and footer lines — became the de facto standard for exchanging cryptographic objects in text form.

The format is simple:

```
-----BEGIN CERTIFICATE-----
```

```
MIIDXTCCAkWgAwIBAgIJAJC1HiIAZAiUMA0GCSqGSIb3Qw0teleBAUAMEUxCzAJ
BgNVBAYTAkFVMRMwEQYDVQQIDApTb21lLVN0YXRlMSEwHwYDVQQKDBhJbnRlcm5l
... (Base64-encoded DER data, typically 64 or 76 characters per line) ...
dGhlIGNlcnRpZmljYXRl
-----END CERTIFICATE-----
```

The header line identifies the content type. Common headers include:

Table 5.2: PEM header types

Header	Content
`-----BEGIN CERTIFICATE-----`	X.509 certificate (DER-encoded)
`-----BEGIN PRIVATE KEY-----`	PKCS#8 unencrypted private key
`-----BEGIN ENCRYPTED PRIVATE KEY-----`	PKCS#8 encrypted private key
`-----BEGIN RSA PRIVATE KEY-----`	PKCS#1 RSA-specific private key (legacy OpenSSL format)
`-----BEGIN PUBLIC KEY-----`	SubjectPublicKeyInfo (X.509) public key
`-----BEGIN CERTIFICATE REQUEST-----`	PKCS#10 certificate signing request (CSR)
`-----BEGIN X509 CRL-----`	Certificate Revocation List

5.2.2 Multiple Objects in One File

A PEM file can contain multiple objects — this is one of its most useful features. A common pattern for PDF signing is to concatenate the certificate chain into a single file:

```
-----BEGIN CERTIFICATE-----
(end-entity certificate)
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
(intermediate CA certificate)
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
(root CA certificate -- optional)
-----END CERTIFICATE-----
```

Java's `CertificateFactory.generateCertificates(InputStream)` (note the plural) reads all certificates from such a file. BouncyCastle's `PEMParser` reads any PEM objects sequentially.

5.2.3 Loading PEM with BouncyCastle

```
import java.io.FileReader;
import java.security.KeyPair;
import java.security.PrivateKey;
import java.security.cert.X509Certificate;

import org.bouncycastle.cert.X509CertificateHolder;
import org.bouncycastle.cert.jcajce.JcaX509CertificateConverter;
import org.bouncycastle.openssl.PEMDecryptorProvider;
import org.bouncycastle.openssl.PEMEncryptedKeyPair;
```

```
import org.bouncycastle.openssl.PEMKeyPair;
import org.bouncycastle.openssl.PEMParser;
import org.bouncycastle.openssl.jcajce.JcaPEMKeyConverter;
import org.bouncycastle.openssl.jcajce.JcePEMDecryptorProviderBuilder;

/**
 * Load certificates and private keys from PEM files using BouncyCastle.
 */
public class PemLoadExample {

    /**
     * Load a private key from a PEM file.
     * Handles both encrypted and unencrypted keys.
     */
    public static PrivateKey loadPrivateKey(
            String pemPath, String password) throws Exception {

        try (PEMParser parser = new PEMParser(new FileReader(pemPath))) {
            Object obj = parser.readObject();

            JcaPEMKeyConverter converter = new JcaPEMKeyConverter()
                    .setProvider("BC");

            if (obj instanceof PEMEncryptedKeyPair) {
                PEMDecryptorProvider decryptor =
                        new JcePEMDecryptorProviderBuilder()
                                .build(password.toCharArray());
                PEMKeyPair decrypted =
                        ((PEMEncryptedKeyPair) obj).decryptKeyPair(decryptor);
                return converter.getKeyPair(decrypted).getPrivate();
            }

            if (obj instanceof PEMKeyPair) {
                return converter.getKeyPair((PEMKeyPair) obj).getPrivate();
            }

            if (obj instanceof org.bouncycastle.asn1.pkcs.PrivateKeyInfo) {
                return converter.getPrivateKey(
                        (org.bouncycastle.asn1.pkcs.PrivateKeyInfo) obj);
            }

            throw new IllegalArgumentException(
                    "Unexpected PEM object: " + obj.getClass().getName());
        }
    }

    /**
     * Load an X.509 certificate from a PEM file.
     */
    public static X509Certificate loadCertificate(
            String pemPath) throws Exception {

        try (PEMParser parser = new PEMParser(new FileReader(pemPath))) {
            Object obj = parser.readObject();
```

```
            if (obj instanceof X509CertificateHolder) {
                return new JcaX509CertificateConverter()
                        .setProvider("BC")
                        .getCertificate((X509CertificateHolder) obj);
            }

            throw new IllegalArgumentException(
                    "Not a certificate: " + obj.getClass().getName());
        }
    }
}
```

The `PEMParser` class reads objects sequentially. If your PEM file contains a private key followed by certificates, you must call `parser.readObject()` once for the key and then again for each certificate. A common mistake is to read only the first object and close the parser, missing the certificates entirely.

5.3 DER: The Binary Encoding

DER (Distinguished Encoding Rules) is the canonical binary encoding for ASN.1 data structures. Every cryptographic object you encounter — certificates, private keys, CMS structures, OCSP responses — has a DER encoding. PEM is just Base64-encoded DER with headers.

5.3.1 Relationship to PEM

The conversion between DER and PEM is trivial:

- **DER to PEM**: Base64-encode the DER bytes, split into 64-character lines, wrap with the appropriate header/footer.
- **PEM to DER**: Strip the header/footer lines, concatenate the remaining lines, Base64-decode.

DER files have no standard extension — you will see `.der`, `.cer`, `.crt`, and even `.pem` used for DER-encoded data (incorrectly, but it happens). The reliable way to distinguish DER from PEM is to check whether the file starts with `-----BEGIN`: if yes, PEM; if no, DER (or something else entirely).

5.3.2 When DER Matters

For PDF signatures, DER encoding is everywhere:

- The CMS `SignedData` in the `/Contents` field is DER-encoded (then hex-encoded)
- Each certificate in the CMS is DER-encoded
- OCSP responses embedded in the DSS are DER-encoded
- CRLs embedded in the DSS are DER-encoded

When you parse these structures with BouncyCastle, the library handles the DER encoding/decoding transparently. You only deal with DER directly when debugging at the byte level — dumping the hex content of `/Contents` and feeding it to `openssl asn1parse`, for example.

```
# Dump the ASN.1 structure of a DER-encoded CMS blob
openssl asn1parse -inform DER -in signature.der

# Convert DER certificate to PEM
openssl x509 -inform DER -in cert.der -outform PEM -out cert.pem

# Convert PEM certificate to DER
openssl x509 -inform PEM -in cert.pem -outform DER -out cert.der
```

5.4 JKS: Java KeyStore (Legacy)

JKS (Java KeyStore) was Java's original proprietary keystore format, introduced in JDK 1.2 (1998). It served the Java ecosystem for nearly two decades and is still encountered in legacy enterprise systems.

5.4.1 Why JKS Is Being Replaced

JKS has several problems:

1. **Proprietary format**: Only Java tools can read it. OpenSSL cannot parse JKS. Non-Java applications cannot use JKS keystores.
2. **Weak encryption**: The private key protection in JKS uses a custom, non-standard algorithm based on password-derived key material and a proprietary obfuscation scheme. It is not PBE by any recognized standard.
3. **Case-insensitive aliases**: JKS converts all aliases to lowercase, which can cause subtle bugs when migrating to PKCS#12 (which is case-sensitive).
4. **No standard for interoperability**: Migrating keys between JKS and non-Java systems always requires format conversion.

Since JDK 9, the default keystore type in Java is **PKCS12**, not JKS. The `keytool` utility creates PKCS#12 keystores by default. If you are starting a new project, always use PKCS#12.

To check which keystore type you are dealing with: `keytool -list -keystore mystore.jks` will prompt for a password and display entries. If you see "Keystore type: jks" in the output, it is a JKS file. If you see "Keystore type: PKCS12", it is PKCS#12. To convert JKS to PKCS#12: `keytool -importkeystore -srckeystore old.jks -destkeystore new.p12 -deststoretype PKCS12`.

5.5 PKCS#8: Private Key Format

PKCS#8 (RFC 5958, originally RSA PKCS#8) is the standard format for private keys, independent of the key algorithm. When you extract a private key from a PKCS#12 file in Java (`keyStore.getKey()`), the key's `getFormat()` method returns `"PKCS#8"` — that is the encoding format of the key bytes.

5.5.1 Unencrypted vs Encrypted

PKCS#8 defines two structures:

- **PrivateKeyInfo** (unencrypted): Contains the algorithm identifier and the raw private key data. Stored in PEM files with the header `-----BEGIN PRIVATE KEY-----`.
- **EncryptedPrivateKeyInfo**: The PrivateKeyInfo wrapped in a password-based encryption layer. Stored in PEM files with the header `-----BEGIN ENCRYPTED PRIVATE KEY-----`.

```
PrivateKeyInfo ::= SEQUENCE {
    version                 Version,
    privateKeyAlgorithm     AlgorithmIdentifier,
    privateKey              OCTET STRING
}

EncryptedPrivateKeyInfo ::= SEQUENCE {
    encryptionAlgorithm     AlgorithmIdentifier,
    encryptedData           OCTET STRING
}
```

The `privateKey` octet string contains the algorithm-specific key data — an RSA private key (PKCS#1 structure) for RSA, or an EC private key structure for ECDSA. PKCS#8 wraps these algorithm-specific formats in a uniform container.

PKCS#8 is the format that the JCA uses internally. When you call `privateKey.getEncoded()`, the returned bytes are PKCS#8 `PrivateKeyInfo`. When you call `publicKey.getEncoded()`, the returned bytes are `SubjectPublicKeyInfo` (the X.509 public key format, defined in RFC 5280 — Internet X.509 PKI Certificate and CRL Profile). These format names — PKCS#8 for private keys, X.509 for public keys — appear in Java's `Key.getFormat()` and in PEM headers.

5.6 PKCS#1: RSA-Specific Key Format

PKCS#1 (RFC 8017) defines the RSA-specific private key structure:

```
RSAPrivateKey ::= SEQUENCE {
    version           Version,
    modulus           INTEGER,  -- n
    publicExponent    INTEGER,  -- e
    privateExponent   INTEGER,  -- d
    prime1            INTEGER,  -- p
    prime2            INTEGER,  -- q
    exponent1         INTEGER,  -- d mod (p-1)
    exponent2         INTEGER,  -- d mod (q-1)
    coefficient       INTEGER   -- (inverse of q) mod p
}
```

This is the format stored inside a PKCS#8 `PrivateKeyInfo` when the algorithm is RSA. It is also the format used in PEM files with the header `-----BEGIN RSA PRIVATE KEY-----` — OpenSSL's "traditional" format.

The distinction between PKCS#1 and PKCS#8 confuses many developers:

- `-----BEGIN RSA PRIVATE KEY-----` is PKCS#1 (RSA-specific, no algorithm identifier wrapper)
- `-----BEGIN PRIVATE KEY-----` is PKCS#8 (algorithm-agnostic, contains an algorithm identifier)

Both contain the same RSA key data. PKCS#8 simply wraps the PKCS#1 structure with an

additional layer that identifies the algorithm. If you try to load a PKCS#1 PEM file using code that expects PKCS#8, you will get a parsing error — and vice versa.

OpenSSL's default output format changed between versions. OpenSSL 1.x used PKCS#1 (`-----BEGIN RSA PRIVATE KEY-----`) by default for RSA keys. OpenSSL 3.x uses PKCS#8 (`-----BEGIN PRIVATE KEY-----`) by default. If you are following a tutorial written for an older OpenSSL version, the key files it produces may be in a different format than your current OpenSSL generates. Always check the PEM header.

5.7 Format Conversion Reference

This matrix shows how to convert between formats using `openssl` and `keytool`. Every PDF signing project eventually needs at least one of these conversions.

Table 5.3: Key and certificate format conversion commands

From	To	Command
PKCS#12	PEM (key + cert)	`openssl pkcs12 -in store.p12 -out bundle.pem -nodes`
PKCS#12	Separate PEM files	`openssl pkcs12 -in store.p12 -nocerts -out key.pem` (key) then `openssl pkcs12 -in store.p12 -nokeys -out cert.pem` (cert)
PEM (key + cert)	PKCS#12	`openssl pkcs12 -export -in cert.pem -inkey key.pem -out store.p12 -name "alias"`
PEM certificate	DER certificate	`openssl x509 -in cert.pem -outform DER -out cert.der`
DER certificate	PEM certificate	`openssl x509 -inform DER -in cert.der -outform PEM -out cert.pem`
PKCS#1 key	PKCS#8 key	`openssl pkcs8 -topk8 -in rsa.pem -out pkcs8.pem -nocrypt`
PKCS#8 key	PKCS#1 key	`openssl rsa -in pkcs8.pem -out rsa.pem -traditional`
JKS	PKCS#12	`keytool -importkeystore -srckeystore old.jks -destkeystore new.p12 -deststoretype PKCS12`
PKCS#12	JKS	`keytool -importkeystore -srckeystore store.p12 -srcstoretype PKCS12 -destkeystore new.jks -deststoretype JKS`
PEM certificate	JKS (trust entry)	`keytool -importcert -file cert.pem -keystore trust.jks -alias "mycert"`

The `-nodes` flag in the OpenSSL PKCS#12 export command means "no DES" — the private key will not be encrypted in the output PEM file. This is convenient for testing but dangerous in production. Never leave unencrypted private keys on disk in production environments. Use `-aes256` instead of `-nodes` to encrypt the output private key with AES-256.

5.8 HSM Integration: When Keys Cannot Leave

A **Hardware Security Module (HSM)** is a dedicated hardware device that generates, stores, and uses cryptographic keys internally. The private key **never leaves the HSM** — all signing operations happen inside the device. This provides the strongest possible key protection: even if the server is completely compromised, the attacker cannot extract the private key.

5.8.1 PKCS#11

The standard interface for HSM access is **PKCS#11** (also called Cryptoki), defined by OASIS (originally by RSA Laboratories). PKCS#11 defines a C API for cryptographic operations: key generation, signing, encryption, etc. The HSM vendor provides a shared library (`.so` on Linux, `.dll` on Windows) that implements the PKCS#11 interface.

Java accesses PKCS#11 via the **SunPKCS11 provider** — a JCA provider that bridges the Java cryptographic API to a PKCS#11 library.

```
import java.security.KeyStore;
import java.security.Provider;
import java.security.Security;

/**
 * Load keys from an HSM via PKCS#11.
 * The private key never leaves the HSM -- signing happens on-device.
 */
public class Pkcs11Example {

    public static KeyStore loadFromHsm(
            String pkcs11ConfigPath, String pin) throws Exception {

        // Configure the SunPKCS11 provider
        Provider p = Security.getProvider("SunPKCS11");
        p = p.configure(pkcs11ConfigPath);
        Security.addProvider(p);

        // Load the "keystore" -- this actually enumerates
        // objects on the HSM token
        KeyStore ks = KeyStore.getInstance("PKCS11", p);
        ks.load(null, pin.toCharArray());

        return ks;
        // ks.getKey() returns a key handle, not the actual key bytes
        // Signing with this key executes on the HSM
    }
}
```

- The config file specifies the PKCS#11 library path, slot number, and other HSM-specific parameters.
- The PIN authenticates to the HSM token. Unlike PKCS#12 passwords, HSM PINs are typically short numeric codes enforced by the hardware.

5.8.2 Why This Matters for PDF Signing

When you use PDFBox's `SignatureInterface.sign()` with an HSM-backed key, the signing operation is transparently delegated to the HSM. The `PrivateKey` object returned by `ks.getKey()` is a **handle**, not the actual key material. The JCA provider intercepts the signing call, forwards it to the HSM via PKCS#11, and returns the result. Your signing code does not change — the same `CMSSignedDataGenerator` code from Chapter 2 works with HSM keys and software keys identically.

This is the elegance of the JCA provider model: the signing code is agnostic to key storage. Whether the key lives in a PKCS#12 file, a JKS keystore, or an HSM, the API is the same.

For eIDAS qualified electronic signatures, the private key *must* be stored in a **Qualified Signature Creation Device (QSCD)** — an HSM or smart card that has been certified to Common Criteria EAL 4+ or equivalent. Software keystores (PKCS#12, JKS) are not QSCDs. If you are building a signing service that needs to produce qualified electronic signatures, HSM integration is not optional — it is a legal requirement under Regulation (EU) No 910/2014 (eIDAS) Article 29.

5.9 Why Format Conversion Goes Wrong

Format conversions are a perennial source of frustration. Here are the most common failure modes:

5.9.1 Password Encoding Mismatch

PKCS#12 passwords can be encoded as either **UTF-8** or **UTF-16BE** (BMPString in ASN.1 terminology) for the key derivation function. The PKCS#12 standard specifies BMPString, but some implementations use UTF-8.

If you create a PKCS#12 with a password containing non-ASCII characters (umlauts, accented characters, CJK characters) using OpenSSL, and then try to open it in Java, the key derivation may produce a different key because the two implementations encode the password differently.

The fix: use only ASCII characters in PKCS#12 passwords. This is not a theoretical recommendation — it is a practical necessity in cross-platform environments.

5.9.2 OpenSSL Legacy Format vs PKCS#8

As mentioned above, OpenSSL's default private key format changed between major versions. A script that worked with OpenSSL 1.1.1 may produce keys that your PKCS#8-expecting Java code cannot parse when run with OpenSSL 3.x, or vice versa.

Always check the PEM header and convert as needed.

5.9.3 Certificate Chain Order in PKCS#12

The PKCS#12 specification does not mandate a specific order for certificates in the file. Some tools store the end-entity certificate first, followed by intermediates. Others store them in reverse order. Java's `KeyStore.getCertificateChain()` returns them in order (end-entity

first), regardless of how they are stored in the file — but only if the certificates are associated with the correct key alias.

If the chain is missing intermediates (because they were not included when the PKCS#12 was created), `getCertificateChain()` returns only the end-entity certificate. This produces the classic ”chain not trusted” error at signing or validation time.

When creating a PKCS#12 for PDF signing, always include the full chain (minus the root). With OpenSSL: `openssl pkcs12 -export -in signer.pem -inkey signer-key.pem -certfile intermediate-chain.pem -out signing.p12`. The `-certfile` flag adds the intermediate certificates. Without it, only the end-entity certificate is included.

5.9.4 DER vs PEM Confusion

Renaming a file’s extension does not change its format. A file named `cert.pem` that is actually DER-encoded will cause a Base64 decode error. A file named `cert.der` that is actually PEM-encoded will cause an ASN.1 parse error.

To reliably detect the format:

```
# If the file starts with "-----BEGIN", it is PEM
head -1 cert.pem

# Try parsing as DER; if it fails, it is probably PEM (or corrupted)
openssl x509 -inform DER -in cert.der -noout 2>&1

# The file command on Linux/macOS can often identify the format
file cert.pem
```

5.10 Summary

Key material comes in several formats, each with its own history, strengths, and failure modes:

- **PKCS#12** is the universal container for private keys and certificate chains. Use it for PDF signing keystores.
- **PEM** is the human-readable, Base64-encoded format for individual certificates, keys, and other cryptographic objects. Use it for interchange with OpenSSL and when you need to inspect contents visually.
- **DER** is the binary ASN.1 encoding underlying everything. You encounter it inside PDF signatures (the CMS blob in `/Contents`), in OCSP responses, and in CRLs.
- **JKS** is Java’s legacy keystore format. Migrate to PKCS#12.
- **PKCS#8** is the standard private key format (algorithm-agnostic). Java uses it internally.
- **PKCS#1** is the RSA-specific key format. Watch for the PEM header to distinguish it from PKCS#8.

For HSM-backed signing (mandatory for eIDAS qualified signatures), keys are accessed via PKCS#11 and never leave the hardware. The JCA provider model makes this transparent to your signing code.

Format conversion problems — password encoding, legacy vs modern defaults, missing chain certificates, DER/PEM confusion — are among the most common issues in PDF signing

deployments. The conversion reference table in this chapter and a systematic approach to checking file formats will save you hours of debugging.

In the next chapter, we examine the CMS SignedData structure — the envelope that carries the cryptographic signature, the certificate chain, and the signed attributes from the signer to the validator, all encoded inside the PDF's `/Contents` hex string.

Chapter 6

CMS/PKCS#7 Signed Data

Every PDF digital signature carries its cryptographic payload inside a CMS `SignedData` structure — a binary envelope defined by RFC 5652 — Cryptographic Message Syntax (CMS) that bundles the signature value, the signer's certificate chain, the digest algorithm, and a set of signed metadata attributes into a single DER-encoded blob. This chapter dissects that envelope layer by layer, from the outermost `ContentInfo` wrapper down to the individual signed attributes, with annotated ASN.1 dumps and BouncyCastle 1.80 code on Java 21. If you understand CMS `SignedData`, you understand the heart of what lives inside the `/Contents` hex string of every signed PDF.

If you have parsed CMS structures before, skip to SignerInfo: The Core of the Signature for the signed attributes encoding trap that has caused more validation failures than any other single issue. If you want code, jump to Java: Parsing CMS SignedData.

6.1 From PKCS#7 to CMS: A History

The Cryptographic Message Syntax did not spring into existence fully formed. It evolved over sixteen years through three distinct phases, each driven by different organizations and different needs. Understanding this lineage matters because the older formats are still embedded in millions of signed PDFs, and the naming confusion between "PKCS#7" and "CMS" persists in specifications, code comments, and error messages to this day.

6.1.1 PKCS#7: RSA Laboratories, 1993

RSA Laboratories published **PKCS#7** (Public-Key Cryptography Standard #7) in November 1993 as a general-purpose format for signed and encrypted data. The full title was "Cryptographic Message Syntax Standard," which is ironic because it was not yet the CMS we know today — it was a proprietary RSA specification, not an IETF standard.

PKCS#7 version 1.5 defined the core structure that survives almost unchanged:

- A `ContentInfo` wrapper with an OID identifying the content type
- A `SignedData` type containing digest algorithms, signer information, certificates, and CRLs
- A `SignerInfo` type containing the actual signature and per-signer metadata
- Support for both *attached* (encapsulated) and *detached* signatures

The design was influenced by the PEM (Privacy-Enhanced Mail) framework from RFCs 1421-1424 and by the emerging S/MIME work at RSA. The goal was a format that could carry signed data for email, file signing, code signing, and any other context where a digital signature needed to travel alongside or separately from its content.

The ”PKCS#7” name persists everywhere. The PDF specification uses `adbe.pkcs7.detached` and `adbe.pkcs7.sha1` as SubFilter values. Java’s `KeyStore.getInstance("PKCS7")` exists. OpenSSL has `pkcs7` commands. In all of these contexts, the actual format in use is CMS as defined by RFC 5652 — Cryptographic Message Syntax (CMS), not the original 1993 PKCS#7. The names are legacy; the structures have evolved.

6.1.2 S/MIME and RFC 2315, 1998

The S/MIME (Secure/Multipurpose Internet Mail Extensions) working group at the IETF took PKCS#7 and brought it into the standards process. **RFC 2315** (March 1998) republished PKCS#7 v1.5 essentially verbatim as an informational RFC, giving it a stable, citable reference in the IETF document series.

But the S/MIME group needed enhancements that RSA’s original design did not provide: better algorithm agility, support for attribute certificates, and a cleaner separation between the content being signed and the signature metadata. This work produced the first true CMS specification.

6.1.3 CMS: RFC 2630 (1999) and RFC 3852 (2004)

RFC 2630 (June 1999) was the first IETF-standards-track CMS specification. It renamed the format from ”PKCS#7” to ”Cryptographic Message Syntax” and introduced several changes:

- The `SignedData` version number advanced from 1 to 3 (with version selection rules based on which optional fields are present)
- `EncapsulatedContentInfo` replaced the old `ContentInfo` for the inner content, allowing non-data content types
- The `SignerIdentifier` was formalized with two choices: `issuerAndSerialNumber` (the classic) and `subjectKeyIdentifier` (new)
- Attribute certificates were added to the `certificates` field as an option alongside X.509 certificates
- The `digestAlgorithms` SET was required to be consistent with the algorithms actually used in `SignerInfo` entries

RFC 3852 (July 2004) was a revision that clarified ambiguities and tightened requirements, particularly around the encoding of signed attributes.

6.1.4 RFC 5652: The Current Standard (2009)

RFC 5652 — Cryptographic Message Syntax (CMS) (September 2009) is the current and authoritative CMS specification. It obsoletes RFC 3852 and is the normative reference used by ISO 32000-2:2020, the PAdES standards (ETSI EN 319 142-1 V1.1.1 (2016-04)), and BouncyCastle.

The changes from RFC 3852 were modest — mostly editorial clarifications and alignment with updated ASN.1 modules. The core structures are stable and have not changed since 1999. When we refer to ”CMS” in this book, we mean RFC 5652 — Cryptographic Message Syntax (CMS) unless otherwise noted.

Table 6.1: CMS evolution timeline

Year	Document	Significance
1993	PKCS#7 v1.5	RSA Laboratories' original specification. Defined SignedData, EnvelopedData, and the ContentInfo wrapper. Still referenced by name in PDF SubFilter values.
1998	RFC 2315	PKCS#7 republished as an informational RFC. No technical changes, but gave the format an IETF-citable reference.
1999	RFC 2630	First standards-track CMS. Introduced version 3 SignedData, SubjectKeyIdentifier as an alternative SignerIdentifier, and formalized attribute handling.
2004	RFC 3852	Clarified signed attributes encoding requirements. Tightened consistency rules for digestAlgorithms SET.
2009	RFC 5652 — Cryptographic Message Syntax (CMS)	Current standard. Obsoletes RFC 3852. The normative reference for all modern PDF signature work.

6.2 Why PDF Uses CMS

When Adobe added digital signatures to PDF 1.3 in 1999, they needed a container format for the cryptographic signature. They chose PKCS#7 (which was becoming CMS at exactly the same time) for pragmatic reasons:

1. **It was the only real option.** In 1999, the choices for cryptographic message formats were PKCS#7/CMS and PGP. PKCS#7/CMS was the enterprise and government standard; PGP was associated with personal email encryption. For a document format targeting business use, PKCS#7/CMS was the obvious choice.
2. **It bundles everything needed for validation.** A CMS `SignedData` structure can carry the signature value, the signer's certificate, intermediate CA certificates, CRLs, and timestamp tokens — everything a validator needs without making network requests. This self-contained property is critical for PDF, where a signed document might be validated years after signing, on an air-gapped system with no network access.
3. **It supports detached signatures natively.** PDF needs the signed content (the byte range) to live in the PDF file, not inside the CMS structure. CMS's detached signature mode was designed for exactly this use case.

The practical consequence is that the `/Contents` value in every PDF signature dictionary is a hex-encoded, DER-encoded CMS `SignedData` structure. When ISO 32000-2:2020 says the `/Contents` field contains "the DER-encoded PKCS#7 binary data object," it means a CMS `SignedData` as defined by RFC 5652 — Cryptographic Message Syntax (CMS). The terms are used interchangeably in the spec, which is historically understandable but terminologically sloppy.

6.3 The ContentInfo Wrapper

Every CMS structure is wrapped in a `ContentInfo` — the outermost ASN.1 envelope that identifies what type of CMS object is inside.

```
ContentInfo ::= SEQUENCE {
    contentType     ContentType,     -- OID identifying the CMS type
    content    [0]  EXPLICIT ANY DEFINED BY contentType
}

ContentType ::= OBJECT IDENTIFIER
```

For a PDF signature, the `contentType` is always:

```
1.2.840.113549.1.7.2   -- id-signedData
```

This OID lives under the RSA Security arc (`1.2.840.113549`) in the PKCS#7 namespace (`.1.7`), with `.2` identifying `SignedData` specifically. The `content` field contains the `SignedData` structure itself, tagged with EXPLICIT `[0]`.

Table 6.2: CMS ContentType OIDs

OID	Name	Usage in PDF
`1.2.840.113549.1.7.1`	`id-data`	Used as the `eContentType` inside SignedData to indicate that the signed content is arbitrary data (the PDF byte range)
`1.2.840.113549.1.7.2`	`id-signedData`	The outermost ContentInfo type for all PDF signatures
`1.2.840.113549.1.7.3`	`id-envelopedData`	Not used in PDF signatures (used for encryption)
`1.2.840.113549.1.9.16.1.4`	`id-ct-TSTInfo`	Used as the `eContentType` for RFC 3161 timestamp tokens embedded in PDF signatures

The OID `1.2.840.113549.1.7.2` is the first thing a CMS parser encounters when reading a PDF signature. If this OID is missing or wrong, the entire structure is unparsable. When you are debugging a broken signature by examining raw hex, look for the DER encoding of this OID near the beginning of the `/Contents` value: `06 09 2A 86 48 86 F7 0D 01 07 02`. Those eleven bytes (tag `06`, length `09`, then the nine OID bytes) are the signature of a CMS `SignedData` structure.

6.4 The SignedData Structure

Inside the `ContentInfo` wrapper, the `SignedData` structure contains everything related to the signature:

```
SignedData ::= SEQUENCE {
    version           CMSVersion,
    digestAlgorithms  DigestAlgorithmIdentifiers,
    encapContentInfo  EncapsulatedContentInfo,
    certificates  [0] IMPLICIT CertificateSet OPTIONAL,
    crls          [1] IMPLICIT RevocationInfoChoices OPTIONAL,
    signerInfos       SignerInfos
}
```

- Version number: 1, 3, 4, or 5, depending on which optional fields are present. For PDF signatures with X.509 certificates and `issuerAndSerialNumber` in SignerInfo, this is typically **1**.
- SET OF digest algorithm identifiers. Must include every algorithm used by any SignerInfo in this structure. For a single-signer PDF, this is a set with one element (e.g., SHA-256).
- The content that was signed. For PDF detached signatures, the `eContent` field is **absent** — the content lives in the PDF file itself.
- The certificate collection. Typically contains the signer's end-entity certificate plus all intermediate CA certificates needed to build the chain to a trust anchor.
- CRLs or OCSP responses for revocation checking. Rarely populated in basic PDF signatures; more common in PAdES-B-LT and PAdES-B-LTA profiles where revocation data is embedded for long-term validation.
- SET OF SignerInfo structures — one per signer. PDF signatures almost always have exactly one SignerInfo.

6.4.1 Version Number Rules

The version field is not arbitrary — RFC 5652 — Cryptographic Message Syntax (CMS) Section 5.1 specifies strict rules:

Table 6.3: SignedData version selection (RFC 5652 — Cryptographic Message Syntax (CMS) Section 5.1)

Version	Condition
1	No attribute certificates in `certificates`, no `OtherRevocationInfoFormat` in `crls`, no version-3 `SignerInfo`, and the `encapContentInfo.eContentType` is `id-data`
3	The `encapContentInfo.eContentType` is other than `id-data` (e.g., `id-ct-TSTInfo` for a timestamp token), OR the structure contains `SignerInfo` version 3
4	Attribute certificates with version 2 are present in `certificates`
5	`OtherRevocationInfoFormat` is present in `crls`

For the vast majority of PDF signatures, the version is **1**. Timestamp tokens embedded as unsigned attributes use version **3** because their inner `eContentType` is `id-ct-TSTInfo` rather than `id-data`.

Some older signing libraries (notably early versions of iText before version 5.3) hardcoded

the version to 1 regardless of the content type. This produced technically non-compliant CMS structures that happened to validate in most tools but failed strict validators like the EU's DSS library. If you encounter a CMS blob with version 1 but a non-`id-data` content type, that is a bug in the signing library, not a feature.

6.4.2 The digestAlgorithms SET

This is a SET OF `AlgorithmIdentifier`, declared at the top level of `SignedData` to allow a parser to identify all digest algorithms in use before reading any `SignerInfo` entries. The intent is optimization: a validator reading a large detached signature can start computing digests in parallel before it reaches the `SignerInfo` structures.

For PDF signatures with a single signer using SHA-256:

```
digestAlgorithms:
  SET (1 element)
    SEQUENCE
      OID: 2.16.840.1.101.3.4.2.1  (SHA-256)
      NULL                           (parameters, absent for SHA-2)
```

The `NULL` parameters field for SHA-2 algorithms is a persistent source of interoperability issues. RFC 5652 — Cryptographic Message Syntax (CMS) says the parameters "SHOULD be omitted" for SHA-256 and related algorithms, but some older implementations include an explicit `NULL` (tag `05 00`), and some validators reject structures where it is absent. BouncyCastle 1.80 includes the explicit `NULL` by default, which is the safe choice for maximum compatibility.

6.4.3 EncapsulatedContentInfo: Detached vs Encapsulated

```
EncapsulatedContentInfo ::= SEQUENCE {
    eContentType    ContentType,
    eContent   [0]  EXPLICIT OCTET STRING OPTIONAL
}
```

This is where detached and encapsulated signatures diverge:

- **Detached** (PDF standard): `eContent` is **absent**. The signed content lives outside the CMS structure — in the PDF file, specified by the `/ByteRange` array. The `eContentType` is `id-data` (`1.2.840.113549.1.7.1`).
- **Encapsulated**: `eContent` is **present** and contains the signed data. This mode is used in S/MIME email signatures where the message body is embedded in the CMS structure.

For the SubFilter `adbe.pkcs7.sha1` (deprecated), the CMS uses a hybrid approach: the `eContent` field contains the SHA-1 hash of the byte range (not the byte range itself), making it technically an encapsulated signature over a hash rather than a detached signature over the content. This design was misguided and is one of the reasons `adbe.pkcs7.sha1` was deprecated.

If you are building a PDF signing implementation and you accidentally pass `true` for the encapsulate parameter in `CMSSignedDataGenerator.generate()`, the entire byte range content will be embedded inside the CMS structure, inside the `/Contents` hex string, inside the PDF. The result is a signature that is dramatically larger than necessary (potentially megabytes), wastes space, and will be rejected by validators that expect a detached signature when the SubFilter is `adbe.pkcs7.detached` or `ETSI.CAdES.detached`.

6.4.4 The certificates Field

The `certificates` field is an IMPLICIT SET that can contain:

- X.509 certificates (`Certificate` type from RFC 5280 — Internet X.509 PKI Certificate and CRL Profile)
- Attribute certificates (v1 or v2) — rarely used in PDF signatures
- Other certificate formats — essentially never used

For PDF signatures, this field should contain at a minimum the signer's end-entity certificate and all intermediate CA certificates needed to chain to a trust anchor. The root CA certificate is traditionally *not* included because the validator must already trust it — including it adds bytes without adding trust.

The ordering of certificates in this SET is **undefined** by RFC 5652 — Cryptographic Message Syntax (CMS). A validator must be prepared to receive certificates in any order and build the chain by matching issuer/subject Distinguished Names and Authority/Subject Key Identifiers.

6.5 SignerInfo: The Core of the Signature

Each `SignerInfo` structure represents one signer's contribution to the `SignedData`:

```
SignerInfo ::= SEQUENCE {
    version               CMSVersion,
    sid                   SignerIdentifier,
    digestAlgorithm       DigestAlgorithmIdentifier,
    signedAttrs      [0]  IMPLICIT SignedAttributes OPTIONAL,
    signatureAlgorithm    SignatureAlgorithmIdentifier,
    signature             SignatureValue,
    unsignedAttrs    [1]  IMPLICIT UnsignedAttributes OPTIONAL
}
```

- Version 1 if `sid` is `issuerAndSerialNumber`, version 3 if `sid` is `subjectKeyIdentifier`.
- Identifies which certificate belongs to this signer.
- The hash algorithm used to compute the message digest (e.g., SHA-256).
- Attributes that are included in the signature computation — critical for PDF signatures.
- The signature algorithm (e.g., SHA-256 with RSA, or ECDSA with SHA-256).
- The raw signature bytes.
- Attributes added after signing — used for timestamps, counter-signatures.

6.5.1 SignerIdentifier: Finding the Signer's Certificate

```
SignerIdentifier ::= CHOICE {
    issuerAndSerialNumber    IssuerAndSerialNumber,
    subjectKeyIdentifier [0] SubjectKeyIdentifier
}
```

The `issuerAndSerialNumber` form is dominant in PDF signatures. It contains the issuer DN and serial number of the signer's certificate — enough to uniquely identify the certificate within the `certificates` SET.

The `subjectKeyIdentifier` form uses the Subject Key Identifier extension from the certificate. It is more compact but less commonly used because older validators do not always support

it.

6.5.2 Signed Attributes: What Gets Signed

When signed attributes are present (and for PDF signatures, they **must** be present per ISO 32000-2:2020), the signature is computed over the DER encoding of the signed attributes — *not* over the content itself. The content's digest is included as a signed attribute, creating an indirect binding:

```
content  --(hash)-->  messageDigest attribute
                            |
signed attributes  --(sign)-->  signature value
```

The mandatory signed attributes for PDF signatures are:

Table 6.4: Required signed attributes in PDF CMS signatures

Attribute	OID	Description
`contentType`	`1.2.840.113549.1.9.3`	Must be `id-data` (`1.2.840.113549.1.7.1`). Identifies the type of content that was signed. Required by RFC 5652 — Cryptographic Message Syntax (CMS) whenever signed attributes are present.
`messageDigest`	`1.2.840.113549.1.9.4`	The hash of the content (the PDF byte range). This is the attribute that binds the CMS signature to the PDF document bytes. The validator recomputes this hash and compares it against the value in this attribute.
`signingTime`	`1.2.840.113549.1.9.5`	The time the signer claims to have signed. This is self-asserted and not cryptographically verified — use a timestamp authority for trusted time (see `unsignedAttrs`).
`ESS-signing-certificate-v2`	`1.2.840.113549.1.9.16.2.47`	Required by PAdES (ETSI EN 319 122-1 V1.1.1 (2016-04)). Contains a hash of the signer's certificate, binding the signature to a specific certificate and preventing certificate substitution attacks. Without this attribute, an attacker could replace the signer's certificate with a different certificate that has the same public key.

The `signingTime` attribute is optional per RFC 5652 — Cryptographic Message Syntax (CMS) but effectively mandatory in practice. Every PDF viewer displays the signing time, and a signature

without one looks suspicious to users. However, the signing time is *not* cryptographically trustworthy — the signer controls the clock. For legally binding timestamps, use an RFC 3161 timestamp token in the unsigned attributes (Chapter 18).

6.5.3 The ESS-signing-certificate-v2 Attribute

This attribute deserves special attention because it is the dividing line between a basic CMS signature and a CAdES-compliant signature — and therefore between a signature that merely works and one that meets European regulatory requirements.

The structure contains:

```
SigningCertificateV2 ::= SEQUENCE {
    certs        SEQUENCE OF ESSCertIDv2,
    policies     SEQUENCE OF PolicyInformation OPTIONAL
}

ESSCertIDv2 ::= SEQUENCE {
    hashAlgorithm  AlgorithmIdentifier DEFAULT {sha-256},
    certHash       Hash,              -- hash of the signer's certificate
    issuerSerial   IssuerSerial OPTIONAL
}
```

The `certHash` is a hash of the signer's DER-encoded certificate. A validator computes this hash independently and compares it against the attribute value, ensuring that the certificate in the CMS structure is the one the signer intended to use.

Without this attribute, a subtle attack is possible: if two certificates share the same public key (which can happen if a certificate is re-issued with different extensions or validity periods), an attacker could swap the certificate in the CMS structure without invalidating the signature. The ESS-signing-certificate-v2 attribute prevents this by binding the signature to the exact certificate, not just the public key.

PAdES baseline profiles (PAdES-B-B and above) require ESS-signing-certificate-v2. A PDF signature that uses the SubFilter `ETSI.CAdES.detached` but lacks this attribute is non-compliant with ETSI EN 319 142-1 V1.1.1 (2016-04). The EU's DSS validation library will flag it as a format failure. If you are implementing PDF signing and need European compliance, this attribute is not optional.

6.6 The Signed Attributes DER Encoding Trap

This section describes the single most common cause of CMS signature validation failures. It has tripped up every signing library author at least once, and it will trip you up too if you are building or debugging a CMS implementation at the byte level.

6.6.1 The Problem

RFC 5652 — Cryptographic Message Syntax (CMS) Section 5.4 specifies how signed attributes are encoded for signature computation:

> The result of the message digest calculation process depends on whether the signedAttrs field is present. When the field is present, [...] the complete DER

> encoding of the SignedAttrs value contained in the signedAttrs field is [used]. [...] the IMPLICIT [0] tag in the signedAttrs is not used for the DER encoding, rather an EXPLICIT SET OF tag is used.
>
> — RFC 5652 — Cryptographic Message Syntax (CMS) Section 5.4

Here is what this means in practice:

1. In the `SignerInfo` structure, the `signedAttrs` field is tagged `[0] IMPLICIT`, meaning the ASN.1 tag byte is `A0` (context class, constructed, tag 0).
2. But when computing the signature over the signed attributes, the tag must be replaced with `31` (UNIVERSAL SET OF) — the tag that would be used if the attributes were a standalone SET OF, not an implicitly tagged field.
3. The rest of the encoding (length bytes and content) remains identical.

This means the bytes that are signed are *not* the bytes that appear in the CMS structure. You must re-tag the signed attributes before computing or verifying the signature.

6.6.2 Why This Exists

The reason is rooted in ASN.1 encoding theory. The `signedAttrs` field uses IMPLICIT tagging to save one byte in the encoding (implicit tagging replaces the original tag; explicit tagging adds a wrapper). But the signature must be computed over the *canonical* encoding of the attributes as a SET OF, not over their implicitly tagged representation within the SignerInfo. The two encodings differ only in the first byte (`A0` vs `31`), but that single byte difference is enough to invalidate the signature.

If you are computing a CMS signature manually (perhaps because you are building a remote signing integration where the hash is computed locally and the signature is applied remotely), you **must** re-tag the signed attributes before hashing them. Use tag `31` (SET OF), not `A0` (IMPLICIT [0]). BouncyCastle handles this correctly in its `CMSSignedDataGenerator`, but if you are working at a lower level — constructing CMS structures from raw ASN.1 objects, or implementing a signature interface for an HSM — this is the byte that will break you.

```
import org.bouncycastle.asn1.ASN1EncodableVector;
import org.bouncycastle.asn1.ASN1Set;
import org.bouncycastle.asn1.BERTags;
import org.bouncycastle.asn1.DERSet;

/**
 * Demonstrate the signed attributes re-tagging requirement.
 * In practice, BouncyCastle handles this internally -- this
 * code is for understanding, not production use.
 */
public class SignedAttrsEncodingExample {

    /**
     * Re-encode signed attributes with SET OF tag for signing.
     * The input is the IMPLICIT [0]-tagged bytes from SignerInfo;
     * the output replaces tag 0xA0 with 0x31.
     */
    public static byte[] retagForSigning(byte[] implicitTaggedAttrs) {
        byte[] retagged = implicitTaggedAttrs.clone();
```

```
            // Replace IMPLICIT [0] tag (0xA0) with SET OF tag (0x31)
            if (retagged[0] == (byte) 0xA0) {
                retagged[0] = (byte) (BERTags.SET | BERTags.CONSTRUCTED); // 0x31
            }
            return retagged;
        }
    }
```

When debugging a signature that fails cryptographic verification but the `messageDigest` attribute matches the content hash, the re-tagging issue is the first thing to check. Extract the signed attributes bytes from the CMS structure, verify that the first byte is `31` (not `A0`) when fed into the signature verification, and compare the hash of those bytes against the decrypted signature value.

6.7 Unsigned Attributes: Post-Signature Additions

Unsigned attributes are added to the `SignerInfo` *after* the signature has been computed. They are not covered by the signature and can be added or modified without invalidating it. This is by design — the primary use case is adding information that is not available at signing time.

The most important unsigned attributes for PDF signatures:

Table 6.5: Common unsigned attributes in PDF CMS signatures

Attribute	OID	Description
`signature-time-stamp`	`1.2.840.113549.1.9.16.2.14`	An RFC 3161 timestamp token over the signature value. Provides a trusted, third-party assertion of the time the signature existed. This is the mechanism for timestamping described in Chapter 18.
`counterSignature`	`1.2.840.113549.1.9.6`	A counter-signature — another signer's signature over the original signature value. Rarely used in PDF signatures directly; timestamps are the modern replacement.

The timestamp token in the `signature-time-stamp` attribute is itself a CMS `SignedData` structure, with its `eContentType` set to `id-ct-TSTInfo` (`1.2.840.113549.1.9.16.1.4`). This creates a nested CMS-within-CMS structure: the outer `SignedData` is the PDF signature, and the inner `SignedData` (inside an unsigned attribute of the outer) is the timestamp.

6.8 CAdES: CMS Advanced Electronic Signatures

The basic CMS `SignedData` structure defined by RFC 5652 — Cryptographic Message Syntax (CMS) is cryptographically complete but lacks features needed for European regulatory

compliance. **CAdES** (CMS Advanced Electronic Signatures), defined in ETSI EN 319 122-1 V1.1.1 (2016-04), extends CMS with additional signed attributes and profile rules.

The key additions CAdES makes to CMS for PDF signing:

1. **ESS-signing-certificate-v2** (discussed above) — mandatory in all CAdES profiles, binds the signature to a specific certificate.
2. **Signature policy identifier** — optional attribute that references a specific signature policy (a set of rules defining the meaning and validation requirements of the signature). Used in some government contexts.
3. **Commitment type indication** — optional attribute declaring what the signer intends by signing (proof of origin, proof of receipt, proof of approval, etc.).
4. **Signer location** — optional attribute with the signer's claimed location at signing time.

The relationship between CMS, CAdES, and PAdES is hierarchical:

```
CMS (RFC 5652)
  |
  +-- CAdES (ETSI EN 319 122) = CMS + mandatory signed attributes
        |
        +-- PAdES (ETSI EN 319 142) = CAdES embedded in PDF per ISO 32000-2
```

A PAdES signature is a CAdES signature is a CMS signature. Each layer adds constraints but does not remove any. A basic `adbe.pkcs7.detached` signature is CMS but not necessarily CAdES. An `ETSI.CAdES.detached` signature is CMS, CAdES, and PAdES.

6.9 Old PKCS#7 vs Modern CMS: What Changed

If you work with PDF signatures long enough, you will encounter CMS structures produced by older libraries that do not quite match what RFC 5652 — Cryptographic Message Syntax (CMS) specifies. The most common issues trace back to libraries that implemented the original PKCS#7 v1.5 rather than CMS, or that implemented an intermediate version with bugs that were never fixed because "it worked in Acrobat."

Table 6.6: Differences between legacy PKCS#7 and modern CMS in the wild

Issue	Legacy PKCS#7 Behavior	Modern CMS (RFC 5652 — Cryptographic Message Syntax (CMS))
SignedData version	Always 1, regardless of content type	Version depends on which optional fields are present (1, 3, 4, or 5)
digestAlgorithms consistency	May list algorithms not actually used by any SignerInfo	Must be consistent with SignerInfo entries
Signed attributes encoding	Some libraries used BER (not DER) encoding for the SET OF, causing non-deterministic byte ordering	Must use DER, which enforces a specific ordering of SET elements

Issue	Legacy PKCS#7 Behavior	Modern CMS (RFC 5652 — Cryptographic Message Syntax (CMS))
ESS-signing-certificate	Absent (not part of original PKCS#7)	Required by CAdES and PAdES
Certificate ordering	Some libraries assumed specific ordering	No ordering defined; validators must handle any order
NULL parameters in AlgorithmIdentifier	Inconsistent — some included, some omitted	SHA-2 algorithms SHOULD omit parameters, but explicit NULL is tolerated

Early versions of iText (before 5.3.x) and some older .NET signing libraries produced CMS structures with BER-encoded signed attributes. BER allows multiple valid encodings for the same data — specifically, SET OF elements may appear in any order. DER requires a specific ordering (by encoded value). If the signing library used one ordering and the validator expects another, the hash of the signed attributes will not match and the signature will fail cryptographic verification. The signature is not actually broken — it was computed over BER-encoded bytes, and if you verify against the same bytes, it passes. But strict DER-only validators will reject it. This is the most common ”it works in Acrobat but fails in DSS” scenario.

6.10 Java: Parsing CMS SignedData

Here is how to parse a CMS `SignedData` structure extracted from a PDF signature’s `/Contents` field, using BouncyCastle 1.80:

```
import java.security.cert.X509Certificate;
import java.util.Collection;

import org.bouncycastle.asn1.ASN1OctetString;
import org.bouncycastle.asn1.ASN1UTCTime;
import org.bouncycastle.asn1.cms.Attribute;
import org.bouncycastle.asn1.cms.CMSAttributes;
import org.bouncycastle.cert.X509CertificateHolder;
import org.bouncycastle.cert.jcajce.JcaX509CertificateConverter;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.SignerInformation;
import org.bouncycastle.cms.SignerInformationStore;
import org.bouncycastle.util.Store;
import org.bouncycastle.util.encoders.Hex;

/**
 * Parse and inspect a CMS SignedData structure from a PDF signature.
 * The input is the raw DER bytes from the /Contents hex string.
 */
public class CmsParsingExample {

    public static void inspectCmsSignature(byte[] cmsBytes) throws Exception {
        CMSSignedData signedData = new CMSSignedData(cmsBytes);
```

```
// -- SignedData version --
int version = signedData.getVersion();
// version 1 = standard PDF signature
// version 3 = timestamp token or SubjectKeyIdentifier used

// -- Certificates embedded in the CMS --
Store<X509CertificateHolder> certStore = signedData.getCertificates();
JcaX509CertificateConverter converter =
        new JcaX509CertificateConverter().setProvider("BC");

// -- Iterate over SignerInfos (usually exactly one) --
SignerInformationStore signerInfoStore = signedData.getSignerInfos();

for (SignerInformation signerInfo : signerInfoStore.getSigners()) {

    // Digest algorithm (e.g., "2.16.840.1.101.3.4.2.1" = SHA-256)
    String digestOid = signerInfo.getDigestAlgOID();

    // Signature algorithm (e.g., "1.2.840.113549.1.1.11" = SHA256withRSA)
    String sigAlgOid = signerInfo.getEncryptionAlgOID();

    // -- Signed attributes --
    if (signerInfo.getSignedAttributes() != null) {

        // messageDigest: the hash of the PDF byte range
        Attribute mdAttr = signerInfo.getSignedAttributes()
                .get(CMSAttributes.messageDigest);
        if (mdAttr != null) {
            byte[] digest = ASN1OctetString.getInstance(
                    mdAttr.getAttrValues().getObjectAt(0)
            ).getOctets();
            // Hex.toHexString(digest) for human-readable output
        }

        // signingTime: the claimed signing time
        Attribute stAttr = signerInfo.getSignedAttributes()
                .get(CMSAttributes.signingTime);
        if (stAttr != null) {
            ASN1UTCTime time = ASN1UTCTime.getInstance(
                    stAttr.getAttrValues().getObjectAt(0));
            // time.getDate() returns a java.util.Date
        }
    }

    // -- Find the signer's certificate --
    Collection<X509CertificateHolder> matches =
            certStore.getMatches(signerInfo.getSID());
    for (X509CertificateHolder holder : matches) {
        X509Certificate cert = converter.getCertificate(holder);
        // cert.getSubjectX500Principal() = signer identity
        // cert.getNotAfter() = certificate expiration
    }

    // -- Raw signature bytes --
```

```
            byte[] signatureBytes = signerInfo.getSignature();
            // signatureBytes.length = 256 for RSA-2048, 384 for RSA-3072, etc.
        }
    }
}
```

- The `CMSSignedData` constructor parses the DER bytes into an in-memory object graph.
- PDF signatures have exactly one SignerInfo. Multiple signers use multiple signature dictionaries, not multiple SignerInfos in one CMS structure.
- The `getSID()` method returns a selector that matches against the `SignerIdentifier` — either `issuerAndSerialNumber` or `subjectKeyIdentifier`.
- The raw signature value. For RSA, this is the encrypted hash. For ECDSA, this is the DER-encoded `(r, s)` pair.

6.11 Java: Building CMS SignedData

Creating a CMS `SignedData` for PDF signing is the reverse of parsing. The following example uses `CMSSignedDataGenerator` with explicit control over the signed attributes:

```
import java.security.PrivateKey;
import java.security.Security;
import java.security.cert.Certificate;
import java.security.cert.X509Certificate;
import java.util.ArrayList;
import java.util.List;

import org.bouncycastle.cert.jcajce.JcaCertStore;
import org.bouncycastle.cms.CMSProcessableByteArray;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.CMSSignedDataGenerator;
import org.bouncycastle.cms.jcajce.JcaSignerInfoGeneratorBuilder;
import org.bouncycastle.jce.provider.BouncyCastleProvider;
import org.bouncycastle.operator.ContentSigner;
import org.bouncycastle.operator.jcajce.JcaContentSignerBuilder;
import org.bouncycastle.operator.jcajce.JcaDigestCalculatorProviderBuilder;

/**
 * Build a detached CMS SignedData suitable for embedding
 * in a PDF signature's /Contents field.
 */
public class CmsBuildingExample {

    static {
        Security.addProvider(new BouncyCastleProvider());
    }

    /**
     * Create a detached CMS SignedData over the given byte range content.
     *
     * @param byteRangeContent  the PDF bytes specified by /ByteRange
     * @param privateKey        the signer's private key
     * @param chain             the certificate chain (end-entity first)
     * @return DER-encoded CMS SignedData bytes
```

```
     */
    public static byte[] createDetachedSignature(
            byte[] byteRangeContent,
            PrivateKey privateKey,
            Certificate[] chain) throws Exception {

        X509Certificate signingCert = (X509Certificate) chain[0];

        // Determine signature algorithm from key type
        String sigAlg = switch (privateKey.getAlgorithm()) {
            case "RSA" -> "SHA256withRSA";
            case "EC"  -> "SHA256withECDSA";
            default -> throw new IllegalArgumentException(
                    "Unsupported key type: " + privateKey.getAlgorithm());
        };

        ContentSigner signer = new JcaContentSignerBuilder(sigAlg)
                .setProvider("BC")
                .build(privateKey);

        CMSSignedDataGenerator generator = new CMSSignedDataGenerator();

        generator.addSignerInfoGenerator(
                new JcaSignerInfoGeneratorBuilder(
                        new JcaDigestCalculatorProviderBuilder()
                                .setProvider("BC")
                                .build()
                ).build(signer, signingCert)
        );

        // Include the full certificate chain
        List<Certificate> certList = new ArrayList<>();
        for (Certificate cert : chain) {
            certList.add(cert);
        }
        generator.addCertificates(new JcaCertStore(certList));

        // Generate DETACHED signature (false = do not encapsulate content)
        CMSProcessableByteArray content =
                new CMSProcessableByteArray(byteRangeContent);
        CMSSignedData signedData = generator.generate(content, false);

        return signedData.getEncoded();
    }
}
```

- `JcaSignerInfoGeneratorBuilder.build()` automatically adds the standard signed attributes (`contentType`, `messageDigest`, `signingTime`). For CAdES compliance, you need to add ESS-signing-certificate-v2 explicitly using a `CMSAttributeTableGenerator`.
- The `false` parameter is critical: detached signature. The byte range content is hashed but not embedded in the CMS structure.

6.12 Annotated ASN.1 Dump of a Real PDF Signature

The following is the output of `openssl asn1parse` run against the DER-decoded `/Contents` of an actual PDF signature. Each line is annotated with what it represents in the CMS structure.

```
    0:d=0  hl=4 l=2417 cons: SEQUENCE              -- ContentInfo
    4:d=1  hl=2 l=   9 prim: OBJECT   :pkcs7-signedData
                                                     -- OID 1.2.840.113549.1.7.2
   15:d=1  hl=4 l=2402 cons: cont [ 0 ]             -- EXPLICIT [0] content
   19:d=2  hl=4 l=2398 cons: SEQUENCE                -- SignedData
   23:d=3  hl=2 l=   1 prim: INTEGER  :01            -- version = 1
   26:d=3  hl=2 l=  15 cons: SET                     -- digestAlgorithms
   28:d=4  hl=2 l=  13 cons: SEQUENCE
   30:d=5  hl=2 l=   9 prim: OBJECT   :sha256        -- OID 2.16.840.1.101.3.4.2.1
   41:d=5  hl=2 l=   0 prim: NULL                    -- parameters (explicit NULL)
   43:d=3  hl=2 l=  11 cons: SEQUENCE                -- encapContentInfo
   45:d=4  hl=2 l=   9 prim: OBJECT   :pkcs7-data    -- eContentType = id-data
                                                     -- OID 1.2.840.113549.1.7.1
                                                     -- (eContent ABSENT = detached)
   56:d=3  hl=4 l=1735 cons: cont [ 0 ]             -- certificates [0] IMPLICIT
   60:d=4  hl=4 l= 872 cons: SEQUENCE                -- Certificate #1 (end-entity)
  936:d=4  hl=4 l= 855 cons: SEQUENCE                -- Certificate #2 (intermediate CA)
 1795:d=3  hl=4 l= 624 cons: SET                     -- signerInfos
 1799:d=4  hl=4 l= 620 cons: SEQUENCE                -- SignerInfo
 1803:d=5  hl=2 l=   1 prim: INTEGER  :01            -- version = 1
 1806:d=5  hl=3 l= 141 cons: SEQUENCE                -- sid = issuerAndSerialNumber
 1809:d=6  hl=3 l= 118 cons: SEQUENCE                --   issuer DN
 1930:d=6  hl=2 l=  19 prim: INTEGER  :...           --   serialNumber
 1951:d=5  hl=2 l=  13 cons: SEQUENCE                -- digestAlgorithm
 1953:d=6  hl=2 l=   9 prim: OBJECT   :sha256        --   SHA-256
 1964:d=6  hl=2 l=   0 prim: NULL
 1966:d=5  hl=3 l= 189 cons: cont [ 0 ]             -- signedAttrs [0] IMPLICIT
 1969:d=6  hl=2 l=  24 cons: SEQUENCE                --   contentType attribute
 1995:d=6  hl=2 l=  47 cons: SEQUENCE                --   messageDigest attribute
 2044:d=6  hl=2 l=  28 cons: SEQUENCE                --   signingTime attribute
 2074:d=6  hl=2 l=  79 cons: SEQUENCE                --   ESS-signing-certificate-v2
 2155:d=5  hl=2 l=  13 cons: SEQUENCE                -- signatureAlgorithm
 2157:d=6  hl=2 l=   9 prim: OBJECT   :sha256WithRSAEncryption
                                                     -- OID 1.2.840.113549.1.1.11
 2168:d=6  hl=2 l=   0 prim: NULL
 2170:d=5  hl=4 l= 256 prim: OCTET STRING            -- signature (256 bytes = RSA-2048)
 2430:d=5  hl=2 l=   7 cons: cont [ 1 ]             -- unsignedAttrs [1] IMPLICIT
                                                     -- (timestamp token would be here)
```

The offsets in this dump are from the start of the DER-encoded CMS blob, not from the start of the PDF file. To get the absolute file offset, add the starting offset of the `/Contents` hex string (as specified by `/ByteRange`) plus the hex-to-binary decoding offset. When debugging with a hex editor, remember that the `/Contents` value is hex-encoded: each byte in the ASN.1 dump corresponds to two hex characters in the PDF file.

6.13 Summary

CMS `SignedData` is the envelope that carries every PDF digital signature. Its lineage stretches from RSA's PKCS#7 in 1993 through the S/MIME working group to the current RFC 5652 — Cryptographic Message Syntax (CMS) standard. The core design has been stable for over twenty-five years, which is both its strength (mature, well-understood, universally supported) and its burden (legacy quirks from 1993 are still with us).

The key points for PDF signature work:

- **ContentInfo wraps everything.** The OID `1.2.840.113549.1.7.2` identifies the CMS as `SignedData`. Learn to recognize its DER encoding (`06 09 2A 86 48 86 F7 0D 01 07 02`) in hex dumps.
- **PDF signatures are detached.** The `eContent` in `EncapsulatedContentInfo` is absent. The signed content lives in the PDF file, addressed by `/ByteRange`.
- **Signed attributes are signed, not the content.** The signature covers the DER-encoded signed attributes, which include the `messageDigest` (hash of the content). This two-level indirection is essential to understand.
- **The DER re-tagging trap is real.** The signed attributes must be re-tagged from `A0` to `31` before hashing for signature computation. BouncyCastle handles this, but any lower-level implementation must get this right.
- **ESS-signing-certificate-v2 is the CAdES requirement.** Without it, the signature is CMS-valid but not CAdES-valid and therefore not PAdES-compliant.
- **Unsigned attributes carry timestamps.** The `signature-time-stamp` attribute contains a nested CMS structure — a timestamp token from an RFC 3161 TSA.
- **Legacy PKCS#7 differences still cause interop failures.** BER vs DER encoding of signed attributes is the most common culprit.

The next part of this book turns to PDF internals — the file structure, incremental updates, and form fields that the CMS `SignedData` is embedded within. Understanding the container is meaningless without understanding the document it lives in.

Chapter 7

PDF Internals

Chapter 8

PDF File Structure

Every PDF file on earth — whether it is a single-page receipt from your corner bakery or a 4,000-page securities filing with twelve nested signatures — is built from the same four structural components, arranged in the same order, governed by the same rules that John Warnock and Chuck Geschke laid down at Adobe in 1993. If you want to understand why a signature's `/ByteRange` array contains raw file offsets, why cross-reference tables use fixed-width 20-byte entries, or why some PDFs have three `%%EOF` markers, you have to understand this structure at the byte level.

This chapter dismantles a PDF file from the first byte to the last. We will trace headers, decode every object type the specification defines, read cross-reference tables in both the classic ASCII format and the newer compressed stream format, examine the trailer dictionary, walk through a minimal PDF byte by byte, and then connect all of it back to the one thing we care about most: digital signatures.

8.1 The Four Sections

ISO 32000-2:2020 Section 7.5 defines the layout. Every conforming PDF file consists of these four parts, in this order:

1. **Header** — a single line declaring the PDF version, plus an optional binary comment
2. **Body** — the collection of indirect objects that define everything in the document: pages, fonts, images, form fields, annotations, and signatures
3. **Cross-reference table** — an index that maps object numbers to byte offsets in the file, enabling random access without linear scanning
4. **Trailer** — a dictionary that tells the parser where to find the cross-reference table, which object is the document root, and (for incremental updates) where to find the previous cross-reference section

This four-part structure has been stable since PDF 1.0. PDF 1.5 introduced an alternative binary representation for the cross-reference data (cross-reference streams), and PDF 2.0 deprecated some legacy metadata keys, but the fundamental anatomy has not changed in over three decades.

8.1.1 The Header

A PDF file begins with a version identifier on the first line:

```
%PDF-1.7
```

That is nine bytes: the five characters `%PDF-`, the version number, and a line ending. The `%` character is the PDF comment delimiter, which means the header line is technically a comment — a design choice that lets non-PDF-aware tools skip it without error.

The version in the header is the *file* version. ISO 32000-2:2020 Section 7.5.2 specifies that a `/Version` key in the document catalog may override this value. In practice, many generators write `%PDF-1.7` in the header and then set `/Version /2.0` in the catalog when they need PDF 2.0 features. This dual-version mechanism exists for backward compatibility: an older parser that does not understand the catalog `/Version` will still see a version number it can process (or at least attempt to process) in the header.

8.1.1.1 The Binary Comment Trick

Immediately after the header line, you will almost always find a second comment line containing four or more bytes with values above 127 (0x80):

```
%âãÏÓ
```

This is not decoration. Its purpose is purely practical: it forces file-type detection heuristics — in web servers, email gateways, FTP clients, operating systems — to classify the file as binary rather than ASCII text. Without this line, a transfer in ASCII mode (remember the FTP `ASCII` vs `BINARY` setting?) could mangle the binary stream data inside the PDF. ISO 32000-2:2020 Section 7.5.2 recommends but does not require this binary hint.

You will occasionally encounter PDFs without the binary comment — usually from minimalist generators or test harnesses. The file is still valid, but it is more vulnerable to transfer corruption.

When debugging PDF files at the byte level, the header is your first sanity check. If `%PDF-` is not the first five bytes, something has happened upstream: a Unicode BOM was prepended, an HTML wrapper was added, a proxy rewrote the response body, or the file was opened as text and line endings were converted. We have encountered every one of these in production systems.

8.1.2 The Body

The body contains all the PDF objects that constitute the document. Page trees, content streams, font dictionaries, image data, form fields, metadata, and — critically for us — signature dictionaries and their CMS payloads.

Objects in the body are written as **indirect objects**, each wrapped in a numbered envelope:

```
7 0 obj
<< /Type /Page /MediaBox [0 0 612 792] /Contents 12 0 R >>
endobj
```

The prefix `7 0 obj` means ”object number 7, generation number 0.” The keyword `endobj` marks the end of the object. Between them is the object’s value — here, a dictionary describing a page.

Objects may appear in any order. There is no requirement that object 1 precedes object 2, or that page objects come before font objects. The cross-reference table handles the mapping from logical object number to physical byte offset. This is a fundamental design principle: **the physical ordering of objects in the file has no bearing on the logical structure of the document**.

Different PDF generators produce wildly different orderings. Adobe Acrobat tends to cluster related objects together. Apache PDFBox writes objects in the order they were allocated in memory. Linearized PDFs (the ”fast web view” format described in ISO 32000-2:2020 Section 7.6) deliberately reorder objects so that the first page can be displayed before the entire file is downloaded. None of this changes the logical content — but it changes the byte offsets, which in turn changes the byte range that a signature covers.

Some tools assume objects appear in numerical order. They do not. If you are writing a PDF parser or debugging one, never assume object N appears before object N+1 in the file. Always consult the cross-reference table.

8.1.3 The Cross-Reference Table

After the body comes the cross-reference table (commonly called the ”xref table”) — an index that records the byte offset where each object starts in the file.

```
xref
0 8
0000000000 65535 f
0000000015 00000 n
0000000078 00000 n
0000000178 00000 n
0000000331 00000 n
0000000409 00000 n
0000000553 00000 n
0000000715 00000 n
```

The keyword `xref` marks the beginning. The line `0 8` means ”this section starts at object number 0 and contains 8 entries.” Each subsequent line is exactly 20 bytes (including the line terminator) and contains three fields:

- **Byte offset** (10 digits, zero-padded) — the position in the file where the object starts
- **Generation number** (5 digits, zero-padded) — used for object versioning; almost always `00000`
- **In-use flag** — `n` means the object is in use; `f` means it is free (deleted)

The very first entry (object 0) is always free, with generation number 65535. It serves as the head of a linked list of free objects. Each free entry’s ”byte offset” field actually contains the object number of the *next* free entry, forming a chain. This mechanism allows PDF writers to reuse deleted object numbers — a feature that mattered when PDFs were small and object numbers precious, and that is now mostly vestigial.

The 20-byte-per-entry format is non-negotiable. This rigidity is deliberate: it enables random access to any entry via arithmetic (`entry_offset = xref_start + header_bytes + object_number * 20`) without scanning the entire table. This random-access property is what allows a viewer to display a single page from a 10,000-page document without loading the whole file into memory.

The generation number increments when an object is deleted and its number is later reused, preventing stale references (`7 0 R`) from resolving to the wrong content. In practice, nearly every object you encounter in the wild has generation 0. The mechanism exists, the specification defines it carefully, and production PDFs almost never exercise it.

8.1.4 The Trailer

Immediately after the cross-reference table comes the trailer:

```
trailer
<<
  /Size 8
  /Root 1 0 R
  /Info 6 0 R
  /ID [<a9f2c74b...> <a9f2c74b...>]
>>
startxref
4892
%%EOF
```

The trailer dictionary is the most important dictionary in the file, because *everything* starts here. A PDF parser begins at the **end** of the file. It searches backward for `%%EOF`, then backward for `startxref`, reads the byte offset (4892), and jumps to that offset to find the cross-reference table. From the xref table, it reads the trailer dictionary. From the trailer dictionary's `/Root` key, it finds the document catalog. From the catalog, it can navigate to every object in the document.

Parsing a PDF starts from the end, not the beginning. This backward-search strategy is fundamental to how incremental updates work (Chapter 8), and it is the reason a PDF can be extended by appending data without rewriting the beginning of the file.

Table 8.1: Trailer Dictionary Keys — ISO 32000-2:2020 Section 7.5.5, Table 15

Key	Type	Description
`/Size`	integer	Total number of entries in the cross-reference table (including object 0). One greater than the highest object number used.
`/Root`	indirect reference	The document catalog — root of the entire logical document tree.
`/Info`	indirect reference	Document information dictionary. Optional and deprecated in PDF 2.0 (replaced by XMP metadata), but still universally present in real-world files.
`/ID`	array of two byte strings	File identifiers. The first element is assigned at creation and never changes; the second is updated on each save. Signatures depend on `/ID` — PAdES profiles strongly recommend it.
`/Prev`	integer	Byte offset of the *previous* cross-reference section. This is how incremental updates chain together (Chapter 8).
`/Encrypt`	dictionary or reference	Encryption dictionary. The signature value itself (`/Contents`) is never encrypted even in encrypted PDFs, but the surrounding content may be.

The `/ID` array is not cosmetic. Some signing tools use the first element as input to seed value dictionaries. Some validators check `/ID` consistency across incremental revisions. If a tool strips or regenerates `/ID` during processing, existing signatures may fail validation. We have seen this happen with document management systems that ”normalize” PDFs on upload.

8.2 PDF Object Types

Every value in a PDF file is one of exactly ten types, defined in ISO 32000-2:2020 Section 7.3. Understanding them is necessary for reading raw PDF structure, debugging signatures at the COS level, and working with PDFBox’s object model.

8.2.1 Boolean, Integer, Real, Null

Boolean — the keywords `true` or `false`. Used in flags like `/NeedAppearances true`.

Integer — a signed whole number with no decimal point: `42`, `-7`, `0`. The four values in a `/ByteRange` array are integers. Integers in PDF have no fixed bit width; implementations typically use 64-bit storage.

Real — a signed number with a decimal point: `3.14`, `-0.001`, `72.0`. Used in coordinate values, transformation matrices, and font metrics.

Null — the keyword `null`. A dictionary key mapped to `null` is equivalent to the key being absent. Null references are resolved as ”object not found.”

8.2.2 String (Literal and Hexadecimal)

PDF has two string representations:

Literal string — parentheses-delimited: `(Hello World)`. Backslash escapes handle special characters: `\(`, `\)`, `\\`, `\n`, `\r`, `\t`, and octal escapes like `\101` (the letter A). Unbalanced parentheses must be escaped; balanced ones need not be.

Hexadecimal string — angle-bracket-delimited hex pairs: `<48656C6C6F>`. Each pair of hex digits represents one byte. The `/Contents` value in a signature dictionary is always a hex string because it contains raw binary CMS data. The hex string is right-padded with zeros to fill the space reserved during signing.

8.2.3 Name

A slash-prefixed identifier: `/Type`, `/Page`, `/Sig`, `/ByteRange`. Names are atoms, not strings — they serve as dictionary keys and enumeration values. Names containing characters outside the printable ASCII range use a `#xx` hex escape: `/F#23oo` represents the name `F#oo`.

8.2.4 Array

An ordered collection enclosed in square brackets: `[0 14000 30408 14592]`. Arrays can contain any mix of types, including other arrays and dictionaries. The `/ByteRange` value is an array of four integers.

8.2.5 Dictionary

Key-value pairs enclosed in double angle brackets:

```
<< /Type /Sig /Filter /Adobe.PPKLite /SubFilter /adbe.pkcs7.detached >>
```

Keys are always names. Values can be any type. Dictionaries are the workhorse of PDF — every page, every font, every annotation, every signature is a dictionary or contains one.

8.2.6 Stream

A dictionary followed by raw binary data, delimited by `stream` and `endstream`:

```
8 0 obj
<< /Type /XObject /Subtype /Image /Length 12345 /Filter /FlateDecode >>
stream
...binary data (12345 bytes, possibly compressed)...
endstream
endobj
```

Streams are the only mechanism for storing bulk data: page drawing instructions, images, embedded font programs, cross-reference data, and CMap tables. The `/Length` key must be exact — if it is wrong, the parser reads too few or too many bytes, corrupting everything that follows.

A wrong `/Length` is one of the most common forms of PDF corruption from buggy generators. When debugging a corrupt PDF, always compare the `/Length` value against the actual byte count between `stream` and `endstream`. QPDF's `--check` flag catches this automatically.

8.2.7 Indirect Object and Indirect Reference

An **indirect object** wraps any value with an object number and generation number: `12 0 obj … endobj`. An **indirect reference** is a pointer to such an object: `12 0 R`. The cross-reference table resolves these: given `12 0 R`, the parser looks up object 12, generation 0, in the xref table, seeks to the recorded byte offset, and reads the object.

Indirect references are the glue that holds a PDF together. The catalog references the page tree, the page tree references individual pages, pages reference content streams and annotations, annotations reference appearance streams — all through indirect references.

8.3 Cross-Reference Streams

PDF 1.5 introduced **cross-reference streams** — an alternative to the classic ASCII xref table that stores the same data in a compressed binary stream object.

8.3.1 Why They Exist

The classic xref table has two problems. First, the 20-bytes-per-entry format is wasteful: a PDF with 100,000 objects needs a 2 MB xref table of pure ASCII text. Cross-reference streams compress this data 5-10x using standard stream filters (typically FlateDecode). Second, the

classic format has no entry type for objects stored inside object streams (covered next); cross-reference streams introduce a Type 2 entry for exactly this purpose.

8.3.2 Structure

```
42 0 obj
<<
  /Type /XRef
  /Size 100
  /W [1 3 1]
  /Index [0 100]
  /Root 1 0 R
  /Info 5 0 R
  /ID [<abc123> <abc123>]
  /Filter /FlateDecode
  /Length 487
>>
stream
...compressed binary data...
endstream
endobj
```

The stream dictionary serves double duty: it contains both the stream parameters (`/Filter`, `/Length`) and the trailer parameters (`/Root`, `/Info`, `/ID`, `/Prev`, `/Size`). There is no separate `trailer` keyword when cross-reference streams are used.

The `/W` array specifies field widths in bytes for each entry. `/W [1 3 1]` means: 1 byte for the type field, 3 bytes for field 2, 1 byte for field 3 — 5 bytes total per entry.

Table 8.2: Cross-Reference Stream Entry Types — ISO 32000-2:2020 Section 7.5.8, Table 18

Type	Field 2 Meaning	Field 3 Meaning
0	Next free object number	Generation number of the free entry
1	Byte offset of the object in the file	Generation number
2	Object number of the containing object stream	Index of this object within that object stream

Type 2 is the key innovation. It says ”this object is not a standalone indirect object in the file body; it is packed inside object stream N, at position M.” Classic xref tables cannot express this.

The `/Index` array specifies which object numbers are covered. `/Index [0 100]` means entries for objects 0 through 99. A cross-reference stream can have multiple subsections: `/Index [0 50 200 10]` covers objects 0-49 and 200-209.

Some PDF libraries produce files that start with a classic xref table but switch to cross-reference streams in incremental updates (or vice versa). This is legal per the specification, but it trips up parsers that expect consistency. PDFBox handles the transition transparently; not all tools do.

8.4 Object Streams

PDF 1.5 also introduced **object streams**: multiple indirect objects packed into a single compressed stream, eliminating the repeated `N 0 obj`...`endobj` framing overhead.

```
10 0 obj
<< /Type /ObjStm /N 5 /First 32 /Filter /FlateDecode /Length 418 >>
stream
...compressed data containing 5 objects...
endstream
endobj
```

The `/N` key gives the count of packed objects; `/First` gives the byte offset within the decompressed data where the first object's value begins (the preceding bytes contain a table of object numbers and offsets).

Not everything can live in an object stream. ISO 32000-2:2020 Section 7.5.7 excludes: stream objects (they have their own `stream`/`endstream` framing), the encryption dictionary, objects with non-zero generation numbers, and the cross-reference stream itself. Signature dictionaries *can* technically live in object streams (they are dictionaries, not streams), but signing tools almost never place them there because it would complicate byte-range calculation.

If you need to find a signature dictionary in a raw PDF, do not search for `/Type /Sig` in the file bytes — it might be compressed inside an object stream. Use `qpdf --show-object=N` where N is the object number. QPDF decompresses object streams transparently.

8.5 Hex Dump Walkthrough: A Minimal Valid PDF

Let us trace through a complete, minimal PDF file, annotating every section:

```
%PDF-1.7
%âãÏÓ

1 0 obj
<< /Type /Catalog /Pages 2 0 R >>
endobj

2 0 obj
<< /Type /Pages /Kids [3 0 R] /Count 1 >>
endobj

3 0 obj
<< /Type /Page /Parent 2 0 R /MediaBox [0 0 612 792]
   /Contents 4 0 R
   /Resources << /Font << /F1 5 0 R >> >> >>
endobj

4 0 obj
<< /Length 44 >>
stream
```

```
BT /F1 24 Tf 100 700 Td (Hello) Tj ET
endstream
endobj

5 0 obj
<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica >>
endobj

xref
0 6
0000000000 65535 f
0000000015 00000 n
0000000066 00000 n
0000000121 00000 n
0000000286 00000 n
0000000384 00000 n

trailer
<< /Size 6 /Root 1 0 R >>
startxref
453
%%EOF
```

- Header: PDF version 1.7
- Binary hint: four bytes above 0x80, forcing binary classification
- Catalog: root of the document tree; `/Pages` points to the page tree root
- Pages node: page tree root with one child page
- Page: defines the media size (US Letter), references a content stream and font resources
- Content stream: drawing instructions — Begin Text, set font F1 at 24pt, move to (100,700), show ”Hello”, End Text
- Font: a reference to standard Helvetica (not embedded — viewers provide a substitute)
- Cross-reference table: six entries mapping object numbers to byte offsets
- Trailer: `/Size 6` (six xref entries), `/Root` points to the catalog
- `startxref`: byte offset of the `xref` keyword
- `%%EOF`: end-of-file marker

A parser reads this *backward*: find `%%EOF`, find `startxref`, jump to byte 453, parse the xref table, read the trailer, follow `/Root` to object 1, and navigate from there.

This minimal PDF omits a `/Info` dictionary, `/ID` array, and font embedding. Production PDFs have dozens to hundreds of objects, but the structural skeleton is identical.

8.6 How Different Generators Produce Different Structures

Not all PDFs are created equal at the byte level, and the differences matter when you are computing byte ranges for signatures.

Adobe Acrobat produces linearized files by default, with objects reordered for progressive display and a hint table at the beginning. It uses cross-reference streams in modern versions.

Apache PDFBox (3.0.0) writes objects in memory-allocation order, uses classic xref tables by default, and does not linearize. Its incremental save mode (`saveIncremental()`) produces clean append-only updates.

iText / OpenPDF frequently uses cross-reference streams even for PDF 1.7 files, and its object ordering varies by version.

LibreOffice produces classic xref tables with object ordering that appears semi-random (it reflects internal processing order).

Microsoft Word (Save as PDF) produces well-structured files with cross-reference streams and object streams, heavily compressed.

QPDF can transform between any of these representations: linearize, de-linearize, convert xref tables to streams or vice versa, unpack object streams, and normalize whitespace. It is the Swiss Army knife of PDF structural debugging.

When comparing two versions of a signed PDF at the structural level:

```
# Show all xref entries and their origins
qpdf --show-xref document.pdf

# Convert to JSON for semantic comparison
qpdf --json document-v1.pdf > /tmp/v1.json
qpdf --json document-v2.pdf > /tmp/v2.json
diff /tmp/v1.json /tmp/v2.json

# Decompress everything for human-readable byte-level comparison
qpdf --qdf --object-streams=disable input.pdf readable.pdf
```

8.7 Why File Structure Matters for Signatures

We have spent this chapter on raw PDF anatomy for a specific reason: **PDF digital signatures operate at the byte level**, not the logical-object level.

The `/ByteRange` array in a signature dictionary specifies exactly which bytes in the physical file are hashed and signed. Verification reads those exact bytes, computes a cryptographic hash, and compares it against the hash embedded in the CMS `SignedData` structure. If a single byte differs — because an object was renumbered, a cross-reference entry was reformatted, whitespace was normalized, or a stream was recompressed — the hash will not match, and the signature is invalid.

This has concrete consequences:

Re-saving breaks signatures. Loading a signed PDF into memory and writing it back out — even without changing any logical content — rewrites the file from scratch: new object ordering, new byte offsets, new xref entries. The bytes change. The signature dies. Signed PDFs must be saved incrementally (Chapter 8).

Linearization affects byte layout. Linearizing a PDF after signing rearranges objects for progressive display. The bytes move. The signature breaks. Always linearize *before* signing, never after.

Cross-reference format affects layout. Whether a file uses classic xref tables or xref streams changes the physical bytes in the file. A signature computed against one format will

not validate if the format changes, even when the logical content is identical.

Object streams affect findability. If a signature dictionary is packed inside an object stream, byte-level tools that search for `/Type /Sig` in the raw file will miss it. Always use proper PDF parsing, never raw byte scanning, when searching for signature-related objects.

8.8 PDFBox COS Layer: Mapping PDF Types to Java

PDFBox's COS (Carousel Object System) layer provides a one-to-one mapping from PDF types to Java classes:

Table 8.3: PDFBox COS Type Mapping

PDF Type	PDFBox Class	Notes
Boolean	`COSBoolean`	Singleton `TRUE`/`FALSE`
Integer	`COSInteger`	Wraps `long`; common values cached
Real	`COSFloat`	Wraps `float`
String	`COSString`	Both literal and hex; `setForceHexForm(true)` for hex output
Name	`COSName`	Interned: `COSName.TYPE`, `COSName.SIG`, custom via `COSName.getPDFName("ByteRange")`
Null	`COSNull`	Singleton `COSNull.NULL`
Array	`COSArray`	Ordered list of `COSBase`
Dictionary	`COSDictionary`	`getItem()` returns raw value; `getDictionaryObject()` dereferences indirect refs
Stream	`COSStream`	Extends `COSDictionary`; `createInputStream()` for decoded data
Indirect ref	`COSObject`	Wraps `COSBase` with object number and generation

The distinction between `getItem()` and `getDictionaryObject()` is critical when working with signatures. `getItem()` returns the raw value — if the entry is an indirect reference, you get the `COSObject` wrapper (which carries the object number). `getDictionaryObject()` follows the reference and returns the target dictionary. Use `getDictionaryObject()` when you need the content; use `getItem()` when you need the object number (e.g., to correlate with xref byte offsets).

Here is how to traverse the file structure from trailer to signature dictionaries using PDFBox 3.0.0:

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.cos.COSArray;
import org.apache.pdfbox.cos.COSDictionary;
import org.apache.pdfbox.cos.COSName;
```

```
import org.apache.pdfbox.cos.COSString;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.interactive.form.PDAcroForm;
import org.apache.pdfbox.pdmodel.interactive.form.PDField;
import org.apache.pdfbox.pdmodel.interactive.form.PDSignatureField;

try (PDDocument doc = Loader.loadPDF(pdfBytes)) {
    // Inspect the trailer
    COSDictionary trailer = doc.getDocument().getTrailer();
    COSArray id = trailer.getCOSArray(COSName.ID);
    log.info("File ID present: {}", id != null);
    log.info("Trailer /Size: {}", trailer.getInt(COSName.SIZE));

    // Navigate to signatures through the AcroForm
    PDAcroForm acroForm = doc.getDocumentCatalog().getAcroForm();
    if (acroForm == null) {
        log.info("No AcroForm -- no form fields or signatures");
        return;
    }

    for (PDField field : acroForm.getFieldTree()) {
        if (field instanceof PDSignatureField sigField) {
            COSDictionary sigDict = sigField.getCOSObject();
            COSDictionary sigValue = sigDict.getCOSDictionary(COSName.V);

            if (sigValue != null) {
                COSName subFilter = sigValue.getCOSName(COSName.SUB_FILTER);
                COSArray byteRange = sigValue.getCOSArray(
                    COSName.getPDFName("ByteRange"));
                COSString contents = (COSString) sigValue.getDictionaryObject(
                    COSName.CONTENTS);

                log.info("Signature '{}': SubFilter={}, ByteRange={}",
                    field.getFullyQualifiedName(), subFilter, byteRange);
                log.info("  /Contents hex length: {} bytes",
                    contents.getBytes().length);
            }
        }
    }
}
```

8.9 Common Structural Problems and Debugging

8.9.1 Incorrect /Length Values

A stream's `/Length` does not match its actual byte count. This corrupts parsing downstream and can shift xref offsets, making byte-range verification fail. QPDF catches this:

```
qpdf --check suspect.pdf
```

8.9.2 Missing /ID Array

Some generators omit `/ID` entirely. While technically optional, its absence can trip up validators that check it as part of integrity verification. Use the PDFBox code above to detect missing `/ID` programmatically.

8.9.3 Multiple %%EOF Markers

A PDF with incremental updates (Chapter 8) will have multiple `%%EOF` markers. This is normal for signed documents. But some web-based PDF processors strip everything after the first `%%EOF`, destroying all incremental updates and all signatures.

8.9.4 Trailing Bytes After Final %%EOF

Bytes after the last `%%EOF` are not part of the PDF. For signed documents, the `/ByteRange` of the outermost signature must cover every byte up to the end of the file. Trailing bytes outside the byte range create a validation gap that the Bochum ISA attacks (Chapter 24) exploited.

8.9.5 Cross-Reference Table Corruption

Byte offsets in the xref table do not match actual object positions. This happens when a tool modifies the body without rebuilding the xref, or when binary transfer corrupts bytes.

QPDF can repair broken xref tables, but repair involves a full rewrite, which invalidates all signatures. Use `--check` for diagnosis without modification. For signed documents, repair is not an option — you need the original uncorrupted file.

8.10 What Comes Next

We have now dissected the complete physical structure of a PDF file: header, body, cross-reference table (in both classic ASCII and compressed stream formats), object streams, trailer, and `%%EOF`. We have mapped every PDF object type to its PDFBox COS class, walked through a minimal file byte by byte, and connected the physical structure to the requirements of digital signatures.

Chapter 8 builds directly on this foundation with **incremental updates** — the append-only mechanism that makes PDF signatures possible in the first place. If this chapter was about the structure of a single snapshot, Chapter 8 is about how those snapshots stack on top of each other, forming the layered history that signatures depend on to survive.

Chapter 9

Incremental Updates

9.1 What Incremental Updates Are

A PDF file, at its simplest, has four parts: a header (`%PDF-1.x` or `%PDF-2.0`), a body of numbered objects, a cross-reference table (or cross-reference stream) that maps object numbers to byte offsets, and a trailer that points to the root object and the cross-reference table. The file ends with the `%%EOF` marker.

Chapter 7 covered this structure in detail. What Chapter 7 did not cover is what happens when you *modify* a PDF.

There are exactly two ways to save changes to a PDF file:

1. **Full rewrite** — rebuild the entire file from scratch, writing a new body, a new cross-reference table, and a new trailer. The old file is replaced entirely.
2. **Incremental update** (also called **incremental save** or **append mode**) — leave the original file untouched and *append* new data to the end. The new data contains only the objects that changed (or were added), a new cross-reference section mapping those objects, and a new trailer. The file now has two `%%EOF` markers: the original one (buried in the middle of the file) and the new one at the end.

ISO 32000-2:2020 Section 7.5.6 defines incremental updates:

> The updated PDF file shall contain the original data of the PDF file plus appended update data. Updates may be appended to a file multiple times. The update data added each time a file is updated shall contain only those objects that have been added or modified, along with a cross-reference section and trailer for those objects.
>
> — ISO 32000-2:2020 Section 7.5.6

The key phrase is ”the original data of the PDF file” — the original bytes are preserved, byte for byte, untouched. This is not a diff or a delta. It is raw appending.

9.2 Why Incremental Updates Exist

Incremental updates solve three distinct problems, and understanding all three explains why the mechanism is designed the way it is.

9.2.1 Problem 1: Performance on Large Documents

A 500-page PDF with embedded fonts and high-resolution images might be 200 MB. If a user fills in a single form field — changing perhaps 50 bytes of actual content — a full rewrite

would require reading, processing, and writing all 200 MB. An incremental update appends a few kilobytes.

This was especially important in the 1990s when the format was designed. Disk I/O was slow, memory was scarce, and rewriting a large file for a minor change was unacceptable. Even today, incremental saves are noticeably faster for large documents.

9.2.2 Problem 2: Preserving Previous State

When a PDF is incrementally updated, the previous revision is still there — it is the bytes before the appended data. This means you can ”undo” an incremental update by truncating the file at the previous `%%EOF` boundary, recovering the exact document as it existed before the change.

This property is not a side effect. It is a design goal. ISO 32000-2:2020 Section 7.5.6 explicitly states that conforming readers should be able to retrieve previous revisions of a document by following the chain of `/Prev` pointers in the trailers.

For workflows where audit trails matter — legal documents, contracts, government forms — this built-in version history is valuable. You do not need an external version control system. The PDF file *is* the version history.

9.2.3 Problem 3: Digital Signatures

This is the reason we are here, and it deserves its own section (Why Incremental Updates Are Critical for Signatures). The short version: a digital signature covers a specific byte range. If you rewrite the file, the bytes change, and the signature becomes invalid. Incremental updates allow modifications *after* signing without disturbing the signed bytes.

Problem 3 is not optional. If you are building any tool that edits PDFs which may contain signatures — even if your tool has nothing to do with signing — you must support incremental saves. A full rewrite of a signed PDF destroys every signature in it. There is no workaround, no clever trick, no way to reassemble the pieces. The signed bytes are gone.

9.3 Anatomy of an Incremental Update

Let us walk through exactly what gets appended during an incremental update. Suppose we start with a simple PDF containing three objects: a catalog (object 1), a page (object 2), and a page content stream (object 3).

9.3.1 The Original File

```
%PDF-1.7
1 0 obj
<< /Type /Catalog /Pages 2 0 R >>
endobj

2 0 obj
<< /Type /Page /Contents 3 0 R /MediaBox [0 0 612 792] >>
endobj
```

```
3 0 obj
<< /Length 44 >>
stream
BT /F1 12 Tf 100 700 Td (Hello) Tj ET
endstream
endobj

xref
0 4
0000000000 65535 f
0000000009 00000 n
0000000066 00000 n
0000000148 00000 n

trailer
<< /Size 4 /Root 1 0 R >>

startxref
252
%%EOF
```

- Three objects in the body
- Cross-reference table mapping object numbers to byte offsets
- Trailer with `/Size` (total object count) and `/Root` (catalog reference)
- `startxref` gives the byte offset of the xref table
- End-of-file marker

Nothing surprising here. This is the standard PDF structure from Chapter 7. Let us say this file is 350 bytes long.

9.3.2 Appending an Incremental Update

Now suppose we want to change the text on the page from ”Hello” to ”Hello World”. We need a new version of object 3. Instead of rewriting the entire file, we append:

```
3 0 obj
<< /Length 50 >>
stream
BT /F1 12 Tf 100 700 Td (Hello World) Tj ET
endstream
endobj

xref
0 1
0000000000 65535 f
3 1
0000000351 00000 n

trailer
```

```
<< /Size 4 /Root 1 0 R /Prev 252 >>

startxref
460
%%EOF
```

- The new version of object 3, with updated content
- A new cross-reference section — but only for changed objects
- The xref entry for object 3 now points to byte offset 351 (the start of the new object 3)
- The new trailer includes a `/Prev` key pointing to the *previous* xref table (at byte 252)
- `startxref` points to the new xref section
- A second `%%EOF` marker

The file now contains *two* copies of object 3: the original at byte offset 148 and the new one at byte offset 351. The original cross-reference table (at byte 252) still points to the old version. The new cross-reference section (at byte 460) points to the new version.

A conforming reader starts from the end of the file, finds the last `startxref`, reads the new xref section, and sees that object 3 is at byte 351. For objects 1 and 2, the new xref section has no entries — so the reader follows the `/Prev` pointer to the previous xref table and finds them there.

The result: the reader sees the updated document. But the old bytes are still present, untouched, in the first 350 bytes of the file.

9.3.3 The `/Prev` Chain

The `/Prev` key in each trailer forms a linked list of cross-reference sections, ordered from newest to oldest. A reader reconstructing the full object map starts with the newest xref section, then walks the `/Prev` chain, adding entries for any object numbers it has not yet seen. The first entry wins — newer revisions override older ones.

For a PDF that has been incrementally updated three times, the chain looks like this:

```
%%EOF  (revision 3 — current)
  startxref -> xref_3
  trailer_3: /Prev -> xref_2

%%EOF  (revision 2)
  startxref -> xref_2
  trailer_2: /Prev -> xref_1

%%EOF  (revision 1 — original)
  startxref -> xref_1
  trailer_1: (no /Prev — this is the first revision)
```

Each revision's trailer may also update the `/Root` reference if the catalog object was modified. In practice, the `/Root` often stays the same — the catalog object number does not change, but the catalog *object* might be replaced with a new version that references new page trees or updated AcroForm dictionaries.

Counting `%%EOF` markers tells you the number of revisions. A freshly created PDF has one

`%%EOF`. After one incremental save, it has two. After n incremental saves, it has $n + 1$... unless some tool has done a full rewrite at some point, collapsing all revisions back to one. A signed PDF with two signatures typically has three `%%EOF` markers: the original document, the first signature, and the second signature.

9.4 How Objects Are Overridden

The override mechanism is simple: same object number, new xref entry.

When a conforming reader encounters object `3 0` in both the new xref section and a previous one, it uses the entry from the newest section. The old object still exists in the file — it occupies bytes, it can be found by parsing — but it is logically superseded.

This has important implications:

- **No object is ever deleted.** You can add an xref entry marking an object as ”free” (generation number incremented, next free object pointer updated), but the bytes of the old object remain in the file. Deleting content from a PDF does not remove it from the file — it merely hides it behind a newer xref section.
- **Object numbers are stable.** Object 3 is always object 3, whether you are looking at revision 1 or revision 5. The *content* of object 3 may differ across revisions, but the number does not change. This is critical for signatures because the `/ByteRange` and cross-reference tables use byte offsets and object numbers as their addressing scheme.
- **Generation numbers exist but are rarely used.** The spec allows incrementing the generation number when an object is freed and reused, but in practice, most tools reuse generation 0 for everything. Do not write code that assumes generation numbers will be non-zero — you will almost never see them in the wild.

Some tools claim to ”optimize” PDFs by removing unused objects and rebuilding the xref table. This is a full rewrite. If the PDF contains signatures, every signature is destroyed. Tools that do this include some versions of Ghostscript’s `pdfwrite` device, QPDF’s `--linearize` option (when not combined with `--preserve-unreferenced-objects`), and various online ”PDF optimizer” services. If you are processing signed PDFs in an automated pipeline, test your tools carefully.

9.5 Cross-Reference Streams and Incremental Updates

PDF 1.5 introduced cross-reference streams as an alternative to the traditional ASCII cross-reference table. Instead of a plaintext table, the cross-reference data is stored as a compressed binary stream object. The trailer dictionary is merged into the stream dictionary — there is no separate `trailer` keyword.

Incremental updates work identically with cross-reference streams. The appended data contains a new cross-reference stream object with entries for modified objects, and the stream dictionary includes a `/Prev` key pointing to the previous cross-reference stream’s byte offset.

```
15 0 obj
<< /Type /XRef
   /Size 16
```

```
   /W [1 3 1]
   /Root 1 0 R
   /Prev 8432
   /Filter /FlateDecode
   /Length 87 >>
stream
...compressed binary xref data...
endstream
endobj

startxref
9150
%%EOF
```

- The cross-reference stream is itself a PDF object (object 15 in this case)
- `/W` defines the width of each field in the binary data: 1 byte for type, 3 bytes for offset, 1 byte for generation
- `/Prev` points to the previous cross-reference stream, just as in the classic trailer
- `startxref` points to this stream object

The `/Prev` chain works exactly the same way. A reader walks the chain, building the complete object map from newest to oldest, using the first entry encountered for each object number.

A PDF file can mix classic xref tables and cross-reference streams across revisions. The original file might use a classic table, and an incremental update might append a cross-reference stream (or vice versa). A conforming reader must handle both. PDFBox handles this transparently — `Loader.loadPDF()` follows the `/Prev` chain regardless of whether each link is a table or a stream.

9.6 Why Incremental Updates Are Critical for Signatures

This is the most important section in this chapter. Everything else — the performance benefits, the revision history, the object override mechanics — is useful context. But for digital signatures, incremental updates are not a convenience. They are a *requirement*.

9.6.1 The Fundamental Constraint

A PDF digital signature covers a specific byte range (see Chapter 13 for the complete deep dive). The `/ByteRange` array in the signature dictionary specifies exactly which bytes in the file are hashed and signed. For the signature to remain valid, those bytes must not change. Not a single one.

If you do a full rewrite of a signed PDF — reorganizing objects, recomputing byte offsets, rebuilding the cross-reference table — the bytes at the signed offsets will be different. The hash will not match. The signature is dead.

An incremental update avoids this problem by *not touching the original bytes*. The signed region (which covers everything from the start of the file through the `/Contents` gap and out to the `%%EOF` of the revision that was signed) remains intact. New content is appended *after* the signed region.

9.6.2 Signing Is an Incremental Update

When PDFBox (or any compliant library) signs a PDF, it performs an incremental update. The signing operation appends:

- A new or updated signature field object (with the `/V` reference pointing to the signature dictionary)
- The signature dictionary object itself (containing `/ByteRange`, `/Contents`, `/Filter`, `/SubFilter`, and the rest)
- An updated AcroForm dictionary (with the new field added to `/Fields` and `/SigFlags` set)
- A new cross-reference section mapping these new/updated objects
- A new trailer with `/Prev` pointing to the previous revision
- A new `%%EOF`

The signature's byte range covers everything from byte 0 to the end of this new `%%EOF` — *including* the bytes of all previous revisions. The signature literally covers the entire file, minus the gap where the CMS blob sits.

9.6.3 Multiple Signatures

Now consider what happens when a second person signs the same PDF. The second signature is another incremental update, appended after the first. The second signature's byte range starts at byte 0 and extends to the end of *its* `%%EOF` — which means it covers the first signature's bytes too.

The first signature's byte range covers bytes up to the first signature's `%%EOF`. The second signature's appended data falls *after* that range. As far as the first signature is concerned, those bytes do not exist — they are outside the range it covers, and that is fine.

This is how multiple signatures coexist:

```
|  Original document          |  Revision 1 (%%EOF)
|  + First signature append   |  Revision 2 (%%EOF)
|  + Second signature append  |  Revision 3 (%%EOF)

First signature's byte range:  [0 .. end of revision 2]   (minus its /Contents gap)
Second signature's byte range: [0 .. end of revision 3]   (minus its /Contents gap)
```

The first signature validates against revisions 1 and 2. The second signature validates against revisions 1, 2, and 3. Neither signature invalidates the other.

This only works because each signature is appended incrementally. If the second signer's tool did a full rewrite instead of an incremental save, the first signature's bytes would be destroyed, and both signatures would be invalid. This is why `/SigFlags` bit 2 (AppendOnly) exists — it tells editors "you must use incremental save, or you will break things."

9.6.4 Form Filling After Signing

The same principle applies to form filling after a certification signature with `/DocMDP` permission level 2 (which allows form filling and signing but no other changes). When a user fills in a form field, the PDF viewer should save the change as an incremental update. The filled

field value is a new object appended to the file, and the certification signature remains valid because its byte range is untouched.

If the viewer does a full save instead, the certification signature breaks. The viewer has just violated the signer's explicit instruction (via `/DocMDP`) that the document should only be modified incrementally.

9.7 What Goes Wrong: Flattening and Full Rewrites

Now that we understand why incremental updates matter, let us catalog the ways they get destroyed.

9.7.1 The Flattening Problem

"Flattening" a PDF typically means merging all incremental updates into a single revision — rewriting the file with one body, one cross-reference table, one trailer, and one `%%EOF`. Some tools call this "optimizing," "linearizing," "compressing," or just "saving."

The result is a smaller, cleaner file. It is also a file in which every digital signature is invalid.

Common tools that flatten by default or by option:

- **Ghostscript** (`gs -sDEVICE=pdfwrite`) — always does a full rewrite; there is no incremental mode
- **QPDF** with `--linearize` — reorganizes the file for progressive loading, destroying incremental history
- **Some online PDF tools** — "compress PDF" or "optimize PDF" services typically do a full rewrite
- **LibreOffice** (depending on version and configuration) — older versions did not always use incremental save, even when `/SigFlags` bit 2 was set
- **Adobe Acrobat** "Save As" vs "Save" — "Save" uses incremental update; "Save As" does a full rewrite (destroying signatures unless the user re-signs)

If you are building an automated PDF processing pipeline and signed PDFs will pass through it, you must audit every tool in the pipeline to ensure it uses incremental saves (or does not modify the file at all). A single non-incremental save anywhere in the pipeline will invalidate every signature. This is the number one cause of "signatures were valid when I sent the PDF but invalid when the recipient opened it."

9.7.2 The "Save" vs "Save As" Trap

Adobe Acrobat's behavior is the one most users encounter:

- **File** > **Save** performs an incremental update. Signatures are preserved.
- **File** > **Save As** writes a new file from scratch. Every signature in the document is destroyed.

Acrobat shows a warning dialog when you attempt Save As on a signed document, but many users click through warnings without reading them. Other PDF editors may not even warn.

When training users who work with signed PDFs, the single most important instruction is: ”Never use Save As. Always use Save.” This one piece of advice prevents more signature breakage than any amount of technical configuration.

9.8 Extracting a Specific Revision

Because incremental updates are purely additive — new bytes appended after the previous `%%EOF` — extracting a previous revision is trivial: truncate the file at the appropriate `%%EOF` boundary.

If a PDF has three `%%EOF` markers (three revisions), you can extract:

- **Revision 1**: all bytes from the start of the file up to and including the first `%%EOF`
- **Revision 2**: all bytes from the start up to and including the second `%%EOF`
- **Revision 3**: the entire file (up to and including the third `%%EOF`)

This is not a diff extraction — each ”revision” is a complete, self-contained PDF file that a reader can open and render. Revision 1 is the original document. Revision 2 is the original plus the first set of changes. And so on.

9.8.1 Extracting with PDFBox

PDFBox does not provide a built-in ”extract revision N” method, but the approach is straightforward: find the N-th `%%EOF` marker and copy bytes from 0 to that position.

```
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;

/**
 * Extracts individual revisions from a PDF with incremental updates.
 * Each revision is a complete, valid PDF file.
 */
public class RevisionExtractor {

    private static final byte[] EOF_MARKER = "%%EOF".getBytes(StandardCharsets.US_ASCII);

    /**
     * Find byte offsets of all %%EOF markers in the PDF.
     */
    public static List<Integer> findEofOffsets(byte[] pdfBytes) {
        List<Integer> offsets = new ArrayList<>();
        for (int i = 0; i <= pdfBytes.length - EOF_MARKER.length; i++) {
            boolean match = true;
            for (int j = 0; j < EOF_MARKER.length; j++) {
                if (pdfBytes[i + j] != EOF_MARKER[j]) {
                    match = false;
                    break;
                }
            }
            if (match) {
```

```
                // Include the newline after %%EOF if present
                int end = i + EOF_MARKER.length;
                if (end < pdfBytes.length && pdfBytes[end] == '\r') end++;
                if (end < pdfBytes.length && pdfBytes[end] == '\n') end++;
                offsets.add(end);
            }
        }
        return offsets;
    }

    /**
     * Extract revision N (1-based) as a complete PDF byte array.
     */
    public static byte[] extractRevision(byte[] pdfBytes, int revision) {
        List<Integer> eofOffsets = findEofOffsets(pdfBytes);
        if (revision < 1 || revision > eofOffsets.size()) {
            throw new IllegalArgumentException(
                "Revision " + revision + " out of range. "
                + "File has " + eofOffsets.size() + " revisions.");
        }
        int endOffset = eofOffsets.get(revision - 1);
        byte[] result = new byte[endOffset];
        System.arraycopy(pdfBytes, 0, result, 0, endOffset);
        return result;
    }
}
```

This approach works for classic incremental updates. It does *not* handle linearized PDFs correctly (see Linearized PDFs and Incremental Updates), because linearized files have a `%%EOF` at the end of the first part that is not a revision boundary. For production code, you should also handle the case where `%%EOF` appears inside a stream object (rare but technically possible in pathological files).

9.8.2 Extracting with QPDF

QPDF provides a more robust approach from the command line:

```
# Show all cross-reference sections and their offsets
qpdf --show-xref input.pdf

# Extract a specific page range (not exactly revision extraction,
# but useful for isolating content)
qpdf input.pdf --pages . 1-5 -- output.pdf
```

QPDF does not have a built-in ”extract revision N” command, but `--show-xref` gives you all the information you need to determine the `%%EOF` boundaries manually.

9.9 How PDFBox Handles Incremental Saves

PDFBox 3.0.0 provides two distinct save methods, and choosing the wrong one is a common source of signature breakage.

9.9.1 `doc.save()` — Full Rewrite

The standard `save()` method writes the entire document from scratch:

```
try (PDDocument doc = Loader.loadPDF(inputBytes)) {
    // Make changes...
    doc.save(outputStream);  // Full rewrite — destroys signatures!
}
```

This produces a clean, single-revision PDF. All incremental update history is lost. All signatures are invalid.

Use `save()` only when:

- The document has no signatures and you do not need to preserve revision history
- You are creating a new document from scratch
- You explicitly want to flatten the file (and accept the consequences)

9.9.2 `doc.saveIncremental()` — Append Mode

The `saveIncremental()` method appends only the changed objects:

```
try (PDDocument doc = Loader.loadPDF(inputBytes)) {
    // Make changes...
    doc.saveIncremental(outputStream);  // Appends — preserves signatures
}
```

This is what you must use when the document contains signatures. PDFBox tracks which objects have been modified (via the COSUpdateInfo mechanism) and writes only those objects, plus a new cross-reference section and trailer, to the output.

`saveIncremental()` requires that the document was loaded from a stream or byte array — PDFBox needs the original bytes to copy them to the output before appending. If you loaded the document from a `File` and then try to write incrementally to a different output stream, the original bytes are read from the file. If you try to overwrite the same file, you will corrupt it. Always load from bytes and write to a new output.

9.9.3 The `saveIncremental()` and Signing Workflow

When signing a PDF, you do not call `saveIncremental()` directly. Instead, you use the signing API:

```
try (PDDocument doc = Loader.loadPDF(inputBytes)) {
    PDSignature signature = new PDSignature();
    signature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
    signature.setSubFilter(PDSignature.SUBFILTER_ADBE_PKCS7_DETACHED);
    signature.setSignDate(Calendar.getInstance());

    SignatureOptions options = new SignatureOptions();
    options.setPreferredSignatureSize(16384);  // Reserve 16KB for CMS

    doc.addSignature(signature, signingInterface, options);

    ByteArrayOutputStream baos = new ByteArrayOutputStream();
    doc.saveIncremental(baos);
```

```
    byte[] signedPdf = baos.toByteArray();
}
```

- `saveIncremental()` here does three things: writes the original bytes, appends the new objects (including the signature dictionary with a placeholder `/Contents`), computes the hash over the byte ranges, calls your `SignatureInterface.sign()` method to get the CMS bytes, and writes them into the placeholder.

The signing operation is *always* an incremental save. PDFBox enforces this — there is no `doc.save()` variant that supports signing, because signing with a full rewrite is logically impossible (the signed byte range would cover a file that no longer exists).

9.10 Multiple `%%EOF` Markers: What They Mean

Every `%%EOF` marker in a PDF represents the end of a revision. Counting them tells you the document's history:

- **1 `%%EOF`**: A single-revision PDF. Never been incrementally updated. Either freshly created, or previously incrementally updated but later flattened by a full rewrite.
- **2 `%%EOF`**: One incremental update after the original. Common for a single signature, a single form fill, or a single annotation addition.
- **3 `%%EOF`**: Two incremental updates. Common for a document with two signatures, or one signature plus a subsequent form fill.
- **N `%%EOF`**: N-1 incremental updates after the original. Documents that have been through complex workflows (multiple signers, multiple form fill rounds, annotation rounds) can have many revisions.

A quick way to count revisions in a PDF is: `grep -c '%%EOF' document.pdf`. This works for most files, but be aware that `%%EOF` could theoretically appear inside a stream object (unlikely in practice). For reliable counting, parse the `/Prev` chain instead.

9.10.1 What Revision Counts Tell You About a Signed PDF

For signature validation, the revision count carries diagnostic information:

- A signed PDF with exactly 2 `%%EOF` markers is the simplest case: original document + one signature. The signature covers everything in revision 1 and 2.
- A signed PDF with 3 `%%EOF` markers might mean: original + signature + post-signature modification (form fill, annotation, second signature). The first signature covers revisions 1-2. Whatever is in revision 3 was added after signing — whether it is permissible depends on the `/TransformMethod` and `/DocMDP` settings.
- A signed PDF with only 1 `%%EOF` is suspicious. Either the signature was created during the initial save (some tools do this), or the file was flattened after signing (which means the signature is almost certainly invalid, even if the bytes were carefully reconstructed — see the discussion in Chapter 24 about the Incremental Saving Attack).

9.11 Tracing the Revision History

To reconstruct the complete revision history of a PDF, you walk the `/Prev` chain from the newest trailer back to the oldest:

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.cos.COSDictionary;
import org.apache.pdfbox.cos.COSName;
import org.apache.pdfbox.pdmodel.PDDocument;

import java.util.ArrayList;
import java.util.List;

/**
 * Traces the /Prev chain to enumerate all cross-reference sections.
 */
public class RevisionTracer {

    public static List<Long> traceXrefOffsets(byte[] pdfBytes) throws Exception {
        List<Long> offsets = new ArrayList<>();
        try (PDDocument doc = Loader.loadPDF(pdfBytes)) {
            COSDictionary trailer = doc.getDocument().getTrailer();
            // The last startxref value
            long currentOffset = doc.getDocument().getStartXref();
            offsets.add(currentOffset);

            // Walk the /Prev chain
            while (trailer.containsKey(COSName.PREV)) {
                long prev = trailer.getLong(COSName.PREV);
                offsets.add(prev);
                // Note: to get the previous trailer, we would need to
                // parse the xref section at the /Prev offset.
                // PDFBox's internal parser does this, but the API
                // does not expose individual revision trailers directly.
                break; // Simplified — full implementation needs xref parsing
            }
        }
        return offsets;
    }
}
```

PDFBox's public API does not expose a convenient "iterate over all revisions" method. The internal `COSParser` follows the `/Prev` chain during loading, but the individual revision trailers are not directly accessible after parsing. For production revision analysis, consider using QPDF's `--show-xref` output (see Debugging Incremental Updates) or writing a custom parser that reads the raw bytes and follows the `/Prev` pointers manually.

9.12 Linearized PDFs and Incremental Updates

Linearized PDFs (sometimes called "optimized for fast web view") have a special structure designed for progressive loading over HTTP. A linearized PDF reorganizes its objects so that the first page can be displayed before the entire file has been downloaded.

The linearization adds a **linearization dictionary** (object 1, typically) and restructures the file into two parts separated by an internal `%%EOF` — but this internal `%%EOF` is *not* a revision boundary. It is part of the linearization structure.

This creates a compatibility problem with incremental updates:

- A linearized PDF already has an unusual structure with cross-reference data at both the beginning and end of the file.
- An incremental update appends data after the final `%%EOF`, which is fine for non-linearized PDFs but technically invalidates the linearization (because the linearization dictionary's `/L` key records the total file length, which is now wrong).
- Some viewers re-linearize on save, which is a full rewrite, which destroys signatures.

ISO 32000-2:2020 Section 7.5.6 states:

> If an incremental update is made to a linearized PDF file, the result is a valid non-linearized PDF file.

In other words, an incremental update on a linearized PDF destroys the linearization but produces a valid PDF. This is acceptable — the file simply loses its "fast web view" optimization.

If your workflow requires both linearization and digital signatures, you must linearize *before* signing, not after. Linearizing after signing is a full rewrite that destroys signatures. Signing after linearizing produces a valid signed PDF that is no longer linearized. There is no way to have a linearized, signed PDF that remains linearized after signing — the incremental update required by signing inherently breaks the linearization structure.

9.13 The Incremental Saving Attack

The incremental update mechanism that makes PDF signatures possible is also the mechanism that attackers have exploited to *bypass* those signatures.

The **Incremental Saving Attack (ISA)**, described by the Ruhr University Bochum team in their CCS 2019 paper, works as follows:

1. The attacker starts with a legitimately signed PDF
2. The attacker appends a new incremental update to the file — adding, modifying, or replacing objects (pages, text, images, form field values)
3. The attacker does *not* touch the signed bytes; the incremental update is appended after the signed region
4. The attacker presents the modified file to a PDF viewer
5. The viewer reads the file from the end, uses the newest cross-reference section, and renders the attacker's content
6. The viewer validates the signature by checking the original byte range — which is untouched — and reports "Signature is valid"

The signature *is* valid — over the bytes it covers. But the viewer is displaying content from *outside* the signed byte range. The user sees a green checkmark and attacker-controlled content.

The defense against ISA is conceptually simple but difficult to implement correctly: a viewer

must detect that content has been appended after the signed revision and warn the user. Specifically, the viewer must check whether the byte range of the last signature covers the entire file. If the file extends beyond the end of the signed byte range, there are unsigned changes.

Whether those changes are permissible (e.g., a subsequent signature, an allowed form fill under DocMDP level 2) or malicious requires deeper analysis of what the appended objects actually do — which is where the complexity lies.

Chapter 24 covers the ISA and its variants in complete technical detail, including the specific techniques attackers used to bypass even the ”unsigned content detected” warnings that some viewers implemented as an initial defense.

9.14 Debugging Incremental Updates

When something goes wrong with a signed PDF — the signature shows as invalid, a form field value disappeared, a viewer crashes — the first thing to check is the incremental update structure.

9.14.1 QPDF: Your Best Friend

QPDF is an indispensable tool for PDF debugging. For incremental update analysis, the key commands are:

```
# Show all cross-reference entries with their types and offsets
qpdf --show-xref signed-document.pdf

# Show the structure of the file (objects, streams, references)
qpdf --show-object=trailer signed-document.pdf

# Check the file for structural errors
qpdf --check signed-document.pdf

# Show a specific object
qpdf --show-object=42,0 signed-document.pdf
```

The `--show-xref` output is particularly valuable. It shows every object number, its byte offset, and which cross-reference section it belongs to. From this output, you can determine:

- How many cross-reference sections exist (i.e., how many revisions)
- Which objects were added or modified in each revision
- Whether the xref entries are consistent (offsets point to actual objects)

9.14.2 Comparing Revisions

To understand what changed between revisions, extract each revision (by truncating at `%%EOF` boundaries as described earlier) and compare them:

```
# Extract revision 1 (original document)
# Suppose the first %%EOF is at byte 45230
head -c 45230 signed-document.pdf > revision1.pdf

# Extract revision 2 (after first signature)
```

```
# Suppose the second %%EOF is at byte 67890
head -c 67890 signed-document.pdf > revision2.pdf

# Compare the objects in each revision
qpdf --show-xref revision1.pdf > xref1.txt
qpdf --show-xref revision2.pdf > xref2.txt
diff xref1.txt xref2.txt
```

The diff will show you exactly which objects were added or modified in the incremental update. For a signature, you should see the signature dictionary object, the updated AcroForm, and possibly an updated catalog or page annotation list.

9.14.3 Hex-Level Inspection

For the most stubborn problems, there is no substitute for looking at the raw bytes. Open the PDF in a hex editor and search for `%%EOF`. Each occurrence marks a revision boundary. The bytes between two consecutive `%%EOF` markers (plus the preceding `startxref` and trailer) constitute one incremental update.

A debugging checklist for incremental update problems:

1. Count `%%EOF` markers — does the number match the expected number of revisions?
2. Check each trailer's `/Prev` value — does the chain form a consistent linked list?
3. Verify that `startxref` in each revision points to a valid xref section
4. For signed PDFs: does the byte range of each signature end at a `%%EOF` boundary?
5. Are there any bytes after the last `%%EOF`? (There should not be — trailing garbage can confuse some parsers.)
6. Open each extracted revision independently — does it render correctly?
7. Compare the object sets across revisions — do the changes make sense for the expected workflow?

9.15 Real-World Scenarios

The theory of incremental updates is clean. The practice is messy. Here are the scenarios you will encounter most often.

9.15.1 Scenario 1: Single Signature

The simplest case. A document is created, then signed. The file has two revisions (two `%%EOF` markers). The signature's byte range covers the entire file. No complications.

9.15.2 Scenario 2: Two Approval Signatures

Alice signs first, then Bob. The file has three revisions: original, Alice's signature, Bob's signature. Alice's byte range covers revisions 1-2. Bob's byte range covers revisions 1-3. When validating Alice's signature, a viewer should check the byte range of Alice's signature and warn if changes between revision 2 and 3 go beyond what Alice's `/TransformMethod` permits (typically, adding more signatures is allowed).

9.15.3 Scenario 3: Certification + Approval

The document author applies a certification signature with `/DocMDP` permission level 2 (form fill and sign allowed). Later, a user fills form fields and applies an approval signature. The file has three or four revisions (original, certification, form fill, approval — or the form fill and approval might be in the same incremental update). The certification signature permits the form fill and the additional signature. A viewer must verify that the changes in the later revisions are within the scope of the DocMDP permissions.

9.15.4 Scenario 4: Annotations After Signing

A user adds a sticky note annotation to a signed PDF. If the editor saves incrementally, a new revision is appended with the annotation object and an updated page annotations array. The original signature's byte range is untouched. Whether the annotation is *allowed* depends on the signature's `/TransformMethod`:

- DocMDP level 1 (no changes allowed): the annotation violates the permissions. A conforming viewer should warn that the document has been modified in a way that exceeds the signer's permissions.
- DocMDP level 2 (form fill and sign only): annotations are not form filling, so this is a violation.
- DocMDP level 3 (annotations, form filling, and signing allowed): the annotation is permitted.

The incremental update itself is structurally fine in all three cases. The question is whether the *content* of the update is semantically permissible. This distinction — structural validity vs. semantic permissibility — is at the heart of the Incremental Saving Attack (Chapter 24).

9.15.5 Scenario 5: The Flattened Disaster

A signed PDF passes through an automated document management system that "optimizes" PDFs for storage. The optimization tool does a full rewrite. The output PDF has one `%%EOF`, no `/Prev` chain, and the signature dictionary's byte range references offsets that no longer correspond to the correct content. The signature is permanently invalid. The signed bytes are gone.

There is no recovery. The original file must be retrieved from backup (if it exists) or the document must be re-signed.

This scenario is *extremely* common in enterprise environments. Document management systems, email gateways, PDF/A conversion tools, and print spoolers all potentially rewrite PDFs. If your organization processes signed PDFs, audit your entire document pipeline for non-incremental saves. Every tool that touches a signed PDF must either leave it completely unmodified or use incremental saves exclusively.

9.15.6 Scenario 6: The Double-Linearization Trap

A document is created with linearization enabled (for fast web view), signed, and then processed by a tool that re-linearizes it. The re-linearization is a full rewrite. The signature is destroyed.

The fix: disable linearization in any tool that processes signed PDFs, or ensure that no post-signing processing step involves linearization.

9.16 Summary

Incremental updates are the mechanism that makes PDF digital signatures possible. Without them, signing a PDF would require that the document never be modified again — no additional signatures, no form filling, no annotations, nothing. With them, a signed PDF can evolve through a controlled sequence of revisions, each one adding new content without disturbing the bytes that previous signatures cover.

The rules are simple:

- A signed PDF must be saved incrementally. Always.
- The byte range of each signature covers a specific set of revisions.
- New content is appended after the signed byte range.
- The `/Prev` chain in the trailers links all revisions together.
- Extracting a specific revision means truncating at the appropriate `%%EOF` boundary.
- A full rewrite destroys all incremental history and all signatures.
- Linearization and incremental updates are incompatible after the first incremental save.

The next chapter (Chapter 9) examines AcroForm fields — the form infrastructure that signature fields are built on. Chapter 12 will bring incremental updates and signature architecture together, and Chapter 13 will dissect the byte range in complete detail. If the byte range is where signatures live or die, incremental updates are the ground they stand on.

Chapter 10

AcroForm & Interactive Forms

10.1 The AcroForm Dictionary

Every PDF that contains interactive form fields has an `/AcroForm` entry in the document catalog. This is the root of all form infrastructure in the document: the field list, the signature flags, the default appearance string, the default resources. If `/AcroForm` is missing from the catalog, the document has no interactive forms — period. Some tools create signature fields and forget to create the AcroForm entry, which leads to signatures that exist as orphaned objects in the file but are invisible to every conforming viewer.

The AcroForm dictionary lives at:

```
trailer -> /Root (Catalog) -> /AcroForm
```

Here is a representative AcroForm dictionary with all keys that matter for signature work:

```
<<
  /Fields [10 0 R 11 0 R 12 0 R]    %
  /SigFlags 3                         %
  /NeedAppearances false                %
  /DA (/Helv 0 Tf 0 g)                %
  /DR <<                                %
    /Font << /Helv 8 0 R /ZaDb 9 0 R >>
  >>
>>
```

- `/Fields` — array of references to the top-level field dictionaries. Every field in the document must be reachable from this array (directly or through parent-child relationships).
- `/SigFlags` — a bit field that controls signature-related behavior. Value 3 means both bit 1 (SignaturesExist) and bit 2 (AppendOnly) are set.
- `/NeedAppearances` — tells the viewer whether to regenerate appearance streams before rendering.
- `/DA` — default appearance string, used for fields that do not have their own `/DA`.
- `/DR` — default resources dictionary, providing fonts and other resources for appearance generation.

Let's examine each of these in the depth they deserve.

10.1.1 /Fields — The Root of the Field Tree

The `/Fields` array contains indirect references to the document's top-level field dictionaries. "Top-level" means fields that have no parent — they are the roots of potentially nested field trees. A flat form with ten independent fields will have ten entries in `/Fields`. A form with a

hierarchical namespace (like `address.city`, `address.zip`) will have a single `address` parent in `/Fields`, with `city` and `zip` as children.

A field that is not reachable from `/Fields` does not exist as far as a conforming reader is concerned. We have seen PDFs where a signing tool created a signature field object, set its `/V` to point at a signature dictionary, added a widget annotation to a page — and forgot to add the field to `/Fields`. The signature data is physically present in the file. The CMS bytes are valid. But no viewer will find it, because the AcroForm field tree does not reference it.

If you are debugging a ”missing signature” problem, check `/Fields` first.

10.1.2 /SigFlags — The Two Bits That Protect Everything

We cover `/SigFlags` in depth here because getting it wrong destroys signatures — silently, irreversibly, and usually long after the signing ceremony is over.

Table 10.1: SigFlags bit definitions — ISO 32000-2:2020 S12.7.2, Table 225

Name	Bit	What it does
`SignaturesExist`	1	Declares that the document contains at least one signature field with a value. Conforming readers use this as a signal to check for signatures and display validation status. Without it, some viewers will not even look for signatures.
`AppendOnly`	2	Mandates that any modifications to the document **shall** use incremental updates only. This is the single most important protection for existing signatures. If a viewer or editor does a full save (rewriting the entire file from scratch), every signature in the document becomes invalid — the bytes have changed, the byte ranges no longer match, and the cryptographic verification will fail.

The only correct value for `/SigFlags` in a signed document is **3** (both bits set). A value of 1 (SignaturesExist without AppendOnly) is technically allowed by the spec but is a bug in practice — it tells viewers that signatures exist but does not protect them from being destroyed by a full save.

Many PDF generation tools — including some versions of LibreOffice, older versions of iText, and various government form generators — set `/SigFlags` to 1 or omit it entirely. If you receive a signed PDF with `/SigFlags` missing or set to 1, the signatures are at risk. Any editor that performs a full save will destroy them.

AppendOnly matters because incremental updates are what make multi-signature workflows possible — each signer appends their changes without moving the bytes that previous signers' byte ranges reference. Without AppendOnly, an editor may do a full rewrite, destroying every signature.

We have also seen the converse: `/SigFlags` is 0 but signed fields exist with valid CMS data. Acrobat shows ”No Signatures” because bit 1 was never set, so it never looked.

The AppendOnly flag is advisory, not enforced by the file format. A malicious or poorly written tool can ignore it. The flag protects against *accidental* destruction by well-behaved editors. Intentional tampering is what the cryptographic signature itself guards against.

10.1.3 /NeedAppearances — The Flag That Breaks Signatures

The `/NeedAppearances` flag (boolean, default `false`) tells the viewer to regenerate appearance streams before rendering. For ordinary fields this is a convenience — skip generating appearances and let the viewer handle it. For signature fields, it is a disaster.

When a viewer regenerates appearances, it modifies the document bytes. If the document is signed, the signature becomes invalid. It does not matter that the viewer was ”just” fixing up visuals — the bytes changed.

If `/NeedAppearances` is `true` in a signed document, either: (1) the viewer regenerates appearances and the signature fails, or (2) the viewer skips regeneration and some fields show blank rectangles. **Never set `/NeedAppearances` to `true` in a document that will be signed.** Always generate complete appearance streams for every field before signing.

10.1.4 /DA — Default Appearance String

The `/DA` (Default Appearance) string is a PDF content stream fragment that specifies the default font, size, and color for variable text in form fields. It typically looks like:

```
/Helv 0 Tf 0 g
```

This means: use the font aliased as `/Helv`, at size 0 (auto-size), in black (`0 g`). The font name `/Helv` must be defined in the `/DR` (Default Resources) dictionary.

Why does this matter for signatures? Because if a signing tool creates a visible signature appearance that contains text — the signer’s name, the date, the reason — and the font referenced in the appearance stream is not available in `/DR` *and* not embedded in the appearance’s own `/Resources`, some viewers will fall back to `/DA` and render the text in Helvetica or not at all.

Always embed the font resources directly in the signature field’s appearance stream `/Resources` dictionary. Do not rely on `/DR`. The appearance stream is a self-contained Form XObject and should carry everything it needs to render correctly, regardless of what the parent AcroForm provides.

10.1.5 /DR — Default Resources

The `/DR` dictionary provides fonts, color spaces, and other resources available to all form fields. The two fonts you will encounter in virtually every AcroForm document are `/Helv` (Helvetica, for text) and `/ZaDb` (ZapfDingbats, for checkbox ticks and radio button bullets).

The `/DR` fonts are *not* the same as page-level fonts. A page’s `/Resources` dictionary is independent. If a signature appearance stream references a font from the page’s resources but not from `/DR` or the appearance’s own resources, rendering may fail in viewers that strictly follow the spec.

10.2 Field Tree Hierarchy

10.2.1 Parent-Child Relationships and Fully Qualified Names

The AcroForm field tree is not flat. Fields can be nested using `/Parent` and `/Kids` keys. A parent field `<< /T (address) /Kids [20 0 R 21 0 R] >>` with a child field `<< /T (city) /Parent 19 0 R /FT /Tx >>` produces the fully qualified name `address.city`.

The fully qualified name is constructed by concatenating `/T` (partial name) values from root to leaf, separated by periods. This is the name you use in PDFBox's `acroForm.getField("address.city")`, in JavaScript, and in FDF data files.

Debugging field-not-found errors: If `acroForm.getField("Signature1")` returns null, check whether the field is nested. It might be `form.Signature1` or `page1.Signature1`. Use `acroForm.getFieldTree()` to iterate every field in the hierarchy and inspect the fully qualified names.

10.2.2 Terminal vs. Non-Terminal Fields

A **terminal field** is a field with no `/Kids`, or whose `/Kids` are all widget annotations (not other fields). Terminal fields have a `/FT` (field type) key — `/Tx`, `/Btn`, `/Ch`, or `/Sig`.

A **non-terminal field** is a pure grouping node. It has `/Kids` that are other fields, and it does *not* have `/FT`. Its only purpose is to provide a namespace prefix.

Some PDF generators create non-terminal fields that also carry `/FT`. This is technically a spec violation (ISO 32000-2:2020 S12.7.4.1), but it happens. PDFBox handles it gracefully in most cases, but other libraries may choke. If you are building a field tree, keep non-terminal nodes clean — no `/FT`, no `/V`, no `/Rect`.

10.3 Field Types

ISO 32000-2:2020 S12.7.4 defines four field types. All of them can appear in the same AcroForm, side by side, and a single document frequently contains all four — especially government and legal forms that combine text input, checkboxes, dropdown selections, and signature fields.

10.3.1 /Tx — Text Fields

Text fields hold user-entered text (value in `/V` as a text string). Key properties: `/MaxLen` (maximum character count), `/Q` (text alignment: 0=left, 1=center, 2=right). Text fields interact with signatures in one important way: if a text field is filled *after* a document is signed, the fill operation is an incremental update. Whether this invalidates the signature depends on the `/Lock` and `/FieldMDP` settings (covered in Chapter 14).

10.3.2 /Btn — Button Fields

Button fields come in three flavors, distinguished by the `/Ff` (field flags) bits: **pushbuttons** (bit 17 set, no persistent value), **checkboxes** (bit 17 clear, bit 16 clear, two-state toggles with values like `/Yes` or `/Off`), and **radio buttons** (bit 17 clear, bit 16 set, mutually exclusive

options). For signatures, button fields matter primarily in the context of form flattening and field locking.

10.3.3 /Ch — Choice Fields

Choice fields present a list of options: **combo boxes** (bit 18 set, a dropdown) and **list boxes** (bit 18 clear, a scrollable list). Options are defined in the `/Opt` array, where each entry can be a simple text string or a two-element array `[export_value, display_text]`.

10.3.4 /Sig — Signature Fields

Signature fields are where form infrastructure meets cryptographic infrastructure. A `/Sig` field is, structurally, just another form field in the AcroForm tree. But its `/V` (value) is not a text string or a name — it is a **signature dictionary** (`/Type /Sig`), which is a complex structure containing the byte range, the CMS data, the signing filter, timestamps, and more.

A signature field before signing:

```
<<
  /FT /Sig
  /T (Signature1)
  /V null                    % or /V absent entirely
  /Rect [100 600 300 650]
  /Subtype /Widget
  /P 5 0 R
>>
```

The same field after signing:

```
<<
  /FT /Sig
  /T (Signature1)
  /V 42 0 R                  % reference to the /Sig dictionary
  /Rect [100 600 300 650]
  /Subtype /Widget
  /P 5 0 R
  /AP << /N 43 0 R >>        % appearance stream added by the signing tool
>>
```

Object 42 is the signature dictionary (covered in depth in Chapter 12). Object 43 is the visual appearance of the signature.

The signature field is *not* the signature. The field is the **container**. The signature dictionary (the `/V` value) is the **content**. A document can have signature fields with no value — these are placeholder fields, pre-positioned by a template author, waiting for a signer to fill them in.

This distinction matters because:

- A document with an empty signature field still has an entry in `/Fields`.
- The `/SigFlags` should *not* have bit 1 set until at least one field actually has a value.
- Validation logic should check `/V`, not just the presence of `/FT /Sig` fields.

What makes `/Sig` fields unique: (1) the value is a complex dictionary, not a simple string; (2) signing computes a hash over the entire file, not just the field; (3) the `/SV` (Seed Value) and `/Lock` dictionaries allow the field to constrain which algorithms, certificates, and post-signing modifications are acceptable; and (4) the widget's `/Rect` determines visibility — a zero-size rect means an invisible signature.

10.4 Widget Annotations

10.4.1 The Connection Between Fields and Pages

A form field is an abstract data object (name, type, value). A widget annotation is its visual representation on a specific page (coordinates, size, appearance).

In the most common case, the field and widget are **merged into a single PDF object** — the same dictionary carries both `/FT` (field type) and `/Subtype /Widget`, both `/T` (field name) and `/Rect` (position). The spec (ISO 32000-2:2020 S12.5.6.19) also allows separating them: a field's `/Kids` array can contain widget annotations, each with `/Parent` pointing back to the field. This allows a single field to have widgets on multiple pages — unusual, but legal.

When debugging, check whether the field and widget are merged or separate. `PDSignatureField.getWidgets()` returns the list of widgets. An empty list means no visual representation — fine for invisible signatures, a bug if you expected a visible one.

10.4.2 Page Association

A widget must be associated with a page via two links: the widget's `/P` key referencing the page dictionary, and the page's `/Annots` array containing a reference to the widget. Both should be present. If the widget is missing from `/Annots`, it will not render — even if `/P` points to the correct page.

Some signing tools add the widget to the field tree but forget to add it to the page's `/Annots` array. The signature is valid cryptographically but invisible on every page. This is one of the most common "invisible signature" bugs that is not actually an invisible signature — it is a missing annotation reference.

10.5 Appearance Streams

10.5.1 Normal, Rollover, and Down

The `/AP` dictionary can define three appearance streams: `/N` (Normal — the only one that matters for signatures), `/R` (Rollover — mouse hover, rarely used), and `/D` (Down — click state, irrelevant for signatures).

Each can be a single Form XObject stream or a **subdictionary** mapping state names to streams. Checkboxes use a subdictionary (`<< /Yes 60 0 R /Off 61 0 R >>`). Signature fields always use a single stream — signatures do not have states.

10.5.2 Why Appearance Streams Matter for Signatures

If the appearance stream is missing, the viewer either renders a blank rectangle, tries to generate one on the fly (dangerous if the document is signed — see `/NeedAppearances` above), or shows a generic placeholder. For signed fields, the only correct approach is to have the signing tool generate a complete appearance stream before signing.

The appearance stream is included in the signed byte range. Modifying it after signing invalidates the signature. However, the appearance is *not* semantically bound to the CMS content — the appearance could say ”Signed by Alice” while the certificate belongs to Bob. The viewer must verify both independently.

10.5.3 Generating Appearances with PDFBox

PDFBox provides the `PDAcroForm.refreshAppearances()` method (and its per-field equivalent) to generate appearance streams for all fields. This is the equivalent of setting `/NeedAppearances` to `true` and having the viewer regenerate — except you do it programmatically, before signing, so the appearances are part of the signed bytes.

```
try (PDDocument doc = Loader.loadPDF(pdfBytes)) {
    PDAcroForm acroForm = doc.getDocumentCatalog().getAcroForm();
    if (acroForm != null) {
        acroForm.setNeedAppearances(false);  // clear the flag
        acroForm.refreshAppearances();        // generate all appearances
    }
}
```

`refreshAppearances()` will overwrite existing appearance streams. If you have custom signature appearances (hand-drawn images, company logos), call this *before* setting up your custom appearances, or call it only on non-signature fields. For signature appearances, most signing workflows build the appearance stream manually using a `PDFormXObject` — Chapter 15 covers this in detail.

10.6 XFA Forms: The Deprecated Complication

10.6.1 What XFA Was and Why It Was Deprecated

XFA (XML Forms Architecture) was Adobe’s attempt to replace the limited AcroForm system with a powerful XML-based form description language. An XFA form embeds XML data inside the PDF file (in the `/XFA` key of the AcroForm dictionary), and a viewer that supports XFA renders the form from XML rather than from the AcroForm field tree. XFA could do things AcroForm could not: dynamic layouts, complex validation rules, repeating sections, and data binding to external schemas.

ISO 32000-2:2020 deprecated XFA for three reasons: (1) only Adobe implemented it fully — interoperability was abysmal; (2) XFA and AcroForm coexist awkwardly in the same file, with the two representations frequently disagreeing about field values; and (3) XFA’s scripting capabilities have been exploited in multiple CVEs.

10.6.2 Why XFA Still Causes Problems

Deprecation does not mean disappearance. Millions of XFA-based PDF forms are still in circulation — government tax forms, insurance applications, healthcare records. When these documents are signed, the signature covers the entire file, including the XFA data.

The problems:

- **PDFBox does not support XFA rendering.** If you load an XFA document in PDFBox, you get the AcroForm fallback. The field values you see may differ from what an XFA-aware viewer shows. If you sign based on the AcroForm representation, the signature covers bytes that an XFA viewer interprets differently.
- **Field names may not match.** XFA and AcroForm can use different naming conventions. A signature field named `form1[0].#subform[0].SignatureField1[0]` in XFA might be `SignatureField1` in the AcroForm fallback.
- **Validation ambiguity.** If the XFA representation says one thing and the signed AcroForm says another, which one did the signer actually see and intend to sign?

If you encounter an XFA document in a signing workflow, the safest approach is to flatten the XFA to pure AcroForm (removing the `/XFA` key and ensuring all field values are represented in the AcroForm structure) *before* any signatures are applied. PDFBox's `PDAcroForm.flatten()` can help, but test thoroughly — XFA flattening is lossy by nature.

10.7 Form Flattening and Signatures

Form flattening converts interactive fields into static page content: field values are ”burned in” to page content streams, widget annotations are removed, and the AcroForm structure is deleted. The result is a static document that looks like the filled-in form but cannot be edited.

Flattening destroys every signature in the document. The signature fields themselves are removed (they are form fields), the byte ranges no longer point at the original bytes, and the AcroForm structure referencing the signatures is gone.

Never flatten a signed document. This sounds obvious, but it happens constantly in automated workflows — sign, then flatten ”for archival” or ”to prevent editing.” The result is a static PDF with no valid signatures. If you need to prevent editing after signing, use field locking (`/Lock`) and certification signatures (`/DocMDP`) — not flattening.

10.8 PDFBox AcroForm API

10.8.1 PDAcroForm: The Entry Point

In PDFBox 3.0.0, the AcroForm is accessed through the document catalog:

```
try (PDDocument doc = Loader.loadPDF(pdfBytes)) {
    PDAcroForm acroForm = doc.getDocumentCatalog().getAcroForm();
    if (acroForm == null) {
        return;  // No interactive forms in this document
    }
    for (PDField field : acroForm.getFields()) {
        log.debug("Field: {} (type: {}, fqn: {})",
```

```
            field.getPartialName(), field.getFieldType(),
            field.getFullyQualifiedName());
    }
}
```

`acroForm.getFields()` returns only top-level fields. To traverse the full tree including children, use `acroForm.getFieldTree()`, which returns a flat iterable over every field in the hierarchy.

10.8.2 Field Tree Traversal

`acroForm.getFieldTree()` returns a flat iterable over every field in the hierarchy — almost always what you want. For finding signature fields specifically:

```
for (PDField field : acroForm.getFieldTree()) {
    if (field instanceof PDSignatureField sigField) {
        log.info("Signature field: {} (signed: {})",
            sigField.getFullyQualifiedName(),
            sigField.getSignature() != null);
    }
}
```

If you need tree depth or explicit parent-child relationships, use `PDNonTerminalField.getChildren()` and recurse manually.

10.8.3 PDSignatureField and PDSignature

`PDSignatureField` is the PDFBox class representing a `/Sig` field. It provides access to the signature value (the `/V` dictionary) through `getSignature()`, which returns a `PDSignature` or `null` if the field is unsigned.

```
PDSignatureField sigField = (PDSignatureField) acroForm.getField("Signature1");
if (sigField == null) { return; }  // field not found -- check fully qualified name

PDSignature sig = sigField.getSignature();
if (sig == null) { return; }       // field exists but is unsigned

log.info("Filter: {}", sig.getFilter());       // e.g. "Adobe.PPKLite"
log.info("SubFilter: {}", sig.getSubFilter()); // e.g. "adbe.pkcs7.detached"
log.info("Name: {}", sig.getName());
log.info("Sign date: {}", sig.getSignDate());
```

`PDAcroForm.getField(String name)` uses the **fully qualified name**. If the field is nested (e.g., `signatures.approval`), you must pass the full path. Passing just `approval` will return `null`. This is one of the most common sources of ”field not found” bugs.

10.8.4 Creating Signature Fields Programmatically

Creating a signature field involves three steps: create the field, create the widget annotation, and wire them into the AcroForm and page structure.

```
// Ensure the AcroForm exists
PDAcroForm acroForm = doc.getDocumentCatalog().getAcroForm();
if (acroForm == null) {
    acroForm = new PDAcroForm(doc);
```

```
    doc.getDocumentCatalog().setAcroForm(acroForm);
}

// Create the signature field and configure its widget
PDSignatureField sigField = new PDSignatureField(acroForm);
sigField.setPartialName("Signature1");
PDAnnotationWidget widget = sigField.getWidgets().get(0);
PDPage targetPage = doc.getPage(0);
widget.setRectangle(new PDRectangle(100, 600, 200, 50));
widget.setPage(targetPage);

// Wire into AcroForm and page structure
targetPage.getAnnotations().add(widget);
List<PDField> fields = new ArrayList<>(acroForm.getFields());
fields.add(sigField);
acroForm.setFields(fields);
acroForm.getCOSObject().setInt(COSName.SIG_FLAGS, 3);
```

Common mistakes when creating signature fields:

1. **Forgetting to add the widget to the page's `/Annots` array.** The field exists in the AcroForm but is invisible.
2. **Forgetting to add the field to `/Fields`.** The widget is on the page but no viewer considers it a form field.
3. **Not setting `/SigFlags`.** The field is there, it gets signed, but editors may do a full save and destroy it.
4. **Using `new PDRectangle(x, y, w, h)` vs. raw array.** PDFBox's `PDRectangle` constructor takes (x, y, width, height), but the raw PDF `/Rect` array is `[llx lly urx ury]`. PDFBox handles the conversion, but if you're debugging raw COS dictionaries, remember the coordinate difference.

10.9 Common Form/Signature Interaction Bugs

Years of debugging have revealed a predictable set of bugs that occur at the intersection of AcroForm infrastructure and signature functionality. These are not edge cases — they are the everyday reality of working with signed PDF forms.

10.9.1 Missing Appearance Streams

Symptom: Signature validates, but the rectangle on the page is blank. **Cause:** No `/AP` entry on the widget, or `/NeedAppearances` is `true` and the viewer destroyed the original appearance during regeneration. **Fix:** Always generate a complete `PDAppearanceStream` during signing and set it as the normal appearance on the widget (see Chapter 15 for details).

10.9.2 Wrong Field Hierarchy

Symptom: PDFBox finds the field but Acrobat does not, or vice versa. **Cause:** Inconsistent parent-child chain — a child references a parent that does not list it in `/Kids`, or the fully qualified name computed by concatenating `/T` values does not match expectations. **Fix:** Validate the tree bidirectionally: every child's `/Parent` must list it in `/Kids`, and vice versa.

10.9.3 SigFlags Not Set or Wrong

Symptom: Signatures are valid, but a subsequent edit does a full save and destroys them all. **Cause:** `/SigFlags` is 0, 1, or absent. **Fix:** Set `/SigFlags` to 3 when creating the first signature field — before signing, not after.

10.9.4 Orphaned Signature Fields

Symptom: Signature panel shows a signature with no visual element on any page; or shows nothing despite signature data existing in the file. **Cause:** Widget is missing from a page's `/Annots` (variant 1), or the field is not referenced from `/Fields` (variant 2). **Fix:** Verify both paths: field AcroForm `/Fields`, and widget page `/Annots`.

10.9.5 Acrobat vs. PDFBox Rendering Differences

Symptom: A form that looks perfect in Acrobat renders differently in PDFBox (or vice versa).

Common causes: Acrobat aggressively substitutes missing fonts (PDFBox does not — it uses only what is embedded or available on the system). Acrobat regenerates appearances when `/NeedAppearances` is `true` (PDFBox requires an explicit `refreshAppearances()` call). Acrobat is lenient about malformed COS dictionaries (a field with `/FT` on a non-terminal node may work in Acrobat but cause unexpected behavior in PDFBox). And Acrobat interprets `/DA` strings flexibly — font size 0 means "auto-size," missing color operators are silently defaulted — while PDFBox parses more strictly.

When debugging cross-viewer differences, compare the raw COS structure. In PDFBox: `field.getCOSObject().toString()`. In Acrobat: Preflight tool or JavaScript console (`this.getField("name").valueAsString`). The differences are almost always at the COS level.

10.9.6 Real-World Scenario: The Government Tax Form

A government tax authority distributes a PDF form designed in Adobe LiveCycle Designer. The form uses XFA internally with an AcroForm fallback. A citizen fills it in using Chrome's built-in PDF viewer (which uses the AcroForm fallback because it does not support XFA). The citizen's tax advisor opens the filled form in Acrobat (which *does* support XFA), sees different field values because the XFA data was never updated, and signs the document.

The signature is technically valid — it covers the bytes that were present at signing time. But the signed content includes XFA data that disagrees with the AcroForm data. There is no clean fix short of removing XFA from the document entirely. This is precisely why ISO 32000-2:2020 deprecated XFA, and why the deprecation matters for real-world signature workflows.

Chapter 11

PDF Coordinate System

The PDF coordinate system is the single most common source of signature placement bugs. It is conceptually simple — an origin at the bottom-left corner, units in points — but it collides head-on with how every screen coordinate system works, how page rotation transforms visual layout without changing coordinates, and how annotation rectangles encode position in a way that rewards precision and punishes guessing.

If you have ever placed a signature at what you thought was the bottom of a page and found it at the top, or created a perfectly valid signature that appeared as a mirror image on a rotated page, or spent an afternoon trying to figure out why your annotation rectangle produced a zero-height invisible box — this chapter is for you.

We will cover the coordinate system from first principles, work through every page box that ISO 32000-2:2020 defines, explain how page rotation interacts with coordinates, show how the Current Transformation Matrix works, and then tie it all back to signature placement with practical PDFBox code and debugging techniques.

11.1 User Space: The Default Coordinate System

Every PDF page has a **user space** coordinate system defined in ISO 32000-2:2020 Section 8.3.2. The rules are few but non-negotiable:

- The **origin** is at the **bottom-left** corner of the page.
- The **x-axis** increases to the **right**.
- The **y-axis** increases **upward**.
- One unit equals **1/72 of an inch** — which is exactly one **typographic point**.

That last property is not a coincidence. PDF was born as a page description language for printing, and the typographic point has been the standard unit of measure in the printing industry since the 18th century. When the specification says a page is 612 units wide and 792 units tall, that's 8.5 by 11 inches — US Letter size. A4 paper (the ISO 216 standard used nearly everywhere outside the US) is 595.28 by 841.89 points (210 by 297 millimetres).

The 1/72-inch definition is a convenient approximation. The traditional printer's point (the Didot point, the Pica point) varies slightly depending on which historical tradition you follow. PDF sidesteps this ambiguity by defining one user space unit as *exactly* 1/72 inch, period. ISO 32000-2:2020 Section 8.3.2.3 makes this explicit.

The bottom-left origin is where most coordinate confusion begins. Every windowing system you have used — HTML, Java Swing, JavaFX, Android, iOS, Win32 — places the origin at the **top-left** corner with the y-axis increasing **downward**. We are trained to think of y=0 as the top. In PDF, y=0 is the bottom.

This means that when you want to place a signature 50 points from the top of a US Letter page, the y-coordinate is not 50 — it is 792 minus 50 minus the height of the signature rectangle. We will return to this conversion formula repeatedly throughout this chapter because it is the root cause of the most common signature positioning bugs.

11.2 Page Boxes: The Five Rectangles

A PDF page does not have a single ”size.” It has up to five nested rectangles, each serving a different purpose. ISO 32000-2:2020 Section 14.11.2 defines them all. Understanding which box matters for signature placement — and which boxes most documents actually define — prevents an entire category of positioning errors.

11.2.1 MediaBox (Required)

The **MediaBox** defines the physical medium — the full extent of the page as it would be printed or rendered. It is the only required page box; every other box defaults to the MediaBox if absent.

For a US Letter page:

```
/MediaBox [0 0 612 792]
```

The array is `[llx lly urx ury]` — lower-left x, lower-left y, upper-right x, upper-right y. The MediaBox almost always starts at the origin `[0 0 …]`, but it does not have to. A MediaBox of `[100 100 712 892]` is perfectly legal and defines the same 612-by-792 point area, just shifted. Most tools normalize to zero, but when debugging coordinates, never assume.

11.2.2 CropBox

The **CropBox** defines the region to which the page contents are clipped when displayed or printed. Think of it as the visible window into the MediaBox. If absent, the CropBox defaults to the MediaBox.

A common use case: a document designed for A4 paper with crop marks. The MediaBox includes the crop marks (larger than the final page), and the CropBox clips to the actual content area.

For signature placement, the CropBox is the relevant boundary in most viewers. Adobe Acrobat positions annotations relative to the CropBox, not the MediaBox, when they differ.

If you compute signature coordinates against the MediaBox but the document has a smaller CropBox, your signature may appear to be shifted or partially off-screen. Always check `page.getCropBox()` in PDFBox and use its dimensions as the reference frame.

11.2.3 BleedBox

The **BleedBox** defines the region to which the page contents are clipped when output in a production (printing) environment. It is slightly larger than the TrimBox to allow for bleed — the area where ink extends beyond the final cut edge to avoid white margins from imprecise cutting.

The BleedBox is relevant for professional printing workflows and has no meaningful impact on digital signature placement. We mention it for completeness.

11.2.4 TrimBox

The **TrimBox** defines the intended final dimensions of the page after trimming. In a print-production workflow, the TrimBox is the ”real” page boundary — the finished product.

Like the BleedBox, the TrimBox is irrelevant for signature placement in digital workflows. However, if you are building a signing service that handles print-ready PDFs, be aware that the TrimBox may be significantly smaller than the MediaBox, and your users' mental model of ”the page” aligns with the TrimBox, not the MediaBox.

11.2.5 ArtBox

The **ArtBox** defines the extent of the page’s ”meaningful content” as intended by the author. It is rarely used in practice and has no impact on signature placement.

11.2.6 Which Box Matters for Signatures?

For digital signature placement, the decision tree is simple:

1. **Use the CropBox** as your reference frame. In PDFBox, `page.getCropBox()` returns the CropBox if defined, otherwise it falls back to the MediaBox. This is the correct default.
2. If you need to verify that your annotation rectangle falls within the visible area, check against the CropBox.
3. Never assume the CropBox starts at `[0 0 …]`. A CropBox of `[50 50 562 742]` shifts the visible origin.

PDFBox’s `PDPage.getCropBox()` handles the fallback-to-MediaBox logic automatically. You almost never need to call `getMediaBox()` directly for positioning purposes. The one exception is when you need to know the physical page dimensions for coordinate translation on pages with a non-zero CropBox origin.

11.3 Annotation Rectangles

Every annotation — including signature widget annotations — has a `/Rect` entry that specifies its position on the page. The `/Rect` is an array of four numbers:

```
/Rect [x1 y1 x2 y2]
```

Where `[x1 y1]` is the **lower-left** corner and `[x2 y2]` is the **upper-right** corner, both in user space coordinates relative to the page’s coordinate system.

For a signature that should appear 100 points from the left edge, 50 points from the bottom edge, and is 200 points wide by 60 points tall:

```
/Rect [100 50 300 110]
```

That gives us: lower-left at (100, 50), upper-right at (100+200, 50+60) = (300, 110).

The convention is lower-left to upper-right. If you accidentally swap these — providing upper-left to lower-right, for example — some viewers will silently normalize the rectangle, while others will render a zero-area or inverted annotation. PDFBox's `PDRectangle` constructor takes `(x, y, width, height)` where `(x, y)` is the lower-left corner, so it handles this correctly if you feed it the right values.

11.4 Why Signature Positioning Goes Wrong

Here is the bug that every developer writes exactly once. We want a signature at the visual top-right corner of a US Letter page. In screen coordinates (top-left origin), that would be approximately `(400, 30)` for a rectangle starting near the top-right. So we write:

```
PDRectangle rect = new PDRectangle(400, 30, 200, 50);
```

And the signature appears at the **bottom-right** of the page. Because y=30 in PDF user space is 30 points from the *bottom*, not 30 points from the top.

The correct conversion is:

```
float pageHeight = page.getCropBox().getHeight();
float desiredTopOffset = 30;    // 30 points from the visual top
float signatureHeight = 50;

float pdfY = pageHeight - desiredTopOffset - signatureHeight;
PDRectangle rect = new PDRectangle(400, pdfY, 200, signatureHeight);
```

The formula is: `pdfY = pageHeight - topOffset - rectangleHeight`.

This is the single most important formula in this chapter. Memorize it, put it in a utility method, and never compute it inline.

If your signing UI lets users click a position on a rendered page, the click coordinates are in screen space (origin top-left). You must invert the y-axis before creating the PDF annotation rectangle. Failing to do so produces signatures that appear to be vertically "mirrored" — a signature placed at the top appears at the bottom, and vice versa.

11.5 Page Rotation

The `/Rotate` key in a page dictionary specifies a rotation angle that viewers should apply when displaying the page. Legal values are 0, 90, 180, and 270 (degrees, clockwise).

Here is the critical insight that trips up even experienced developers: **page rotation does not change the coordinate system**. It changes how the viewer *displays* the coordinates, but the coordinates themselves — and the annotation rectangles — remain in the original, unrotated space.

Consider a US Letter page with `/Rotate 90`. The viewer displays it in landscape orientation: 792 points wide and 612 points tall. But the MediaBox is still `[0 0 612 792]` and the coordinate system still has its origin at the bottom-left corner of the *unrotated* page.

This means that if you compute an annotation rectangle for a 90-degree-rotated page using the visual (rotated) dimensions, your coordinates will be wrong. The annotation will appear

in a completely unexpected location because the viewer applies the rotation *after* positioning the annotation in the unrotated coordinate space.

11.5.1 Rotation and Signature Placement

To place a signature at a specific visual position on a rotated page, we must think in the *unrotated* coordinate space and account for the rotation.

For a page rotated 90 degrees clockwise:

- The visual width (what the user sees) corresponds to the unrotated height (792 for US Letter).
- The visual height corresponds to the unrotated width (612 for US Letter).
- A point at visual position (vx, vy) — where vy is measured from the visual top — maps to unrotated coordinates (vy_from_bottom, vx) approximately, depending on the rotation direction.

The general approach in PDFBox:

```
PDRectangle cropBox = page.getCropBox();
int rotation = page.getRotation();

float pageWidth, pageHeight;
if (rotation == 90 || rotation == 270) {
    // Visual dimensions are swapped
    pageWidth = cropBox.getHeight();
    pageHeight = cropBox.getWidth();
} else {
    pageWidth = cropBox.getWidth();
    pageHeight = cropBox.getHeight();
}

// Now compute visual position
float visualX = 400;                          // from left of visual page
float visualTopOffset = 30;                   // from top of visual page
float sigWidth = 200;
float sigHeight = 50;

// Convert to PDF coordinates depending on rotation
float pdfX, pdfY;
switch (rotation) {
    case 0:
        pdfX = cropBox.getLowerLeftX() + visualX;
        pdfY = cropBox.getUpperRightY() - visualTopOffset - sigHeight;
        break;
    case 90:
        pdfX = cropBox.getLowerLeftX() + visualTopOffset;
        pdfY = cropBox.getLowerLeftY() + visualX;
        // Width and height swap in the annotation rect
        float temp = sigWidth;
        sigWidth = sigHeight;
        sigHeight = temp;
        break;
    case 180:
```

```
            pdfX = cropBox.getUpperRightX() - visualX - sigWidth;
            pdfY = cropBox.getLowerLeftY() + visualTopOffset;
            break;
        case 270:
            pdfX = cropBox.getUpperRightX() - visualTopOffset - sigHeight;
            pdfY = cropBox.getUpperRightY() - visualX - sigWidth;
            temp = sigWidth;
            sigWidth = sigHeight;
            sigHeight = temp;
            break;
        default:
            throw new IllegalArgumentException("Invalid rotation: " + rotation);
    }

    PDRectangle signatureRect = new PDRectangle(pdfX, pdfY, sigWidth, sigHeight);
```

Wrap this coordinate transformation in a dedicated utility method and unit-test it for all four rotation values. This is the kind of code where an off-by-one error in a single case produces a bug that only manifests for rotated pages — which may be 5% of your input documents, ensuring the bug survives initial testing and surfaces in production.

11.6 Transformation Matrices and the CTM

ISO 32000-2:2020 Section 8.3.3 defines the **Current Transformation Matrix (CTM)** — a 3x3 affine transformation matrix that maps user space coordinates to device space coordinates. Every content stream operates under a CTM that controls translation, scaling, rotation, and skewing.

The CTM is represented as six numbers `[a b c d e f]`, corresponding to the matrix:

```
| a  b  0 |
| c  d  0 |
| e  f  1 |
```

For signature appearances, the CTM matters because the appearance stream (the Form XObject that draws the visual signature) has its own coordinate system. The `/BBox` of the appearance stream defines the drawing area, and the viewer maps this BBox into the annotation rectangle using a transformation derived from the rectangle dimensions and the BBox dimensions.

When the appearance stream's BBox does not match the annotation rectangle's aspect ratio, the viewer scales the appearance to fit — potentially distorting the signature image. If the BBox is `[0 0 200 50]` but the annotation rectangle is 200 by 100 points, the appearance will be stretched vertically by a factor of 2.

When creating signature appearances in PDFBox, always ensure the appearance stream's BBox matches the dimensions of the annotation rectangle. If you use `PDRectangle` for both and create them from the same width and height values, this happens automatically. The bug occurs when one is constructed independently of the other, or when the annotation rectangle is modified after the appearance is created.

Content streams use operators like `cm` (concatenate matrix) to modify the CTM:

```
1 0 0 1 100 200 cm    % translate by (100, 200)
```

```
0.5 0 0 0.5 0 0 cm    % scale by 50%
```

For signature placement, you rarely need to manipulate the CTM directly. PDFBox handles the coordinate mapping between the appearance stream and the annotation rectangle. But when debugging a signature that appears at the wrong size or position, checking the CTM in the appearance stream's content is a valuable diagnostic step.

11.7 Signature Appearance Rectangles and Page Coordinates

The annotation rectangle (`/Rect`) positions the signature on the page. The appearance stream's BBox defines the drawing canvas. The relationship between them determines how the visual signature is rendered.

Here is the complete chain:

1. The page has a coordinate system defined by its MediaBox (or CropBox).
2. The annotation's `/Rect` specifies where on the page the annotation sits, in page coordinates.
3. The appearance stream has a `/BBox` that defines its internal drawing area.
4. The viewer computes a transformation from the BBox to the Rect — scaling and translating the appearance stream content to fill the annotation rectangle.
5. If the page has a `/Rotate` value, the viewer rotates the entire visual output *after* all of the above.

This chain explains why a signature appearance that looks correct on one page can appear distorted on another: if the target page has different dimensions or a different rotation, and you reuse the same appearance stream without adjusting the annotation rectangle, the scaling transformation changes.

11.8 PDFBox Coordinate Handling

PDFBox provides `PDRectangle` as the primary class for working with rectangles in PDF user space. The key methods:

```
PDRectangle mediaBox = page.getMediaBox();   // Physical page extent
PDRectangle cropBox = page.getCropBox();     // Visible page extent (falls back to MediaBox)
int rotation = page.getRotation();           // 0, 90, 180, or 270

// PDRectangle stores lower-left corner + width/height
float llx = cropBox.getLowerLeftX();
float lly = cropBox.getLowerLeftY();
float urx = cropBox.getUpperRightX();
float ury = cropBox.getUpperRightY();
float width = cropBox.getWidth();
float height = cropBox.getHeight();
```

`PDRectangle` predefined constants include `PDRectangle.A4` (595.28 x 841.89) and `PDRectangle.LETTER` (612 x 792). These are useful for creating new pages but should never be used as the reference for positioning on an existing page — always query the actual page's CropBox.

11.8.1 Practical Example: Placing a Visible Signature

The following complete example places a visible signature rectangle at a specific visual position on a page, correctly handling rotation:

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.PDPage;
import org.apache.pdfbox.pdmodel.common.PDRectangle;

public class SignaturePositionCalculator {

    /**
     * Computes the PDF annotation rectangle for a signature at a given
     * visual position, accounting for page rotation and CropBox offset.
     *
     * @param page         the target page
     * @param visualX      x offset from the left of the visual page (points)
     * @param visualY      y offset from the top of the visual page (points)
     * @param visualWidth  desired visual width of the signature (points)
     * @param visualHeight desired visual height of the signature (points)
     * @return the annotation rectangle in PDF user space coordinates
     */
    public static PDRectangle computeSignatureRect(
            PDPage page,
            float visualX, float visualY,
            float visualWidth, float visualHeight) {

        PDRectangle crop = page.getCropBox();
        int rotation = page.getRotation();

        float pdfX, pdfY, pdfW, pdfH;

        switch (rotation) {
            case 0:
                pdfX = crop.getLowerLeftX() + visualX;
                pdfY = crop.getUpperRightY() - visualY - visualHeight;
                pdfW = visualWidth;
                pdfH = visualHeight;
                break;
            case 90:
                pdfX = crop.getLowerLeftX() + visualY;
                pdfY = crop.getLowerLeftY() + visualX;
                pdfW = visualHeight;
                pdfH = visualWidth;
                break;
            case 180:
                pdfX = crop.getUpperRightX() - visualX - visualWidth;
                pdfY = crop.getLowerLeftY() + visualY;
                pdfW = visualWidth;
                pdfH = visualHeight;
                break;
            case 270:
                pdfX = crop.getUpperRightX() - visualY - visualHeight;
                pdfY = crop.getUpperRightY() - visualX - visualWidth;
```

```
                    pdfW = visualHeight;
                    pdfH = visualWidth;
                    break;
                default:
                    throw new IllegalArgumentException(
                        "Unsupported rotation: " + rotation);
            }

            return new PDRectangle(pdfX, pdfY, pdfW, pdfH);
        }
    }
```

Call this with visual coordinates — ”200 points from the left, 50 points from the top, 180 wide, 40 tall” — and it returns the correct PDF rectangle regardless of rotation.

11.9 Common Coordinate Bugs

Over years of building PDF signing tools, we have seen the same bugs recur. Here is a catalog.

11.9.1 Flipped Signatures (Y-axis Inversion)

Symptom: The signature appears at the vertical opposite of where it should be. A signature placed near the top of the page appears near the bottom.

Cause: Using screen-space y-coordinates (origin top-left) directly as PDF y-coordinates (origin bottom-left) without inverting.

Fix: Apply the formula `pdfY = cropBox.getUpperRightY() - screenY - signatureHeight`.

11.9.2 Signatures on the Wrong Page Position (Rotated Pages)

Symptom: The signature appears in a completely unexpected location — not just vertically flipped, but seemingly random — and only on certain documents.

Cause: The page has a non-zero `/Rotate` value, and the coordinate calculation does not account for it. Many developers test exclusively with non-rotated pages and never encounter this until a customer submits a rotated document.

Fix: Always check `page.getRotation()` and apply the rotation-aware coordinate transformation shown above.

11.9.3 Zero-Area Annotation (Invisible Signature Becomes Truly Invisible)

Symptom: The signature field exists (it shows in the signature panel), but nothing is visible on the page.

Cause: The annotation rectangle has zero width or height, typically because the lower-left and upper-right corners are identical, or because a coordinate calculation produced a negative dimension that the viewer clamped to zero.

Fix: Always validate that the computed rectangle has positive width and height before creating the annotation. Log the rectangle values during development:

```
log.debug("Signature rect: x={}, y={}, w={}, h={}",
    rect.getLowerLeftX(), rect.getLowerLeftY(),
    rect.getWidth(), rect.getHeight());
```

11.9.4 Signature Appears Off-Page

Symptom: The signature field exists, the signature validates, but nothing is visible. Unlike the zero-area case, the rectangle has positive dimensions — it's just outside the visible CropBox.

Cause: Coordinates were computed against the MediaBox, but the CropBox is smaller. Or the CropBox has a non-zero origin that was not accounted for.

Fix: Always compute against `page.getCropBox()` and include the CropBox's lower-left offset in your calculations.

11.9.5 Distorted Signature Appearance

Symptom: The signature image or text appears stretched or squashed.

Cause: The appearance stream's BBox has a different aspect ratio than the annotation rectangle. The viewer scales the BBox to fit the Rect, distorting the content.

Fix: Ensure the appearance stream's BBox dimensions match the annotation rectangle's width and height.

11.10 Debugging Coordinates with QPDF and RUPS

When a signature appears in the wrong place, you need to inspect the raw PDF objects. Two tools are indispensable.

11.10.1 QPDF

QPDF (https://qpdf.sourceforge.io/) is a command-line tool that can dump every object in a PDF in a human-readable JSON format:

```
qpdf --json input.pdf > /tmp/structure.json
```

Then search for the annotation rectangle:

```
grep -A2 '"Rect"' /tmp/structure.json
```

You will see the raw `/Rect` array values. Compare them against the page's `/MediaBox` and `/CropBox` to verify they fall within the visible area.

QPDF can also show page-level dictionaries:

```
qpdf --show-object=3 input.pdf   # show object 3 (a page, for example)
```

11.10.2 RUPS (Reading and Updating PDF Syntax)

RUPS is a Java Swing application from the iText team that provides a tree view of every PDF object. It is invaluable for visual debugging because you can navigate the object tree, inspect annotation rectangles, and see the page boxes side by side.

To inspect a signature's coordinates in RUPS:

1. Open the PDF in RUPS.
2. Navigate to **Pages** → **Page N** → **Annots** (the annotations array).
3. Find the signature widget annotation.
4. Check the `/Rect` value — this is the annotation rectangle in page coordinates.
5. Go back to the page and check `/MediaBox`, `/CropBox`, and `/Rotate`.
6. Verify that the Rect falls within the CropBox and that you've accounted for the rotation.

RUPS also lets you inspect the appearance stream's BBox. Navigate into the annotation's `/AP` → `/N` and check the `/BBox` entry. If the BBox dimensions don't match the Rect dimensions, you've found your distortion bug.

11.10.3 PDFBox Programmatic Inspection

You can also inspect coordinates programmatically, which is useful in automated tests:

```
try (PDDocument doc = Loader.loadPDF(new File("signed.pdf"))) {
    for (PDPage page : doc.getPages()) {
        log.info("Page MediaBox: {}", page.getMediaBox());
        log.info("Page CropBox:  {}", page.getCropBox());
        log.info("Page Rotation: {}", page.getRotation());

        for (var annot : page.getAnnotations()) {
            if ("Sig".equals(annot.getSubtype())) {
                PDRectangle rect = annot.getRectangle();
                log.info("Sig annotation Rect: [{}, {}, {}, {}]",
                    rect.getLowerLeftX(), rect.getLowerLeftY(),
                    rect.getUpperRightX(), rect.getUpperRightY());
            }
        }
    }
}
```

When writing integration tests for signature placement, always test with at least four documents: one for each rotation value (0, 90, 180, 270). Also test with a document that has a CropBox smaller than its MediaBox. These five test cases catch the vast majority of coordinate bugs before they reach production.

11.11 Summary

The PDF coordinate system is a solved problem — the rules are clear and have not changed since PDF 1.0. The difficulty lies entirely in the mismatch between PDF's bottom-left origin and the top-left origin of every UI framework, compounded by page rotation's effect on visual layout.

For signature placement, remember these principles:

- Always use the CropBox (via `page.getCropBox()`) as your reference frame.
- Always invert the y-axis when converting from screen coordinates: `pdfY = cropBoxTop - screenY - height`.
- Always check `page.getRotation()` and transform coordinates accordingly.
- Always ensure the appearance stream BBox matches the annotation rectangle dimensions.

- Always test with rotated pages and non-standard CropBoxes.

These are not complex algorithms. They are simple arithmetic applied consistently. The bugs come from skipping a step, not from the math being hard.

Chapter 12

Font Embedding in PDF

Fonts are the invisible infrastructure of every PDF document. When they work, nobody notices. When they break, the document becomes unreadable — and if that document contains a digital signature, the signature appearance becomes a row of blank rectangles or missing-glyph squares that make your legally binding signature look like a rendering error.

This chapter explains how fonts work inside PDF files, how they get embedded (or not), what goes wrong when they don't, and how all of this matters specifically for digital signatures. We cover the font types defined by ISO 32000-2:2020, the encoding models, the mechanics of embedding and subsetting, and the practical reality of getting text to render correctly in signature appearance streams across different viewers.

If you have ever seen a signature block that shows `.notdef` squares where a signer's name should be, or a PDF where the signature text vanishes entirely on one viewer but renders fine on another, this chapter explains why.

12.1 Why Fonts Matter for Digital Signatures

A PDF digital signature has two independent representations: the **cryptographic** representation (the CMS `SignedData` structure, the byte range, the certificate chain) and the **visual** representation (the appearance stream that shows the signer's name, date, and reason on the page). Chapter 15 covers how appearance streams are structured; this chapter covers the font infrastructure that makes the *text* inside those appearances render correctly.

The visual appearance of a signature is a Form XObject — a miniature PDF page with its own resource dictionary, its own content stream, and its own font references. When that content stream contains a `Tj` or `TJ` operator to draw text, the PDF viewer must resolve the font reference, find the glyph outlines, and render them. If any step in that chain fails — the font reference points to a font that isn't embedded, the encoding doesn't map the character code to the right glyph, the font descriptor is missing, the font file stream is corrupt — the text doesn't render.

This matters more for signatures than for regular document text, for three reasons:

1. **Signature appearances are generated programmatically.** When a human creates a PDF in a word processor, the word processor embeds the fonts it uses. When your signing code creates a signature appearance, *your code* is responsible for font handling. If you get it wrong, nobody catches the error until a user reports that the signature looks broken.
2. **Signature appearances must survive across viewers.** A signature created on a server using PDFBox must render correctly in Adobe Acrobat, Foxit Reader, browser-based viewers using pdf.js, LibreOffice, and Preview.app. Each viewer has its own font fallback

behavior, its own interpretation of encoding, and its own handling of missing fonts. The only way to guarantee consistent rendering is to embed the font.

3. **Signature text often contains names with non-ASCII characters.** Signer names like ”Muller” are fine. Signer names like ”Muller” with an umlaut, ”Gonzalez” with an accent, or any East Asian, Arabic, or Cyrillic name will fail if the font and encoding aren’t set up correctly. In a global signing system, ”works for ASCII names” is not good enough.

12.2 Font Types in PDF

ISO 32000-2:2020 Section 9.6 defines several font types. The distinctions matter because each type has different embedding rules, different encoding models, and different failure modes.

12.2.1 Type 1 Fonts (PostScript Era)

Type 1 fonts are the original PostScript outline fonts, defined by Adobe in 1984. They use cubic Bezier curves for glyph outlines and a custom PostScript-based encoding scheme.

In PDF, a Type 1 font dictionary has `/Subtype /Type1` and references a font program (the glyph outlines) via the `/FontDescriptor` dictionary’s `/FontFile` entry. The font program is a stream containing the PostScript font data in PFB (Printer Font Binary) format.

Type 1 fonts are limited to a maximum of 256 glyphs per font instance — a consequence of the single-byte encoding model inherited from PostScript. This makes them unsuitable for CJK scripts, complex scripts like Arabic or Devanagari, or any language that needs more than 256 distinct glyphs.

Adobe officially ended support for Type 1 fonts in January 2023, removing the ability to author content with Type 1 fonts in their Creative Cloud applications. Existing PDFs with embedded Type 1 font programs still render correctly (the outlines are in the file), but this signals the industry’s long-term direction: TrueType and OpenType are the future.

12.2.2 The Standard 14 Fonts

PDF defines fourteen fonts that conforming readers are expected to have available without embedding. These are the **Standard 14 fonts**, specified in ISO 32000-2:2020 Section 9.6.2.2:

Table 12.1: The Standard 14 font families

Family	Regular / Bold / Italic / BoldItalic	Notes
Helvetica	Helvetica, Helvetica-Bold, Helvetica-Oblique, Helvetica-BoldOblique	Sans-serif. The default choice for most programmatic PDF generation.
Times	Times-Roman, Times-Bold, Times-Italic, Times-BoldItalic	Serif. Common in academic and legal documents.
Courier	Courier, Courier-Bold, Courier-Oblique, Courier-BoldOblique	Monospace. Used for code listings and form data.

Family	Regular / Bold / Italic / BoldItalic	Notes
Symbol	Symbol	Mathematical and Greek symbols.
ZapfDingbats	ZapfDingbats	Decorative symbols (checkmarks, arrows, etc.).

The critical word is ”expected.” ISO 32000-2:2020 Section 9.6.2.2 states that conforming readers *should* provide these fonts, but PDF 2.0 actually recommends embedding even the Standard 14 fonts to ensure consistent rendering. The practical reality is that most viewers do provide metrics-compatible substitutes for these fonts, but the substitutes are not identical — a document typeset with Helvetica on one viewer may render with Arial or Liberation Sans on another, causing slight differences in character widths, line breaks, and text positioning.

For signature appearances, these differences are usually cosmetic and acceptable. But if your signature appearance calculates text positions based on precise Helvetica metrics (e.g., centering text in the signature rectangle), a substitute font with slightly different widths will misalign the text.

For signature appearances, the Standard 14 fonts are ”good enough” for ASCII text. They are available everywhere, they don’t require embedding, and the rendering differences between substitute fonts are typically invisible at the font sizes used in signature blocks (8-12 points).

For non-ASCII text — accented characters, umlauts, CJK characters — the Standard 14 fonts are *not* sufficient, because their built-in encoding (WinAnsiEncoding) cannot represent characters outside the Windows-1252 code page. If your signers have names with characters outside that range, you must embed a TrueType or OpenType font with the appropriate glyph coverage.

12.2.2.1 PDFBox 3.x and the Standard 14 API Change

If you’ve migrated from PDFBox 2.x to 3.x, you’ve likely encountered this:

```
// PDFBox 2.x -- this no longer compiles in 3.x
PDFont font = PDType1Font.HELVETICA;
```

In PDFBox 2.x, the Standard 14 fonts were available as static constants on `PDType1Font`. PDFBox 3.x removed these constants and replaced them with an enum-based API:

```
// PDFBox 3.x -- the correct way
PDFont font = new PDType1Font(Standard14Fonts.FontName.HELVETICA);

// Other examples:
PDFont bold = new PDType1Font(Standard14Fonts.FontName.HELVETICA_BOLD);
PDFont courier = new PDType1Font(Standard14Fonts.FontName.COURIER);
PDFont times = new PDType1Font(Standard14Fonts.FontName.TIMES_ROMAN);
```

The reason for this change is that the old static constants were shared mutable objects — every caller that obtained `PDType1Font.HELVETICA` got the same instance, which could cause subtle threading and state pollution issues when multiple documents were processed concurrently. The new constructor creates a fresh instance each time, eliminating the shared-state problem.

If you search the internet for PDFBox font examples, the majority of results still show the 2.x API (`PDType1Font.HELVETICA`). These examples will not compile with PDFBox 3.0.0. Always verify that code examples match the major version of PDFBox you are using.

12.2.3 TrueType Fonts

TrueType fonts, developed jointly by Apple and Microsoft in the late 1980s as a competitor to Adobe's Type 1, use quadratic Bezier curves for glyph outlines. They are the most common font format in modern computing and the recommended choice for PDF font embedding.

In PDF, a TrueType font dictionary has `/Subtype /TrueType`. The font program is embedded via the `/FontDescriptor` dictionary's `/FontFile2` entry, which contains the raw TrueType font data (a `.ttf` file, essentially).

TrueType fonts in PDF can be either **simple** (single-byte encoding, max 256 glyphs per instance) or embedded as part of a **composite** font (Type 0 + CIDFont, described below) for full Unicode coverage. When used as a simple font, TrueType has the same 256-glyph limitation as Type 1. The composite font mechanism removes this limitation.

```
// Loading and embedding a TrueType font in PDFBox 3.x
PDFont font = PDType0Font.load(document, new File("/path/to/DejaVuSans.ttf"));

// The font is now embedded in the document.
// It will be automatically subsetted when the document is saved,
// including only the glyphs actually used.
```

`PDType0Font.load()` is the PDFBox 3.x method for loading TrueType and OpenType fonts. Despite the name suggesting Type 0, this method creates the full composite font structure (Type 0 wrapper + CIDFont descendant) needed for Unicode text support. The font file is embedded in the document, and PDFBox automatically subsets it on save.

12.2.4 Type 0 / CIDFont (Composite Fonts)

Type 0 fonts — also called composite fonts — are the mechanism PDF uses to support large character sets (CJK scripts, Arabic, Devanagari, and any script requiring more than 256 glyphs).

A Type 0 font consists of two linked dictionaries:

1. **The Type 0 font dictionary** (`/Subtype /Type0`): the "wrapper" that the content stream references. It contains a `/DescendantFonts` array pointing to the CIDFont dictionary and a `/Encoding` entry specifying the CMap that maps character codes to CIDs (Character IDs).
2. **The CIDFont dictionary** (`/Subtype /CIDFontType0` or `/CIDFontType2`): the actual font data. CIDFontType0 wraps a PostScript CID-keyed font; CIDFontType2 wraps a TrueType font. In practice, CIDFontType2 (TrueType-based) is overwhelmingly more common.

```
% Type 0 composite font structure in a PDF
10 0 obj
<< /Type /Font
   /Subtype /Type0
   /BaseFont /ABCDEF+DejaVuSans
   /Encoding /Identity-H
```

```
   /DescendantFonts [11 0 R]
   /ToUnicode 12 0 R
>>
endobj

11 0 obj
<< /Type /Font
   /Subtype /CIDFontType2
   /BaseFont /ABCDEF+DejaVuSans
   /CIDSystemInfo << /Registry (Adobe) /Ordering (Identity) /Supplement 0 >>
   /FontDescriptor 13 0 R
   /W [3 [278] 36 [722] 68 [556 500]]
>>
endobj
```

- The `/BaseFont` name includes a subset prefix (ABCDEF+) — we'll explain this below
- `/Identity-H` means character codes map directly to CIDs (horizontal writing mode)
- The descendant CIDFont dictionary
- The `/ToUnicode` CMap enables text extraction — critical for accessibility and search
- CIDFontType2: the actual glyph data comes from an embedded TrueType font
- The `/W` (Widths) array defines per-glyph widths for precise text positioning

When PDFBox's `PDType0Font.load()` embeds a TrueType font, it creates exactly this structure. You don't need to build it manually, but understanding it helps when debugging font issues in existing PDFs.

12.3 Font Embedding vs. Referencing

There are two ways a PDF can use a font: **embed** it (include the font program inside the PDF file) or **reference** it (include only the font name and metrics, relying on the viewer to provide the actual glyphs).

12.3.1 Referenced Fonts

A referenced font includes the font dictionary and metrics but *not* the font program. The viewer must find a matching font on the system or use a fallback. The Standard 14 fonts are the canonical example — they're designed to be used without embedding because every viewer is expected to have compatible substitutes.

Referenced non-standard fonts are a recipe for rendering failures. If your PDF references "CompanyBrandFont-Regular" without embedding it, the viewer will substitute whatever it can find — usually a generic serif or sans-serif font — and the text will look wrong. Character widths will differ, glyphs may be missing, and text positioning will be off.

For signature appearances, **never reference non-standard fonts**. The signing server has the font file; the recipient's PDF viewer probably doesn't.

12.3.2 Embedded Fonts

An embedded font includes the complete font program (or a subset of it) as a stream inside the PDF. The viewer uses the embedded data to render glyphs, regardless of what fonts are installed on the system.

Embedding guarantees consistent rendering across all viewers and all platforms. It increases the PDF file size — a full TrueType font can be several megabytes — but subsetting (described below) reduces the impact dramatically.

The font program is stored as a stream referenced by the `/FontDescriptor` dictionary:

Table 12.2: Font embedding streams by font type — ISO 32000-2:2020 Table 123

Font Type	FontDescriptor Key	Stream Contents
Type 1	`/FontFile`	PFB (Printer Font Binary) data — the Type 1 font program
TrueType	`/FontFile2`	Complete TrueType font file (`.ttf`)
CFF / OpenType	`/FontFile3`	CFF (Compact Font Format) data, or OpenType font program. The stream's `/Subtype` distinguishes: `/CIDFontType0C` for CID-keyed CFF, `/Type1C` for simple CFF, `/OpenType` for OpenType.

The distinction between `/FontFile`, `/FontFile2`, and `/FontFile3` is not just metadata — the stream contents are in fundamentally different binary formats. A viewer that tries to parse a CFF stream as TrueType (or vice versa) will get corrupted glyph outlines or crash. Getting the key wrong is a common source of font rendering failures in programmatically generated PDFs.

12.4 Font Subsetting

Embedding a full font file — especially a CJK font with tens of thousands of glyphs — adds significant bulk to the PDF. **Font subsetting** solves this by embedding only the glyphs actually used in the document.

A typical signature appearance uses fewer than 100 distinct glyphs (the signer's name, a date, a reason string). Subsetting a 2 MB TrueType font down to just those glyphs produces a subset of perhaps 10-30 KB.

12.4.1 The Subset Prefix Convention

ISO 32000-2:2020 Section 9.6.4 defines a naming convention for subsetted fonts: the `/BaseFont` name is prefixed with six uppercase letters followed by a plus sign.

```
/BaseFont /ABCDEF+DejaVuSans
```

The prefix "ABCDEF+" indicates that this is a subset — it does not contain all glyphs from the original DejaVuSans font. The six letters are arbitrary (PDFBox generates them randomly);

their only purpose is to make the subset's `/BaseFont` name unique within the document, preventing different subsets of the same font from being confused with each other.

A font *without* the subset prefix:

```
/BaseFont /DejaVuSans
```

This indicates the full font is embedded (or the font is referenced without embedding).

When debugging font issues, the subset prefix is your first clue. If you see `/ABCDEF+Helvetica`, the font is subsetted. If a glyph is missing from the rendered output, it may not have been included in the subset — meaning the glyph was not used (or not *known* to be used) when the subset was created. This is a common bug when signature text is modified after the font was subsetted.

12.4.2 How PDFBox Handles Subsetting

PDFBox 3.0.0 performs automatic font subsetting when saving a document. When you load a TrueType font with `PDType0Font.load()`, PDFBox tracks which glyphs are used during text drawing operations and includes only those glyphs in the final embedded subset.

```
// PDFBox tracks glyph usage automatically
PDFont font = PDType0Font.load(document, new File("DejaVuSans.ttf"));

// When we write text, PDFBox notes which glyphs are needed
try (OutputStream os = formXObject.getStream().createOutputStream()) {
    // ... write content stream using this font ...
}

// On save, PDFBox subsets the font to include only used glyphs
document.save(outputStream);
```

There is one catch. If you write raw content stream operators directly (as we must for signature appearance Form XObjects in PDFBox 3.x, since `PDPageContentStream` does not support Form XObjects), PDFBox may not automatically track glyph usage. In this case, you must encode the text through the font to ensure the glyph tracking is triggered:

```
// Ensure glyph tracking when writing raw content streams
byte[] encodedText = font.encode("Digitally signed by: Muller");
// Use encodedText in your content stream, not the raw string
```

If you bypass PDFBox's text encoding and write raw character codes directly into the content stream, the subset may not include all required glyphs. The result: some characters render as `.notdef` (the missing glyph symbol, typically a small rectangle) in viewers. This is one of the most common causes of broken signature appearances.

12.5 Encoding

Encoding is the mapping between **character codes** (the byte values in the content stream) and **glyphs** (the visual shapes the viewer renders). Getting encoding right is essential for correct text rendering, and getting it wrong is the most common cause of garbled or missing text in PDF documents.

12.5.1 Simple Font Encoding (Single-Byte)

For simple fonts (Type 1 and simple TrueType), each character code is a single byte (0-255). The encoding maps each byte value to a glyph name, and the font program maps glyph names to outlines.

PDF defines two built-in encodings:

WinAnsiEncoding (ISO 32000-2:2020 Table D.1): Corresponds closely to Windows-1252. Covers Western European characters: basic Latin, accented characters (a, e, u, n, etc.), and common symbols. This is the default for most PDF generators and the encoding used by the Standard 14 fonts.

MacRomanEncoding (ISO 32000-2:2020 Table D.2): The classic Macintosh character set. Rarely used in modern PDFs, but you'll encounter it in documents created by older Mac applications.

A font dictionary can also specify a **custom encoding** via the `/Encoding` key, using a `/Differences` array that overrides individual code-to-glyph mappings:

```
/Encoding <<
  /Type /Encoding
  /BaseEncoding /WinAnsiEncoding
  /Differences [
    128 /Euro
    160 /space /exclamdown /cent /sterling
  ]
>>
```

- Maps byte value 128 to the Euro sign glyph, overriding the WinAnsiEncoding mapping

12.5.2 The 256-Glyph Wall

Single-byte encoding means a simple font can address at most 256 glyphs. This is fine for English and most Western European languages, but it's a hard wall for:

- **CJK languages**: Chinese alone requires thousands of glyphs
- **Arabic and Hebrew**: need contextual glyph shaping (the same character looks different depending on position in a word)
- **Devanagari, Thai, Georgian**: scripts with large glyph inventories
- **Mixed-script text**: a signature appearance showing a name in Cyrillic with a date in Latin

For any of these, you need a composite font (Type 0 + CIDFont) with a multi-byte encoding.

12.5.3 Composite Font Encoding

Composite fonts use a **CMap** (Character Map) to map multi-byte character codes to CIDs. The most common CMap in modern PDFs is `/Identity-H`, which maps character codes directly to CIDs as a simple identity function — code 0 maps to CID 0, code 1 to CID 1, and so on.

With `/Identity-H`, the character codes in the content stream are raw CID values encoded as two-byte big-endian integers. This gives access to the full 65,536 CID range, more than

enough for any single font.

The downside of `/Identity-H` is that the character codes have no inherent semantic meaning — they're just glyph indices. Without additional mapping information, a viewer cannot extract the text content or determine what Unicode characters the glyphs represent. That's where the `/ToUnicode` CMap comes in.

12.6 The ToUnicode CMap

The `/ToUnicode` entry in a font dictionary maps character codes to Unicode values. It is not used for rendering — the viewer uses the font's glyph outlines for that. Instead, it is used for **text extraction**: copy-paste, search, and accessibility.

For signature appearances, the `/ToUnicode` CMap matters because:

1. **Accessibility tools** read the `/ToUnicode` CMap to extract text from the appearance stream. A signature appearance without `/ToUnicode` mappings will appear as empty or garbled text to screen readers.
2. **PDF validators** (including veraPDF) check for the presence of `/ToUnicode` CMaps as a compliance requirement for PDF/A. If your signed PDFs must be PDF/A-compliant, the fonts in your signature appearance must have proper `/ToUnicode` entries.
3. **Text search** in the document depends on `/ToUnicode`. Without it, searching for the signer's name will not find the text in the signature appearance.

A `/ToUnicode` CMap is a PDF stream containing a CMap program in PostScript-like syntax:

```
/CIDInit /ProcSet findresource begin
12 dict begin
begincmap
/CIDSystemInfo
  << /Registry (Adobe) /Ordering (UCS) /Supplement 0 >> def
/CMapName /Adobe-Identity-UCS def
/CMapType 2 def
1 begincodespacerange
<0000> <FFFF>
endcodespacerange
5 beginbfchar
<0024> <0044>        % CID 36 -> U+0044 (D)
<004C> <0069>        % CID 76 -> U+0069 (i)
<004E> <0067>        % CID 78 -> U+0067 (g)
endbfchar
endcmap
CMapName currentdict /CMap defineresource pop
end
end
```

PDFBox generates the `/ToUnicode` CMap automatically when you use `PDType0Font.load()` and write text through the font's encoding methods. You do not need to create it manually — but understanding the structure helps when you're debugging a PDF where text extraction returns garbled results.

In PDFBox 3.x, the method `PDFont.getToUnicodeCMap()` is *not* publicly accessible. To inspect the `/ToUnicode` CMap of a font, you must access the COS dictionary directly: `font.getCOSObject().getDictionaryObject(COSName.TO_UNICODE)`. This is a common stumbling block when writing font inspection or diagnostic tools.

12.7 Font Descriptors

Every embedded font (and many referenced fonts) has a **font descriptor** dictionary (`/Type /FontDescriptor`) that provides metrics and metadata about the font. The font descriptor is essential — it tells the viewer how to render the font even before it looks at the glyph outlines.

Table 12.3: Key entries in the font descriptor — ISO 32000-2:2020 Table 122

Key	Type	Purpose
`/FontName`	name	The PostScript name of the font (e.g., `/DejaVuSans`)
`/Flags`	integer	Bit field describing the font: FixedPitch (bit 1), Serif (bit 2), Symbolic (bit 3), Script (bit 4), Nonsymbolic (bit 6), Italic (bit 7), AllCap (bit 17), SmallCap (bit 18), ForceBold (bit 19)
`/FontBBox`	array	The bounding box of all glyphs: `[llx lly urx ury]` in glyph coordinate space
`/ItalicAngle`	number	Angle of the italic slant (0 for upright fonts)
`/Ascent`	number	The maximum height above the baseline
`/Descent`	number	The maximum depth below the baseline (typically negative)
`/CapHeight`	number	The height of capital letters
`/StemV`	number	The dominant stem width (vertical strokes) — affects rendering weight
`/FontFile`	stream	Type 1 font program (PFB format)
`/FontFile2`	stream	TrueType font program
`/FontFile3`	stream	CFF or OpenType font program (with `/Subtype` to distinguish)

A font descriptor with all metrics but *no* font file stream describes a **referenced** font — the viewer must find a substitute. If the descriptor includes a font file stream, the font is **embedded**. Checking whether `/FontFile`, `/FontFile2`, or `/FontFile3` exists is the definitive way to determine if a font is embedded.

12.8 PDFBox Font Handling in 3.x

PDFBox 3.0.0 made several breaking changes to its font API compared to 2.x. Understanding these changes is essential for writing correct font handling code.

12.8.1 Loading Fonts

```
// Standard 14 font (no embedding needed)
PDFont helvetica = new PDType1Font(Standard14Fonts.FontName.HELVETICA);

// Embed a TrueType font from a file
PDFont dejaVu = PDType0Font.load(document, new File("DejaVuSans.ttf"));

// Embed a TrueType font from an InputStream
try (InputStream is = getClass().getResourceAsStream("/fonts/NotoSans-Regular.ttf")) {
    PDFont notoSans = PDType0Font.load(document, is);
}

// Embed a TrueType font WITHOUT subsetting
// (include all glyphs -- larger file, but no risk of missing glyphs)
PDFont fullFont = PDType0Font.load(document, new File("DejaVuSans.ttf"), false);
```

The `false` parameter to `PDType0Font.load()` disables subsetting. This is rarely needed, but can be useful during debugging to rule out subsetting as the cause of missing glyphs.

12.8.2 The getEncoding() Removal

PDFBox 2.x provided a `PDFont.getEncoding()` method that returned the font's encoding object. PDFBox 3.x **removed** this method.

If you need to inspect a font's encoding in PDFBox 3.0.0, you must access the COS dictionary directly:

```
// PDFBox 2.x -- no longer works
// Encoding encoding = font.getEncoding();

// PDFBox 3.x -- access the COS dictionary
COSBase encodingObj = font.getCOSObject().getDictionaryObject(COSName.ENCODING);

if (encodingObj instanceof COSName encodingName) {
    // Named encoding: /WinAnsiEncoding, /MacRomanEncoding, etc.
    log.info("Font uses named encoding: {}", encodingName.getName());
} else if (encodingObj instanceof COSDictionary encodingDict) {
    // Custom encoding with /Differences array
    COSArray differences = encodingDict.getDictionaryObject(
            COSName.getPDFName("Differences"), COSArray.class);
    if (differences != null) {
        log.info("Font has custom encoding with {} difference entries",
                differences.size());
    }
}
```

12.8.3 Inspecting Font Embedding Programmatically

A common diagnostic task is checking whether a font in a PDF is embedded. Here's a complete example:

```
private boolean isFontEmbedded(PDFont font) {
    PDFontDescriptor descriptor = font.getFontDescriptor();
    if (descriptor == null) {
```

```
            // Standard 14 fonts may not have a descriptor
            return false;
        }
        return descriptor.getFontFile() != null
            || descriptor.getFontFile2() != null
            || descriptor.getFontFile3() != null;
    }
```

12.9 Why Signature Text Shows as Squares or Blanks

This section addresses the most common font-related failures in signature appearances, what causes them, and how to fix them.

12.9.1 Missing Font

Symptom: The signature block is empty, or shows substituted text in a different font than intended.

Cause: The signature appearance references a font that is not embedded and not available on the viewer's system.

Fix: Always embed fonts used in signature appearances. For basic ASCII text, the Standard 14 fonts are a safe choice because viewers provide substitutes. For anything else, embed a TrueType font using `PDType0Font.load()`.

12.9.2 Wrong Encoding

Symptom: Some characters render correctly, but accented characters (e, u, n) show as wrong characters or `.notdef` squares.

Cause: The text contains characters outside the font's encoding range. WinAnsiEncoding covers codes 32-255 corresponding to Windows-1252, which includes most Western European accented characters. But characters outside this range — em-dash at certain positions, certain ligatures, or any non-Latin script — will fail.

Fix: Use a composite font (Type 0) with `/Identity-H` encoding and an embedded TrueType font. This provides full Unicode coverage. In PDFBox: use `PDType0Font.load()` instead of the Standard 14 fonts.

12.9.3 Unembedded Font with Non-Standard Glyphs

Symptom: Text renders as squares or blank space on some viewers but works on others.

Cause: The font is referenced but not embedded. Viewers that happen to have a compatible font installed render the text correctly; viewers without the font substitute a generic font that may not have the required glyphs.

Fix: Embed the font.

12.9.4 Subset Missing Glyphs

Symptom: Most text renders correctly, but specific characters are missing.

Cause: The font was subsetted, but the subset does not include all required glyphs. This typically happens when text is written to the content stream *after* the font subsetting decision was made, or when glyph usage tracking was bypassed.

Fix: Ensure all text is written through PDFBox's font encoding methods *before* saving. If the problem persists, try loading the font with subsetting disabled (`PDType0Font.load(document, file, false)`) to confirm subsetting is the cause.

When writing raw content stream operators for a Form XObject (which PDFBox 3.x requires for signature appearances), make sure to encode text through the font *before* writing the encoded bytes to the stream. Skipping this step means PDFBox won't know which glyphs are needed and may produce a subset that's missing characters.

```
// WRONG: writing raw text bypasses glyph tracking
os.write("(John Muller) Tj".getBytes(StandardCharsets.US_ASCII));

// RIGHT: encode through the font first
byte[] encoded = font.encode("John Muller");
// Then write the encoded bytes as a hex string in the content stream
os.write(("<" + bytesToHex(encoded) + "> Tj").getBytes(StandardCharsets.US_ASCII));
```

12.10 Font Licensing and the fsType Flag

TrueType and OpenType fonts contain a licensing flag called **fsType** in the OS/2 table. This flag indicates what the font vendor permits in terms of embedding and editing:

Table 12.4: fsType embedding permission bits

Bits	Name	Meaning
0x0000	Installable	No restrictions. The font may be embedded and used freely.
0x0002	Restricted	The font must not be embedded in documents. Period.
0x0004	Preview & Print	The font may be embedded in read-only documents. The recipient can view and print but should not edit.
0x0008	Editable	The font may be embedded, and the recipient may edit the document.

Most free and open-source fonts (DejaVu, Noto, Liberation) have fsType 0x0000 (Installable). Many commercial fonts (fonts you pay for) have restrictive fsType values.

PDF generators typically respect the fsType flag — PDFBox will emit a warning (but not throw an exception) when embedding a font with restrictive fsType. Some tools refuse to embed the font entirely.

For signature appearances:

- **Use fonts with fsType 0x0000** (Installable) to avoid any licensing complications.

- The **Google Noto font family** (`Noto Sans`, `Noto Serif`, `Noto Sans CJK`) covers virtually every script in Unicode and is Apache-2.0 licensed with fsType 0x0000. It is the best general-purpose choice for internationalized signature text.
- **DejaVu Sans** covers Latin, Greek, Cyrillic, Armenian, Georgian, and many symbols. Free, well-tested, and widely deployed.
- **Liberation Sans/Serif/Mono** are metrics-compatible with Arial/Times New Roman/Courier New. Licensed under SIL Open Font License.

Even if your PDF generator embeds a font with restrictive fsType, the PDF viewer will render it. The fsType flag is a licensing statement, not a technical enforcement mechanism. But distributing PDFs with improperly embedded commercial fonts exposes your organization to licensing claims. For server-side signing operations that produce thousands of PDFs, use only fonts whose license explicitly permits embedding.

12.11 Practical Advice for Signature Appearances

Drawing from everything in this chapter, here is the decision framework for choosing and configuring fonts in signature appearance streams:

12.11.1 For ASCII-Only Signatures

If your signer names, dates, and reasons are guaranteed to be ASCII only (a-z, A-Z, 0-9, basic punctuation):

```
// Simple, reliable, zero-cost approach
PDFont font = new PDType1Font(Standard14Fonts.FontName.HELVETICA);
```

This works everywhere. No embedding needed, no licensing concerns, no subsetting complications. The text will render in Helvetica or a metrics-compatible substitute on every viewer.

12.11.2 For International Signatures

If signer names may contain non-ASCII characters (accented Latin, Cyrillic, CJK, Arabic, or anything beyond Windows-1252):

```
// Embed a Unicode-capable font
try (InputStream is = getClass().getResourceAsStream("/fonts/NotoSans-Regular.ttf")) {
    PDFont font = PDType0Font.load(document, is);
    // Now you can write any Unicode character the font covers
}
```

Bundle the font file with your application (in `src/main/resources/fonts/` for a Maven project). Do not rely on system-installed fonts — the signing server may not have the same fonts as your development machine.

12.11.3 Font Size Guidelines

Signature appearance text is typically rendered at small sizes (8-12 points) in a rectangle that's often only 150-300 points wide. Keep these guidelines in mind:

- **10pt** is a good default for signer name and date

- **8pt** works for secondary information (reason, location, certificate details)
- **Never go below 7pt** — some viewers render small text poorly, and the text becomes unreadable when printed
- **Calculate text width** before rendering to ensure it fits the signature rectangle; truncation or overflow beyond the bounding box results in clipped text that viewers may or may not display

12.12 Debugging Font Issues

When a signature appearance renders incorrectly, the following diagnostic steps will identify the root cause.

12.12.1 Step 1: Identify the Font

Open the PDF in a tool that shows the raw object structure (PDFalyzer, QPDF, or `PDFDebugger` from PDFBox's tools module). Navigate to the signature field's widget annotation, then to `/AP /N` (the normal appearance stream), then to `/Resources /Font`. Note the font name, subtype, and whether a font descriptor with a font file stream exists.

12.12.2 Step 2: Check Embedding

Look for `/FontFile`, `/FontFile2`, or `/FontFile3` in the font descriptor. If none exists, the font is not embedded — that's likely the problem.

12.12.3 Step 3: Check Encoding

For simple fonts, check the `/Encoding` entry. Is it `/WinAnsiEncoding`? If the problematic characters are outside Windows-1252, the encoding is the cause.

For composite fonts, check the `/Encoding` entry on the Type 0 dictionary. It should be `/Identity-H` for a properly structured modern PDF.

12.12.4 Step 4: Check the ToUnicode CMap

If the text *renders* correctly but copy-paste or search returns garbled results, the `/ToUnicode` CMap is missing or incorrect.

12.12.5 Step 5: veraPDF Validation

veraPDF provides font-specific validation rules that catch a wide range of font issues:

```
# Run veraPDF validation focusing on font issues
verapdf --format text --flavour 2b input.pdf 2>&1 | grep -i font
```

veraPDF checks for embedded fonts (required for PDF/A), valid `/ToUnicode` mappings, correct font descriptor entries, and consistent glyph widths. Its validation messages are often the fastest way to identify exactly what's wrong with a font configuration.

12.12.6 Step 6: PDFBox Font Inspection

For programmatic inspection, PDFBox can enumerate all fonts in a page or appearance stream:

```
private void inspectAppearanceFonts(PDAnnotationWidget widget) {
    PDAppearanceStream appearance = widget.getAppearance().getNormalAppearance();
    if (appearance == null) {
        log.warn("Signature widget has no normal appearance stream");
        return;
    }

    PDResources resources = appearance.getResources();
    if (resources == null) {
        log.warn("Appearance stream has no resources");
        return;
    }

    for (COSName fontName : resources.getFontNames()) {
        PDFont font = resources.getFont(fontName);
        log.info("Font /{}: type={}, baseFont={}, embedded={}",
                fontName.getName(),
                font.getClass().getSimpleName(),
                font.getName(),
                isFontEmbedded(font));

        // Check for ToUnicode CMap
        COSBase toUnicode = font.getCOSObject()
                .getDictionaryObject(COSName.TO_UNICODE);
        log.info("  ToUnicode CMap: {}",
                toUnicode != null ? "present" : "MISSING");
    }
}
```

When debugging font issues in production, always capture the raw bytes of the appearance stream and the font dictionary's COS structure. The visual symptom (squares, blanks, wrong characters) tells you *something* is wrong; the COS structure tells you *what*. A tool that shows both the rendered appearance and the underlying object tree — the way PDFalyzer does — saves enormous debugging time compared to flipping between a viewer and a hex editor.

12.13 Summary

Fonts in PDF are a deceptively deep topic. For digital signatures specifically, the key takeaways are:

- **Embed your fonts.** The Standard 14 fonts are safe for ASCII text; everything else must be embedded.
- **Use composite fonts (Type 0) for non-ASCII text.** Simple fonts are limited to 256 glyphs.
- **Encode text through the font API** to ensure glyph tracking for subsetting works correctly.
- **Use freely licensed fonts** (Noto Sans, DejaVu, Liberation) to avoid fsType complications.

- **Test in multiple viewers** — what works in Adobe Acrobat may fail in pdf.js or LibreOffice.
- **When debugging, look at the COS structure**, not just the rendered output. The font dictionary, font descriptor, encoding, and `/ToUnicode` CMap tell the full story.

The next chapter — Chapter 12 — moves from the document infrastructure we've built up across Part II into the core subject of this book: the architecture of PDF digital signatures themselves.

Chapter 13

PDF Digital Signatures

13.1 Signature Architecture

This chapter dissects the internal architecture of PDF digital signatures as specified in ISO 32000-2:2020 §12.8. We examine every dictionary, every key, and every byte — from the AcroForm field tree down to the raw CMS `SignedData` blob embedded in `/Contents`.

But more importantly, we talk about what actually matters when you're staring at a broken signature at 2 AM, wondering why Adobe says "invalid" while your code says "fine."

13.1.1 Overview: What a PDF Signature Actually Is

Here's the thing most people get wrong about PDF signatures: they think of them as something *attached to* a document, like a wax seal on an envelope. They're not.

A PDF digital signature is a set of **PDF objects embedded within the document itself**. The signature *is part of the document*. This seemingly minor architectural choice has profound consequences — it's why PDF signatures are simultaneously powerful and maddening, why they survive copy operations but break on re-saves, and why you can have a perfectly valid signature on a document that's been modified after signing (yes, really).

The four pieces of a PDF signature:

1. A **signature field** (`PDSignatureField`) in the AcroForm field tree — the form field that "holds" the signature
2. A **signature dictionary** (`/Type /Sig`) — the actual cryptographic payload and metadata
3. A **widget annotation** — the visual rectangle on a page (may be zero-size for invisible signatures)
4. An **appearance stream** (`/AP`) — what the viewer renders in the signature rectangle

The signature field is a form field like any other (`/FT /Sig`). The signature dictionary is the *value* of that field (`/V`). An unsigned field has `/V` absent or null — the field exists but has no signature value yet.

Think of it like an empty checkbox on a paper form. The box is there, waiting to be checked. A document author can place signature fields in a PDF template, and signers fill them in later — each one independently, at different times, possibly on different continents.

When you encounter a PDF with signature problems, start here. Open the document in a COS-level viewer (like PDFalyzer, or even a hex editor) and check:

1. Does the AcroForm exist? (Some PDFs have signature fields floating without an AcroForm — instant headache.)
2. Does `/SigFlags` include bit 2 (AppendOnly)? If not, any editor can destroy all signatures with a simple save.
3. Is the `/V` entry present on the signature field? If absent, the field is unsigned.
4. Does the widget actually have a page reference? Some tools create orphaned widgets.

This five-minute checklist saves hours of debugging.

13.1.2 Signature Fields in the AcroForm Tree

> A signature field (field type Sig) shall represent a digital signature that may or may not have a signature value.
>
> — ISO 32000-2:2020 §12.7.4.5

Signature fields live in the document's interactive form structure:

13.1.2.1 The SigFlags Trap

The `/SigFlags` entry is one of those things that's easy to overlook and devastating to get wrong.

Table 13.1: SigFlags — ISO 32000-2:2020 §12.7.2, Table 225

Name	Bit	Req	Spec Ref	Effect
`SignaturesExist`	1	Required	§12.7.2	Tells the viewer "this document contains at least one signature field." Conforming readers **shall not** allow saving in a way that invalidates existing signatures.
`AppendOnly`	2	Required	§12.7.4.5	The document **shall** be saved using incremental updates only. This is the mechanism that protects existing signatures from being destroyed by a full rewrite.

Setting `/SigFlags` to 3 (both bits) is mandatory when a document contains signatures. If bit 2 is missing, a naive PDF editor may do a full save and destroy all signatures. Many real-world PDFs get this wrong.

Here's what happens in practice: someone creates a PDF with LibreOffice, adds a signature field, signs it — and LibreOffice doesn't set bit 2. The document gets emailed to a colleague who opens it in some random PDF editor to add an annotation, hits save, and the editor does a full rewrite. Every signature is now invalid. The colleague has no idea what happened. The signer gets an angry phone call.

The fix is always the same: if you're building a tool that creates signature fields, set `/SigFlags` to 3 the moment any field has a value. And if you're building a tool that edits PDFs,

check for `/SigFlags` bit 2 before saving — if it's set, you *must* use incremental save. If your tool doesn't support incremental save… well, you need to support it now. Welcome to PDF.

13.1.2.2 Field Dictionary Structure

A signature field's COS dictionary looks like this. Every key matters, and I'll explain why:

Here is the complete COS dictionary of a typical signature field. In most PDFs, the field and its widget annotation are merged into a single object — the spec (ISO 32000-2:2020 §12.5.6.19) allows splitting them, but few tools bother:

```
<<
  /Type /Annot
  /Subtype /Widget
  /FT /Sig
  /T (Signature1)
  /V 42 0 R
  /Rect [100 500 300 550]
  /AP << /N 43 0 R >>
  /P 7 0 R
  /Lock <<
    /Type /SigFieldLock
    /Action /All
  >>
  /SV <<
    /Type /SV
    /Filter /Adobe.PPKLite
    /SubFilter [/adbe.pkcs7.detached /ETSI.CAdES.detached]
    /DigestMethod [/SHA256 /SHA384 /SHA512]
    /Reasons [(Approved)(Reviewed)]
    /MDP << /P 2 >>
  >>
>>
```

The keys fall into three groups: annotation/widget keys (how it looks and where it lives), signature field keys (what it is and what it points to), and sub-dictionaries that constrain signing behavior.

Table 13.2: Annotation/Widget Keys — ISO 32000-2:2020 §12.5.6.19

Key	Type	Req	Spec Ref	Description
`/Type`	name	Optional	§12.5.6	Always `/Annot` for the merged field+widget.
`/Subtype`	name	Required	§12.5.6	Always `/Widget` — this is a widget annotation.
`/Rect`	rectangle	Required	§12.5.6	Widget rectangle in page coordinates: `[llx lly urx ury]`. If `[0 0 0 0]`, the signature is invisible.

Key	Type	Req	Spec Ref	Description
`/P`	reference	Optional	§12.5.6	Page reference — which page this widget is on. Some poorly generated PDFs omit this, and then viewers have to guess. Don't be that tool.
`/AP`	dictionary	Optional	§12.5.6.19	Appearance stream dictionary. `/N` is the normal appearance — what the viewer draws. Without this, many viewers show a generic placeholder. Adobe is particularly opinionated about appearances.

Table 13.3: Signature Field Keys — ISO 32000-2:2020 §12.7.5.5

Key	Type	Req	Spec Ref
`/FT`	name	Required	§12.7.4.1
Field type — `/Sig` marks this as a signature field. Non-negotiable. If you see `/FT /Tx` on something that's supposed to be a signature, someone made a mistake.			
`/T`	text string	Required	§12.7.4.1
Partial field name — used to identify the field programmatically. This is how your code finds "the signature field called Signature1."			
`/V`	reference	Optional	§12.7.5.5
Reference to the **signature dictionary** (`/Type /Sig`) — the cryptographic payload. If absent, the field is unsigned. When present, this is the key entry that connects the form field to the actual signature. The signature dictionary is detailed in The Signature Dictionary ().			

The remaining two keys — `/Lock` and `/SV` — are sub-dictionaries that deserve their own tables.

Table 13.4: Lock Dictionary (`/Type /SigFieldLock`) — ISO 32000-2:2020 §12.7.5.5, Table 237

Key	Type	Req	Spec Ref
`/Type`	name	Optional	§12.7.5.5
Always `/SigFieldLock`.			
`/Action`	name	Required	§12.7.5.5, Table 237
What to lock after signing: `/All` (lock everything), `/Include` (lock fields listed in `/Fields`), or `/Exclude` (lock everything *except* fields listed in `/Fields`). This is how approval signatures can lock specific fields — see ???.			

Key	Type	Req	Spec Ref
`/Fields`	array	Conditional	§12.7.5.5, Table 237
Array of field names to include or exclude from locking. Required when `/Action` is `/Include` or `/Exclude`.			

13.1.2.3 Seed Value Dictionary (`/SV`): The Unsung Hero

> The seed value dictionary provides constraint information that shall be used at the time the signature is applied.
>
> — ISO 32000-2:2020 §12.7.5.5

The seed value dictionary is one of the most powerful and underused features in the entire PDF signature specification. Most people don't even know it exists, which is a shame, because it solves a problem that plagues every organization deploying PDF signatures at scale: **how do you ensure signers use the right algorithm, the right format, and the right handler?**

Table 13.5: Seed Value Dictionary (`/Type /SV`) — ISO 32000-2:2020 §12.7.5.5, Table 238

Key	Type	Req	Spec Ref
`/Filter`	name	Optional	§12.7.5.5, Table 238
Required signature handler (e.g., `/Adobe.PPKLite`). Use this to ensure your documents work with specific software.			
`/SubFilter`	array of names	Optional	§12.7.5.5, Table 238
Acceptable signature formats. Set this to `[/ETSI.CAdES.detached]` if you need PAdES compliance. The signer's software **must** use one of these formats.			
`/DigestMethod`	array of names	Optional	§12.7.5.5, Table 238
Acceptable hash algorithms. Set this to `[/SHA256 /SHA384 /SHA512]` — and for the love of all that is holy, **do not include `/SHA1`**. It's 2026. SHA-1 has been broken since 2017.			
`/Reasons`	array of strings	Optional	§12.7.5.5, Table 238
Pre-defined reason strings the signer must choose from. Great for audit trails — instead of "asdf" in the reason field, signers pick from "Approved", "Reviewed", "Authorized".			
`/LegalAttestation`	array of strings	Optional	§12.7.5.5, Table 238
Legal attestation strings — rarely used, but available if you need legal boilerplate attached to the signature.			
`/Ff`	integer	Optional	§12.7.5.5, Table 238
Flags indicating which seed values are **required** (vs. recommended). If bit 1 is set on a constraint, it's mandatory — the signer's software must enforce it, not just suggest it.			

The `/MDP` and `/TimeStamp` entries are sub-dictionaries with their own structure:

Table 13.6: Seed Value MDP Dictionary (`/MDP`) — ISO 32000-2:2020 §12.7.5.5

Key	Type	Req	Spec Ref	Description
`/P`	integer	Required	§12.7.5.5	The DocMDP permission level this signature must enforce: 1 (no changes), 2 (form fills + signatures), 3 (form fills + signatures + annotations). This is how a template author says "whoever signs this field must also lock the document."

Table 13.7: Seed Value TimeStamp Dictionary (`/TimeStamp`) — ISO 32000-2:2020 §12.7.5.5

Key	Type	Req	Spec Ref	Description
`/URL`	ASCII string	Required	§12.7.5.5	URL of the RFC 3161 timestamp server.
`/Ff`	integer	Optional	§12.7.5.5	If bit 1 is set, timestamping is required (not just recommended). If you need PAdES-B-T, set this. The signer's software will contact the TSA automatically.

Practical advice for template authors:

If you're creating PDF templates that others will sign, use seed values aggressively. Don't trust signers to pick the right options — they won't. A financial services company once sent us PDFs where half the signatures used SHA-1 because some signers had old software and nobody constrained the algorithm. Those signatures were technically valid but would fail compliance audits.

Set `/DigestMethod` to `[/SHA256]`, set `/SubFilter` to `[/ETSI.CAdES.detached]`, and set the `/Ff` flags to make them mandatory. Your future self will thank you.

13.1.3 The Signature Dictionary (`/Type /Sig`)

The signature dictionary is the heart of a PDF digital signature. It's referenced from the field's `/V` entry, and it's where the real action is. Everything else — the field, the widget, the appearance — is just scaffolding. This dictionary is where the cryptographic proof lives.

Here is the complete structure of a typical signature dictionary — every key you'll encounter in production:

```
<<
  /Type /Sig
  /Filter /Adobe.PPKLite
  /SubFilter /ETSI.CAdES.detached
```

```
  /ByteRange [0 15000 15500 34500]
  /Contents <308206A7...00000000>
  /M (D:20260306120000+00'00')
  /Name (John Doe)
  /Location (Vienna, Austria)
  /Reason (Approved)
  /ContactInfo (john@example.com)
  /Reference [
    <<
      /Type /SigRef
      /TransformMethod /DocMDP
      /TransformParams <<
        /Type /TransformParams
        /P 2
        /V /1.2
      >>
      /DigestMethod /SHA256
    >>
  ]
  /Prop_Build <<
    /App << /Name /PDFalyzer >>
    /Filter << /Name /Adobe.PPKLite >>
  >>
>>
```

The table below dissects each key. The core cryptographic keys come first, then metadata, then sub-dictionaries.

13.1.3.1 Core Signature Keys

These five keys define the cryptographic signature itself. Get any of them wrong and the signature is broken.

Table 13.8: Core keys of the Signature Dictionary (`/Type /Sig`) — ISO 32000-2:2020 §12.8.1, Table 255

Key	Type	Req	Spec Ref
`/Type`	name	Optional	§12.8.1
Always `/Sig` when present. Identifies this dictionary as a signature value. Optional per the spec, but always include it — defensive parsers rely on it.			
`/Filter`	name	Required	§12.8.1, Table 255
The signature handler name. In theory, this identifies which software can process the signature. In practice, it's always `/Adobe.PPKLite`. Always. Even when the signing software has nothing to do with Adobe. The name stuck because Adobe defined the original signature spec in 1999 and everyone just... went with it.			
`/SubFilter`	name	Required	§12.8.1, Table 255

Key	Type	Req	Spec Ref
The actual encoding format of the signature value. *This* is the key that determines how `/Contents` is structured — see SubFilter Values: A Brief History of Getting It Right for the full history and current recommendation.			
`/ByteRange`	array	Required	§12.8.1, Table 255
Four integers `[offset1 length1 offset2 length2]` defining which bytes of the file are signed. The gap between the two ranges is where `/Contents` lives. The most important array in the entire dictionary — a deep dive is in ???.			
`/Contents`	string	Required	§12.8.1, Table 255
Hex-encoded CMS `SignedData` blob (RFC 5652 — Cryptographic Message Syntax (CMS) §5), padded with zeros. This is the actual cryptographic signature. Everything else in this dictionary is metadata. This is the payload. See The Value: CMS SignedData.			

13.1.3.2 Metadata Keys

These keys carry informational data about the signing event. They are part of the signed bytes and cannot be changed after signing — but they are set by whoever controls the signing process, **not verified against any external authority**.

Table 13.9: Metadata keys of the Signature Dictionary — ISO 32000-2:2020 §12.8.1, Table 255

Key	Type	Req	Spec Ref	Description
`/M`	date string	Optional	§12.8.1	Signing time in PDF date format. **Do not trust this.** It's the signer's local clock. Use the CMS timestamp attribute for authoritative time.
`/Name`	text string	Optional	§12.8.1	Signer's name. **Do not trust this either.** It's free text. The authoritative name is in the X.509 certificate inside `/Contents`.
`/Location`	text string	Optional	§12.8.1	Claimed signing location. Informational, unverified.
`/Reason`	text string	Optional	§12.8.1	Claimed signing reason (e.g., "Approved", "Reviewed"). Informational, unverified.
`/ContactInfo`	text string	Optional	§12.8.1	Contact info. Informational, optional, you see the pattern.

The `/M`, `/Name`, `/Location`, `/Reason`, and `/ContactInfo` fields are **unauthenticated metadata**. Yes, they're part of the signed bytes. Yes, they can't be changed after signing. But they're set *before* signing, by whoever controls the signing process. If a malicious signer puts "Barack

Obama" in the `/Name` field and signs with a self-signed certificate, the signature will be structurally valid. **The only trustworthy identity information is the X.509 certificate in `/Contents`.** Never display `/Name` to users as if it were verified identity. This mistake is shockingly common in PDF viewer implementations.

13.1.3.3 Signature Reference Dictionary (`/Reference`)

The `/Reference` entry is an array of **signature reference dictionaries** (`/Type /SigRef`) that define transform methods — rules about what can and cannot change after signing. This is the mechanism behind certification signatures and field-level locking (covered in depth in ???).

Table 13.10: Signature Reference Dictionary (`/Type /SigRef`) — ISO 32000-2:2020 §12.8.1, Table 256

Key	Type	Req	Spec Ref
`/Type`	name	Optional	§12.8.1, Table 256
Always `/SigRef`.			
`/TransformMethod`	name	Required	§12.8.1, Table 256
The transform method: `/DocMDP` (document modification detection and prevention), `/FieldMDP` (field-level locking), or `/UR` (usage rights — deprecated in PDF 2.0). DocMDP is by far the most common. See ???.			
`/TransformParams`	dictionary	Optional	§12.8.1, Table 256
Parameters for the transform method. For DocMDP, this contains `/P` (permission level: 1 = no changes, 2 = form fills + signatures, 3 = form fills + signatures + annotations) and `/V` (version).			
`/DigestMethod`	name	Required	§12.8.1, Table 256
Hash algorithm for this reference (e.g., `/SHA256`). Must match the algorithm used in the CMS structure.			

13.1.3.4 Build Properties Dictionary (`/Prop_Build`)

The `/Prop_Build` dictionary identifies the software that created the signature. It's optional and purely informational — but surprisingly useful for debugging.

Table 13.11: Build Properties Dictionary — ISO 32000-2:2020 §12.8.1, Table 258

Key	Type	Req	Spec Ref	Description
`/App`	dictionary	Optional	§12.8.1	Application that created the signature. Contains `/Name` (e.g., `/PDFalyzer`, `/AdobeAcrobatDC`).
`/Filter`	dictionary	Optional	§12.8.1	Signature handler info. Contains `/Name` (e.g., `/Adobe.PPKLite`).

Key	Type	Req	Spec Ref	Description
/PubSec	dictionary	Optional	§12.8.1	Public security handler info. Rarely populated.

The /Prop_Build dictionary is your friend when debugging.

When a signature doesn't validate, the first thing I check is /Prop_Build. It tells you which software created the signature. If it says /iText with a version from 2015, you know you might be dealing with an old CMS structure. If it says /Adobe Acrobat and the signature is invalid, something is genuinely wrong because Adobe's own tool should produce signatures that Adobe validates. If /Prop_Build is missing entirely, the signing software was lazy — which tells you something about the quality of the rest of the signature, too.

13.1.3.5 SubFilter Values: A Brief History of Getting It Right

The /SubFilter value determines how the /Contents bytes are structured. Getting this right is the difference between "signature works everywhere" and "signature works in Adobe but nowhere else" (or vice versa).

The history of SubFilter values is essentially the history of PDF signatures trying — and sometimes failing — to keep up with cryptographic best practices:

The first two SubFilter values are deprecated — they use SHA-1, which has been broken for collision resistance since the SHAttered attack of February 2017. You will encounter them in legacy documents, but you should never use them for new signatures.

Table 13.12: Deprecated SubFilter Values (do not use for new signatures)

SubFilter	Standard	Req	Spec Ref
/adbe.x509.rsa_sha1	PDF 1.3+	Deprecated	ISO 32000-2:2020 §12.8.3.2
Do not use. Raw RSA signature with the certificate in a separate /Cert key instead of inside /Contents. No CMS wrapper. SHA-1 only. This was PDF's first attempt at digital signatures in 1999. It made sense at the time. It doesn't anymore.			
/adbe.pkcs7.sha1	PDF 1.3+	Deprecated	ISO 32000-2:2020 §12.8.3.4
Do not use. CMS SignedData where the SHA-1 digest of the byte range is *encapsulated* in the CMS. This is the worst of both worlds: SHA-1 (broken) and encapsulated content (unnecessary complexity). Nobody misses this format.			

The following three values are the current standard. For new signatures, use /ETSI.CAdES.detached unless you have a specific reason not to.

Table 13.13: Current SubFilter Values

SubFilter	Standard	Req	Spec Ref
`/adbe.pkcs7.detached`	PDF 1.3+	Current	ISO 32000-2:2020 §12.8.3.3
CMS `SignedData` with detached content. The workhorse format — widely supported since PDF 1.3. The signed bytes are the PDF byte range, not embedded in CMS. If you need maximum compatibility with older software, this is your format.			
`/ETSI.CAdES.detached`	PDF 2.0 / PAdES	Modern	ETSI EN 319 142-1 V1.1.1 (2016-04) §5.1
CMS `SignedData` with mandatory CAdES signed attributes, specifically the `signing-certificate-v2` attribute. Required for PAdES-B-B and higher. **This is the modern standard. Use this.** The CMS structure is identical to `/adbe.pkcs7.detached` at the container level — the difference is one signed attribute (~50 bytes).			
`/ETSI.RFC3161`	PDF 2.0 / PAdES	Modern	ETSI EN 319 142-1 V1.1.1 (2016-04) §5.4
RFC 3161 timestamp token — not a signature, but a document-level timestamp. Used for PAdES-B-T and long-term archival. You'll encounter these when dealing with LTV (Long-Term Validation).			

Here's a trap that catches people all the time: Adobe Acrobat and most viewers accept *all* SubFilter values for backward compatibility. Your signature will "work" in Acrobat regardless of which SubFilter you use. But **only** `/ETSI.CAdES.detached` is valid for PAdES-compliant signatures under eIDAS. If you need legal standing in the EU — and increasingly, that's everyone — this is the only acceptable value. Don't let "it works in Acrobat" lull you into using `/adbe.pkcs7.detached` for new signatures. Acrobat is not the arbiter of legal validity; the eIDAS regulation is.

The compatibility question everyone asks:

"Can I use `/ETSI.CAdES.detached` if my users have old PDF readers?"

Yes. The CMS structure is identical to `/adbe.pkcs7.detached` at the container level. Old readers that don't understand the SubFilter name will typically fall back to parsing the `/Contents` as generic CMS/PKCS#7 — and it will work. The difference is one signed attribute (`signing-certificate-v2`), which old readers simply ignore.

The only scenario where it breaks: truly ancient software (pre-2010) that strictly checks the SubFilter name against a hardcoded whitelist and rejects unknown values. If your users are running PDF readers from 2008, they have bigger problems than SubFilter compatibility.

13.1.4 The `/Contents` Value: CMS SignedData

The `/Contents` entry is a **hex-encoded byte string** containing a DER-encoded CMS `SignedData` structure (RFC 5652 — Cryptographic Message Syntax (CMS) §5).

This is where the actual cryptographic signature lives. Everything else in the signature dictionary is metadata. This is the payload.

13.1.4.1 Why Hex-Encoded?

PDF strings can be literal `(…)` or hex `<…>`. The hex form is used because CMS blobs contain arbitrary binary data (DER-encoded ASN.1), and literal strings would need extensive escaping for bytes like `\(`, `\)`, `\\`. Hex encoding doubles the size but is safe and unambiguous.

A typical `/Contents` value looks like `<308206A7…several thousand hex characters…0000000000>`. The leading bytes (`3082…`) are the DER-encoded ASN.1 SEQUENCE tag, and the trailing zeros are padding. Which brings us to one of the more elegant hacks in the PDF specification:

13.1.4.2 Pre-allocation: The Chicken-and-Egg Problem

Here's the fundamental problem with PDF signatures:

1. The signature covers the file's bytes (including the signature dictionary itself)
2. But the signature dictionary must contain the CMS blob
3. But the CMS blob can only be computed *after* you know which bytes to sign
4. But you can't know which bytes to sign until the dictionary is written

The solution is a trick that every PDF signing implementation uses: **pre-allocate** a large enough `/Contents` value filled with zeros, write the file, *then* overwrite the zeros with the actual CMS bytes:

The pre-allocated size must be estimated accurately, and this is where things go wrong in the real world:

- **Too small** — the CMS blob doesn't fit — signing fails. You'll get an exception from PDFBox and no output file. This is the most common "my signing suddenly broke" bug, usually triggered by adding a certificate chain or a timestamp that makes the CMS blob bigger than expected.
- **Too large** — wasted bytes, but the signature is valid. A few KB of zeros at the end of the file never hurt anyone.

Always err on the side of too large. Here are sensible defaults:

Scenario	Hex Size	Bytes
Basic signature (no chain, no timestamp)	`0x2500` (PDFBox default)	~9 KB
Signature with certificate chain	`0x4000`	~16 KB
PAdES-B-T (with RFC 3161 timestamp)	`0x6000`	~24 KB
PAdES-B-LT (embedded OCSP + CRLs)	`0x18000`	~96 KB
PAdES-B-LTA (archival, large CRL)	`0x28000`	~160 KB

I've seen production systems fail because someone used the PDFBox default (`0x2500`) for PAdES-B-T signatures. The first 500 signatures worked fine because the TSA responses were

small. Then the TSA renewed its certificate with a longer chain, the responses grew by 2 KB, and every signing operation started failing on a Friday afternoon. Set it to `0x6000` minimum and forget about it.

13.1.4.3 CMS SignedData: The Cryptographic Core

The CMS blob inside `/Contents` follows RFC 5652 — Cryptographic Message Syntax (CMS) §5. If you've worked with S/MIME email signatures or code signing, this structure will look familiar — it's the same `SignedData` format used across the entire PKI ecosystem:

```
SignedData ::= SEQUENCE {
    version            CMSVersion,
    digestAlgorithms   DigestAlgorithmIdentifiers,
    encapContentInfo   EncapsulatedContentInfo,
    certificates   [0] IMPLICIT CertificateSet OPTIONAL,
    crls           [1] IMPLICIT RevocationInfoChoices OPTIONAL,
    signerInfos        SignerInfos
}
```

- Version: 1 for basic CMS, 3 if attribute certificates are present
- Set of digest algorithms used (typically just `{SHA-256}`)
- Content type: `id-data` (1.2.840.113549.1.7.1) with **absent** content — this is what "detached" means. The signed data (the PDF byte range) is not inside the CMS structure. It's outside, in the PDF file itself.
- Certificates — the signer's certificate and optionally the full chain (intermediates, possibly root). **Always include the full chain.** I know it makes the blob bigger. I know storage is a concern. Do it anyway. A signature without a chain is a signature that can only be validated by someone who already has all your certificates. That's almost nobody.
- CRLs — rarely embedded here in the modern PAdES world. In PAdES-B-LT and later, revocation data goes in the DSS dictionary instead.
- Signer information — the actual signature value and signed attributes. This is where the cryptography lives.

13.1.4.3.1 SignerInfo: Where the Cryptography Lives

```
SignerInfo ::= SEQUENCE {
    version                   CMSVersion,
    sid                       SignerIdentifier,
    digestAlgorithm           DigestAlgorithmIdentifier,
    signedAttrs           [0] IMPLICIT SignedAttributes OPTIONAL,
    signatureAlgorithm        SignatureAlgorithmIdentifier,
    signature                 SignatureValue,
    unsignedAttrs         [1] IMPLICIT UnsignedAttributes OPTIONAL
}
```

- Identifies the signer's certificate — either by issuer DN + serial number, or by subject key identifier. Both work. Issuer+serial is more traditional; subject key identifier is more robust when certificates are reissued.
- Hash algorithm used to digest the PDF byte range. SHA-256 is the minimum acceptable. SHA-384 or SHA-512 if you're paranoid or work in defense.

- **Signed attributes** — authenticated metadata. See the critical attributes table below. These are *signed*, meaning any modification invalidates the signature.
- Signature algorithm (e.g., SHA256withRSA, SHA256withECDSA). Note: the *signature* is computed over the DER-encoded signed attributes, not directly over the PDF bytes. This is a CMS subtlety that trips up many implementers.
- The actual cryptographic signature bytes.
- **Unsigned attributes** — unauthenticated extensions. Timestamps go here.

13.1.4.3.2 The Signed Attributes That Matter The first two attributes are required by CMS whenever signed attributes are present. The remaining attributes add PAdES compliance and defense-in-depth.

Table 13.14: Required CMS Signed Attributes — RFC 5652 — Cryptographic Message Syntax (CMS) §11

Attribute	OID	Req	Spec Ref
`content-type`	1.2.840.113549.1.9.3	Required	RFC 5652 — Cryptographic Message Syntax (CMS) §11.1
Must be `id-data` (1.2.840.113549.1.7.1). Required by CMS when signed attributes are present. Forgetting this is a classic implementation bug — the signature appears valid in some tools but fails in others.			
`message-digest`	1.2.840.113549.1.9.4	Required	RFC 5652 — Cryptographic Message Syntax (CMS) §11.2
SHA-256 hash of the PDF byte range. This is what's actually verified during validation. If this hash doesn't match the file bytes, the signature is invalid. Full stop.			

Table 13.15: PAdES/CAdES Signed Attributes

Attribute	OID	Req	Spec Ref
`signing-certificate-v2`	1.2.840.113549.1.9.16.2.47	Required (PAdES)	RFC 5035 §3
Hash of the signer's certificate (SHA-256). **Required for PAdES / CAdES.** This single attribute prevents certificate substitution attacks — an attacker can't swap in a different certificate without breaking the signature. It's ~50 bytes. There is no reason not to include it.			
`signing-time`	1.2.840.113549.1.9.5	Optional	RFC 5652 — Cryptographic Message Syntax (CMS) §11.3
Claimed signing time. Not authoritative — it's whatever the signer's clock said. For authoritative time, you need a timestamp (see unsigned attributes). Optional, but almost everyone includes it.			
`algorithm-protection`	1.2.840.113549.1.9.52	Recommended	RFC 6211
Protects against algorithm substitution attacks by binding the digest and signature algorithms into the signed attributes. Good hygiene to include.			

The `signing-certificate-v2` attribute is what makes the difference between `/adbe.pkcs7.detached` and `/ETSI.CAdES.detached`.

Without it: plain CMS (PKCS#7) — vulnerable to certificate substitution attacks. With it: CAdES — the signer's certificate is bound to the signature by hash.

This single attribute is the gateway to PAdES compliance. It's about 50 bytes in the CMS structure. There is no reason not to include it, even if you're using `/adbe.pkcs7.detached` as your SubFilter.

Certificate substitution attacks are not theoretical. In 2019, researchers at Ruhr University Bochum demonstrated practical attacks against multiple PDF viewers that exploited the absence of certificate binding. The fix was trivially simple: check `signing-certificate-v2`. Several major viewers had to be patched.

13.1.4.3.3 Unsigned Attributes: The Afterthought That Became Essential

Table 13.16: Unsigned Attributes — RFC 5652 — Cryptographic Message Syntax (CMS) §11

Attribute	OID	Req	Spec Ref
`signature-time-stamp`	1.2.840.113549.1.9.16.2.14	Required (PAdES-B-T)	RFC 3161 — Internet X.509 PKI Time-Stamp Protocol (TSP), ETSI EN 319 142-1 V1.1.1 (2016-04) §5.3
RFC 3161 timestamp token over the signature value. Provides authoritative proof of signing time from a trusted TSA. This is the only way to prove *when* a signature was made. The timestamp token has its own signature from the TSA, making it self-authenticating — which is why it's safe as an unsigned attribute.			
`certificate-values`	1.2.840.113549.1.9.16.2.23	Optional	RFC 5126 §5.6.1
Full certificate chain values. Used in CAdES-X-Long. In the PAdES world, these typically go in the DSS dictionary instead (see ???).			
`revocation-values`	1.2.840.113549.1.9.16.2.24	Optional	RFC 5126 §5.6.1
OCSP responses and CRLs. Used in CAdES-X-Long. In PAdES, these go in the DSS dictionary instead.			

The timestamp attribute deserves special attention. Unsigned attributes are, by definition, not protected by the signature — they can be added after signing without invalidating anything. This is *by design*. A timestamp proves that the signature existed at a certain time, and it's intentionally added after the signature is created (because you need the signature value to timestamp it).

This is also why you should never put anything security-critical in unsigned attributes that isn't self-authenticating (like a timestamp token, which has its own signature from the TSA).

13.1.5 The Signing Process: Step by Step

The process of creating a PDF digital signature is intricate because the signature must cover bytes that include the signature dictionary itself — a chicken-and-egg problem solved by the

`/Contents` pre-allocation we discussed above.

Here's the full sequence, and understanding it is essential for debugging:

13.1.5.1 Step-by-Step Detail

Step	Detail
1	**Create or locate the signature field.** If the document already has an unsigned field (`/FT /Sig` with no `/V`), use it. Otherwise, create a new field in the AcroForm `/Fields` array with a widget annotation on the desired page. *Practical note:* if the document was prepared by someone else (a template), always prefer using existing unsigned fields. Creating new fields in a document that already has signatures is technically possible via incremental save, but it makes some validators nervous.
2	**Build the signature dictionary** with placeholder values. `/ByteRange` is set to `[0 0 0 0]` (will be overwritten). `/Contents` is filled with `00` bytes at the estimated maximum size. All other keys (`/Filter`, `/SubFilter`, `/M`, `/Name`, `/Reason`, `/Reference`) are set to their final values *now* — they become part of the signed bytes and can't change later.
3	**Write the PDF using incremental save.** The new objects (signature dict, updated field, updated xref) are appended after the existing `%%EOF`. This is critical — a full rewrite would invalidate any existing signatures. *This is non-negotiable.* Even if the document has no prior signatures, use incremental save. It's the only safe habit.
4	**Compute actual byte offsets.** After writing, the exact byte positions of the `/Contents` hex string are known. Update `/ByteRange` to `[0, contentsStart, contentsEnd, fileLength - contentsEnd]`. The two segments cover everything *except* the `/Contents` value itself.
5	**Read the signed bytes.** Concatenate `file[0 .. contentsStart-1]` and `file[contentsEnd .. fileLength-1]`. These are the exact bytes the signature covers — everything except the `/Contents` hex value. *Debug tip:* if you suspect byte range issues, dump these bytes to a file and hash them manually. Compare with the `message-digest` attribute in the CMS.
6	**Hash the signed bytes.** Compute SHA-256 (or SHA-384/512) over the concatenated byte ranges. This hash becomes the `message-digest` signed attribute in the CMS structure.
7–8	**Sign the hash.** The private key (RSA or ECDSA) signs the DER-encoded signed attributes (which include the hash). This may happen locally or via a remote HSM/signing service. In the remote case, only the hash leaves your system — the private key never does. More on remote signing in ???.
9	**Build the CMS `SignedData` structure.** Include: digest algorithm, signer certificate, certificate chain, signed attributes (content-type, message-digest, signing-certificate-v2), signature value. DER-encode the result.

Step	Detail
10	**Write CMS bytes into `/Contents`.** Hex-encode the DER bytes and write them at the exact byte offset of the `/Contents` value in the file. Remaining space is left as `00` padding. No other bytes in the file are modified.

Step 10 is a **raw byte write** — not a PDF object update. The file is opened as a random-access binary stream, the cursor is positioned at the `/Contents` hex string's start offset, and the hex chars are overwritten in place. If the CMS blob is larger than the pre-allocated space, signing fails. If even a single byte is written at the wrong offset, the signature is invalid and the PDF may be corrupted. This is why you don't implement PDF signing from scratch — you use PDFBox or a similar library that handles the byte-level bookkeeping for you.

13.1.6 Incremental Save: The Foundation of Multi-Signature

Every signature operation uses **incremental save** (ISO 32000-2:2020 §7.5.6). The original file bytes are never modified — new objects are appended after the existing `%%EOF`.

This is the mechanism that makes multiple independent signatures possible. Without incremental save, adding a second signature would modify the bytes covered by the first signature, invalidating it. With incremental save, the first signature's bytes remain untouched — the second signature's data is simply appended at the end.

Each signature's `/ByteRange` covers everything up to and including its own incremental update — but **not** the updates that come after:

- Signature 1's byte range: `[0, …, …, M]` — covers original + update #1
- Signature 2's byte range: `[0, …, …, P]` — covers original + update #1 + update #2

This is why Signature 1 remains valid after Signature 2 is added: Signature 2's bytes are **appended after** the range that Signature 1 covers. From Signature 1's perspective, those bytes are "trailing" — present in the file but outside the signed range.

"The file has been modified since signing" — don't panic.

This is the single most common false alarm in PDF signature validation. A validator detects trailing bytes after a signature's byte range and flags it as "modified." But in the vast majority of cases, those trailing bytes are simply a later incremental update — another signature, a form fill, or an annotation.

What to check:

1. Is the trailing data a valid incremental update? (Look for xref table + trailer + %%EOF.)
2. Does it correspond to an expected operation? (Another signature, a form field being filled.)
3. Did the byte range change? (The original signature's bytes at their original offsets should be byte-for-byte identical.)

If the answers are yes, yes, and no — the signature is fine. The "modification" is just the normal operation of PDF's append-only architecture.

When to actually worry:

- Trailing bytes that aren't a valid incremental update — raw data appended to the file
- An incremental update that modifies objects covered by an existing signature in unexpected ways (e.g., page content changes)
- Multiple %%EOF markers where the trailing section isn't a proper PDF update

13.1.7 Widget Annotations and Page Placement

Every signature field has an associated widget annotation (ISO 32000-2:2020 §12.5.6.19) that defines its visual presence on a page.

13.1.7.1 The `/Rect` Coordinate System

The widget's `/Rect` array specifies `[llx, lly, urx, ury]` in **default user space** coordinates:

PDF coordinates have **origin at lower-left** with Y increasing upward. This is the opposite of screen coordinates (origin top-left, Y increasing downward). This will bite you. It bites everyone. You will write the conversion wrong the first time. Here it is, so you can copy it:

```
// viewport = page.getViewport({ scale: scale })
var left = bbox[0] * scale;
var top  = viewport.height - (bbox[1] + bbox[3]) * scale;
var width = bbox[2] * scale;
var height = bbox[3] * scale;
```

- X translates directly (both coordinate systems have X increasing rightward)
- Y must be flipped: PDF Y=0 is bottom, screen Y=0 is top. Note the `bbox[1] + bbox[3]` — that's lower-left-Y plus height, giving the top edge in PDF space, which maps to the top edge in screen space.
- Width scales linearly
- Height scales linearly

Where `bbox` is `[lowerLeftX, lowerLeftY, width, height]` as extracted by PDFBox's `PDRectangle`. Note that PDFBox gives you `[x, y, width, height]` while the raw PDF `/Rect` is `[llx, lly, urx, ury]` — PDFBox does the conversion for you. Don't convert twice.

13.1.7.2 Invisible Signatures

A signature can be **invisible** — having no visual appearance on the page:

- **Zero-size rect:** `/Rect [0 0 0 0]` — the widget exists but has no area
- **No widget at all:** The field has no annotation entry (valid per spec, but some viewers complain)

Invisible signatures are common for:

- Certification signatures — the document author certifies the document without cluttering the layout
- Machine-generated signatures — automated workflows where nobody's looking at the PDF
- Counter-signatures — additional approval signatures in complex chains where visual space is limited

Should your signature be visible or invisible?

For approval signatures intended for human signers: make them visible. Users expect to *see* their signature on the page. A visible signature provides a visual anchor — ”I signed here” — and helps in audit scenarios.

For certification signatures and automated workflows: invisible is usually fine. The certification status is shown in the viewer’s signature panel, not on the page.

For machine-to-machine: always invisible. No human is looking at the page. Adding a visual appearance just wastes space and processing time.

One exception: if you’re in a regulated industry where auditors review printed copies of signed PDFs (yes, this still happens in 2026), visible signatures are essential because auditors can’t check digital signatures on paper. In that case, include the signer’s name, the signing time, and ideally a note like ”Digitally signed — verify electronically.”

13.1.8 AcroForm `/SigFlags`: Document-Level Signature Metadata

> If the document contains signature fields, the AcroForm dictionary shall include a SigFlags entry.
>
> — ISO 32000-2:2020 §12.7.2, Table 225

Table 13.17: SigFlags Bit Values — ISO 32000-2:2020 §12.7.2, Table 225

Name	Bit	Req	Spec Ref
`SignaturesExist`	1 (value 1)	Required	§12.7.2, Table 225
At least one signature field has a value. Readers should present signature-specific UI (e.g., the blue bar at the top of Adobe Acrobat). Must be set whenever any signature field contains a `/V` entry.			
`AppendOnly`	2 (value 2)	Required	§12.7.2, Table 225
The document **shall** only be saved incrementally. A conforming reader that allows editing **must not** do a full save — it must append. This is the single most important flag for signature preservation. Without it, a naive editor may do a full rewrite and destroy every signature.			

In practice, both bits should be set (`/SigFlags 3`) whenever the document contains any signed signature field. This was already discussed above, but it bears repeating: if you get this wrong, signatures will be destroyed by innocent save operations, and the blame will be on your tool.

13.1.9 PDFBox API Mapping

For those of you who skipped here — I don’t blame you, this is the section with the code — here’s how the spec concepts map to PDFBox 3.x classes:

PDF Concept	PDFBox Class	Key Methods
Signature field	`PDSignatureField`	`getSignature()`, `getWidgets()`, `getPartialName()`
Signature dictionary	`PDSignature`	`getByteRange()`, `getContents(byte[])`, `getSubFilter()`, `getName()`, `getSignDate()`
AcroForm	`PDAcroForm`	`getFields()`, `getFieldTree()`, `setSignaturesExist()`, `setAppendOnly()`
Widget annotation	`PDAnnotationWidget`	`getRectangle()`, `getPage()`, `getNormalAppearanceStream()`
Appearance stream	`PDFormXObject`	`getStream()`, `getBBox()`, `getResources()`
Adding a signature	`PDDocument`	`addSignature(PDSignature, SignatureInterface, SignatureOptions)`
Incremental save	`PDDocument`	`saveIncremental(OutputStream)`

13.1.9.1 Minimal Signing Code (Key Lines)

The essential PDFBox signing code in ~20 lines. This is the skeleton — the full, production-ready version is in Appendix A, but these are the lines that matter:

```
PDDocument doc = Loader.loadPDF(pdfBytes);
PDAcroForm acroForm = doc.getDocumentCatalog().getAcroForm();

PDSignature signature = new PDSignature();
signature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
signature.setSubFilter(PDSignature.SUBFILTER_ETSI_CADES_DETACHED);
signature.setName("John Doe");
signature.setSignDate(Calendar.getInstance());

SignatureOptions options = new SignatureOptions();
options.setPreferredSignatureSize(0x6000);
options.setVisualSignature(appearanceStream);

doc.addSignature(signature, signingInterface, options);
doc.saveIncremental(outputStream);
```

- Always load from bytes. Don't create a new `PDDocument()` and try to `saveIncremental()` — PDFBox 3.x will throw `IllegalStateException: document was not loaded from a file or a stream`. Yes, this error message has confused every single person who's ever encountered it.
- Filter is always `Adobe.PPKLite` for interoperability. Don't be creative here.

- SubFilter: use `ETSI.CAdES.detached` for PAdES compliance. See the earlier section for why.
- Pre-allocate 24KB for the CMS blob. **Not** the PDFBox default of 9KB. See the pre-allocation table above.
- Visual appearance — set this to `null` for invisible signatures. Or provide an `InputStream` of a pre-built appearance.
- Register the signature and the `SignatureInterface`. The interface's `sign()` method will be called during `saveIncremental()`.
- **Incremental save.** There is no other option. Do not call `save()` on a document you're signing.

13.1.9.2 The `SignatureInterface`: Your Extension Point

```
public interface SignatureInterface {
    byte[] sign(InputStream content) throws IOException;
}
```

- Called by PDFBox during `saveIncremental()`. The `InputStream` provides the concatenated byte ranges — this is the data you need to hash and sign. Your implementation builds the CMS `SignedData` blob and returns the DER-encoded bytes.

This is the **only thing you need to implement**. PDFBox handles the PDF structure, the byte range calculation, the incremental save, the widget creation, the appearance stream embedding. You handle the cryptography.

Keep your `SignatureInterface` implementation stateless and testable.

Don't put PDF logic in there. Don't reference the `PDDocument`. Your `sign()` method receives an `InputStream` of bytes and returns CMS bytes. That's it. This makes it trivially testable — pass in any byte stream, check that the output is valid CMS. It also makes it easy to swap between local signing (BouncyCastle) and remote signing (HSM, cloud) without touching any PDF code.

13.1.10 Common Mistakes and How to Avoid Them

After years of working with PDF signatures, the same problems come up over and over. Here's the hit list:

#	Mistake	Fix
1	**Trusting `/Name` for signer identity.** Many systems display the `/Name` field from the signature dictionary as if it's the verified signer name. It's not. It's free-text set by whoever controls the signing process.	Always extract the signer's identity from the X.509 certificate in the CMS structure. Display the certificate's Subject CN (or the full Subject DN for enterprise contexts).

#	Mistake	Fix
2	**Using `PDDocument.save()` instead of `saveIncremental()`.** Full save rewrites the entire file, destroying all existing signatures.	Always use `saveIncremental()`. Make it a code review checklist item: "Does this code path call `save()` on a signed document? Reject."
3	**Pre-allocating too little space for `/Contents`.** Works fine in testing with simple certificates, breaks in production when chains get longer or timestamps are added.	Use `0x6000` (24KB) minimum. For PAdES-B-LT, use `0x18000` (96KB). Test with your actual TSA and certificate chain, not with test certificates.
4	**Forgetting `signing-certificate-v2` in the CMS attributes.** Technically valid as PKCS#7, but not PAdES-compliant and vulnerable to certificate substitution.	Use BouncyCastle's `JcaSignerInfoGeneratorBuilder` with the correct `DigestCalculatorProvider`. It adds the attribute automatically when configured correctly. See the code in Appendix A.
5	**Not including the certificate chain.** A signature with just the signer's certificate can't be validated by anyone who doesn't already have the issuer certificates.	Always include the full chain (signer + intermediates). You can omit the root CA — validators typically have their own trust stores. But include everything else.
6	**Creating a fresh `PDDocument` and calling `saveIncremental()`.** PDFBox 3.x requires the document to have been loaded from bytes.	Create a blank PDF, save it to bytes, load it back, then sign. See the test code in this chapter.
7	**Setting `/SigFlags` to 1 instead of 3.** Bit 1 alone means "signatures exist" but doesn't enforce append-only saving.	Always set `/SigFlags` to 3. No exceptions.
8	**Modifying the PDF between adding the signature and calling `saveIncremental()`.** Any change to the document graph between `addSignature()` and `saveIncremental()` may change byte offsets, breaking the signature.	Set up everything before calling `addSignature()`. Once you've added the signature, the very next call should be `saveIncremental()`. Nothing in between.

13.1.11 Key Points

- A PDF signature = signature field + signature dictionary + widget + appearance stream — all embedded in the document
- The signature dictionary's `/Contents` holds a CMS `SignedData` blob (hex-encoded, zero-padded)

- `/ByteRange` defines exactly which file bytes are signed — everything except `/Contents` itself
- `/SubFilter` determines the signature format: use `/ETSI.CAdES.detached` for PAdES compliance — there's no good reason not to
- The `signing-certificate-v2` attribute is what distinguishes CAdES from plain PKCS#7 — and what prevents certificate substitution attacks
- All signatures use incremental save — original bytes are never modified, ever
- `/SigFlags` must be set to 3 (SignaturesExist + AppendOnly) to protect existing signatures — getting this wrong destroys other people's signatures
- Seed value dictionaries (`/SV`) can constrain how a field is signed — use them in templates to prevent signer mistakes
- Widget `/Rect` uses PDF coordinates (origin lower-left, Y up) — you will get the conversion wrong the first time, and that's OK
- PDFBox's `SignatureInterface.sign(InputStream)` is the single extension point for all signing logic — keep it clean, stateless, and testable
- When in doubt, pre-allocate more space for `/Contents` — wasted bytes are cheap, failed signatures are expensive
- "The file has been modified since signing" is usually not a problem — it's usually just a later incremental update

13.1.12 Specification References

Chapter 14

Byte Range Deep Dive

The byte range is where PDF signatures live or die. Everything in Chapter 12 — the `/Sig` dictionary, the `/SubFilter`, the certificate chain, the CMS structure — all of it is meaningless if the byte range is wrong. A single off-by-one error in the `/ByteRange` array, and the signature is cryptographically invalid. A subtle gap in coverage, and an attacker can inject content that the signature appears to endorse.

This chapter takes the byte range apart byte by byte. We will trace exactly which bytes are signed, where the gap is, why the gap exists, what lives in the gap, and what goes wrong when any of these assumptions are violated. We will look at hex dumps of real PDFs, visualize the byte range geometry, walk through multi-signature scenarios, and examine the exact attacks that exploit byte range weaknesses.

If Chapter 12 taught you *what* a byte range is, this chapter teaches you *how* it actually works in the raw file — and how to verify that it works correctly.

14.1 The Byte Range Model

14.1.1 What the Spec Says

ISO 32000-2:2020 §12.8.1 defines the `/ByteRange` array as:

> An array of pairs of integers (starting byte offset, length in bytes) that shall describe the exact byte range for the digest computation. Multiple discontiguous byte ranges shall be used to describe a digest that does not include the signature value itself.

In practice, every PDF signature uses exactly **one pair of byte ranges** — two segments — with a gap between them. The gap contains the `/Contents` value (the hex-encoded CMS signature). The two segments cover everything *else* in the file.

The `/ByteRange` array always has exactly four integers:

```
/ByteRange [offset1 length1 offset2 length2]
```

Where:

- `offset1` = 0 (always — the signed region starts at the first byte of the file)
- `length1` = number of bytes from the start of the file to the opening < of the `/Contents` hex string
- `offset2` = byte offset immediately after the closing > of the `/Contents` hex string
- `length2` = number of bytes from `offset2` to the end of the file

The math is simple:

- **Signed bytes** = `[0, length1)` + `[offset2, offset2 + length2)`
- **Unsigned gap** = `[length1, offset2)` — this contains only the `/Contents` hex string value
- **Total file size** = `offset2 + length2`

14.1.2 Visualizing the Byte Range

To make this concrete, let's look at a real signed PDF. Suppose we have a signed file of 45,000 bytes with a 32KB (32,768 hex characters = 16,384 bytes) reserved `/Contents` value:

```
File layout (not to scale):

Byte 0                                                        Byte 44,999
|==============================================================|
|  PDF header, objects, pages,    |  /Contents  |  xref, trailer |
|  /Sig dictionary (up to the    |  hex string |  %%EOF         |
|  opening < of /Contents)       |  (the gap)  |                |
|================================|=============|================|
|<---- length1 = 14,000 -------->|<-- gap  --->|<-- length2 -->|
|                                 |  = 16,408   |   = 14,592    |
|                                 |             |                |
|  SIGNED (segment 1)            |  NOT SIGNED |  SIGNED (seg 2)|
|================================|=============|================|

/ByteRange [0 14000 30408 14592]
```

The gap is slightly larger than 16,384 bytes because it includes the < and > delimiters that bracket the hex string, plus the '/Contents ' key prefix. The exact gap size depends on how PDFBox (or whatever library produced the file) serializes the signature dictionary.

14.1.3 Why the Gap Exists: The Chicken-and-Egg Problem

The gap exists because of a fundamental bootstrapping problem:

1. To **compute the signature**, you need to hash the bytes of the file
2. To **write the file**, you need to include the signature (inside `/Contents`)
3. But the signature is *part of* the file — so how can you hash a file that doesn't exist yet?

The solution: **exclude the signature value from the hash**. The `/Contents` hex string is the one part of the file that is *not* covered by the signature. Everything else — the PDF header, all objects, the signature dictionary itself (including `/ByteRange`, `/Filter`, `/SubFilter`, and all other keys), the cross-reference table, and the trailer — is signed.

This means the signing process works like this:

1. Write the entire PDF file, including the `/Sig` dictionary, but leave the `/Contents` value as a placeholder (all zeros)
2. Record the byte offsets of where the placeholder starts and ends
3. Compute the hash over everything *except* the placeholder: `hash = SHA256(segment1 || segment2)`
4. Construct the CMS `SignedData` using this hash
5. Hex-encode the CMS bytes and write them into the placeholder space

6. If the CMS is shorter than the reserved space (it almost always is), pad the remainder with zeros

This is why PDFBox's `SignatureInterface.sign(InputStream)` receives an `InputStream` — that stream contains the concatenation of segment 1 and segment 2 (the bytes *outside* the gap), ready to be hashed.

The `/ByteRange` array itself is part of the signed region. This is critical for security: if an attacker could modify the `/ByteRange` after signing, they could redirect the signature to cover different content. The byte range is committed before the hash is computed, and any change to it invalidates the signature.

14.2 The Gap in Detail

14.2.1 What's Actually in the Gap

Let's look at a hex dump of the gap region. In a properly signed PDF, the gap contains exactly one thing: the hex-encoded CMS `SignedData`, padded with zeros:

```
Byte 14000:  3C                                          -- opening '<'
Byte 14001:  30 38 30 36 30 39 ...                       -- CMS bytes (hex-encoded)
             ...
             ... (several KB of hex characters)
             ...
Byte 28500:  30 30 30 30 30 30 30 30 30 30 30 30 ...     -- zero padding ("000000...")
             ...
Byte 30407:  3E                                          -- closing '>'
```

The hex encoding doubles the size: a 4,096-byte CMS signature becomes 8,192 hex characters. The reserved space is typically 16KB (32,768 hex characters = 16,384 bytes of actual CMS capacity), which is enough for most PAdES-B-B signatures.

PAdES-B-LTA signatures are much larger than PAdES-B-B. A B-LTA signature includes:

- The CMS `SignedData` itself (~2–4 KB)
- The signing certificate and full chain (~2–5 KB)
- The timestamp token from the TSA (~2–4 KB)
- OCSP responses for each certificate in the chain (~1–3 KB each)
- CRL data if OCSP is not available (~5–50 KB)

A B-LTA signature can easily reach 20–30 KB, which requires 40–60 KB of hex-encoded space. If you're producing B-LTA signatures, reserve at least **64 KB** for `/Contents`:

```
// PDFBox: set signature content size (in bytes, before hex encoding)
SignatureOptions options = new SignatureOptions();
options.setPreferredSignatureSize(65536); // 64 KB
doc.addSignature(signature, signingInterface, options);
```

If the CMS exceeds the reserved space, PDFBox throws an `IOException` — the signing fails silently in the sense that no file is produced. This is the #1 cause of "signing worked in development but fails in production" — development certificates have short chains and no timestamps, production certificates have full chains, OCSP responses, and timestamps that exceed the default buffer.

14.2.2 The Zero-Padding Question

The zero padding at the end of the `/Contents` hex string is not part of the CMS. It's just filler — the hex string `00` decodes to a null byte, and CMS parsers stop reading at the end of the DER-encoded ASN.1 structure, ignoring trailing zeros.

Some validators are stricter than others about this padding:

- **Adobe Acrobat**: Ignores trailing zero padding — works fine
- **LibreOffice**: Ignores trailing zeros — works fine
- **Some custom validators**: May compute the hash of the entire `/Contents` hex string (including padding) and fail because the hash doesn't match what the signer computed over just the CMS bytes. This is a bug in the validator, not in the signed PDF.

When writing validation code, always parse the `/Contents` hex string as DER and stop at the end of the ASN.1 structure. Do not hash the raw hex string — decode it to binary, find the end of the DER sequence (the total length indicated in the ASN.1 header), and ignore everything after that.

14.3 Incremental Saves and the Byte Range

14.3.1 How Signing Creates an Incremental Update

When PDFBox signs a PDF, it performs an **incremental save** — it appends new data to the end of the existing file rather than rewriting the entire file. This is not optional for signing; it is how signing must work.

An incrementally saved PDF looks like this:

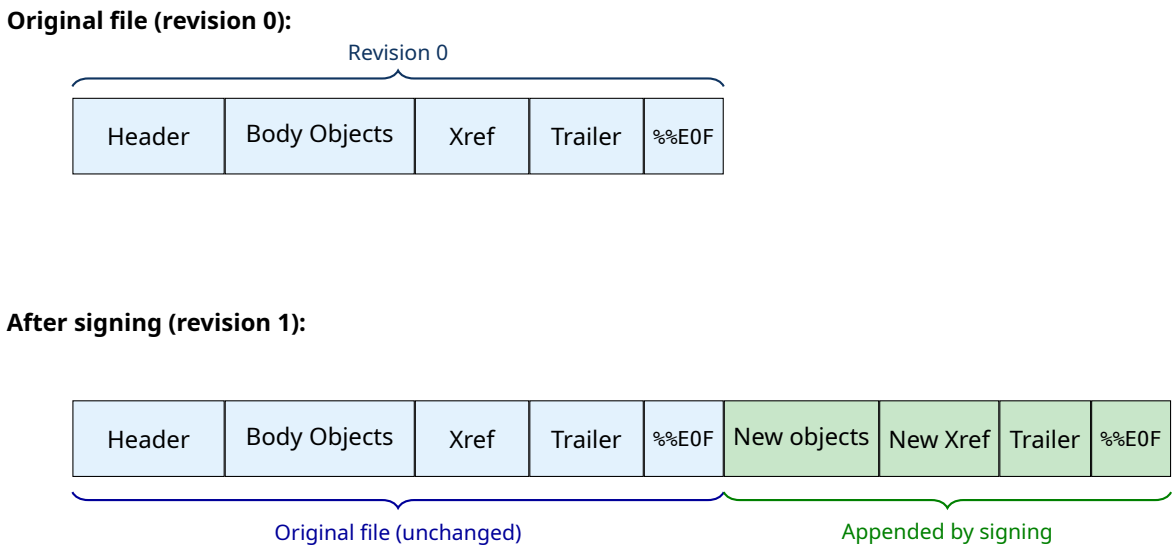


Figure 14.1: PDF file before and after signing — incremental update appends new objects without modifying the original file

The "new objects" include the signature dictionary (or modifications to an existing signature field), any new annotation objects for the visual appearance, and updated AcroForm entries.

The ”new xref” is a cross-reference section that records the locations of these new objects, and the ”new trailer” points to the new xref.

The entire appended region — including the new `%%EOF` — is covered by the byte range of the signature. The byte range starts at byte 0 (the original file’s header) and extends to the end of the appended data, with only the `/Contents` gap excluded.

This means:

- The signature covers the **original file** in its entirety
- The signature covers the **new objects** added during signing (the signature dictionary, appearances, etc.)
- The signature covers the **new cross-reference table** and **new trailer**
- The only thing not covered is the CMS signature value itself

14.3.2 What Happens After Signing: The Trailing Bytes Problem

Here is one of the most security-critical aspects of byte range handling: **what happens to bytes that appear after the signed region?**

Consider this scenario:

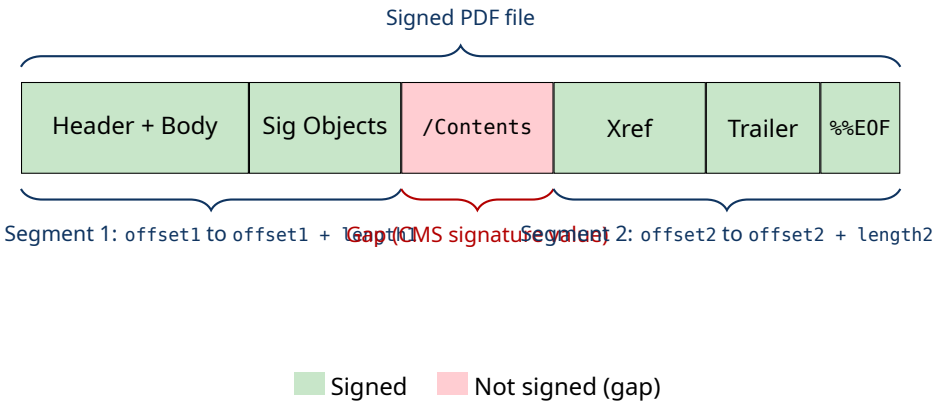


Figure 14.2: Byte range coverage — the signature covers two segments with a gap for `/Contents`

If `offset2 + length2` equals the total file size, the signature covers everything — good.

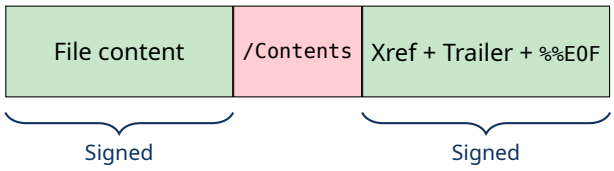
But what if someone appends data *after* the signed region?

The byte range still validates — the hash over the signed segments hasn’t changed. But the file now contains unsigned data at the end. If this unsigned data is a valid incremental update (with its own xref, trailer, and `%%EOF`), a PDF viewer might load it and display the *attacker’s* content while showing the original signature as valid.

This is exactly the **Incremental Saving Attack (ISA)** that the Bochum team demonstrated in 2019 (Chapter 24). The attack works because PDF viewers before the patches would happily load incremental updates appended after the signed byte range, display the new content, and still report the signature as valid — because the signed bytes hadn’t been modified.

The trailing bytes check is the most critical validation step.

After signing:



After tampering (ISA):

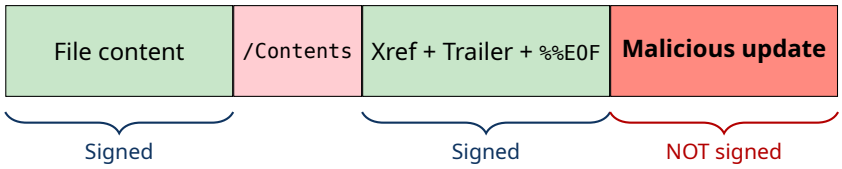


Figure 14.3: Trailing bytes after the signed region — the Incremental Saving Attack surface

When verifying a PDF signature, you **must** verify that:

```
offset2 + length2 == actual file size
```

If the file is larger than the signed region, something was appended after signing. This doesn't necessarily mean the file was tampered with — a second legitimate signature would also append data — but it means you must validate the second revision's signature independently.

If there is unsigned trailing data that is *not* a valid subsequent signature revision, the document has been modified after signing and the signature should be treated as covering only the original content.

Chapter 24 provides the full attack taxonomy and defense code.

14.3.3 Multi-Signature Scenarios

Enterprise PDF workflows often involve multiple signatures — a document might be signed by the author, countersigned by a manager, and certified by a compliance officer. Each signature creates its own incremental update.

The byte range geometry for a multiply-signed PDF looks like this:

Each signature covers everything before its own gap (including previous signatures' gaps — the hex bytes of `/Contents1` and `/Contents2` are part of the signed data for signatures 2 and 3). This nesting means:

- **Signature 1** covers the original PDF + the objects and xref added during the first signing
- **Signature 2** covers everything that signature 1 covers PLUS the objects added during the second signing. This means signature 2 implicitly endorses signature 1's existence (though not necessarily its validity)
- **Signature 3** covers everything, providing the broadest coverage

Each signature is a snapshot of the document at the time it was applied. Signature 1 saw the document in state S1. Signature 2 saw it in state S2 (which includes signature 1). If you need

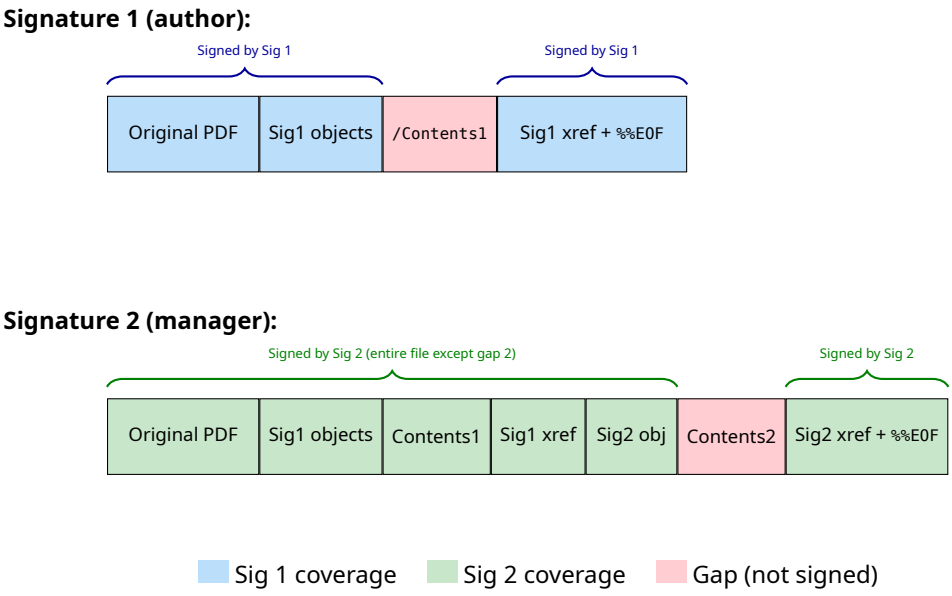


Figure 14.4: Multi-signature byte range nesting — each subsequent signature covers everything except its own gap

to know what the document looked like when signature 1 was applied, you must extract revision 1 by truncating the file at the end of signature 1's byte range — everything after that was added later.

PDFBox provides this capability:

```
// Extract the revision that a specific signature covers
PDDocument doc = Loader.loadPDF(signedFile);
List<PDSignature> signatures = doc.getSignatureDictionaries();
for (PDSignature sig : signatures) {
    int[] byteRange = sig.getByteRange();
    int revisionEnd = byteRange[2] + byteRange[3];

    // Extract this revision
    byte[] revision = new byte[revisionEnd];
    try (InputStream is = new FileInputStream(signedFile)) {
        is.readNBytes(revision, 0, revisionEnd);
    }
    // 'revision' is now the PDF as it existed when this signature was applied
}
```

14.3.4 Permission-Based Restrictions: What Can Change After Signing?

Not all changes after signing are attacks. The PDF specification defines three **modification detection and prevention (MDP)** permission levels that control what changes are allowed after a certification signature:

Table 14.1: DocMDP Permission Levels (ISO 32000-2:2020 §12.8.2.2.2)

Level	Name	Req	Description
P=1	No changes	Optional	No changes whatsoever are allowed. Any modification — including filling form fields or adding annotations — invalidates the signature. This is the most restrictive level.
P=2	Form filling + signing	Default	Users may fill form fields and apply additional signatures. Other changes (adding pages, modifying text, changing form field definitions) invalidate the signature. This is the most common level for government forms.
P=3	Form filling + signing + annotations	Optional	Users may fill form fields, sign, and add annotations (comments, highlights, stamps). This is the most permissive level while still detecting unauthorized modifications.

When validating a signature with MDP restrictions, you must check not only whether the byte range is intact, but also whether the *content* of subsequent incremental updates falls within the allowed permissions. This is where validation gets complex — you need to compare the PDF objects in the signed revision against the current revision and determine whether the differences are permitted.

Adobe Acrobat does this analysis automatically and shows warnings like ”The document has been altered since the signature was applied, but the changes are permitted.” Building this analysis yourself requires understanding of the PDF object model at a level covered in Chapters 7 and 8.

14.4 Size Estimation: How Much Space to Reserve

14.4.1 The Problem

Before you can sign a PDF, you must write it to disk with a placeholder for `/Contents`. The placeholder must be large enough to hold the final CMS signature. If it’s too small, the signing fails. If it’s too large, you waste space (and the file contains kilobytes of zero padding).

The challenge: the exact CMS size depends on:

- **The signing certificate** — larger key sizes (RSA-4096 vs RSA-2048) produce larger signatures
- **The certificate chain** — more intermediates = more bytes
- **The signed attributes** — PAdES requires ESS-signing-certificate-v2, content-type, message-digest; more attributes = more bytes
- **Timestamps** — a TSA response is itself a CMS `SignedData`, typically 2–4 KB

- **OCSP responses** — each certificate in the chain may have a 1–3 KB OCSP response
- **CRL data** — if a certificate's revocation information is provided via CRL instead of OCSP, CRLs can be 5–500 KB depending on the CA's revocation volume

You won't know the exact sizes until you've fetched the timestamp and revocation data — but you need to commit the space *before* computing the hash.

14.4.2 Estimation Strategies

Strategy 1: Fixed generous buffer (simple, recommended)

Reserve a fixed buffer that's large enough for worst-case scenarios:

```
// Signature size estimates by PAdES profile
int estimatedSize;
switch (profile) {
    case B_B:   estimatedSize = 16384;  break; // 16 KB — basic CMS, no timestamp
    case B_T:   estimatedSize = 32768;  break; // 32 KB — CMS + timestamp token
    case B_LT:  estimatedSize = 65536;  break; // 64 KB — CMS + timestamp + OCSP/CRL
    case B_LTA: estimatedSize = 131072; break; // 128 KB — CMS + timestamp + OCSP/CRL + archival TS
}
SignatureOptions options = new SignatureOptions();
options.setPreferredSignatureSize(estimatedSize);
```

This wastes some space (a B-B signature typically needs only 4–8 KB of the 16 KB reserved), but the waste is negligible compared to the content of a typical PDF. A 100 KB PDF with 16 KB of zero padding in `/Contents` is 116 KB — nobody cares.

Strategy 2: Pre-compute exact size (complex, rarely necessary)

If you're generating millions of signed PDFs per day and storage costs matter, you can:

1. Fetch the timestamp and revocation data *before* creating the placeholder
2. Compute the CMS size exactly
3. Reserve only the exact space needed (plus a small safety margin)

This is significantly more complex because you need to construct a "dry run" CMS to measure it, and the timestamp/OCSP data might change between the dry run and the actual signing. In practice, this optimization is almost never worth the complexity.

Just use 64 KB (65536 bytes) as your default. It's enough for PAdES-B-LT in the vast majority of cases, it wastes a trivial amount of space, and it eliminates the entire class of "signature too large for reserved space" failures. Only reduce it if you have a specific reason (embedded systems with extreme storage constraints) and only increase it if you're producing B-LTA signatures with CRLs from large CAs.

14.4.3 What Happens When the Estimate Is Wrong

If the CMS signature is **smaller** than the reserved space: no problem. The remaining space is filled with zero bytes. All conforming validators ignore the padding.

If the CMS signature is **larger** than the reserved space: **the signing fails**. PDFBox throws an `IOException` with a message like:

```
IOException: Can't write signature, not enough space
```

When this happens:

1. The output file is corrupted (partial write) — discard it
2. Increase `SignatureOptions.setPreferredSignatureSize()` and retry
3. To debug: log the actual CMS size from your `SignatureInterface.sign()` method and compare it to the reserved size

```
@Override
public byte[] sign(InputStream content) throws IOException {
    byte[] cmsBytes = createCMSSignature(content);
    log.info("CMS signature size: {} bytes (reserved: {} bytes)",
             cmsBytes.length, reservedSize);
    return cmsBytes;
}
```

14.5 Byte Range Verification: How to Check It

14.5.1 The Verification Algorithm

When validating a signature, the byte range verification follows these steps:

1. **Read the `/ByteRange` array** from the signature dictionary: `[offset1, length1, offset2, length2]`
2. **Verify structural sanity**:
   - `offset1` must be 0
   - `length1` must be positive
   - `offset2` must equal `length1` + gap size (where gap size = the length of the `/Contents` hex string including delimiters)
   - `length2` must be positive
   - `offset2 + length2` should equal the file size (or the size of the revision, for multi-signature documents)
3. **Verify the gap contains only `/Contents`**: the bytes in `[length1, offset2)` should be a hex string containing the CMS signature value
4. **Extract the signed bytes**: concatenate `[0, length1)` and `[offset2, offset2 + length2)`
5. **Compute the hash**: `hash = SHA-256(signed_bytes)` (or whatever digest algorithm the CMS specifies)
6. **Verify the CMS**: parse the CMS `SignedData` from `/Contents`, extract the signed `messageDigest` attribute, and compare it to the hash you just computed. If they match, and the CMS cryptographic verification passes, the byte range is intact.

14.5.2 Implementation in Java

```
/**
 * Verifies the byte range integrity of a PDF signature.
 * Returns true if the byte range covers the expected regions
 * and the hash matches the CMS digest.
 */
public boolean verifyByteRange(byte[] pdfBytes, PDSignature signature)
        throws IOException, CMSException {
```

```
    int[] byteRange = signature.getByteRange();

    // Step 1: Structural checks
    if (byteRange.length != 4) {
        log.error("ByteRange must have exactly 4 elements, got {}", byteRange.length);
        return false;
    }

    int offset1 = byteRange[0];
    int length1 = byteRange[1];
    int offset2 = byteRange[2];
    int length2 = byteRange[3];

    if (offset1 != 0) {
        log.error("ByteRange offset1 must be 0, got {}", offset1);
        return false;
    }

    // Step 2: Check coverage
    int signedEnd = offset2 + length2;
    if (signedEnd != pdfBytes.length) {
        log.warn("ByteRange ends at {} but file size is {} — " +
                 "{} trailing bytes are unsigned",
                 signedEnd, pdfBytes.length, pdfBytes.length - signedEnd);
        // This is a WARNING, not necessarily an error —
        // subsequent signatures would extend the file
    }

    // Step 3: Extract and hash signed bytes
    MessageDigest digest = MessageDigest.getInstance("SHA-256");
    digest.update(pdfBytes, offset1, length1);       // segment 1
    digest.update(pdfBytes, offset2, length2);        // segment 2
    byte[] computedHash = digest.digest();

    // Step 4: Parse CMS and compare
    byte[] cmsBytes = signature.getContents();
    CMSSignedData cmsData = new CMSSignedData(cmsBytes);
    SignerInformation signer = cmsData.getSignerInfos()
            .getSigners().iterator().next();

    // Extract the messageDigest from signed attributes
    Attribute digestAttr = signer.getSignedAttributes()
            .get(CMSAttributes.messageDigest);
    byte[] cmsDigest = ASN1OctetString.getInstance(
            digestAttr.getAttrValues().getObjectAt(0)).getOctets();

    boolean hashMatch = MessageDigest.isEqual(computedHash, cmsDigest);
    if (!hashMatch) {
        log.error("Hash mismatch: computed={} vs CMS={}",
                  Hex.toHexString(computedHash), Hex.toHexString(cmsDigest));
    }
    return hashMatch;
}
```

The code above verifies that the **hash** matches — it does not verify the **CMS signature** (the

RSA/ECDSA computation over the signed attributes). Full verification requires both:

1. Byte range hash matches CMS `messageDigest` attribute (shown above)
2. CMS signature over `signedAttrs` verifies with the signer's certificate

Chapter 31 provides complete verification code that covers both steps plus certificate chain validation.

14.6 Hex Dump Walkthrough: A Real Signed PDF

Let's trace the byte range through an actual signed PDF, byte by byte. This is the kind of analysis you do when debugging — you open a hex editor (or use `xxd`, `hexdump`, or QPDF) and look at the raw bytes.

14.6.1 Finding the Signature Dictionary

First, locate the signature dictionary in the file. With QPDF:

```
qpdf --show-object=15 signed-document.pdf
```

(Where object 15 is the signature dictionary — find it by tracing from the AcroForm Fields [signature field] /V.)

Output:

```
15 0 obj
<<
  /ByteRange [0 14231 47007 1849]
  /ContactInfo ()
  /Contents <3082...0000>
  /Filter /Adobe.PPKLite
  /Location ()
  /M (D:20260308120000+01'00')
  /Name (Test Signer)
  /Reason ()
  /SubFilter /adbe.pkcs7.detached
  /Type /Sig
>>
endobj
```

Now we know:

- **Segment 1**: bytes `[0, 14231)` — 14,231 bytes from file start
- **Gap**: bytes `[14231, 47007)` — 32,776 bytes containing `/Contents`
- **Segment 2**: bytes `[47007, 48856)` — 1,849 bytes at file end
- **Total file size**: 47,007 + 1,849 = 48,856 bytes

14.6.2 Examining the Gap Boundaries

Let's look at the bytes right around the gap:

```
xxd -s 14220 -l 30 signed-document.pdf
```

Output:

```
0000378c: 2f43 6f6e 7465 6e74 7320 3c30 3830 3230  /Contents <08020
0000379c: 3036 3032 3031 3030 3330 3832 3032 3032  060201003082020
```

We can see `/Contents <` at offset ~14,223. The < delimiter is at the byte just before the hex string begins. The `/ByteRange` says segment 1 ends at offset 14,231 — which is the < character. Actually, the exact offset of < depends on how PDFBox serializes the dictionary.

Now the end of the gap:

```
xxd -s 47000 -l 20 signed-document.pdf
```

Output:

```
0000b798: 3030 3030 3030 3030 3e0a 2f46 696c 7465  00000000>./Filte
```

The > delimiter is at byte 47,007 (approximately), followed by a newline and the next dictionary key. Segment 2 starts immediately after the >.

14.6.3 Verifying No Trailing Bytes

```
wc -c signed-document.pdf
# Output: 48856 signed-document.pdf
```

Byte range says: offset2 + length2 = 47,007 + 1,849 = 48,856. File size = 48,856. They match — no trailing unsigned bytes. The signature covers the entire file (minus the gap).

14.7 Common Byte Range Problems

14.7.1 Problem 1: Trailing Bytes After Signed Region

Symptom: Signature validates cryptographically, but the file contains content not covered by the signature.

Cause: Something was appended to the file after signing — possibly a legitimate incremental update (second signature, form fill), or possibly an Incremental Saving Attack.

Detection:

```
int signedEnd = byteRange[2] + byteRange[3];
int fileSize = pdfBytes.length;
if (signedEnd < fileSize) {
    int trailingBytes = fileSize - signedEnd;
    log.warn("{} bytes of unsigned data after signed region", trailingBytes);
    // Analyze the trailing data to determine if it's a legitimate revision
}
```

Resolution: Parse the trailing data. If it contains a valid cross-reference section and trailer pointing to a subsequent signature, it's likely a legitimate multi-signature document. If it contains modified objects (changed page content, replaced fonts, new pages), it may be an attack. Chapter 24 provides detailed analysis code.

14.7.2 Problem 2: `/Contents` Size Exceeded

Symptom: `IOException: Can't write signature, not enough space` during signing.

Cause: The CMS `SignedData` (after hex encoding) exceeds the reserved `/Contents` space.

Fix: Increase `SignatureOptions.setPreferredSignatureSize()`. See the size estimation section above for recommended values by PAdES profile.

14.7.3 Problem 3: ByteRange Doesn't Start at Zero

Symptom: Validator reports "invalid byte range" or "signature does not cover document header."

Cause: Extremely rare — would indicate a buggy signing library. The spec requires that the signed region start at byte 0 to cover the PDF header.

Resolution: This is unfixable after the fact — the PDF must be re-signed.

14.7.4 Problem 4: Gap Contains More Than Just `/Contents`

Symptom: Validator reports "unexpected data in signature gap" or hash mismatch.

Cause: The bytes between the two signed segments should contain *only* the `/Contents` hex string. If any other data exists in this gap — for example, if a tool inserted extra whitespace or a stray dictionary key — the byte range geometry is broken.

Detection: Parse the gap region and verify it matches the pattern `<[0-9A-Fa-f]+>` with optional zero padding.

14.7.5 Problem 5: Hash Algorithm Mismatch

Symptom: Computed hash doesn't match CMS `messageDigest` attribute, even though the byte range looks correct.

Cause: You're hashing with a different algorithm than the one specified in the CMS. The CMS `SignerInfo.digestAlgorithm` tells you which algorithm was used — it might be SHA-256, SHA-384, SHA-512, or (for legacy documents) SHA-1.

Fix: Always read the digest algorithm from the CMS before computing the hash:

```
SignerInformation signer = cmsData.getSignerInfos().getSigners().iterator().next();
String digestAlgOid = signer.getDigestAlgOID();
// Map OID to algorithm name
String algName = switch (digestAlgOid) {
    case "2.16.840.1.101.3.4.2.1" -> "SHA-256";
    case "2.16.840.1.101.3.4.2.2" -> "SHA-384";
    case "2.16.840.1.101.3.4.2.3" -> "SHA-512";
    case "1.3.14.3.2.26"          -> "SHA-1";   // deprecated
    default -> throw new IllegalArgumentException("Unknown digest OID: " + digestAlgOid);
};
MessageDigest digest = MessageDigest.getInstance(algName);
```

14.8 Key Takeaways

- The byte range divides a signed PDF into exactly **two signed segments** with **one unsigned gap** between them

- The gap contains only the hex-encoded CMS signature value (the `/Contents` string) — nothing else
- `offset2 + length2` must equal the file size; any trailing bytes are unsigned and potentially malicious
- Each signature in a multi-signature document covers all previous signatures, creating a nested coverage model
- Always reserve generous space for `/Contents` — 64 KB is a safe default for most PAdES profiles
- The `/ByteRange` array itself is signed — it cannot be modified after signing without detection
- Byte range verification is the **first step** of signature validation — if it fails, there is no need to check the CMS
- The Incremental Saving Attack (ISA) exploits the trailing bytes problem; the trailing bytes check is your primary defense

Chapter 15

Approval vs. Certification Signatures

PDF digital signatures come in two fundamentally different flavors, and confusing them is one of the most common mistakes in signature implementation. An **approval signature** says ”I have reviewed and agree with this document.” A **certification signature** says ”I am the author of this document, and I am defining what changes are allowed after I sign it.”

The difference is not just semantic — it is structural. A certification signature carries a `/DocMDP` (Modification Detection and Prevention) transform that defines permission levels, restricts what subsequent signers and form fillers can do, and triggers different validation behavior in viewers. Getting this wrong means your signatures either restrict too much (form fields that can’t be filled after signing) or too little (no protection against unauthorized modifications).

This chapter explains both signature types, the DocMDP permission model, Lock and Field-MDP dictionaries, and the interaction rules that govern what happens when multiple signatures coexist.

15.1 The Two Signature Types

15.1.1 Approval Signatures

An **approval signature** is what most people think of when they hear ”digital signature.” It means: ”I, the signer, have reviewed the document and give my approval.”

Technically, an approval signature is a `/Sig` dictionary with:

- `/Filter`: `/Adobe.PPKLite`
- `/SubFilter`: `adbe.pkcs7.detached` or `ETSI.CAdES.detached`
- `/ByteRange`: covers the document up to this point
- No `/TransformMethod` in any associated signature reference dictionary

Approval signatures have these properties:

- **Multiple allowed**: A document can have any number of approval signatures. Each one adds an incremental update.
- **No permission restrictions**: An approval signature does not limit what subsequent users can do with the document. After you apply an approval signature, someone else can add another signature, fill form fields, add annotations, or even modify the document (though modifications will cause the signature to show as ”modified since signing”).
- **Order matters**: Each approval signature covers everything up to the point it was applied. Signature 2 covers signature 1. If the document is modified between signature

1 and signature 2, signature 1 will show as ”document modified” while signature 2 will show as ”valid” (because it was applied after the modification).

15.1.2 Certification Signatures

A **certification signature** (also called an ”author signature” or ”MDP signature”) is structurally different. It carries a `/TransformMethod` of `/DocMDP` in its signature reference dictionary, which embeds a set of permission rules into the signature.

Technically, a certification signature includes:

- Everything an approval signature has
- A `/Reference` array in the signature dictionary containing a `/SigRef` (Signature Reference) dictionary
- The `/SigRef` dictionary specifies `/TransformMethod /DocMDP`
- A `/TransformParams` dictionary with a `/P` key (the permission level)

And at the document level:

- The document catalog’s `/Perms` dictionary includes a `/DocMDP` entry pointing to the certifying signature

Certification signatures have these properties:

- **Only one per document**: A document can have at most one certification signature. It is always the first signature applied. If you try to add a certification signature to a document that already has one, the behavior is undefined — some viewers will reject it, others will ignore the previous one.
- **Defines permissions**: The `/P` value (1, 2, or 3) controls what changes are allowed after certification. This is the defining feature that makes certification signatures different from approval signatures.
- **First in the chain**: The certification signature is always the foundational signature. All subsequent signatures are approval signatures that build on the certified baseline.

```
<< /Type /Sig
   /Filter /Adobe.PPKLite
   /SubFilter /adbe.pkcs7.detached
   /ByteRange [0 14231 47007 1849]
   /Contents <3082...>
   /Reference [
     << /Type /SigRef
        /TransformMethod /DocMDP
        /TransformParams << /Type /TransformParams /P 2 /V /1.2 >>
     >>
   ]
   /Name (Document Author)
   /M (D:20260308120000+01'00')
>>
```

The document catalog connects this signature to the document-level permission system:

```
<< /Type /Catalog
```

```
   /AcroForm << ... >>
   /Perms << /DocMDP 15 0 R >>    % Reference to the certifying signature object
   /Pages 3 0 R
>>
```

15.2 The DocMDP Permission Model

15.2.1 Permission Level 1: No Changes Allowed

`/P 1` is the strictest level. After certification with P=1, **no changes whatsoever are allowed** to the document. This means:

- No form fields can be filled
- No additional signatures can be added
- No annotations can be created
- No pages can be added or removed
- Any incremental update invalidates the certification

This level is appropriate for documents that are complete and final — archived records, published regulations, completed certificates. In practice, it is rarely used because most signing workflows require at least form filling or countersigning.

P=1 is so restrictive that it prevents the document from being countersigned. If your workflow requires multiple signatures (author signs, then reviewer countersigns), **do not use P=1**. Use P=2 instead — it allows additional signatures while preventing content modifications.

15.2.2 Permission Level 2: Form Filling and Signing Only

`/P 2` is the **default and most common** permission level. After certification with P=2:

- **Allowed**: Filling form fields, applying additional (approval) signatures, adding signature appearances
- **Not allowed**: Modifying page content, adding pages, removing pages, changing form field definitions (adding/removing fields), modifying JavaScript actions
- Any change beyond form filling and signing invalidates the certification

P=2 is the standard for government forms, contracts, and regulated documents. The document author sets up the form (fields, layout, instructions), certifies it, and then the form recipient fills in the fields and applies their own approval signature — without being able to alter the form structure or content.

This is the level you should use unless you have a specific reason to choose P=1 or P=3.

15.2.3 Permission Level 3: Form Filling, Signing, and Annotations

`/P 3` is the most permissive certification level. After certification with P=3:

- **Allowed**: Everything P=2 allows, plus adding annotations (comments, highlights, stamps, sticky notes, text markup)
- **Not allowed**: Modifying page content, adding or removing pages, changing form field definitions

P=3 is appropriate for review workflows where multiple people need to comment on a certified document without altering its content. Think of a contract that needs review comments from legal, compliance, and management before final signatures.

15.2.4 What "Invalidates the Certification" Actually Means

When we say a change "invalidates the certification," we mean:

- The **byte range** of the certification signature remains intact — the hash still matches
- The **CMS signature** still verifies cryptographically
- But a **conforming viewer** detects that the modifications in subsequent revisions exceed the allowed permissions and displays a warning

In Adobe Acrobat, this shows as: "The document has been altered or corrupted since the Signature was applied." In LibreOffice, it shows as: "Document has been modified since it was last signed."

The exact wording varies by viewer, and — this is important — **not all viewers enforce MDP permissions correctly**. Some viewers only check whether the byte range is intact and don't analyze the incremental updates for permission compliance. This is one of the weaknesses that the Bochum team exploited in the Incremental Saving Attack (Chapter 24).

15.3 Lock Dictionaries: Field-Level Protection

15.3.1 What Lock Dictionaries Do

Beyond document-level MDP permissions, PDF supports **field-level locking** via the `/Lock` dictionary on a signature field. A Lock dictionary specifies which form fields should be locked (made read-only) once this particular signature is applied.

The Lock dictionary lives in the signature field dictionary (not the signature value dictionary), and it applies when the field is signed:

```
<< /Type /Annot
   /Subtype /Widget
   /FT /Sig
   /T (SignatureField1)
   /V 15 0 R               % Reference to the /Sig dictionary
   /Lock <<
     /Type /SigFieldLock
     /Action /Include       % or /Exclude or /All
     /Fields [(FieldA) (FieldB)]
   >>
>>
```

15.3.2 Lock Actions

Table 15.1: Signature Field Lock Actions (ISO 32000-2:2020 §12.8.2.4)

Action	Type	Req	Description
`/All`	name	—	Lock all fields in the document. No `/Fields` array is needed. After this signature is applied, every form field in the document becomes read-only. This is the simplest but most restrictive lock.
`/Include`	name	—	Lock only the fields listed in the `/Fields` array. The listed fields become read-only; all other fields remain fillable. Use this when you want to protect specific fields (e.g., ”Price” and ”Total”) while allowing others (e.g., ”Customer Signature”) to be filled.
`/Exclude`	name	—	Lock all fields **except** those listed in the `/Fields` array. The listed fields remain fillable; everything else is locked. Use this when most fields should be locked but a few specific fields must remain editable (e.g., a ”Date Received” field that an administrator fills after the signer has signed).

15.3.3 The `/Fields` Array

The `/Fields` array in the Lock dictionary contains the **fully qualified field names** of the fields to include or exclude. In a hierarchical field tree, the fully qualified name is the concatenation of parent and child names separated by periods:

```
/Fields [(PersonalInfo.FirstName) (PersonalInfo.LastName) (Signature2)]
```

Field name matching is case-sensitive and must be exact. A typo in the Lock dictionary’s `/Fields` array means the field won’t be locked — and there will be no error message. The signature applies successfully, the Lock dictionary is present, but the misspelled field remains editable.

Always test your Lock configurations by:

1. Signing the document with the Lock dictionary
2. Opening it in Adobe Acrobat
3. Attempting to edit the fields that should be locked
4. Verifying that Acrobat prevents editing and shows the fields as read-only

15.4 FieldMDP: Fine-Grained Modification Detection

15.4.1 FieldMDP vs. DocMDP

While DocMDP controls what categories of changes are allowed (form filling, annotations, etc.), **FieldMDP** controls which specific fields can be modified. It is a more targeted mecha-

nism.

FieldMDP uses the same `/Reference` array mechanism as DocMDP but with `/TransformMethod /FieldMDP`:

```
<< /Type /SigRef
   /TransformMethod /FieldMDP
   /TransformParams <<
     /Type /TransformParams
     /Action /Include
     /Fields [(ProtectedField1) (ProtectedField2)]
     /V /2.0
   >>
>>
```

The `/Action` and `/Fields` keys work identically to Lock dictionary actions — `/Include`, `/Exclude`, and `/All` with the same semantics.

The difference from Lock dictionaries:

- **Lock dictionary**: Controls whether a field is **editable** in the viewer UI (read-only vs. writable). It's a presentation-level restriction — the viewer *should* prevent the user from typing in a locked field.
- **FieldMDP**: Controls whether modifications to specific fields are **detected** during validation. If a FieldMDP-protected field is changed after signing, the signature shows as modified. It's a validation-level mechanism.

In practice, Lock and FieldMDP are often used together: Lock prevents casual editing in a UI, and FieldMDP ensures that any bypass of the Lock (via direct file manipulation) is detected during validation.

15.5 Interaction Rules: When Multiple Signatures Coexist

15.5.1 The Certification + Approval Pattern

The most common multi-signature pattern is:

1. **Author creates** the document with form fields
2. **Author certifies** with P=2 (form filling and signing allowed) and optionally locks specific fields
3. **Recipient fills** form fields
4. **Recipient applies** an approval signature (which adds an incremental update with the filled values and the new signature)

This works because:

- The certification (P=2) allows form filling and additional signatures
- The approval signature is applied to a document that already has a certification
- The certification signature remains valid because the changes (form fill + new signature) are within the P=2 permissions

15.5.2 What Happens When Permissions Conflict

Things get complicated when:

Scenario 1: Approval signature before certification

Technically, any signature can be applied to any document. If someone applies an approval signature first and then a certification signature second, the certification signature covers (signs over) the approval signature. But the certification's MDP permissions only apply to changes *after* the certification — they don't retroactively validate the approval signature's changes.

In practice, this scenario is unusual and may confuse validators. **Always apply the certification signature first.**

Scenario 2: Lock dictionary conflicts with MDP

If the certification says P=2 (form filling allowed) but a Lock dictionary locks all fields — the Lock takes precedence for the locked fields. The MDP permission defines the ceiling (what's *possible*), and Lock dictionaries further restrict within that ceiling.

Scenario 3: Multiple Lock dictionaries from different signatures

Each signature field can have its own Lock dictionary. When multiple signatures with Lock dictionaries coexist:

- Locks are cumulative — each new signature can lock additional fields
- Locks are permanent — once a field is locked by any signature, no subsequent signature can unlock it
- A field is locked if *any* Lock dictionary in the chain locks it

This means the first signer can lock fields A and B, the second signer can lock fields C and D, and after both signatures are applied, fields A, B, C, and D are all locked.

```
// Signature 1 (Author): Lock fields A, B
SignatureField1.Lock = { Action: Include, Fields: [A, B] }

// After Sig1: A=locked, B=locked, C=editable, D=editable

// Signature 2 (Reviewer): Lock fields C, D
SignatureField2.Lock = { Action: Include, Fields: [C, D] }

// After Sig2: A=locked, B=locked, C=locked, D=locked
```

15.6 Implementation: Certification Signatures with PDFBox

15.6.1 Creating a Certification Signature

```
PDDocument doc = Loader.loadPDF(inputFile);

// Create the signature
PDSignature signature = new PDSignature();
signature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
signature.setSubFilter(PDSignature.SUBFILTER_ADBE_PKCS7_DETACHED);
```

```
signature.setName("Document Author");
signature.setSignDate(Calendar.getInstance());

// Set certification (MDP) permissions — P=2 is recommended
doc.addSignature(signature, signingInterface);

// Set the access permissions
// 2 = form filling and signing allowed
AccessPermission accessPermission = new AccessPermission();
accessPermission.setDocMDP(2);
doc.setDocumentPermissions(signature, accessPermission);

doc.saveIncremental(outputStream);
doc.close();
```

PDFBox's API for certification signatures evolved between versions. In PDFBox 2.x, you used `doc.addSignature()` with a `SignatureOptions` and separately configured the MDP via the catalog's `/Perms` dictionary. In PDFBox 3.x, the `AccessPermission` class provides a cleaner API, but the underlying mechanism is the same — writing the `/DocMDP` reference into the catalog.

15.6.2 Adding Lock Dictionaries

Lock dictionaries are configured on the signature field before signing:

```
// Get or create the signature field
PDSignatureField sigField = findOrCreateSignatureField(doc, "AuthorSignature");

// Create lock dictionary
COSDictionary lockDict = new COSDictionary();
lockDict.setName(COSName.TYPE, "SigFieldLock");
lockDict.setName(COSName.getPDFName("Action"), "Include");

COSArray fieldsToLock = new COSArray();
fieldsToLock.add(new COSString("PriceField"));
fieldsToLock.add(new COSString("TotalField"));
fieldsToLock.add(new COSString("ContractTerms"));
lockDict.setItem(COSName.getPDFName("Fields"), fieldsToLock);

// Attach lock dictionary to the signature field
sigField.getCOSObject().setItem(COSName.getPDFName("Lock"), lockDict);

// Now sign the document
doc.addSignature(signature, signingInterface);
doc.saveIncremental(outputStream);
```

15.7 Seed Value Dictionaries: Constraining How a Signature Field Is Signed

15.7.1 Purpose of Seed Values

A **seed value dictionary** (`/SV`) on a signature field constrains *how* the field can be signed. It does not sign anything itself — it provides instructions to the signing tool about what

certificate to use, what algorithm to use, which SubFilter is required, and other signing parameters.

This is the mechanism that government form creators use to ensure that form fillers sign with the correct certificate type, use the correct algorithm, and produce signatures that will pass validation.

15.7.2 Key Seed Value Entries

Table 15.2: Seed Value Dictionary Keys (ISO 32000-2:2020 §12.8.2.3)

Key	Type	Req	Spec Ref
`/Filter`	name	Optional	§12.8.2.3, Table 237
The signature handler to use. In practice, always `/Adobe.PPKLite`. If set, the signing tool should use this handler — if a different handler is selected, the signing tool should warn the user.			
`/SubFilter`	array	Optional	§12.8.2.3, Table 237
Array of acceptable SubFilter values. The signing tool should use one of these SubFilter values. Example: `[/adbe.pkcs7.detached /ETSI.CAdES.detached]` means either detached format is acceptable.			
`/DigestMethod`	array	Optional	§12.8.2.3, Table 237
Array of acceptable digest algorithms. Example: `[/SHA256 /SHA384 /SHA512]`. If set, the signing tool must use one of these algorithms. `/SHA1` is deprecated and should not be included.			
`/Reasons`	array	Optional	§12.8.2.3, Table 237
Array of text strings presenting acceptable signing reasons. The signing tool should let the user choose from this list. Example: `[(I approve this document) (I have reviewed this document) (I am the authorized signatory)]`.			
`/MDP`	dictionary	Optional	§12.8.2.3, Table 237
The MDP parameters dictionary, specifying the default DocMDP permission level for signatures applied to this field.			
`/TimeStamp`	dictionary	Optional	§12.8.2.3, Table 237
Timestamp seed value dictionary specifying a timestamp authority URL and whether a timestamp is required. If the `/Ff` flag is set to 1 (required), the signing tool must include a timestamp from the specified TSA.			
`/Ff`	integer (flags)	Optional	§12.8.2.3, Table 237
Flags indicating which seed values are mandatory (bit 1 set) vs. recommended (bit 1 clear). If a seed value's corresponding Ff bit is set, the signing tool must respect the constraint — failure to do so should prevent the signature from being applied.			
`/LegalAttestation`	array	Optional	§12.8.2.3
Array of legal attestation strings. The signer may be required to accept one of these attestations as part of the signing ceremony.			

15.7.3 Timestamp Seed Values

The `/TimeStamp` sub-dictionary in seed values is particularly important for PAdES compliance:

```
/SV <<
  /TimeStamp <<
    /URL (http://tsa.example.com/timestamp)
    /Ff 1              % 1 = timestamp required, 0 = optional
  >>
  /SubFilter [/ETSI.CAdES.detached]
  /DigestMethod [/SHA256 /SHA384 /SHA512]
  /Ff 1                % All seed values are mandatory
>>
```

This seed value dictionary says: ”When signing this field, you must use CAdES detached format, SHA-256 or better, and you must include a timestamp from `tsa.example.com`. All of these requirements are mandatory (Ff=1).”

A conforming signing tool (Adobe Acrobat, for example) will enforce these constraints automatically. PDFBox-based signing code must read and respect these seed values manually — the library does not enforce them automatically.

Always read the seed value dictionary before signing. If you’re building a signing service that signs arbitrary PDFs, the seed values may impose constraints that your default configuration doesn’t meet. Ignoring them produces a signature that appears valid cryptographically but violates the form author’s intent — and may fail validation in systems that check seed value compliance.

```
PDSignatureField sigField = ...; // the field you're about to sign
COSDictionary sv = sigField.getCOSObject()
    .getCOSDictionary(COSName.getPDFName("SV"));
if (sv != null) {
    // Check SubFilter requirements
    COSArray subFilters = sv.getCOSArray(COSName.getPDFName("SubFilter"));
    // Check DigestMethod requirements
    COSArray digestMethods = sv.getCOSArray(COSName.getPDFName("DigestMethod"));
    // Check Timestamp requirements
    COSDictionary ts = sv.getCOSDictionary(COSName.getPDFName("TimeStamp"));
    if (ts != null) {
        String tsaUrl = ts.getString(COSName.getPDFName("URL"));
        int tsFlags = ts.getInt(COSName.getPDFName("Ff"), 0);
        boolean timestampRequired = (tsFlags & 1) != 0;
    }
}
```

15.8 Real-World Patterns and Mistakes

15.8.1 Mistake 1: Using Certification When Approval Is Needed

Scenario: A developer creates a multi-signature workflow where three people sign sequentially. They make the first signature a certification signature with P=1 (no changes allowed).

Result: The second and third signatures fail or invalidate the certification, because P=1

prohibits additional signatures.

Fix: Use P=2 for the first (certification) signature if additional signatures are needed. Or use approval signatures for all three if no MDP restrictions are needed.

15.8.2 Mistake 2: Not Setting MDP When It's Needed

Scenario: A government form is meant to be filled and signed by the citizen, with no other modifications allowed. The form creator signs it as a plain approval signature without MDP.

Result: The form has no modification restrictions. Anyone can modify the form structure, add fields, remove fields, or alter content — and the original signature only indicates "document has been modified" without specifying what was allowed.

Fix: Use a certification signature with P=2 to allow form filling while preventing structural changes.

15.8.3 Mistake 3: Lock Dictionary with Wrong Field Names

Scenario: The signature is configured to lock fields "Price", "Total", and "Terms". But the actual field names in the PDF are "price_field", "total_amount", and "contract_terms" (because the form designer used different naming).

Result: The Lock dictionary is present and syntactically correct, but no fields are actually locked — the names don't match. There is no error message or warning.

Fix: Enumerate all field names before configuring the Lock dictionary. In PDFBox:

```
PDAcroForm acroForm = doc.getDocumentCatalog().getAcroForm();
for (PDField field : acroForm.getFieldTree()) {
    log.info("Field: {} (type: {})",
             field.getFullyQualifiedName(), field.getFieldType());
}
```

15.8.4 Mistake 4: Assuming All Viewers Enforce MDP

Scenario: A developer relies on MDP P=2 to prevent content modifications, assuming all viewers will refuse to let users edit non-form content after certification.

Result: Adobe Acrobat enforces MDP correctly. Foxit enforces it mostly correctly. LibreOffice enforces it partially. PDFBox (when used as a library) does not enforce MDP at all — it allows any modification programmatically. Browser-based PDF viewers (Chrome, Firefox) often ignore MDP entirely because they don't support editing.

Lesson: MDP is a *validation* mechanism, not an *access control* mechanism. It tells you whether changes were permitted, not whether they were prevented. A determined user with a hex editor or a programmatic tool can always modify a PDF — MDP ensures you can *detect* that the modification occurred.

15.9 Key Takeaways

- **Approval signatures**: "I approve this document." Multiple allowed, no restrictions on subsequent changes, each covers all previous content.

- **Certification signatures**: ”I authored this document and define what can change.” Only one per document, must be first, carries DocMDP permissions (P=1/2/3).
- **P=2 is the default and most common** permission level — allows form filling and additional signatures while preventing content modifications.
- **Lock dictionaries** control which fields become read-only after a specific signature is applied. Cumulative and permanent.
- **FieldMDP** provides validation-level detection of changes to specific fields — complements Lock dictionaries.
- **Seed value dictionaries** constrain how a signature field must be signed — SubFilter, digest algorithm, timestamp requirements.
- MDP is a **detection mechanism**, not a prevention mechanism. Not all viewers enforce it. Always verify compliance programmatically.
- **Always apply certification signatures first**, then approval signatures. The certification establishes the permission framework.

Chapter 16

Visual Appearance of Signatures

A digital signature has two faces. The cryptographic face — the CMS `SignedData`, the byte range, the certificate chain — is what makes the signature *valid*. The visual face — what the user sees in the PDF viewer — is what makes the signature *trustworthy to a human reader*. These two faces are entirely independent, and understanding their independence is critical for both security and usability.

The visual appearance of a PDF signature is an **annotation appearance stream** — a miniature PDF drawing that shows the signer's name, date, reason, and optionally a handwritten signature image, a company logo, or other graphical elements. This appearance stream is part of the signed bytes (it lives inside the signature field's widget annotation), but it is *not* cryptographically bound to the CMS in any meaningful way. A valid CMS signature does not guarantee that the visual appearance accurately represents the signer — it only guarantees that the bytes haven't changed.

This distinction is exactly what Shadow Attacks (Chapter 24) exploit: an attacker can create a PDF where the visual appearance shows one thing to the signer and something different to the recipient, with the signature validating correctly in both cases.

This chapter explains how signature appearances work, how to create them with PDFBox, what the layer model means, and how different viewers handle appearance rendering.

16.1 Appearance Streams: The Basics

16.1.1 What an Appearance Stream Is

Every annotation in a PDF — not just signatures — can have an **appearance stream**. An appearance stream is a Form XObject (a `/Type /XObject /Subtype /Form` stream) that contains the drawing instructions for how the annotation looks.

For a signature widget annotation, the appearance stream is stored under the `/AP` (Appearance) key:

```
<< /Type /Annot
   /Subtype /Widget
   /FT /Sig
   /T (SignatureField1)
   /V 15 0 R                    % the /Sig dictionary
   /Rect [100 600 300 650]      % position on the page (x1 y1 x2 y2)
   /AP <<
     /N 20 0 R                  % Normal appearance stream
   >>
>>
```

The `/N` key holds the **normal** appearance — the one shown when the annotation is not being interacted with. Annotations can also have `/R` (rollover — shown on mouse hover) and `/D` (down — shown when clicked), but signature appearances only use `/N`.

The appearance stream (object 20 in this example) is a Form XObject:

```
20 0 obj
<< /Type /XObject
   /Subtype /Form
   /BBox [0 0 200 50]          % bounding box in the stream's coordinate space
   /Resources << /Font << /F1 21 0 R >> >>
   /Length 142
>>
stream
BT
/F1 10 Tf
0 35 Td
(Digitally signed by: Test Signer) Tj
0 -15 Td
(Date: 2026-03-08 12:00:00 UTC) Tj
0 -15 Td
(Reason: I approve this document) Tj
ET
endstream
endobj
```

This is a complete miniature PDF page: it has a bounding box, font resources, and drawing instructions using the standard PDF content stream operators (`BT`/`ET` for text blocks, `Tf` for font selection, `Td` for positioning, `Tj` for showing text).

16.1.2 Visible vs. Invisible Signatures

Not all signatures need a visual appearance. A signature can be:

Visible: The signature field has a non-zero `/Rect` (annotation rectangle) and an appearance stream. The user sees a signature block on a specific page.

Invisible: The signature field either has a zero-size `/Rect` (`[0 0 0 0]`), or has no appearance stream, or is not associated with any page. The signature exists cryptographically but doesn't appear on any page.

Invisible signatures are common in automated signing workflows where the PDF is being signed programmatically (e.g., a server signing all outgoing invoices) and there's no need for a visual indicator on the document. The signature is still listed in the viewer's signature panel — the user can still inspect it, verify the certificate, and check the validation status.

Even for invisible signatures, some viewers (notably Adobe Acrobat) display a blue banner at the top of the document saying "Signed and all signatures are valid" or similar. This banner is not part of the PDF content — it's the viewer's UI.

16.2 The Layer Model: n0, n2, and n4

16.2.1 Adobe's Legacy Layer System

Adobe Acrobat uses a layer-based approach to compose signature appearances. This system was designed by Adobe for their own implementation and is not formally part of the ISO 32000 specification — but because Adobe Acrobat is the dominant viewer, understanding it matters.

The layers are:

- **n0** (Layer 0 — Background): Contains the background graphic and a static validity indicator. Adobe Acrobat uses this to show the blue "valid" checkmark or the red "invalid" X. This layer is generated and updated by Acrobat itself — it is not typically set by the signing application.
- **n2** (Layer 2 — Content): Contains the signer's information — name, date, reason, location, and optionally a handwritten signature image or company logo. **This is the layer you create** when building a visual signature appearance.
- **n4** (Layer 4 — Text overlay): An optional text overlay, rarely used.

In practice, most signing applications (including PDFBox-based implementations) create a single appearance stream that serves as the entire appearance — they don't use the n0/n2/n4 layer decomposition. Adobe Acrobat, when it encounters a signature without layer-based appearances, creates its own composite appearance using the provided stream as the content layer and adding its own validity indicators.

The layer model is an implementation detail of Adobe Acrobat, not a spec requirement. When you create a signature appearance with PDFBox, you create a single Form XObject for the `/AP /N` entry. Acrobat will overlay its own validity graphics on top. Other viewers (LibreOffice, Foxit, browser-based viewers) each handle the appearance differently — some show your appearance as-is, some add their own decorations.

16.2.2 What This Means for Your Implementation

When building a signature appearance with PDFBox:

1. **Create a single Form XObject** with the signer's information
2. **Don't worry about n0/n2/n4 layers** — the single appearance works across all viewers
3. **Set reasonable fonts and sizes** — the appearance will be rendered in the space defined by the signature field's `/Rect`
4. **Include key information**: signer's name, signing date/time, reason (if applicable)
5. **Test in multiple viewers** — Acrobat, Foxit, LibreOffice, and a browser-based viewer — to ensure the appearance renders correctly in each

16.3 Creating Signature Appearances with PDFBox

16.3.1 Basic Text Appearance

The simplest appearance shows the signer's name and date:

```
private PDFormXObject createSignatureAppearance(PDDocument doc,
        PDRectangle rect, String signerName, Calendar signDate)
        throws IOException {

    PDFormXObject form = new PDFormXObject(doc.getDocument().createCOSStream());
    form.setResources(new PDResources());
    form.setBBox(new PDRectangle(rect.getWidth(), rect.getHeight()));

    PDFont font = new PDType1Font(Standard14Fonts.FontName.HELVETICA);
    form.getResources().put(COSName.getPDFName("F1"), font);

    float fontSize = 8;
    String dateStr = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss z")
            .format(signDate.getTime());

    // Build the content stream manually
    try (OutputStream os = form.getStream().createOutputStream()) {
        String content = String.join("\n",
            "BT",
            "/F1 " + fontSize + " Tf",
            "0 " + (rect.getHeight() - fontSize - 2) + " Td",
            "(" + escapeForPdf(signerName) + ") Tj",
            "0 -" + (fontSize + 2) + " Td",
            "(Date: " + escapeForPdf(dateStr) + ") Tj",
            "ET"
        );
        os.write(content.getBytes(StandardCharsets.US_ASCII));
    }

    return form;
}

private String escapeForPdf(String text) {
    return text.replace("\\", "\\\\")
               .replace("(", "\\(")
               .replace(")", "\\)");
}
```

In PDFBox 3.x, you **cannot** use `PDPageContentStream` to write to a Form XObject. `PDPageContentStream` is strictly for page content. For Form XObjects, you must write raw PDF operators to the stream's output stream, as shown above.

This is one of the breaking changes from PDFBox 2.x, where `PDPageContentStream` could be used with Form XObjects via a constructor overload that accepted a `PDFormXObject` parameter. That constructor no longer exists in 3.x.

16.3.2 Appearance with Signature Image

Many signing workflows include a handwritten signature image — either captured via a canvas on a web page or tablet, or uploaded as a PNG/JPEG file. Adding an image to the appearance requires embedding it as a PDF image XObject:

```
private PDFormXObject createAppearanceWithImage(PDDocument doc,
        PDRectangle rect, String signerName, Calendar signDate,
        byte[] signatureImagePng) throws IOException {
```

```
    PDFormXObject form = new PDFormXObject(doc.getDocument().createCOSStream());
    form.setResources(new PDResources());
    form.setBBox(new PDRectangle(rect.getWidth(), rect.getHeight()));

    // Add font
    PDFont font = new PDType1Font(Standard14Fonts.FontName.HELVETICA);
    form.getResources().put(COSName.getPDFName("F1"), font);

    // Add signature image
    PDImageXObject image = PDImageXObject.createFromByteArray(
            doc, signatureImagePng, "signature");
    form.getResources().put(COSName.getPDFName("Img0"), image);

    // Layout: image on the left, text on the right
    float imgWidth = rect.getWidth() * 0.4f;
    float imgHeight = rect.getHeight() - 4;
    float textX = imgWidth + 5;
    float fontSize = 7;
    String dateStr = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss z")
            .format(signDate.getTime());

    try (OutputStream os = form.getStream().createOutputStream()) {
        StringBuilder sb = new StringBuilder();
        // Draw image
        sb.append("q\n");  // save graphics state
        sb.append(imgWidth).append(" 0 0 ").append(imgHeight)
          .append(" 2 2 cm\n");  // transform matrix
        sb.append("/Img0 Do\n");  // draw image
        sb.append("Q\n");  // restore graphics state
        // Draw text
        sb.append("BT\n");
        sb.append("/F1 ").append(fontSize).append(" Tf\n");
        sb.append(textX).append(" ")
          .append(rect.getHeight() - fontSize - 2).append(" Td\n");
        sb.append("(Digitally signed by:) Tj\n");
        sb.append("0 -").append(fontSize + 2).append(" Td\n");
        sb.append("(").append(escapeForPdf(signerName)).append(") Tj\n");
        sb.append("0 -").append(fontSize + 2).append(" Td\n");
        sb.append("(Date: ").append(escapeForPdf(dateStr)).append(") Tj\n");
        sb.append("ET\n");
        os.write(sb.toString().getBytes(StandardCharsets.US_ASCII));
    }

    return form;
}
```

16.3.3 Attaching the Appearance to the Signature

After creating the appearance Form XObject, attach it to the signature field's widget annotation:

```
// Create signature field and widget annotation
PDSignature signature = new PDSignature();
// ... set signature properties ...
```

```
// Set the visual rectangle on page 1
PDPage page = doc.getPage(0);
PDRectangle rect = new PDRectangle(100, 600, 200, 50);

SignatureOptions options = new SignatureOptions();
options.setPreferredSignatureSize(65536);
options.setPage(0);  // page index (0-based)
options.setVisualSignature(
    createSignatureAppearance(doc, rect, "Test Signer", Calendar.getInstance()));

doc.addSignature(signature, signingInterface, options);
doc.saveIncremental(outputStream);
```

16.4 Appearance Rendering Across Viewers

16.4.1 How Different Viewers Handle Appearances

One of the most frustrating aspects of PDF signature appearances is that different viewers render them differently — even when the appearance stream is identical.

Table 16.1: Signature appearance rendering behavior across major viewers

Viewer	Behavior
Adobe Acrobat	Overlays its own validity indicator (blue checkmark / red X) on layer n0. Your appearance stream is rendered on layer n2. Acrobat may resize your content to fit its layer model. If `/NeedAppearances` is true in the AcroForm, Acrobat may regenerate appearances from scratch.
Foxit Reader	Renders the appearance stream mostly as provided. Adds a small validity icon in the corner. Generally the most faithful rendering of custom appearances.
LibreOffice Draw	Renders the appearance stream as-is. Does not overlay validity indicators on the appearance itself — shows validation status in a separate panel. Good for testing that your raw appearance looks correct.
Chrome (built-in viewer)	Renders appearance streams but does not validate signatures. No validity indicators. The appearance is what you see — useful for checking visual layout.
Firefox (pdf.js)	Renders appearance streams. Does not validate signatures (though experimental validation support has been discussed). Like Chrome, useful for layout testing.
Okular (KDE)	Renders appearances and provides signature validation. May show validity status differently from other viewers.

Always test your signature appearances in at least three viewers — Adobe Acrobat, a non-Adobe desktop viewer (Foxit or LibreOffice), and a browser. Appearances that look

perfect in Acrobat may render differently in other viewers due to:

- Different font fallback behavior (if the font isn't embedded)
- Different handling of the coordinate system and bounding box
- Different overlay/decoration behavior
- Different treatment of transparency and blending

16.4.2 The `/NeedAppearances` Problem

The AcroForm dictionary has a boolean flag `/NeedAppearances` that tells viewers whether to regenerate form field appearances from scratch. If this flag is `true`, some viewers (notably Adobe Acrobat) will ignore the existing appearance streams and generate their own based on the field's value.

For signature fields, this is usually not what you want. If `/NeedAppearances` is true and Acrobat regenerates the signature's appearance, your carefully crafted visual (with the signer's image, custom layout, etc.) gets replaced with Acrobat's default rendering.

Always set `/NeedAppearances` to `false` (or remove the key entirely — the default is `false`) before signing:

```
PDAcroForm acroForm = doc.getDocumentCatalog().getAcroForm();
if (acroForm != null) {
    acroForm.getCOSObject().setBoolean(COSName.NEED_APPEARANCES, false);
}
```

16.5 Positioning: Getting the Rectangle Right

16.5.1 The Coordinate System

PDF uses a **bottom-left origin** coordinate system where (0, 0) is at the lower-left corner of the page. The y-axis goes *up*, not down. This is the opposite of most screen coordinate systems and is a common source of positioning errors.

The signature field's `/Rect` array defines the annotation rectangle as `[x1 y1 x2 y2]`, where:

- `(x1, y1)` is the lower-left corner
- `(x2, y2)` is the upper-right corner

So a signature field at the bottom-right of an A4 page (595 x 842 points) might have:

```
/Rect [395 50 575 100]    % 180pt wide, 50pt tall, near bottom-right
```

A common mistake: specifying the rectangle with screen coordinates (origin at top-left, y going down). If you compute `y = pageHeight - desiredY`, make sure you do this for *both* y1 and y2, and that y1 < y2.

Debug tip: If your signature appearance shows up in the wrong position, check:

1. Is the page rotated? (A `/Rotate` key on the page dictionary changes the visible coordinate system)
2. Is the `MediaBox` the expected size? (Custom page sizes shift everything)
3. Did you flip y1 and y2? (The lower-left corner must have the smaller y value)

16.5.2 Handling Page Rotation

PDF pages can have a `/Rotate` key that specifies a rotation angle (0, 90, 180, or 270 degrees). This rotation affects how the page is *displayed* but does not change the coordinate system of the page content or annotations.

This means that on a 90-degree rotated page (landscape orientation achieved via rotation), the annotation rectangle coordinates are still in the unrotated coordinate space. To place a signature at the visual bottom-right of a rotated page, you need to account for the rotation.

```
PDPage page = doc.getPage(pageIndex);
int rotation = page.getRotation();
PDRectangle mediaBox = page.getMediaBox();

float sigWidth = 200;
float sigHeight = 50;
float margin = 30;

PDRectangle sigRect;
switch (rotation) {
    case 0:   // Normal portrait
        sigRect = new PDRectangle(
            mediaBox.getWidth() - sigWidth - margin,
            margin,
            sigWidth, sigHeight);
        break;
    case 90:  // Landscape (rotated clockwise)
        sigRect = new PDRectangle(
            margin,
            margin,
            sigHeight, sigWidth);  // swap width/height
        break;
    case 180: // Upside-down portrait
        sigRect = new PDRectangle(
            margin,
            mediaBox.getHeight() - sigHeight - margin,
            sigWidth, sigHeight);
        break;
    case 270: // Landscape (rotated counter-clockwise)
        sigRect = new PDRectangle(
            mediaBox.getWidth() - sigHeight - margin,
            mediaBox.getHeight() - sigWidth - margin,
            sigHeight, sigWidth);
        break;
    default:
        sigRect = new PDRectangle(margin, margin, sigWidth, sigHeight);
}
```

16.6 Security Implications of Visual Appearances

16.6.1 The Appearance/Content Disconnect

The visual appearance of a signature is stored in the PDF's object structure, which is covered by the byte range. But the appearance is not cryptographically bound to the *identity* of the

signer. The appearance can show anything — ”Signed by Alice” while the CMS is signed by Bob’s certificate.

A conforming viewer should display the certificate information from the CMS (not from the appearance) in its signature properties panel. But what the user *sees* on the page is the appearance — and most users trust the visual appearance more than the signature properties panel.

This is a deliberate design trade-off in PDF: the signer (or the signing application) controls what the appearance looks like, and the viewer overlays validity information. The validity indicators (checkmarks, status banners) come from the viewer’s own analysis of the CMS — they don’t come from the appearance stream.

16.6.2 Shadow Attacks and Visual Appearances

Shadow Attacks (Chapter 24) exploit the gap between what a signer sees and what the document contains. In the **Replace** variant, the attacker creates a form field with a specific visual appearance, then changes the field’s value (and thus its appearance) after the signer certifies the document. Because the certification allows form filling (P=2 or P=3), the change is permitted — but the visual content of the document has changed dramatically.

The defense is the same as for all Shadow Attacks: validate not just the cryptographic integrity but also the *semantic* integrity — what changed between revisions, and were those changes consistent with the displayed content?

For maximum security: When validating a signed PDF, don’t just check the signature’s validity. Extract the revision that was signed (by truncating the file at the byte range end — see Chapter 13) and compare its visual rendering to the current document. If they differ, the document has been modified since signing — even if the modifications were technically ”permitted” by the MDP level.

16.7 Common Appearance Problems

16.7.1 Problem: Blank Signature Rectangle

Symptom: The signature field shows as an empty box in the viewer.

Causes:

- No appearance stream set (the `/AP` dictionary is missing)
- `/NeedAppearances` is `true`, and the viewer regenerated an empty appearance
- The Form XObject has an empty stream (no drawing instructions)
- The bounding box (`/BBox`) is [0 0 0 0] or has zero dimensions

16.7.2 Problem: Text Appears Clipped or Off-Screen

Symptom: Only part of the signature text is visible, or the text appears outside the signature rectangle.

Causes:

- The text positioning (`Td` operators) exceeds the Form XObject’s `/BBox`

- The font size is too large for the available space
- The `/BBox` doesn't match the annotation's `/Rect` dimensions

Fix: Ensure the Form XObject's `/BBox` width and height match the annotation `/Rect` dimensions, and that all drawing instructions fall within the `/BBox`.

16.7.3 Problem: Font Not Rendering

Symptom: Text appears as squares, dots, or is completely invisible.

Causes:

- The font resource is not declared in the Form XObject's `/Resources`
- The font name in the content stream (`/F1`) doesn't match the resource key
- Non-ASCII characters used with a non-embedded Standard 14 font (Standard 14 fonts only support WinAnsiEncoding)

Fix: For text beyond basic Latin characters (accented letters, CJK, etc.), embed a TrueType or Type0 font. Standard 14 fonts (Helvetica, Times, Courier, etc.) only support a limited character set.

16.8 Key Takeaways

- Signature appearance is an **annotation appearance stream** — a Form XObject with drawing instructions
- The appearance is **visually independent** from the cryptographic signature — they can show different things
- **Invisible signatures** have zero-size rectangles or no appearance stream — they exist cryptographically but don't show on the page
- The **n0/n2/n4 layer model** is Adobe's implementation detail, not a spec requirement — create a single Form XObject for maximum compatibility
- **PDFBox 3.x** requires writing raw PDF operators to Form XObject streams — `PDPageContentStream` only works for pages
- **Always set `/NeedAppearances` to `false`** before signing to prevent viewers from regenerating your custom appearance
- **Test in multiple viewers** — Acrobat, Foxit, LibreOffice, and browsers all render appearances differently
- The **coordinate system** is bottom-left origin with y going up — account for page rotation when positioning
- Visual appearances are a **security surface** — Shadow Attacks exploit the disconnect between appearance and content

Chapter 17

Signature Validation

Signing a PDF is the easy part. Validating a signed PDF — correctly, securely, and in a way that matches what other validators produce — is where the real complexity lives. A signature can fail validation for dozens of reasons, and the error messages from different viewers are often vague or contradictory. Adobe says ”valid.” LibreOffice says ”signature cannot be verified.” Your custom validator throws a `CMSException`. Which one is right?

This chapter walks through the complete validation process, from the first byte range check to the final certificate chain verification. We cover what each validation step checks, what can go wrong at each step, how different viewers handle each step, and how to debug validation failures systematically.

17.1 The Five Steps of Signature Validation

Signature validation is not a single check — it is a pipeline of five independent verifications, each of which must pass. A failure at any step means the signature is invalid (or at least questionable), regardless of whether the other steps pass.

17.1.1 Step 1: Structural Integrity (Byte Range)

Question answered: ”Does the signature cover the expected bytes of the file?”

What to check:

1. The `/ByteRange` array has exactly 4 elements: `[offset1 length1 offset2 length2]`
2. `offset1` is 0 (signature covers from the start of the file)
3. `offset2 + length2` equals the file size (or revision size for multi-signature documents)
4. The gap `[length1, offset2)` contains only the `/Contents` hex string
5. No trailing unsigned bytes exist after the signed region (unless they are a subsequent legitimate revision)

Common failures:

- Trailing bytes after the signed region — possible Incremental Saving Attack (Chapter 24)
- ByteRange doesn’t start at 0 — broken signing implementation
- Gap contains unexpected data — corrupted or manually edited PDF

Chapter 13 covers this step in exhaustive detail.

17.1.2 Step 2: Hash Verification (Digest Match)

Question answered: ”Does the hash of the signed bytes match what the CMS claims was signed?”

What to check:

1. Read the digest algorithm from the CMS `SignerInfo.digestAlgorithm`
2. Compute the hash over the byte range segments: `hash = algo(segment1 || segment2)`
3. Extract the `messageDigest` attribute from the CMS `signedAttrs`
4. Compare the computed hash to the CMS-provided hash

Common failures:

- Hash algorithm mismatch — you computed SHA-256 but the CMS used SHA-384
- Byte range computation error — you read the wrong bytes
- Modified file — someone changed the file after signing

```
// Step 2: Compute and compare hash
String digestAlgOid = signerInfo.getDigestAlgOID();
String algName = mapOidToAlgorithm(digestAlgOid);
MessageDigest digest = MessageDigest.getInstance(algName);

int[] byteRange = signature.getByteRange();
digest.update(pdfBytes, byteRange[0], byteRange[1]);
digest.update(pdfBytes, byteRange[2], byteRange[3]);
byte[] computedHash = digest.digest();

Attribute digestAttr = signerInfo.getSignedAttributes()
        .get(CMSAttributes.messageDigest);
byte[] claimedHash = ASN1OctetString.getInstance(
        digestAttr.getAttrValues().getObjectAt(0)).getOctets();

if (!MessageDigest.isEqual(computedHash, claimedHash)) {
    log.error("Hash mismatch — document was modified after signing");
    return ValidationResult.HASH_MISMATCH;
}
```

17.1.3 Step 3: Cryptographic Verification (CMS Signature)

Question answered: ”Was the CMS `SignedData` actually created by the claimed signer’s private key?”

What to check:

1. Extract the signer’s certificate from the CMS `SignedData.certificates` collection
2. Verify that the `SignerInfo.signatureAlgorithm` matches the certificate’s key type (RSA cert with RSA signature, ECDSA cert with ECDSA signature)
3. Verify the signature over the `signedAttrs` using the signer’s public key
4. Verify that the signed attributes include the required fields: `contentType`, `messageDigest`, and (for PAdES) `ESS-signing-certificate-v2`

Common failures:

- Wrong certificate used for verification — the CMS contains multiple certificates, and you picked an intermediate instead of the signer
- Algorithm mismatch — rare, but can happen with misconfigured BouncyCastle providers
- Missing `ESS-signing-certificate-v2` — not a cryptographic failure, but a PAdES compliance failure

```
// Step 3: CMS cryptographic verification
CMSSignedData cmsData = new CMSSignedData(
        new CMSProcessableByteArray(signedBytes), signature.getContents());

SignerInformationStore signerStore = cmsData.getSignerInfos();
SignerInformation signerInfo = signerStore.getSigners().iterator().next();

// Find the signer's certificate
X509CertificateHolder signerCert = findSignerCertificate(
        cmsData.getCertificates(), signerInfo.getSID());

// Verify the CMS signature
boolean cmsValid = signerInfo.verify(
        new JcaSimpleSignerInfoVerifierBuilder()
                .setProvider("BC")
                .build(signerCert));

if (!cmsValid) {
    log.error("CMS signature verification failed - " +
              "signature was not created by the claimed certificate");
    return ValidationResult.CMS_INVALID;
}
```

17.1.4 Step 4: Certificate Chain Validation

Question answered: ”Is the signer’s certificate trustworthy? Was it valid at the time of signing?”

What to check:

1. Build the certificate chain from the signer’s certificate up to a trusted root
2. Verify each certificate’s signature (each cert is signed by its issuer)
3. Check validity periods — was each certificate valid at the signing time?
4. Check revocation status — was any certificate revoked at or before the signing time?
5. Check key usage — does the signer’s certificate allow digital signatures (`digitalSignature` or `nonRepudiation` key usage)?
6. Check basic constraints — are intermediate certificates properly marked as CA certificates?

Common failures:

- **Self-signed certificate**: The signer’s certificate is not issued by a recognized CA. This is technically valid (the CMS signature will verify) but the certificate is not trusted. Most viewers show ”signature is valid but the identity of the signer could not be verified.”
- **Expired certificate**: The certificate was valid when the signature was created, but has since expired. With timestamps, this is not a problem (the timestamp proves the

signature was created before expiration). Without timestamps, viewers may warn about an expired certificate.

- **Revoked certificate**: A certificate in the chain was revoked (compromised, or the CA issued it in error). Revocation checking requires OCSP or CRL lookup — which may fail if the revocation servers are unreachable.
- **Missing intermediate**: The CMS doesn't include all intermediate certificates, and the validator can't build the chain. This is the most common real-world certificate chain failure.
- **Not in trust store**: The root certificate is not in the validator's trust store. Different validators have different trust stores — Adobe uses AATL, operating systems use their own, and EU validators use the EUTL.

17.1.5 Step 5: Trust Evaluation

Question answered: "Do I trust the entity that signed this document?"

This is the most subjective step, and it's where validators diverge the most. Trust evaluation is not a purely technical question — it depends on context:

Adobe Acrobat trust model: Acrobat trusts certificates that chain to a root in the **Adobe Approved Trust List (AATL)**. If the certificate chains to an AATL root, Acrobat shows a green checkmark. If not — even if the certificate is perfectly valid, issued by a reputable CA, and chains to a root in the operating system's trust store — Acrobat shows a yellow warning: "At least one signature has problems."

This is one of the most frustrating aspects of PDF signature validation for developers. Your signature is cryptographically valid, the certificate is issued by a recognized CA, the certificate is not expired or revoked — but Acrobat shows a warning because the CA is not on Adobe's list. Getting on the AATL requires the CA to apply to Adobe and pay fees.

EU validation model: EU validators (like the DSS demonstration webapp, or validators used by EU government services) trust certificates that chain to a root in the **EU Trusted List (EUTL)**. The EUTL is the authoritative registry of Qualified Trust Service Providers. A signature is a "qualified electronic signature" — with the legal weight of a handwritten signature under eIDAS — only if the certificate chains to an EUTL-listed QTSP.

Operating system trust model: Some applications (like LibreOffice) use the operating system's certificate store for trust evaluation. This means the set of trusted roots varies by OS and by the user's system configuration.

Custom trust model: Your application can define its own trust store — a specific set of root certificates that you trust for your use case. This is common in enterprise environments where the signing certificates are issued by an internal CA.

When debugging trust failures, ask three questions:

1. What trust store is the validator using? (AATL, EUTL, OS store, custom)
2. Is the signer's root certificate in that trust store?
3. Are all intermediates available? (Either embedded in the CMS or fetchable via the certificate's Authority Information Access extension)

The answer to question 1 determines everything else. A signature can be "valid" in one validator and "untrusted" in another, purely because of trust store differences.

17.2 Validation Differences Across Viewers

One of the most confusing aspects of PDF signature validation is that different viewers produce different results for the same PDF. This is not a bug — it's a consequence of the five-step validation model, where each viewer makes different choices at each step.

Table 17.1: Validation behavior comparison across major PDF viewers

Check	Acrobat	Foxit	LibreOffice	PDFBox	EU DSS
Byte range trailing bytes	Warns	Warns	Partial	Manual	Warns
Hash verification	Yes	Yes	Yes	Manual	Yes
CMS signature	Yes	Yes	Yes	Manual	Yes
Certificate chain	AATL	Mixed	OS store	Manual	EUTL
Revocation (OCSP)	Yes	Yes	Limited	Manual	Yes
Revocation (CRL)	Yes	Yes	Yes	Manual	Yes
MDP enforcement	Full	Mostly	Partial	None	Full
Timestamp verification	Yes	Yes	Partial	Manual	Yes
PAdES compliance	Partial	Partial	No	Manual	Full

"Manual" for PDFBox means the library provides the primitives (signature extraction, byte range access, CMS parsing) but does not perform the validation automatically — you must write the validation logic yourself. This is by design: PDFBox is a PDF library, not a validation service.

EU DSS provides the most complete automated validation, including ETSI-compliant validation reports.

17.3 Debugging Validation Failures

17.3.1 The Systematic Approach

When a signature fails validation, resist the urge to guess. Instead, walk through the five steps systematically:

Step 1: Is the byte range intact?

```
# Check byte range with QPDF
qpdf --show-object=<sig-obj-number> document.pdf
# Look at /ByteRange — does offset2 + length2 == file size?
wc -c document.pdf
```

Step 2: Does the hash match?

```
// Compute the hash yourself and compare to the CMS messageDigest
// (Code in Chapter 13)
```

Step 3: Does the CMS verify?

```
# Extract /Contents, decode hex, verify with OpenSSL
openssl cms -inform DER -in signature.der -verify \
    -content signed-bytes.bin -noverify -out /dev/null
# -noverify skips certificate verification — tests only the CMS math
```

Step 4: Is the certificate chain valid?

```
# Extract the signer's certificate and verify the chain
openssl cms -inform DER -in signature.der -noout -cmsout -print | \
    grep -A 5 "cert:"
openssl verify -CAfile trusted-roots.pem signer-cert.pem
```

Step 5: Is the certificate trusted?

Check the signer's root against the relevant trust store (AATL list, EUTL, OS store).

17.3.2 Common Error Messages and Their Meaning

Table 17.2: PDF signature error messages decoded

Error Message (Viewer)	Step	What It Actually Means
"Signature is INVALID" (Acrobat)	2 or 3	Hash mismatch or CMS verification failure. The document was modified after signing, or the CMS is malformed.
"At least one signature has problems" (Acrobat)	5	The certificate does not chain to an AATL root. The signature may be cryptographically perfect — Acrobat just doesn't trust the CA.
"Document has been altered or corrupted since the Signature was applied" (Acrobat)	1 or MDP	Either the byte range doesn't cover the full file (trailing bytes), or modifications after signing exceed the MDP permissions.
"Signature is valid, but the identity of the signer could not be verified" (Acrobat)	4 or 5	CMS is valid, but the certificate chain is incomplete or the root is not trusted.
"The signature cannot be verified" (LibreOffice)	3, 4, or 5	Generic error — LibreOffice doesn't distinguish well between CMS failure and trust failure. Check the signature details panel.
`CMSException: message-digest attribute value does not match calculated value` (BouncyCastle)	2	Hash mismatch. The bytes you passed to the verifier don't match what was signed.
`CertPathValidatorException: validity check failed` (Java)	4	A certificate in the chain has expired.

Error Message (Viewer)	Step	What It Actually Means
`CertPathValidatorException: no valid CRL found` (Java)	4	Revocation checking failed because the CRL distribution point is unreachable or the CRL is not cached.

17.4 Timestamp Verification

17.4.1 Why Timestamps Matter for Validation

A digital signature proves that a specific certificate holder signed specific bytes. But it doesn't prove *when* the signing happened. Without a timestamp, you can only say "the signature was created sometime before now."

This matters because certificates expire and can be revoked. If a certificate was valid on March 1 but expired on March 31, and you validate the signature on April 15:

- **Without timestamp**: The certificate is expired. Some validators will flag the signature as questionable.
- **With timestamp**: The timestamp proves the signature was created on March 1, when the certificate was valid. The signature remains valid despite the certificate's expiration.

17.4.2 Verifying the Timestamp

A signature timestamp is embedded as an unsigned attribute (`id-aa-signatureTimeStampToken`, OID 1.2.840.113549.1.9.16.2.14) in the CMS `SignerInfo`:

```
// Extract and verify the signature timestamp
AttributeTable unsignedAttrs = signerInfo.getUnsignedAttributes();
if (unsignedAttrs != null) {
    Attribute tsAttr = unsignedAttrs.get(
            PKCSObjectIdentifiers.id_aa_signatureTimeStampToken);
    if (tsAttr != null) {
        CMSSignedData tsToken = new CMSSignedData(
                tsAttr.getAttrValues().getObjectAt(0).toASN1Primitive().getEncoded());

        // Verify the timestamp's CMS signature
        SignerInformation tsSigner = tsToken.getSignerInfos()
                .getSigners().iterator().next();
        X509CertificateHolder tsCert = findSignerCertificate(
                tsToken.getCertificates(), tsSigner.getSID());

        boolean tsValid = tsSigner.verify(
                new JcaSimpleSignerInfoVerifierBuilder()
                        .setProvider("BC").build(tsCert));

        if (tsValid) {
            // Extract the timestamp time
            TimeStampToken tst = new TimeStampToken(tsToken);
            Date signingTime = tst.getTimeStampInfo().getGenTime();
            log.info("Signature was timestamped at: {}", signingTime);
```

```
                // Verify that the timestamp covers the signature value
                byte[] tsDigest = tst.getTimeStampInfo()
                        .getMessageImprintDigest();
                // This digest should match the hash of the signature value
            }
        }
    }
```

17.4.3 Timestamp Chain of Trust

The timestamp itself is a CMS signature — signed by the TSA's certificate. This certificate has its own chain and trust requirements:

1. The TSA certificate must have the `id-kp-timeStamping` extended key usage (OID 1.3.6.1.5.5.7.3.8)
2. The TSA certificate chain must be valid at the time of verification
3. The TSA's root must be in your trust store

If the TSA's certificate has also expired by the time you validate (years later), you need **long-term validation (LTV)** — which is the subject of Chapter 18.

17.5 Building a Complete Validation Pipeline

17.5.1 Architecture

A production validation pipeline typically follows this structure:

```
Input: PDF file
  │
  ├─→ Step 1: Extract all signatures
  │      └─→ For each signature:
  │            ├─→ Step 2: Byte range check
  │            ├─→ Step 3: Hash verification
  │            ├─→ Step 4: CMS verification
  │            ├─→ Step 5: Certificate chain
  │            ├─→ Step 6: Revocation check
  │            ├─→ Step 7: Timestamp verification (if present)
  │            ├─→ Step 8: MDP compliance (if certification sig)
  │            └─→ Step 9: Trust evaluation
  │
  └─→ Output: Validation report
        ├─→ Per-signature results
        ├─→ Overall document status
        └─→ Detailed diagnostics
```

Chapter 31 provides a complete Java implementation of this pipeline.

17.5.2 When to Use EU DSS Instead

If you need to produce **ETSI-compliant validation reports** (ETSI TS 119 102-2) — for example, because you're building a validation service for a government or regulated industry — use

the EU DSS framework (Chapter 23) instead of building the pipeline yourself. EU DSS handles all validation steps, produces standards-compliant reports, and has been tested against the full range of PAdES profiles and edge cases.

Building your own pipeline is appropriate when:

- You need custom trust evaluation logic
- You need to validate signatures in a specific context (e.g., your own application's signing workflow)
- You want to understand every step of the validation process (which is the point of this book)
- You're building a specialized tool (forensic analysis, security research)

17.6 Key Takeaways

- Signature validation is a **five-step pipeline**: byte range hash CMS certificate chain trust
- Each step is independent — a failure at any step means the signature is invalid or questionable
- **Different viewers produce different results** because they use different trust stores and enforce different rules
- Adobe Acrobat uses **AATL** for trust; EU validators use **EUTL**; LibreOffice uses the **OS trust store**
- The most common real-world failure is **trust evaluation** — the signature is cryptographically perfect but the root CA isn't in the validator's trust store
- **Timestamps** prove when a signature was created — essential for handling expired certificates and building long-term validation
- **Debug systematically**: walk through the five steps in order, using QPDF, OpenSSL, and BouncyCastle to isolate the failure
- **EU DSS** provides the most complete automated validation with ETSI-compliant reports
- PDFBox provides the **primitives** for validation but does not automate the pipeline — you build it yourself

Chapter 18

PAdES Profiles (ETSI EN 319 142)

ISO 32000-2 defines *how* a digital signature is embedded in a PDF. PAdES defines *what* that signature must contain to be legally meaningful in Europe — and, increasingly, everywhere else.

PAdES — PDF Advanced Electronic Signatures — is a set of standards published by the **European Telecommunications Standards Institute (ETSI)** that constrain PDF signatures to meet specific compliance requirements. A PAdES signature is always a valid ISO 32000-2 signature, but the reverse is not true. PAdES adds mandatory signed attributes, requires specific SubFilter values, mandates timestamp handling, and defines four progressively more complete signature profiles.

Understanding PAdES is not optional if your signatures need to be validated by European government services, if your customers require eIDAS-compliant signatures, or if you want your signatures to remain verifiable for years or decades. Even if you're not operating in the EU, PAdES profiles have become the de facto standard for "properly structured" PDF signatures worldwide — Adobe Acrobat increasingly favors PAdES-structured signatures in its validation logic.

This chapter explains each PAdES profile, what it requires, how the profiles relate to each other, and how to produce each one.

18.1 The Four PAdES Baseline Profiles

PAdES defines four **baseline profiles**, each building on the previous one. They are formally named PAdES-B-B, PAdES-B-T, PAdES-B-LT, and PAdES-B-LTA. The naming follows the pattern: PAdES-B(aseline)-<level>.

Table 18.1: PAdES Baseline Profile Summary

Profile	Full Name	What It Adds
B-B	Baseline — Basic	The minimum PAdES signature. CMS `SignedData` with mandatory signed attributes including `ESS-signing-certificate-v2`. The signature itself — no timestamps, no revocation data.
B-T	Baseline — Timestamp	B-B + a **signature timestamp** from a trusted TSA. Proves when the signature was created. The timestamp is an unsigned CMS attribute (`id-aa-signatureTimeStampToken`).

Profile	Full Name	What It Adds
B-LT	Baseline — Long-Term	B-T + embedded **revocation data** (OCSP responses and/or CRLs) and the full certificate chain in the PDF's **DSS** (Document Security Store) dictionary. Everything needed to validate the signature offline, without contacting any external servers.
B-LTA	Baseline — Long-Term with Archive timestamp	B-LT + a **document timestamp** signature that covers the entire file (including the DSS dictionary). Enables re-timestamping decades later, creating an indefinite chain of trust that outlives any individual certificate.

18.1.1 The Progression: Why Four Levels?

Each profile addresses a specific failure mode:

- **B-B** fails when: the signing certificate expires (validators may warn), or when you need to prove *when* the signature was created (there's no timestamp)
- **B-T** fails when: the OCSP responder or CRL distribution point goes offline years later, and you can't check revocation status
- **B-LT** fails when: the timestamp's certificate, or the signing certificate's root CA, expires or is removed from trust stores decades later
- **B-LTA** handles all of these by providing a renewable chain of timestamps

In practice:

- **B-B** is sufficient for short-lived documents that will be validated within the signing certificate's validity period (typically 1-3 years)
- **B-T** is the minimum for any document with legal significance — you need to prove when it was signed
- **B-LT** is required for long-term archival (10+ years) where you can't assume external services will remain available
- **B-LTA** is the gold standard for documents that must be verifiable indefinitely — government records, notarized documents, regulatory filings

18.2 PAdES-B-B: The Baseline Signature

18.2.1 Requirements

A PAdES-B-B signature must include:

1. **SubFilter**: Must be `ETSI.CAdES.detached` (not `adbe.pkcs7.detached`)
2. **Mandatory signed attributes in CMS**:
   - `content-type` (OID 1.2.840.113549.1.9.3) — must be `id-data`
   - `message-digest` (OID 1.2.840.113549.1.9.4) — the hash of the signed bytes
   - `ESS-signing-certificate-v2` (OID 1.2.840.113549.1.9.16.2.47) — identifies the signing certificate by its hash, preventing certificate substitution attacks

3. **Signing certificate embedded in CMS**: The signer's certificate (and ideally the full chain up to but not including the root) must be embedded in the CMS `SignedData.certificates` collection
4. **Digest algorithm**: Must be SHA-256 or stronger (SHA-384, SHA-512). SHA-1 is **not** PAdES-compliant.

18.2.2 The ESS-signing-certificate-v2 Attribute

This attribute is the key difference between a basic ISO 32000-2 signature and a PAdES-B-B signature. It binds the CMS signature to a specific certificate by including a hash of the certificate in the signed attributes.

Why this matters: without `ESS-signing-certificate-v2`, an attacker could theoretically substitute a different certificate into the CMS `certificates` collection — one with the same public key but different identity information. The CMS signature would still verify (same key), but the identity would be wrong. The ESS attribute prevents this by cryptographically committing to a specific certificate.

```
// Creating the ESS-signing-certificate-v2 attribute with BouncyCastle
MessageDigest certDigest = MessageDigest.getInstance("SHA-256");
byte[] certHash = certDigest.digest(signingCertificate.getEncoded());

ESSCertIDv2 essCertId = new ESSCertIDv2(
        new AlgorithmIdentifier(NISTObjectIdentifiers.id_sha256),
        certHash);

SigningCertificateV2 signingCertV2 = new SigningCertificateV2(
        new ESSCertIDv2[] { essCertId });

// Add to signed attributes
CMSAttributeTableGenerator signedAttrGen = new DefaultSignedAttributeTableGenerator(
        new AttributeTable(new Hashtable<>() {{
            put(PKCSObjectIdentifiers.id_aa_signingCertificateV2,
                new Attribute(PKCSObjectIdentifiers.id_aa_signingCertificateV2,
                    new DERSet(signingCertV2)));
        }}));
```

A common mistake: using `ESS-signing-certificate` (v1, OID 1.2.840.113549.1.9.16.2.12) instead of `ESS-signing-certificate-v2` (OID 1.2.840.113549.1.9.16.2.47). The v1 attribute uses SHA-1 for the certificate hash, which is deprecated. PAdES requires v2, which uses SHA-256 by default.

iText 2.x (and old versions of some other libraries) produced v1 attributes. If you encounter a PDF with `ESS-signing-certificate` (v1), it's likely from a legacy signing implementation.

18.2.3 SubFilter: `ETSI.CAdES.detached` vs. `adbe.pkcs7.detached`

PAdES requires the SubFilter value `ETSI.CAdES.detached`. This is structurally identical to `adbe.pkcs7.detached` — both produce a detached CMS signature embedded in the `/Contents` hex string. The difference is semantic: `ETSI.CAdES.detached` signals that the signature follows the CAdES (CMS Advanced Electronic Signatures) profile, which includes the ESS-signing-certificate-v2 attribute and other CAdES requirements.

In practice, many validators accept both SubFilter values for PAdES validation. Adobe Acrobat treats `ETSI.CAdES.detached` and `adbe.pkcs7.detached` similarly. But for strict PAdES compliance — especially when dealing with EU government validators — use `ETSI.CAdES.detached`.

18.3 PAdES-B-T: Adding a Timestamp

18.3.1 What B-T Adds

PAdES-B-T is PAdES-B-B plus a **signature timestamp** — a timestamp token from a Trusted Timestamp Authority (TSA) that proves when the signature was created.

The timestamp is embedded as an **unsigned attribute** (`id-aa-signatureTimeStampToken`, OID 1.2.840.113549.1.9.16.2.14) in the CMS `SignerInfo`. It's unsigned because it's applied *after* the signature — you sign first, then request a timestamp over the signature value.

18.3.2 How Timestamping Works

The timestamp workflow:

1. **Sign the document** — produce the CMS `SignedData` with all signed attributes (B-B level)
2. **Hash the signature value** — compute SHA-256 over the CMS `SignerInfo.signature` bytes
3. **Send a timestamp request** — send the hash to a TSA using RFC 3161 (HTTP POST to the TSA's URL)
4. **Receive the timestamp response** — the TSA returns a `TimeStampToken` (which is itself a CMS `SignedData` signed by the TSA's certificate)
5. **Embed the timestamp** — add the `TimeStampToken` as an unsigned attribute in the CMS

```
// Requesting a timestamp from a TSA
TimeStampRequestGenerator tsReqGen = new TimeStampRequestGenerator();
tsReqGen.setCertReq(true);  // Request TSA's certificate in the response

byte[] signatureHash = MessageDigest.getInstance("SHA-256")
        .digest(signerInfo.getSignature());

TimeStampRequest tsReq = tsReqGen.generate(
        TSPAlgorithms.SHA256, signatureHash);

// Send to TSA via HTTP
byte[] tsReqBytes = tsReq.getEncoded();
HttpURLConnection conn = (HttpURLConnection)
        new URL(tsaUrl).openConnection();
conn.setDoOutput(true);
conn.setRequestMethod("POST");
conn.setRequestProperty("Content-Type", "application/timestamp-query");
conn.getOutputStream().write(tsReqBytes);

TimeStampResponse tsResp = new TimeStampResponse(conn.getInputStream());
tsResp.validate(tsReq);  // Verify the response matches our request

TimeStampToken tsToken = tsResp.getTimeStampToken();
```

18.3.3 Choosing a TSA

Choosing a TSA is a non-trivial decision. The TSA's certificate must be trusted by the validators that will check your signatures. Key considerations:

- **For Adobe Acrobat**: The TSA's root must be in the AATL, or the timestamp will show a warning
- **For EU compliance**: The TSA should be a Qualified Trust Service Provider (QTSP) listed in the EUTL
- **For availability**: The TSA must be reachable when you sign — if the TSA is down, your signing fails (or you produce a B-B signature without a timestamp)
- **For cost**: Some TSAs are free (FreeTSA, DigiCert's free tier), others charge per timestamp or per certificate

Chapter 18 covers timestamps in depth, including TSA selection, failover strategies, and the Document Security Store.

18.4 PAdES-B-LT: Long-Term Data

18.4.1 What B-LT Adds

PAdES-B-LT is PAdES-B-T plus **long-term validation data** embedded directly in the PDF. This data ensures that the signature can be validated years from now, even if the signing certificate's CA has gone out of business, the OCSP responder has been decommissioned, and the CRL distribution points return 404.

The long-term data is stored in the **DSS** (Document Security Store) dictionary, which is added to the document catalog:

```
<< /Type /Catalog
   /AcroForm << ... >>
   /DSS <<
     /Certs [30 0 R 31 0 R 32 0 R]       % Certificate chain as DER-encoded streams
     /OCSPs [33 0 R]                       % OCSP responses as DER-encoded streams
     /CRLs [34 0 R]                        % CRLs as DER-encoded streams
     /VRI <<
       /A1B2C3D4... <<                     % Hash-keyed VRI (Validation Related Info)
         /Cert [30 0 R 31 0 R]
         /OCSP [33 0 R]
         /TU (D:20260308120000Z)           % Time of validation
       >>
     >>
   >>
>>
```

18.4.2 The DSS Dictionary

Table 18.2: DSS Dictionary Keys (ISO 32000-2:2020 §12.8.4.3)

Key	Type	Req	Spec Ref
`/Certs`	array of streams	Optional	§12.8.4.3
DER-encoded certificates — the full chain for every signature and timestamp in the document. Include the signer's certificate, all intermediates, and optionally the root.			
`/OCSPs`	array of streams	Optional	§12.8.4.3
DER-encoded OCSP responses for certificate revocation checking. Each response covers a specific certificate at a specific point in time.			
`/CRLs`	array of streams	Optional	§12.8.4.3
DER-encoded Certificate Revocation Lists. Used when OCSP is not available for a certificate.			
`/VRI`	dictionary	Optional	§12.8.4.3
Validation Related Information — a dictionary keyed by uppercase hex SHA-1 hashes of signature values, mapping each signature to its specific validation data.			

18.4.3 Why B-LT Matters

Without B-LT, validating a signature requires contacting external servers:

- OCSP responder to check revocation status
- CRL distribution point to download the revocation list
- CA's AIA (Authority Information Access) URL to fetch intermediate certificates

These servers may be unreachable years from now. CAs shut down, merge, change URLs, or decommission infrastructure. With B-LT, everything needed for validation is embedded in the PDF itself — the document is self-contained.

18.5 PAdES-B-LTA: Archival Timestamps

18.5.1 What B-LTA Adds

PAdES-B-LTA is PAdES-B-LT plus a **document timestamp** — a timestamp signature that covers the entire PDF file, including the DSS dictionary.

This timestamp is not part of the original signature's CMS — it's a separate signature dictionary with `/SubFilter /ETSI.RFC3161`:

```
<< /Type /Sig
   /Filter /Adobe.PPKLite
   /SubFilter /ETSI.RFC3161
   /ByteRange [0 52000 84776 2100]
   /Contents <3082...>
>>
```

This timestamp signature covers the entire file (including the original signature, the DSS, and everything else) and proves that all of this data existed at the time the timestamp was created.

18.5.2 The Archival Chain

The power of B-LTA is that it creates a **renewable chain of trust**:

1. The original signature (signed by the signer's certificate, valid for 1-3 years)
2. The signature timestamp (signed by the TSA's certificate, valid for 10-20 years)
3. The archival timestamp (signed by the TSA's certificate, covers everything)
4. Years later: add another archival timestamp (covers the previous archival timestamp)
5. Repeat indefinitely

Each archival timestamp extends the provable lifetime of the signature. Even after the original signing certificate has expired, been revoked, and the CA has gone out of business — the chain of archival timestamps proves that the signature was valid at the time it was created.

This is the mechanism that national archives use to preserve the legal validity of digitally signed documents for decades.

18.5.3 Re-Timestamping

To extend a B-LTA signature's validity:

1. Fetch fresh OCSP responses / CRLs for the archival timestamp's certificate chain
2. Add them to the DSS dictionary
3. Apply a new archival timestamp (document timestamp) over the entire file

This can be done without the original signer's involvement — it's a maintenance operation performed by the document custodian.

Automated re-timestamping should be a scheduled process in any document management system that stores B-LTA signed documents. Before the current archival timestamp's TSA certificate expires (check the certificate's `notAfter` date), apply a new archival timestamp. This keeps the chain of trust alive.

18.6 Profile Comparison Matrix

Table 18.3: PAdES profile feature comparison

Feature	B-B	B-T	B-LT	B-LTA
CMS with ESS-signing-certificate-v2	Yes	Yes	Yes	Yes
Signature timestamp	No	Yes	Yes	Yes
Embedded revocation data (DSS)	No	No	Yes	Yes
Embedded certificate chain (DSS)	No	No	Yes	Yes
Document timestamp (archival)	No	No	No	Yes
Offline validation	No	Partial	Yes	Yes
Validity beyond cert expiry	No	Yes (with TS)	Yes	Yes (indefinite)

Feature	B-B	B-T	B-LT	B-LTA
EU/eIDAS suitability	Minimal	Short-term	Good	**Full compliance**
Typical use case	Internal docs	Contracts	Regulated docs	Archival

18.7 Migrating Between Profiles

One of the elegant properties of PAdES profiles is that you can **upgrade** a signature's profile without re-signing:

- **B-B B-T**: Add a timestamp (unsigned attribute to the CMS). Requires the original file and the CMS.
- **B-T B-LT**: Fetch revocation data and certificates, create the DSS dictionary, add as incremental update.
- **B-LT B-LTA**: Add a document timestamp covering the entire file.

This progression means you can sign with B-B today (fast, no TSA dependency) and upgrade to B-LTA later as a batch operation. EU DSS provides methods for exactly this:

```
// EU DSS: Extend a B-B signature to B-LTA
PAdESService service = new PAdESService(certificateVerifier);
service.setTspSource(tspSource);

DSSDocument extendedDoc = service.extendDocument(
    signedDocument,
    new PAdESSignatureParameters() {{
        setSignatureLevel(SignatureLevel.PAdES_BASELINE_LTA);
    }});
```

18.8 Key Takeaways

- PAdES constrains ISO 32000-2 signatures for legal and regulatory compliance — a PAdES signature is always a valid PDF signature, but not vice versa
- **Four profiles**: B-B (basic), B-T (timestamped), B-LT (long-term data), B-LTA (archival)
- **ESS-signing-certificate-v2** is the mandatory attribute that makes a signature PAdES-compliant — without it, you have a valid ISO 32000-2 signature but not PAdES
- **SubFilter `ETSI.CAdES.detached`** is required for strict PAdES compliance
- The **DSS dictionary** (B-LT and above) embeds certificates, OCSP responses, and CRLs directly in the PDF for offline validation
- **B-LTA** provides indefinite validity through renewable archival timestamps
- Profiles can be **upgraded** without re-signing — B-B B-T B-LT B-LTA as incremental updates
- For EU/eIDAS compliance, **B-LTA is the target** — anything less may not meet long-term archival requirements

Chapter 19

Timestamps & Long-Term Validation

A digital signature without a timestamp is like a contract without a date. You can prove *who* signed and *what* they signed, but not *when*. And in the world of digital signatures, ”when” is often the most important question — because certificates expire, keys get compromised, algorithms get broken, and the difference between ”signed before the certificate expired” and ”signed after the certificate expired” can be the difference between a legally binding document and an invalid one.

This chapter covers everything about timestamps and long-term validation (LTV): how RFC 3161 timestamps work, how they’re embedded in PDF signatures, how the Document Security Store (DSS) enables offline validation, how archival timestamps create indefinite validity, and how to build a complete LTV infrastructure.

19.1 RFC 3161: The Timestamp Protocol

19.1.1 What a Timestamp Proves

An RFC 3161 timestamp is a cryptographic proof that a specific piece of data existed at a specific point in time. The proof comes from a **Trusted Timestamp Authority (TSA)** — a server operated by a trusted third party that:

1. Receives a hash of the data to be timestamped
2. Combines the hash with the current time
3. Signs the combination with the TSA’s private key
4. Returns the signed timestamp token

The timestamp token is a CMS `SignedData` structure (yes, a timestamp is itself a CMS signature) containing a `TSTInfo` (Timestamp Token Info) structure with:

- The hash algorithm and hash value of the timestamped data
- The timestamp time (UTC)
- The TSA’s name
- A serial number (unique per timestamp)
- An accuracy indication (how precise the time is)

19.1.2 The Timestamp Request/Response Flow

The protocol uses HTTP POST with content type `application/timestamp-query` for the request and `application/timestamp-reply` for the response. The request and response are DER-encoded ASN.1 structures defined in RFC 3161.

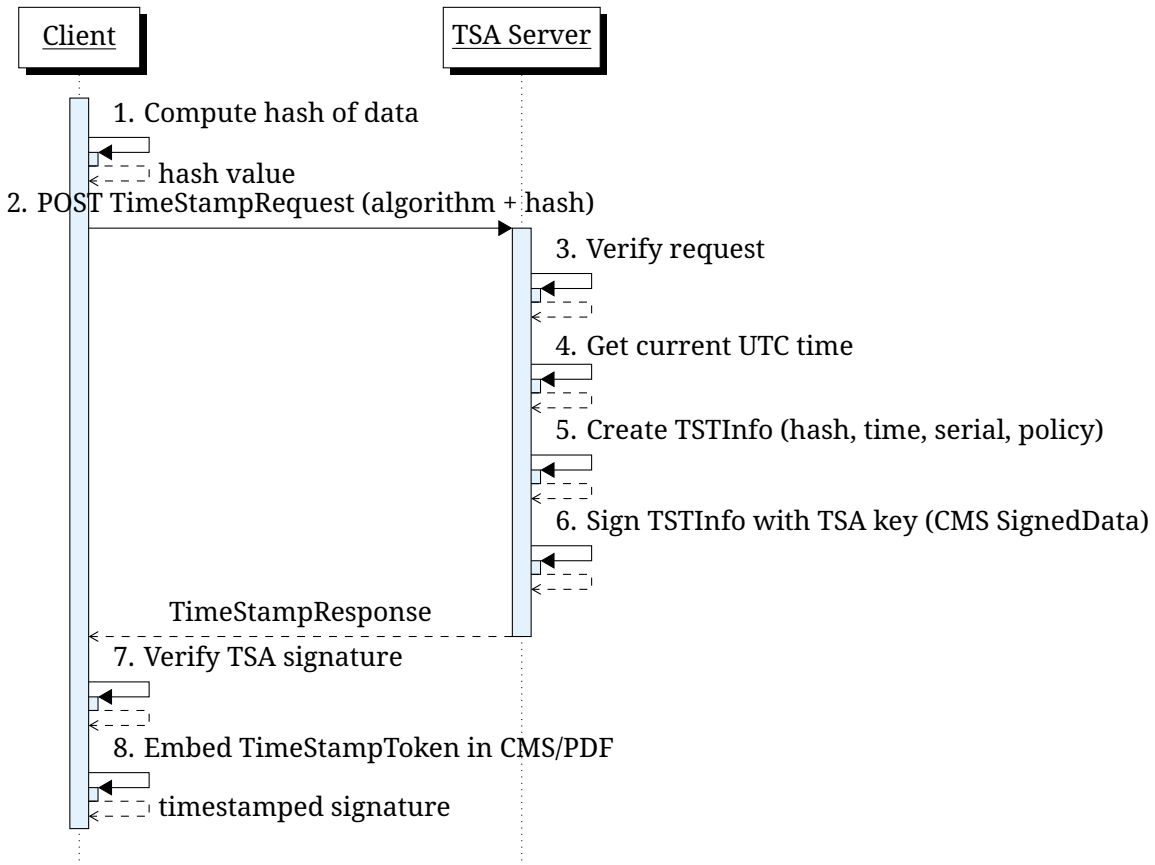


Figure 19.1: RFC 3161 timestamp request/response flow

19.1.3 Implementation with BouncyCastle

```
/**
 * Requests a timestamp from a TSA for the given data.
 * Returns the TimeStampToken that can be embedded as an unsigned CMS attribute.
 */
public TimeStampToken requestTimestamp(byte[] dataToTimestamp, String tsaUrl)
        throws Exception {

    // 1. Hash the data
    MessageDigest digest = MessageDigest.getInstance("SHA-256");
    byte[] hash = digest.digest(dataToTimestamp);

    // 2. Create the timestamp request
    TimeStampRequestGenerator reqGen = new TimeStampRequestGenerator();
    reqGen.setCertReq(true);  // Include TSA's cert in the response
    TimeStampRequest tsReq = reqGen.generate(TSPAlgorithms.SHA256, hash);

    // 3. Send to TSA via HTTP
    byte[] reqBytes = tsReq.getEncoded();
    HttpURLConnection conn = (HttpURLConnection)
            new URL(tsaUrl).openConnection();
    conn.setDoOutput(true);
    conn.setRequestMethod("POST");
    conn.setRequestProperty("Content-Type", "application/timestamp-query");
    conn.setConnectTimeout(10000);
    conn.setReadTimeout(30000);

    try (OutputStream out = conn.getOutputStream()) {
        out.write(reqBytes);
    }

    // 4. Read the response
    byte[] respBytes;
    try (InputStream in = conn.getInputStream()) {
        respBytes = in.readAllBytes();
    }

    // 5. Parse and validate the response
    TimeStampResponse tsResp = new TimeStampResponse(respBytes);
    tsResp.validate(tsReq);  // Verify it matches our request

    if (tsResp.getStatus() != PKIStatus.GRANTED
            && tsResp.getStatus() != PKIStatus.GRANTED_WITH_MODS) {
        throw new TSPException("TSA rejected request: status=" +
                tsResp.getStatus() + ", statusString=" +
                tsResp.getStatusString());
    }

    TimeStampToken tsToken = tsResp.getTimeStampToken();
    log.info("Timestamp obtained: time={}, serial={}, TSA={}",
            tsToken.getTimeStampInfo().getGenTime(),
            tsToken.getTimeStampInfo().getSerialNumber(),
            tsToken.getTimeStampInfo().getTsa());
```

```
    return tsToken;
}
```

19.1.4 TSA Selection Criteria

Choosing the right TSA is critical. A timestamp is only as trustworthy as the TSA that produced it.

Table 19.1: TSA selection criteria for different use cases

Criterion	Considerations
Trust store compatibility	For Adobe Acrobat: TSA's root must be in AATL. For EU compliance: TSA must be a QTSP in the EUTL. For general use: TSA's root should be widely trusted.
Availability & SLA	TSAs can go down. For production signing, use a TSA with a published SLA. Consider a fallback TSA if the primary is unreachable.
Cost	Free TSAs exist (FreeTSA.org, some DigiCert endpoints) but may not be suitable for production. Commercial TSAs charge per timestamp or per annual subscription. Costs range from free to several thousand euros per year.
Accuracy	TSAs report their time accuracy (e.g., ±1 second). For most use cases, sub-second accuracy is not critical.
Geographic location	Some regulations require the TSA to be operated within specific jurisdictions. EU regulations may require a TSA operated by an EU-based QTSP.
Response time	Network latency + TSA processing time. Free TSAs may be slower. For high-volume signing (thousands per minute), TSA response time matters.
Certificate lifetime	The TSA's signing certificate has an expiry date. Longer-lived TSA certificates mean your timestamps are verifiable for longer before re-timestamping is needed.

Never hardcode a single TSA URL without a fallback strategy. If the TSA is unreachable when your application needs to sign, the signing fails. At minimum, implement:

1. A configurable TSA URL (not hardcoded)
2. A reasonable timeout (10-30 seconds)
3. A retry with backoff (the TSA may be temporarily overloaded)
4. Optionally, a fallback TSA URL
5. A graceful degradation path: if no TSA is available, produce a B-B signature and log a warning for later B-T upgrade

19.2 Signature Timestamps vs. Document Timestamps

PDF supports two kinds of timestamps, and they serve different purposes:

19.2.1 Signature Timestamps

A **signature timestamp** is embedded *inside* the CMS `SignedData` as an unsigned attribute of the `SignerInfo`. It timestamps the *signature value* — proving when the signature was created.

- **Where it lives**: CMS `SignerInfo.unsignedAttrs.id-aa-signatureTimeStampToken`
- **What it timestamps**: The bytes of the `SignerInfo.signature` field
- **PAdES level**: Required for PAdES-B-T and above
- **Who creates it**: The signing application, immediately after computing the CMS signature

19.2.2 Document Timestamps

A **document timestamp** is a separate signature in the PDF — a `/Sig` dictionary with `/SubFilter /ETSI.RFC3161` — that timestamps the *entire document*.

- **Where it lives**: A separate signature dictionary in the PDF, applied as an incremental update
- **What it timestamps**: The entire PDF file (via byte range), including all previous signatures and the DSS
- **PAdES level**: Required for PAdES-B-LTA (the archival timestamp)
- **Who creates it**: The document management system, as a maintenance operation

The key difference: a signature timestamp proves when a *specific signature* was created. A document timestamp proves that the *entire document* (including all signatures and validation data) existed at a specific time.

```
// Signature timestamp (inside CMS)
/Sig <<
  /SubFilter /ETSI.CAdES.detached
  /Contents <CMS SignedData:
    SignerInfo:
      signedAttrs: [contentType, messageDigest, ESS-signing-certificate-v2]
      signature: <RSA/ECDSA signature bytes>
      unsignedAttrs: [
        signatureTimeStampToken: <TimeStampToken>  ← THIS
      ]
  >
>>

// Document timestamp (separate signature)
/Sig <<
  /SubFilter /ETSI.RFC3161            ← THIS
  /ByteRange [0 ... ... ...]          ← Covers entire file
  /Contents <TimeStampToken as CMS>
>>
```

19.3 The Document Security Store (DSS)

19.3.1 Purpose

The DSS is a dictionary in the document catalog that stores all the data needed to validate every signature in the document *offline* — without contacting any external servers.

Without a DSS, validating a signature requires:

- Fetching the signer's certificate chain from the CMS and/or AIA URLs
- Contacting OCSP responders to check revocation status
- Downloading CRLs from distribution points

All of these require network access and functioning servers. Years from now, these servers may not exist. The DSS solves this by embedding everything locally.

19.3.2 Building the DSS

Building a DSS requires collecting all the validation data and embedding it as an incremental update:

```
/**
 * Adds a DSS (Document Security Store) dictionary to the PDF,
 * embedding certificates, OCSP responses, and CRLs for all signatures.
 */
public void addDSS(PDDocument doc, List<ValidationData> validationData)
        throws IOException {

    COSDictionary dssDictionary = new COSDictionary();
    COSArray certsArray = new COSArray();
    COSArray ocspsArray = new COSArray();
    COSArray crlsArray = new COSArray();

    for (ValidationData vd : validationData) {
        // Embed certificates
        for (byte[] certBytes : vd.getCertificateChain()) {
            PDStream certStream = new PDStream(doc);
            try (OutputStream os = certStream.createOutputStream()) {
                os.write(certBytes);
            }
            certsArray.add(certStream.getCOSObject());
        }

        // Embed OCSP responses
        for (byte[] ocspBytes : vd.getOcspResponses()) {
            PDStream ocspStream = new PDStream(doc);
            try (OutputStream os = ocspStream.createOutputStream()) {
                os.write(ocspBytes);
            }
            ocspsArray.add(ocspStream.getCOSObject());
        }

        // Embed CRLs
        for (byte[] crlBytes : vd.getCrls()) {
```

```
                PDStream crlStream = new PDStream(doc);
                try (OutputStream os = crlStream.createOutputStream()) {
                    os.write(crlBytes);
                }
                crlsArray.add(crlStream.getCOSObject());
            }
        }

        if (certsArray.size() > 0) dssDictionary.setItem(COSName.getPDFName("Certs"), certsArray);
        if (ocspsArray.size() > 0) dssDictionary.setItem(COSName.getPDFName("OCSPs"), ocspsArray);
        if (crlsArray.size() > 0) dssDictionary.setItem(COSName.getPDFName("CRLs"), crlsArray);

        // Add DSS to document catalog
        doc.getDocumentCatalog().getCOSObject()
                .setItem(COSName.getPDFName("DSS"), dssDictionary);
    }
```

19.3.3 VRI: Validation Related Information

The DSS can optionally include a `/VRI` sub-dictionary that maps each signature to its specific validation data. The VRI keys are the uppercase hex SHA-1 hash of the signature value (the bytes in `/Contents`):

```
/DSS <<
  /Certs [...]
  /OCSPs [...]
  /VRI <<
    /A1B2C3D4E5F6... <<        % SHA-1 hash of signature 1's /Contents
      /Cert [30 0 R 31 0 R]    % Certs for this specific signature
      /OCSP [33 0 R]           % OCSP for this specific signature
      /TU (D:20260308120000Z)  % Time of validation
    >>
    /F6E5D4C3B2A1... <<        % SHA-1 hash of signature 2's /Contents
      /Cert [30 0 R 32 0 R]
      /OCSP [34 0 R]
      /TU (D:20260309140000Z)
    >>
  >>
>>
```

VRI is useful for multi-signature documents where each signature has different validation data. Without VRI, a validator must try all certificates and OCSP responses in the DSS against each signature to find the right match. With VRI, the mapping is explicit.

The VRI key is a SHA-1 hash — yes, SHA-1. This is a historical artifact from when the DSS was first specified. SHA-1 is used here only as a key derivation function (to generate a unique dictionary key from the signature value), not for cryptographic security. The security of the DSS does not depend on SHA-1 collision resistance.

19.4 Fetching Revocation Data

19.4.1 OCSP (Online Certificate Status Protocol)

OCSP is the preferred mechanism for checking certificate revocation. You send a request to the CA's OCSP responder with the certificate's serial number, and it responds with "good," "revoked," or "unknown."

```
/**
 * Fetches an OCSP response for the given certificate.
 * Returns the DER-encoded OCSP response bytes.
 */
public byte[] fetchOCSPResponse(X509Certificate cert,
        X509Certificate issuer) throws Exception {

    // 1. Find the OCSP responder URL from the certificate's AIA extension
    String ocspUrl = extractOcspUrl(cert);
    if (ocspUrl == null) {
        log.warn("No OCSP URL in certificate AIA for: {}",
                cert.getSubjectX500Principal());
        return null;
    }

    // 2. Build the OCSP request
    CertificateID certId = new CertificateID(
            new JcaDigestCalculatorProviderBuilder().build()
                    .get(CertificateID.HASH_SHA1),
            new JcaX509CertificateHolder(issuer),
            cert.getSerialNumber());

    OCSPReqBuilder reqBuilder = new OCSPReqBuilder();
    reqBuilder.addRequest(certId);
    OCSPReq ocspReq = reqBuilder.build();

    // 3. Send via HTTP
    HttpURLConnection conn = (HttpURLConnection)
            new URL(ocspUrl).openConnection();
    conn.setDoOutput(true);
    conn.setRequestMethod("POST");
    conn.setRequestProperty("Content-Type", "application/ocsp-request");
    conn.getOutputStream().write(ocspReq.getEncoded());

    byte[] respBytes = conn.getInputStream().readAllBytes();

    // 4. Validate the response
    OCSPResp ocspResp = new OCSPResp(respBytes);
    if (ocspResp.getStatus() != OCSPResp.SUCCESSFUL) {
        log.error("OCSP response status: {}", ocspResp.getStatus());
        return null;
    }

    BasicOCSPResp basicResp = (BasicOCSPResp) ocspResp.getResponseObject();
    SingleResp singleResp = basicResp.getResponses()[0];
```

```
    if (singleResp.getCertStatus() == CertificateStatus.GOOD) {
        log.info("OCSP: certificate is GOOD (produced: {})",
                basicResp.getProducedAt());
        return respBytes;
    } else {
        log.error("OCSP: certificate status is NOT GOOD: {}",
                singleResp.getCertStatus());
        return null;  // Certificate is revoked or unknown
    }
}
```

19.4.2 CRL (Certificate Revocation List)

CRLs are the fallback when OCSP is not available. A CRL is a signed list of all revoked certificates issued by a CA.

```
/**
 * Downloads a CRL from the certificate's CRL Distribution Point.
 * Returns the DER-encoded CRL bytes.
 */
public byte[] fetchCRL(X509Certificate cert) throws Exception {
    String crlUrl = extractCrlUrl(cert);
    if (crlUrl == null) {
        log.warn("No CRL URL in certificate for: {}",
                cert.getSubjectX500Principal());
        return null;
    }

    HttpURLConnection conn = (HttpURLConnection) new URL(crlUrl).openConnection();
    conn.setConnectTimeout(10000);
    conn.setReadTimeout(30000);

    byte[] crlBytes = conn.getInputStream().readAllBytes();

    // Validate the CRL
    CertificateFactory cf = CertificateFactory.getInstance("X.509");
    X509CRL crl = (X509CRL) cf.generateCRL(new ByteArrayInputStream(crlBytes));

    log.info("CRL downloaded: issuer={}, entries={}, nextUpdate={}",
            crl.getIssuerX500Principal(),
            crl.getRevokedCertificates() != null ?
                    crl.getRevokedCertificates().size() : 0,
            crl.getNextUpdate());

    return crlBytes;
}
```

CRLs can be **very large** — some CAs have CRLs of 500 KB or more. Embedding a large CRL in the DSS significantly increases the PDF file size. Prefer OCSP when available — OCSP responses are typically 1-3 KB.

Also be aware of **delta CRLs** — some CAs publish only the changes since the last full CRL. If the certificate's CRL Distribution Point specifies a delta CRL, you may need both the full CRL and the delta CRL for complete validation.

19.5 The Complete B-LTA Workflow

Putting it all together, here's the end-to-end workflow for producing a PAdES-B-LTA signature:

```
1. Create PAdES-B-B signature
   └── CMS with ESS-signing-certificate-v2, SHA-256, SubFilter ETSI.CAdES.detached

2. Add signature timestamp → PAdES-B-T
   └── Request RFC 3161 timestamp over the signature value
   └── Embed as unsigned CMS attribute

3. Fetch validation data
   └── Build certificate chain (signer → intermediates → root)
   └── Fetch OCSP response for each certificate
   └── Fetch CRL as fallback where OCSP unavailable
   └── Fetch TSA's certificate chain and revocation data too

4. Create DSS dictionary → PAdES-B-LT
   └── Embed all certificates as streams
   └── Embed all OCSP responses as streams
   └── Embed all CRLs as streams
   └── Optionally create VRI entries
   └── Add DSS to document catalog as incremental update

5. Add document timestamp → PAdES-B-LTA
   └── Request RFC 3161 timestamp over the entire file (including DSS)
   └── Apply as separate /Sig dictionary with SubFilter ETSI.RFC3161
   └── Save as incremental update
```

Each step is an incremental update to the PDF. The final file contains the original document, the signature, the DSS, and the document timestamp — everything needed for indefinite validation.

19.6 Key Takeaways

- **RFC 3161 timestamps** are CMS-signed proofs that data existed at a specific time — issued by Trusted Timestamp Authorities
- **Signature timestamps** (in CMS unsigned attributes) prove when a specific signature was created
- **Document timestamps** (separate `/Sig` with SubFilter `ETSI.RFC3161`) prove when the entire document existed — used for archival
- The **DSS dictionary** embeds certificates, OCSP responses, and CRLs directly in the PDF for offline validation
- **VRI entries** map specific signatures to their validation data using SHA-1 hash keys
- **OCSP** is preferred over CRLs for revocation checking (smaller, more specific)
- **TSA selection** matters: trust store compatibility (AATL/EUTL), availability, cost, and response time
- **PAdES-B-LTA** is the complete package: signature + timestamp + validation data + archival timestamp

- **Re-timestamping** extends validity indefinitely — a scheduled maintenance operation for archival documents
- Always implement **TSA failover** — a TSA outage should not prevent signing

Chapter 20

eIDAS & Qualified Signatures

Chapter 21

The eIDAS Regulation

In 2014, the European Union did something that no other jurisdiction had managed: it created a legal framework where a digital signature — a sequence of bytes produced by a cryptographic algorithm — carries the same legal weight as ink on paper, in all 27 member states simultaneously. That framework is called eIDAS, and whether you love it or hate it, if your software produces or validates digital signatures that touch Europe, you need to understand it.

The Regulation (EU) No 910/2014 (eIDAS) — its full title is ”Regulation (EU) No 910/2014 of the European Parliament and of the Council of 23 July 2014 on electronic identification and trust services for electronic transactions in the internal market” — is a 38-page regulation that created the legal and institutional infrastructure for electronic signatures, seals, timestamps, and related trust services across the entire European Union. It replaced a failed directive that had been limping along for 15 years, and it did so by making one critical architectural decision: it chose to be a *regulation*, not a *directive*.

That distinction matters enormously, and understanding why requires a brief detour through European law.

21.1 A Brief History: The Electronic Signatures Directive

21.1.1 Directive 1999/93/EC

Before eIDAS, there was the **Electronic Signatures Directive** (Directive 1999/93/EC), adopted on December 13, 1999. The directive was born in the heady optimism of the dot-com era, when legislators across the world were scrambling to give electronic commerce a legal foundation. The United States had passed the E-Sign Act. UNCITRAL had published a Model Law on Electronic Signatures. Europe wanted its own framework, and it wanted one that would work across all (then 15) member states.

The directive’s core idea was sound: define what an ”advanced electronic signature” is, define what a ”qualified certificate” is, establish that qualified electronic signatures should not be denied legal effect, and require member states to set up supervisory bodies for certification service providers.

21.1.2 Why Directives Fail

Here is where European institutional mechanics become relevant to your code.

In EU law, a **directive** tells member states *what result to achieve* but lets each country decide *how* to achieve it through national legislation. A **regulation**, by contrast, is directly applicable in all member states without any national transposition — it becomes law everywhere on the same day, with the same text.

The Electronic Signatures Directive was a directive. This meant that each member state had to transpose it into national law, and each one did so differently. Germany created the *Signaturgesetz* (Signature Law). France created *le decret n. 2001-272*. Italy created the *Codice dell'Amministrazione Digitale*. Each country interpreted the directive's requirements through its own legal tradition, its own bureaucratic structures, and its own technical preferences.

The result was exactly what you would predict: 15 (later 28) different national implementations of the "same" legal framework, with subtly different definitions of what counted as an "advanced" signature, different requirements for certification service providers, different supervisory structures, and — critically for developers — different technical standards that qualified certificates had to meet.

The Electronic Signatures Directive used the term "Certification Service Provider" (CSP). eIDAS renamed these to "Trust Service Provider" (TSP) and, for the supervised ones, "Qualified Trust Service Provider" (QTSP). The terminology change was deliberate: eIDAS covers more than just certificates — it covers timestamps, seals, website authentication, and electronic delivery services. If you encounter "CSP" in older documentation, read it as "TSP."

21.1.3 Cross-Border Failure

The most damaging consequence of the directive's fragmentation was the failure of cross-border recognition. In theory, a qualified electronic signature from Germany should have been accepted in France, and vice versa. In practice, it often was not. Each country's government procurement portals, tax systems, and regulatory filing platforms had been built to recognize only their own national qualified certificates. A German doctor trying to sign a cross-border prescription electronically for a French patient would find that the French system did not recognize the German qualified certificate.

A 2012 study by the European Commission found that cross-border recognition of electronic signatures was "marginal at best." Of 13 member states surveyed, only 5 had any mechanism at all for recognizing qualified certificates from other EU countries. The directive had been in force for 13 years, and the internal market for electronic signatures was essentially non-existent.

This was the political context in which eIDAS was born.

21.1.4 The Lessons Learned

The Commission identified several root causes for the directive's failure:

1. **Directive vs. Regulation**: national transposition created divergent implementations
2. **No mandatory trust infrastructure**: there was no EU-wide list of qualified providers that all member states were required to accept
3. **Vague technical requirements**: the directive referenced "secure signature creation devices" without specifying what that meant technically
4. **No supervision framework**: how supervisory bodies should operate, what they should check, and what consequences non-compliance should carry were all left to national discretion
5. **Certificates only**: the directive covered only electronic signatures, ignoring related services like timestamps, seals, and electronic delivery

eIDAS addressed every single one of these failures.

21.2 eIDAS: Architecture of the Regulation

21.2.1 Regulation, Not Directive

eIDAS is a regulation. Article 288 of the Treaty on the Functioning of the European Union says: ”A regulation shall have general application. It shall be binding in its entirety and directly applicable in all Member States.”

This means: - The text is the same in all 27 member states (plus EEA countries through incorporation) - No national transposition is needed or permitted — the regulation *is* the law - National courts apply the regulation directly; citizens and businesses can invoke it directly - Divergent national interpretations can be challenged at the Court of Justice of the European Union

For developers, this has a profound practical consequence: the rules are the same everywhere. When eIDAS says that a qualified electronic signature has the legal effect equivalent to a handwritten signature, it means exactly that in Germany, France, Italy, Estonia, and all other member states. You do not need to check 27 different national laws to know what your signature is worth.

21.2.2 Scope and Structure

The eIDAS regulation has two main parts:

Chapter II: Electronic Identification (Articles 6-12) — establishes a framework for mutual recognition of national electronic identification schemes (eID). This is the part that deals with government-issued digital identities (like the German *Personalausweis* or the Estonian *ID-kaart*) and how they should be recognized across borders. We will not cover this in detail because it relates to identity management rather than document signing.

Chapter III: Trust Services (Articles 13-45) — this is where the electronic signature framework lives, and it is the focus of this chapter and the next four chapters.

The trust services chapter covers: - Electronic signatures (Articles 25-27) - Electronic seals (Articles 35-40) - Electronic timestamps (Articles 41-42) - Electronic delivery services (Articles 43-44) - Website authentication certificates (Article 45) - General provisions on trust service providers (Articles 13-24)

eIDAS does not mandate *how* electronic signatures must be implemented technically. It defines *legal categories* and *legal effects*, then defers to ETSI and CEN for the technical standards. The regulation deliberately separates legal requirements from technical implementation — a wise decision that has allowed the technical standards (PAdES, CAdES, XAdES) to evolve independently of the legal framework.

When eIDAS says ”advanced electronic signature,” it is defining a legal concept with four properties. When ETSI EN 319 142 says ”PAdES-B-T,” it is defining a technical implementation that satisfies those properties. The mapping between legal categories and technical standards is documented in Commission Implementing Decision (EU) 2015/1506.

21.3 The Three Tiers of Electronic Signatures

The heart of eIDAS for developers is its three-tier classification of electronic signatures. Each tier has progressively stricter requirements and progressively stronger legal effects.

21.3.1 Tier 1: Electronic Signature (Article 3(10))

The broadest tier. Article 3(10) defines an electronic signature as:

> ”electronic signature” means data in electronic form which is attached to or logically associated with other data in electronic form and which is used by the signatory to sign
>
> — Regulation (EU) No 910/2014 (eIDAS), Article 3(10)

That is it. An electronic signature under eIDAS can be: - A scanned image of a handwritten signature pasted into a PDF - A typed name at the bottom of an email - A checkbox on a web form saying ”I agree” - A click of a ”Sign” button that records user intent

There is no cryptographic requirement. There is no certificate requirement. There is no requirement for any specific technology at all.

The legal effect of a basic electronic signature is defined in Article 25(1): it ”shall not be denied legal effect and admissibility as evidence in legal proceedings solely on the grounds that it is in an electronic form.” This is a non-discrimination principle — courts cannot reject evidence *just* because it is electronic. But it says nothing about the *weight* of that evidence. A typed name has much less evidentiary value than a cryptographic signature with a qualified certificate.

For developers building signing workflows: many business processes do not require more than a basic electronic signature. Click-to-sign, draw-to-sign, and type-to-sign all count as electronic signatures under eIDAS. If your use case is internal corporate approvals, non-binding agreements, or informal consent, a basic electronic signature may be legally sufficient. Check with your legal team, but do not assume you always need cryptography.

21.3.2 Tier 2: Advanced Electronic Signature (Article 3(11) and Article 26)

The middle tier. Article 3(11) defines an advanced electronic signature as an electronic signature that meets the requirements set out in Article 26.

Article 26 specifies four requirements:

> An advanced electronic signature shall meet the following requirements:
>
> (a) it is uniquely linked to the signatory;
>
> (b) it is capable of identifying the signatory;
>
> (c) it is created using electronic signature creation data that the signatory can, with a high level of confidence, use under his sole control; and
>
> (d) it is linked to the data signed therewith in such a way that any subsequent change in the data is detectable.
>
> — Regulation (EU) No 910/2014 (eIDAS), Article 26

Let us examine each requirement from a technical perspective.

21.3.2.1 (a) Uniquely linked to the signatory

The signature must be produced using data (typically a private key) that is uniquely associated with one specific natural person. In practice, this means: a private key that belongs to exactly one person, used to produce a cryptographic signature that can only have been created with that key.

A shared private key used by multiple employees would fail this requirement. A department-level signing key would fail this requirement. The key must be tied to a specific individual.

21.3.2.2 (b) Capable of identifying the signatory

The signature (together with its associated data, such as a certificate) must make it possible to determine *who* signed. A raw RSA or ECDSA signature over a hash does not, by itself, identify anyone — it just proves that whoever held the private key produced the signature. Combined with an X.509 certificate that binds the public key to an identity (name, organization, email), the requirement is satisfied.

21.3.2.3 (c) Sole control

The signatory must be the only person who can use the signing key. ”With a high level of confidence” is the qualifier — absolute sole control is impossible to guarantee (keys can be stolen), but the system must make unauthorized use difficult.

Sole control mechanisms include: - **Password-protected PKCS#12 keystore**: the signer must know the password - **Smart card with PIN**: the signer must possess the card and know the PIN - **HSM with authentication**: the signer must authenticate (username + password + OTP) before the HSM will sign - **Biometric activation**: fingerprint or face recognition to authorize signing

”Sole control” is where many implementations fail the advanced signature test. If a signing key is stored in a server-side keystore and an application signs on behalf of users without meaningful per-signature authentication, the resulting signatures are arguably *not* advanced electronic signatures — because the user does not have sole control over the key.

The fact that the user logged into the application is not sufficient. The authentication must be linked to the *signing action*, not just the *session*. This is the distinction that the SCAL (Sole Control Assurance Level) framework addresses — more on this in Chapter 22.

21.3.2.4 (d) Linked to the data

Any change to the signed data must be detectable. This is the integrity property that every digital signature scheme provides: if you change even one bit of the signed data, the signature will no longer verify. For PDF signatures, the byte range mechanism (Chapter 13) ensures this linkage.

21.3.3 Tier 3: Qualified Electronic Signature (Article 3(12))

The highest tier, and the most consequential. Article 3(12) defines a qualified electronic signature as:

> ”qualified electronic signature” means an advanced electronic signature that is created by a qualified electronic signature creation device, and which is based on a qualified certificate for electronic signatures
>
> — Regulation (EU) No 910/2014 (eIDAS), Article 3(12)

A QES is an advanced electronic signature (all four Article 26 properties) that *additionally*: 1. Uses a **Qualified Signature Creation Device (QSCD)** — certified hardware that protects the signing key 2. Is based on a **Qualified Certificate** — issued by a Qualified Trust Service Provider (QTSP) listed in the EU Trusted List

The legal effect of a QES is defined in Article 25(2): ”A qualified electronic signature shall have the equivalent legal effect of a handwritten signature.” This is not a non-discrimination principle — it is an *equivalence* rule. A QES is legally equal to pen on paper. No court in any EU member state can treat it differently.

Furthermore, Article 25(3) establishes mutual recognition: ”A qualified electronic signature based on a qualified certificate issued in one Member State shall be recognised as a qualified electronic signature in all other Member States.” This is the cross-border recognition that the 1999 directive failed to achieve.

21.4 Key Articles in Detail

21.4.1 Article 25: Legal Effects of Electronic Signatures

Article 25 is the most cited provision in eIDAS, and it deserves careful reading.

Table 21.1: Article 25 — Legal effects summarized

Paragraph	Rule	Practical Meaning
25(1)	Electronic signatures shall not be denied legal effect solely because they are electronic	Non-discrimination: courts cannot reject electronic evidence just because it is not paper
25(2)	QES shall have the equivalent legal effect of a handwritten signature	Legal equivalence: a QES *is* a handwritten signature for all legal purposes
25(3)	QES based on a qualified certificate from one member state shall be recognized in all member states	Cross-border recognition: no country can reject another country’s QES

Note what Article 25 does *not* say: it does not say that advanced electronic signatures (AdES) have the legal effect of handwritten signatures. An AdES benefits only from the non-discrimination rule of 25(1) — it cannot be rejected *solely* because it is electronic, but a

court is free to assign it less evidentiary weight than a handwritten signature based on the circumstances.

This creates a clear hierarchy: - **QES**: automatic legal equivalence to handwritten (Article 25(2)) - **AdES**: cannot be dismissed as evidence just for being electronic (Article 25(1)), but the court decides its weight - **Basic electronic signature**: same as AdES — cannot be dismissed for being electronic, but its evidentiary weight is at the court's discretion

Many developers (and their managers) misunderstand Article 25(1) as meaning "all electronic signatures are legally valid." That is not what it says. It says they cannot be denied legal effect *solely* because they are electronic. A typed name at the bottom of an email is an electronic signature under eIDAS, but it provides very little evidence about who actually sent the email, whether they intended to be bound by its contents, or whether the email was tampered with. A court will weigh this evidence accordingly.

Only QES gets automatic equivalence. Everything else is a matter of evidence and judicial discretion.

21.4.2 Article 28: Qualified Certificates for Electronic Signatures

Article 28 requires that qualified certificates contain specific information, listed in Annex I of the regulation. A qualified certificate must contain:

1. An indication (at least in machine-processable form) that the certificate is a qualified certificate
2. A set of data representing the QTSP, including the member state where it is established and its name
3. The signatory's name (or a pseudonym, clearly identified as such)
4. Electronic signature validation data (the public key) corresponding to the electronic signature creation data (the private key)
5. The period of validity
6. The certificate serial number
7. The advanced electronic signature or seal of the issuing QTSP
8. The location where the certificate supporting the QTSP's signature/seal is available free of charge
9. The location of the QTSP's certificate revocation status services

In X.509 terms, this translates to specific extensions and naming conventions: - The certificate policies extension must contain the OID for qualified certificates (0.4.0.194112.1.2 for QES) - The subject DN must contain the signatory's name - The issuer DN identifies the QTSP - CRL Distribution Points and/or OCSP responder URLs must be present - The QCStatements extension (OID 0.4.0.1862.1) carries machine-readable qualification information

```
-- QCStatements extension (ETSI EN 319 412-5)
QCStatements ::= SEQUENCE OF QCStatement

QCStatement ::= SEQUENCE {
    statementId   OBJECT IDENTIFIER,
    statementInfo ANY DEFINED BY statementId OPTIONAL
}

-- Key OIDs:
```

```
-- id-etsi-qcs-QcCompliance  (0.4.0.1862.1.1)  = "this is a qualified cert"
-- id-etsi-qcs-QcSSCD        (0.4.0.1862.1.4)  = "key is on a QSCD"
-- id-etsi-qcs-QcType         (0.4.0.1862.1.6)  = type (esign, eseal, web)
```

21.4.3 Article 29: Requirements for Qualified Signature Creation Devices

Article 29 states that qualified electronic signature creation data (the private key) must be managed by a QSCD that meets the requirements of Annex II.

Annex II requires that QSCDs must:

1. Ensure the confidentiality of the electronic signature creation data
2. Ensure that the electronic signature creation data used for signature creation can occur only once (practical uniqueness)
3. Ensure that the electronic signature creation data cannot be derived with reasonable assurance, and the signature is reliably protected against forgery using currently available technology
4. Ensure that the electronic signature creation data can be reliably protected by the legitimate signatory against use by others

In practice, this means: the private key must be generated on, and never leave, a certified hardware device. The certification is typically Common Criteria EAL4+ (or equivalent) for the device, plus specific Protection Profiles defined by CEN and ETSI.

Examples of certified QSCDs: - Smart cards (e.g., IDPrime MD from Thales, CardOS from Atos) - USB tokens (e.g., SafeNet eToken 5110) - Hardware Security Modules (e.g., Thales Luna Network HSM, Utimaco SecurityServer) - Remote QSCDs operated by a QTSP (the key lives in an HSM in a data center, accessed via API)

The rise of remote QSCDs has been the biggest practical change since eIDAS entered into force. The original assumption was that QES required a physical device in the signer's possession (a smart card in their wallet). But eIDAS Annex II does not require *physical possession* — it requires sole control. A private key in an HSM operated by a QTSP, accessible only through strong multi-factor authentication specific to the signer and the signing operation, satisfies the requirement. This opened the door to remote signing (Chapter 22), which is now the dominant model for QES in practice.

21.4.4 Article 35-40: Electronic Seals

Electronic seals are the organizational counterpart to electronic signatures. While a signature is created by a natural person (a human), a seal is created by a legal person (a company, government agency, association).

The seal articles mirror the signature articles almost exactly: - Article 35(1): non-discrimination for electronic seals - Article 35(2): qualified electronic seal enjoys a presumption of integrity and origin - Article 36: AdESeal requirements (same four properties as AdES, adapted for legal persons) - Article 38: qualified certificates for electronic seals (same as Art. 28, but for organizations)

The key difference: Article 35(2) says a qualified seal enjoys a "presumption of integrity of the data and of correctness of the origin of that data." Note the careful wording — it says

nothing about the legal *intent* behind the seal. A signature expresses consent or approval. A seal proves origin and integrity. Chapter 21 explores this distinction in depth.

21.5 Mutual Recognition

21.5.1 Within the EU

Article 25(3) establishes that a QES based on a qualified certificate from one member state is recognized as a QES in all member states. Article 35(3) does the same for qualified electronic seals.

This means: - An Austrian QES is valid in Portugal - A Lithuanian qualified seal is valid in Spain - An Estonian qualified timestamp is valid in Germany

Member states **cannot** impose additional requirements for cross-border recognition. They cannot require that QES certificates be issued by domestic QTSPs. They cannot require additional technical features beyond what eIDAS specifies.

The mechanism that makes this work is the **EU Trusted List (EUTL)** — a machine-readable, cryptographically signed registry of all QTSPs in the EU. If a certificate chains to a QTSP on the EUTL, it qualifies for cross-border recognition. Chapter 20 covers the EUTL in detail.

In practice, cross-border recognition is still imperfect. While the *legal* framework is clear, many national IT systems have not been updated to consume the EUTL and accept certificates from other member states. A 2021 European Commission survey found that while most member states had updated their legal frameworks, actual system-level interoperability lagged significantly.

If your software validates signatures against a fixed list of trusted certificates (rather than dynamically checking the EUTL), you are likely rejecting valid QES from other member states. This is a compliance failure under eIDAS — Article 25(3) does not have exceptions for ”we haven’t updated our trust store yet.”

21.5.2 Non-EU Recognition

eIDAS Article 14 allows the European Commission to adopt implementing acts recognizing electronic identification schemes from third countries, provided they meet equivalent requirements.

In practice, non-EU recognition is sparse: - **Switzerland** has a bilateral agreement with the EU that provides for mutual recognition of qualified trust services under the ZertES (Swiss Electronic Signatures Act) - **United Kingdom** post-Brexit: the UK retained eIDAS as domestic law (the UK eIDAS regulation) but mutual recognition lapsed; UK QES are not automatically recognized in the EU and vice versa - **United States**: no formal equivalence arrangement; US digital signatures have no special status under eIDAS (and eIDAS signatures have no special status under US law)

For developers serving global markets, this means: do not assume that a qualified certificate from a non-EU QTSP will be recognized as ”qualified” in the EU, even if the QTSP provides technically identical services. Qualification is a *legal status* granted by an EU member state’s supervisory body, not a technical property of the certificate.

21.6 eIDAS 2.0: Regulation (EU) 2024/1183

On April 11, 2024, the revised eIDAS regulation — commonly called **eIDAS 2.0** or Regulation (EU) 2024/1183 (eIDAS 2.0) — entered into force. The revision keeps the trust services framework largely intact but adds a transformative new component: the **European Digital Identity Wallet (EUDI Wallet)**.

21.6.1 What Changed

The key changes in eIDAS 2.0:

1. **EUDI Wallet**: Every member state must provide a digital identity wallet to its citizens and residents by 2026. The wallet will carry government-issued Person Identification Data (PID) and Qualified Electronic Attestations of Attributes (QEAAs). Crucially, the wallet will be able to create QES — making qualified signatures accessible to every EU citizen from their smartphone.
2. **Qualified Electronic Attestations of Attributes (QEAAs)**: A new trust service type. QEAAs are digitally signed attestations of specific attributes (age, professional qualifications, university degrees) that can be presented through the EUDI wallet. Think of them as digital equivalents of physical certificates and credentials.
3. **Remote QSCD in the wallet**: The EUDI wallet can use a remote QSCD operated by a QTSP for creating qualified signatures. The private key lives in an HSM in the QTSP's data center, but the wallet provides the authentication and sole control mechanisms.
4. **Free QES for citizens**: Article 45e requires that natural persons be offered QES creation through the wallet free of charge for non-professional use. This is a dramatic change — until now, QES required purchasing a certificate from a commercial QTSP, which typically costs 50-300 EUR per year.
5. **Qualified Electronic Ledgers**: A new trust service type for distributed ledger technology (blockchain). ETSI is currently developing the technical standards.

21.6.2 Impact on Developers

For PDF signature developers, eIDAS 2.0 means:

- **Expect a surge in QES volume**: Free QES through the wallet will dramatically increase the number of qualified signatures in circulation. Your validation code needs to handle EUTL lookups efficiently.
- **Wallet-initiated signing flows**: The EUDI wallet defines a new interaction pattern where the wallet app initiates a signing flow with a remote QSCD. If your application accepts signed documents, you need to support the signature formats the wallet produces.
- **QEAA validation**: You may need to validate not just the signature on a document, but also QEAAs presented alongside it (e.g., a doctor signing a prescription presents their medical qualification attestation).

The EUDI wallet's Architecture Reference Framework (ARF) is publicly available and under active development. As of early 2026, large-scale pilots are running in multiple sectors across the EU. The technical specifications are not yet final, but the direction is clear: QES will move from a niche enterprise technology to a consumer feature on smartphones. Plan your

architecture accordingly.

21.7 The Gap Between Regulation and Reality

21.7.1 What the Regulation Says

eIDAS paints an elegant picture: three clearly defined tiers, qualified trust services supervised by competent national bodies, mutual recognition across borders, machine-readable trusted lists that software can consume programmatically.

21.7.2 What Actually Happens

The reality is more complicated.

National implementation varies despite the regulation's direct applicability. While member states cannot transpose the regulation differently (it is directly applicable), they *can* adopt supplementary national measures. Germany's *Vertrauensdienstegesetz* (Trust Services Act), France's *decret relatif a la signature electronique*, and Italy's *Codice dell'Amministrazione Digitale* all contain national provisions that fill gaps the regulation left open. These national provisions sometimes create practical differences in how QES is implemented and accepted.

EUTL consumption is not universal. Many government and enterprise systems still validate signatures against hardcoded trust stores rather than dynamically checking the EUTL. This means they accept domestic QES but reject equally valid QES from other member states.

QSCD certification is slow and expensive. Getting a signature creation device certified as a QSCD takes 12-24 months and costs hundreds of thousands of euros. This limits the number of available QSCDs and concentrates the market among a few hardware vendors.

The "qualified" premium. Qualified certificates cost significantly more than standard certificates — typically 50-300 EUR per year for individuals, and more for organizations. This price barrier has limited adoption, particularly among small businesses and individuals. eIDAS 2.0's free QES through the wallet aims to address this, but the wallet is not yet widely deployed.

Interoperability testing is ongoing. The European Commission funds regular interoperability testing events (called "plugtests") through ETSI, where QTSPs and software vendors test their implementations against each other. These events consistently reveal interoperability issues — signatures that validate in one tool but fail in another, timestamps that are accepted by some validators but rejected by others, EUTL entries that are ambiguous about the services a QTSP provides.

21.8 Common Misconceptions

21.8.1 "eIDAS makes all digital signatures legal"

No. eIDAS creates a *framework* with three tiers. Only QES gets automatic legal equivalence to handwritten signatures. Basic electronic signatures and advanced electronic signatures benefit only from the non-discrimination principle — they cannot be rejected *solely* for being electronic, but their legal weight is a matter for the court to decide.

21.8.2 ”If my signature uses a certificate, it’s advanced”

Not necessarily. An advanced electronic signature must satisfy all four requirements of Article 26: uniquely linked to the signatory, capable of identifying them, under sole control, and linked to the data. A certificate-based signature where the private key is stored in a shared server-side keystore, accessible to multiple system administrators, arguably fails the ”sole control” requirement.

21.8.3 ”PAdES = eIDAS compliance”

Not automatically. PAdES defines technical profiles for PDF signatures. Using PAdES-B-LTA with a qualified certificate on a QSCD produces a QES. Using PAdES-B-LTA with a self-signed certificate produces a technically well-structured signature with zero eIDAS qualification. The eIDAS tier depends on *who issued the certificate* and *where the key is stored*, not on the PDF signature format alone.

21.8.4 ”My country’s signatures are recognized everywhere in the EU”

Only if they are *qualified*. A non-qualified signature from Germany has no special status in France — it is just evidence, to be evaluated by the French court on its merits. Only QES enjoys automatic cross-border recognition under Article 25(3).

21.8.5 ”eIDAS applies outside the EU”

eIDAS is EU law. It applies within the EU, the EEA (through the EEA Agreement), and countries with specific bilateral arrangements (Switzerland). It does not apply in the United States, China, Japan, or anywhere else, unless those jurisdictions choose to recognize eIDAS-qualified signatures under their own legal frameworks.

However, eIDAS has become a de facto global reference model. Countries designing their own electronic signature frameworks often look to eIDAS for inspiration. The three-tier model (basic, advanced, qualified) has been adopted in various forms by multiple non-EU jurisdictions.

21.9 What Your Code Needs to Do

For developers building PDF signature creation or validation software, eIDAS imposes specific technical requirements depending on the tier you are targeting:

21.9.1 For Basic Electronic Signatures

Nothing beyond what PDF already requires. Any valid PDF signature — even a simple `/SubFilter /adbe.pkcs7.detached` with a self-signed certificate — is an electronic signature under Article 3(10).

21.9.2 For Advanced Electronic Signatures

Your signing workflow must ensure: 1. **Unique linkage**: one private key per signer (never share keys) 2. **Identification**: the certificate must contain the signer’s identity (name, at

minimum) 3. **Sole control**: meaningful per-signature authentication (not just session login) 4. **Data linkage**: the signature must cover the entire document (standard byte range coverage)

Technically, this means: PAdES-B-B or higher, with a certificate issued to a specific individual, and a signing flow that authenticates the signer for each signature.

21.9.3 For Qualified Electronic Signatures

All of the above, plus: 1. **Qualified certificate**: issued by a QTSP listed on the EUTL 2. **QSCD**: the private key must be on a certified qualified signature creation device 3. **Recommendation: PAdES-B-LTA**: while not strictly required by eIDAS, PAdES-B-LTA ensures long-term verifiability

21.9.4 For Validation

If your software validates signatures and needs to determine the eIDAS tier:

```
/**
 * Determines the eIDAS tier of a PDF signature.
 * This is a simplified illustration - production code should use EU DSS
 * which handles the full complexity of EUTL parsing and chain validation.
 */
public EidasTier determineEidasTier(X509Certificate signerCert,
                                    boolean keyOnQscd) {
    // Check if the certificate is qualified (issued by a QTSP)
    boolean isQualifiedCert = isListedOnEutl(signerCert);

    // Check for QCStatements extension
    boolean hasQcCompliance = hasQcStatement(signerCert,
            "0.4.0.1862.1.1");  // id-etsi-qcs-QcCompliance
    boolean hasQscdStatement = hasQcStatement(signerCert,
            "0.4.0.1862.1.4");  // id-etsi-qcs-QcSSCD

    if (isQualifiedCert && hasQcCompliance && (keyOnQscd || hasQscdStatement)) {
        return EidasTier.QUALIFIED;     // QES - Art. 25(2) applies
    }

    if (meetsArticle26Requirements(signerCert)) {
        return EidasTier.ADVANCED;      // AdES - Art. 25(1) applies
    }

    return EidasTier.BASIC;            // Basic - Art. 25(1) applies
}

private boolean hasQcStatement(X509Certificate cert, String oid) {
    byte[] extValue = cert.getExtensionValue("1.3.6.1.5.5.7.1.3");
    if (extValue == null) {
        return false;
    }
    // Parse QCStatements ASN.1 structure
    ASN1Sequence qcStatements = ASN1Sequence.getInstance(
            ASN1OctetString.getInstance(extValue).getOctets());
    for (ASN1Encodable statement : qcStatements) {
        ASN1Sequence seq = ASN1Sequence.getInstance(statement);
```

```
            ASN1ObjectIdentifier statementId =
                    ASN1ObjectIdentifier.getInstance(seq.getObjectAt(0));
            if (statementId.getId().equals(oid)) {
                return true;
            }
        }
        return false;
    }
```

The code above is a simplified illustration. Production eIDAS tier determination is significantly more complex because:

- EUTL lookup requires parsing the trusted list XML, validating its signature, and traversing the LOT hierarchy
- Certificate chain validation must check revocation status for every certificate in the chain
- QSCD status may need to be inferred from the certificate's QCStatements rather than verified directly
- Some QTSPs issue certificates that are qualified for some services but not others

For production use, the **EU DSS framework** (Chapter 23) provides a complete, tested implementation of eIDAS tier determination. Do not implement this from scratch unless you have very specific requirements that EU DSS cannot meet.

21.10 Key Takeaways

- eIDAS (Regulation (EU) No 910/2014 (eIDAS)) is an EU regulation — directly applicable in all member states, no national transposition required
- It replaced the failed Electronic Signatures Directive (1999/93/EC), which suffered from fragmented national implementations
- Three tiers: basic electronic signature (any electronic indication of signing intent), advanced electronic signature (four properties: unique linkage, identification, sole control, data linkage), qualified electronic signature (advanced + QSCD + qualified certificate)
- Only QES has automatic legal equivalence to a handwritten signature (Article 25(2))
- Cross-border recognition is mandatory for QES (Article 25(3)) — enforced through the EU Trusted List
- eIDAS 2.0 (Regulation (EU) 2024/1183 (eIDAS 2.0)) adds the EUDI wallet, making QES accessible to all EU citizens via smartphone
- The gap between regulation and implementation is real — code that hardcodes trust stores instead of checking the EUTL violates the mutual recognition principle
- For production eIDAS validation, use the EU DSS framework (Chapter 23) rather than implementing from scratch

Chapter 22

Qualified Certificates & Trust Services

A certificate is just a data structure — some ASN.1 fields, a public key, and a signature from someone who claims to be a Certificate Authority. What makes a certificate *qualified* is not a technical property you can detect by parsing bytes alone. It is a legal status, conferred by a government supervisory body, published in a machine-readable trust infrastructure, and backed by an institutional apparatus that most developers have never heard of, let alone queried programmatically.

This chapter takes you through that entire apparatus: the Qualified Trust Service Providers that issue qualified certificates, the EU Trusted List that publishes who they are, the technical requirements that separate a qualified certificate from an ordinary one, the QSCD hardware that protects the keys, and the Java code you need to actually parse and verify all of this. If Chapter 19 was the legal framework, this chapter is the plumbing.

22.1 Qualified Trust Service Providers (QTSPs)

22.1.1 What a QTSP Is

A **Qualified Trust Service Provider** is an organization that has been audited, approved, and listed by a national supervisory body within the EU as meeting the requirements of Regulation (EU) No 910/2014 (eIDAS) Articles 20-24. The word ”qualified” is doing serious work here — it is not a marketing term, not a self-declaration, and not something you obtain by installing software. It is a legal status, analogous to a bank receiving a banking license: you cannot call yourself a QTSP unless a government body says you are one.

A QTSP can provide one or more qualified trust services:

- **Qualified certificates for electronic signatures** (natural persons)
- **Qualified certificates for electronic seals** (legal persons)
- **Qualified timestamps** (RFC 3161 tokens from a qualified TSA)
- **Qualified electronic delivery services** (certified email equivalents)
- **Qualified website authentication certificates** (the controversial Article 45, rewritten in eIDAS 2.0)
- **Qualified electronic attestations of attributes** (new in Regulation (EU) 2024/1183 (eIDAS 2.0))

The qualification is *per-service*, not per-organization. A provider might be qualified for timestamps but not for certificates, or for seals but not for signatures. This matters when you are parsing the EU Trusted List — you need to check not just whether the provider is listed, but *which services* are qualified.

22.1.2 How to Become a QTSP

The path to becoming a QTSP is neither quick nor cheap. Here is the process, simplified but accurate:

1. **Establish the infrastructure.** You need HSMs (certified to Common Criteria EAL4+ or equivalent), a secure data center (typically ISO 27001 certified), redundant systems, trained personnel, and documented processes for every aspect of certificate lifecycle management — issuance, suspension, revocation, key generation, key destruction.
2. **Write a Certificate Practice Statement (CPS).** This is the document that describes, in exhaustive detail, how you operate: identity verification procedures, key management, certificate profiles, revocation handling, disaster recovery, incident response. A CPS for a QTSP is typically 80-150 pages long.
3. **Get a conformity assessment.** An accredited **Conformity Assessment Body (CAB)** evaluates your infrastructure, processes, and CPS against Regulation (EU) No 910/2014 (eIDAS) and the applicable ETSI standards (primarily ETSI EN 319 401 for general TSP requirements and ETSI EN 319 411-1/411-2 for certificate issuance). The assessment produces a **Conformity Assessment Report (CAR)**.
4. **Submit to the national supervisory body.** Each EU member state designates a supervisory body for QTSPs: in Germany, the **Bundesnetzagentur** (Federal Network Agency); in France, **ANSSI**; in Italy, **AgID** (Agenzia per l'Italia Digitale). You submit the CAR, your CPS, and supporting documentation.
5. **Supervisory body review and approval.** The supervisory body reviews everything, may request additional information or on-site inspections, and eventually grants or denies qualified status.
6. **Listing on the national Trusted List.** Once approved, the supervisory body adds your service(s) to the national **Trusted Service Status List (TSL)** — the country's portion of the EU Trusted List. Your qualified status takes effect when the updated TSL is published.
7. **Ongoing supervision.** Qualification is not a one-time event. Regulation (EU) No 910/2014 (eIDAS) Article 20(1) mandates audits at least every 24 months. Regulation (EU) No 910/2014 (eIDAS) Article 19(2) requires QTSPs to report security incidents to the supervisory body within 24 hours.

The entire process from initial infrastructure setup to EUTL listing typically takes **18-36 months** and costs **several hundred thousand to several million euros**, depending on the scope of services and the jurisdiction. This is by design: the barrier to entry ensures that only well-resourced, professionally operated organizations can issue qualified certificates.

If someone offers you "qualified certificates" from a provider that is not on the EUTL, they are either confused about what "qualified" means or deliberately misleading you. Always verify against the EUTL.

22.1.3 The Supervision Model

The eIDAS supervision model is a two-layer structure:

National layer: Each member state's supervisory body is responsible for QTSPs established in its territory. The supervisory body grants and revokes qualified status, maintains the national TSL, conducts or commissions audits, handles incident reports, and imposes sanctions for

non-compliance.

EU layer: The European Commission maintains the **List of Trusted Lists (LOTL)** — a master document that points to every national TSL. The Commission does not supervise individual QTSPs; it supervises the supervisory bodies indirectly, through the mutual evaluation mechanism and through the eIDAS Expert Group.

Table 22.1: Supervision hierarchy

Level	Entity	Responsibility
EU	European Commission	Maintains LOTL, sets technical standards via implementing acts, coordinates mutual evaluations
National	Supervisory Body (Bundesnetzagentur, ANSSI, AgID, etc.)	Grants/revokes qualified status, maintains national TSL, audits QTSPs, handles incidents
Provider	QTSP (DigiCert, D-Trust, InfoCert, etc.)	Operates trust services, issues certificates, complies with ETSI standards, reports incidents

22.2 The EU Trusted List (EUTL)

22.2.1 Architecture: The LOT Hierarchy

The EUTL is not a single list — it is a **hierarchy of cryptographically signed XML documents**:

1. The **LOTL (List of Trusted Lists)** is the root. Maintained by the European Commission, published at a well-known URL. The LOTL contains pointers (URLs) to every national TSL, along with the certificates used to verify each national TSL's signature.
2. Each **national TSL (Trusted Service Status List)** lists the trust service providers in that country, their services, the status of each service (granted, withdrawn, deprecated), and the CA certificates associated with each service.
3. Each TSL is **digitally signed** using an XML Advanced Electronic Signature (XAdES) — the same trust-no-one verification model applied to the trust infrastructure itself.

The LOTL URL is:

```
https://ec.europa.eu/tools/lotl/eu-lotl.xml
```

This URL has been stable for years, but production systems should make it configurable. The official reference for the LOTL location is Commission Implementing Decision (EU) 2015/1505. The LOTL itself contains a `NextUpdate` field indicating when the next version will be published. National TSLs are updated on varying schedules — some monthly, some quarterly, some whenever a change occurs.

22.2.2 XML Schema

The TSL XML schema is defined in ETSI TS 119 612 (replacing the older ETSI TS 102 231). The schema is verbose, deeply nested, and frankly exhausting to read — but it is well-structured

and consistent.

```
<!-- Simplified LOTL structure -->
<TrustServiceStatusList xmlns="http://uri.etsi.org/02231/v2#">
  <SchemeInformation>
    <TSLVersionIdentifier>5</TSLVersionIdentifier>
    <TSLSequenceNumber>321</TSLSequenceNumber>
    <TSLType>
      http://uri.etsi.org/TrstSvc/TrustedList/TSLType/EUlistofthelists
    </TSLType>
    <SchemeOperatorName>
      <Name xml:lang="en">European Commission</Name>
    </SchemeOperatorName>
    <ListIssueDateTime>2026-02-15T00:00:00Z</ListIssueDateTime>
    <NextUpdate>
      <dateTime>2026-08-15T00:00:00Z</dateTime>
    </NextUpdate>
  </SchemeInformation>

  <TrustServiceProviderList>
    <TrustServiceProvider>
      <TSPInformation>
        <TSPName><Name xml:lang="en">Germany</Name></TSPName>
      </TSPInformation>
      <TSPServices>
        <TSPService>
          <ServiceInformation>
            <ServiceTypeIdentifier>
              http://uri.etsi.org/TrstSvc/Svctype/TLIssuer
            </ServiceTypeIdentifier>
            <ServiceSupplyPoints>
              <ServiceSupplyPoint>
                https://www.bundesnetzagentur.de/.../TrustedList.xml
              </ServiceSupplyPoint>
            </ServiceSupplyPoints>
          </ServiceInformation>
        </TSPService>
      </TSPServices>
    </TrustServiceProvider>
    <!-- ... 26 more countries ... -->
  </TrustServiceProviderList>
</TrustServiceStatusList>
```

Each **national TSL** then contains the actual QTSP entries with their service certificates:

```
<!-- Simplified national TSL structure (Germany) -->
<TrustServiceStatusList xmlns="http://uri.etsi.org/02231/v2#">
  <TrustServiceProviderList>
    <TrustServiceProvider>
      <TSPInformation>
        <TSPName>
          <Name xml:lang="en">D-Trust GmbH</Name>
        </TSPName>
      </TSPInformation>
      <TSPServices>
        <TSPService>
```

```
            <ServiceInformation>
              <ServiceTypeIdentifier>
                http://uri.etsi.org/TrstSvc/Svctype/CA/QC
              </ServiceTypeIdentifier>
              <ServiceStatus>
                http://uri.etsi.org/TrstSvc/TrustedList/Svcstatus/granted
              </ServiceStatus>
              <StatusStartingTime>2018-07-01T00:00:00Z</StatusStartingTime>
              <ServiceDigitalIdentity>
                <DigitalId>
                  <X509Certificate>MIIE...base64...</X509Certificate>
                </DigitalId>
              </ServiceDigitalIdentity>
              <ServiceInformationExtensions>
                <Extension Critical="true">
                  <Qualifications>
                    <QualificationElement>
                      <Qualifiers>
                        <Qualifier uri="http://uri.etsi.org/TrstSvc/
                          TrustedList/SvcInfoExt/QCWithQSCD"/>
                      </Qualifiers>
                    </QualificationElement>
                  </Qualifications>
                </Extension>
              </ServiceInformationExtensions>
            </ServiceInformation>
          </TSPService>
        </TSPServices>
      </TrustServiceProvider>
    </TrustServiceProviderList>
  </TrustServiceStatusList>
```

22.2.3 Key Service Types and Statuses

Table 22.2: EUTL service type identifiers for PDF-relevant services

Service Type URI	Meaning
…/Svctype/CA/QC	CA issuing **qualified certificates** (check qualifiers for esign/eseal/web)
…/Svctype/TSA/QTST	**Qualified timestamp** authority
…/Svctype/Certstatus/OCSP/QC	**Qualified OCSP** responder
…/Svctype/Certstatus/CRL/QC	**Qualified CRL** issuer

Table 22.3: EUTL service status values

Status URI	Meaning
…/`Svcstatus/granted`	Currently qualified — this is what you are looking for
…/`Svcstatus/withdrawn`	Qualification has been withdrawn
… `/Svcstatus/recognisedatnationallevel`	Not EU-qualified, but recognized under national law
…/`Svcstatus/deprecated`	Service deprecated by the supervisory body

A common mistake in EUTL parsing is to check only the current status. You also need to check the **service history**. A service that was `granted` from 2019-01-01 to 2023-06-30 and then `withdrawn` was qualified during that window. A certificate issued during the `granted` period is still a qualified certificate, even though the service is now withdrawn.

The TSL includes a `ServiceHistory` element with `ServiceHistoryInstance` entries recording every status change. Your validation logic must check whether the certificate was issued (and the signature created) during a period when the service status was `granted`.

22.2.4 Cryptographic Signing of the TSL

Every TSL (including the LOTL) is signed with an XAdES signature. To trust the contents, you must:

1. **Verify the XML signature** on the TSL document
2. **Verify the signing certificate** against the LOTL (for national TSLs) or against the Commission's well-known certificate (for the LOTL itself)
3. **Check freshness** — the TSL must not be past its `NextUpdate` date (or not past it by more than a configurable grace period)

The LOTL signing certificate is published by the European Commission at a well-known URL. When the signing certificate rotates, the new certificate is announced in advance in the LOTL itself — this is the **pivot mechanism**, and it is one of the trickiest parts of EUTL parsing to implement correctly.

For production systems, implement a scheduled refresh — daily or weekly — that re-downloads, re-verifies, and re-parses the entire EUTL hierarchy. The EU DSS library (Chapter 23) provides a `TrustedListsCertificateSource` that handles all of this automatically, including caching, refresh scheduling, signature verification, and the pivot mechanism.

22.3 Qualified Certificates: What Makes Them Qualified

A certificate is "qualified" when three conditions are simultaneously true:

1. It is issued by a CA whose certificate issuance service has `granted` status in the EUTL
2. The certificate contains specific technical indicators (the QCStatements extension)

3. The CA followed the procedures defined in its CPS, which was approved during the conformity assessment

Condition 3 is not something you can verify programmatically — it is an institutional guarantee, backed by the supervisory body's ongoing oversight. Conditions 1 and 2 are what your code checks.

22.3.1 The QCStatements Extension

The technical indicator of a qualified certificate is the **QCStatements extension** (OID `1.3.6.1.5.5.7.1.3`), defined in ETSI EN 319 412-5. This extension contains a sequence of statements, each identified by an OID:

Table 22.4: QCStatements OIDs relevant to PDF signatures

OID	Short Name	Meaning
`0.4.0.1862.1.1`	id-etsi-qcs-QcCompliance	This certificate is issued as a **qualified certificate** under Regulation (EU) No 910/2014 (eIDAS)
`0.4.0.1862.1.4`	id-etsi-qcs-QcSSCD	The private key resides on a **Qualified Signature/Seal Creation Device**
`0.4.0.1862.1.6`	id-etsi-qcs-QcType	The type: esign (…`6.1` = signature), eseal (…`6.2` = seal), web (…`6.3`)
`0.4.0.1862.1.3`	id-etsi-qcs-QcRetentionPeriod	How long the CA retains registration information after certificate expiry
`0.4.0.1862.1.5`	id-etsi-qcs-QcPDS	URL of the PKI Disclosure Statement (human-readable CPS summary)
`0.4.0.1862.1.2`	id-etsi-qcs-QcLimitValue	Monetary transaction limit (rarely used)

Do **not** trust the QCStatements extension alone. Anyone can put `id-etsi-qcs-QcCompliance` in a self-signed certificate. The QCStatements assertion is only meaningful when the issuing CA is listed in the EUTL with `granted` status for a `CA/QC` service.

The correct validation logic is:

1. Check that the issuing CA's certificate matches one in the EUTL with `ServiceTypeIdentifier` of `CA/QC` and status `granted`
2. Check the period: the certificate's `notBefore` date must fall within the service's `granted` status period
3. *Then* check QCStatements for the specific type (esign, eseal, web)
4. *Then* check for QcSSCD if you need to verify QSCD usage

Steps 1-2 without steps 3-4: you know the CA is qualified, but not necessarily this specific certificate. Steps 3-4 without steps 1-2: you have a self-asserted claim with no institutional backing. You need all four.

22.3.2 Certificate Policy OIDs

Qualified certificates also carry specific **Certificate Policy** OIDs in the `certificatePolicies` extension (OID `2.5.29.32`), defined in ETSI EN 319 411-2:

Table 22.5: Certificate Policy OIDs for qualified certificates

OID	Meaning
`0.4.0.194112.1.2`	QCP-n: Qualified Certificate Policy for **natural persons** (signatures)
`0.4.0.194112.1.3`	QCP-l: Qualified Certificate Policy for **legal persons** (seals)
`0.4.0.194112.1.1`	QCP-n-qscd: Natural person certificates with QSCD
`0.4.0.194112.1.4`	QCP-l-qscd: Legal person certificates with QSCD
`0.4.0.194112.1.0`	QCP-w: Website authentication

The `-qscd` variants indicate that the CA has verified that the subscriber's private key is on a QSCD. This is a stronger assertion than the QcSSCD statement alone — the policy OID says "the CA has verified the key is on a QSCD according to the procedures defined in ETSI EN 319 411-2."

22.3.3 QSCD Requirements

A Qualified Signature Creation Device (or Qualified Seal Creation Device — QSCD covers both) must meet the requirements of Regulation (EU) No 910/2014 (eIDAS) Annex II:

1. The private key must be generated on the device and never leave it
2. The signing operation must occur on the device itself
3. Access to the private key must require authentication of the legitimate holder (PIN, biometric, etc.)
4. The private key cannot be derived from signatures or the public key with reasonable assurance

Certification path: Common Criteria evaluation (ISO/IEC 15408) at EAL4+ against Protection Profiles defined in EN 419 211 (for smart cards and tokens) or EN 419 221 (for HSMs and remote signing). A national certification body (BSI in Germany, ANSSI in France) performs the evaluation.

Hardware categories in practice:

- **Smart cards** (Thales IDPrime MD, Atos CardOS): private key generated on chip, never exported, signing performed on-card
- **USB tokens** (SafeNet eToken 5110, YubiKey 5 FIPS): same security model as smart cards, USB form factor
- **HSMs** (Thales Luna, Utimaco SecurityServer): rack-mounted devices for server-side signing; basis for remote signing (Chapter 22)

Not every HSM is a certified QSCD. An HSM may be FIPS 140-2 Level 3 or Common Criteria EAL4+ certified for *general cryptographic operations* without being certified as a QSCD under

the specific EN 419 211/221 Protection Profiles. The QSCD certification must reference the eIDAS-specific Protection Profiles.

22.4 Real-World QTSPs

When you are debugging why a particular qualified certificate does or does not validate, knowing who issued it and where they are listed is invaluable. Here are the QTSPs you are most likely to encounter:

D-Trust GmbH (Germany) — a subsidiary of the Bundesdruckerei (the organization that prints German passports). Listed in the German TSL, supervised by the Bundesnetzagentur. Dominant in German government and enterprise contexts.

DigiCert / QuoVadis (Netherlands, Belgium) — DigiCert acquired QuoVadis in 2019, inheriting QTSP status in multiple EU countries. One of the few providers with both AATL and eIDAS qualified status — useful when documents must validate in both Adobe Acrobat and European government systems.

GlobalSign (Belgium) — listed as a QTSP in Belgium, supervised by the FPS Economy. Strong enterprise presence. AATL membership plus EUTL listing gives broad compatibility.

Swisscom Trust Services (Austria, Switzerland) — the Swiss telecoms giant operates a QTSP listed in the *Austrian* TSL (not Swiss — because Switzerland is not in the EU) plus ZertES qualification in Switzerland. Their remote signing service is popular for high-volume enterprise signing.

InfoCert (Italy) — one of the largest European QTSPs by volume. Listed in the Italian TSL, supervised by AgID. Part of the Tinexta Group. Deeply embedded in Italian public administration.

Asseco Data Systems / Certum (Poland) — the dominant QTSP in Central/Eastern Europe. If you encounter qualified certificates from Polish organizations, Certum is the likely issuer.

This list is not exhaustive — there are over 200 qualified trust services listed across the EUTL. The EU maintains a browsable version at `https://eidas.ec.europa.eu/efda/tl-browser/` where you can filter by country, service type, and status.

22.5 Java Code: Parsing the EUTL

22.5.1 Downloading and Parsing the LOTL

```
import java.io.InputStream;
import java.net.URI;
import java.util.ArrayList;
import java.util.List;

import javax.xml.parsers.DocumentBuilderFactory;

import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.NodeList;
```

```
import lombok.Data;
import lombok.extern.slf4j.Slf4j;

/**
 * Parses the EU List of Trusted Lists (LOTL) to extract
 * pointers to each national Trusted Service Status List.
 */
@Slf4j
public class EutlLotlParser {

    private static final String LOTL_URL =
            "https://ec.europa.eu/tools/lotl/eu-lotl.xml";
    private static final String TSL_NS =
            "http://uri.etsi.org/02231/v2#";

    @Data
    public static class TslPointer {
        private String countryCode;
        private String tslUrl;
    }

    public List<TslPointer> parseLotl() throws Exception {
        log.info("Downloading LOTL from: {}", LOTL_URL);

        DocumentBuilderFactory factory =
                DocumentBuilderFactory.newInstance();
        factory.setNamespaceAware(true);
        // Prevent XXE attacks
        factory.setFeature(
                "http://apache.org/xml/features/"
                + "disallow-doctype-decl", true);

        Document doc;
        try (InputStream in = URI.create(LOTL_URL).toURL()
                .openStream()) {
            doc = factory.newDocumentBuilder().parse(in);
        }

        List<TslPointer> pointers = new ArrayList<>();
        NodeList nodes = doc.getElementsByTagNameNS(
                TSL_NS, "OtherTSLPointer");

        for (int i = 0; i < nodes.getLength(); i++) {
            Element pointer = (Element) nodes.item(i);

            NodeList locations = pointer.getElementsByTagNameNS(
                    TSL_NS, "TSLLocation");
            if (locations.getLength() == 0) {
                continue;
            }
            String url = locations.item(0).getTextContent().trim();

            NodeList territories = pointer.getElementsByTagNameNS(
                    TSL_NS, "SchemeTerritory");
            String country = territories.getLength() > 0
```

```
                    ? territories.item(0).getTextContent().trim()
                    : "UNKNOWN";

            TslPointer p = new TslPointer();
            p.setCountryCode(country);
            p.setTslUrl(url);
            pointers.add(p);

            log.debug("Found TSL: country={}, url={}", country, url);
        }

        log.info("Parsed LOTL: {} national TSL pointers",
                pointers.size());
        return pointers;
    }
}
```

22.5.2 Matching Certificates Against the EUTL

```
import java.io.ByteArrayInputStream;
import java.io.InputStream;
import java.net.URI;
import java.security.cert.CertificateFactory;
import java.security.cert.X509Certificate;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Base64;
import java.util.List;

import javax.xml.parsers.DocumentBuilderFactory;

import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.NodeList;

import lombok.Data;
import lombok.extern.slf4j.Slf4j;

/**
 * Parses a national TSL and matches signer certificates
 * against qualified CA services listed in the EUTL.
 */
@Slf4j
public class EutlCertificateMatcher {

    private static final String TSL_NS =
            "http://uri.etsi.org/02231/v2#";

    @Data
    public static class QualifiedCaEntry {
        private String providerName;
        private String status;
        private X509Certificate caCertificate;
    }
```

```
/**
 * Downloads a national TSL and extracts all CA/QC services
 * with "granted" status.
 */
public List<QualifiedCaEntry> parseNationalTsl(String tslUrl)
        throws Exception {
    log.info("Parsing national TSL: {}", tslUrl);

    DocumentBuilderFactory factory =
            DocumentBuilderFactory.newInstance();
    factory.setNamespaceAware(true);
    factory.setFeature(
            "http://apache.org/xml/features/"
            + "disallow-doctype-decl", true);

    Document doc;
    try (InputStream in = URI.create(tslUrl).toURL()
            .openStream()) {
        doc = factory.newDocumentBuilder().parse(in);
    }

    List<QualifiedCaEntry> entries = new ArrayList<>();
    NodeList services = doc.getElementsByTagNameNS(
            TSL_NS, "TSPService");
    CertificateFactory cf =
            CertificateFactory.getInstance("X.509");

    for (int i = 0; i < services.getLength(); i++) {
        Element svc = (Element) services.item(i);

        // Check service type
        NodeList typeNodes = svc.getElementsByTagNameNS(
                TSL_NS, "ServiceTypeIdentifier");
        if (typeNodes.getLength() == 0) {
            continue;
        }
        String type = typeNodes.item(0)
                .getTextContent().trim();
        if (!type.contains("CA/QC")) {
            continue;
        }

        // Check status
        NodeList statusNodes = svc.getElementsByTagNameNS(
                TSL_NS, "ServiceStatus");
        if (statusNodes.getLength() == 0) {
            continue;
        }
        String status = statusNodes.item(0)
                .getTextContent().trim();
        if (!status.contains("granted")) {
            continue;
        }
```

```
        // Extract CA certificate
        NodeList certNodes = svc.getElementsByTagNameNS(
                TSL_NS, "X509Certificate");
        if (certNodes.getLength() == 0) {
            continue;
        }
        String b64 = certNodes.item(0).getTextContent()
                .replaceAll("\\s", "");
        byte[] certBytes = Base64.getDecoder().decode(b64);
        X509Certificate cert = (X509Certificate)
                cf.generateCertificate(
                    new ByteArrayInputStream(certBytes));

        QualifiedCaEntry entry = new QualifiedCaEntry();
        entry.setStatus(status);
        entry.setCaCertificate(cert);
        entries.add(entry);

        log.info("Found qualified CA: {}",
                cert.getSubjectX500Principal());
    }

    return entries;
}

/**
 * Checks whether the given signer certificate was issued
 * by a CA listed in the EUTL with "granted" status.
 */
public boolean isIssuedByQualifiedCa(
        X509Certificate signerCert,
        X509Certificate issuerCert,
        List<QualifiedCaEntry> eutlEntries) {
    byte[] issuerEncoded;
    try {
        issuerEncoded = issuerCert.getEncoded();
    } catch (Exception e) {
        log.error("Failed to encode issuer cert", e);
        return false;
    }

    for (QualifiedCaEntry entry : eutlEntries) {
        try {
            byte[] eutlEncoded =
                    entry.getCaCertificate().getEncoded();
            if (Arrays.equals(issuerEncoded, eutlEncoded)) {
                log.info("Issuer matched EUTL entry: {}",
                        entry.getCaCertificate()
                            .getSubjectX500Principal());
                return true;
            }
        } catch (Exception e) {
            log.warn("Failed to encode EUTL cert", e);
        }
    }
```

```
            return false;
        }
    }
```

The code above is educational — it demonstrates the mechanics of EUTL parsing and certificate matching. A production implementation must also:

- **Verify the XML signatures** on the LOTL and every national TSL
- **Handle the LOTL pivot mechanism** when the signing certificate rotates
- **Parse service history** to support time-based qualification checks
- **Cache and refresh** the hierarchy on a schedule (not on every validation)
- **Parse QCStatements** from the signer's certificate to determine the qualification type

For production use, the **EU DSS library** (Chapter 23) provides `TrustedListsCertificateSource`, which handles all of this correctly. Do not reimplement this from scratch unless you have a very specific reason.

22.6 Key Takeaways

- A **QTSP** is not a self-declared status — it requires conformity assessment, supervisory body approval, and listing in the EUTL; the process takes 18-36 months and significant investment
- The **EUTL** is a hierarchy: the LOTL (Commission) points to national TSLs (supervisory bodies); every document is cryptographically signed with XAdES
- A certificate is qualified when: its issuer is in the EUTL with `granted` status for `CA/QC`, *and* the certificate contains the `QcCompliance` QCStatement
- **QCStatements** carry machine-readable qualification data: QcCompliance, QcSSCD, QcType (esign/eseal/web)
- **Certificate Policy OIDs** from ETSI EN 319 411-2 encode the qualification policy: QCP-n, QCP-l, with `-qscd` variants
- **QSCDs** require Common Criteria EAL4+ certification against EN 419 211/221 Protection Profiles — not every HSM qualifies
- Check **service history** when validating: a certificate issued during a `granted` period is qualified even if the service is now `withdrawn`
- Real-world QTSPs include D-Trust (DE), DigiCert/QuoVadis (NL/BE), GlobalSign (BE), Swisscom (AT/CH), InfoCert (IT), Certum (PL), and over 200 others
- For production EUTL validation, use the EU DSS framework's `TrustedListsCertificateSource` — do not reimplement the parsing, pivot handling, and signature verification from scratch

Chapter 23

Qualified Signatures vs Seals

Here is a question that trips up almost every developer the first time they encounter it: if an electronic signature and an electronic seal use exactly the same cryptographic mechanism — the same RSA or ECDSA algorithm, the same CMS SignedData structure, the same PAdES profile, the same qualified certificate from the same QTSP, stored on the same model of QSCD — what, exactly, is the difference?

The answer is not technical. It is legal. A signature is created by a *natural person* (a human being) and expresses that person's intent. A seal is created by a *legal person* (a company, government agency, association) and proves the origin and integrity of data. Same bytes, same math, different legal meaning — and getting it wrong can mean the difference between a legally binding contract and an authenticated invoice.

23.1 The Fundamental Distinction

23.1.1 Signatures: Natural Persons

An electronic signature under Regulation (EU) No 910/2014 (eIDAS) is always attributed to a **natural person**. Article 3(10) defines an electronic signature as "data in electronic form which is attached to or logically associated with other data in electronic form and which is used by the signatory to sign." Article 3(9) defines "signatory" as "a natural person who creates an electronic signature."

A company cannot sign. A department cannot sign. A server cannot sign. Only a human can sign, and a signature expresses that human's **intent** — consent, approval, acknowledgment, or whatever the signing context requires.

23.1.2 Seals: Legal Persons

An electronic seal is attributed to a **legal person** — a company, government agency, or any entity with legal personality. Regulation (EU) No 910/2014 (eIDAS) Article 3(25) defines an electronic seal as "data in electronic form, which is attached to or logically associated with other data in electronic form to ensure the latter's origin and integrity."

Note the careful wording. A seal does not express *intent*. It proves **origin** (the data came from this organization) and **integrity** (the data has not been tampered with). When Acme Corp seals an invoice, the seal says: "This invoice was produced by Acme Corp, and it has not been modified since." It says nothing about which employee generated it or whether anyone reviewed it.

23.2 eIDAS Articles 25 vs 35

23.2.1 Article 25: Signatures

- **25(1)**: Non-discrimination — an electronic signature shall not be denied legal effect solely because it is electronic
- **25(2)**: A qualified electronic signature has the **equivalent legal effect of a handwritten signature**
- **25(3)**: A QES from one member state is recognized in all member states

The critical phrase in 25(2) is ”equivalent legal effect.” This is the strongest possible legal recognition — a QES *is* a handwritten signature for all legal purposes.

23.2.2 Article 35: Seals

- **35(1)**: Non-discrimination (same principle as 25(1))
- **35(2)**: A qualified electronic seal enjoys a **presumption of integrity of the data and of correctness of the origin**
- **35(3)**: Mutual recognition across member states
- **Article 36**: Advanced seal requirements (same four properties as Art. 26, adapted for legal persons)
- **Article 38**: Qualified certificates for seals must meet Annex III requirements
- **Article 40**: Member states may define additional legal effects beyond the presumption

Table 23.1: Legal effects comparison

Aspect	Qualified Signature (QES)	Qualified Seal (QESeal)
Created by	Natural person (human)	Legal person (organization)
Legal effect	Equivalent to handwritten signature (Art. 25(2))	Presumption of integrity and origin (Art. 35(2))
Expresses intent?	Yes — the signer intends to be bound	No — proves origin and integrity only
Cross-border	Mandatory recognition (Art. 25(3))	Mandatory recognition (Art. 35(3))

The difference between ”legal equivalence” and ”presumption of integrity and origin” is not academic. A QES provides **non-repudiation of commitment** — the signer personally committed to the content. A QESeal provides **non-repudiation of origin** — the data came from the stated organization.

If a contract requires a signature, a seal will not suffice — even a qualified seal. A seal on a contract proves only that the document came from the company, not that any authorized representative agreed to its terms. Conversely, for automated invoices, a seal is the *correct* choice — attributing personal intent to a batch process would be legally inappropriate.

23.3 Same Code, Different Certificate

23.3.1 What Differs in the Certificate

From a cryptographic perspective, there is zero difference. Both are X.509 certificates, both use the same algorithms, both can be issued by the same QTSP. The differences are in the metadata:

Table 23.2: Certificate differences: signature vs seal

Field	Signature Certificate	Seal Certificate
Subject CN	Natural person's name ("Maria Schmidt")	Organization name ("Acme GmbH")
Certificate Policy OID	`0.4.0.194112.1.2` (QCP-n) or …`1.1` (QCP-n-qscd)	`0.4.0.194112.1.3` (QCP-l) or …`1.4` (QCP-l-qscd)
QCStatements: QcType	`0.4.0.1862.1.6.1` (esign)	`0.4.0.1862.1.6.2` (eseal)
eIDAS Annex	Annex I	Annex III

23.3.2 Creating a Seal vs a Signature in Java

The code is **identical**. The only difference is which certificate and private key you load.

```
import java.io.ByteArrayOutputStream;
import java.security.PrivateKey;
import java.security.cert.X509Certificate;

import org.apache.pdfbox.Loader;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;

/**
 * Creates either a qualified signature or a qualified seal on a
 * PDF document. The cryptographic operation is identical -- the
 * distinction is entirely in the certificate used.
 *
 * Signature: cert has QcType esign (0.4.0.1862.1.6.1),
 *            subject = "CN=Alice Smith"
 * Seal:      cert has QcType eseal (0.4.0.1862.1.6.2),
 *            subject = "O=Acme Corp, C=DE"
 */
@Slf4j
@RequiredArgsConstructor
public class PdfSignOrSealService {

    private final PrivateKey privateKey;
    private final X509Certificate[] certificateChain;
```

```
    public byte[] signOrSeal(byte[] pdfBytes, String reason,
            String location) throws Exception {

        PDDocument doc = Loader.loadPDF(pdfBytes);

        PDSignature sig = new PDSignature();
        sig.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
        sig.setSubFilter(
                PDSignature.SUBFILTER_ETSI_CADES_DETACHED);
        sig.setReason(reason);
        sig.setLocation(location);

        CmsSignatureCreator signer = new CmsSignatureCreator(
                privateKey, certificateChain);

        doc.addSignature(sig, signer);

        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        doc.saveIncremental(baos);
        doc.close();

        log.info("PDF signed/sealed: cert={}, reason={}",
                certificateChain[0].getSubjectX500Principal(),
                reason);
        return baos.toByteArray();
    }
}
```

- The reason field typically differs: ”Contract approval by Alice Smith” for a signature vs ”Authenticated by Acme Corp” for a seal

23.3.3 Detecting Signature vs Seal During Validation

```
import java.security.cert.X509Certificate;

import org.bouncycastle.asn1.ASN1Encodable;
import org.bouncycastle.asn1.ASN1ObjectIdentifier;
import org.bouncycastle.asn1.ASN1OctetString;
import org.bouncycastle.asn1.ASN1Sequence;

import lombok.extern.slf4j.Slf4j;

/**
 * Determines whether a certificate is for electronic signatures
 * (natural person) or electronic seals (legal person) by
 * examining the QcType statement in QCStatements.
 */
@Slf4j
public class SignatureOrSealDetector {

    private static final String QC_STATEMENTS_OID =
            "1.3.6.1.5.5.7.1.3";
    private static final String QC_TYPE_OID = "0.4.0.1862.1.6";
    private static final String ESIGN_OID = "0.4.0.1862.1.6.1";
```

```
    private static final String ESEAL_OID = "0.4.0.1862.1.6.2";

    public enum CertType { SIGNATURE, SEAL, UNKNOWN }

    public CertType detect(X509Certificate cert) {
        byte[] extValue = cert.getExtensionValue(QC_STATEMENTS_OID);
        if (extValue == null) {
            return CertType.UNKNOWN;
        }

        try {
            byte[] octets = ASN1OctetString.getInstance(extValue)
                    .getOctets();
            ASN1Sequence statements =
                    ASN1Sequence.getInstance(octets);

            for (ASN1Encodable statement : statements) {
                ASN1Sequence seq =
                        ASN1Sequence.getInstance(statement);
                String oid = ASN1ObjectIdentifier.getInstance(
                        seq.getObjectAt(0)).getId();

                if (!QC_TYPE_OID.equals(oid) || seq.size() < 2) {
                    continue;
                }

                ASN1Sequence typeSeq = ASN1Sequence.getInstance(
                        seq.getObjectAt(1));
                for (ASN1Encodable entry : typeSeq) {
                    String typeOid = ASN1ObjectIdentifier
                            .getInstance(entry).getId();
                    if (ESIGN_OID.equals(typeOid)) {
                        log.info("Certificate type: SIGNATURE");
                        return CertType.SIGNATURE;
                    }
                    if (ESEAL_OID.equals(typeOid)) {
                        log.info("Certificate type: SEAL");
                        return CertType.SEAL;
                    }
                }
            }
        } catch (Exception e) {
            log.warn("Failed to parse QCStatements", e);
        }
        return CertType.UNKNOWN;
    }
}
```

If the QcType statement is absent (some older certificates predate ETSI EN 319 412-5), fall back to the certificate policy OID: `0.4.0.194112.1.2` / …`1.1` indicate signature certificates, `0.4.0.194112.1.3` / …`1.4` indicate seal certificates. As a last resort, check the subject DN — a certificate with only an organization name (no personal CN) is likely a seal, though this heuristic is not definitive.

23.4 Use Cases: When to Sign, When to Seal

23.4.1 Contracts and Agreements: Signature

Any document requiring personal commitment — contracts, NDAs, medical prescriptions, regulatory filings, court submissions — needs a **signature**. The signer is the specific human being who agrees. Regulation (EU) No 910/2014 (eIDAS) Article 27 requires member states that mandate handwritten signatures for public services to accept QES.

23.4.2 Invoices and Automated Documents: Seal

Documents needing organizational authentication without personal approval — invoices, bank statements, payslips, automated reports — are candidates for **seals**. EU VAT Directive 2006/112/EC (Article 233) recognizes electronic seals as meeting invoice authenticity and integrity requirements.

23.4.3 Hybrid Workflows

Many real-world workflows combine both:

1. An organization **seals** a document template (e.g., a bank seals a loan agreement)
2. A natural person **signs** to express personal agreement (e.g., the customer signs the loan)
3. The organization's representative **signs** to accept (a natural person acting as agent)

The seal and signatures coexist as separate signature dictionaries in the same PDF, each with its own byte range and CMS structure. Validators should display both, clearly distinguishing "Sealed by Acme Corp" from "Signed by Alice Smith."

23.5 Server-Side Seal Creation and Batch Sealing

Because seals represent organizations rather than individuals, they are ideal for **automated, server-side creation**. No human needs to be present at the moment of sealing.

23.5.1 The Architecture

1. The organization holds a **qualified seal certificate** with its private key on an **HSM** (certified as a Qualified Seal Creation Device)
2. An application server generates documents (invoices, reports, certificates)
3. The server calls the HSM to sign each document — no human interaction per document
4. The HSM authenticates the application server (mutual TLS, API key), not a person

This is the critical difference from signatures: for QES, the signatory must authenticate *per signing operation* (sole control). For seals, the organization's security policies ensure only authorized systems can trigger sealing. This makes batch processing feasible.

At high volumes, watch for two bottlenecks:

1. **HSM throughput**: a single signing operation takes 1-10ms (RSA-2048 is slower than ECDSA P-256). At 50,000 documents per minute, you may need multiple HSM partitions or load balancing.

2. **TSA throughput**: if adding qualified timestamps (PAdES-B-T), the TSA becomes the constraint. Most commercial TSAs handle hundreds of requests per second, but verify against your volume requirements.

23.5.2 Cost Implications

Signature certificates are per-person, per-year: 50-300 EUR per natural person. For 500 employees, the annual cost is substantial.

Seal certificates are per-organization, per-year. A single seal certificate can produce millions of seals at a fixed cost. The per-document cost approaches zero at scale.

This asymmetry drives the architectural decision: seals for volume, signatures for commitment.

23.6 Common Pitfalls

23.6.1 Displaying Seals as Signatures

If your validation UI says ”Signed by Acme Corp” when it encounters a seal, you are misleading the user. The correct display is ”Sealed by” or ”Authenticated by” — language that communicates origin without implying personal intent.

23.6.2 Using Signature Certificates for Automation

If your batch pipeline signs 10,000 documents per hour with Alice’s personal qualified certificate, you have a problem. Alice is not exercising sole control over each operation, and may not even be aware of most of them. Use a seal certificate for automation and reserve signature certificates for workflows requiring individual human approval.

23.6.3 Ignoring the Certificate Type in Code

Because the signing code is identical, it is easy to accidentally use a seal certificate for a signature workflow or vice versa. Your application must validate the certificate *before* signing: check QcType, reject mismatches, and never rely on the user to ”use the right certificate.”

Some older qualified certificates lack a QcType statement entirely. For these, you must fall back to the EUTL service entry’s qualifiers or the certificate policy OID. This is yet another reason to use the EU DSS library (Chapter 23), which handles these edge cases.

23.7 Key Takeaways

- **Signatures** are for natural persons and express **intent** — Article 25(2) gives QES the legal equivalence of a handwritten signature
- **Seals** are for legal persons and prove **origin and integrity** — Article 35(2) gives qualified seals a presumption of integrity and correct origin, not equivalence to a signature
- The cryptographic mechanism is **identical** — same CMS, same PAdES, same QSCD; the only difference is the certificate (QcType esign vs eseal, certificate policy QCP-n vs QCP-l)

- Use **signatures** for contracts, agreements, approvals — anything requiring personal human commitment
- Use **seals** for invoices, automated reports, official publications — organizational authentication without personal approval
- **Server-side batch sealing** is the dominant seal use case: HSM-backed, no per-operation human auth, millions of documents at fixed cost
- **Enforce the certificate type in code** — do not allow seal certs for signature workflows or vice versa
- Display seals correctly: ”Sealed by” not ”Signed by” — the legal distinction matters
- Regulation (EU) 2024/1183 (eIDAS 2.0) focuses on personal QES via the EUDI wallet; seals continue on HSM infrastructure, with rising volume from QEAAs

Chapter 24

Remote Signing

For the first two decades of digital signatures, the assumption was simple: the private key lives with the signer. A smart card in a wallet, a PKCS#12 file on a laptop, a USB token plugged into a desktop. The signer has physical custody of their key material, they authenticate locally (enter a PIN, type a password), and the signing happens on their device.

That model is dead — or at least, it is dying.

In the remote signing model, the private key never leaves a server-side Hardware Security Module (HSM). The signer authenticates remotely — via OAuth 2.0, via SMS OTP, via biometric verification on their phone — and the server performs the cryptographic signing operation on their behalf. The signer never touches the key, never downloads it, never even knows the key's technical parameters. They authorize a signing operation, and the server produces the signature.

This chapter covers the architecture of remote signing: why it exists, how the CSC API standardizes it, how HSMs and Signature Activation Modules work together, what the SCAL levels mean for sole control, and how to integrate with remote signing services in practice. If your application needs to produce qualified electronic signatures (QES) at scale — for thousands of users who do not have smart cards and do not want USB tokens — remote signing is how you get there.

24.1 Why Remote Signing Won

24.1.1 The Smart Card Problem

The traditional QES model required every signer to have:

1. A qualified certificate issued by a QTSP
2. A Qualified Signature Creation Device (QSCD) — a certified smart card or USB token
3. A card reader connected to their computer
4. Driver software for the card reader and the QSCD
5. Middleware that bridges the QSCD to the signing application (typically PKCS#11 or Microsoft CNG)

This worked in tightly controlled enterprise environments — banks, government agencies, regulated industries where IT departments could provision hardware to every employee. It failed completely for consumer scenarios, mobile scenarios, and any context where "plug in a USB device and install a driver" is not feasible.

The practical failure modes were numerous:

- **Driver hell**: Smart card drivers are notoriously platform-dependent. A card that works on Windows 10 may not work on Windows 11 without a driver update. macOS support

is inconsistent. Linux support depends on the specific combination of card, reader, and PKCS#11 module.
- **Mobile impossibility**: Smartphones have no USB-A ports, no smart card readers, and limited support for external cryptographic hardware. The NFC-based smart card reading that some phones support is slow and unreliable.
- **Certificate lifecycle**: Qualified certificates typically expire after 1-3 years. Renewal requires physically visiting a Registration Authority (for identity verification) and provisioning a new certificate onto the QSCD. Users forget, certificates expire, signing fails.
- **Cost**: A qualified certificate costs 50-300 EUR per year. A smart card costs 20-50 EUR. A card reader costs 15-40 EUR. For an organization with 10,000 employees, the hardware cost alone is significant — before you account for help desk tickets from users who cannot get their cards to work.

24.1.2 The Remote Alternative

Remote signing eliminates every one of these problems:

- **No hardware**: The signer needs only a web browser or a mobile app. No drivers, no readers, no tokens.
- **Mobile-native**: Authentication happens through the phone itself — push notifications, biometric confirmation, SMS OTP. The signing experience on a smartphone is identical to the desktop experience.
- **Managed lifecycle**: The QTSP manages the key lifecycle entirely. Certificate issuance, renewal, and revocation happen server-side. The signer never handles key material.
- **Lower per-user cost**: While the QTSP charges per signature or per subscription, the elimination of hardware provisioning and support often makes remote signing cheaper at scale.

Remote signing does not lower the *security* bar — it moves the security boundary. Instead of trusting the signer's device to protect the key (which, for a password-protected PKCS#12 on a laptop, is a dubious proposition), you trust the QTSP's HSM infrastructure and authentication system. For most threat models, a professionally managed HSM with multi-factor authentication provides stronger key protection than a smart card with a four-digit PIN that the user wrote on a sticky note.

24.1.3 eIDAS and Remote QSCDs

The Regulation (EU) No 910/2014 (eIDAS) does not require physical possession of a QSCD — it requires sole control (Article 26(c)) and that the device meet the requirements of Annex II. An HSM operated by a QTSP, where the signer's key partition is accessible only through strong multi-factor authentication tied to the specific signing operation, satisfies both requirements.

The European Committee for Standardization (CEN) formalized this in **EN 419 241**, which defines requirements for trustworthy systems supporting server signing:

- **EN 419 241-1**: General requirements for server signing
- **EN 419 241-2**: Protection profile for QSCDs for server signing

These standards, combined with the SCAL framework (discussed below), provide the certifi-

cation path for remote QSCDs. By 2024, remote signing had become the dominant model for QES in practice — more qualified signatures are produced via remote HSMs than via local smart cards.

24.2 The Hash-Signing Model

24.2.1 Why Send the Hash, Not the Document

Remote signing uses a **hash-signing model**: the client computes the hash of the data to be signed and sends only the hash to the server. The server applies the private key to the hash (RSA-PSS, ECDSA, etc.) and returns the raw signature value. The client then embeds the signature value into the CMS structure and the PDF.

This design has three critical advantages:

1. **Privacy**: The document content never leaves the client. The server sees only a 32-byte (SHA-256) or 64-byte (SHA-512) hash. It cannot reconstruct the document from the hash. For sensitive documents — medical records, legal contracts, financial statements — this is essential.
2. **Bandwidth**: Sending a 32-byte hash is dramatically faster than sending a 50 MB PDF over a network connection. For mobile clients on cellular networks, the difference matters.
3. **Separation of concerns**: The client handles PDF parsing, byte range computation, CMS structure building, and timestamp embedding. The server handles key management, access control, and the raw cryptographic operation. Neither needs to understand the other's domain.

24.2.2 The Flow Step by Step

Steps 1-3 and 11-14 happen entirely on the client. Only steps 4, 6, and the server-side steps 5-10 involve the remote server. The document never crosses the network.

The "hash" sent to the server in step 6 is not the hash of the document bytes. It is the hash of the **DER-encoded signed attributes** of the CMS SignerInfo. The signed attributes include the `messageDigest` attribute, which itself contains the hash of the document bytes. This two-level hashing is standard CMS behavior (defined in RFC 5652 — Cryptographic Message Syntax (CMS) Section 5.4) and is easy to get wrong.

If you hash the document bytes and send that to the remote server, the resulting signature will not verify — because CMS validators expect the signature to be over the signed attributes, not the raw data.

24.3 The CSC API (Cloud Signature Consortium)

24.3.1 What the CSC Is

The **Cloud Signature Consortium (CSC)** is an industry body that developed an open standard API for remote signing. Before the CSC API, every remote signing provider had its own proprietary API — different authentication flows, different request/response formats,

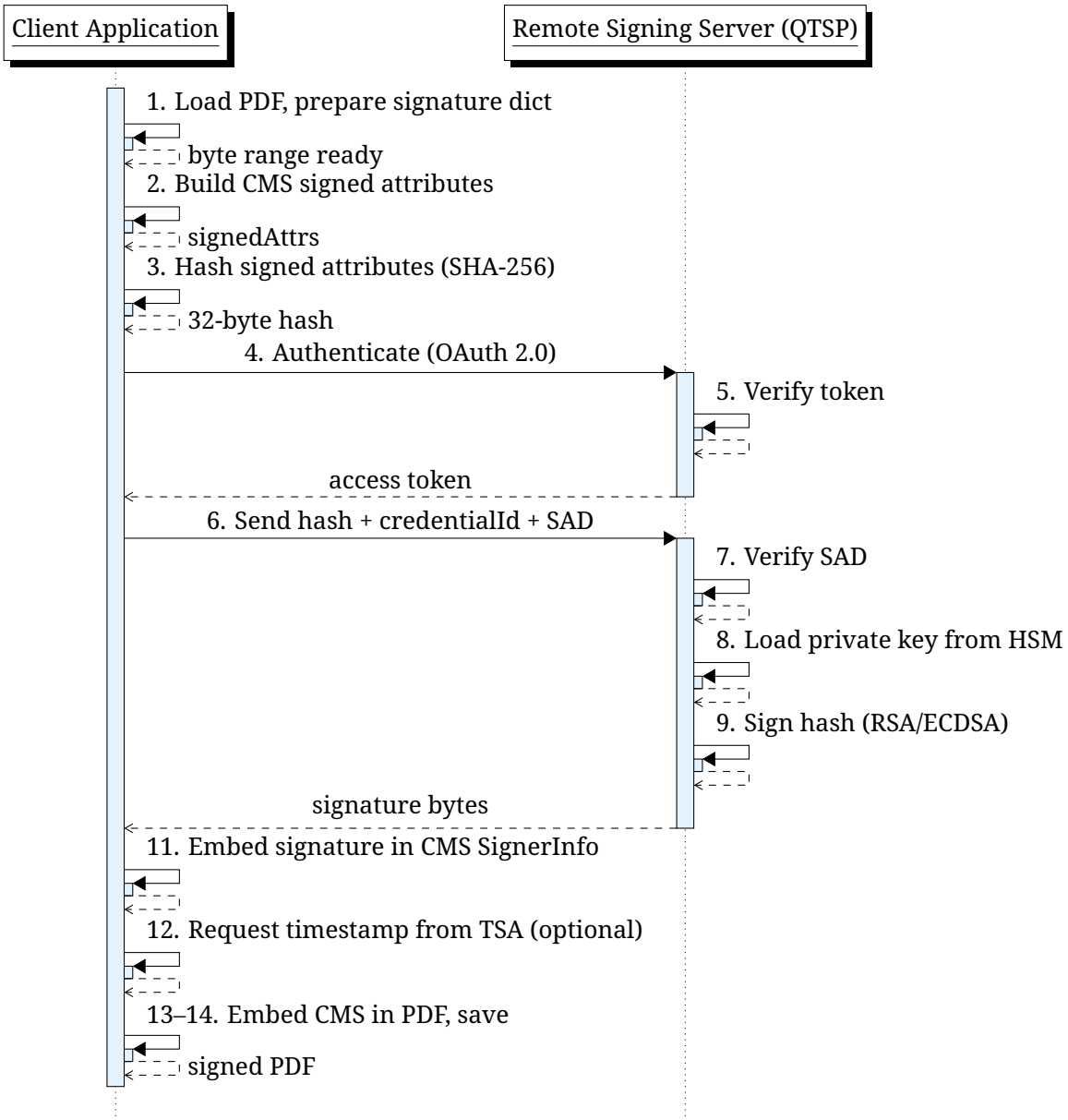


Figure 24.1: Remote signing flow — client and server responsibilities

different error codes. Integrating with Swisscom AIS required completely different code than integrating with DigiCert, which required completely different code than integrating with InfoCert.

The CSC API v2.0 (the current version, published in 2019 and updated since) standardizes:

- **Authentication**: OAuth 2.0 with specific grant types
- **Credential discovery**: listing available signing credentials for a user
- **Signature activation**: the mechanism by which the signer authorizes a specific signing operation
- **Hash signing**: the actual cryptographic operation

The API is RESTful, uses JSON for request and response bodies, and is designed to be consumed by web and mobile clients.

24.3.2 API Endpoints

The CSC API defines the following core endpoints:

Table 24.1: CSC API v2.0 Core Endpoints

Endpoint	Method	Purpose
`/auth/login`	POST	Authenticate the user with the remote signing service. Returns an OAuth 2.0 access token.
`/auth/revoke`	POST	Revoke an active access token.
`/credentials/list`	POST	List the signing credentials available to the authenticated user. Returns credential IDs, certificate chains, key algorithms, and SCAL levels.
`/credentials/info`	POST	Get detailed information about a specific credential, including the certificate chain and supported algorithms.
`/credentials/authorize`	POST	Authorize a specific credential for signing. For SCAL2 credentials, this triggers explicit user authentication (OTP, biometric) and returns a Signature Activation Data (SAD) token.
`/credentials/sendOTP`	POST	Send a one-time password to the user (SMS, email) as part of SCAL2 authorization.
`/signatures/signHash`	POST	Sign one or more hashes using a previously authorized credential. Returns the raw signature values.
`/signatures/timestamp`	POST	Request an RFC 3161 timestamp. Some providers expose timestamping through the CSC API as well.

24.3.3 Authentication: OAuth 2.0

The CSC API uses OAuth 2.0 for authentication. The specific grant type depends on the deployment scenario:

- **Authorization Code Grant**: For web applications where the user authenticates in a browser. The signing application redirects the user to the QTSP's authorization server, the user logs in, and the application receives an authorization code that it exchanges for an access token.
- **Client Credentials Grant**: For server-to-server scenarios where a system (not a human) needs to sign. The application authenticates with its own client ID and secret.
- **Resource Owner Password Grant**: For scenarios where the application collects the user's credentials directly. Discouraged by OAuth 2.0 best practices but still supported by some providers.

```
/**
 * Authenticates with a CSC-compliant remote signing service.
 * Returns an OAuth 2.0 access token for subsequent API calls.
 */
public String authenticate(String cscBaseUrl, String username,
        String password) throws IOException {

    String loginUrl = cscBaseUrl + "/auth/login";
    String requestBody = """
            {
                "remoteUser": "%s",
                "remoteSecret": "%s"
            }
            """.formatted(username, password);

    HttpURLConnection conn = (HttpURLConnection)
            new URL(loginUrl).openConnection();
    conn.setDoOutput(true);
    conn.setRequestMethod("POST");
    conn.setRequestProperty("Content-Type", "application/json");
    conn.setConnectTimeout(10000);
    conn.setReadTimeout(30000);

    try (OutputStream out = conn.getOutputStream()) {
        out.write(requestBody.getBytes(StandardCharsets.UTF_8));
    }

    if (conn.getResponseCode() != 200) {
        throw new IOException("CSC auth/login failed: HTTP " +
                conn.getResponseCode());
    }

    // Parse JSON response — extract access_token
    String responseJson;
    try (InputStream in = conn.getInputStream()) {
        responseJson = new String(in.readAllBytes(), StandardCharsets.UTF_8);
    }

    // In production, use a proper JSON parser (Jackson, Gson)
```

```
    // This is simplified for illustration
    log.info("CSC authentication successful for user: {}", username);
    return extractJsonField(responseJson, "access_token");
}
```

24.3.4 Credential Discovery

Once authenticated, the client can discover what signing credentials are available:

```
/**
 * Lists the signing credentials available to the authenticated user.
 * Returns credential IDs that can be used for signing.
 */
public List<CscCredential> listCredentials(String cscBaseUrl,
        String accessToken) throws IOException {

    String url = cscBaseUrl + "/credentials/list";
    String requestBody = """
            {
                "maxResults": 100
            }
            """;

    HttpURLConnection conn = (HttpURLConnection)
            new URL(url).openConnection();
    conn.setDoOutput(true);
    conn.setRequestMethod("POST");
    conn.setRequestProperty("Content-Type", "application/json");
    conn.setRequestProperty("Authorization", "Bearer " + accessToken);

    try (OutputStream out = conn.getOutputStream()) {
        out.write(requestBody.getBytes(StandardCharsets.UTF_8));
    }

    String responseJson;
    try (InputStream in = conn.getInputStream()) {
        responseJson = new String(in.readAllBytes(), StandardCharsets.UTF_8);
    }

    // Response contains:
    // {
    //   "credentialIDs": ["cred-001", "cred-002"],
    //   "credentialInfos": [
    //     {
    //       "credentialID": "cred-001",
    //       "key": { "algo": ["1.2.840.113549.1.1.11"], "len": 2048 },
    //       "cert": { "certificates": ["MIIC...", "MIID..."] },
    //       "SCAL": "2",
    //       "description": "Qualified certificate for John Doe"
    //     }
    //   ]
    // }

    log.info("CSC credentials/list returned credentials for user");
    return parseCscCredentials(responseJson);
```

```
}
```

The credential response includes the full certificate chain (Base64-encoded), which the client needs to build the CMS structure *before* requesting the hash signature. This is a critical detail: the client must know the signer's certificate at CMS construction time (for the ESS-signing-certificate-v2 attribute), not just at signing time.

24.3.5 Credential Authorization and SAD

For SCAL2 credentials (see below), the client must explicitly authorize the credential before signing. This triggers an additional authentication step — the QTSP sends an OTP to the signer, or the signer confirms via a mobile app, or biometric verification occurs.

The result of successful authorization is a **Signature Activation Data (SAD)** token — an opaque string that the client presents with the signing request to prove that the signer authorized this specific operation.

```
/**
 * Authorizes a credential for signing (SCAL2).
 * Triggers user authentication (OTP, biometric, etc.).
 * Returns a SAD token valid for a limited time and number of signatures.
 */
public String authorizeCredential(String cscBaseUrl, String accessToken,
        String credentialId, int numSignatures) throws IOException {

    // Step 1: Trigger OTP delivery
    String sendOtpUrl = cscBaseUrl + "/credentials/sendOTP";
    String otpRequestBody = """
            {
                "credentialID": "%s"
            }
            """.formatted(credentialId);

    HttpURLConnection otpConn = (HttpURLConnection)
            new URL(sendOtpUrl).openConnection();
    otpConn.setDoOutput(true);
    otpConn.setRequestMethod("POST");
    otpConn.setRequestProperty("Content-Type", "application/json");
    otpConn.setRequestProperty("Authorization", "Bearer " + accessToken);
    try (OutputStream out = otpConn.getOutputStream()) {
        out.write(otpRequestBody.getBytes(StandardCharsets.UTF_8));
    }

    log.info("OTP sent for credential: {}", credentialId);

    // Step 2: Collect OTP from user (your UI must handle this)
    String otp = collectOtpFromUser();

    // Step 3: Authorize the credential with the OTP
    String authUrl = cscBaseUrl + "/credentials/authorize";
    String authRequestBody = """
            {
                "credentialID": "%s",
                "numSignatures": %d,
```

```
                "OTP": "%s"
            }
            """.formatted(credentialId, numSignatures, otp);

    HttpURLConnection authConn = (HttpURLConnection)
            new URL(authUrl).openConnection();
    authConn.setDoOutput(true);
    authConn.setRequestMethod("POST");
    authConn.setRequestProperty("Content-Type", "application/json");
    authConn.setRequestProperty("Authorization", "Bearer " + accessToken);
    try (OutputStream out = authConn.getOutputStream()) {
        out.write(authRequestBody.getBytes(StandardCharsets.UTF_8));
    }

    String responseJson;
    try (InputStream in = authConn.getInputStream()) {
        responseJson = new String(in.readAllBytes(), StandardCharsets.UTF_8);
    }

    // Response: { "SAD": "eyJhbGciOiJSUzI1NiIs..." }
    String sad = extractJsonField(responseJson, "SAD");
    log.info("Credential authorized, SAD obtained for {} signatures",
            numSignatures);
    return sad;
}
```

The SAD token has a limited lifetime (typically 5-10 minutes) and a limited number of uses (specified in `numSignatures`). If you request authorization for 1 signature, the SAD can only be used once. If you need to sign multiple documents in a batch, request a higher `numSignatures` value — but be aware that some QTSPs limit the maximum.

If the SAD expires before you use it, you must re-authorize. Design your signing flow so that SAD acquisition happens immediately before hash signing, not minutes before.

24.3.6 Hash Signing

The core operation — sending a hash and receiving a signature:

```
/**
 * Signs one or more hashes using a CSC-compliant remote signing service.
 * Returns raw signature values (one per hash).
 */
public List<byte[]> signHashes(String cscBaseUrl, String accessToken,
        String credentialId, String sad,
        List<byte[]> hashes) throws IOException {

    String signUrl = cscBaseUrl + "/signatures/signHash";

    // Convert hashes to Base64 for JSON transport
    List<String> hashesBase64 = new ArrayList<>();
    for (byte[] hash : hashes) {
        hashesBase64.add(Base64.getEncoder().encodeToString(hash));
    }

    // Build JSON array of hashes
```

```
        StringBuilder hashArray = new StringBuilder("[");
        for (int i = 0; i < hashesBase64.size(); i++) {
            if (i > 0) hashArray.append(",");
            hashArray.append("\"").append(hashesBase64.get(i)).append("\"");
        }
        hashArray.append("]");

        String requestBody = """
                {
                    "credentialID": "%s",
                    "SAD": "%s",
                    "hashes": %s,
                    "hashAlgo": "2.16.840.1.101.3.4.2.1",
                    "signAlgo": "1.2.840.113549.1.1.11"
                }
                """.formatted(credentialId, sad, hashArray.toString());

        HttpURLConnection conn = (HttpURLConnection)
                new URL(signUrl).openConnection();
        conn.setDoOutput(true);
        conn.setRequestMethod("POST");
        conn.setRequestProperty("Content-Type", "application/json");
        conn.setRequestProperty("Authorization", "Bearer " + accessToken);

        try (OutputStream out = conn.getOutputStream()) {
            out.write(requestBody.getBytes(StandardCharsets.UTF_8));
        }

        String responseJson;
        try (InputStream in = conn.getInputStream()) {
            responseJson = new String(in.readAllBytes(), StandardCharsets.UTF_8);
        }

        // Response: { "signatures": ["MEUCIQD...", "MEQCIBD..."] }
        List<byte[]> signatures = parseSignatureValues(responseJson);
        log.info("CSC signHash returned {} signature values", signatures.size());
        return signatures;
    }
```

The OIDs in the request body specify the algorithms:

Table 24.2: Common algorithm OIDs for CSC signHash

OID	Algorithm	Notes
`2.16.840.1.101.3.4.2.1`	SHA-256	Hash algorithm. The hash you send must use this algorithm.
`2.16.840.1.101.3.4.2.2`	SHA-384	Stronger hash alternative.
`1.2.840.113549.1.1.11`	SHA256withRSA (PKCS#1 v1.5)	RSA signature with SHA-256. The server applies PKCS#1 v1.5 padding.

OID	Algorithm	Notes
`1.2.840.113549.1.1.10`	RSASSA-PSS	RSA-PSS signature. Preferred over PKCS#1 v1.5 for new deployments.
`1.2.840.10045.4.3.2`	SHA256withECDSA	ECDSA signature with SHA-256. Shorter signatures, equivalent security.

24.4 SCAL: Sole Control Assurance Levels

24.4.1 The Problem SCAL Solves

The Regulation (EU) No 910/2014 (eIDAS) Article 26(c) requires that an advanced electronic signature be ”created using electronic signature creation data that the signatory can, with a high level of confidence, use under his sole control.”

For local signing with a smart card, sole control is straightforward: the signer physically possesses the card and knows the PIN. For remote signing, sole control is more nuanced. The key lives in a server-side HSM. Multiple users' keys may reside in the same physical HSM. The QTSP’s system administrators have physical access to the HSM. How do you prove that the signer — and only the signer — authorized a specific signing operation?

The **SCAL** (Sole Control Assurance Level) framework, defined in CEN EN 419 241-1, provides two levels of assurance:

24.4.2 SCAL1: Server-Side Sole Control

In SCAL1, the remote signing server ensures sole control through its own authentication and authorization mechanisms, but there is no explicit per-signature authorization from the user beyond the initial session authentication.

Characteristics: - The signer authenticates to the service (username + password, possibly with MFA) - Once authenticated, the service can sign on the signer’s behalf for the duration of the session - The service’s access control mechanisms prevent unauthorized use - There is no additional user interaction for each signing operation

SCAL1 is suitable for: - **Advanced electronic signatures (AdES)**: The sole control requirement of Article 26(c) can be met if the authentication is sufficiently strong - **Automated signing workflows**: Where a human authorizes a batch of signatures upfront - **Internal enterprise signing**: Where the trust model accepts session-level authorization

SCAL1 is *not* suitable for QES, because the sole control assurance is not strong enough — the server could theoretically sign without the user’s knowledge during an active session.

24.4.3 SCAL2: Explicit Per-Signature Authorization

In SCAL2, every signing operation requires explicit authorization from the signer, independent of session authentication. The authorization is linked to the specific data being signed.

Characteristics: - The signer authenticates to the service (session-level) - For each signing operation, the signer must provide additional authorization: OTP, biometric confirmation, mobile app push approval - The authorization produces a SAD (Signature Activation Data) token that is bound to the specific credential and limited in scope (time, number of signatures) - The HSM will not perform the signing operation without a valid SAD

SCAL2 is required for: - **Qualified electronic signatures (QES)**: The per-signature authorization satisfies Article 26(c) with a high level of confidence - **Any scenario where you need to prove that the signer actively authorized each specific signature**

Table 24.3: SCAL1 vs. SCAL2 Comparison

Aspect	SCAL1	SCAL2
Authentication	Session-level (login once)	Session-level + per-signature authorization
User interaction per signature	None after login	Required (OTP, biometric, push)
SAD token	Not required	Required for each signing operation
Suitable for QES	No	Yes
Suitable for AdES	Yes	Yes
Batch signing	Easy (sign N documents in a loop)	Possible but requires batch SAD (N signatures authorized at once)
eIDAS Article 26(c) assurance	Moderate	High

The distinction between SCAL1 and SCAL2 is not merely theoretical — it determines whether the resulting signature can be a QES. If your application needs to produce qualified electronic signatures via remote signing, you *must* implement a SCAL2 flow. A SCAL1 flow, no matter how strong the session authentication, will not produce a QES because it lacks per-signature authorization.

This is the single most common mistake in remote signing integrations: assuming that a strong login (username + password + TOTP) is sufficient for QES. It is not. The authentication must be linked to the signing operation, not just the session.

24.5 HSM and SAM Architecture

24.5.1 The HSM (Hardware Security Module)

The HSM is the physical device that stores private keys and performs cryptographic operations. In a remote signing deployment, the HSM is typically:

- **FIPS 140-2 Level 3** or **Level 4** certified (US standard)
- **Common Criteria EAL4+** certified (international standard)

- Deployed in a physically secured data center with access logging, video surveillance, and multi-person access controls
- Connected to the signing server via a high-speed internal network (never directly exposed to the internet)

Popular HSMs for remote signing: - **Thales Luna Network HSM**: Widely used in European QTSPs. Supports PKCS#11, JCE (Java Cryptography Extension), and Microsoft CNG. Available in various throughput tiers. - **Utimaco SecurityServer**: German manufacturer, popular with European banks and government agencies. Strong PKCS#11 support. - **Entrust nShield Connect**: Used by large enterprises and some QTSPs. Offers a proprietary "Security World" key management architecture alongside standard PKCS#11. - **AWS CloudHSM / Azure Dedicated HSM**: Cloud-hosted HSMs. Convenient for cloud-native architectures, but may not meet all QSCD certification requirements (check the specific certification status).

24.5.2 The SAM (Signature Activation Module)

The SAM sits between the remote signing API and the HSM. Its job is to enforce sole control:

1. **Receive the signing request** with the signer's credentials and SAD
2. **Verify the SAD** — check that it is valid, not expired, bound to the correct credential, and that the number of allowed signatures has not been exhausted
3. **Authorize the HSM operation** — unlock the signer's key partition in the HSM for exactly one (or N) signing operations
4. **Return the signature value** to the API layer

The SAM is the component that makes SCAL2 possible. Without a SAM, the signing server could theoretically use any key in the HSM at any time. The SAM ensures that keys are only activated when the legitimate signer has provided explicit, verifiable authorization.

```
Architecture of a Remote Signing Service


                        ┌──────────────────────┐
Internet                │   Load Balancer /    │
────────────────────────│   API Gateway        │
                        └──────────────────────┘
                                   │
                                   │
                        ┌──────────┴───────────┐
                        │   CSC API Server     │
                        │   (REST endpoints)   │
                        │   OAuth 2.0 token    │
                        │   validation         │
                        └──────────────────────┘
                                   │
                                   │
                        ┌──────────┴───────────┐
                        │   Signature          │
                        │   Activation Module  │
                        │   (SAM)              │
                        │   • SAD validation   │
                        │   • SCAL enforcement │
                        │   • Audit logging    │
```

```
└──────────────────────┘
           │
           │ PKCS#11
┌──────────┴──────────┐
│   Hardware Security   │
│   Module (HSM)        │
│   • Key storage       │
│   • Crypto operations │
│   • Tamper-evident    │
└──────────────────────┘
```

24.5.3 PKCS#11: The HSM Interface

PKCS#11 (also called Cryptoki) is the standard API for communicating with HSMs and smart cards. It is a C-based API with Java bindings available through the JCE `SunPKCS11` provider or through dedicated libraries.

In a remote signing server, the PKCS#11 flow for signing looks like this:

```
/**
 * Signs a hash using an HSM via PKCS#11.
 * This runs on the remote signing server, NOT on the client.
 */
public byte[] signWithHsm(byte[] hash, String keyAlias,
        char[] keyPin) throws Exception {

    // 1. Load the PKCS#11 provider
    //    (configured once at server startup, not per request)
    Provider pkcs11Provider = Security.getProvider("SunPKCS11");

    // 2. Load the keystore from the HSM
    KeyStore ks = KeyStore.getInstance("PKCS11", pkcs11Provider);
    ks.load(null, keyPin);

    // 3. Retrieve the private key
    PrivateKey privateKey = (PrivateKey) ks.getKey(keyAlias, keyPin);
    if (privateKey == null) {
        throw new IllegalStateException(
                "Key not found in HSM: " + keyAlias);
    }

    // 4. Sign the hash
    //    Note: for RSA, Signature.sign() expects the raw data, not a
    //    pre-computed hash. Use "NONEwithRSA" to sign a raw hash, or
    //    use "SHA256withRSA" and pass the original data.
    Signature sig = Signature.getInstance("SHA256withRSA", pkcs11Provider);
    sig.initSign(privateKey);
    sig.update(hash);
    byte[] signatureValue = sig.sign();

    log.info("HSM signing completed: key={}, sigLen={}",
            keyAlias, signatureValue.length);
    return signatureValue;
}
```

The JCE `SunPKCS11` provider requires a configuration that specifies the PKCS#11 library path, slot number, and other HSM-specific parameters. This configuration is typically loaded from a file or a dynamic `InputStream`:

```
# pkcs11.cfg
name = MyHSM
library = /opt/hsm/lib/libCryptoki2_64.so
slot = 0
```

Different HSM vendors provide different PKCS#11 shared libraries (`.so` on Linux, `.dll` on Windows). The library path is the most common source of "HSM not found" errors in deployments.

24.5.4 Key Lifecycle in an HSM

Private keys in an HSM follow a strict lifecycle:

1. **Generation**: The key pair is generated *inside* the HSM. The private key never exists outside the HSM's tamper-resistant boundary. This is a fundamental requirement for QSCD certification.
2. **Activation**: The key is associated with a certificate (issued by the QTSP's CA) and made available for signing. For SCAL2, activation for each signing operation requires a valid SAD.
3. **Use**: The HSM performs signing operations using the key. Each use is logged for audit purposes.
4. **Suspension**: The key can be temporarily suspended (e.g., if the signer reports a compromised authentication factor). The certificate is not revoked, but signing is disabled.
5. **Revocation**: The key is permanently disabled, and the associated certificate is revoked. The key material may be destroyed or archived depending on the QTSP's policy.

For QSCD certification under EN 419 241-2, the HSM must guarantee that private keys are generated internally and can never be extracted — not even by the QTSP's own administrators. This is what "the key never leaves the HSM" means in practice. The HSM must be designed so that there is no API, no debug mode, no firmware backdoor that could extract key material. Physical attacks (opening the HSM casing) must trigger key destruction.

24.6 Vendor Landscape

24.6.1 Major Remote Signing Providers

The remote signing market is dominated by a handful of European QTSPs, each offering CSC-compliant APIs with varying degrees of completeness:

Table 24.4: Remote Signing Provider Comparison

Provider	Headquarters	CSC API	SCAL Levels	Notable Features
Swisscom AIS	Switzerland	v2.0	SCAL1, SCAL2	Integrated with Swiss law (ZertES). Strong mobile authentication via Mobile ID. High throughput for batch signing.
DigiCert + QuoVadis	US / Switzerland	v2.0	SCAL1, SCAL2	Global presence. QuoVadis (acquired by DigiCert) provides the EU-qualified certificates. AATL-listed roots.
InfoCert	Italy	v2.0	SCAL1, SCAL2	Largest QTSP in Europe by volume. SPID integration for Italian digital identity. Strong in Italian government sector.
Namirial	Italy	v2.0	SCAL1, SCAL2	Focus on document workflow integration. Offers both remote signing and a full document management platform.

Provider	Headquarters	CSC API	SCAL Levels	Notable Features
ZealID	Lithuania	v2.0	SCAL2	Mobile-first approach. Signing via smartphone app with biometric authentication. Popular in Baltic states.
D-Trust (Bundes-druckerei)	Germany	v2.0	SCAL1, SCAL2	German Federal Printing Office subsidiary. Strong in German government and healthcare sectors.

24.6.2 Choosing a Provider

When selecting a remote signing provider, consider:

1. **Trusted list presence**: The provider must be a QTSP listed in the EUTL of the member state where they are supervised. Verify this at the EU's official Trusted List Browser.
2. **AATL listing**: If your signed PDFs will be opened in Adobe Acrobat, the provider's root certificate should be in the Adobe Approved Trust List (AATL). Without AATL listing, Acrobat will show a warning even for EU-qualified signatures.
3. **Geographic coverage**: Some providers issue certificates qualified in one member state; others have qualified status in multiple states. For cross-border use, mutual recognition under eIDAS Article 25(3) applies, but AATL listing is a separate concern.
4. **API completeness**: Not all providers implement the full CSC API. Some support only SCAL1, some have proprietary extensions, some are still on CSC v1.x. Request the provider's API documentation and test against their sandbox environment.
5. **Authentication options**: SCAL2 requires per-signature authorization. Providers offer different mechanisms: SMS OTP, mobile app push, biometric (fingerprint/face), hardware token. Choose based on your users' capabilities.
6. **Pricing model**: Per-signature pricing (typically 0.10-2.00 EUR per signature), per-certificate pricing (annual fee per signer), or volume-based tiers. Calculate your total cost based on expected signing volume.
7. **SLA and availability**: For production signing, you need a provider with a published SLA. Remote signing is a synchronous operation in most workflows — if the provider is down, your users cannot sign.

Do not assume that all CSC API implementations are interchangeable. The CSC specification leaves room for provider-specific behavior in areas like:

- OTP delivery mechanisms (some providers trigger OTP automatically, others require

an explicit `/credentials/sendOTP` call)
- SAD token format and lifetime
- Batch signing limits (how many hashes can be signed with a single SAD)
- Error response formats and error codes

Always develop and test against the specific provider's sandbox before going to production. The CSC specification is a standard, but implementations have edges.

24.7 Complete Integration Example

The following example shows a complete remote signing flow — from PDF preparation through CSC API calls to final PDF assembly:

```
/**
 * Signs a PDF using a CSC-compliant remote signing service.
 * Produces a PAdES-B-T compliant signature.
 */
public byte[] signPdfRemotely(byte[] pdfBytes, String cscBaseUrl,
        String accessToken, String credentialId,
        String tsaUrl) throws Exception {

    // --- Phase 1: Prepare the PDF and CMS structure ---

    // 1a. Get the signer's certificate from the CSC service
    CscCredentialInfo credInfo = getCredentialInfo(
            cscBaseUrl, accessToken, credentialId);
    X509Certificate signerCert = credInfo.getCertificateChain().get(0);
    List<X509Certificate> certChain = credInfo.getCertificateChain();

    // 1b. Load the PDF and prepare the signature dictionary
    PDDocument doc = Loader.loadPDF(pdfBytes);
    PDSignature pdSignature = new PDSignature();
    pdSignature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
    pdSignature.setSubFilter(
            COSName.getPDFName("ETSI.CAdES.detached"));
    pdSignature.setSignDate(Calendar.getInstance());

    // 1c. Register our SignatureInterface
    // (PDFBox calls this to get the CMS bytes)
    RemoteSignatureInterface sigInterface =
            new RemoteSignatureInterface(
                    cscBaseUrl, accessToken, credentialId,
                    signerCert, certChain, tsaUrl);

    doc.addSignature(pdSignature, sigInterface);

    // 1d. Save incrementally — this triggers sigInterface.sign()
    ByteArrayOutputStream baos = new ByteArrayOutputStream();
    doc.saveIncremental(baos);
    doc.close();

    log.info("PDF signed remotely: resultSize={} bytes", baos.size());
    return baos.toByteArray();
```

```
}

/**
 * PDFBox SignatureInterface implementation for remote signing.
 * Called by PDFBox during saveIncremental() with the byte range content.
 */
public class RemoteSignatureInterface
        implements SignatureInterface {

    private final String cscBaseUrl;
    private final String accessToken;
    private final String credentialId;
    private final X509Certificate signerCert;
    private final List<X509Certificate> certChain;
    private final String tsaUrl;

    // Constructor omitted for brevity

    @Override
    public byte[] sign(InputStream content) throws IOException {
        try {
            // 2a. Read the byte range content (the data to be signed)
            byte[] contentBytes = content.readAllBytes();

            // 2b. Build the CMS signed attributes
            CMSSignedDataGenerator gen = new CMSSignedDataGenerator();

            // Add certificates to the CMS
            List<X509CertificateHolder> certHolders = new ArrayList<>();
            for (X509Certificate cert : certChain) {
                certHolders.add(new JcaX509CertificateHolder(cert));
            }
            gen.addCertificates(
                    new CollectionStore<>(certHolders));

            // 2c. Compute the message digest of the content
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            byte[] contentHash = md.digest(contentBytes);

            // 2d. Build signed attributes manually
            ASN1EncodableVector signedAttrs =
                    new ASN1EncodableVector();

            // content-type attribute
            signedAttrs.add(new Attribute(
                    CMSAttributes.contentType,
                    new DERSet(CMSObjectIdentifiers.data)));

            // message-digest attribute
            signedAttrs.add(new Attribute(
                    CMSAttributes.messageDigest,
                    new DERSet(new DEROctetString(contentHash))));

            // ESS-signing-certificate-v2 attribute
            byte[] certHash = MessageDigest.getInstance("SHA-256")
```

```
                .digest(signerCert.getEncoded());
        ESSCertIDv2 essCertId = new ESSCertIDv2(
                new AlgorithmIdentifier(
                        NISTObjectIdentifiers.id_sha256),
                certHash);
        SigningCertificateV2 sigCertV2 =
                new SigningCertificateV2(
                        new ESSCertIDv2[]{essCertId});
        signedAttrs.add(new Attribute(
                PKCSObjectIdentifiers
                        .id_aa_signingCertificateV2,
                new DERSet(sigCertV2)));

        // 2e. DER-encode the signed attributes
        DERSet signedAttrSet = new DERSet(signedAttrs);
        byte[] signedAttrBytes = signedAttrSet.getEncoded();

        // 2f. Hash the signed attributes — THIS is what we
        //     send to the remote signing server
        byte[] signedAttrHash = MessageDigest
                .getInstance("SHA-256")
                .digest(signedAttrBytes);

        // --- Phase 2: Remote signing via CSC API ---

        // 3a. Authorize the credential (SCAL2)
        String sad = authorizeCredential(
                cscBaseUrl, accessToken, credentialId, 1);

        // 3b. Sign the hash remotely
        List<byte[]> signatures = signHashes(
                cscBaseUrl, accessToken, credentialId, sad,
                List.of(signedAttrHash));
        byte[] signatureValue = signatures.get(0);

        // --- Phase 3: Assemble the CMS and add timestamp ---

        // 4a. Build the complete CMS SignedData manually
        //     (embedding the remote signature value)
        byte[] cmsBytes = assembleCmsSignedData(
                signedAttrSet, signatureValue,
                certChain, signerCert);

        // 4b. Request a timestamp (for PAdES-B-T)
        if (tsaUrl != null) {
            cmsBytes = addSignatureTimestamp(
                    cmsBytes, signatureValue, tsaUrl);
        }

        return cmsBytes;

    } catch (Exception e) {
        throw new IOException("Remote signing failed", e);
    }
}
```

```
}
```

The code above is a simplified illustration of the remote signing flow. A production implementation must also handle:

- **Error recovery**: What happens if the CSC API call fails after SAD authorization? The SAD may be consumed. You may need to re-authorize.
- **Timeout handling**: Network calls to the CSC API and TSA can fail. Implement timeouts and retries.
- **Content size estimation**: PDFBox's `addSignature` requires you to pre-estimate the size of the `/Contents` value. If you underestimate, the CMS will not fit. Set `SignatureOptions.setPreferredSignatureSize()` generously (e.g., 16384 bytes).
- **Thread safety**: If multiple users sign concurrently, your CSC API client must handle concurrent requests correctly. Use separate HTTP connections per request.

For production remote signing with 3.0.0, consider using the EU DSS framework (Chapter 23), which provides a complete, tested implementation of the remote signing flow with CSC API support.

24.8 Security Considerations

24.8.1 Transport Security

All communication with the CSC API must use TLS 1.2 or later. The access token is a bearer token — anyone who intercepts it can use it to sign. In production:

- Pin the CSC API server's TLS certificate (or at least its root CA) to prevent man-in-the-middle attacks
- Use short-lived access tokens (minutes, not hours)
- Transmit tokens only in the `Authorization` header, never in query parameters or request bodies
- Log all API calls for audit purposes, but never log access tokens or SAD values

24.8.2 Hash Manipulation

The remote signing server signs whatever hash you send it. It does not — and cannot — verify that the hash corresponds to any particular document. This means: if an attacker compromises the client application, they can send arbitrary hashes to the server and obtain valid signatures over arbitrary data.

Mitigations: - **SCAL2 authorization**: The signer must actively authorize each signing operation, limiting the window for automated abuse - **Audit logging**: Every signing operation is logged on the server, creating a forensic trail - **Rate limiting**: Unusual signing patterns (hundreds of signatures per minute from a single user) should trigger alerts - **Document description in SAD**: Some CSC implementations allow including a document description in the authorization request, which the signer can verify before approving

24.8.3 Key Compromise

If an HSM is compromised, all keys stored in it are compromised. This is the catastrophic failure mode of remote signing — a single breach affects all users.

HSM-level mitigations: - Physical security (data center access controls, tamper detection) - Multi-person key ceremony for HSM initialization - Regular firmware updates - Separate HSMs for different security levels (production vs. test) - Geographic redundancy with key replication

Operational mitigations: - Certificate revocation within minutes of detection - Automatic alerts for anomalous signing patterns - Regular penetration testing of the signing infrastructure - Insurance against breach liability (some QTSPs carry specific cyber insurance)

24.9 Key Takeaways

- **Remote signing** moves the private key from the signer's device to a server-side HSM operated by a QTSP — the signer authenticates remotely, the server signs
- The **hash-signing model** preserves document confidentiality: only a 32-byte hash crosses the network, never the document content
- The **CSC API v2.0** standardizes remote signing: OAuth 2.0 authentication, credential discovery, signature activation, hash signing
- **SCAL1** provides session-level sole control (suitable for AdES); **SCAL2** provides per-signature authorization (required for QES)
- The **SAD token** is the proof that the signer authorized a specific signing operation — it has limited lifetime and use count
- **HSMs** store keys and perform cryptographic operations; **SAMs** enforce sole control and verify SAD tokens
- PKCS#11 is the standard interface between the signing server and the HSM — vendor-specific libraries are the most common deployment issue
- Major providers (Swisscom AIS, DigiCert/QuoVadis, InfoCert, Namirial, ZealID, D-Trust) all offer CSC-compliant APIs with varying degrees of completeness
- **Always test against the specific provider's sandbox** — CSC API implementations have provider-specific behaviors
- For production PDF signing via remote services, consider the EU DSS framework (Chapter 23), which provides tested CSC API integration

Chapter 25

The EU DSS Framework

Throughout this book, we have been building PDF signing and validation code from the ground up — constructing CMS structures by hand with BouncyCastle, computing byte ranges manually, assembling DSS dictionaries one stream at a time. This is the right approach for understanding what is happening, but it is the wrong approach for production software.

The **European Digital Signature Services (EU DSS)** framework is the European Commission's open-source reference implementation for digital signature creation, extension, and validation. It is maintained by the European Commission's ISA2 programme (now the Digital Europe Programme), developed primarily by the Luxembourg company Nowina Solutions, and used by dozens of EU member states in their national validation services. It handles PAdES, CAdES, XAdES, and JAdES. It validates against the EU Trusted List. It implements every PAdES profile from B-B through B-LTA. It supports remote signing via the CSC API. And it is free — released under the LGPL 2.1 license.

If you are building production software that creates or validates digital signatures in a European context, EU DSS is not just an option — it is the benchmark against which your implementation will be measured. This chapter covers everything you need to use it: architecture, core modules, the signing flow, validation, Spring Boot integration, and the honest trade-offs between using EU DSS and rolling your own.

25.1 History and Governance

25.1.1 Origins

The EU DSS project began in 2011 as part of the European Commission's **ISA programme** (Interoperability Solutions for European Public Administrations). The goal was to provide member states with a reusable software library that correctly implemented the ETSI signature standards — so that each country would not have to write its own implementation from scratch and risk incompatibilities.

The project was motivated by the same cross-border recognition failures that led to eIDAS itself (Chapter 19). Even when countries agreed on the legal framework, their software implementations often produced or validated signatures differently. A common reference implementation was the practical answer to "why doesn't my signature validate in your system?"

25.1.2 Development

The framework has been developed primarily by **Nowina Solutions** (formerly Nowina Technologies), a Luxembourg-based company specializing in electronic signatures and trust

services. Nowina won the European Commission's procurement for DSS development and has been the principal technical contributor since 2015.

The source code is hosted on the European Commission's **GitHub organization** at `github.com/esig/dss` and on the Commission's own code repository at `ec.europa.eu/digital-building-blocks/code/projects/ESIG`.

25.1.3 Version History

Table 25.1: EU DSS Major Version History

Version	Date	Key Changes
4.x	2015-2017	First widely adopted version. PAdES, CAdES, XAdES support. Java 7/8.
5.0	2018	Major rewrite. Modular architecture (separate Maven modules). Java 8 minimum. Introduced pluggable token architecture.
5.6	2020	JAdES (JSON Advanced Electronic Signatures) support. ETSI EN 319 102-1 validation policy. Improved EU Trusted List parsing.
5.10	2022	Java 11 minimum. CSC API support for remote signing. Updated BouncyCastle dependency (1.70+).
5.13	2023	Improved PKCS#11 token support. PSD2 (Payment Services Directive) certificate validation. Updated CRL/OCSP handling.
6.0	2024	Java 17 minimum. Module system (JPMS) support. Restructured Maven coordinates (groupId changed from `eu.europa.ec.joinup.sd-dss` to `eu.europa.dss`). Updated 1.80 dependency.
6.2	2025	Java 17+. Enhanced EUDI wallet integration. Improved TSL (Trusted List) refresh mechanisms. This is the current stable release as of early 2026.

The groupId change in version 6.0 is a common source of confusion. If your project depends on `eu.europa.ec.joinup.sd-dss:dss-pades`, that is DSS 5.x. If it depends on `eu.europa.dss:dss-pades`, that is DSS 6.x. The two groupIds are not interchangeable — do not mix modules from different major versions.

25.2 Architecture

25.2.1 Design Principles

EU DSS is built around three core architectural principles:

1. **Format independence**: The same signing and validation workflow works for PAdES (PDF), CAdES (CMS), XAdES (XML), and JAdES (JSON). The format-specific logic is encapsulated in format modules; the core signing flow is shared.
2. **Pluggable token architecture**: The private key can come from anywhere — a PKCS#12 file, a PKCS#11 smart card, a Microsoft CNG keystore, a remote CSC API service, or a custom `SignatureTokenConnection` implementation. The signing flow does not need to know where the key is.
3. **Policy-based validation**: Validation is driven by a configurable policy (an XML file) that defines which checks to perform and what their outcomes mean. You can use the default ETSI validation policy, or you can define your own — accepting certain non-conformities for your use case while rejecting others.

25.2.2 Module Overview

EU DSS is organized into Maven modules, each with a specific responsibility. You do not need all of them — include only what you use.

Table 25.2: Core EU DSS Modules

Module	Purpose
`dss-spi`	Service Provider Interface. Core abstractions: `DSSDocument`, `SignatureTokenConnection`, `CertificateToken`, `DSSPrivateKeyEntry`. All other modules depend on this.
`dss-model`	Data model classes: `SignatureLevel`, `DigestAlgorithm`, `EncryptionAlgorithm`, `TimestampType`, etc. Enumerations and DTOs used across the framework.
`dss-token`	Token implementations: `Pkcs12SignatureToken`, `Pkcs11SignatureToken`, `MSCAPISignatureToken`, `KeyStoreSignatureToken`. These connect to key material.
`dss-pades`	PAdES (PDF) signing and validation. Depends on Apache PDFBox internally. Produces and validates PAdES-B-B through B-LTA.
`dss-cades`	CAdES (CMS) signing and validation. Used internally by `dss-pades` for the CMS structures inside PDF signatures.
`dss-xades`	XAdES (XML) signing and validation. Not needed for PDF work unless you also handle XML signatures.
`dss-jades`	JAdES (JSON) signing and validation. The newest format support.
`dss-tsl-validation`	EU Trusted List (TSL/EUTL) parsing, validation, and caching. Downloads the trusted lists, verifies their signatures, and builds a trust store that other modules use for validation.
`dss-crl-parser`	CRL downloading and parsing.
`dss-ocsp-parser`	OCSP response fetching and parsing.

Module	Purpose
`dss-service`	Higher-level service classes: `OnlineOCSPSource`, `OnlineCRLSource`, `OnlineTSPSource`, `CommonsDataLoader` (HTTP client). Implements the data-fetching operations that validation requires.
`dss-document`	Document-level abstractions: `SignatureParameters`, `ToBeSigned`, `SignatureValue`, `SignedDocumentValidator`. The signing flow API lives here.
`dss-diagnostic-data-jaxb`	JAXB model for the diagnostic data XML that validation produces. Useful for detailed validation reports.
`dss-detailed-report-jaxb`	JAXB model for the detailed validation report. Human-readable validation results.
`dss-simple-report-jaxb`	JAXB model for the simple validation report. Summary-level pass/fail results.

25.2.3 Maven Coordinates

For a typical PAdES signing and validation project with EU DSS 6.2, you need:

```
<properties>
    <dss.version>6.2</dss.version>
</properties>

<dependencies>
    <!-- Core signing and validation for PAdES -->
    <dependency>
        <groupId>eu.europa.dss</groupId>
        <artifactId>dss-pades</artifactId>
        <version>${dss.version}</version>
    </dependency>

    <!-- Token support (PKCS#12, PKCS#11) -->
    <dependency>
        <groupId>eu.europa.dss</groupId>
        <artifactId>dss-token</artifactId>
        <version>${dss.version}</version>
    </dependency>

    <!-- Online sources for OCSP, CRL, TSA -->
    <dependency>
        <groupId>eu.europa.dss</groupId>
        <artifactId>dss-service</artifactId>
        <version>${dss.version}</version>
    </dependency>

    <!-- EU Trusted List validation -->
    <dependency>
        <groupId>eu.europa.dss</groupId>
        <artifactId>dss-tsl-validation</artifactId>
        <version>${dss.version}</version>
```

```
    </dependency>
</dependencies>
```

EU DSS depends on 3.0.0 internally for PDF operations. If your project also uses PDFBox directly, ensure version alignment — having two different PDFBox versions on the classpath causes unpredictable behavior. Check the DSS BOM (Bill of Materials) for the exact PDFBox version it expects.

25.3 The Signing Flow: ToBeSigned / SignatureValue

25.3.1 The Two-Phase Signing Model

EU DSS uses a two-phase signing model that maps directly to the hash-signing model described in Chapter 22. This separation is what makes remote signing possible — the private key operation can happen anywhere, including on a remote HSM via the CSC API.

Phase 1: Compute what needs to be signed

```
// 1. Create the PAdES service
PAdESService service = new PAdESService(certificateVerifier);

// 2. Set the TSA source (for B-T and above)
service.setTspSource(onlineTSPSource);

// 3. Configure signature parameters
PAdESSignatureParameters parameters = new PAdESSignatureParameters();
parameters.setSignatureLevel(SignatureLevel.PAdES_BASELINE_T);
parameters.setDigestAlgorithm(DigestAlgorithm.SHA256);
parameters.setSigningCertificate(new CertificateToken(signingCert));
parameters.setCertificateChain(certificateChainTokens);

// 4. Get the data to be signed (the hash)
DSSDocument toSignDocument = new InMemoryDocument(pdfBytes);
ToBeSigned dataToSign = service.getDataToSign(
        toSignDocument, parameters);
```

The `ToBeSigned` object contains the bytes that must be signed — specifically, the DER-encoded signed attributes of the CMS SignerInfo, exactly as described in Chapter 22's hash-signing flow. EU DSS has already computed the message digest over the PDF byte range, built the ESS-signing-certificate-v2 attribute, and assembled the signed attributes.

Phase 2: Sign the hash and finalize

```
// 5. Sign the hash (locally or remotely)
//    This is where the private key is used
SignatureValue signatureValue = signingToken.sign(
        dataToSign, parameters.getDigestAlgorithm(),
        privateKeyEntry);

// 6. Finalize the signature — embed in PDF
DSSDocument signedDocument = service.signDocument(
        toSignDocument, parameters, signatureValue);

// 7. Write the result
signedDocument.writeTo(new FileOutputStream("signed.pdf"));
```

The beauty of this design: step 5 is the *only* step that touches the private key. Everything else — PDF parsing, CMS construction, timestamp embedding, incremental save — is handled by EU DSS. If you want to use a remote signing service instead of a local key, you replace step 5 with a CSC API call and everything else stays the same.

25.3.2 SignatureTokenConnection Implementations

The `SignatureTokenConnection` interface abstracts access to the private key. EU DSS ships with several implementations:

```
// PKCS#12 file (local keystore)
Pkcs12SignatureToken token = new Pkcs12SignatureToken(
        new FileInputStream("keystore.p12"),
        new KeyStore.PasswordProtection("password".toCharArray()));
List<DSSPrivateKeyEntry> keys = token.getKeys();
DSSPrivateKeyEntry privateKey = keys.get(0);

// PKCS#11 smart card / HSM
Pkcs11SignatureToken pkcs11Token = new Pkcs11SignatureToken(
        "/usr/lib/opensc-pkcs11.so",  // PKCS#11 library path
        new KeyStore.PasswordProtection("pin".toCharArray()),
        0);  // slot index
List<DSSPrivateKeyEntry> smartCardKeys = pkcs11Token.getKeys();

// Microsoft CNG (Windows certificate store)
MSCAPISignatureToken mscapiToken = new MSCAPISignatureToken();
List<DSSPrivateKeyEntry> windowsKeys = mscapiToken.getKeys();
```

For remote signing via CSC API, you implement `SignatureTokenConnection` with HTTP calls to the CSC endpoints. The `sign()` method sends the hash to `/signatures/signHash` and returns the signature value.

25.4 Complete PAdES-B-T Signing Example

Here is a complete, working example that signs a PDF with PAdES-B-T using a PKCS#12 keystore and a TSA:

```
import eu.europa.dss.cades.signature.CAdESService;
import eu.europa.dss.enumerations.DigestAlgorithm;
import eu.europa.dss.enumerations.SignatureLevel;
import eu.europa.dss.model.DSSDocument;
import eu.europa.dss.model.InMemoryDocument;
import eu.europa.dss.model.SignatureValue;
import eu.europa.dss.model.ToBeSigned;
import eu.europa.dss.pades.PAdESSignatureParameters;
import eu.europa.dss.pades.signature.PAdESService;
import eu.europa.dss.service.http.commons.CommonsDataLoader;
import eu.europa.dss.service.http.commons.OCSPDataLoader;
import eu.europa.dss.service.ocsp.OnlineOCSPSource;
import eu.europa.dss.service.crl.OnlineCRLSource;
import eu.europa.dss.service.tsp.OnlineTSPSource;
import eu.europa.dss.spi.x509.CertificateSource;
import eu.europa.dss.spi.x509.CommonTrustedCertificateSource;
```

```
import eu.europa.dss.token.DSSPrivateKeyEntry;
import eu.europa.dss.token.Pkcs12SignatureToken;
import eu.europa.dss.validation.CommonCertificateVerifier;

/**
 * Complete PAdES-B-T signing with EU DSS.
 * This example uses a PKCS#12 keystore and a public TSA.
 */
public byte[] signWithEuDss(byte[] pdfBytes, String keystorePath,
        String keystorePassword, String tsaUrl) throws Exception {

    // 1. Open the PKCS#12 token
    Pkcs12SignatureToken signingToken = new Pkcs12SignatureToken(
            new FileInputStream(keystorePath),
            new KeyStore.PasswordProtection(
                    keystorePassword.toCharArray()));

    // 2. Get the private key entry (first key in the keystore)
    List<DSSPrivateKeyEntry> keys = signingToken.getKeys();
    DSSPrivateKeyEntry privateKey = keys.get(0);

    // 3. Configure the certificate verifier
    //    (used during signing to validate the chain)
    CommonCertificateVerifier certificateVerifier =
            new CommonCertificateVerifier();

    // OCSP source for online revocation checking
    OnlineOCSPSource ocspSource = new OnlineOCSPSource();
    ocspSource.setDataLoader(new OCSPDataLoader());
    certificateVerifier.setOcspSource(ocspSource);

    // CRL source as fallback
    OnlineCRLSource crlSource = new OnlineCRLSource();
    crlSource.setDataLoader(new CommonsDataLoader());
    certificateVerifier.setCrlSource(crlSource);

    // 4. Configure the TSA source
    OnlineTSPSource tspSource = new OnlineTSPSource(tsaUrl);
    tspSource.setDataLoader(new CommonsDataLoader());

    // 5. Create the PAdES service
    PAdESService padesService = new PAdESService(certificateVerifier);
    padesService.setTspSource(tspSource);

    // 6. Configure signature parameters
    PAdESSignatureParameters parameters =
            new PAdESSignatureParameters();
    parameters.setSignatureLevel(SignatureLevel.PAdES_BASELINE_T);
    parameters.setDigestAlgorithm(DigestAlgorithm.SHA256);
    parameters.setSigningCertificate(
            privateKey.getCertificate());
    parameters.setCertificateChain(
            privateKey.getCertificateChain());

    // Optional: set visual signature parameters
```

```
        // parameters.setSignatureImageParameters(...);

        // 7. Phase 1: Get the data to be signed
        DSSDocument toSignDocument = new InMemoryDocument(pdfBytes);
        ToBeSigned dataToSign = padesService.getDataToSign(
                toSignDocument, parameters);

        // 8. Phase 2: Sign the hash
        SignatureValue signatureValue = signingToken.sign(
                dataToSign, parameters.getDigestAlgorithm(),
                privateKey);

        // 9. Finalize: embed signature in PDF
        DSSDocument signedDocument = padesService.signDocument(
                toSignDocument, parameters, signatureValue);

        // 10. Extract the result
        ByteArrayOutputStream result = new ByteArrayOutputStream();
        signedDocument.writeTo(result);

        log.info("PAdES-B-T signature created: {} bytes",
                result.size());
        return result.toByteArray();
    }
```

That is roughly 60 lines of code — including logging, comments, and error handling setup — to produce a fully compliant PAdES-B-T signature. Compare this to the hundreds of lines required to build the CMS structure manually with BouncyCastle (Chapter 12), and the value proposition of EU DSS becomes clear.

25.5 Extending Signatures

One of EU DSS's most powerful features is **signature extension**: upgrading a signature's PAdES profile without re-signing.

```
    /**
     * Extends a PAdES-B-B or B-T signature to PAdES-B-LTA.
     * Fetches revocation data, builds the DSS, adds an archival timestamp.
     */
    public byte[] extendToLta(byte[] signedPdfBytes,
            String tsaUrl) throws Exception {

        // 1. Set up the certificate verifier with online sources
        CommonCertificateVerifier verifier =
                new CommonCertificateVerifier();
        verifier.setOcspSource(new OnlineOCSPSource());
        verifier.setCrlSource(new OnlineCRLSource());

        // 2. Add the EU Trusted List as a trust source
        //    (required for validating qualified certificates)
        // TrustedListsCertificateSource is populated by the
        // TSL loading job — see Spring Boot integration below

        // 3. Create the PAdES service with a TSA
```

```
    PAdESService service = new PAdESService(verifier);
    OnlineTSPSource tspSource = new OnlineTSPSource(tsaUrl);
    service.setTspSource(tspSource);

    // 4. Configure extension parameters
    PAdESSignatureParameters extensionParams =
            new PAdESSignatureParameters();
    extensionParams.setSignatureLevel(
            SignatureLevel.PAdES_BASELINE_LTA);

    // 5. Extend the document
    DSSDocument signedDoc =
            new InMemoryDocument(signedPdfBytes);
    DSSDocument extendedDoc =
            service.extendDocument(signedDoc, extensionParams);

    ByteArrayOutputStream result = new ByteArrayOutputStream();
    extendedDoc.writeTo(result);

    log.info("Signature extended to PAdES-B-LTA: {} bytes",
            result.size());
    return result.toByteArray();
}
```

The `extendDocument` method handles all the complexity described in Chapter 17 (PAdES Profiles) and Chapter 18 (Timestamps and LTV):

1. Parses the existing signature to determine its current level
2. Fetches OCSP responses and CRLs for every certificate in the chain
3. Builds the DSS dictionary with all validation data
4. Requests an archival timestamp from the TSA
5. Saves everything as an incremental update

This is the operation you would run as a scheduled job to keep archived documents verifiable indefinitely.

25.6 Validation with EU DSS

25.6.1 Basic Validation

Validation is where EU DSS provides the most value. Building a correct signature validator from scratch requires implementing the entire ETSI EN 319 102-1 validation algorithm — certificate chain building, revocation checking at the correct point in time, timestamp validation, DSS consumption, policy evaluation. EU DSS does all of this:

```
/**
 * Validates all signatures in a signed PDF.
 * Returns a structured validation report.
 */
public Reports validateSignatures(byte[] signedPdfBytes)
        throws Exception {

    // 1. Set up the certificate verifier
```

```
    CommonCertificateVerifier verifier =
            new CommonCertificateVerifier();
    verifier.setOcspSource(new OnlineOCSPSource());
    verifier.setCrlSource(new OnlineCRLSource());

    // For qualified certificate validation, add the EUTL:
    // verifier.addTrustedCertSources(trustedListSource);

    // 2. Create a validator for the signed document
    DSSDocument document = new InMemoryDocument(signedPdfBytes);
    SignedDocumentValidator validator =
            SignedDocumentValidator.fromDocument(document);
    validator.setCertificateVerifier(verifier);

    // 3. Validate — this runs the full ETSI EN 319 102-1
    //    validation algorithm
    Reports reports = validator.validateDocument();

    // 4. Access the results
    SimpleReport simpleReport = reports.getSimpleReport();
    for (String sigId : simpleReport.getSignatureIdList()) {
        log.info("Signature {}: indication={}, subIndication={}",
                sigId,
                simpleReport.getIndication(sigId),
                simpleReport.getSubIndication(sigId));
        log.info("  Signed by: {}",
                simpleReport.getSignedBy(sigId));
        log.info("  Signing time: {}",
                simpleReport.getSigningTime(sigId));
        log.info("  Signature level: {}",
                simpleReport.getSignatureFormat(sigId));
    }

    return reports;
}
```

25.6.2 Validation Reports

EU DSS produces three levels of validation reports:

1. **Simple Report**: A summary of each signature's validity — pass/fail, signer name, signing time, signature level. Suitable for end-user display.
2. **Detailed Report**: A structured breakdown of every validation check — certificate chain validation, revocation checking, timestamp validation, policy conformance. Each check has an indication (PASSED, FAILED, INDETERMINATE) and a sub-indication explaining why.
3. **Diagnostic Data**: The raw data collected during validation — all certificates encountered, all revocation data fetched, all timestamps found. This is the lowest level and is useful for debugging validation failures.

```
// Accessing the detailed report
DetailedReport detailedReport = reports.getDetailedReport();

// Get the basic building blocks conclusion
```

```
//  (certificate chain and revocation status)
for (String sigId : detailedReport.getSignatureIds()) {
    Indication bbsIndication =
            detailedReport.getBasicBuildingBlocksIndication(sigId);
    SubIndication bbsSubIndication =
            detailedReport.getBasicBuildingBlocksSubIndication(sigId);

    log.info("Signature {} basic building blocks: {} / {}",
            sigId, bbsIndication, bbsSubIndication);

    // Get timestamp validation results
    List<String> tsIds =
            detailedReport.getTimestampIds(sigId);
    for (String tsId : tsIds) {
        log.info("  Timestamp {}: {}",
                tsId,
                detailedReport.getTimestampIndication(tsId));
    }
}
```

25.6.3 EU Trusted List Integration

For validating qualified certificates, EU DSS can load and parse the EU Trusted List (EUTL):

```
/**
 * Configures EU DSS to validate against the EU Trusted List.
 * The EUTL is downloaded, its signature verified, and the qualified
 * trust service providers are extracted as trusted certificate sources.
 */
public TrustedListsCertificateSource loadEuTrustedList()
        throws Exception {

    TrustedListsCertificateSource trustedListSource =
            new TrustedListsCertificateSource();

    // The LOTL (List of the Lists) is the entry point
    LOTLSource lotlSource = new LOTLSource();
    lotlSource.setUrl(
            "https://ec.europa.eu/tools/lotl/eu-lotl.xml");

    // The LOTL is signed — we need the signing certificate
    // to verify it (bootstrapping problem — this cert is
    // distributed as part of the EU DSS distribution)
    lotlSource.setCertificateSource(
            officialJournalCertificateSource());

    // Configure the TSL validation job
    TLValidationJob tlValidationJob = new TLValidationJob();
    tlValidationJob.setListOfTrustedListSources(lotlSource);
    tlValidationJob.setTrustedListCertificateSource(
            trustedListSource);

    // Use a caching data loader to avoid re-downloading
    // the entire EUTL on every validation
    FileCacheDataLoader cacheDataLoader =
```

```
            new FileCacheDataLoader();
    cacheDataLoader.setCacheExpirationTime(
            24 * 60 * 60 * 1000L); // 24 hours
    cacheDataLoader.setFileCacheDirectory(
            new File("/tmp/dss-tl-cache"));
    tlValidationJob.setOnlineDataLoader(cacheDataLoader);

    // Run the initial load
    tlValidationJob.onlineRefresh();

    log.info("EU Trusted List loaded: {} trusted certificates",
            trustedListSource.getNumberOfTrustedCertificates());

    return trustedListSource;
}
```

The EU Trusted List is large — the LOTL (List of the Lists) references 27+ national trusted lists, each containing dozens of trust service providers and their certificates. The initial download and parsing can take 30-60 seconds.

In production: - Load the EUTL at application startup, not on the first validation request - Cache the downloaded lists on disk (use `FileCacheDataLoader`) - Refresh periodically (every 24 hours is typical) rather than on every request - If the EUTL is unreachable, fall back to the cached version

Never skip EUTL validation — if you validate qualified signatures without checking the EUTL, you are accepting certificates that may not be qualified.

25.7 Spring Boot Integration

EU DSS provides a Spring Boot starter module that auto-configures most of the framework:

```
<dependency>
    <groupId>eu.europa.dss</groupId>
    <artifactId>dss-spring-boot-starter</artifactId>
    <version>${dss.version}</version>
</dependency>
```

25.7.1 Configuration Properties

The starter exposes configuration through `application.properties`:

```
# TSA configuration
dss.tsp.url=http://timestamp.digicert.com
dss.tsp.digest-algorithm=SHA256

# EU Trusted List
dss.lotl.url=https://ec.europa.eu/tools/lotl/eu-lotl.xml
dss.lotl.cache-directory=/var/cache/dss/tl
dss.lotl.cache-expiration=86400000

# OCSP/CRL timeouts
dss.ocsp.connect-timeout=5000
dss.ocsp.read-timeout=10000
```

```
dss.crl.connect-timeout=5000
dss.crl.read-timeout=30000

# Proxy (if behind a corporate proxy)
dss.proxy.https.host=proxy.example.com
dss.proxy.https.port=8443
```

25.7.2 Auto-Configured Beans

The starter auto-configures the following Spring beans:

- `CommonCertificateVerifier` — pre-configured with OCSP and CRL sources
- `OnlineTSPSource` — configured from `dss.tsp.url`
- `OnlineOCSPSource` and `OnlineCRLSource` — with timeout configuration
- `TrustedListsCertificateSource` — loaded from the EUTL on startup
- `TLValidationJob` — scheduled to refresh the EUTL periodically
- `CommonsDataLoader` — HTTP client with proxy and timeout support

You can inject these directly into your services:

```
@Service
@Slf4j
public class SignatureService {

    private final CommonCertificateVerifier certificateVerifier;
    private final OnlineTSPSource tspSource;

    public SignatureService(
            CommonCertificateVerifier certificateVerifier,
            OnlineTSPSource tspSource) {
        this.certificateVerifier = certificateVerifier;
        this.tspSource = tspSource;
    }

    public byte[] signPdf(byte[] pdfBytes,
            SignatureTokenConnection token,
            DSSPrivateKeyEntry keyEntry) throws Exception {

        PAdESService service =
                new PAdESService(certificateVerifier);
        service.setTspSource(tspSource);

        PAdESSignatureParameters params =
                new PAdESSignatureParameters();
        params.setSignatureLevel(
                SignatureLevel.PAdES_BASELINE_T);
        params.setDigestAlgorithm(DigestAlgorithm.SHA256);
        params.setSigningCertificate(
                keyEntry.getCertificate());
        params.setCertificateChain(
                keyEntry.getCertificateChain());

        DSSDocument doc = new InMemoryDocument(pdfBytes);
        ToBeSigned tbs = service.getDataToSign(doc, params);
```

```
        SignatureValue sv = token.sign(
                tbs, params.getDigestAlgorithm(), keyEntry);
        DSSDocument signed =
                service.signDocument(doc, params, sv);

        ByteArrayOutputStream out = new ByteArrayOutputStream();
        signed.writeTo(out);
        return out.toByteArray();
    }
}
```

25.8 EU DSS vs. Rolling Your Own

25.8.1 When to Use EU DSS

Use EU DSS when:

- **You need eIDAS compliance**: EU DSS is the reference implementation. If your validation disagrees with EU DSS, regulators will assume your code is wrong, not EU DSS.
- **You validate qualified signatures**: EUTL parsing, QCStatements interpretation, SCAL assessment — EU DSS handles all of this. Implementing it correctly from scratch is months of work.
- **You need multiple signature formats**: If your application handles PAdES, CAdES, and XAdES, EU DSS's format-independent architecture saves enormous effort.
- **You need long-term maintenance**: EU DSS is actively maintained with a clear release cadence. ETSI standards evolve, new algorithms are added, old ones deprecated. EU DSS tracks these changes.

25.8.2 When to Consider Alternatives

EU DSS may not be the right choice when:

- **You need total control over CMS construction**: EU DSS abstracts away the CMS building process. If your use case requires non-standard CMS attributes, custom signature dictionary entries, or unusual byte range handling, EU DSS may fight you. In these cases, using BouncyCastle directly (as shown throughout this book) gives you full control.
- **You have strict dependency constraints**: EU DSS brings a significant dependency tree — PDFBox, BouncyCastle, Apache HttpClient, JAXB, SLF4J, and dozens of transitive dependencies. If your project has strict dependency policies, the EU DSS footprint may be too large.
- **You only create signatures, never validate**: If you only sign PDFs and never validate them, EU DSS brings a lot of validation machinery you do not need. A leaner approach with PDFBox + BouncyCastle may be more appropriate.
- **You need to support PDF features beyond signing**: EU DSS is focused on digital signatures. It does not help with form filling, content editing, font embedding, or other PDF operations. You will still need PDFBox or another PDF library for those.

25.8.3 The Hybrid Approach

In practice, many projects use a hybrid approach:

- **EU DSS for validation**: Use `SignedDocumentValidator` to validate incoming signed PDFs. This gives you correct ETSI EN 319 102-1 validation with EUTL support.
- **PDFBox + BouncyCastle for signing**: Use PDFBox for PDF manipulation and BouncyCastle for CMS construction. This gives you full control over the signing process while still producing standard-compliant signatures.
- **EU DSS for extension**: Use `PAdESService.extendDocument()` to upgrade signatures from B-B/B-T to B-LTA. This is complex to implement manually (DSS dictionary construction, archival timestamp embedding) and EU DSS does it reliably.

This hybrid model gives you the best of both worlds: full control where you need it, and battle-tested reference code where correctness is paramount.

If you adopt the hybrid approach, be mindful of PDFBox version alignment. EU DSS uses a specific PDFBox version internally. If your direct PDFBox dependency uses a different version, classloader conflicts can cause subtle, hard-to-diagnose bugs — wrong method signatures, missing classes, or (worst case) silent data corruption. Always align your PDFBox version with the one in EU DSS's dependency tree.

25.9 Common Pitfalls

25.9.1 1. Forgetting to Set the Certificate Chain

```
// WRONG – only sets the signing certificate, not the chain
parameters.setSigningCertificate(keyEntry.getCertificate());
// Missing: parameters.setCertificateChain(...)

// RIGHT – set both
parameters.setSigningCertificate(keyEntry.getCertificate());
parameters.setCertificateChain(keyEntry.getCertificateChain());
```

Without the certificate chain, the CMS `certificates` collection will contain only the signer's certificate. Validators that do not have access to the intermediate certificates (from AIA or a local store) will fail to build the chain.

25.9.2 2. Not Configuring a Certificate Verifier

```
// WRONG – empty verifier, cannot check revocation
CommonCertificateVerifier verifier =
        new CommonCertificateVerifier();
// No OCSP source, no CRL source set

// RIGHT – configure revocation sources
CommonCertificateVerifier verifier =
        new CommonCertificateVerifier();
verifier.setOcspSource(new OnlineOCSPSource());
verifier.setCrlSource(new OnlineCRLSource());
```

An unconfigured verifier cannot check certificate revocation. EU DSS will still produce a signature, but validation will report INDETERMINATE for revocation checking.

25.9.3 3. Underestimating /Contents Size

When EU DSS constructs the PDF signature, it must pre-allocate space for the `/Contents` hex string. If the allocated space is too small, the CMS signature (with certificates, timestamps, etc.) will not fit. EU DSS handles this internally with reasonable defaults, but if you are building very large certificate chains or using multiple timestamps, you may need to increase the allocation:

```
PAdESSignatureParameters params =
        new PAdESSignatureParameters();
params.setContentSize(32768);  // 32 KB for /Contents
// Default is typically 16384 (16 KB)
```

25.9.4 4. Blocking the EUTL Load on First Request

The EUTL download takes 30-60 seconds. If you trigger it lazily (on the first validation request), your first user will experience a 30-second delay. Always load the EUTL eagerly at application startup:

```
@PostConstruct
public void initTrustedLists() {
    log.info("Loading EU Trusted List — this may take 30-60 seconds");
    tlValidationJob.onlineRefresh();
    log.info("EU Trusted List loaded: {} certificates",
            trustedListSource.getNumberOfTrustedCertificates());
}
```

25.9.5 5. Ignoring Validation Sub-Indications

A common mistake is checking only the top-level `Indication` (TOTAL_PASSED, TOTAL_FAILED, INDETERMINATE) and ignoring the `SubIndication`. The sub-indication tells you *why* the validation reached that conclusion:

- `NO_SIGNING_CERTIFICATE_FOUND` — the signer's certificate is missing
- `REVOKED` — a certificate in the chain has been revoked
- `EXPIRED` — a certificate has expired and there is no proof that signing occurred before expiry
- `TRY_LATER` — revocation data could not be fetched (OCSP/CRL unreachable)
- `CRYPTO_CONSTRAINTS_FAILURE` — weak algorithms detected (e.g., SHA-1)
- `HASH_FAILURE` — the message digest does not match (document has been modified)

Always log the sub-indication alongside the indication for debugging.

25.10 Key Takeaways

- **EU DSS** is the European Commission's open-source reference implementation for digital signature creation, extension, and validation — LGPL 2.1 licensed, actively maintained
- The **two-phase signing model** (ToBeSigned / SignatureValue) cleanly separates CMS construction from the private key operation, enabling local and remote signing through the same API

- **SignatureTokenConnection** abstracts key access: PKCS#12, PKCS#11, MS CAPI, or custom (CSC API) — swap the token, keep the signing flow
- EU DSS handles **PAdES-B-B through B-LTA** creation and the full ETSI EN 319 102-1 **validation algorithm**, including EUTL lookup for qualified certificate determination
- **Signature extension** (`extendDocument`) upgrades a signature's PAdES profile without re-signing — the most practical feature for archival workflows
- The **Spring Boot starter** auto-configures certificate verifiers, TSA sources, and EUTL loading — production-ready with minimal configuration
- **Validation reports** come in three levels: Simple (pass/fail), Detailed (per-check breakdown), Diagnostic (raw collected data)
- The **EUTL** must be loaded at startup and cached on disk — lazy loading causes unacceptable latency, missing EUTL data causes incorrect qualified status determination
- **Hybrid approach** is common in practice: EU DSS for validation and extension, PDFBox + BouncyCastle for signing with full CMS control
- Always align **PDFBox versions** between your direct dependency and EU DSS's transitive dependency — version mismatches cause classloader conflicts

Chapter 26

Security, Incidents & Outlook

26.1 PDF Signature Attacks: A Complete Technical Analysis

> We found 21 out of 22 desktop viewer applications and 5 out of 7 online validation services to be vulnerable to at least one of our attacks.
>
> — Vladislav Mladenov et al. 1 Trillion Dollar Refund

For nearly two decades, the PDF signature ecosystem operated on an assumption that turned out to be wrong: that viewers actually verified what they claimed to verify. Between 2019 and 2021, a research team at Ruhr University Bochum systematically dismantled this assumption. They showed that an attacker could take a validly signed PDF, modify it in specific ways, and present the result to any major PDF viewer — Adobe Acrobat, Foxit Reader, Nitro, LibreOffice, Okular, dozens of others — and the viewer would display a green checkmark and the text ”Signature is valid.”

The signed content had been altered. The viewer said it hadn’t. Every viewer was wrong.

This chapter is a complete technical analysis of every major attack against PDF digital signatures — from the Bochum research program to earlier CVEs, from the theoretical mechanics to working code that demonstrates each attack class. The purpose is not to enable attacks but to make them impossible to misunderstand. If you build or maintain a PDF signature validation system, you need to understand exactly how these attacks work so that your code cannot be fooled by them. If you are a security researcher, this chapter gives you the foundation to find the next generation of PDF signature vulnerabilities — because the Bochum team found the obvious ones, but the attack surface is far from fully explored.

26.1.1 The Researchers

The PDF signature attacks of 2019–2021 were the work of the **Chair of Network and Data Security** (Lehrstuhl für Netz- und Datensicherheit) at Ruhr University Bochum, Germany, led by **Prof. Dr. Jörg Schwenk**.

The core research team:

- **Vladislav Mladenov** — PhD researcher, co-first author of the CCS 2019 paper, led the implementation and testing of the USF, ISA, and SWA attack variants
- **Christian Mainka** — post-doctoral researcher, co-first author of both the CCS 2019 and NDSS 2021 papers, designed the Shadow Attack taxonomy and coordinated responsible disclosure
- **Karsten Meyer zu Selhausen** — researcher and co-founder of **Hackmanit GmbH**, the security consultancy that spun out of the Bochum group; contributed to attack implementation and vendor coordination

- **Martin Grothe** — researcher, contributed to the CCS 2019 paper's systematic evaluation across 22 viewers
- **Simon Rohlmann** — researcher, co-author of the NDSS 2021 Shadow Attacks paper, developed the Hide-and-Replace variant
- **Prof. Dr. Jörg Schwenk** — senior author on both papers, provided the research direction and the formal security model for PDF signatures

The team maintained a coordinated disclosure website at **pdf-insecurity.org**, where they published proof-of-concept PDFs, detailed attack descriptions, and a viewer-by-viewer vulnerability matrix. Hackmanit GmbH later developed **PDF Detector**, a tool for checking PDFs against known attack patterns.

The two landmark papers:

1. *"1 Trillion Dollar Refund: How To Spoof PDF Signatures"* — presented at **ACM Conference on Computer and Communications Security (CCS)**, November 2019, London. This paper introduced three attack classes (USF, ISA, SWA) and tested them against 22 desktop viewers and 7 online validation services.
2. *"Shadow Attacks: Hiding and Replacing Content in Signed PDFs"* — presented at **Network and Distributed Systems Security Symposium (NDSS)**, February 2021 (virtual). This paper introduced a fundamentally different attack model where malicious content is prepared *before* the document is signed.

Both conferences are top-tier academic venues in computer security. The CCS 2019 paper won a **Distinguished Paper Award** — a recognition given to only a handful of papers at each conference.

26.1.2 Before Bochum: The State of PDF Signature Validation

To understand why the Bochum attacks worked, you need to understand what PDF viewers *were* doing when they claimed to "validate" a signature.

A correct signature validation must verify at least the following:

1. **Structural integrity**: The byte range specified in the `/ByteRange` array must cover the entire file except the gap reserved for the `/Contents` value. There must be no unaccounted bytes before, between, or after the byte ranges.
2. **Cryptographic verification**: The CMS `SignedData` structure in `/Contents` must contain a valid signature over the hash of the byte range data. The digest algorithm, signing algorithm, and signed attributes must all be consistent.
3. **Certificate chain validation**: The signer's certificate must chain to a trusted root. The chain must be valid at the time of signing (or at the time of a trusted timestamp). Revocation status must be checked.
4. **Modification detection**: If additional data has been appended to the file after the signature (an incremental update), the viewer must determine whether those changes are permissible under the signature's `/TransformMethod` (e.g., DocMDP permission level) or whether they indicate tampering.

What the Bochum team discovered was that most viewers implemented step 3 reasonably well (they checked certificate chains) but failed catastrophically at steps 1, 2, and 4. Some viewers didn't check the cryptographic signature value at all. Some didn't verify that the

byte range covered the entire file. Some allowed arbitrary changes after signing without any warning.

The result: a perfectly executed certificate chain validation was protecting… nothing. The viewers were carefully verifying *who* signed the document, while completely ignoring *whether the document had been changed since they signed it*.

26.1.3 Attack Class 1: Universal Signature Forgery (USF)

26.1.3.1 The Concept

Universal Signature Forgery is the simplest and most devastating of the three original attack classes. The premise is almost embarrassingly straightforward: what happens if the CMS signature value in `/Contents` is wrong?

A correct implementation should reject the signature immediately — if the cryptographic value doesn't verify, nothing else matters. What the Bochum team found was that some viewers never checked.

In the most extreme cases, the `/Contents` value could be empty (all zeros), filled with random bytes, or contain a valid CMS structure that signed completely different data — and the viewer would still display "Signature is valid" with a green checkmark.

26.1.3.2 How It Works

The attack requires no special tools and no knowledge of the signer's private key. The attacker takes a legitimately signed PDF, modifies the document content (changes text, replaces pages, adds content), and re-calculates any internal structures (cross-reference table, object offsets) needed to keep the file syntactically valid. The `/Contents` value — the CMS `SignedData` blob — is left unchanged, even though it now contains a signature over the *original* document bytes, not the modified ones.

In a correct implementation, the viewer would:

1. Read the `/ByteRange` to determine which bytes are signed
2. Hash those bytes using the digest algorithm specified in the CMS structure
3. Extract the signature value from the CMS `SignerInfo`
4. Verify the signature over the hash using the signer's public key
5. If verification fails reject the signature

Vulnerable viewers skipped step 4, step 3, or in some cases both. They parsed the CMS structure to extract the signer's certificate (for display purposes), checked that the certificate chained to a trust anchor, and then displayed the result — without ever verifying that the signature value was mathematically valid for the current document content.

26.1.3.3 The Variants

The Bochum team identified three USF variants with increasing severity:

USF Variant 1: Manipulated SignerInfo

The `SignerInfo.signature` value (the actual RSASSA-PKCS1-v1_5 or ECDSA bytes) is replaced with random data. A correct implementation rejects this immediately during RSA/ECDSA verification. Vulnerable viewers never reached that step.

USF Variant 2: Manipulated ContentInfo

The outer CMS `ContentInfo` wrapping is modified so the `SignedData` structure is structurally invalid — wrong OIDs, missing fields, or inconsistent version numbers. A correct implementation rejects this during CMS parsing. Vulnerable viewers used lenient parsing that silently ignored structural errors.

USF Variant 3: Manipulated `/Contents` Value

The entire `/Contents` hex string is replaced — either with zeros, with a truncated CMS structure, or with a CMS structure copied from a different document. This tests whether the viewer checks the relationship between the CMS content and the actual document bytes at all.

26.1.3.4 Demonstration: Detecting USF Vulnerability

The following code demonstrates how to detect whether a signed PDF's CMS signature actually validates cryptographically — the check that USF-vulnerable viewers failed to perform. This is the *defense*, not the attack: if you are building a validator, this is the minimum you must implement.

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;
import org.bouncycastle.cert.X509CertificateHolder;
import org.bouncycastle.cert.jcajce.JcaX509CertificateConverter;
import org.bouncycastle.cms.CMSProcessableByteArray;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.SignerInformation;
import org.bouncycastle.cms.jcajce.JcaSimpleSignerInfoVerifierBuilder;
import org.bouncycastle.jce.provider.BouncyCastleProvider;
import org.bouncycastle.util.Store;

import java.io.FileInputStream;
import java.io.InputStream;
import java.security.Security;
import java.security.cert.X509Certificate;
import java.util.Collection;

/**
 * Demonstrates the cryptographic signature verification that USF-vulnerable
 * viewers failed to perform. This is the MINIMUM check for signature integrity.
 */
public class UsfDefenseCheck {

    static {
        Security.addProvider(new BouncyCastleProvider());
    }

    public static void verifySignature(byte[] pdfBytes) throws Exception {
        PDDocument doc = Loader.loadPDF(pdfBytes);
```

```
for (PDSignature sig : doc.getSignatureDictionaries()) {
    // Step 1: Extract the signed byte ranges
    byte[] signedContent = sig.getSignedContent(
            new ByteArrayInputStream(pdfBytes));

    // Step 2: Extract the CMS SignedData from /Contents
    byte[] cmsBytes = sig.getContents(
            new ByteArrayInputStream(pdfBytes));

    // Step 3: Parse the CMS structure
    CMSSignedData cmsSignedData = new CMSSignedData(
            new CMSProcessableByteArray(signedContent),
            cmsBytes);

    // Step 4: Verify each signer's cryptographic signature
    Store<X509CertificateHolder> certStore =
            cmsSignedData.getCertificates();

    for (SignerInformation signer :
            cmsSignedData.getSignerInfos().getSigners()) {

        Collection<X509CertificateHolder> matches =
                certStore.getMatches(signer.getSID());

        if (matches.isEmpty()) {
            System.err.println(
                    "FAIL: No certificate found for signer");
            continue;
        }

        X509CertificateHolder certHolder = matches.iterator().next();
        X509Certificate cert = new JcaX509CertificateConverter()
                .setProvider("BC")
                .getCertificate(certHolder);

        // THIS IS THE CHECK THAT USF-VULNERABLE VIEWERS SKIPPED
        boolean cryptoValid = signer.verify(
                new JcaSimpleSignerInfoVerifierBuilder()
                        .setProvider("BC")
                        .build(cert));

        if (!cryptoValid) {
            System.err.println(
                    "FAIL: Cryptographic signature verification "
                    + "FAILED for signer: " + cert.getSubjectX500Principal());
            System.err.println(
                    "This document may have been tampered with (USF attack)");
        } else {
            System.out.println(
                    "PASS: Cryptographic signature valid for: "
                    + cert.getSubjectX500Principal());
        }
    }
}
```

```
        doc.close();
    }
}
```

- Extracts the bytes covered by `/ByteRange` — this is the data that was supposedly signed
- Extracts the raw CMS `SignedData` blob from the `/Contents` hex string
- Creates a CMS structure with the signed content as the detached data — BouncyCastle will use this for digest comparison
- If no certificate matches the `SignerInfo.sid`, the CMS structure is malformed — reject immediately
- **This is the critical line.** `signer.verify()` performs the actual RSA/ECDSA verification: it computes the digest over the signed content, verifies the signed attributes, and checks the signature value against the signer's public key. USF-vulnerable viewers never called the equivalent of this method.

If your signature validation code does not include a step equivalent to `signer.verify()` — an actual cryptographic verification of the signature value against the document bytes — your code is vulnerable to Universal Signature Forgery. This is non-negotiable. It must happen before you display any "valid" indicator to the user.

26.1.3.5 Constructing a USF Attack (Proof of Concept)

To understand USF fully, here is how an attacker constructs one. This is *not* production exploit code — it is a minimal demonstration of the mechanics that every PDF signature validator must defend against.

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.PDPage;
import org.apache.pdfbox.pdmodel.PDPageContentStream;
import org.apache.pdfbox.pdmodel.font.PDType1Font;
import org.apache.pdfbox.pdmodel.font.Standard14Fonts;

import java.io.ByteArrayOutputStream;
import java.io.File;
import java.nio.file.Files;

/**
 * Demonstrates USF attack mechanics. Takes a signed PDF, modifies the visible
 * content, and writes the result. The CMS signature value is left unchanged --
 * it now signs the ORIGINAL content, not the modified content.
 *
 * A correct validator will reject this because the digest over the byte range
 * no longer matches the digest in the CMS SignerInfo.
 * A USF-vulnerable validator will show "Signature is valid."
 */
public class UsfAttackDemo {

    public static void main(String[] args) throws Exception {
        byte[] originalPdf = Files.readAllBytes(
                new File("signed-document.pdf").toPath());
```

```
        // Load the signed PDF
        PDDocument doc = Loader.loadPDF(originalPdf);

        // Add a new page with attacker-controlled content
        PDPage attackPage = new PDPage();
        doc.addPage(attackPage);

        PDPageContentStream cs = new PDPageContentStream(doc, attackPage);
        cs.beginText();
        cs.setFont(
                new PDType1Font(Standard14Fonts.FontName.HELVETICA_BOLD), 24);
        cs.newLineAtOffset(50, 700);
        cs.showText("APPROVED - Payment of $1,000,000 authorized");
        cs.endText();
        cs.close();

        // Save with full rewrite — this changes object offsets,
        // xref table, and file length. The /ByteRange in the original
        // signature now covers different bytes, but the /Contents
        // CMS value still contains the hash of the ORIGINAL bytes.
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        doc.save(baos);
        doc.close();

        Files.write(
                new File("usf-attack-result.pdf").toPath(), baos.toByteArray());

        // A correct validator: REJECTS (digest mismatch)
        // A USF-vulnerable validator: shows green checkmark
    }
}
```

- Modifying the document structure after it was signed — adding a page, changing text, anything
- The attacker can insert any content they want
- A full `save()` (not `saveIncremental()`) rewrites the entire file, changing byte offsets everywhere. The original `/ByteRange` and `/Contents` are preserved structurally, but the bytes they reference have changed. The CMS signature now signs data that no longer exists at those byte positions.

This code produces a PDF that any *correct* validator will reject. The point is that in 2019, most validators were not correct. The digest computed over the new byte range content will not match the digest inside the CMS `SignerInfo.digestAlgorithm` — the very first check in proper CMS verification detects this. USF worked because viewers displayed the signer's certificate information *without performing this check*.

26.1.4 Attack Class 2: Incremental Saving Attack (ISA)

26.1.4.1 The Concept

The Incremental Saving Attack is more subtle than USF and exploits a feature that is fundamental to how PDF signatures work: **incremental updates**.

When a PDF is signed, the signing process:

1. Appends the signature dictionary and any new objects to the end of the file
2. Appends a new cross-reference section pointing to the new/modified objects
3. Appends a new trailer
4. The `/ByteRange` in the signature covers everything from the beginning of the file through the end of this new revision — with a gap for the `/Contents` value

The critical insight is that PDF is designed to allow *additional* incremental updates after signing. This is intentional — you need to be able to add a second signature to an already-signed document, or fill in form fields on a signed form. ISO 32000-2:2020 §12.8.1 explicitly allows certain modifications after signing, depending on the signature's `/TransformMethod` and permission level.

The question is: does the viewer correctly distinguish between *permissible* incremental updates (adding another signature) and *impermissible* ones (changing the page content)?

The Bochum team found that most viewers did not.

26.1.4.2 How It Works

The attacker takes a legitimately signed PDF and appends a new incremental update — exactly the way PDFBox's `saveIncremental()` method does. But instead of adding a second signature or filling in a form field, the attacker appends objects that change the visible content: modified page content streams, replaced images, rewritten text, or entirely new pages.

The original signature remains intact — its byte range still covers the original revision, and its CMS signature is still mathematically valid over those bytes. The appended revision contains the modifications, complete with a new cross-reference section and trailer that redirect the viewer to the modified objects.

A correct validator must:

1. Verify the signature is cryptographically valid (steps from USF defense)
2. Determine the document's revision at the time of signing (the byte range endpoint)
3. Compare the current document state to the signed revision
4. If there are changes beyond the signed revision, evaluate whether those changes are permissible under the signature's DocMDP permissions
5. If impermissible changes are detected, display a clear warning: "The document has been modified after signing"

Vulnerable viewers either:

- Did not compare the current document state to the signed revision at all
- Compared but accepted all incremental updates as permissible
- Only checked whether the appended data was "syntactically valid PDF" rather than whether it changed the visible content

26.1.4.3 The Variants

ISA Variant 1: Content Replacement via Incremental Update

The attacker appends a new revision that redefines existing page content stream objects. The cross-reference in the new revision points to new objects at the end of the file that replace the original content. The viewer renders the new content; the signature covers the old content.

ISA Variant 2: Page Insertion/Deletion

The attacker appends a new revision that modifies the page tree — adding new pages, removing existing pages, or reordering them. The `/Pages` dictionary in the new revision points to a different set of page objects.

ISA Variant 3: Annotation and Form Field Manipulation

The attacker appends annotations or modifies form field values. This variant is particularly insidious because some annotation changes *are* legitimately allowed after signing (e.g., adding signature fields for subsequent signers). The boundary between ”allowed” and ”not allowed” is defined by the DocMDP permission level, and most viewers did not enforce it correctly.

26.1.4.4 Demonstration: ISA Attack Mechanics

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.cos.COSDictionary;
import org.apache.pdfbox.cos.COSName;
import org.apache.pdfbox.cos.COSArray;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.PDPage;
import org.apache.pdfbox.pdmodel.PDPageContentStream;
import org.apache.pdfbox.pdmodel.common.PDRectangle;
import org.apache.pdfbox.pdmodel.font.PDType1Font;
import org.apache.pdfbox.pdmodel.font.Standard14Fonts;

import java.io.ByteArrayOutputStream;
import java.io.File;
import java.nio.file.Files;

/**
 * Demonstrates ISA mechanics. Takes a signed PDF and appends an incremental
 * update that replaces the visible content of page 1.
 *
 * The original signature remains cryptographically valid over the original
 * byte range. The appended revision changes what the user sees.
 *
 * A correct validator detects the impermissible modification.
 * An ISA-vulnerable validator shows "Signature is valid" on the modified content.
 */
public class IsaAttackDemo {

    public static void main(String[] args) throws Exception {
        byte[] signedPdf = Files.readAllBytes(
                new File("signed-document.pdf").toPath());

        // Load for incremental save — preserves the original bytes
        PDDocument doc = Loader.loadPDF(signedPdf);

        // Get the first page and replace its content
```

```
        PDPage page = doc.getPage(0);

        // Create a new content stream that overwrites existing content
        PDPageContentStream cs = new PDPageContentStream(
                doc, page,
                PDPageContentStream.AppendMode.OVERWRITE,
                true);

        cs.beginText();
        cs.setFont(
                new PDType1Font(Standard14Fonts.FontName.HELVETICA_BOLD), 36);
        cs.newLineAtOffset(50, 700);
        cs.showText("CONTRACT VOIDED");
        cs.endText();
        cs.close();

        // Save incrementally — appends new objects + xref + trailer
        // The original signature's byte range is UNTOUCHED
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        doc.saveIncremental(baos);
        doc.close();

        Files.write(
                new File("isa-attack-result.pdf").toPath(), baos.toByteArray());

        // The file now looks like this:
        // [original PDF bytes ... signature ... %%EOF]  <-- signed revision
        // [new objects ... new xref ... new trailer ... %%EOF]  <-- attacker's appendage
        //
        // The signature covers only the first section.
        // The viewer renders the second section.
        // A correct validator: warns "Document modified after signing"
        // An ISA-vulnerable validator: shows green checkmark
    }
}
```

- `AppendMode.OVERWRITE` replaces the existing content stream. PDFBox tracks this as a modified object, which will be written in the incremental update
- The attacker-controlled content — can be anything
- `saveIncremental()` appends to the existing file bytes without modifying them. The original signature's `/ByteRange` and `/Contents` remain at their original byte positions. The new revision simply adds new objects and a new cross-reference section after the original `%%EOF`

26.1.4.5 Detecting ISA: Revision Comparison

The defense against ISA requires comparing the document as rendered from the signed revision to the document as rendered from the current (potentially modified) revision.

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;

import java.io.ByteArrayInputStream;
```

```
import java.util.Arrays;

/**
 * Demonstrates how to extract the signed revision of a PDF and compare it
 * to the full file — the core defense against ISA.
 */
public class IsaDefenseCheck {

    public static void checkForAppendedContent(byte[] pdfBytes)
            throws Exception {

        PDDocument doc = Loader.loadPDF(pdfBytes);

        for (PDSignature sig : doc.getSignatureDictionaries()) {
            int[] byteRange = sig.getByteRange();

            if (byteRange == null || byteRange.length != 4) {
                System.err.println("FAIL: Invalid or missing /ByteRange");
                continue;
            }

            // The signed region ends at byteRange[2] + byteRange[3]
            int signedEnd = byteRange[2] + byteRange[3];

            // Check: does the file extend beyond the signed region?
            if (pdfBytes.length > signedEnd) {
                int extraBytes = pdfBytes.length - signedEnd;
                System.err.println(
                        "WARNING: File contains " + extraBytes
                        + " bytes AFTER the signed byte range");
                System.err.println(
                        "This indicates an incremental update after signing.");
                System.err.println(
                        "These bytes are NOT covered by the signature.");
                System.err.println(
                        "Investigate whether the changes are permissible "
                        + "(e.g., additional signatures, form fills) "
                        + "or malicious (content replacement).");

                // Extract the signed revision for comparison
                byte[] signedRevision = Arrays.copyOfRange(
                        pdfBytes, 0, signedEnd);

                // You can now load signedRevision as a separate PDDocument
                // and compare its rendered pages to the full document
                // to determine exactly what changed
            } else if (pdfBytes.length == signedEnd) {
                System.out.println(
                        "PASS: File ends exactly at the signed byte range. "
                        + "No incremental updates after signing.");
            } else {
                System.err.println(
                        "FAIL: File is SHORTER than the signed byte range. "
                        + "File is truncated or corrupted.");
            }
```

```
        }

        doc.close();
    }
}
```

- `/ByteRange` is an array of four integers: `[offset1, length1, offset2, length2]`. The two ranges cover everything before and after the `/Contents` gap
- The signed content ends at the start of range 2 plus its length — this is the last byte the signature covers
- If the file extends beyond this point, there is appended content that the signature does not cover. This is the ISA detection point
- Extracting the signed revision as a standalone PDF allows you to render it separately and compare to the current document state

A naive "does the file extend beyond the byte range?" check catches obvious ISA attacks, but it is not sufficient. A legitimate second signature also extends the file. The real check requires parsing the appended incremental update and determining *what* changed — was it another signature (allowed), a form fill (allowed under DocMDP P=3), or a content modification (never allowed)? This is the hard part of ISA defense, and it is where most viewers failed in 2019.

26.1.5 Attack Class 3: Signature Wrapping Attack (SWA)

26.1.5.1 The Concept

The Signature Wrapping Attack is the most technically sophisticated of the three original attack classes. Where USF exploits missing cryptographic verification and ISA exploits missing modification detection, SWA exploits the way viewers *locate* the signature within the file.

The core insight: the `/ByteRange` array tells the viewer which bytes are signed, but *nothing prevents an attacker from moving the signed bytes to a different location in the file* — as long as the byte values at the `/ByteRange` offsets still produce the same hash.

Actually, SWA is even simpler than that. The attacker doesn't need the same hash. The attacker modifies the `/ByteRange` array itself to point to a *different* region of the file — one that contains attacker-controlled content — while keeping the original CMS signature that was computed over the original content.

Wait — if the `/ByteRange` is modified, wouldn't the signature be invalid? Yes, *if the viewer reads the `/ByteRange` from the signed revision*. But many viewers read the `/ByteRange` from the *current* cross-reference table, which may have been modified by an incremental update. The attacker appends a new revision that:

1. Redefines the signature dictionary object to point to a new `/ByteRange`
2. Places attacker-controlled content at the byte positions specified by the new `/ByteRange`
3. Ensures the CMS signature (in the original `/Contents`) is structurally present

The viewer follows the new cross-reference to the redefined signature dictionary, reads the attacker's `/ByteRange`, hashes the attacker's content, and — if it doesn't actually verify the hash against the CMS signature (shades of USF) — displays "valid."

In some variants, the viewer *does* check the hash, but the attacker has been clever about which bytes are at the new `/ByteRange` positions, potentially including a copy of the original signed content or exploiting hash-algorithm weaknesses.

26.1.5.2 How It Works

```
Original file layout:
  [header ... objects ... page content ... ]
  [signature dict: /ByteRange [0 840 3000 1200] /Contents <CMS...>]
  [xref ... trailer ... %%EOF]

  Bytes 0-839 and 3000-4199 are signed.
  Bytes 840-2999 are the /Contents gap (CMS signature value).
  The CMS signature is valid over hash(bytes[0..839] + bytes[3000..4199]).

Attacker appends an incremental update:
  [... %%EOF]                              <-- original file ends here
  [new object: modified signature dict     <-- attacker redefines the sig dict
      /ByteRange [0 840 5000 1200]         <-- points to NEW byte positions
      /Contents <original CMS bytes>]      <-- keeps the original CMS value
  [attacker content at byte offset 5000]   <-- whatever the attacker wants
  [new xref pointing to new sig dict]
  [new trailer ... %%EOF]
```

The viewer follows the new xref, finds the redefined signature dictionary, reads `/ByteRange [0 840 5000 1200]`, and hashes bytes at those positions. If it checks the CMS signature against this hash, it will fail (because the CMS signed the *original* byte range content). But if the viewer is also vulnerable to USF (skips the hash verification), the attack succeeds.

SWA is often combined with elements of ISA and USF. The Bochum team found that many real-world exploits used a combination of attack techniques.

26.1.5.3 Demonstration: Detecting SWA

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;

/**
 * Demonstrates SWA detection: verify that the /ByteRange covers the entire
 * file without gaps (except the /Contents gap) and without overlaps.
 */
public class SwaDefenseCheck {

    public static void checkByteRangeIntegrity(byte[] pdfBytes)
            throws Exception {

        PDDocument doc = Loader.loadPDF(pdfBytes);

        for (PDSignature sig : doc.getSignatureDictionaries()) {
            int[] byteRange = sig.getByteRange();
```

```
if (byteRange == null || byteRange.length != 4) {
    System.err.println("FAIL: Missing or malformed /ByteRange");
    continue;
}

int offset1 = byteRange[0]; // Should be 0
int length1 = byteRange[1];
int offset2 = byteRange[2];
int length2 = byteRange[3];
int signedEnd = offset2 + length2;

// Check 1: First range must start at byte 0
if (offset1 != 0) {
    System.err.println(
            "FAIL: /ByteRange does not start at byte 0. "
            + "Starts at: " + offset1);
}

// Check 2: The gap between ranges is the /Contents value
int gapStart = offset1 + length1;
int gapEnd = offset2;
int gapSize = gapEnd - gapStart;

if (gapSize <= 0) {
    System.err.println(
            "FAIL: /ByteRange gap is zero or negative. "
            + "Possible SWA indicator");
}

// Check 3: Verify /Contents hex string is at the gap position
// The gap should contain exactly the hex-encoded CMS signature
// surrounded by angle brackets: <hex...>
if (pdfBytes[gapStart] != '<') {
    System.err.println(
            "FAIL: Expected '<' at gap start (offset "
            + gapStart + "), found: 0x"
            + Integer.toHexString(
                    pdfBytes[gapStart] & 0xFF));
}

// Check 4: The signed region should cover the entire file
// (for a single-signature document with no subsequent updates)
if (signedEnd < pdfBytes.length) {
    System.err.println(
            "WARNING: /ByteRange ends at byte " + signedEnd
            + " but file is " + pdfBytes.length
            + " bytes. Unsigned trailing content detected");
}

// Check 5: Cross-validate that the /ByteRange in the file
// matches what we'd expect from the signed revision
// This requires parsing the original revision's xref
System.out.println(
        "ByteRange analysis: "
        + "range1=[" + offset1 + "," + (offset1 + length1) + ") "
```

```
                        + "gap=[" + gapStart + "," + gapEnd + ") "
                        + "range2=[" + offset2 + "," + signedEnd + ") "
                        + "fileSize=" + pdfBytes.length);
            }

            doc.close();
        }
    }
```

- The first byte range must start at offset 0. If it doesn't, there are unsigned bytes at the beginning of the file — a clear SWA indicator
- The gap between the two ranges must be positive and should correspond to the `/Contents` hex string. A zero or negative gap indicates structural manipulation
- The gap should contain the hex-encoded CMS value. Checking that it starts with < and ends with > is a basic sanity check

SWA defense requires more than just checking `/ByteRange` arithmetic. A sophisticated attacker can construct a valid-looking byte range that passes arithmetic checks but points to carefully crafted content. The complete defense is: (a) verify `/ByteRange` covers the file correctly, (b) verify the CMS signature cryptographically (USF defense), and (c) verify that the `/ByteRange` values come from the signed revision's cross-reference, not from an appended revision.

26.1.6 Attack Class 4: Shadow Attacks (2021)

26.1.6.1 A Fundamentally Different Threat Model

The three attacks from CCS 2019 (USF, ISA, SWA) all shared a common assumption: the attacker modifies a PDF *after* it has been legitimately signed. The signed document is genuine; the attacker's manipulation comes afterward.

Shadow Attacks, published at NDSS 2021, inverted the model. The attacker prepares the PDF *before* it is signed, embedding hidden malicious content that the signer cannot see during the signing process. The signer reviews the document, sees nothing suspicious, and signs it. The signed PDF is then sent to the recipient, who opens it — and sees different content from what the signer saw.

The signature is cryptographically valid. The signer genuinely signed those exact bytes. The bytes just don't mean what the signer thought they meant.

This is not a cryptographic failure. It is a semantic failure: the gap between what a human *sees* when they review a document and what the *bytes* actually encode. And it is far harder to defend against than USF, ISA, or SWA.

26.1.6.2 Shadow Attack Variant 1: Hide

The attacker creates a PDF with two layers of content — one visible, one hidden. The hidden content is placed *behind* the visible content using PDF features like:

- **Overlapping objects**: A white rectangle drawn on top of text, hiding it. The text is in the content stream; the rectangle covers it. After signing, the attacker removes the rectangle via an incremental update, revealing the hidden text.

- **Optional Content Groups (OCG)**: PDF's layer mechanism. Content can be tagged with `/OC` (Optional Content) markers and made visible or invisible via the `/OCProperties` dictionary. The attacker sets a layer to invisible before signing, then sets it to visible after signing.
- **Referenced XObjects**: Content referenced as `Form XObject` but not actually rendered on any page. After signing, an incremental update adds the XObject to a page's content stream.

The key constraint: *all of the hidden content must be present in the file at the time of signing*, so that it falls within the signed byte range. The attacker only changes the *visibility* via an incremental update after signing — which some viewers allowed without warning.

26.1.6.3 Shadow Attack Variant 2: Replace

The attacker creates a PDF where the visible content can be *replaced* by different visible content without modifying any objects in the signed byte range. This exploits **interactive form features**:

- A text field with a default value ("Contract for $1,000") and a different actual value ("Contract for $1,000,000") — the default is displayed initially, but changing which value is rendered requires only an annotation flag change
- Font references that can be swapped — the same Unicode code points render as different glyphs depending on which font is active
- Images referenced by name from a page's Resources dictionary — changing which image a name resolves to requires only modifying the Resources dictionary

The Replace variant is particularly dangerous because it exploits features that are *designed* to change document appearance (form filling, annotation updates) — the viewer must distinguish between a legitimate form fill and an attacker swapping content.

26.1.6.4 Shadow Attack Variant 3: Hide-and-Replace

The most powerful variant combines both techniques. The attacker creates a PDF that:

1. Contains the content the signer will see (layer A)
2. Contains the content the recipient will see (layer B), hidden behind layer A
3. Is structured so that a single incremental update (changing a few bytes in an annotation dictionary or OCG configuration) switches from layer A to layer B

After signing, the attacker appends the minimal incremental update needed to switch layers. If the viewer allows annotation or OCG changes after signing (as many did, because these are often considered "permissible" changes), the attack succeeds.

26.1.6.5 Demonstration: Shadow Attack (Hide Variant) Preparation

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.PDPage;
import org.apache.pdfbox.pdmodel.PDPageContentStream;
import org.apache.pdfbox.pdmodel.common.PDRectangle;
import org.apache.pdfbox.pdmodel.font.PDType1Font;
```

```
import org.apache.pdfbox.pdmodel.font.Standard14Fonts;

import java.io.ByteArrayOutputStream;
import java.io.File;
import java.nio.file.Files;

/**
 * Demonstrates the preparation phase of a Shadow Attack (Hide variant).
 * The attacker creates a PDF with hidden text behind a white overlay.
 *
 * BEFORE signing: Signer sees "Payment: $1,000"
 * AFTER signing + reveal: Recipient sees "Payment: $1,000,000"
 *
 * The hidden text ("Payment: $1,000,000") is present in the signed bytes.
 * The overlay (white rectangle) hides it. An incremental update that removes
 * the overlay reveals the hidden text — and the signature remains valid.
 */
public class ShadowAttackHideDemo {

    public static void main(String[] args) throws Exception {
        PDDocument doc = new PDDocument();
        PDPage page = new PDPage(PDRectangle.A4);
        doc.addPage(page);

        PDType1Font font = new PDType1Font(
                Standard14Fonts.FontName.HELVETICA);
        PDType1Font fontBold = new PDType1Font(
                Standard14Fonts.FontName.HELVETICA_BOLD);

        PDPageContentStream cs = new PDPageContentStream(doc, page);

        // Layer 1: The hidden content (written FIRST, so it's behind)
        // This text will be present in the signed bytes
        cs.beginText();
        cs.setFont(fontBold, 16);
        cs.newLineAtOffset(50, 700);
        cs.showText("Payment: $1,000,000");
        cs.endText();

        // Layer 2: The overlay that hides it
        // A white rectangle drawn ON TOP of the hidden text
        cs.setNonStrokingColor(1.0f, 1.0f, 1.0f); // White
        cs.addRect(45, 695, 250, 25); // Covers the text above
        cs.fill();

        // Layer 3: The visible content (what the signer sees)
        cs.setNonStrokingColor(0.0f, 0.0f, 0.0f); // Black
        cs.beginText();
        cs.setFont(fontBold, 16);
        cs.newLineAtOffset(50, 700);
        cs.showText("Payment: $1,000");
        cs.endText();

        // Legitimate-looking document body
        cs.beginText();
```

```
        cs.setFont(font, 12);
        cs.newLineAtOffset(50, 650);
        cs.showText("This agreement confirms the payment amount above.");
        cs.endText();

        cs.close();

        // Save — this PDF is now ready to be sent to the signer
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        doc.save(baos);
        doc.close();

        Files.write(
                new File("shadow-prepared.pdf").toPath(), baos.toByteArray());

        // The signer opens this PDF, sees "Payment: $1,000", and signs it.
        // The signed byte range includes ALL content: both the hidden
        // "$1,000,000" text and the white rectangle that covers it.
        //
        // After signing, the attacker creates an incremental update that
        // removes or makes transparent the white rectangle, revealing
        // "Payment: $1,000,000" — and the signature validates.
    }
}
```

- The hidden text is written to the content stream first. In PDF rendering order, later operators paint *over* earlier ones on the same area
- A white filled rectangle covers the hidden text completely. The signer sees only white space at this position
- The visible text is drawn on top of the white rectangle. This is what the signer reads when reviewing the document before signing

Shadow Attacks are fundamentally harder to prevent than USF, ISA, or SWA because the malicious content is *part of the signed data.* The signature is cryptographically valid over the exact bytes the signer intended to sign. The defense must go beyond byte-level verification into *semantic* analysis: does the document contain hidden content? Are there overlapping objects? Are there unused Form XObjects? Are there Optional Content Groups with suspicious visibility settings? This is an open research problem with no complete solution.

26.1.6.6 The Scope of Shadow Attack Vulnerability

The Bochum team tested 29 PDF viewers against the Shadow Attack variants. The results were comprehensive:

Table 26.1: Shadow Attack test results across PDF viewers (NDSS 2021)

Viewer	Hide	Replace	Hide-and-Replace
Adobe Acrobat Reader DC	Vulnerable	Vulnerable	Vulnerable
Foxit Reader	Vulnerable	Vulnerable	Vulnerable

Viewer	Hide	Replace	Hide-and-Replace
LibreOffice Draw	Vulnerable	Partially vulnerable	Vulnerable
Nitro Pro	Vulnerable	Vulnerable	Vulnerable
PDF-XChange	Vulnerable	Not vulnerable	Vulnerable
Okular	Vulnerable	Vulnerable	Vulnerable
Evince	Vulnerable	Vulnerable	Vulnerable
Chrome PDF Viewer	No signature validation	—	—
macOS Preview	Vulnerable	Vulnerable	Vulnerable

This table is a representative subset. The full results covering 29 viewers are available in the NDSS 2021 paper and at pdf-insecurity.org. Chrome's built-in PDF viewer does not validate signatures at all (it displays them but does not indicate validity), so Shadow Attacks are not applicable in the traditional sense — but the user also receives no warning that the document has been signed and potentially tampered with.

26.1.7 Pre-Bochum Vulnerabilities: Earlier PDF Signature Issues

The Bochum research was the most systematic analysis of PDF signature security, but it was not the first time vulnerabilities were discovered. Several earlier issues laid the groundwork — and reveal that the *category* of bug (failing to verify what you claim to verify) has been a persistent pattern in PDF signature implementations.

26.1.7.1 CVE-2018-16042 — Adobe Acrobat Signature Bypass (2018)

In 2018, Adobe patched **CVE-2018-16042** in Acrobat DC and Acrobat Reader DC (versions before 2019.008.20081). The vulnerability allowed a crafted PDF to display a valid signature indicator even when the document had been modified.

The technical details were not fully disclosed (Adobe's advisory was characteristically brief), but the vulnerability class — signature validation bypass via document manipulation — is the same category as the ISA attacks that Bochum would systematize a year later. This CVE was one of several signals that the PDF signature validation code in major viewers had not been subject to rigorous adversarial testing.

Affected versions: Acrobat DC/Reader DC before 2019.008.20081
Impact: Signature validation bypass — modified documents displayed as validly signed
Fix: Updated signature validation logic in the January 2019 security update

26.1.7.2 CVE-2018-4993 — Adobe Acrobat NTLM Credential Theft (2018)

While not strictly a signature bypass, **CVE-2018-4993** demonstrated a related risk: a crafted PDF could trigger NTLM authentication requests to an attacker-controlled server when

opened in Adobe Acrobat. Combined with a forged signature (via any of the bypass techniques), this creates a compound attack: the victim opens a ”validly signed” PDF, and their Windows credentials are silently exfiltrated.

Affected versions: Acrobat DC/Reader DC before 2018.011.20040
Impact: NTLM hash theft via SMB relay when opening a crafted PDF
Fix: Restricted external resource loading in the May 2018 security update

26.1.7.3 SHAttered and the SHA-1 Deprecation (2017)

In February 2017, researchers from Google and CWI Amsterdam published **SHAttered** — the first practical collision attack against SHA-1. They produced two different PDF documents with the same SHA-1 hash, proving that SHA-1 could no longer be relied upon for collision resistance.

For PDF signatures, this had immediate implications. The `adbe.pkcs7.sha1` SubFilter — which embeds a SHA-1 hash of the entire document as the CMS content data — was now theoretically vulnerable to a collision attack: an attacker could prepare two documents with the same SHA-1 hash, have the victim sign one, and then substitute the other.

The practical impact was limited (the collision required approximately 2^{63} computations and the two colliding documents had to be carefully constructed), but the theoretical break was sufficient to trigger industry-wide deprecation:

- ISO 32000-2:2020 deprecated `adbe.pkcs7.sha1` entirely
- Adobe Acrobat began showing warnings for SHA-1 signed documents
- The CA/Browser Forum required SHA-256 as the minimum for all new certificates
- PAdES profiles already mandated SHA-256 as the minimum digest algorithm

Table 26.2: SHA-1 timeline for PDF signatures

Year	Event
2005	Theoretical weakness in SHA-1 published (Wang et al.)
2011	NIST deprecated SHA-1 for digital signatures in federal use
2013	Microsoft announced SHA-1 certificate sunset for 2016
2017	SHAttered: first practical SHA-1 collision (Google/CWI). Two PDFs with identical SHA-1 hashes publicly demonstrated
2017	ISO 32000-2 deprecated `adbe.pkcs7.sha1` SubFilter
2020	SHA-1 collisions reduced to 2^{63} operations (Leurent and Peyrin, ”SHA-1 is a Shambles”), making chosen-prefix collisions feasible

If your code still generates signatures using SHA-1 (either via `adbe.pkcs7.sha1` or using SHA-1 as the `digestAlgorithm` in the CMS `SignerInfo`), stop immediately. SHA-1 is broken. Use SHA-256 or SHA-384. There is no legitimate reason to use SHA-1 for any new signature created after 2017.

26.1.7.4 The Comodo Registration Authority Breach (2011)

In March 2011 — the same month as DigiNotar, though discovered slightly earlier — an attacker compromised a **registration authority (RA)** affiliated with Comodo, one of the world's largest certificate authorities at the time.

The attacker used the compromised RA account to issue nine fraudulent SSL certificates for high-profile domains: `login.live.com`, `mail.google.com`, `www.google.com`, `login.yahoo.com`, `login.skype.com`, `addons.mozilla.org`, and three others. The certificates were issued by Comodo's legitimate intermediate CA, meaning they chained correctly to a trusted root.

The breach was discovered quickly because the attacker tested one of the certificates against a live server, and certificate pinning in Google Chrome detected the mismatch. Without Chrome's certificate pinning, the fraudulent certificates could have been used for undetectable man-in-the-middle attacks.

For PDF signatures: The Comodo breach demonstrated that even without compromising a root CA's private key, an attacker could obtain legitimate-looking certificates through a *registration authority* compromise — a weaker link in the chain. A document signed with one of these fraudulent certificates would have validated correctly in any PDF viewer that trusted Comodo's root. The signer would appear to be "Google" or "Microsoft" or "Yahoo" — entities that obviously did not sign the document.

The incident led to:

- Mandatory two-factor authentication for all RA accounts at major CAs
- Domain control validation (DCV) requirements tightened by the CA/Browser Forum
- Increased scrutiny of the RA model, eventually leading several CAs to eliminate external RAs entirely

26.1.7.5 The Symantec Distrust (2017–2018)

Between 2015 and 2017, Google's Chrome security team and the CA/Browser Forum discovered that **Symantec** (which at that time owned the Thawte, GeoTrust, VeriSign, and RapidSSL certificate brands) had mis-issued thousands of certificates — including certificates for domains whose owners had not authorized them.

The problems were systemic:

- Symantec had delegated certificate issuance to third-party partners (RAs) without adequate oversight
- Test certificates for real domains were issued to unauthorized parties
- Audit logs were incomplete or missing
- Symantec initially denied the scope of the problem, then acknowledged it in stages

Google announced a graduated distrust of all Symantec-issued certificates:

- Chrome 66 (April 2018): distrusted certificates issued before June 1, 2016
- Chrome 70 (October 2018): distrusted all remaining Symantec certificates

Mozilla Firefox and Apple Safari followed with their own distrust timelines. Symantec sold its CA business to DigiCert, which re-issued certificates for affected customers under DigiCert's infrastructure.

For PDF signatures: Symantec was a major CA in the AATL (Adobe Approved Trust List). Documents signed with Symantec-issued certificates before the distrust cutoff dates would validate correctly when signed, but might show warnings or failures in PDF viewers that updated their trust stores to reflect the browser distrust. This created a time-dependent validation problem: a PDF signed with a Symantec certificate in 2015 was ”valid” when signed, ”suspect” in 2018, and — depending on the viewer’s trust configuration — might show as ”invalid” or ”valid with warnings” today.

This is exactly the problem that PAdES-B-LTA (long-term validation with archival timestamps) was designed to solve: by embedding the full certificate chain and revocation data at the time of signing, and then protecting it with periodic archival timestamps, the signature’s validity at the time of signing can be verified even after the issuing CA is no longer trusted.

26.1.8 Exploits in the Wild

A natural question: were any of these attacks exploited before or after the Bochum disclosures?

The honest answer is: **we don’t know for certain**.

PDF signatures are used in financial contracts, government documents, legal filings, tax returns, insurance claims, and medical records. If an attacker successfully forged a signature on a financial contract, the victim might not discover it until a dispute arises — and even then, the forensic analysis might not identify the specific attack technique.

What we do know:

26.1.8.1 Indicators of Real-World Exploitation

1. **The ”1 Trillion Dollar Refund” title was not hyperbole.** The paper’s title referenced the fact that tax authorities in multiple countries accept signed PDFs as official submissions. The German tax system (ELSTER), for example, processes millions of electronically signed documents per year. If USF or ISA could forge signatures on tax submissions, the financial exposure was enormous. The Bochum team coordinated with CERT-Bund (Germany’s national CERT) before publication.
2. **Adobe’s response was unusually fast.** Adobe issued CVE-2020-9592 and CVE-2020-9596 and released patches within weeks of the responsible disclosure. For a company that typically operates on monthly security update cycles, the accelerated timeline suggests Adobe assessed the risk as immediate and severe — consistent with either active exploitation or intelligence indicating exploitation was imminent.
3. **The EU Digital Signature Service (DSS) was vulnerable.** The EU’s own reference implementation for signature validation — the tool member states use to validate eIDAS-compliant signatures — was vulnerable to ISA attacks. If the EU’s own validator could be fooled, signatures submitted to government services across 27 member states could potentially have been forged without detection.
4. **Commercial signing services had no detection capability.** Before the Bochum research, no commercial PDF signing or validation service tested for these attack classes. There were no automated tools to detect ISA, SWA, or Shadow Attack patterns in submitted PDFs. Any exploitation that occurred before 2019 would have been invisible to the existing security infrastructure.

26.1.8.2 Post-Disclosure Exploitation Risk

After the papers were published and patches released, the risk shifted:

- **Unpatched viewers remain vulnerable.** Not all users update their PDF viewers. Enterprise environments, government agencies with slow update cycles, and users in regions with limited internet access may still be running vulnerable versions of Acrobat, Foxit, or other viewers.
- **The attack techniques are public.** The Bochum team published detailed descriptions and proof-of-concept PDFs. Any attacker with basic PDF knowledge can reproduce the attacks.
- **Shadow Attacks remain partially unresolved.** While most viewers now detect the Hide and Replace variants, the fundamental problem — that PDF semantics allow content to be visually hidden — is inherent to the format. New variants of Shadow Attacks are likely still discoverable.

26.1.9 The Patch Response: What Changed

26.1.9.1 Adobe's Response

Adobe's patches for CVE-2020-9592 and CVE-2020-9596 (released in the May 2020 Acrobat security update, version 2020.006.20042) implemented:

1. **Cryptographic signature verification** — closing the USF vulnerability by ensuring the CMS signature value is always verified against the document hash
2. **Byte range coverage validation** — checking that the `/ByteRange` covers the entire file from offset 0 to the file's end (minus the `/Contents` gap), closing the SWA vulnerability
3. **Incremental update analysis** — warning users when an incremental update modifies content beyond what the signature's permission level allows, closing the ISA vulnerability
4. **Shadow content detection** — partial detection of Hidden content patterns (overlapping objects, OCGs with suspicious visibility settings)

Acrobat's signature panel now displays more granular information:

- "Signed and all signatures are valid" — no modifications detected
- "Signed but the document has been modified since signing" — incremental updates detected, with details about what changed
- "At least one signature has problems" — cryptographic or structural verification failed
- "The document has been altered or corrupted since the signature was applied" — clear indication of tampering

26.1.9.2 Foxit's Response

Foxit Reader patched its signature validation in version 10.1 (released October 2020), implementing similar checks: CMS verification, byte range validation, and incremental update analysis. Foxit was notably slower than Adobe in responding — roughly five months after Adobe's patch.

26.1.9.3 LibreOffice's Response

LibreOffice (which uses the NSS library for cryptographic operations) patched its PDF signature validation in version 7.0. The fix was more complex because LibreOffice's PDF import/export pipeline differs significantly from dedicated PDF viewers — it converts PDF to an internal format for editing, which complicates byte-level signature verification.

26.1.9.4 The Broader Industry

By mid-2021, most actively maintained PDF viewers had released patches addressing the three original attack classes (USF, ISA, SWA). Shadow Attack mitigations were slower to arrive and remain incomplete in some viewers as of 2026.

The Bochum research also influenced the standards bodies:

- **ISO TC 171** (the committee responsible for ISO 32000-2:2020) incorporated the attack findings into the errata and clarifications for the second edition of PDF 2.0
- **ETSI** updated the PAdES standard (EN 319 142-1 V1.2.1, published 2022) to include explicit requirements for byte range coverage verification and incremental update validation
- **The EU Digital Signature Service (DSS)** — the open-source reference validator maintained by the European Commission — was patched and now includes specific test cases for each attack class

26.1.10 Building a Robust Validator: Lessons from the Attacks

Every attack in this chapter exploits a specific gap between what a viewer *should* check and what it *actually* checks. Here is the complete checklist, informed by every vulnerability discussed above.

Table 26.3: PDF Signature Validation Checklist (post-Bochum)

#	Check	Defends Against
1	Parse the CMS `SignedData` structure and verify it is well-formed (correct OIDs, consistent version numbers, all required fields present)	USF Variant 2
2	Verify the `SignerInfo.signature` value cryptographically against the signer's public key and the computed digest	USF Variant 1
3	Verify the message digest in the signed attributes matches the hash of the byte range content	USF Variant 3, SWA
4	Verify `/ByteRange` starts at offset 0	SWA
5	Verify `/ByteRange` gap contains the `/Contents` hex string	SWA

#	Check	Defends Against
6	Verify `/ByteRange` covers the entire file (for the signed revision) — no unaccounted bytes before, between, or after the ranges	SWA, ISA
7	If the file extends beyond the signed byte range, parse the incremental update and determine what changed	ISA
8	Verify that any changes in the incremental update are permissible under the signature's DocMDP permission level (P=1: no changes; P=2: form fills + signatures; P=3: form fills + signatures + annotations)	ISA Variant 3
9	Verify the certificate chain from the signer's certificate to a trusted root, checking revocation (OCSP/CRL) and validity dates	Certificate forgery, CA breaches
10	If a timestamp is present, verify the TSA certificate chain independently and check that the timestamp covers the correct data	Backdated signatures
11	Check for hidden content patterns: overlapping objects, Optional Content Groups with invisible layers, unreferenced Form XObjects, form fields with discrepant default/actual values	Shadow Attacks (Hide, Replace)
12	Ensure the digest algorithm is SHA-256 or stronger — reject SHA-1	SHAttered, hash collision attacks

Checks 1–6 are non-negotiable — any validator that skips them is vulnerable to at least one of USF, ISA, or SWA. Checks 7–10 are necessary for real-world security but require more complex implementation. Checks 11–12 are defense-in-depth against attacks that remain partially unresolved.

No checklist is complete. The Bochum team found these attack classes by thinking adversarially about the gap between what the spec *requires* and what implementations *do*. There will be more attacks. The defense is not just implementing these twelve checks — it is building a culture of adversarial testing into your validation code.

26.1.11 A Note on Responsible Disclosure

The Bochum team followed responsible disclosure for both papers. They notified affected vendors months before publication and provided proof-of-concept PDFs for testing.

For the CCS 2019 paper, the disclosure timeline was approximately:

- **March–April 2019**: Initial vendor notifications
- **May–August 2019**: Back-and-forth with vendors; some requested extended timelines; some did not respond

- **November 2019**: Paper presented at CCS; public disclosure
- **May 2020**: Adobe patches released (CVE-2020-9592, CVE-2020-9596)
- **October 2020**: Foxit patches released

The six-month gap between public disclosure and Adobe's patch — and the eleven-month gap before Foxit's patch — is notable. The Bochum team publicly criticized the slow response, particularly from viewers that took more than six months to patch after detailed proof-of-concept material was shared privately.

For the NDSS 2021 paper (Shadow Attacks), the disclosure process was similar but vendors responded faster, having already been through the CCS 2019 cycle.

The research materials — including proof-of-concept PDFs, the vulnerability matrix, and detailed attack descriptions — remain available at **pdf-insecurity.org**. The Hackmanit GmbH **PDF Detector** tool (available as a web service and standalone library) can check PDFs for known attack patterns.

26.1.12 Summary

PDF signature security before 2019 was, to put it bluntly, an illusion. The cryptographic primitives were sound — RSA, ECDSA, SHA-256, CMS are all well-designed and well-analyzed. The failure was entirely in how PDF viewers *used* those primitives. They verified certificates meticulously and signatures not at all.

The Bochum research (CCS 2019, NDSS 2021) exposed four attack classes:

- **USF**: The viewer doesn't check the cryptographic signature value — forge anything
- **ISA**: The viewer doesn't check for modifications after signing — append anything
- **SWA**: The viewer doesn't validate the byte range — redirect it to anything
- **Shadow Attacks**: The viewer doesn't analyze what the signed content *means* — hide anything

Every major PDF viewer was vulnerable. The patches took months to years. Some Shadow Attack variants remain partially unresolved.

Earlier events — the DigiNotar breach (2011), the Comodo RA compromise (2011), the Symantec distrust (2017–2018), and the SHAttered SHA-1 collision (2017) — had already weakened the trust infrastructure underneath PDF signatures. The Bochum attacks showed that even if you trusted the infrastructure, the validators themselves couldn't be trusted.

The lesson for implementers is clear: **verify everything, trust nothing, and test adversarially**. The validation checklist in this chapter is the minimum. The next attack class is already being researched.

Chapter 27

Certificate Authority Breaches

Every PDF digital signature carries an implicit assertion: ”I trust the certificate authority that issued this certificate.” That assertion is not a mathematical fact — it is an organizational, operational, and political judgment. And like all judgments, it can turn out to be catastrophically wrong.

This chapter chronicles the most consequential certificate authority failures of the past two decades. Not because they are historical curiosities, but because each one contains lessons that directly affect how you should build, validate, and maintain PDF digital signatures today. A CA breach does not just compromise web traffic. It retroactively undermines every document signed with certificates issued by that CA — contracts, tax filings, court submissions, medical records. The signatures do not become invalid in a cryptographic sense; the math still checks out. They become *untrustworthy*, which in the real world is worse.

27.1 Why CA Breaches Matter for PDF Signatures

When you sign a PDF, the recipient validates your signature by chaining your signing certificate back to a trusted root. That chain passes through one or more intermediate CAs, each vouching for the next, until it reaches a root certificate that the validator explicitly trusts — either because it is in Adobe’s AATL, in the EU Trusted List, in the operating system’s trust store, or in a custom trust configuration.

The entire chain is only as strong as its weakest link. If any CA in the chain is compromised, an attacker can issue fraudulent certificates — certificates that look legitimate but are controlled by the attacker. Documents signed with those fraudulent certificates will pass validation in any viewer that trusts the compromised CA.

Worse, when a compromised CA is discovered and revoked, the collateral damage extends to *every* legitimate certificate that CA ever issued. If you signed a contract last year with a certificate from CA-X, and CA-X is discovered to be compromised this year, your signature’s trustworthiness is now in question — not because your key was compromised, but because the entity vouching for your identity can no longer be trusted.

This is the fundamental fragility of the PKI trust model: trust is transitive, delegated, and revocable. You cannot sign a document and walk away confident that the signature will still be trusted in five years, because you cannot predict whether the CA that issued your certificate will still be trusted in five years.

The only mitigation for CA compromise is **embedding revocation evidence at signing time**. A PAdES-B-LTA signature includes OCSP responses and CRLs captured when the signature was created. If the CA is later revoked, the embedded evidence proves that the certificate was valid *at the time of signing* — before the breach was discovered. Without LTV data, a

validator checking the signature after the CA revocation has no way to distinguish between a legitimate pre-breach signature and a fraudulent one.

27.2 DigiNotar (2011): The Breach That Broke a Country

27.2.1 Background: A Small CA With a Big Responsibility

DigiNotar was a Dutch certificate authority founded in 1998, headquartered in Beverwijk, a small town near Amsterdam. The company was originally an offshoot of the Dutch notarial profession — the name is a portmanteau of ”Digital Notary” — and it carved out a niche issuing certificates for Dutch government services, notaries, and businesses.

In 2010, VASCO Data Security International (now OneSpan) acquired DigiNotar for approximately 10 million euros. At the time, DigiNotar was one of the root CAs trusted by every major browser and operating system. Its certificates were embedded in the trust stores of Windows, macOS, Firefox, Chrome, and by extension, in the trust stores used by PDF viewers.

DigiNotar also operated **PKIoverheid** (PKI for Government) — the Dutch government’s public key infrastructure. PKIoverheid certificates were used across Dutch government digital services, including the DigiD electronic identity system, tax filing, and — critically for our purposes — official government document signing.

27.2.2 The Attack

In June 2011, an attacker gained access to DigiNotar’s certificate issuance infrastructure. The attack was later attributed to a state-sponsored actor operating from Iran, though the attacker also posted messages claiming to be a lone Iranian hacker named ”Comodohacker” — the same handle used in the Comodo breach earlier that year.

The attacker exploited multiple vulnerabilities in DigiNotar’s infrastructure:

- The CA’s servers ran outdated, unpatched software
- Network segmentation between the public-facing servers and the certificate issuance systems was inadequate
- The HSMs (Hardware Security Modules) that should have required physical authorization for certificate issuance were configured to allow automated remote issuance
- Audit logs were incomplete and inconsistent — DigiNotar did not have reliable records of which certificates had been issued

Once inside, the attacker issued over **531 fraudulent certificates** for high-value domains. The confirmed targets included:

Table 27.1: Known fraudulently issued DigiNotar certificates (partial list)

Domain	Significance
google.com, www.google.com	Gmail, Google Search — primary communication tool for millions
login.yahoo.com	Yahoo Mail
login.live.com	Microsoft Hotmail/Outlook

Domain	Significance
addons.mozilla.org	Firefox extension repository
login.skype.com	Skype communications
global.sts.microsoft.com	Microsoft federation services
www.sis.gov.uk	UK Secret Intelligence Service (MI6)
www.cia.gov	US Central Intelligence Agency
mossad.gov.il	Israeli intelligence service
azadegi.com	Iranian opposition news website
Several wildcard certs	Covering entire TLDs and subdomains

27.2.3 The Use: Surveillance of Iranian Citizens

The fraudulent certificates were not issued for financial fraud. They were used for **mass surveillance**.

In July and August 2011, the fraudulent `google.com` certificate was deployed in man-in-the-middle (MITM) attacks targeting Iranian internet users. The Iranian government (or an actor with access to Iranian internet infrastructure) intercepted HTTPS connections to Google services — primarily Gmail — by presenting the fraudulent DigiNotar certificate. Because DigiNotar was a trusted root CA, browsers accepted the certificate without warning.

The victims were Iranian citizens communicating via Gmail during a period of significant political tension. The Arab Spring protests had swept through Tunisia, Egypt, Libya, and Syria. Iran's Green Movement, which had erupted after the disputed 2009 presidential election, was still active. The surveillance targeted people who thought their communications were private.

The scale of the interception was significant. Fox-IT, the Dutch forensics firm that later investigated the breach, estimated that approximately **300,000 unique IP addresses** — almost all Iranian — had been subjected to MITM attacks using the fraudulent Google certificate.

27.2.4 Discovery: How Certificate Pinning Saved the Day

The attack was discovered on August 28, 2011, by an Iranian user of Google Chrome.

Chrome has a feature called **certificate pinning** (also known as HPKP, HTTP Public Key Pinning, or in Chrome's case, a hardcoded pin set). For a small number of high-value domains — including google.com — Chrome maintains a built-in list of expected certificate public keys. If the presented certificate's public key does not match any of the pinned keys, Chrome refuses the connection and displays a prominent error, even if the certificate otherwise validates correctly against the trust store.

When the Iranian Chrome user attempted to connect to Google, Chrome detected that the presented certificate (signed by DigiNotar) did not match Google's pinned public keys. Chrome displayed an error. The user reported the error to Google.

Certificate pinning was not designed as a comprehensive defense against CA compromise. It only worked for the handful of domains that Google had pre-configured pins for. If the attacker had only issued certificates for domains that Chrome did not pin — the intelligence agency websites, the Iranian opposition news site, the Yahoo and Skype domains — the attack might have continued undetected for months.

The lesson is humbling: the discovery of one of the most significant CA compromises in internet history was essentially an accident — it depended on one user, in one country, using one specific browser, connecting to one specific domain that happened to have certificate pinning enabled.

27.2.5 The Response: Death of a CA

Once Google confirmed the fraudulent certificate, events moved rapidly:

- **August 29, 2011**: Google published a Chrome update blocking the specific DigiNotar intermediate certificate used to sign the fraudulent Google certificate
- **August 30, 2011**: Mozilla pushed a Firefox update removing DigiNotar's root from the trust store entirely
- **September 2, 2011**: The Dutch government's GovCERT (Government Computer Emergency Response Team) issued an advisory acknowledging the breach
- **September 3, 2011**: Google, Microsoft, and Apple all pushed updates removing DigiNotar from their trust stores
- **September 5, 2011**: The Dutch government formally revoked DigiNotar's authorization to issue PKIoverheid certificates
- **September 20, 2011**: DigiNotar's parent company VASCO filed for DigiNotar's bankruptcy in Dutch court

The entire collapse — from discovery to bankruptcy — took less than four weeks.

Fox-IT's forensic investigation, published in their report "DigiNotar Certificate Authority breach — 'Operation Black Tulip'" (named with Dutch irony), found that the breach was far more extensive than initially reported. The attacker had been inside DigiNotar's systems since at least June 17, 2011, had compromised multiple CA servers, and had access to issuing infrastructure for both DigiNotar's commercial certificates and the PKIoverheid government certificates.

27.2.6 Impact on PDF Signatures

The immediate impact on PDF signatures was severe and irreversible.

Any PDF document signed with a DigiNotar-issued certificate became **unverifiable** the moment browsers and operating systems removed DigiNotar from their trust stores. Adobe Acrobat, which uses the Windows certificate store (on Windows) or its own AATL (on macOS), stopped trusting DigiNotar certificates. A signed PDF that showed a green checkmark on September 2 showed a warning on September 4.

The documents themselves had not changed. The signatures had not been tampered with. The signing keys had not been compromised (in most cases — the legitimate certificate holders were not the ones attacked). But the trust anchor for those certificates no longer existed.

For Dutch government agencies, this was particularly painful. PKIoverheid certificates were used to sign official government documents, notarial deeds, and legal filings. The Dutch government had to issue emergency replacement certificates through other CAs and re-sign critical documents — a process that took months.

This is the nightmare scenario for long-lived signed documents. A contract signed in 2010 with a DigiNotar certificate and stored in an archive had its trust chain severed in 2011. If the signature did not include LTV data (OCSP responses and CRLs embedded at signing time, as in PAdES-B-LTA), there was *no way* to prove after the fact that the certificate was valid when the signature was created. The document was, for all practical purposes, unsigned.

This is why PAdES-B-LTA exists. This is why you embed revocation data. This is not a theoretical concern — it happened.

27.2.7 Lessons from DigiNotar

The DigiNotar breach taught the PKI community several painful lessons:

1. **A single CA compromise can retroactively invalidate thousands of signatures.** Trust is not permanent. It is not even stable. It can be revoked at any time, with retrospective effect.
2. **CA security is only as strong as the weakest operational practice.** DigiNotar had HSMs, but they were misconfigured. They had audit logs, but they were incomplete. They had network segmentation, but it was insufficient. The technology existed to prevent the breach; the organization failed to deploy it correctly.
3. **Certificate pinning is a band-aid, not a solution.** It helped detect *this* breach, but it does not scale. You cannot pin every domain's certificate in every application.
4. **Government PKI is not immune.** DigiNotar operated critical Dutch government infrastructure. Government CAs are not inherently more trustworthy than commercial ones — they are subject to the same operational failures.
5. **Bankruptcy is the final revocation.** When a CA goes bankrupt, its certificates do not gracefully expire. They are abruptly distrusted. There is no migration path, no deprecation period, no gentle sunset. One day the certificates work; the next day they do not.

27.3 Comodo (2011): The Parallel Breach

27.3.1 The Attack

Just five months before DigiNotar's collapse, in March 2011, a similar attack targeted Comodo — at the time, one of the world's largest commercial CAs.

The attacker compromised **registration authority (RA) accounts** at Comodo's reseller partners — specifically, the Italian firm GlobalTrust.it and the Argentine firm InstantSSL.it. Registration authorities are entities authorized to perform identity verification on behalf of a CA; they can request certificate issuance without having direct access to the CA's root key.

Using the compromised RA credentials, the attacker obtained **nine fraudulent certificates** for high-value domains:

- login.live.com (Microsoft)

- mail.google.com (Google)
- www.google.com (Google)
- login.yahoo.com (Yahoo, three separate certificates)
- login.skype.com (Skype)
- addons.mozilla.org (Mozilla)
- An additional ”global trustee” certificate

The target list was remarkably similar to the DigiNotar targets — because the same attacker was behind both breaches.

27.3.2 The Response: How Comodo Got It Right

Comodo’s response was dramatically faster and more effective than DigiNotar’s would be five months later.

Comodo detected the fraudulent issuance **within hours** — their monitoring systems flagged the unusual certificate requests from the compromised RA accounts. The nine certificates were revoked within the same day. Comodo published a detailed incident report, contacted browser vendors, and cooperated fully with the investigation.

The key differences between Comodo’s response and DigiNotar’s later failure:

Table 27.2: Comodo vs DigiNotar incident response comparison

Factor	Comodo	DigiNotar
Detection time	Hours	Weeks (possibly months)
Number of fraudulent certs	9	531+
Disclosure	Immediate and transparent	Delayed and incomplete
Certificate revocation	Same day	Gradual, over weeks
Root trust impact	Root preserved (reseller revoked)	Entire root removed
Business outcome	Continued operating	Bankruptcy
CA audit logs	Complete and useful	Incomplete and unreliable

Comodo survived because it detected the breach quickly, responded transparently, and demonstrated that the compromise was limited to reseller accounts — not the root CA infrastructure. The attackers got nine certificates; at DigiNotar, they got the keys to the kingdom.

27.3.3 The Attacker’s Manifesto

The attacker posted a manifesto online claiming to be a 21-year-old Iranian, acting alone, motivated by anger at Western governments' treatment of Iran. The manifesto, written in broken English, contained enough technical detail about both the Comodo and DigiNotar breaches to be credible. Whether the attacker was truly a lone wolf or a front for state-sponsored activity remains debated. The operational sophistication of the MITM infrastructure deployed

against Iranian citizens (requiring cooperation with or access to Iranian ISP infrastructure) suggests state involvement, regardless of who wrote the code.

From a PDF signing perspective, the Comodo incident reinforces a critical point: **your signing certificate's trustworthiness depends not only on your CA, but on every reseller, registration authority, and partner in the CA's issuance chain**. When selecting a CA for document signing certificates, ask about their RA model. How many resellers can request certificate issuance? What monitoring exists for anomalous issuance patterns? How quickly can fraudulent certificates be detected and revoked?

27.4 Symantec Distrust (2017-2018): The Slow-Motion Collapse

27.4.1 Background: The World's Largest CA

If DigiNotar was a small CA that died quickly, Symantec was a giant that died slowly.

Through a series of acquisitions — **VeriSign** (2010), **Thawte** (via VeriSign), **GeoTrust** (via VeriSign), and **RapidSSL** (via GeoTrust) — Symantec had become the world's largest certificate authority by 2015. At its peak, Symantec and its subsidiary brands had issued approximately **30% of all TLS certificates on the public internet**.

That market share made Symantec's CA operation systemically important. It also made it a tempting target for scrutiny.

27.4.2 The Mis-Issuance Scandal

In September 2015, Google's Certificate Transparency (CT) monitoring detected that Symantec had issued several test certificates for google.com domains — without Google's authorization. Symantec claimed these were test certificates issued by employees during internal testing, and that they had been revoked quickly.

Google investigated further and discovered that the problem was much larger:

- Symantec had issued **at least 164 certificates** for domains without proper authorization
- Some were issued by Symantec employees, others by RA partners (including CrossCert, a Korean RA)
- The unauthorized issuance spanned multiple years
- Symantec's internal controls had failed to detect the mis-issuances
- When confronted, Symantec's initial disclosures underreported the scope of the problem

In March 2017, Google engineers Ryan Sleevi and Devon O'Brien published a detailed analysis on the Chromium bug tracker, proposing a phased distrust of all Symantec-issued certificates. The proposal was unprecedented: no one had ever suggested distrusting certificates from the world's largest CA.

27.4.3 The Distrust Timeline

What followed was a two-year negotiation between Google, Mozilla, Symantec, and the broader CA/Browser Forum. The timeline:

- **March 2017**: Google proposes phased Symantec distrust in Chrome

- **April 2017**: Symantec contests the proposal, arguing the issues were limited
- **July 2017**: Google, Mozilla, and Symantec reach a compromise plan: Symantec must transition to a new infrastructure operated by a different CA
- **August 2017**: DigiCert announces acquisition of Symantec's CA business for approximately $950 million
- **December 2017**: DigiCert completes the acquisition and begins re-issuing certificates under DigiCert roots
- **April 2018**: Chrome 66 distrusts Symantec certificates issued before June 1, 2016
- **October 2018**: Chrome 70 distrusts ALL remaining Symantec certificates
- **October 2018**: Firefox follows with similar distrust in Firefox 63

The distrust was progressive: older certificates were distrusted first, giving website operators time to obtain replacement certificates. But the endpoint was absolute: after October 2018, no Symantec-issued certificate was trusted by Chrome or Firefox.

27.4.4 Impact on PDF Signatures

The Symantec distrust had a less dramatic but more insidious impact on PDF signatures than the DigiNotar breach.

DigiNotar's collapse was sudden — trust was revoked in days, and the relatively small number of affected certificates limited the blast radius. Symantec's distrust was gradual, but the scale was enormous.

- **Thousands of document signing certificates** had been issued under Symantec, VeriSign, Thawte, and GeoTrust brands
- Many of these certificates were in Adobe's AATL (Adobe Approved Trust List)
- As Symantec certificates were distrusted, PDF signatures chaining to those certificates showed warnings in validators that followed Chrome/Firefox trust decisions
- Adobe's trust store (AATL) moved independently from browsers — Adobe did not automatically distrust Symantec certificates just because Chrome did
- This created a confusing situation where a signed PDF might validate in Acrobat (using AATL) but fail in a web-based validator (using the browser trust store)

The Symantec case illustrates an important distinction: **PDF trust stores and web browser trust stores are not the same thing**. Adobe Acrobat maintains its own trust list (AATL), which is updated on its own schedule, with its own criteria. A certificate can be distrusted by Chrome but still trusted by Acrobat — or vice versa. This is both a feature (PDF signatures are not automatically affected by browser trust decisions) and a bug (trust status becomes inconsistent across different validation contexts).

The transition to DigiCert mitigated some of the damage — DigiCert re-issued certificates under its own roots, and existing Symantec customers could obtain replacement certificates. But for documents already signed with Symantec-chain certificates, the question of long-term trust remained unresolved.

```
/**
 * Checks whether a signing certificate chains to a currently trusted root.
 * This is the kind of check you need to perform when validating old signatures
 * whose CAs may have been distrusted since signing time.
 */
public CertificateChainStatus validateChainTrust(
```

```
        X509Certificate signerCert,
        List<X509Certificate> chain,
        Set<TrustAnchor> currentTrustAnchors,
        Date signingTime) throws Exception {

    // Build the certificate path
    CertificateFactory cf = CertificateFactory.getInstance("X.509");
    CertPath certPath = cf.generateCertPath(chain);

    // Configure the path validator with the signing time
    PKIXParameters params = new PKIXParameters(currentTrustAnchors);
    params.setDate(signingTime);  // Validate as of signing time, not now
    params.setRevocationEnabled(false);  // We check revocation separately

    try {
        CertPathValidator validator = CertPathValidator.getInstance("PKIX");
        PKIXCertPathValidatorResult result =
                (PKIXCertPathValidatorResult) validator.validate(certPath, params);

        TrustAnchor anchor = result.getTrustAnchor();
        log.info("Certificate chain valid at signing time. Trust anchor: {}",
                anchor.getTrustedCert().getSubjectX500Principal());

        return CertificateChainStatus.builder()
                .valid(true)
                .trustAnchor(anchor.getTrustedCert().getSubjectX500Principal().getName())
                .validationTime(signingTime)
                .build();

    } catch (CertPathValidatorException e) {
        log.warn("Certificate chain validation failed at index {}: {}",
                e.getIndex(), e.getMessage());

        return CertificateChainStatus.builder()
                .valid(false)
                .failureReason(e.getMessage())
                .failedCertificateIndex(e.getIndex())
                .validationTime(signingTime)
                .build();
    }
}
```

27.5 Let's Encrypt: The Free CA's Growing Pains

27.5.1 The 2020 Mass Revocation

Let's Encrypt, the free, automated CA that has issued over three billion certificates since its launch in 2015, does not issue document signing certificates. Its certificates are exclusively for TLS (web server) use. So why does a Let's Encrypt incident appear in a book about PDF signatures?

Because the 2020 mass revocation event demonstrated something important about the fragility of CA infrastructure at scale — a fragility that applies equally to every CA in the PDF

signing ecosystem.

On March 3, 2020, Let's Encrypt announced that it had discovered a bug in its **CAA (Certificate Authority Authorization) checking code**. CAA is a DNS-based mechanism that allows domain owners to specify which CAs are authorized to issue certificates for their domain. Before issuing a certificate, a CA must check the domain's CAA records and refuse issuance if the domain specifies a different CA.

The bug: Let's Encrypt's validation system checked CAA records when a domain was first validated, but did not re-check them when issuing a certificate if the validation result was cached. This meant that if a domain's CAA record changed after the initial validation (for example, to explicitly prohibit Let's Encrypt from issuing certificates), Let's Encrypt would still issue the certificate based on the stale cached result.

This violated the CA/Browser Forum's Baseline Requirements, which mandate that CAA records be checked within 8 hours of certificate issuance.

27.5.2 The Chaos

Let's Encrypt identified approximately **3,048,289 certificates** that had been issued without a compliant CAA check at issuance time. The Baseline Requirements required these certificates to be revoked within 5 days.

Let's Encrypt gave certificate holders **24 hours** to renew their certificates before revocation. For the ACME-based automated renewal ecosystem that Let's Encrypt had built, this should have been manageable — the whole point of Let's Encrypt is automated certificate management.

In practice, it was chaos. Not every certificate holder had functional automated renewal. Not every server was properly configured for ACME. Some certificates were on devices that could not easily be updated. The announcement came on a Tuesday evening (US time), giving operators one business day to respond.

27.5.3 Relevance to PDF Signing

The Let's Encrypt incident did not directly affect any PDF signatures. But it demonstrated three principles that every PDF signing architect should internalize:

1. **Scale amplifies errors.** A single bug in a CA's issuance pipeline can affect millions of certificates. The larger the CA, the larger the blast radius. When choosing a CA for document signing, understand that the CA's operational quality affects every certificate they issue — including yours.
2. **Revocation at scale is operationally brutal.** Even with 24 hours notice and automated tooling, millions of certificates could not be renewed in time. Now imagine the same scenario with document signing certificates, where "renewal" means re-signing every document that was signed with the revoked certificate. There is no automated tool for that.
3. **Compliance requirements drive revocation timelines.** The Baseline Requirements gave Let's Encrypt no discretion — the certificates *had* to be revoked within 5 days, regardless of the impact. EU regulations impose similar requirements on Qualified Trust Service Providers. If a QTSP discovers a compliance issue, it may be required to revoke certificates on a timeline that does not allow for graceful migration.

If your QTSP discovers a compliance issue and is required to revoke your signing certificate, every PDF you signed with that certificate becomes suspect — unless you embedded LTV data at signing time. This is not a far-fetched scenario. It has happened to web certificates. It will happen to document signing certificates. Plan accordingly.

27.6 Certificate Transparency: The Response to CA Failures

27.6.1 What Certificate Transparency Is

Certificate Transparency (CT) was invented by Google engineers **Ben Laurie** and **Adam Langley** in 2012 (published as RFC 5280 — Internet X.509 PKI Certificate and CRL Profile companion document RFC 6962, and later updated in RFC 9162), directly in response to the DigiNotar and Comodo breaches.

The core insight: if CAs cannot be trusted to only issue authorized certificates, we need a public audit mechanism that makes *every* issued certificate visible to domain owners and the broader community.

CT works as follows:

1. When a CA issues a certificate, it submits the certificate to one or more **CT logs** — append-only, cryptographically verifiable public databases
2. The CT log returns a **Signed Certificate Timestamp (SCT)** — a promise that the certificate will be included in the log within a specified time (the ”Maximum Merge Delay,” typically 24 hours)
3. The SCT is embedded in the certificate (as an X.509 extension), in the TLS handshake (as a TLS extension), or in the OCSP response (as an OCSP extension)
4. Browsers verify that certificates come with valid SCTs — if a certificate has no SCT, Chrome treats it as untrusted (since April 2018)
5. Domain owners (and anyone else) can monitor CT logs for certificates issued for their domains — if an unauthorized certificate appears, they can demand revocation

27.6.2 How CT Detects Rogue Certificates

CT does not *prevent* a compromised CA from issuing rogue certificates. It makes rogue certificates *detectable*.

If the DigiNotar attack had happened in a CT world, the fraudulent `google.com` certificate would have appeared in public CT logs within 24 hours of issuance. Google’s CT monitoring infrastructure would have detected it immediately — not because a user in Iran happened to be running Chrome, but because Google continuously monitors every CT log for certificates mentioning google.com.

The detection would have been systematic, automatic, and fast.

27.6.3 CT and PDF: Limited but Growing

Certificate Transparency was designed for the web PKI ecosystem. Its adoption in the document signing world has been slower:

- **Document signing certificates are not required to have SCTs.** The CA/Browser Forum Baseline Requirements mandate CT for TLS certificates, but no equivalent mandate exists for document signing certificates.
- **Adobe's AATL does not require CT.** CAs in the AATL are not required to log their document signing certificates in CT logs.
- **The EU Trusted List does not reference CT.** The eIDAS regulatory framework does not mention Certificate Transparency.

However, CT is increasingly relevant for PDF signatures:

- Some CAs voluntarily log their document signing certificates in CT logs
- Organizations building their own signature validation can incorporate CT log monitoring as an additional trust signal
- The EUDI Wallet infrastructure (Chapter 27) may incorporate CT-like transparency mechanisms for wallet-issued certificates

```
/**
 * Fetches and embeds revocation data (OCSP and CRL) at signing time,
 * creating the foundation for long-term validation that survives CA revocation.
 * This is the essential defense against CA breach impact.
 */
public LtvData collectRevocationData(List<X509Certificate> chain)
        throws Exception {

    List<byte[]> ocspResponses = new ArrayList<>();
    List<byte[]> crls = new ArrayList<>();

    // For each certificate in the chain (except the root),
    // fetch OCSP response and/or CRL
    for (int i = 0; i < chain.size() - 1; i++) {
        X509Certificate cert = chain.get(i);
        X509Certificate issuer = chain.get(i + 1);

        // Try OCSP first (preferred — smaller, more specific)
        byte[] ocspResponse = fetchOcspResponse(cert, issuer);
        if (ocspResponse != null) {
            log.info("OCSP response obtained for: {}",
                    cert.getSubjectX500Principal());
            ocspResponses.add(ocspResponse);
        }

        // Also fetch CRL as fallback
        byte[] crl = fetchCrl(cert);
        if (crl != null) {
            log.info("CRL obtained for: {} ({} bytes)",
                    cert.getSubjectX500Principal(), crl.length);
            crls.add(crl);
        }

        if (ocspResponse == null && crl == null) {
            log.warn("No revocation data available for: {}. " +
                    "LTV validation may fail for this certificate.",
                    cert.getSubjectX500Principal());
        }
```

```
    }

    return LtvData.builder()
            .certificates(chain.stream()
                    .map(c -> {
                        try { return c.getEncoded(); }
                        catch (Exception e) { throw new RuntimeException(e); }
                    })
                    .collect(Collectors.toList()))
            .ocspResponses(ocspResponses)
            .crls(crls)
            .collectionTime(new Date())
            .build();
}
```

27.7 What These Incidents Taught Us

27.7.1 Trust Is Fragile and Revocable

The PKI model assumes that root CAs are trustworthy. DigiNotar proved that assumption can fail catastrophically. Symantec proved it can fail gradually. Comodo proved it can fail and be recovered from — but only if the CA responds competently.

For PDF signature architects, the lesson is: **design for CA failure**. Assume that any CA in your signing certificate's chain might be distrusted at some point in the future. Your architecture should handle that scenario gracefully.

27.7.2 Long-Term Validity Requires Embedded Evidence

The single most important defense against CA breach impact is **embedding revocation data at signing time**.

A PAdES-B-LTA signature includes:

- The complete certificate chain (signer, intermediates, root)
- OCSP responses for every certificate in the chain, obtained at signing time
- CRLs as fallback revocation evidence
- A signature timestamp proving when the signature was created
- A document timestamp proving when the validation data was embedded

With all this data embedded, a validator can determine that the signing certificate was valid *at the time of signing* — even if the CA is later revoked. The embedded OCSP response from 2024 proves the certificate was not revoked in 2024, regardless of what happens to the CA in 2025.

Without this data, the validator is left with an impossible question: was this certificate valid when the signature was created? If the CA is gone, there is no one to ask.

27.7.3 You Cannot Predict CA Longevity

DigiNotar: bankrupt in 4 weeks. Symantec CA: sold and sunset in 18 months. Let's Encrypt: a compliance bug affecting 3 million certificates.

When you select a CA for your document signing certificates, you are making a bet on that CA's continued trustworthiness. History suggests this bet is riskier than most organizations realize.

Practical mitigations:

1. **Use PAdES-B-LTA for anything that matters.** The 5 seconds of extra processing time to fetch and embed revocation data is negligible compared to the cost of re-signing thousands of documents after a CA revocation.
2. **Have a certificate migration plan.** Know how you will transition to a new CA if your current CA is compromised, acquired, or distrusted. This includes having backup certificates from a second CA.
3. **Monitor your CA's compliance status.** Subscribe to CA/Browser Forum mailing lists, Google's security blog, and Mozilla's CA program communications. You want to know about problems with your CA before your customers do.
4. **Re-timestamp archived documents periodically.** A PAdES-B-LTA archival timestamp can be refreshed by adding a new document timestamp — extending the signature's verifiability beyond the original TSA's certificate lifetime.

27.8 Timeline: Major CA Incidents (2001-2024)

Table 27.3: Major CA security incidents and their impact on digital signatures

Year	Incident	Impact and Outcome
2001	VeriSign issues fraudulent Microsoft code-signing certs	Two certificates issued to an impersonator claiming to be Microsoft. Revoked quickly, but demonstrated that even the largest CA could be socially engineered. Led to improved identity verification procedures.
2008	Debian weak key vulnerability (CVE-2008-0166)	A Debian-specific OpenSSL bug caused all keys generated on Debian/Ubuntu systems for 18 months to be predictable. Affected SSL certificates, SSH keys, and potentially signing keys. Not a CA breach per se, but CAs had to revoke certificates whose keys were generated on affected systems.
2011 (Mar)	Comodo reseller compromise	9 fraudulent certificates issued via compromised RA accounts. Revoked within hours. CA survived. Attacker claimed Iranian motivation.
2011 (Jul-Sep)	DigiNotar breach	531+ fraudulent certificates. Used for mass MITM surveillance of Iranian citizens. CA bankrupt within 4 weeks. Dutch government PKI severely disrupted. All DigiNotar-chained PDF signatures became unverifiable.

Year	Incident	Impact and Outcome
2012	Trustwave subordinate CA for DLP	Trustwave issued a subordinate CA certificate to a private company for use in DLP (Data Loss Prevention) traffic inspection. Technically allowed the company to issue certificates for any domain. Caused industry debate about acceptable CA practices.
2013	ANSSI/Google France intermediate cert	A French government intermediate CA (operated by ANSSI) issued unauthorized certificates for Google domains. Google detected via CT/pinning. Limited scope but raised concerns about government CAs.
2014	India CCA/NIC breach	India's National Informatics Centre (NIC), operating under the Indian Controller of Certifying Authorities (CCA), issued unauthorized certificates for Google and Yahoo domains. Google detected via CT. Indian CCA root was constrained in trust stores.
2015	CNNIC/MCS Holdings	China Internet Network Information Center (CNNIC) delegated certificate issuance to MCS Holdings (an Egyptian company), which issued unauthorized certificates for Google domains via a MITM proxy. Google and Mozilla distrusted CNNIC's root certificates.
2015-2018	Symantec mis-issuance and distrust	Thousands of certificates issued without proper authorization. Progressive distrust by Chrome and Firefox over 2 years. Symantec CA sold to DigiCert for $950M. Massive impact on document signing certificates issued under Symantec, VeriSign, Thawte, and GeoTrust brands.
2017	WoSign/StartCom distrust	Chinese CA WoSign acquired StartCom (Israeli CA) without disclosure, backdated SHA-1 certificates to avoid browser deadlines, and made false statements to browser vendors. Both CAs distrusted by all major browsers.
2020	Let's Encrypt CAA bug revocation	3 million certificates revoked due to CAA checking non-compliance. 24-hour notice to certificate holders. Demonstrated operational fragility of CA infrastructure at scale.

Year	Incident	Impact and Outcome
2022	TrustCor allegations	Investigative reporting linked TrustCor (a Panamanian CA) to a company involved in distributing spyware. Mozilla, Google, and Microsoft removed TrustCor from their trust stores. No evidence of fraudulent certificate issuance, but trust was revoked on the basis of the CA operator's associations.
2023	e-Tugra suspended from CCADB	Turkish CA e-Tugra suspended from the Common CA Database (CCADB) after security researchers discovered critical vulnerabilities in its infrastructure, including exposed admin panels and improperly secured HSMs.

This timeline only includes publicly known incidents. The actual number of CA compromises is certainly higher — not every breach is detected, and not every detected breach is publicly disclosed. The incidents we know about are the ones where detection mechanisms worked. The ones we should worry about are the ones where they did not.

27.9 Impact on PDF Signature Validation: What Happens in Practice

When a CA is revoked, the practical impact on PDF signature validation depends on several factors:

27.9.1 The Validator's Trust Store

Different validators use different trust stores:

- **Adobe Acrobat (Windows)**: Uses AATL plus optionally the Windows certificate store
- **Adobe Acrobat (macOS)**: Uses AATL plus optionally the macOS Keychain
- **Foxit Reader**: Uses the operating system's trust store
- **LibreOffice**: Uses the operating system's trust store (via NSS on Linux)
- **EU DSS validation service**: Uses the EU Trusted List (EUTL)
- **Custom Java applications**: Use the JVM's `cacerts` trust store or a custom `TrustStore`

When a CA is revoked, the timing and scope of the revocation varies across these stores. AATL updates are published by Adobe on their own schedule. OS trust store updates come via system updates. The EUTL is updated when a member state's supervisory body modifies its trusted list.

This means a signed PDF can show different validation results in different viewers during the transition period — "valid" in one that has not yet received the revocation update, "invalid" in one that has.

27.9.2 Whether LTV Data Is Present

If the signature includes embedded revocation data (PAdES-B-LT or B-LTA), a competent validator can perform **historical validation**: it checks the embedded OCSP response or CRL to determine the certificate's status at the time of signing, rather than its status now.

If the signature does not include LTV data, the validator must check the certificate's current status — and if the CA has been revoked, the chain cannot be validated at all.

27.9.3 The Certificate's Position in the Chain

If only an intermediate CA is revoked (as in the Comodo case), the impact is limited to certificates issued through that specific intermediate. Other intermediates under the same root may be unaffected.

If the root CA is revoked (as in DigiNotar's case), *every* certificate in the chain is affected — there is no way to validate the chain without a trusted root.

27.10 Key Takeaways

- **CA breaches retroactively undermine PDF signatures** — the math does not change, but the trust does
- **DigiNotar (2011)** showed that a small CA's failure can have outsized impact — 531 rogue certs, mass surveillance, bankruptcy in 4 weeks
- **Comodo (2011)** showed that fast, transparent response can save a CA — 9 rogue certs, revoked in hours, CA survived
- **Symantec (2015-2018)** showed that even the largest CA is not too big to distrust — 30% of TLS certificates, distrusted over 2 years
- **Let's Encrypt (2020)** showed that compliance bugs at scale force brutal revocation timelines — 3 million certs, 24-hour notice
- **Certificate Transparency** makes rogue certificates detectable but does not prevent them
- **PAdES-B-LTA is the only reliable defense** — embedded revocation data at signing time proves certificate validity regardless of what happens to the CA later
- **Always have a CA migration plan** — your CA's trustworthiness is not guaranteed for the lifetime of your signed documents
- **Monitor your CA's status** — you want to know about problems before your validators do
- **Re-timestamp archived documents** periodically to extend their verifiability beyond any single CA's lifetime

Chapter 28

Oddities & Annoyances

Every technology has its dark corners — the design decisions that made sense in 1999 but haunt us today, the interoperability failures that no spec can fix, the political arrangements that masquerade as technical standards. PDF digital signatures have more dark corners than most.

This chapter is a tour of the things that will make you mutter unprintable words during implementation. Not bugs, exactly — bugs get fixed. These are *structural annoyances*: the problems baked into the specifications, the ecosystem, and the economics of PDF signing. Each one has wasted thousands of engineering hours across the industry. Understanding them will not make them go away, but it will stop you from wasting your own hours rediscovering what everyone else already knows and nobody bothered to write down.

28.1 The Adobe Approved Trust List Monopoly

28.1.1 What the AATL Is

Adobe Acrobat is the world's dominant PDF viewer. When a user opens a signed PDF in Acrobat, the first thing they see is a blue or green banner at the top of the document. Green means "Signed and all signatures are valid." Blue means "At least one signature has problems." Yellow means "Signed, but the signature cannot be verified."

The difference between green and yellow, in the vast majority of cases, is not whether the signature is cryptographically correct. It is whether the signing certificate chains to a root in the **Adobe Approved Trust List (AATL)**.

The AATL is a curated list of root certificates that Adobe trusts for document signing. It is published at `https://helpx.adobe.com/acrobat/kb/approved-trust-list2.html` and ships embedded in Acrobat. If your signing certificate chains to an AATL root, you get the green checkmark. If it does not — even if the certificate is perfectly valid, issued by a reputable CA, chains to a trust anchor in the operating system's trust store, and was created with impeccable cryptographic rigor — you get yellow.

Yellow, in the real world, means "the customer calls support."

28.1.2 The Economics of Green

Getting your root certificate into the AATL requires meeting Adobe's technical requirements (documented in the "AATL Technical Requirements" PDF, which is itself signed with an AATL certificate — recursion intended) and paying a fee. The requirements include:

- The root must be at least 4096-bit RSA or equivalent
- The CA must carry an annual WebTrust for CAs audit or equivalent (ETSI EN 319 411)

- The CA must use HSMs for root key protection
- Document signing certificates must include specific key usage extensions
- The CA must have a published CPS (Certification Practice Statement)

These are reasonable requirements for a public trust anchor. The problem is not the technical bar — it is the gatekeeping.

Adobe is a private company. The AATL is Adobe's private trust list. Adobe decides who gets in, Adobe decides who stays in, and Adobe decides the criteria. There is no appeals process, no regulatory oversight, and no obligation to accept any particular CA.

For CAs, inclusion in the AATL is not optional — it is a commercial necessity. Enterprise customers demand the green checkmark. Government procurement specifications require it. A CA that is not in the AATL effectively cannot sell document signing certificates at competitive prices, because the certificates will produce yellow warnings in the software that 80% of users use to view PDFs.

The AATL creates a two-tier market for document signing certificates. AATL-listed certificates cost significantly more than non-AATL certificates — typically $200-$500 per year for an individual signing certificate, versus $30-$50 for a certificate from a non-AATL CA. The price premium has nothing to do with cryptographic quality and everything to do with the color of a banner in one company's software.

If you are building a signing service, the single most important commercial decision you will make is whether your use case requires the green checkmark. If your recipients will open signed PDFs in Acrobat (and they almost certainly will), you need AATL. If your recipients use a custom validation workflow that checks against the EU Trusted List or your own trust store, you do not.

28.1.3 The EUTL Alternative

The EU Trusted List (Regulation (EU) No 910/2014 (eIDAS), Article 22) is the European counterpart to the AATL. Every Qualified Trust Service Provider in the EU is listed on the EUTL, and a certificate chaining to a QTSP on the EUTL qualifies for the highest eIDAS tier regardless of whether Adobe has heard of the CA.

But here is the catch: Acrobat does not validate against the EUTL by default. Acrobat uses the AATL. There is an option to enable EUTL validation in Acrobat (Edit > Preferences > Signatures > Verification > select "European Union Trusted Lists"), but it is off by default, and the vast majority of users never touch it.

This means a signature with a qualified certificate from a QTSP that is *not* in the AATL will show a yellow warning in Acrobat — even though it is legally a qualified electronic signature with the same standing as a handwritten signature under EU law.

The legal framework says: this is a QES, as valid as ink. The software says: "The validity of the document certification is UNKNOWN."

The user believes the software.

If you are distributing signed PDFs to recipients who use Acrobat, and your QTSP is on the EUTL but not the AATL, include instructions for enabling EUTL validation in Acrobat. Better yet, embed the full certificate chain and LTV data so that validation depends on the trust

store configuration rather than online lookups. Best of all: choose a QTSP that is on *both* the AATL and the EUTL. Most major European QTSPs have pursued AATL membership precisely because of this problem.

28.1.4 The Political Dimension

The AATL gives Adobe significant influence over the digital signing ecosystem. Adobe can, at its discretion, adjust requirements, add fees, or remove CAs. This is not a theoretical concern — Adobe has adjusted AATL requirements multiple times, each adjustment requiring CAs to re-certify and sometimes change their infrastructure.

Some in the industry argue that this is appropriate: someone needs to curate trust, and Adobe has a legitimate interest in ensuring that signatures validated by its software are trustworthy. Others argue that a single company should not be the gatekeeper for a technology used in legal, governmental, and financial transactions across the globe.

The emergence of the EUDI wallet (Chapter 27) may shift this dynamic. If QES becomes a consumer technology available to every EU citizen, the volume of qualified signatures will dwarf the current market — and the EUTL, not the AATL, will be the authoritative trust anchor for those signatures. Whether Adobe adjusts its defaults accordingly remains to be seen.

28.2 Broken Validation Across Viewers

28.2.1 The Problem

Take a signed PDF — any signed PDF — and open it in five different viewers. You will get five different answers about whether the signature is valid.

This is not an exaggeration. It is the daily reality of PDF signature interoperability.

The fundamental problem is that ISO 32000-2:2020 defines the *structure* of a PDF signature but leaves enormous latitude in how viewers should *validate* that structure. The specification says what a `/Sig` dictionary must contain, what the `/ByteRange` must cover, and how the CMS `SignedData` must be embedded. It does not say, with sufficient precision, what a viewer should do when the trust chain is incomplete, when the timestamp is missing, when the revocation data is stale, or when the certificate uses an algorithm the viewer does not support.

ETSI EN 319 142 (PAdES) adds more precision, but it constrains *signing* more than it constrains *validation*. A PAdES-B-LTA signature is well-defined in terms of what it must contain. What a validator must do with each piece of that content — and what it should report to the user when something is missing or ambiguous — is still underspecified.

28.2.2 The Comparison

The following table shows how five common validators handle the same edge cases. These are real behaviors observed in testing, not theoretical predictions.

Table 28.1: Signature validation behavior across viewers (observed, not specified)

Scenario	Acrobat DC	Foxit Reader	LibreOffice	macOS Preview	PDFBox
Valid PAdES-B-LTA, AATL cert	Green checkmark	Valid	Valid	Valid	Valid (programmatic)
Valid CMS, self-signed cert	Yellow warning	Unknown signer	Valid (if cert trusted)	Invalid	Valid (if cert in trust store)
Valid CMS, expired cert, no timestamp	Invalid (expired)	Invalid	Valid at signing time	Invalid	Depends on validation date
Valid CMS, EUTL cert, not in AATL	Yellow (unknown signer)	Valid (if EUTL enabled)	Valid	Unknown	Depends on trust store
SHA-1 digest algorithm	Warning (weak hash)	Valid (no warning)	Valid (no warning)	Valid	Valid (no warning)
Missing OCSP/CRL data	Warning (revocation unknown)	Valid (ignores)	Valid (ignores)	Valid (ignores)	No revocation check by default
Incremental update after signing	Modified after signing	Modified	May not detect	Does not detect	Detects (if coded)
SubFilter `ETSI.CAdES.detached`	Full validation	Full validation	Full validation	Partial	Full (if coded)
SubFilter `adbe.pkcs7.detached`	Full validation	Full validation	Full validation	Full validation	Full (if coded)
SubFilter `adbe.x509.rsa_sha1`	Legacy support	May fail	May fail	Fails	Not supported

The most dangerous row in this table is ”Incremental update after signing.” This is exactly the scenario that the Bochum Incremental Saving Attack (Chapter 24) exploits. If a viewer does not detect that the file was modified after signing, it will show a valid signature on a document whose content has been changed. As of 2026, most major viewers have patched this — but ”most” is not ”all,” and not every installed version has been updated.

28.2.3 Why Viewers Disagree

The disagreements stem from several root causes:

Different trust stores. Acrobat uses AATL. Foxit uses the OS trust store. LibreOffice uses NSS (on Linux) or the OS store (on Windows/macOS). macOS Preview uses the Keychain. A certificate trusted by one store may not be trusted by another.

Different algorithm support. Some viewers still accept SHA-1; others warn or reject. Some support ECDSA; others are RSA-only. Some parse CAdES extensions; others only understand basic CMS.

Different revocation policies. Acrobat actively checks revocation status (OCSP and CRL) if the information is available. Foxit and LibreOffice are more lenient — they may skip revocation checks if the data is not embedded. macOS Preview performs minimal revocation checking.

Different byte range verification. Acrobat carefully verifies that the byte range covers the entire file except the `/Contents` gap. Some other viewers are less rigorous — they check that the CMS signature verifies over the byte range data, but do not verify that the byte range actually covers the whole file.

Different handling of incremental updates. This is the critical one. A proper validator must compare the signed byte range to the total file size and, if there are bytes after the signed range, analyze the incremental update to determine whether it contains permitted changes (adding signatures, annotations) or impermissible modifications (changing page content). Not all viewers perform this analysis with equal rigor.

28.2.4 What You Can Do About It

If you are a signature *creator*, your goal is to produce signatures that validate consistently across all major viewers:

1. **Use PAdES-B-LTA.** It is the most broadly supported profile.
2. **Use SubFilter `ETSI.CAdES.detached`.** All modern viewers support it, and it enables PAdES validation.
3. **Use SHA-256 or SHA-384.** Never SHA-1. Avoid SHA-512 only because some older viewers have edge-case issues with it.
4. **Embed the full certificate chain.** Do not rely on the viewer fetching missing intermediates — some viewers will not.
5. **Embed OCSP responses and CRLs.** If the revocation data is in the DSS, viewers that check revocation will find it; viewers that do not check will simply ignore it.
6. **Allocate sufficient space for `/Contents`.** We discuss this in detail later in this chapter.
7. **Test in at least three viewers.** Acrobat, Foxit/LibreOffice, and one programmatic validator (PDFBox or EU DSS). If it validates in all three, it will validate almost everywhere.

If you are a signature *validator*, your goal is to implement the most rigorous checks and report the most precise results:

1. **Verify the byte range covers the entire file minus the `/Contents` gap.** This catches the Signature Wrapping Attack (Chapter 24).
2. **Detect incremental updates after signing.** Compare the signed byte range to the file size. If there is trailing data, parse it and determine whether it represents permitted or impermissible changes.
3. **Check revocation status.** Use embedded OCSP/CRL data if available; fall back to online checks if not; report clearly when revocation data is unavailable.
4. **Report the PAdES profile level.** Tell the user whether the signature is B-B, B-T, B-LT, or B-LTA — this is much more informative than a binary ”valid/invalid.”

28.3 SubFilter Confusion

28.3.1 The Five SubFilter Values

The `/SubFilter` key in a signature dictionary tells the viewer what format the signature value (in `/Contents`) uses. Over the history of PDF, five SubFilter values have been defined:

Table 28.2: SubFilter values defined across PDF versions and PAdES

SubFilter	Defined In	Status and Usage
`adbe.pkcs7.detached`	PDF 1.3 (1999)	The original. CMS `SignedData` where the signed content is the byte range data (not included in the CMS structure). Still widely used. Supported by every viewer. Not PAdES-compliant on its own.
`adbe.pkcs7.sha1`	PDF 1.4 (2001)	The SHA-1 hash of the entire document is embedded as the CMS `EncapsulatedContentInfo`. Deprecated in ISO 32000-2:2020. SHA-1 is broken for collision resistance since the SHAttered attack (2017). Do not use.
`adbe.x509.rsa_sha1`	PDF 1.3 (1999)	Raw RSA signature over SHA-1 hash. The certificate is in a separate `/Cert` key, not embedded in CMS. Extremely legacy. Most modern viewers cannot validate it. Do not use.
`ETSI.CAdES.detached`	PAdES (ETSI TS 102 778, 2009)	The PAdES SubFilter. CMS `SignedData` with CAdES extensions (ESS-signing-certificate-v2, optionally timestamp attributes). Required for all PAdES profiles. The recommended choice for any new implementation.
`ETSI.RFC3161`	ISO 32000-2:2020 (2017)	Document-level timestamp. The `/Contents` contains an RFC 3161 `TimeStampToken` instead of a regular CMS signature. Used for archive timestamps in PAdES-B-LTA. Not a ”signature” in the user-facing sense — it proves the document existed at a specific time.

28.3.2 Which One Should You Use?

If you are implementing a new signing feature in 2026, the answer is simple: **`ETSI.CAdES.detached`** for signatures and **`ETSI.RFC3161`** for document-level timestamps.

There is no reason to use `adbe.pkcs7.detached` in new implementations unless you have a specific compatibility requirement with a legacy system that cannot parse CAdES extensions. The practical difference between `adbe.pkcs7.detached` and `ETSI.CAdES.detached` is the presence of the `ESS-signing-certificate-v2` signed attribute (Chapter 17) — adding it makes the signature PAdES-B-B compliant at zero additional cost.

`adbe.pkcs7.sha1` is deprecated and cryptographically broken. Do not use it. If you encounter it in a legacy document, validate it with the understanding that SHA-1 collision resistance has been publicly demonstrated to be insufficient since February 2017 (the SHAttered attack by Stevens et al.).

`adbe.x509.rsa_sha1` is a relic of PDF 1.3. It separates the certificate from the signature in a way that no modern library handles well. If you encounter it, you are looking at a document from the early 2000s.

28.3.3 What Happens with Unrecognized SubFilter Values

ISO 32000-2:2020 §12.8.1 states that if a viewer encounters a SubFilter value it does not recognize, it should not attempt to validate the signature. In practice, viewers handle this inconsistently:

- **Acrobat**: displays "The signature format is not supported" — a clear and honest message
- **Foxit**: may attempt validation anyway and produce confusing results
- **LibreOffice**: typically shows "Signature could not be verified" — accurate but unhelpful
- **macOS Preview**: ignores the signature entirely — the document appears unsigned

If you are implementing a validator and encounter an unknown SubFilter, the correct behavior is to report that the signature format is not recognized. Do not attempt to guess the format. Do not silently skip the signature. Report it.

A common source of SubFilter confusion in the wild: some signing tools produce a SubFilter of `adbe.pkcs7.detached` but include CAdES extensions (ESS-signing-certificate-v2) in the CMS signed attributes. Technically, this is a valid `adbe.pkcs7.detached` signature that *also* happens to be PAdES-compliant — because PAdES compliance is about the content of the CMS, not just the SubFilter label. But some validators treat the SubFilter as a strict indicator: if it says `adbe.pkcs7.detached`, they skip CAdES-specific checks. The result: a signature that is PAdES-compliant by content but not recognized as such by label.

For maximum compatibility, use the SubFilter that matches your intent. If you are producing PAdES signatures, use `ETSI.CAdES.detached`.

28.4 PAdES vs ISO 32000-2:2020: The Divergence

28.4.1 Two Standards, One Problem

ISO 32000-2:2020 defines how to embed a digital signature in a PDF. ETSI EN 319 142 (PAdES) defines additional requirements for that signature to be legally meaningful in Europe.

These two standards *should* be perfectly aligned. They are not.

The divergence is not dramatic — PAdES is a strict subset of ISO 32000-2:2020, so a PAdES-compliant signature is always a valid ISO 32000-2:2020 signature. But the two standards were

developed by different organizations (ISO 32000-2:2020 by ISO TC 171, PAdES by ETSI ESI), on different timelines, with different constituencies. The result is a set of subtle mismatches that catch developers in the gap.

28.4.2 Where They Disagree

Signed attributes. ISO 32000-2:2020 requires the CMS `SignedData` to include `content-type` and `message-digest` signed attributes (RFC 5652 — Cryptographic Message Syntax (CMS) §11.1). PAdES *additionally* requires `ESS-signing-certificate-v2` (ETSI EN 319 122-1 §5.7.3). A signature that satisfies ISO 32000-2:2020 but omits `ESS-signing-certificate-v2` is a valid PDF signature but *not* a valid PAdES signature. This is the single most common PAdES compliance failure.

SubFilter semantics. ISO 32000-2:2020 treats SubFilter as informational — it tells the viewer what format to expect, but the viewer is expected to parse the CMS and determine the format from the content. PAdES treats `ETSI.CAdES.detached` as a *requirement* — a signature claiming PAdES compliance must use this SubFilter. A technically identical CMS blob wrapped in `adbe.pkcs7.detached` is not PAdES-compliant by label, even if the CMS content is identical.

Timestamp handling. ISO 32000-2:2020 defines document-level timestamps (SubFilter `ETSI.RFC3161`) and signature timestamps (unsigned attribute `id-aa-signatureTimeStampToken` in the CMS). PAdES defines the same mechanisms but adds requirements about timestamp token format, the relationship between signature timestamps and document timestamps, and the order in which they must be embedded in PAdES-B-LTA. The PAdES requirements are stricter — a timestamp arrangement that is valid under ISO 32000-2:2020 may not satisfy PAdES-B-LTA's requirements for archive timestamps.

DSS dictionary. The Document Security Store (DSS) dictionary — where certificates, OCSP responses, and CRLs are stored for LTV validation — is defined in ISO 32000-2:2020 §12.8.4.3 and also referenced in PAdES. The specifications agree on the structure but differ slightly on when DSS data is mandatory. ISO 32000-2:2020 treats DSS as optional enrichment. PAdES-B-LT and B-LTA *require* DSS data — without it, the signature cannot claim the LT or LTA profile level.

28.4.3 The Practical Consequence

If you are building a signing service that needs to produce signatures valid in both the ISO 32000-2:2020 sense and the PAdES sense (and you almost certainly are), always target PAdES. PAdES is the stricter standard. A PAdES-B-LTA signature satisfies both standards simultaneously. A signature that targets only ISO 32000-2:2020 without PAdES awareness will fail PAdES validation — and PAdES validation is what matters for EU compliance, for AATL-listed CAs, and for most enterprise validation workflows.

The EU DSS validation framework (Chapter 23) validates signatures against PAdES profiles, not against raw ISO 32000-2:2020. If your signatures validate in EU DSS, they satisfy both standards. If you are building your own validator, validate against PAdES — it is the superset of requirements that covers both.

28.5 The Timestamp Chicken-and-Egg Problem

28.5.1 The Problem

A PAdES-B-T signature requires a signature timestamp — an RFC 3161 `TimeStampToken` embedded as an unsigned CMS attribute. The timestamp proves when the signature was created.

To obtain the timestamp, you must:

1. Create the CMS `SignedData` (including the signed attributes and the signature value)
2. Hash the signature value
3. Send the hash to a Time Stamp Authority (TSA)
4. Receive the `TimeStampToken` from the TSA
5. Embed the token as an unsigned attribute in the CMS `SignedData`

Here is the problem: the CMS `SignedData` is the `/Contents` value in the signature dictionary. The `/Contents` value is a hex string with a *fixed* length — the space for it was allocated when the signature dictionary was written into the PDF, before the CMS was created.

So the size of `/Contents` must be determined *before* you know how large the CMS will be. And the size of the CMS depends on whether you include a timestamp — and how large that timestamp is. And you cannot obtain the timestamp until you have the CMS signature value. And you cannot have the signature value until you have the byte range. And the byte range depends on the size of `/Contents`.

This is the chicken-and-egg.

28.5.2 The Standard Solution

The standard solution, used by every major signing library, is to **pre-allocate a fixed-size `/Contents` field**, create the CMS signature, obtain the timestamp, embed the timestamp in the CMS, and then pad the CMS to fit the pre-allocated space.

```
// Pre-allocate /Contents with estimated size
// The hex encoding doubles the byte count, so allocate 2x the expected CMS size
int estimatedCmsSize = 16384;  // 16 KB — usually sufficient for B-T
SignatureOptions sigOptions = new SignatureOptions();
sigOptions.setPreferredSignatureSize(estimatedCmsSize);

// The PDF is written with a /Contents placeholder of 2 * estimatedCmsSize hex chars
// After signing, the actual CMS (with timestamp) is padded with zeros to fill the space
```

The padding is simple: after the CMS `SignedData` DER encoding, the remaining bytes in `/Contents` are filled with `0x00`. The hex encoding of these zero bytes produces `00` characters. The viewer knows to ignore trailing zeros after the DER-encoded CMS — the DER encoding is self-delimiting, so the parser can determine where the CMS ends and the padding begins.

28.5.3 The 32 KB Problem

How much space should you pre-allocate? This is a more consequential question than it appears.

The CMS `SignedData` contains:

- The DER-encoded `SignedData` structure itself (~100-300 bytes of overhead)
- The signer's certificate (~1-2 KB)
- Any intermediate certificates (~1-2 KB each, typically 1-3 intermediates)
- The signature value (~256-512 bytes for RSA-2048/4096, ~64-72 bytes for ECDSA)
- Signed attributes (~200-500 bytes)
- The signature timestamp token (~3-8 KB, depending on the TSA)

A typical PAdES-B-T signature with one intermediate certificate and a timestamp requires approximately 8-12 KB. A PAdES-B-LTA signature with embedded OCSP responses and CRLs in the CMS (rather than in the DSS) can require 15-30 KB.

Most implementations pre-allocate **32 KB** (32,768 bytes, producing 65,536 hex characters in `/Contents`). This is sufficient for the vast majority of signatures.

But not all.

Table 28.3: Scenarios that exceed 32 KB

Scenario	Why It Is Large
Long certificate chain	4-5 intermediates, each 2 KB. Happens with some government CAs.
Large timestamp token	Some TSAs include their own certificate chain in the TimeStampToken. If the TSA has 3 intermediates, the token alone can be 10-15 KB.
Embedded CRLs	If a CRL is embedded directly in the CMS (as an unsigned attribute), it can be 50-200 KB. This is unusual for new implementations (CRLs go in the DSS instead), but some legacy signing tools do this.
Cross-certified chains	Government PKIs that cross-certify with multiple roots can produce chains with redundant certificates, inflating the CMS.
Multiple timestamps	Some workflows add both a signature timestamp and a content timestamp, doubling the timestamp overhead.

If the CMS exceeds the pre-allocated `/Contents` size, the signing operation fails. The failure mode depends on the library:

- **PDFBox**: throws an `IOException` with a message about the signature being too large for the placeholder
- **iText**: throws a similar exception
- **OpenPDF**: throws an exception

The fix is simple: increase the pre-allocated size. But the fix means you allocated *more* space that is filled with zero padding — wasted bytes in every PDF you sign. There is a tension between allocating enough space to handle edge cases and not bloating every PDF with unused padding.

A practical compromise: pre-allocate 32 KB for PAdES-B-T signatures and 64 KB for PAdES-B-LTA signatures. If you know your signing chain is unusually long (government PKIs,

cross-certified CAs), increase to 96 KB or 128 KB. The wasted space is insignificant compared to the risk of a signing failure in production.

If you want to be precise, you can estimate the CMS size before allocating:

1. Build a ”dry run” CMS with a dummy signature value and a dummy timestamp of the expected size
2. Measure its DER-encoded length
3. Add a 50% safety margin
4. Use that as your pre-allocation size

This is more complex but avoids both undersizing and excessive oversizing.

28.6 Silent Signing Failures

28.6.1 The Taxonomy of Silence

PDF signing can fail in ways that produce a valid-looking PDF with an invalid signature. The document opens. The pages render. But the signature is broken — and the failure may not be obvious until someone tries to validate it.

These silent failures are particularly dangerous because they often are not detected at signing time. The signing code runs without exceptions. The PDF is saved. It is emailed, archived, or submitted to a government portal. And then, days or weeks later, someone discovers that the signature is invalid.

28.6.2 Empty `/Contents`

The most egregious silent failure: the `/Contents` value is present but contains only zero bytes. No CMS structure at all.

This happens when the signing pipeline fails to inject the CMS into the `/Contents` placeholder after the byte range is computed and the CMS is generated. The PDF has the correct structure — the signature dictionary exists, the byte range is set, the `/Contents` key is there — but the value is all zeros.

Some signing libraries have had bugs where exceptions during CMS generation were swallowed (caught and not re-thrown), leaving the `/Contents` empty. The resulting PDF looks signed — it has a signature field, possibly even a visual appearance — but the signature is cryptographically void.

After every signing operation, *always* verify the result. Open the output PDF, extract the `/Contents` value, and confirm it contains a valid DER-encoded CMS `SignedData`. This adds a few milliseconds to your signing pipeline and catches the most catastrophic failure mode.

```
// Post-signing verification — always do this
byte[] contents = extractContentsFromSignedPdf(signedPdfBytes);
CMSSignedData cms = new CMSSignedData(contents);
if (cms.getSignerInfos().getSigners().isEmpty()) {
    throw new IllegalStateException(
            "Signed PDF has no signers in CMS — signing failed silently");
}
```

```
log.info("Post-signing verification passed: {} signer(s) in CMS",
        cms.getSignerInfos().getSigners().size());
```

28.6.3 Bad Byte Range

The `/ByteRange` array must specify two ranges that together cover the entire file except the `/Contents` hex string. A bad byte range — one that does not start at 0, does not end at the file's last byte, or has gaps other than the `/Contents` region — produces a signature that covers only part of the file.

This can happen when:

- The signing library miscalculates the offset of the `/Contents` value in the file
- The file is modified (even trivially — whitespace changes, metadata updates) between byte range calculation and signature injection
- A streaming writer flushes partial content, causing offset shifts

The PDF opens fine. The signature exists. But a proper validator will detect that the byte range does not cover the entire file and report the signature as invalid or the document as modified.

28.6.4 Missing Certificate Chain

A CMS `SignedData` should include the signer's certificate and all intermediate certificates needed to build a chain to a trusted root. If the intermediates are missing, the validator must fetch them — and may not be able to.

Missing intermediates are common in testing (developers use self-signed certificates and forget to include the chain) but also happen in production when:

- The signing code includes only the signer's certificate, not the intermediates
- The PKCS#12 keystore contains only the signer's certificate (the intermediates were not imported)
- A library upgrade changed the default behavior for chain inclusion

The signature validates if the validator can fetch or already has the missing intermediates. It fails if it cannot. This is a Heisenbug — the signature works on the developer's machine (where the intermediates are in the trust store) but fails on the customer's machine (where they are not).

28.6.5 Expired Timestamp

A signature timestamp proves when the signature was created. But the timestamp itself is signed by the TSA's certificate — and that certificate can expire.

If the TSA certificate expires and the timestamp token does not include sufficient chain data for historical validation, validators may reject the timestamp. The signature itself may still verify, but the timestamp — which was the proof of when the signature was created — becomes unverifiable.

For PAdES-B-LTA, this is handled by the archive timestamp mechanism: a document-level timestamp covers the entire file (including the DSS with revocation data for the signature

timestamp). As long as the archive timestamp chain is valid, the signature timestamp can be verified historically.

For PAdES-B-T signatures without LTV data, an expired TSA certificate means the timestamp can no longer be verified. The signature degrades from B-T to effectively B-B — you lose the proof of when it was created.

Monitor the expiration dates of the TSAs you use. TSA certificates typically have 10-15 year validity periods, so this is not an urgent problem — but it *is* a problem that will arrive eventually. When it does, any PAdES-B-T signature without LTV data that used the expired TSA will lose its timestamp verifiability.

This is another reason to prefer PAdES-B-LTA: the archive timestamp mechanism provides a renewable chain that survives individual certificate expirations.

28.7 Reader-Enabled PDFs vs Signing Conflicts

28.7.1 What Reader-Enabling Is

Adobe Reader (the free version) historically could not save filled-in forms. To allow Reader users to fill in and save a form, the PDF author had to "enable" the PDF for Reader using Adobe Acrobat Pro. This was done by applying a special **usage rights signature** — a certification-level signature with a specific set of usage rights extensions that told Reader which features to unlock.

The usage rights signature is stored in the document's permissions dictionary (`/Perms`) and uses a special Adobe-issued certificate. When Reader opens a document with a valid usage rights signature, it enables the specified features (fill-in, save, comment, etc.).

28.7.2 The Conflict with Document Signing

The problem arises when a Reader-enabled PDF needs to be signed.

The usage rights signature is a certification signature — it was the first signature on the document, and it claims `/DocMDP` permissions. Adding an approval signature after the certification signature requires an incremental update, which is permitted under the DocMDP permissions level set by the usage rights signature.

But here is the catch: some PDF editors, when they save the document after adding a signature, perform operations that invalidate the usage rights signature. This happens because:

1. The editor modifies objects that the usage rights signature's byte range covers
2. The editor performs a full save instead of an incremental save, destroying the incremental structure that the usage rights signature depends on
3. The editor adds or modifies AcroForm entries in a way that the usage rights signature's `/FieldMDP` transform does not permit

When the usage rights signature is invalidated, Reader reverts to its restricted mode — the user can no longer save the form. And if the document's approval signature depends on the AcroForm structure (which it does, since signature fields *are* form fields), the interplay

between the invalidated usage rights and the new signature can produce validation errors in Acrobat.

28.7.3 The Historical Context

Reader-enabling was a significant source of revenue for Adobe: Acrobat Pro was the only tool that could create the usage rights signature (because it used an Adobe-controlled certificate). Third-party libraries could not Reader-enable PDFs.

ISO 32000-2:2020 deprecated the `/Perms` usage rights mechanism. PDF 2.0 assumes that all viewers can save forms without special enablement. But the real world is not PDF 2.0 — millions of Reader-enabled PDF 1.7 documents are still in circulation, and enterprises still generate new ones because their workflows depend on the Reader experience.

If your signing service processes documents that may be Reader-enabled, test the signing workflow with Reader-enabled PDFs specifically. Ensure your signing code performs an incremental save (not a full save) and does not modify objects covered by the usage rights signature. PDFBox's `PDDocument.saveIncremental()` is designed for this — but you must be careful not to load and re-save objects that the usage rights signature covers.

28.7.4 The Modern Alternative

The practical alternative to Reader-enabling is to stop caring about it. Adobe Acrobat Reader DC (the current free version) can save filled-in forms without usage rights. The limitation existed in older versions of Reader (pre-DC) and was removed as Adobe shifted to a subscription model for Acrobat.

If your user base has upgraded to Reader DC (and by 2026, most have), Reader-enabling is no longer necessary. If you must support legacy Reader versions, the only safe approach is to Reader-enable the PDF *after* all signatures have been applied — which is usually not possible, since the usage rights signature must be the first signature.

The real answer is: test your specific workflow, document the behavior, and warn your users if they are operating in a scenario where Reader-enabling and signing conflict.

28.8 Trailing Bytes and Garbage After `%%EOF`

28.8.1 The Spec Says

ISO 32000-2:2020 §7.5.5 states that a PDF file must end with `%%EOF`. Anything after the last `%%EOF` marker is not part of the PDF.

In an incrementally saved PDF (which every signed PDF is — signing requires an incremental save), each revision ends with its own `%%EOF`. The last `%%EOF` in the file marks the end of the last revision.

28.8.2 The Reality

In practice, many tools append data after `%%EOF`:

- Some PDF generators append a newline or carriage return after `%%EOF` (harmless but technically non-conformant)
- Some content management systems append metadata or tracking information
- Some signing tools, due to buffer sizing bugs, write a few extra bytes after the CMS injection
- Some email systems append MIME boundary markers when the PDF is transmitted as an attachment

These trailing bytes create a problem for signature validation: the signed byte range (as specified by `/ByteRange`) covers the file up to and including the last byte *as calculated at signing time*. If additional bytes appear after that, the file is longer than the signed byte range. A strict validator will flag this as ”the document was modified after signing” — because, technically, it was, even if the modification was just a trailing newline.

When your validator detects bytes beyond the signed byte range, examine what those bytes actually are before reporting ”modified after signing.” A few bytes of whitespace or a line break after `%%EOF` is cosmetically messy but not a security concern. A complete incremental update after the signed range is a genuine modification that needs careful analysis.

Distinguish between ”there are trailing bytes that are not an incremental update” (cosmetic) and ”there is an incremental update that was not covered by any signature” (potentially malicious).

28.9 Key Takeaways

- **The AATL is a private trust list** controlled by Adobe that determines whether Acrobat shows a green or yellow banner — this gatekeeping creates a two-tier market for signing certificates
- **Validation results differ across viewers** — the same PDF can show valid, invalid, or unknown depending on trust stores, algorithm support, revocation policies, and byte range verification rigor
- **Use `ETSI.CAdES.detached`** for all new signatures — it is PAdES-compliant and universally supported
- **PAdES is stricter than ISO 32000-2:2020** — always target PAdES, because a PAdES signature satisfies both standards
- **Pre-allocate 32-64 KB for `/Contents`** — undersizing causes signing failures; oversizing wastes a few KB of padding
- **Always verify the result after signing** — check that `/Contents` contains valid CMS, the byte range is correct, and the chain is complete
- **Monitor TSA certificate expiration** — expired TSA certs degrade B-T signatures to effectively B-B
- **Reader-enabled PDFs conflict with signing** — test specifically if your workflow involves Reader-enabled documents
- **Trailing bytes after `%%EOF`** are common and usually harmless, but your validator should distinguish cosmetic trailing data from actual post-signature modifications

Chapter 29

EUDI Wallet & Digital Identity

For three decades, qualified electronic signatures have been a niche technology — expensive, complicated, and reserved for enterprises, notaries, and government agencies. In 2024, the European Union decided to change that. The revised eIDAS regulation puts a qualified signature creation capability in the pocket of every EU citizen, on a device they already own: their smartphone.

This is not incremental improvement. This is a category shift. If it works — and the ”if” is substantial — QES moves from a tool that a few thousand specialized users understand to a feature that 450 million Europeans can use as casually as they use contactless payment.

This chapter covers the regulatory, architectural, and technical dimensions of the EUDI wallet and its impact on PDF digital signatures. We start with the political context (because regulation drives everything in this space), move through the architecture, and end with what you — the developer building signing or validation software — need to prepare.

29.1 eIDAS 2.0: What Changed and Why

29.1.1 The Regulatory History

The original eIDAS regulation (Regulation (EU) No 910/2014 (eIDAS), Chapter 19) entered into force in 2014 and became fully effective in 2016. It was a success in the narrow sense: it created a functioning framework for qualified trust services, established the EU Trusted List, and enabled cross-border recognition of QES.

It was a failure in the broad sense: almost nobody used it.

The adoption numbers tell the story. By 2022, eight years after eIDAS entered into force, fewer than 14% of EU citizens had ever used a qualified electronic signature. The number who used QES regularly was a fraction of that. In some member states, the number of active QES users was in the low thousands — in countries with populations of tens of millions.

The reasons for low adoption were structural:

1. **Cost.** A qualified signing certificate costs 50-300 EUR per year. Most citizens have no reason to pay that.
2. **Complexity.** Creating a QES required a smart card or USB token, driver software, and middleware that worked reliably on exactly zero consumer devices.
3. **No killer application.** There was no consumer use case compelling enough to justify the cost and complexity. Tax filing, the most common candidate, was solved by other means in most member states.
4. **eID fragmentation.** Each member state had its own electronic identity scheme, with its own technology stack, its own enrollment process, and its own level of adoption.

Cross-border recognition of eIDs was technically possible under eIDAS but practically rare.

The European Commission recognized by 2020 that voluntary adoption was not going to achieve the digital single market it wanted. Something more forceful was needed.

29.1.2 Regulation (EU) 2024/1183

On April 11, 2024, the revised eIDAS regulation — Regulation (EU) 2024/1183 (eIDAS 2.0), commonly called eIDAS 2.0 — entered into force. The revision keeps the trust services framework from eIDAS 1.0 largely intact (the three-tier signature model, QTSPs, the EU Trusted List — all unchanged) but adds a transformative new component.

The headline change: **every EU member state must provide a European Digital Identity Wallet to its citizens and residents**.

The timeline:

Table 29.1: eIDAS 2.0 key dates

Date	Milestone
April 11, 2024	eIDAS 2.0 enters into force
By November 2024	European Commission publishes the first set of implementing acts defining technical specifications
2024-2025	Large-Scale Pilots (LSPs) across the EU test wallet implementations in real-world scenarios
By end of 2026	Member states must make the EUDI wallet available to all citizens and residents
2027 onwards	Mandatory acceptance of the wallet by public services and, for certain use cases, private sector reliance parties

29.1.3 The Political Context

eIDAS 2.0 did not emerge from a vacuum. It was shaped by several converging political forces:

COVID-19 and the Digital COVID Certificate. The EU Digital COVID Certificate (2021) demonstrated that a pan-European digital credential system could be built and deployed at scale in months. Over 1.8 billion certificates were issued across 46 countries. The technical infrastructure — QR codes, digital signatures, trust frameworks — proved that citizens would adopt digital credentials if the use case was compelling enough. The political lesson was clear: mandates work, voluntary adoption does not.

Strategic autonomy. The European Commission increasingly frames digital identity as a matter of strategic autonomy. European citizens authenticate to online services using Google, Apple, and Facebook identity providers — American companies, subject to American law, storing European identity data on American infrastructure. The EUDI wallet is, in part, an assertion that European identity should be managed by European institutions.

The digital single market. The EU's single market depends on cross-border transactions. Those transactions require identity verification. If every member state has its own identity system, cross-border identity verification remains fragmented — exactly the problem eIDAS 1.0 was supposed to solve and did not.

Article 45 and the browser wars. The original eIDAS 2.0 proposal included a controversial provision (Article 45) that would have required web browsers to trust Qualified Website Authentication Certificates (QWACs) issued by QTSPs, without the ability to apply additional security requirements. Browser vendors — particularly Mozilla and Google — objected strenuously, arguing that this would undermine browser security by forcing them to trust certificates that did not meet their own security standards. The final text was amended to address some of these concerns, but the episode illustrates the tension between regulatory mandates and technical security communities.

29.2 The EUDI Wallet Concept

29.2.1 What the Wallet Is

The EUDI wallet is a smartphone application — provided by or on behalf of a member state — that allows citizens and residents to:

1. **Store government-issued identity data (PID):** name, date of birth, nationality, and a unique persistent identifier — the digital equivalent of a national ID card or passport
2. **Store qualified electronic attestations of attributes (QEAAs):** digitally signed attestations of specific attributes — university degrees, professional qualifications, driving licenses, age verification, right to work
3. **Present credentials selectively:** share only the specific attributes required for a transaction (e.g., "I am over 18" without revealing date of birth or name)
4. **Create qualified electronic signatures:** sign documents with QES, using a remote QSCD accessed through the wallet

The wallet is *not* a single application developed by the European Commission. Each member state is responsible for providing its own wallet implementation. The Commission defines the technical specifications and ensures interoperability; member states build or procure the implementations.

29.2.2 What the Wallet Is Not

The EUDI wallet is not a blockchain. It is not decentralized in the cryptocurrency sense. It is not self-sovereign identity (SSI), although it borrows some concepts from that space.

The wallet is a *government-issued* credential system. The identity data comes from authoritative government sources (civil registries, passport offices). The trust model is top-down: you trust the wallet because you trust the member state that issued it, and other member states trust it because the EU framework mandates mutual recognition.

The wallet is also not mandatory for citizens. eIDAS 2.0 requires member states to *offer* the wallet; it does not require citizens to *use* it. However, Article 5b requires that a wide range of public and private sector entities accept the wallet — which creates strong incentives for adoption even without an explicit mandate.

The voluntary nature of the wallet is politically important. A mandatory government identity app on every citizen's phone would face significant political resistance, particularly in countries with strong privacy traditions (Germany, Austria, the Netherlands). By making the wallet optional for citizens but mandatory for reliance parties, eIDAS 2.0 creates adoption through demand-side pressure rather than supply-side compulsion.

29.3 Architecture Reference Framework (ARF)

29.3.1 Overview

The Architecture Reference Framework (ARF) is the technical blueprint for the EUDI wallet ecosystem. Published and maintained by the European Commission, the ARF defines the architecture, protocols, data models, and trust framework for the wallet.

As of early 2026, the ARF is at version 1.4 and still evolving. The core architecture is stable, but specific protocol choices and data formats are being refined based on pilot feedback.

The ARF defines four main roles:

Table 29.2: EUDI wallet ecosystem roles

Role	Description
Wallet User	The citizen or resident who holds the wallet on their device
Wallet Provider	The member state (or entity acting on its behalf) that issues and manages the wallet application
PID Provider / Attestation Provider	The authoritative source that issues identity data (PID) or attribute attestations (QEAAs) into the wallet
Relying Party	The entity (public or private sector) that requests and validates credentials from the wallet

29.3.2 The Trust Model

The EUDI wallet trust model has multiple layers:

1. **Device trust.** The wallet runs on the user's smartphone. The device's secure element or Trusted Execution Environment (TEE) provides hardware-backed key storage. The wallet checks that the device meets minimum security requirements (OS version, no root/jailbreak).
2. **Wallet trust.** Each wallet implementation is certified by the member state that issues it. The certification confirms that the wallet meets the ARF's security and functional requirements. Wallet instances are registered with the member state's infrastructure.
3. **PID/QEAA trust.** PID and QEAAs are digitally signed by the issuing authority (the PID Provider for identity data, the Attestation Provider for attributes). The signature uses a key that chains to the member state's entry in the EU Trusted List. Relying parties validate the attestation signatures against the EUTL, just as PDF signature validators check the signer's certificate against a trust store.
4. **Relying party trust.** Relying parties that request credentials from the wallet must be registered and authenticated. The wallet verifies the relying party's identity before

releasing any data — the user knows *who* is requesting their data, and the wallet ensures the request is legitimate.

The trust model for the EUDI wallet is fundamentally the same trust model as PDF digital signatures: X.509 certificates, certificate chains, trusted lists, and CMS/COSE signatures. If you understand how PDF signature validation works (Chapters 16-17), you understand the core of the wallet's trust model. The wallet adds selective disclosure and user consent on top, but the cryptographic foundations are identical.

29.4 PID and QEAA

29.4.1 Person Identification Data (PID)

The PID is the wallet's core credential — the digital equivalent of a national ID card. It is issued by the member state's PID Provider (typically the civil registry or passport authority) and contains a minimum set of identity attributes.

Table 29.3: Mandatory PID attributes (ARF v1.4)

Attribute	Description
`family_name`	Current family name(s)
`given_name`	Current given name(s)
`birth_date`	Date of birth
`unique_id`	A persistent, unique identifier assigned by the member state
`age_over_18`	Boolean: is the holder over 18? (Derived attribute for age verification without revealing birth date)
`issuance_date`	When the PID was issued
`expiry_date`	When the PID expires
`issuing_authority`	The authority that issued the PID
`issuing_country`	The member state that issued the PID (ISO 3166-1 alpha-2)

The PID is signed by the PID Provider using a key that chains to the member state's trust infrastructure. The signature format is currently specified as either **ISO/IEC 18013-5 mdoc** (the same format used for mobile driving licenses) or **SD-JWT** (Selective Disclosure JSON Web Token, IETF draft). The ARF supports both formats, with ongoing discussion about whether to mandate one or allow both.

29.4.2 Qualified Electronic Attestations of Attributes (QEAA)

QEAAs are the wallet's extensibility mechanism. While the PID provides core identity, QEAAs attest to specific attributes beyond identity:

- **Educational credentials:** university degrees, professional certifications

- **Professional qualifications:** medical licenses, legal bar admissions, engineering certifications
- **Organizational identity:** employment status, company role, powers of representation
- **Financial attributes:** bank account verification, creditworthiness attestations
- **Travel documents:** digital driving licenses, digital travel credentials

A QEAA is issued by a Qualified Trust Service Provider (QTSP) or a government authority. When issued by a QTSP, it carries the same legal weight as the paper credential it replaces — a QEAA attesting a university degree, issued by a QTSP after verification with the university, is legally equivalent to the paper diploma.

The ”qualified” designation is critical. An Electronic Attestation of Attributes (EAA) without the ”qualified” prefix does not enjoy the same legal presumptions. The distinction mirrors the QES/AdES distinction: qualified attestations carry a legal presumption of correctness; non-qualified attestations are evidence to be evaluated on their merits.

29.5 Selective Disclosure and SD-JWT

29.5.1 The Privacy Problem

Traditional digital credentials have a privacy problem: they are all-or-nothing. If your credential contains your name, date of birth, address, and nationality, presenting it to verify your age requires revealing all four attributes — even though the relying party only needs to know that you are over 18.

The EUDI wallet solves this through **selective disclosure** — the ability to present only a subset of the attributes in a credential, without invalidating the cryptographic signature.

29.5.2 How SD-JWT Works

The ARF specifies **SD-JWT (Selective Disclosure for JWTs)** as one of the credential formats for selective disclosure. SD-JWT is an IETF specification (currently in the RFC track) that extends JSON Web Tokens with selective disclosure capability.

The mechanism works as follows:

1. **Issuance.** The issuer creates a JWT containing all attributes, but each disclosable attribute is replaced with a hash of the attribute value combined with a random salt. The original attribute values and their salts are provided to the wallet holder as ”disclosures” — separate, individually presentable data items.
2. **Presentation.** When the wallet presents the credential to a relying party, it includes the JWT (with the hashed placeholders) plus only the disclosures for the attributes the relying party needs. The relying party can verify that each disclosed attribute matches its hash in the JWT, confirming it was part of the original signed credential — but attributes whose disclosures are withheld remain hidden.
3. **Verification.** The relying party verifies the JWT signature (confirming it was issued by a trusted authority), then verifies each disclosed attribute against its hash in the JWT. Attributes that were not disclosed cannot be reconstructed from the hashes (the random salts ensure this).

```
// Simplified SD-JWT structure (conceptual, not a real token)
{
  "iss": "https://pid.example-country.eu",
  "iat": 1709913600,
  "exp": 1741449600,
  "cnf": { "jwk": { ... } },  // Holder binding key

  // Selectively disclosable claims (hashed placeholders)
  "_sd": [
    "H3JQXF...base64url_hash_of_family_name_disclosure",
    "K9PLWT...base64url_hash_of_given_name_disclosure",
    "M2VRZN...base64url_hash_of_birth_date_disclosure"
  ],

  // Non-disclosable claims (always visible)
  "age_over_18": true,
  "issuing_country": "DE"
}
```

29.5.3 The ISO/IEC 18013-5 mdoc Alternative

The ARF also supports the **mdoc** format defined in ISO/IEC 18013-5 (originally for mobile driving licenses). mdoc uses CBOR (Concise Binary Object Representation) instead of JSON and COSE (CBOR Object Signing and Encryption) instead of JOSE.

Both formats support selective disclosure. The choice between them is still under discussion in the ARF, with mdoc favored for proximity (NFC/BLE) use cases and SD-JWT favored for remote (online) use cases.

For PDF signing purposes, the credential format does not directly matter — what matters is that the wallet can authenticate the user and authorize a signing operation. The credential format affects how the wallet proves the user's identity to the signing service, not how the PDF signature itself is structured.

If you are building a service that accepts signed PDFs and also needs to verify the signer's attributes (e.g., a medical prescription service that needs to confirm the signer is a licensed doctor), you will need to handle QEAA validation *in addition to* PDF signature validation. The QEAA is presented separately from the PDF — it is not embedded in the signature's CMS. Your validation workflow will have two parallel tracks: verify the PDF signature, and verify the QEAA.

29.6 Signing with the Wallet

29.6.1 The Signing Flow

The most consequential feature of the EUDI wallet for this book's audience is its ability to create qualified electronic signatures. eIDAS 2.0 Article 5a(4)(d) requires that the wallet support QES creation. Article 45e goes further: member states must ensure that QES creation through the wallet is offered to natural persons free of charge for non-professional purposes.

Free QES for every EU citizen. That sentence alone justifies this chapter.

The signing flow works as follows:

1. **The user initiates a signing request.** This could be from within the wallet app itself, or from an external application (a web portal, a desktop application, a government service) that redirects to the wallet via a protocol defined in the ARF (OpenID4VP — OpenID for Verifiable Presentations).
2. **The wallet authenticates the user.** The user confirms their identity using the wallet's authentication mechanism — typically biometric (fingerprint or face recognition) or device PIN. This is the "sole control" requirement from eIDAS Article 26(c) and Article 29.
3. **The wallet presents the document hash to a remote QSCD.** The wallet does *not* contain the private signing key on the device itself. Instead, it communicates with a **remote QSCD** operated by a QTSP. The wallet sends a hash of the document to be signed (not the document itself — the hash is small and privacy-preserving).
4. **The remote QSCD creates the signature.** The QSCD — an HSM in the QTSP's data center, certified as a Qualified Signature Creation Device — signs the hash with the user's private key. The private key was generated on and never leaves the QSCD.
5. **The signature is returned to the requesting application.** The CMS `SignedData` (or the raw signature value, depending on the integration pattern) is returned to the application that initiated the signing request. The application embeds the signature in the PDF.

Table 29.4: EUDI wallet signing flow

Step	Description
1	Application presents document to user and requests signature (via OpenID4VP)
2	Wallet displays document summary and requests user authentication (biometric/PIN)
3	Wallet sends document hash to remote QSCD at QTSP (over mTLS, authenticated by wallet instance credentials)
4	QSCD signs the hash with user's private key; returns CMS `SignedData` with qualified certificate and signature timestamp
5	Application receives CMS, embeds it in PDF `/Contents`, adds DSS data for LTV, and saves the signed PDF

29.6.2 Remote QSCD: Why the Key Is Not on the Phone

A natural question: why not put the signing key on the phone itself? Modern smartphones have secure elements and TEEs that can generate and store cryptographic keys.

The answer is QSCD certification.

A QSCD must be certified under Common Criteria EAL4+ (or equivalent) against specific Protection Profiles (EN 419 211 or EN 419 221). Smartphone secure elements are certified for payment (EMVCo) and some for FIDO authentication, but as of 2026, no smartphone secure element has been certified as a QSCD under the eIDAS Protection Profiles.

The certification gap is not primarily technical — modern secure elements are arguably as secure as certified smart cards. The gap is procedural: QSCD certification requires extensive evaluation against specific attack models, documentation of the manufacturing process, and

ongoing compliance monitoring. Smartphone vendors have not pursued this certification because, until eIDAS 2.0, there was no market demand for it on consumer devices.

This may change. The ARF explicitly contemplates the possibility of ”local QSCD” on a device — a future scenario where smartphone secure elements achieve QSCD certification and the signing key can reside on the phone. But for the initial wallet deployments, the architecture is remote QSCD.

Remote QSCD is not a compromise — it is the dominant model for QES today. Even before the EUDI wallet, most QES in the EU were created using remote QSCDs. Services like DocuSign (through its European qualified signing infrastructure), Swisscom, InfoCert, and D-Trust all operate remote QSCDs. The wallet extends this existing model to a new client (the smartphone) but does not fundamentally change the architecture.

The Cloud Signature Consortium (CSC) API — the industry-standard REST API for remote signing — is being adapted for wallet integration. If your application already supports CSC-based remote signing (Chapter 22), adapting it for wallet-initiated signing is a relatively small change.

29.6.3 The Sole Control Assurance Problem

eIDAS requires that the signing key be under the signatory’s ”sole control” (Article 26(c)). When the key is on a smart card in the signer’s wallet, sole control is straightforward: the signer possesses the card and knows the PIN.

When the key is in a remote QSCD, sole control is more complex. The signer does not physically possess the key. The key is in an HSM in a data center, managed by a QTSP. Sole control must be established through authentication mechanisms that ensure only the authorized signer can trigger a signing operation with that key.

The SCAL (Sole Control Assurance Level) framework, defined in CEN EN 419 241-1, distinguishes two levels:

- **SCAL1**: The QTSP relies on an external authentication mechanism (e.g., the wallet app’s biometric authentication) to authorize signing. The QTSP trusts the authentication assertion from the wallet.
- **SCAL2**: The QTSP independently verifies the signer’s authorization — the signer must authenticate directly to the QTSP’s infrastructure (e.g., through a separate OTP sent by the QTSP), in addition to any wallet-level authentication.

SCAL2 is required for QES. This means the remote QSCD must have its own authentication of the signer, not just trust the wallet’s claim that the user authenticated.

In practice, this is implemented through a combination of: - Wallet-level biometric authentication (proves the user is holding the phone) - A cryptographic challenge from the QSCD that the wallet’s secure element signs (proves the specific wallet instance is authorized) - An additional PIN or biometric confirmation presented to the QSCD’s authentication service (proves the user explicitly authorized *this specific signing operation*)

The user experience goal is to make this feel like a single authentication step (touch fingerprint, confirm signing), while the underlying protocol satisfies SCAL2’s multi-factor requirement.

29.7 Impact on PDF Signing

29.7.1 What Changes

The EUDI wallet does not change the PDF signature format. A PDF signed through the wallet contains the same structures as any other PAdES signature: a `/Sig` dictionary with `/SubFilter` `ETSI.CAdES.detached`, a `/ByteRange` covering the file, a `/Contents` value containing CMS `SignedData`, and (for PAdES-B-LTA) a DSS dictionary with certificates and revocation data.

What changes is the *volume*, the *source*, and the *validation path*.

Volume. Free QES for 450 million EU citizens will produce orders of magnitude more qualified signatures than the current market. Your validation code must handle this volume. If your validation involves synchronous EUTL lookups for every signature, you need caching. If your trust store is updated manually, you need automation.

Source. Wallet-generated signatures will come from certificates issued by QTSPs participating in the wallet ecosystem. These QTSPs will be on the EUTL. They may or may not be on Adobe's AATL. If they are not on the AATL, Acrobat users will see yellow warnings on legitimate QES — which will create pressure on Adobe to either auto-consume the EUTL or expand the AATL.

Validation path. Wallet-generated signatures chain through a trust path that includes: the signer's qualified certificate (issued by a QTSP) □ the QTSP's intermediate/root certificate □ the EUTL entry for the QTSP. This is the same chain as any other QES. The novelty is that the signer is a consumer on a smartphone, not an enterprise user with a signing portal.

29.7.2 PAdES-B-LTA from Mobile

The wallet ecosystem targets PAdES-B-LTA as the default profile for wallet-generated signatures. This is the correct engineering decision: wallet-generated signatures will be used for contracts, government submissions, and official documents that require long-term verifiability.

Producing a PAdES-B-LTA signature from a mobile device requires:

1. **Signature creation** (CMS `SignedData` with `ESS-signing-certificate-v2`, via remote QSCD)
2. **Signature timestamp** (RFC 3161 `TimeStampToken` from a TSA, embedded as unsigned CMS attribute)
3. **DSS population** (certificates, OCSP responses, CRLs for the entire chain)
4. **Document timestamp** (a separate signature dictionary with SubFilter `ETSI.RFC3161` covering the entire file including DSS)

Steps 1-2 are performed by the QSCD/QTSP infrastructure. Steps 3-4 are performed by the application that embeds the signature in the PDF — which may be a server-side service, not the mobile device itself.

The typical architecture:

1. The mobile app sends the document hash to the QTSP via the wallet
2. The QTSP signs the hash on the QSCD and obtains a signature timestamp from its TSA

3. The CMS (with timestamp) is returned to the application server
4. The server embeds the CMS in the PDF, populates the DSS, and adds a document timestamp
5. The signed PDF is returned to the user or submitted to the relying party

The mobile device never needs to perform the PDF manipulation itself — that is handled by the server. This is a practical necessity: PDF signing with LTV data is computationally modest but requires PDF parsing libraries that are not available on all mobile platforms.

```
/**
 * Server-side method that completes a wallet-initiated signing operation.
 * The wallet has obtained a CMS SignedData from the remote QSCD;
 * this method embeds it in the PDF and adds LTV data.
 */
public byte[] completeWalletSignature(byte[] pdfBytes,
                                      byte[] cmsSignedData,
                                      List<X509Certificate> chain) throws Exception {

    PDDocument doc = Loader.loadPDF(pdfBytes);

    // Create signature and set CMS from wallet/QSCD
    PDSignature signature = new PDSignature();
    signature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
    signature.setSubFilter(PDSignature.SUBFILTER_ETSI_CADES_DETACHED);
    signature.setSignDate(Calendar.getInstance());

    // Add signature with pre-built CMS
    doc.addSignature(signature, content -> cmsSignedData);

    // Save incrementally
    ByteArrayOutputStream baos = new ByteArrayOutputStream();
    doc.saveIncremental(baos);
    byte[] signedPdf = baos.toByteArray();
    doc.close();

    // Add DSS with LTV data (separate incremental save)
    signedPdf = addDssWithLtvData(signedPdf, chain);

    // Add document timestamp (another incremental save)
    signedPdf = addDocumentTimestamp(signedPdf);

    log.info("Wallet signature completed: PAdES-B-LTA, {} bytes", signedPdf.length);
    return signedPdf;
}
```

The code above is simplified for illustration. A production implementation must handle: pre-allocating the `/Contents` placeholder, coordinating the byte range computation with the CMS injection, error handling for TSA timeouts, retry logic for remote QSCD communication failures, and verification of the completed signature. Chapter 29 covers the full PDFBox signing pipeline.

29.8 Pilot Programs

29.8.1 The Large-Scale Pilots

The European Commission is funding four Large-Scale Pilots (LSPs) that test the EUDI wallet in real-world scenarios across multiple member states. Each pilot covers different use cases and involves different member states.

Table 29.5: EUDI wallet Large-Scale Pilots (as of early 2026)

Pilot	Lead Countries	Primary Use Cases
POTENTIAL	France, Germany, Italy, Spain	Government services (eID authentication), banking (customer onboarding), mobile driving license, SIM registration, QES signing
NOBID	Norway, Denmark, Sweden, Finland, Iceland, Latvia, Italy	Cross-border payment account opening, PID issuance and verification, age verification, organizational identity
EWC (EU Digital Identity Wallet Consortium)	Sweden, Finland, France, Germany, Czech Republic, Croatia, Italy, Austria, Greece, Romania	Travel (digital travel credentials), education (diploma attestation), digital payments, organizational wallet use cases
DC4EU (Digital Credentials for Europe)	Spain, Austria, Belgium, France, Germany, Italy, Luxembourg, Poland, Romania, Sweden	Education (European Digital Credentials for Learning), social security (Professional Qualifications, Portable Document A1)

29.8.2 What the Pilots Are Testing

The signing-relevant findings from the pilots (as of early 2026, with pilots still running):

QES from mobile works. The core signing flow — user authentication, hash submission to remote QSCD, CMS return, PDF embedding — has been demonstrated successfully across multiple implementations. The user experience typically involves a single biometric confirmation (fingerprint or face recognition) with a clear ”sign this document” prompt.

Cross-border validation is the challenge. Creating the signature is (relatively) straightforward. Validating it across borders — ensuring that a signature created by a French citizen’s wallet validates correctly in a German government portal — requires all parties to consume the EUTL, support the same PAdES profiles, and handle certificate chains from unfamiliar QTSPs. This is exactly the same interoperability problem discussed earlier in this chapter, amplified by scale.

TSA integration needs standardization. Different wallet implementations use different TSAs, with different timestamp token sizes, different certificate chains, and different response times. The pilots have exposed edge cases where timestamp tokens from one TSA cause validation issues in validators that are not prepared for the TSA’s specific certificate chain

length.

User comprehension is limited. In pilot user testing, most participants understood ”sign this document” but did not understand what a qualified electronic signature *is*, how it differs from a basic electronic signature, or why it has legal equivalence to a handwritten signature. This is not a technical problem, but it has technical consequences: if users do not understand the significance of QES, they may not treat the signing authentication with appropriate care.

29.9 What Developers Need to Prepare

29.9.1 If You Build Signing Software

1. **Support wallet-initiated signing flows.** The wallet communicates with your application via OpenID4VP (OpenID for Verifiable Presentations). You will need to implement the RP (Relying Party) side of the OpenID4VP protocol to receive signing requests from the wallet.
2. **Handle remote QSCD signatures.** Your application will receive a CMS `SignedData` blob from the QTSP, not generate it locally. Your signing pipeline must support ”external CMS” injection — computing the byte range, sending the hash for external signing, receiving the CMS, and embedding it in the PDF.
3. **Pre-allocate generously.** Wallet-generated CMS may include longer certificate chains than you are accustomed to (government PKI chains are often deeper than commercial CA chains). Pre-allocate at least 64 KB for PAdES-B-LTA.
4. **Add DSS and document timestamps server-side.** The mobile device will not populate the DSS or add document timestamps. Your server must do this, which means your server needs access to OCSP responders and CRL distribution points for the signer’s certificate chain, plus a TSA for the document timestamp.

29.9.2 If You Build Validation Software

1. **Consume the EUTL programmatically.** Wallet-generated signatures chain to QTSPs on the EUTL. If your validator uses a static trust store, it will reject these signatures. Implement automated EUTL consumption — fetch the LOT (List of Trusted Lists), parse each member state’s trusted list, extract the trust anchors, and update your trust store on a regular schedule (daily is typical).
2. **Cache the EUTL.** With millions of wallet-generated signatures to validate, you cannot fetch the EUTL for every validation. Cache the parsed trust anchors with a TTL (time-to-live) and refresh periodically.
3. **Support deep certificate chains.** Government PKI chains can be 4-5 certificates deep (signer intermediate 1 intermediate 2 root). Ensure your chain-building code handles this without performance degradation.
4. **Report the PAdES profile level.** Wallet-generated signatures should be PAdES-B-LTA. If a wallet-generated signature is missing LTV data (B-T or B-B instead of B-LTA), this indicates a problem in the signing pipeline. Report the profile level to help the signer or their application diagnose the issue.
5. **Prepare for QEAA validation.** If your application needs to verify not just *who signed* but *what qualifications the signer has* (e.g., a medical prescription signed by a doctor), you will need to validate QEAAs in addition to the PDF signature. This is a separate

validation path: the QEAA is a signed credential (SD-JWT or mdoc) that you verify against the issuing QTSP's certificate on the EUTL.

29.9.3 If You Build Enterprise Systems

1. **Update your trust store management.** If your enterprise trust store is managed manually (a Java `cacerts` file, a Windows certificate store), automate it. Wallet-generated signatures will come from QTSPs that may not be in your current trust store.
2. **Budget for EUTL integration.** Consuming the EUTL is not trivial: the LOT and individual trusted lists are XML documents signed with specific certificates, using a specific schema (ETSI TS 119 612). The EU DSS framework (Chapter 23) provides a reference implementation, but integrating it into your architecture requires engineering effort.
3. **Plan for volume.** If your system currently validates hundreds of qualified signatures per day, prepare for thousands or tens of thousands. The wallet will democratize QES, and your systems will see the increase.
4. **Test with pilot certificates.** The EUDI wallet pilots issue test certificates that can be used for integration testing. Engage with the pilot programs relevant to your member state and obtain test credentials for your validation pipeline.

29.10 The Open Questions

The EUDI wallet is ambitious, unprecedented, and not yet fully defined. Several significant questions remain open as of early 2026:

Will users actually adopt it? Mandating reliance parties to accept the wallet creates demand, but the wallet still needs a compelling user experience. If the signing flow feels cumbersome — multiple authentication steps, slow QSCD response times, confusing consent screens — users will avoid it.

Will Apple and Google cooperate? The wallet runs on smartphones, which means it needs integration with iOS and Android. Both platforms control access to NFC, secure elements, and biometric APIs. The EU Digital Markets Act (DMA) requires gatekeepers to allow third-party wallet apps to access device hardware, but the implementation details are being negotiated, and the mobile platforms have commercial interests in their own wallet ecosystems (Apple Pay, Google Pay).

What happens with AATL? If hundreds of millions of qualified signatures are created through the wallet but the signing certificates are not in Adobe's AATL, Acrobat users will see yellow warnings on legitimate QES. This creates political pressure on Adobe to either accept the EUTL as a trust anchor by default or face regulatory action. The outcome will shape the PDF signature ecosystem for years.

Will the credential format converge? The current ARF supports both SD-JWT and mdoc. Supporting two formats doubles the implementation effort for relying parties. The industry is watching to see if one format emerges as dominant or if the dual-format approach persists.

How will cross-border QES work in practice? The technical framework is defined. The legal framework mandates mutual recognition. But whether a German government portal will actually accept a Romanian wallet-generated QES without friction — that depends

on whether all the implementation details are correct across two different member state implementations, two different QTSP certificate chains, and two different validation stacks.

The answers to these questions will determine whether the EUDI wallet becomes the most significant development in digital signing since eIDAS itself — or another ambitious European digital project that works in theory and disappoints in practice.

The EUDI wallet is a moving target. The ARF is being updated, implementing acts are being published, and pilots are generating findings that feed back into the specifications. If you are reading this in 2027 or later, some of the specifics in this chapter may have changed. The architectural principles — remote QSCD, selective disclosure, EUTL-based trust, PAdES-B-LTA as the target profile — are stable. The protocol details and credential format choices may have evolved.

29.11 Key Takeaways

- **eIDAS 2.0 (Regulation (EU) 2024/1183 (eIDAS 2.0))** mandates that every EU member state offer a digital identity wallet to citizens by end of 2026, with QES creation capability included
- **The EUDI wallet** is a smartphone application that stores government-issued identity data (PID) and qualified attribute attestations (QEAAs) and can create qualified electronic signatures
- **Remote QSCD** is the signing architecture: the private key lives in a certified HSM operated by a QTSP, not on the phone itself — the wallet provides authentication and sole control
- **Selective disclosure** (via SD-JWT or mdoc) allows users to share only the attributes a relying party needs, without revealing their full identity
- **Free QES for citizens** (Article 45e) will dramatically increase the volume of qualified signatures — plan your infrastructure for orders-of-magnitude growth
- **PAdES-B-LTA is the target profile** for wallet-generated signatures — server-side DSS population and document timestamping are required
- **EUTL consumption is essential** — wallet-generated signatures chain to QTSPs that may not be in your current trust store; automate EUTL updates
- **Four Large-Scale Pilots** (POTENTIAL, NOBID, EWC, DC4EU) are testing the wallet across sectors and member states; engage with them for test credentials
- **Open questions remain** about user adoption, platform cooperation (Apple/Google), AATL integration, credential format convergence, and real-world cross-border validation
- **The architectural principles are PDF-native** — if you understand PAdES, certificate chains, and the EUTL, you understand the cryptographic foundation of the EUDI wallet

Chapter 30

Code, Tools & Implementation

Chapter 31

The Java PDF Library Landscape

Before we write a single line of signing code, we need to talk about the tools we're using — where they came from, why they exist, and what baggage they carry. If you've ever wondered why there are three different "iText" libraries on Maven Central, why PDFBox's API changed so dramatically between version 2 and version 3, or why a library from the year 2000 with known bugs is still running in Fortune 500 production systems — this chapter is for you.

The Java PDF ecosystem is not just a collection of libraries. It is a story of open-source idealism, licensing betrayal (depending on who you ask), corporate acquisitions, community forks, and the kind of technical debt that accumulates when enterprises build critical infrastructure on a dependency and then the rules change. Understanding this history is not optional background reading — it directly affects the code you write, the signatures you produce, and the compatibility problems you will encounter.

31.1 The iText Saga: From PhD Project to Corporate Licensing Drama

31.1.1 Bruno Lowagie and the Birth of iText (2000–2009)

The story of Java PDF processing begins, for practical purposes, with Bruno Lowagie. In 2000, Lowagie — a Belgian mathematician working at a printing company — needed to generate PDF documents from Java. The options at the time were essentially nonexistent. Adobe had published the PDF specification, but the only tools that could create PDFs were Adobe's own products (Acrobat, Distiller) and a handful of expensive commercial libraries. There was nothing usable in the open-source Java world.

So Lowagie wrote his own library. He called it **iText**, released it under the **Mozilla Public License (MPL)** and later the **GNU Lesser General Public License (LGPL)**, and published it on SourceForge — the GitHub of its era. The LGPL license was critically important: it allowed companies to use iText in proprietary software without open-sourcing their own code, as long as they didn't modify the iText library itself. This is the license model that made iText ubiquitous.

The timing was perfect. Java was exploding in enterprise environments. J2EE application servers were the dominant architecture. Companies needed to generate invoices, reports, contracts, and forms as PDFs — millions of them per day, in automated workflows. iText was free, it worked, and its API was straightforward:

```
// iText 2.x — the classic API that a generation of Java developers learned
Document document = new Document();
PdfWriter.getInstance(document, new FileOutputStream("output.pdf"));
document.open();
document.add(new Paragraph("Hello, PDF"));
document.close();
```

This simplicity was revolutionary. You didn't need to understand PDF internals. You didn't need to manipulate dictionaries or worry about cross-reference tables. You told iText what you wanted on the page, and it produced a valid PDF. The library handled fonts, images, tables, forms, encryption, and — crucially for our purposes — **digital signatures**.

By the mid-2000s, iText was everywhere. It was embedded in JBoss, Alfresco, Jasper Reports, BIRT, and hundreds of other Java projects. Large banks, insurance companies, government agencies, and healthcare organizations had built their document generation pipelines on iText. The Maven coordinates `com.lowagie:itext` were among the most downloaded artifacts in the Java ecosystem.

31.1.2 The LGPL-to-AGPL Switch: The Day the Rules Changed (2009)

On December 1, 2009, Bruno Lowagie changed the license of iText from **LGPL** to **AGPL** (GNU Affero General Public License), starting with version **5.0.0**.

This was, depending on your perspective, either a reasonable business decision or an act of betrayal against the community that had made iText successful. Both perspectives have merit, and understanding the controversy matters because its consequences are still playing out in production systems today.

What AGPL means in practice: The AGPL is like the GPL, but with one additional requirement: if you use an AGPL-licensed library in software that users interact with over a network (which includes web applications, SaaS products, and API services), you must release the *entire source code* of your application under the AGPL. Not just your modifications to iText — your entire application.

For the thousands of companies using iText in their proprietary Java backends — generating invoices, processing insurance claims, signing contracts — this was devastating. They had exactly three options:

1. **Open-source their entire application** under AGPL — unthinkable for most enterprises
2. **Buy a commercial license** from iText Group NV (the company Lowagie founded to monetize iText) — expensive, typically €5,000–€50,000+ per year depending on deployment scale
3. **Stay on iText 2.1.7** — the last LGPL-licensed version, released in 2009 — forever

Most chose option 3.

Why Lowagie made the switch: Lowagie's stated reasons were straightforward: he had spent nearly a decade maintaining a complex library for free, while companies worth billions used it to generate revenue without contributing back. He wanted to build a business. The AGPL created a forcing function: use iText for free in open-source projects, or pay for a commercial license. This is a perfectly legitimate business model — MongoDB, Elasticsearch, and Redis would later make similar moves (though with different licenses).

Why the community was angry: The anger wasn't about the business model itself — it was about the trust violation. Enterprise architects had chosen iText *because* of its LGPL license. They had written it into procurement documents, gotten legal approval, and built systems that depended on it. The license change felt like a bait-and-switch: attract an enormous user base with a permissive license, then change the terms once everyone is locked in.

The fact that existing versions remained LGPL was cold comfort. iText 2.1.7 was already showing its age in 2009. By 2012, it had known bugs in font handling, incomplete PDF 1.7 support, and no support for the newer CMS structures that PAdES was beginning to require. But upgrading to iText 5.x meant either paying or going AGPL.

If you encounter a PDF signed by a Java application that uses `com.lowagie.text.pdf.PdfSignatureAppeara` in its imports — that's iText 2.x. The class was renamed to `com.itextpdf.text.pdf.PdfSignatureAppearanc` in iText 5.x (note the package change from `com.lowagie` to `com.itextpdf`). The package name tells you which era of iText produced the signature.

31.1.3 The CMS Structure Problem: Why Old iText Signatures Look Different

Here's where the licensing drama directly impacts signature compatibility.

iText 2.x (the LGPL version that much of the world is still using) produces CMS `SignedData` structures that are technically valid per RFC 5652 — Cryptographic Message Syntax (CMS), but differ from what modern validators expect in several ways:

Encapsulated vs. Detached content: Early iText versions could produce CMS signatures with *encapsulated* content — meaning the signed hash was embedded inside the `SignedData` structure's `encapContentInfo` field rather than being referenced externally via the PDF byte range. This was not wrong per the CMS spec, but it created problems:

- Modern PAdES validators expect detached signatures (SubFilter `adbe.pkcs7.detached` or `ETSI.CAdES.detached`), where the signed content is the byte range in the PDF itself — not a copy inside the CMS
- Some validators would see encapsulated content and assume the signature was standalone (not bound to the PDF), failing validation even though the cryptographic math was correct
- The double-hash problem: if the signed bytes in the CMS didn't match what a byte-range-aware validator computed from the PDF file, the signature appeared invalid

Missing signed attributes: iText 2.x did not always include the `ESS-signing-certificate-v2` attribute that PAdES requires. Without this attribute, the signature is valid per ISO 32000-2 but *not* PAdES-compliant — which means it fails validation in any system that enforces PAdES profiles (including EU government validation services and Adobe Acrobat in its stricter validation modes).

SHA-1 default: iText 2.1.7 defaulted to SHA-1 for the message digest. SHA-1 was already theoretically weakened by 2009 (Wang and Yu's collision attacks were published in 2005), and it was definitively broken in 2017 when the SHAttered attack produced actual colliding files. PDFs signed with iText 2.x before SHA-256 became the norm carry SHA-1 digests, which modern validators increasingly flag or reject.

If your organization has a corpus of PDFs signed between 2005 and 2015 by Java applications, there is a significant chance they were signed with iText 2.x using SHA-1 and potentially using encapsulated (not detached) CMS content. These signatures may fail validation in modern tools.

To verify: extract the CMS SignedData from the `/Contents` entry and check:

1. Is `encapContentInfo.eContentType` set with actual content? (encapsulated — old style)
2. Is the digest algorithm SHA-1 (`OID 1.3.14.3.2.26`)? (deprecated)

3. Is the `ESS-signing-certificate-v2` attribute present in `signedAttrs`? (missing = not PAdES-compliant)

Chapter 31 provides code to perform all three checks.

31.1.4 iText 5.x, 7.x, and 8.x: The Commercial Evolution

After the AGPL switch, iText evolved rapidly — but behind a paywall for commercial use.

iText 5.x (2009–2016): Maintained the original API design but added PDF 2.0 features, improved CMS handling, and better PAdES support. The package name changed from `com.lowagie` to `com.itextpdf`. If you see both package prefixes in the same project's classpath — and this happens more often than you'd think — you're looking at a project that tried to use both the old LGPL version and the new AGPL version simultaneously, which is a recipe for classloading nightmares.

iText 7.x (2016–2023): A complete rewrite with a modular architecture. The API was entirely different from iText 5.x — migration was not trivial. iText 7 introduced separate modules for core PDF operations, forms, signing, and other features. The signing module (`itext7-sign`) included comprehensive PAdES support, including B-LTA profiles with embedded timestamps and revocation data. The commercial license became the primary offering, with AGPL remaining for open-source projects.

iText 8.x (2023–present): The current generation, continuing the modular architecture with added support for PDF 2.0 features, improved accessibility tagging, and enhanced signing capabilities. iText Group NV (now iText Software, headquartered in Ghent, Belgium, with offices in the US and Singapore) is a substantial commercial entity with paying enterprise customers worldwide.

The commercial iText is, by all accounts, a high-quality library. Its signing implementation is mature, well-tested, and PAdES-compliant. But at commercial license prices that can reach tens of thousands of euros per year for enterprise deployments, it is not an option for every project — which is why the open-source alternatives exist.

31.1.5 The IBM Connection

IBM's involvement with iText is a lesser-known but important part of the story. In the early 2000s, IBM was one of the largest users of iText — embedding it in several of their enterprise products, including **IBM Content Manager**, **IBM Filenet**, and various WebSphere-based document processing systems.

IBM contributed patches and improvements back to the project during the LGPL era, which was entirely within the LGPL's collaborative spirit. However, when the license changed to AGPL, IBM — like most large enterprises — was unwilling to either open-source their proprietary products or pay iText's commercial licensing fees for a dependency that had previously been free.

This created an awkward situation: IBM's products contained iText 2.x code, their customers expected continued PDF functionality, and the upgrade path required either legal procurement (commercial iText license) or engineering investment (migrate to a different library). IBM's response, like most enterprises in this position, was a combination of staying on old

versions and gradually migrating new development to Apache PDFBox — which was, by then, becoming mature enough for production use.

The IBM story is representative of what happened across the industry: hundreds of enterprises had iText deeply embedded in their systems, and the license change created a slow-motion migration that, in many cases, is still incomplete in 2026.

31.2 OpenPDF: The Community Fork That Refuses to Die

31.2.1 The Fork (2016)

When it became clear that iText 5.x was permanently AGPL for commercial use, and that iText 2.1.7 was accruing unfixed bugs and missing modern features, a group of developers forked iText 2.1.7 under the name **OpenPDF** (initially called **LibrePDF**).

The fork was hosted on GitHub under the `LibrePDF/OpenPDF` repository, and it was explicitly positioned as the LGPL-licensed continuation of the original iText codebase. The initial stated goals were modest: fix bugs, add basic modern features (SHA-256 support, better font handling), and keep the library usable for projects that needed LGPL licensing.

31.2.2 What OpenPDF Is (and Isn't)

OpenPDF today (version 2.0.x, as of 2026) is a capable PDF generation library that maintains API compatibility with iText 2.x while adding features that the original 2.1.7 never had:

- **SHA-256/384/512 message digests** for signing (critical — SHA-1 is no longer acceptable)
- **AES-256 encryption** support
- **Improved font handling**, including better CJK support and TrueType subsetting
- **Bug fixes** for dozens of issues that were never patched in iText 2.1.7

However, OpenPDF also carries significant limitations:

- It is built on the iText 2.x architecture, which was designed for PDF 1.7 and earlier
- **No PDF 2.0 support** — cross-reference streams, tagged PDF improvements, and other 2.0 features are absent
- **Limited PAdES support** — basic PAdES-B-B signatures are possible, but B-LT and B-LTA profiles (which require DSS dictionary construction and archival timestamps) are not natively supported
- **No built-in LTV (Long-Term Validation)** — you'll need to construct the DSS dictionary yourself
- The contributor community is small — a handful of maintainers, not the large team that iText Group NV employs
- **Documentation is sparse** — the iText 2.x book (Lowagie's "iText in Action") is the closest thing to comprehensive documentation, and it describes an API from 2008

OpenPDF lives at `com.github.librepdf:openpdf` on Maven Central. Do not confuse it with `com.lowagie:itext` (the original iText 2.1.7) or `com.itextpdf:itextpdf` (the AGPL iText 5.x). All three exist on Maven Central. All three have different licenses, different capabilities, and different bugs. Getting the wrong one in your dependency tree is a common source of confusion — and potential legal liability.

31.2.3 Should You Use OpenPDF?

For **PDF generation** (creating invoices, reports, forms) where you need an LGPL-licensed library and don't require PDF 2.0 features: OpenPDF is a reasonable choice.

For **digital signatures**: OpenPDF can produce basic CMS signatures, but for PAdES compliance, long-term validation, and the kind of robust signing infrastructure this book teaches, **PDFBox is the better choice**. PDFBox's signing API is more actively maintained, better tested against the Bochum attack scenarios, and has a larger community contributing security fixes.

That said — if you have an existing codebase built on iText 2.x that you cannot easily migrate, OpenPDF is the path of least resistance for getting off the unmaintained 2.1.7 version while keeping your existing API calls intact.

31.3 Apache PDFBox: The Open-Source Standard

31.3.1 Origin Story: Ben Litchfield and the Apache Incubation

Apache PDFBox started as **PDFBox** — a standalone project created by **Ben Litchfield** in 2002. Litchfield, working as a Java developer, needed to *read* existing PDF documents — not just create them (which was iText's primary strength at the time). He needed to extract text, parse structures, and manipulate the low-level PDF object model. iText was designed as a PDF *writer*; PDFBox was designed as a PDF *reader* that could also write.

This fundamental design difference — reader-first vs. writer-first — is still visible in the APIs today:

- **iText/OpenPDF** excels at high-level document construction: "add a paragraph here, put a table there, generate the PDF"
- **PDFBox** excels at low-level document manipulation: "give me the COS dictionary for object 42, let me modify the annotation rectangle, show me the raw font encoding"

For digital signature work, PDFBox's reader-first architecture is a significant advantage. Signing a PDF requires understanding the existing document structure, computing byte ranges, parsing existing signatures, and manipulating specific dictionaries — all tasks where low-level access is essential.

In 2008, PDFBox entered the **Apache Software Foundation** incubator process, and in 2009 it graduated as a top-level Apache project under the **Apache License 2.0** — one of the most permissive open-source licenses available. Unlike iText's LGPL□AGPL journey, PDFBox has been Apache-licensed since inception and will remain so. This licensing certainty is one of the reasons enterprises migrated to PDFBox.

31.3.2 PDFBox 2.x: The Workhorse (2016–2023)

PDFBox 2.0 was the version that brought PDFBox into mainstream production use. It featured:

- A complete COS (Carousel Object System) model for low-level PDF manipulation
- Text extraction with position information
- PDF rendering (via `PDFRenderer`)

- AcroForm support (creating, filling, and flattening forms)
- Digital signature support via `PDDocument.addSignature()`
- Font handling (Type1, TrueType, CID fonts, subsetting)

The signing API in PDFBox 2.x was functional but had rough edges. The `SignatureInterface` required you to provide a `sign(InputStream)` method that received the bytes to sign and returned the CMS `SignedData` as a byte array. The actual CMS construction was your responsibility — typically using BouncyCastle (we'll discuss this later). PDFBox handled the PDF-specific parts: creating the signature dictionary, computing the byte range, reserving space for `/Contents`, and performing the incremental save.

```
// PDFBox 2.x signing pattern
PDDocument doc = PDDocument.load(inputFile);
PDSignature signature = new PDSignature();
signature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
signature.setSubFilter(PDSignature.SUBFILTER_ADBE_PKCS7_DETACHED);
signature.setName("Signer Name");
signature.setSignDate(Calendar.getInstance());

doc.addSignature(signature, new SignatureInterface() {
    @Override
    public byte[] sign(InputStream content) throws IOException {
        // Your BouncyCastle CMS construction goes here
        return createCMSSignature(content, certificate, privateKey);
    }
});

doc.saveIncremental(outputStream);
doc.close();
```

This separation of concerns — PDFBox handles the PDF, you handle the cryptography — is elegant in theory but demanding in practice. Getting the CMS structure right for PAdES compliance requires deep knowledge of signed attributes, certificate embedding, and timestamp handling. Chapters 29 and 30 provide the complete implementation.

31.3.3 PDFBox 3.x: The Breaking Change (2024)

PDFBox 3.0 was released in 2024 as a major version bump with **breaking API changes**. The migration was significant enough to catch many projects off guard:

PDFBox 3.x is **not** a drop-in replacement for 2.x. If you're upgrading, budget significant development time. The changes are pervasive and affect every area of the API.

The most impactful changes for signing:

Loading documents:

```
// PDFBox 2.x
PDDocument doc = PDDocument.load(new File("document.pdf"));
PDDocument doc = PDDocument.load(byteArray);

// PDFBox 3.x — PDDocument.load() is GONE
PDDocument doc = Loader.loadPDF(new File("document.pdf"));
PDDocument doc = Loader.loadPDF(byteArray);
```

Font construction:

```
// PDFBox 2.x
PDType1Font font = PDType1Font.HELVETICA;

// PDFBox 3.x — static font constants are GONE
PDType1Font font = new PDType1Font(Standard14Fonts.FontName.HELVETICA);
```

AcroForm API:

```
// PDFBox 2.x
boolean needsAppearances = acroForm.isNeedAppearances();

// PDFBox 3.x — method renamed
boolean needsAppearances = acroForm.getNeedAppearances();
```

Font encoding access:

```
// PDFBox 2.x
Encoding encoding = font.getEncoding(); // Worked for most font types

// PDFBox 3.x — getEncoding() removed from PDFont interface
// Must access encoding through COS dictionary directly
COSDictionary fontDict = font.getCOSObject();
COSBase encoding = fontDict.getDictionaryObject(COSName.ENCODING);
```

Content streams:

```
// PDFBox 2.x — could write to form XObjects with PDPageContentStream
PDPageContentStream cs = new PDPageContentStream(doc, formXObject, ...);

// PDFBox 3.x — PDPageContentStream is for pages ONLY
// For form XObjects, write raw PDF operators via the stream's output
OutputStream os = formXObject.getStream().createOutputStream();
```

The signing API itself (`PDDocument.addSignature()`, `SignatureInterface`, `PDSignature`) remained structurally similar, but the surrounding changes — especially in document loading, font handling, and content stream creation — mean that any signing code that also creates visual signature appearances needs substantial rework.

Migration strategy: Don't try to upgrade PDFBox in-place. Create a new branch, update the dependency, let the compiler tell you every break, and fix them systematically. The PDFBox team maintains a migration guide that covers the major changes, but the edge cases (especially around font encoding and content stream APIs) will require reading the 3.x Javadoc carefully.

31.3.4 The PDFBox Signing Infrastructure: What's Under the Hood

PDFBox's signing implementation is one of the most impressive pieces of open-source PDF engineering. It's worth understanding the high-level architecture, because when something goes wrong during signing — and it will — you need to know where to look.

The signing process in PDFBox involves these internal steps:

1. **Signature dictionary creation**: PDFBox creates the `/Sig` dictionary with the properties you set (`/Filter`, `/SubFilter`, `/Name`, `/M`, etc.)

2. **Space estimation**: PDFBox needs to know how much space to reserve for the `/Contents` hex string *before* the signature is computed — because the signature covers the bytes outside the `/Contents` gap, and the file must be written before the signature can be calculated. The default is 16KB (32,768 hex characters), but PAdES-B-LTA signatures with embedded timestamps and revocation data can easily exceed this.
3. **Byte range computation**: PDFBox calculates the `/ByteRange` array — the two regions of the file that the signature covers (everything before and after the `/Contents` value)
4. **Incremental save**: The modified PDF is written as an incremental update — appended to the original file, preserving previous revisions
5. **Signing callback**: PDFBox passes the byte range data (as an `InputStream`) to your `SignatureInterface.sign()` implementation
6. **Signature injection**: The returned CMS bytes are hex-encoded and written into the reserved `/Contents` space

If the CMS signature is larger than the reserved space, the signing fails. If it's smaller (which is normal), the remaining space is filled with zeros — the zero-padding that you see in the hex dump of any signed PDF's `/Contents` entry.

This architecture means that **PDFBox does not construct the CMS signature itself**. It provides the bytes to sign, and you provide the signature. This is both a strength (maximum flexibility — you can use any CMS library, any algorithm, any certificate) and a burden (you must get the CMS right yourself). That's why the next chapter is dedicated entirely to BouncyCastle CMS construction.

31.4 The Legion of the Bouncy Castle

31.4.1 Why BouncyCastle Exists

Java has a built-in cryptographic provider — the `SunJSSE` and `SunJCE` providers that ship with every JDK. So why does almost every Java PDF signing project use BouncyCastle instead?

Three reasons:

1. **CMS support**: The JDK's built-in providers do not include a CMS (`SignedData`) implementation. They provide the raw cryptographic primitives (RSA, ECDSA, SHA-256, etc.), but not the higher-level CMS container format that PDF signatures require. BouncyCastle's `bcpkix` module provides `CMSSignedDataGenerator`, `CMSSignedData`, and all the ASN.1 infrastructure needed to construct and parse CMS structures.
2. **Algorithm breadth**: BouncyCastle supports algorithms that the JDK providers don't — including EdDSA (Ed25519, Ed448), GOST, SM2, and various other national or emerging algorithms. While PDF signatures currently use RSA and ECDSA almost exclusively, the ability to work with less common algorithms is valuable for government and specialized applications.
3. **ASN.1 library**: BouncyCastle's ASN.1 library (`bcutil` and `bcprov`) is the de facto standard for Java ASN.1 encoding/decoding. When you need to parse a certificate, construct a signed attribute, or decode an OCSP response, BouncyCastle's ASN.1 classes are what you use.

31.4.2 The BouncyCastle Ecosystem

BouncyCastle is not a single JAR — it's a family of related libraries:

Table 31.1: BouncyCastle modules relevant to PDF signing

Module	Maven Artifact	Purpose
Provider	`bcprov-jdk18on`	Core cryptographic provider: symmetric/asymmetric ciphers, hashes, key generation, random number generation. Register via `Security.addProvider(new BouncyCastleProvider())`
PKIX/CMS/EAC	`bcpkix-jdk18on`	CMS `SignedData` construction/parsing, PKCS#10 certificate requests, OCSP, timestamp protocol (RFC 3161), certificate path validation. **This is the module you need for PDF signing.**
Utilities	`bcutil-jdk18on`	PEM reader/writer, ASN.1 dump utilities, base64 encoding. Pulled in as a transitive dependency.

You must explicitly register the BouncyCastle provider before using it. If you forget this step, you'll get cryptic `NoSuchAlgorithmException` errors at runtime.

```
import org.bouncycastle.jce.provider.BouncyCastleProvider;
import java.security.Security;

// Do this once at application startup
if (Security.getProvider("BC") == null) {
    Security.addProvider(new BouncyCastleProvider());
}
```

In Spring Boot applications, put this in a `@Configuration` class with a `@PostConstruct` method. In standalone applications, put it in your `main()` method. Forgetting this registration is the #1 cause of "it works in my IDE but fails in production" signing issues.

31.4.3 BouncyCastle Version History and the `-jdk18on` Suffix

BouncyCastle's artifact naming has changed over the years, which causes dependency confusion:

- `bcprov-jdk15` — for JDK 1.5 (ancient, do not use)
- `bcprov-jdk15on` — for JDK 1.5 and later (the long-lived artifact ID)
- `bcprov-jdk18on` — for JDK 1.8 (Java 8) and later (current, use this)

The `jdk18on` variants (introduced with BouncyCastle 1.71) dropped support for Java versions before 8 and added Java module system (`module-info.java`) support. If you're on Java 11 or later (and for PDF signing with PDFBox 3.x, you should be on Java 17+), use the `jdk18on` artifacts exclusively.

Do not mix `jdk15on` and `jdk18on` artifacts in the same classpath. BouncyCastle uses a provider registration model, and having two versions of the provider registered simultaneously

causes unpredictable failures — the wrong version may handle a given algorithm, producing incompatible ASN.1 structures.

31.5 EU DSS: The European Commission's Signing Framework

31.5.1 What EU DSS Is

There is one more library that deserves its own section — not because it replaces PDFBox or BouncyCastle, but because it *builds on top of them* and adds a layer that would take months to implement yourself.

EU DSS (Digital Signature Services) is an open-source Java framework developed and maintained by **Nowina Solutions** under contract from the **European Commission**. It is the reference implementation for creating and validating digital signatures that comply with the European eIDAS regulation — which means it implements the full PAdES, CAdES, XAdES, and JAdES signature profiles, including the complex long-term validation (LTV) workflows that individual libraries like PDFBox cannot do alone.

The project lives at `github.com/esig/dss` and is released under the **LGPL 2.1** license. The Maven coordinates follow the pattern `eu.europa.ec.joinup.sd-dss:dss-*`.

31.5.2 Why EU DSS Matters for PDF Signatures

If your application needs to produce **PAdES-B-LTA** signatures — the gold standard for long-term archival signatures that remain verifiable for decades — you have two choices:

1. **Build it yourself**: Use PDFBox for the PDF layer, BouncyCastle for CMS construction, write your own timestamp client (RFC 3161), implement OCSP and CRL fetching, construct the DSS (Document Security Store) dictionary, embed the VRI (Validation Related Information), and manage archival timestamp chains. This is what Chapters 29–31 of this book teach. It works. It is also hundreds of hours of development and testing.
2. **Use EU DSS**: Import the `dss-pades` module, configure a `PAdESSignatureParameters` object, provide a `SignatureTokenConnection` (which wraps your certificate and private key), and let the framework handle everything — CMS construction, signed attribute inclusion, timestamp requests, revocation data embedding, DSS dictionary creation, and incremental save.

Here's what a PAdES-B-LTA signature looks like with EU DSS:

```
// EU DSS — PAdES-B-LTA signing (simplified)
PAdESSignatureParameters params = new PAdESSignatureParameters();
params.setSignatureLevel(SignatureLevel.PAdES_BASELINE_LTA);
params.setDigestAlgorithm(DigestAlgorithm.SHA256);
params.setSigningCertificate(new CertificateToken(certificate));
params.setCertificateChain(Arrays.asList(intermediateCerts));

// TSA (Timestamp Authority) for the signature timestamp
OnlineTSPSource tspSource = new OnlineTSPSource("http://tsa.example.com/timestamp");
params.setContentTimestampParameters(new TimestampParameters(DigestAlgorithm.SHA256));

PAdESService service = new PAdESService(commonCertificateVerifier);
service.setTspSource(tspSource);
```

```
// Two-step signing: digest, then sign
ToBeSigned dataToSign = service.getDataToSign(documentToSign, params);
SignatureValue signatureValue = signingToken.sign(dataToSign, params.getDigestAlgorithm(), privateKey);
DSSDocument signedDocument = service.signDocument(documentToSign, params, signatureValue);
```

Compare this to implementing the same B-LTA profile manually with PDFBox + BouncyCastle — the EU DSS version handles the timestamp embedding, OCSP/CRL fetching, DSS dictionary construction, and VRI population automatically.

31.5.3 EU DSS Architecture

EU DSS is modular. You don't need to import the entire framework — pick the modules relevant to your signature format:

Table 31.2: EU DSS modules relevant to PDF signing

Module	Purpose
`dss-pades`	PAdES signature creation and extension (B-B B-T B-LT B-LTA). Uses PDFBox internally for PDF manipulation.
`dss-cades`	CAdES signature creation. The PAdES module delegates CMS construction to this.
`dss-document`	Core document model (`DSSDocument`, `InMemoryDocument`, `FileDocument`) and service interfaces.
`dss-service`	Online data loaders — HTTP clients for TSA, OCSP, and CRL fetching.
`dss-tsl-validation`	EU Trusted List (EUTL) parsing and validation. Essential for determining whether a signing certificate is qualified under eIDAS.
`dss-validation-report`	Generates detailed validation reports following ETSI TS 119 102-2 — the same validation report format that EU government validation services produce.
`dss-token`	`SignatureTokenConnection` implementations: PKCS#12 file, PKCS#11 (HSM), JKS keystore, MS-CAPI (Windows certificate store).

31.5.4 EU DSS vs. Rolling Your Own

The honest comparison:

Advantages of EU DSS:

- **Compliance out of the box**: EU DSS is the European Commission's reference implementation. If you use it correctly, your signatures will pass validation on the EU's official validation service (DSS demonstration webapp) and on member-state validation services. This is not a guarantee you get from hand-rolling with PDFBox + BouncyCastle.
- **LTV complexity hidden**: Long-term validation requires fetching OCSP responses, downloading CRLs, embedding them in the PDF's DSS dictionary, creating VRI entries,

and adding archival timestamps. EU DSS does all of this. Implementing it yourself is a significant effort.
- **Trusted List integration**: The `dss-tsl-validation` module can load and parse the EU Trusted List, letting you programmatically determine whether a signing certificate is qualified. Building this yourself means implementing ETSI TS 119 612 XML parsing and certificate matching — nontrivial.
- **Validation reports**: EU DSS generates ETSI-compliant validation reports that are accepted by auditors and compliance officers.

Advantages of rolling your own (PDFBox + BouncyCastle):

- **Full control**: You understand every byte. When something fails, you know exactly where to look because you wrote every layer.
- **No framework weight**: EU DSS pulls in a substantial dependency tree. If you only need PAdES-B-B (baseline signature without timestamps or LTV), you can do it with PDFBox + BouncyCastle in about 200 lines of code — EU DSS adds a megabytes of framework for something achievable with direct library calls.
- **Learning**: Building it yourself teaches you the spec. Using EU DSS teaches you the framework's API. For a book like this one, understanding comes from building.
- **Customization**: EU DSS makes assumptions about how signatures should be constructed. If your use case deviates (unusual SubFilter values, custom signed attributes, non-standard timestamp configurations), you may fight the framework rather than benefit from it.

Our recommendation: Learn to build signatures with PDFBox + BouncyCastle first (Chapters 29–31). Then, when you need PAdES-B-LTA or EU Trusted List integration in production, use EU DSS — you'll understand what it's doing under the hood, and you'll be able to debug it when it behaves unexpectedly.

Chapter 23 provides a complete walkthrough of EU DSS for PAdES signing and validation.

31.5.5 EU DSS Version History

EU DSS follows a rapid release cycle, with major versions roughly annually:

- **DSS 5.x** (2017–2022): The long-lived stable series. Most documentation and blog posts you'll find online reference 5.x APIs.
- **DSS 6.x** (2023–present): The current generation, with Java 17+ as the minimum, updated dependencies (including PDFBox 3.x for the PAdES module), and improved validation reporting. The API changed significantly between 5.x and 6.x — if you're following tutorials, make sure they match your version.

EU DSS 6.x uses PDFBox 3.x internally. If your project also uses PDFBox directly (for example, for document manipulation before signing), you must ensure version alignment. Having DSS pull in PDFBox 3.0.0 while your code uses PDFBox 3.0.1 can cause subtle `NoSuchMethodError` exceptions at runtime. Use Maven's `<dependencyManagement>` to lock the PDFBox version across your entire project.

31.6 Analysis and Debugging Tools

A signing implementation is only as good as your ability to inspect what it produces. The following tools are indispensable for PDF signature development.

31.6.1 RUPS: The PDF Object Explorer

RUPS (Reading and Updating PDF Syntax) is a Java Swing application that lets you browse the internal object structure of a PDF file. It was originally developed by Bruno Lowagie as a companion to iText, and it shows every COS object, dictionary key, stream, and cross-reference entry in the file.

For signature work, RUPS is invaluable because it lets you:

- Inspect the `/Sig` dictionary and all its keys (`/ByteRange`, `/Contents`, `/SubFilter`, `/Filter`)
- Navigate the field tree to see how signature fields are structured
- View the `/AcroForm` dictionary and its `/SigFlags`
- Examine `/Lock` and `/SV` (Seed Value) dictionaries
- Browse incremental updates to see what changed between revisions
- Inspect the raw byte content of the `/Contents` hex string

RUPS is available on GitHub at `LibrePDF/OpenPDF` (it's bundled with the OpenPDF project). It's also available as a standalone download. The interface is spartan — it's a developer tool, not a user-facing product — but once you learn to navigate the tree view, it becomes the fastest way to understand what's actually inside a PDF.

When debugging a signature failure, the first thing you should do is open the PDF in RUPS and check:

1. Does the `/Sig` dictionary exist? (Navigate: Root AcroForm Fields [signature field] V)
2. Is the `/ByteRange` array plausible? (Four integers, the first should be 0, the last should be close to the file size)
3. Is `/Contents` non-empty? (Should be a long hex string, not all zeros)
4. What is the `/SubFilter`? (Should be `adbe.pkcs7.detached` or `ETSI.CAdES.detached`)

If any of these are wrong, you've found your problem before even looking at the CMS.

31.6.2 veraPDF: The Compliance Validator

veraPDF is the industry-standard PDF/A validation tool, developed by the Open Preservation Foundation and the PDF Association. While its primary mission is PDF/A compliance checking, it includes PDF analysis capabilities that are useful for signature work:

- **Policy-based validation** — you can define custom validation profiles that check signature-specific requirements
- **Feature extraction** — enumerate all signatures, certificates, and cryptographic properties in a PDF
- **Structural analysis** — verify that the PDF structure conforms to the specification

veraPDF is available as a CLI tool, a Java library (which you can embed in your validation pipeline), and a GUI application. It's the validator that national archives and government agencies use to verify PDF/A compliance — if your signed PDFs also need to be PDF/A compliant (as many government document workflows require), veraPDF is essential.

31.6.3 QPDF: The Swiss Army Knife

QPDF is a C++ command-line tool (created by Jay Berkenbilt) that can inspect, transform, and repair PDF files at the structural level. It's not Java, but every Java PDF developer should have it installed.

Key capabilities for signature work:

- `qpdf --show-xref` — displays the cross-reference table, showing where every object lives in the file. Critical for understanding byte range issues.
- `qpdf --show-object=N` — displays a specific object by number. Faster than navigating a GUI when you know the object number from an error message.
- `qpdf --check` — validates the PDF's structural integrity. If this reports problems, your signing code has a bug.
- `qpdf --qdf` — produces a "QDF" (QPDF data format) output where the PDF is restructured for human readability, with objects in order and comments showing object numbers. Invaluable for understanding what incremental saves changed.
- `qpdf --linearize` — linearizes the PDF for fast web viewing. **Do not linearize after signing** — it destroys the signature by reorganizing the file.

Several QPDF operations will destroy existing signatures:

- `--linearize` — reorganizes the entire file
- `--normalize-content=y` — reprocesses content streams
- `--object-streams=generate` — restructures object storage
- Any operation that produces a non-incremental output

Always use `--copy-encryption=source.pdf` and check whether the operation preserves incremental structure before running it on a signed PDF.

31.6.4 MuTool (from MuPDF)

MuTool is the command-line companion to MuPDF (Artifex's open-source PDF renderer). Its `mutool show` command provides quick access to PDF objects:

```
mutool show document.pdf trailer          # Show trailer dictionary
mutool show document.pdf xref             # Show cross-reference table
mutool show document.pdf 42 0             # Show object 42, generation 0
mutool show document.pdf outline          # Show document outline/bookmarks
```

MuTool is lightweight and fast — useful for quick inspections when you don't want to launch a GUI tool.

31.6.5 OpenSSL: CMS Verification Outside the PDF Context

For verifying the CMS signature independently of the PDF wrapper, OpenSSL is the standard tool:

```
# Extract the raw signature bytes from /Contents (after hex-decoding)
# and save as signature.der

# Verify the CMS structure
openssl cms -inform DER -in signature.der -noout -cmsout -print

# Verify the signature against the signed data
openssl cms -inform DER -in signature.der -verify \
    -content signed-data.bin \
    -CAfile ca-bundle.pem \
    -out /dev/null
```

This separates the CMS verification from the PDF byte-range logic — if OpenSSL says the CMS is valid but your PDF validator says the signature is invalid, the problem is in the byte range handling, not the cryptography.

31.6.6 ASN.1 Dump Tools

When debugging CMS issues, being able to read the raw ASN.1 structure is essential. Several tools help:

- **`openssl asn1parse`** — parses DER-encoded ASN.1, showing the tag-length-value tree
- **BouncyCastle's `ASN1Dump`** — `org.bouncycastle.asn1.util.ASN1Dump.dumpAsString(obj)` — prints any ASN.1 object as a readable tree from Java code
- **`dumpasn1`** (Peter Gutmann) — the original ASN.1 dumper, still the most detailed output
- **Web-based tools** — several websites parse ASN.1/DER online; useful for quick checks but do not upload production signatures to third-party websites

31.7 Choosing the Right Library for Your Project

After all this history, here is the practical guidance:

31.7.1 Decision Matrix

Table 31.3: Library selection for PDF signature projects

Requirement	PDFBox + BC	EU DSS	OpenPDF	iText 7/8 (AGPL)	iText 7/8 (Commercial)
License	Apache 2.0	LGPL 2.1	LGPL 2.1	AGPL 3.0	Proprietary
Cost	Free	Free	Free	Free (if AGPL-compatible)	€€€€
PAdES-B-B	Yes (manual CMS)	Yes (built-in)	Basic	Yes (built-in)	Yes (built-in)
PAdES-B-LTA	Yes (manual DSS)	Yes (built-in)	No	Yes (built-in)	Yes (built-in)

Requirement	PDFBox + BC	EU DSS	OpenPDF	iText 7/8 (AGPL)	iText 7/8 (Commercial)
EUTL / eIDAS validation	No (manual)	Yes (built-in)	No	Partial	Yes
PDF 2.0	Yes	Yes (via PDFBox)	No	Yes	Yes
Community size	Large (Apache)	Medium (EU-backed)	Small	N/A (AGPL)	Vendor-supported
Signing API maturity	High	Very high	Moderate	High	High
Low-level PDF access	Excellent	Limited (framework)	Limited	Good	Good
Our recommendation	**Learn + custom**	**Production PAdES**	Legacy migration	Open-source projects	Budget permits

31.7.2 Our Recommendation

For the projects this book targets — production signing services, validation pipelines, signature analysis tools — **Apache PDFBox 3.x with BouncyCastle** is the default recommendation:

1. **Apache 2.0 license**: No surprises, no AGPL obligations, no license fees. This will not change.
2. **Low-level access**: Signing requires understanding and manipulating the raw PDF structure. PDFBox's COS model gives you direct access. iText and OpenPDF abstract this away, which is convenient until you need to debug a byte-range computation or inspect a malformed signature dictionary.
3. **Active maintenance**: PDFBox has a dedicated Apache community, regular releases, and responsive handling of security issues. The Bochum attacks were patched quickly.
4. **Separation of concerns**: PDFBox handles the PDF, BouncyCastle handles the crypto. You understand both layers because you built both layers. When something breaks, you know exactly where to look.

The trade-off is that PDFBox makes you build more yourself — especially the CMS layer. That's what Chapters 29–31 are for.

The best way to learn is to build it yourself. Using a library that does everything for you (like commercial iText) gets you to production faster, but leaves you helpless when something goes wrong. With PDFBox + BouncyCastle, you understand every byte of the signature because you put it there. When a customer reports that their signature "shows invalid in Adobe but valid in Foxit," you'll know exactly where to look — because you wrote the code that constructed the CMS, computed the byte range, and injected the signature.

31.8 The Ecosystem at a Glance

To close this chapter, here is a summary of every tool and library we've discussed, organized by function:

Table 31.4: The Java PDF signature ecosystem

Function	Tool/Library	License	Notes
PDF manipulation (read/write)	Apache PDFBox 3.x	Apache 2.0	Default choice for this book. Reader-first architecture.
PDF manipulation (read/write)	OpenPDF 2.x	LGPL 2.1	Community fork of iText 2.x. Good for legacy migration.
PDF manipulation (read/write)	iText 7/8	AGPL / Commercial	Feature-rich but expensive or license-restrictive.
CMS / Cryptography	BouncyCastle 1.80+	MIT (Bouncy Castle License)	De facto standard for Java CMS. Register provider explicitly.
PAdES signing framework	EU DSS 6.x	LGPL 2.1	European Commission reference implementation. PAdES-B-B through B-LTA, validation reports, EU Trusted List integration.
PDF structure inspection	RUPS	LGPL	GUI object browser. First tool to reach for.
PDF/A compliance	veraPDF	Open source (MPL 2.0)	Industry-standard validator. Also useful for general PDF analysis.
PDF structural tools	QPDF	Apache 2.0	CLI Swiss army knife. `--show-xref`, `--qdf`, `--check`.
PDF rendering/inspection	MuTool (MuPDF)	AGPL	Lightweight CLI. Quick object inspection.
CMS verification	OpenSSL	Apache 2.0	CMS verification independent of PDF context.
ASN.1 inspection	BouncyCastle `ASN1Dump`	MIT	Java-based ASN.1 tree dump.
ASN.1 inspection	`dumpasn1` (Gutmann)	Public domain	The classic ASN.1 dumper.

Every tool in this table will appear at some point in the remaining chapters. Install them now — when you're debugging a signature failure at 2 AM, you don't want to be downloading tools.

31.9 Key Takeaways

- **iText's license change** (LGPL AGPL in 2009) split the Java PDF world in two. The consequences — old versions in production, community forks, enterprise migrations — are still visible in 2026.
- **OpenPDF** is the LGPL continuation of iText 2.x. Useful for legacy migration but limited for modern PAdES signing.
- **PDFBox** + **BouncyCastle** is the recommended stack for new signing projects. Apache-licensed, actively maintained, gives you low-level control.
- **PDFBox 3.x** broke API compatibility with 2.x. Budget migration time. Key changes: `Loader.loadPDF()`, new font constructors, content stream API changes.
- **BouncyCastle** provides the CMS layer that PDFBox doesn't. Always register the provider explicitly. Use `jdk18on` artifacts.
- **RUPS, QPDF, veraPDF, OpenSSL** — know your tools. PDF signature debugging is 50% knowing what to look at and 50% having the right tool to look at it.
- **The library you choose is a 10-year decision.** Choose based on license stability, community health, and technical fit — not just today's feature checklist.

Chapter 32

PDFBox Signing — Complete Walk-through

Chapter 28 gave us the lay of the land. Now we write code. This chapter is a complete, production-quality walkthrough of PDF signing with Apache PDFBox 3.0.0 and BouncyCastle 1.80 on Java 21. Every code listing compiles, every class is self-contained, and every pitfall we describe is one we have hit in production — or watched someone else hit at 2 AM on a Friday.

We start with the core PDFBox signing API — `PDDocument.addSignature()`, `SignatureInterface`, and `SignatureOptions` — then build outward: the `PDSignature` dictionary and its fields, visible and invisible signature appearances, multi-signature workflows, external/HSM signing, certification signatures with DocMDP, timestamp embedding, and the migration traps waiting for anyone upgrading from PDFBox 2.x to 3.x.

By the end of this chapter, you will have a complete, production-ready signing class that produces PAdES-B-B and PAdES-B-T compliant signatures. Chapter 30 goes deeper into the BouncyCastle CMS construction that powers the actual cryptography.

32.1 The PDFBox Signing API: Three Moving Parts

PDFBox signing revolves around three objects that you wire together before calling `saveIncremental()`. Understanding what each one does — and what happens when you get the order wrong — is essential.

32.1.1 PDSignature: The Signature Dictionary

`PDSignature` is PDFBox's Java representation of the PDF `/Sig` dictionary (ISO 32000-2:2020 S12.8.1). When you create a `PDSignature` and set its properties, you are building the dictionary that will be written into the PDF file.

```
PDSignature signature = new PDSignature();
signature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);       // /Filter
signature.setSubFilter(COSName.getPDFName("ETSI.CAdES.detached")); // /SubFilter
signature.setSignDate(Calendar.getInstance());                // /M
signature.setName("Alice Smith");                             // /Name
signature.setReason("I approve this document");               // /Reason
signature.setLocation("Berlin, Germany");                     // /Location
signature.setContactInfo("alice@example.com");                // /ContactInfo
```

Every setter maps directly to a key in the `/Sig` dictionary. The following table lists all the fields you can set, what they map to, and when you should care.

Table 32.1: PDSignature Fields — ISO 32000-2:2020 S12.8.1, Table 255

Java Method	PDF Key	Required	What It Does
`setFilter(COSName)`	`/Filter`	Yes	The signature handler. Almost always `Adobe.PPKLite`. PDFBox provides `FILTER_ADOBE_PPKLITE` as a constant. Other values (`Entrust.PPKEF`, `CICI.SignIt`, `VeriSign.PPKVS`) exist in the spec but are rarely seen in the wild.
`setSubFilter(COSName)`	`/SubFilter`	Yes	The encoding format of the signature value. For PAdES, use `ETSI.CAdES.detached`. For legacy compatibility, `adbe.pkcs7.detached`. The deprecated `adbe.pkcs7.sha1` embeds the document hash as CMS content — do not use it in new code.
`setSignDate(Calendar)`	`/M`	No	Claimed signing time. This is *not* the authoritative timestamp — it is the signer's local clock. A TSA timestamp (PAdES-B-T) provides cryptographic proof of time. Set it anyway, because viewers display it when no TSA timestamp exists.
`setName(String)`	`/Name`	No	Signer's name as a display string. PAdES validators typically extract the name from the signing certificate's CN instead. Still useful for the human reader.
`setReason(String)`	`/Reason`	No	Why the document was signed. Free text. Some workflows require specific values (”Approved”, ”Reviewed”).
`setLocation(String)`	`/Location`	No	Where the document was signed. Free text. ”Berlin, Germany” or ”Server Room 3” — whatever is meaningful.
`setContactInfo(String)`	`/ContactInfo`	No	Contact information for the signer. Rarely used in practice, but mandated by some government workflows.
`setByteRange(int[])`	`/ByteRange`	Auto	The byte ranges of the signed content. *You do not set this yourself.* PDFBox computes it during `saveIncremental()`. If you manually set it, you will break the signature.

Do not call `setByteRange()` manually. PDFBox calculates the byte range during the incremental save process. If you set it yourself, the byte range will not match the actual file offsets

after writing, and the signature will be invalid. We have seen production code that does this — it always fails.

32.1.2 SignatureInterface: The Cryptographic Callback

`SignatureInterface` is a single-method interface:

```
public interface SignatureInterface {
    byte[] sign(InputStream content) throws IOException;
}
```

PDFBox calls your `sign()` method during `saveIncremental()`. The `InputStream` parameter contains the bytes identified by the `/ByteRange` — that is, the entire PDF file *minus* the gap reserved for the `/Contents` hex string. Your job is to read those bytes, produce a CMS `SignedData` structure (or whatever your `/SubFilter` requires), and return the DER-encoded result.

The returned `byte[]` is hex-encoded and written into the `/Contents` field of the signature dictionary. If the returned bytes are larger than the space reserved for `/Contents`, PDFBox throws an exception. If they are smaller, the remaining space is zero-padded.

The `InputStream` you receive in `sign()` is the concatenation of the two byte ranges — the bytes before the `/Contents` gap and the bytes after it. You must read *all* of them with `content.readAllBytes()` (or equivalent). If you only read part of the stream, you are signing incomplete data, and the signature will fail validation.

Here is the simplest possible implementation — it signs the content with RSA/SHA-256 and produces a basic CMS `SignedData`:

```
public class SimpleCmsSignatureInterface implements SignatureInterface {

    private final PrivateKey privateKey;
    private final X509Certificate certificate;
    private final List<X509Certificate> chain;

    public SimpleCmsSignatureInterface(PrivateKey privateKey,
                                       X509Certificate certificate,
                                       List<X509Certificate> chain) {
        this.privateKey = privateKey;
        this.certificate = certificate;
        this.chain = chain;
    }

    @Override
    public byte[] sign(InputStream content) throws IOException {
        try {
            // Read all byte-range content
            byte[] contentBytes = content.readAllBytes();

            // Build the CMS SignedData (Chapter 30 covers this in full detail)
            CMSSignedDataGenerator gen = new CMSSignedDataGenerator();

            ContentSigner signer = new JcaContentSignerBuilder("SHA256withRSA")
                    .setProvider("BC")
                    .build(privateKey);
```

```
            DigestCalculatorProvider dcProvider =
                    new JcaDigestCalculatorProviderBuilder()
                            .setProvider("BC")
                            .build();

            gen.addSignerInfoGenerator(
                    new JcaSignerInfoGeneratorBuilder(dcProvider)
                            .build(signer, certificate));

            gen.addCertificates(new JcaCertStore(chain));

            CMSProcessableByteArray cmsContent =
                    new CMSProcessableByteArray(contentBytes);
            CMSSignedData signedData = gen.generate(cmsContent, false);

            return signedData.getEncoded();
        } catch (Exception e) {
            throw new IOException("CMS signing failed: " + e.getMessage(), e);
        }
    }
}
```

- The `false` parameter means ”detached signature” — the content is not embedded inside the CMS structure. This is mandatory for PDF signatures: the content lives in the PDF file itself, not inside the CMS blob.

32.1.3 SignatureOptions: Container Size and Visual Appearance

`SignatureOptions` controls two things: the size of the `/Contents` container (how many bytes are reserved in the PDF for the CMS signature) and the visual appearance stream.

```
SignatureOptions sigOptions = new SignatureOptions();
sigOptions.setPreferredSignatureSize(32768); // 32 KB for the CMS container
```

The `preferredSignatureSize` is the number of *bytes* reserved for the hex-encoded signature value in `/Contents`. Since the hex encoding doubles the size, a 32768-byte container can hold a CMS blob of up to about 16 KB.

Table 32.2: Recommended Container Sizes

Scenario	Size (bytes)	Why
Basic signature (no timestamp)	32768	A CMS `SignedData` with one signer, RSA-2048, and a 3-certificate chain typically occupies 4-8 KB. 32 KB gives ample room.
Signature with TSA timestamp (PAdES-B-T)	65536	The timestamp token adds another CMS `SignedData` structure (the TSA’s signature, its certificate chain, and the `TSTInfo`). 64 KB handles even large TSA responses.

Scenario	Size (bytes)	Why
Signature with embedded OCSP/CRL (PAdES-B-LT)	131072 or more	CRL responses can be very large (100 KB+). If you embed revocation data in the CMS structure rather than the DSS, you need a correspondingly large container.

If your CMS blob exceeds the reserved container size, `saveIncremental()` throws an `IOException` with a message like ”can’t write signature, not enough space.” The fix is to increase `preferredSignatureSize`. But do not set it to absurd values (e.g., 10 MB) because the entire container is written as hex zeros into the PDF — a 10 MB container adds 20 MB of hex padding to the file.

32.2 Complete Basic Signing Example

Let us put the three pieces together into a complete, compilable example. This is the minimum code needed to sign a PDF with PDFBox 3.0.0.

```
import java.io.ByteArrayOutputStream;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.security.KeyStore;
import java.security.PrivateKey;
import java.security.cert.X509Certificate;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.Enumeration;
import java.util.List;

import org.apache.pdfbox.Loader;
import org.apache.pdfbox.cos.COSName;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.SignatureInterface;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.SignatureOptions;
import org.bouncycastle.cert.jcajce.JcaCertStore;
import org.bouncycastle.cms.CMSProcessableByteArray;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.CMSSignedDataGenerator;
import org.bouncycastle.cms.jcajce.JcaSignerInfoGeneratorBuilder;
import org.bouncycastle.jce.provider.BouncyCastleProvider;
import org.bouncycastle.operator.ContentSigner;
import org.bouncycastle.operator.jcajce.JcaContentSignerBuilder;
import org.bouncycastle.operator.jcajce.JcaDigestCalculatorProviderBuilder;

/**
 * Minimal PDF signing example with PDFBox 3.x and BouncyCastle.
 * Signs a PDF with an invisible signature using a PKCS#12 keystore.
```

```
 */
public class BasicPdfSigner {

    public static void main(String[] args) throws Exception {
        // Register BouncyCastle provider -- must happen before any crypto operation
        java.security.Security.addProvider(new BouncyCastleProvider());

        Path inputPdf = Path.of("document.pdf");
        Path keystorePath = Path.of("keystore.p12");
        char[] keystorePassword = "changeit".toCharArray();

        // Load key material from PKCS#12
        KeyStore ks = KeyStore.getInstance("PKCS12");
        try (FileInputStream fis = new FileInputStream(keystorePath.toFile())) {
            ks.load(fis, keystorePassword);
        }

        Enumeration<String> aliases = ks.aliases();
        String alias = aliases.nextElement();
        PrivateKey privateKey = (PrivateKey) ks.getKey(alias, keystorePassword);
        X509Certificate cert = (X509Certificate) ks.getCertificate(alias);

        List<X509Certificate> chain = new ArrayList<>();
        for (java.security.cert.Certificate c : ks.getCertificateChain(alias)) {
            chain.add((X509Certificate) c);
        }

        // Load the PDF
        byte[] pdfBytes = Files.readAllBytes(inputPdf);

        try (PDDocument doc = Loader.loadPDF(pdfBytes)) {
            // Create the signature dictionary
            PDSignature signature = new PDSignature();
            signature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
            signature.setSubFilter(COSName.getPDFName("ETSI.CAdES.detached"));
            signature.setSignDate(Calendar.getInstance());
            signature.setName(cert.getSubjectX500Principal().getName());
            signature.setReason("Document approval");

            // Configure container size
            SignatureOptions options = new SignatureOptions();
            options.setPreferredSignatureSize(32768);

            // Wire everything together
            SignatureInterface sigInterface = new SimpleCmsSignatureInterface(
                    privateKey, cert, chain);
            doc.addSignature(signature, sigInterface, options);

            // Save incrementally -- this triggers the sign() callback
            ByteArrayOutputStream baos = new ByteArrayOutputStream();
            doc.saveIncremental(baos);

            // Write the signed PDF
            Path outputPdf = Path.of("document-signed.pdf");
            Files.write(outputPdf, baos.toByteArray());
```

```
            System.out.println("Signed PDF written to " + outputPdf
                    + " (" + baos.size() + " bytes)");
        }
    }
}
```

- PDFBox 3.x uses `Loader.loadPDF(byte[])`. The old `PDDocument.load()` is gone. See the migration section later in this chapter.
- `addSignature()` registers the signature dictionary, the callback, and the options. Nothing is written yet.
- `saveIncremental()` writes the PDF as an incremental update, computes the byte range, calls your `sign()` method, hex-encodes the result, and patches it into the `/Contents` field. This is where all the magic happens.

Always use `saveIncremental()`, never `save()`. A regular `save()` rewrites the entire PDF, which destroys any existing signatures. `saveIncremental()` appends a new revision to the file, preserving all previous signatures. This is not optional — it is the only correct way to save a signed PDF.

32.3 The InputStream in sign(): What You Are Actually Signing

The `InputStream` passed to your `sign()` method is one of the most misunderstood parts of the PDFBox signing API. Let us be very precise about what it contains.

When PDFBox writes the PDF during `saveIncremental()`, it creates the file with a placeholder for the `/Contents` value. The byte range is computed as two ranges that together cover the entire file *except* the `/Contents` hex string:

```
/ByteRange [0 offset1 offset2 length2]
```

Where:

- Range 1: bytes 0 through `offset1` (everything before the `/Contents` hex string)
- Range 2: bytes `offset2` through `offset2 + length2` (everything after the `/Contents` hex string)

The `InputStream` you receive is the concatenation of these two ranges. You do not see the gap. You do not need to parse the byte range. PDFBox handles all of that.

Your `sign()` method receives a continuous stream of bytes, and you sign all of them.

```
@Override
public byte[] sign(InputStream content) throws IOException {
    byte[] contentBytes = content.readAllBytes();
    // contentBytes now contains Range1 + Range2
    // Sign these bytes and return the CMS SignedData
}
```

- You *must* read the entire stream. If you read only part of it (e.g., using a fixed-size buffer and stopping early), the hash you compute will not match the hash a validator computes over the full byte range, and the signature will be invalid.

A common bug in PDFBox 2.x code (which people copy into 3.x projects) is using `IOUtils.toByteArray()` from Apache Commons IO with a size limit. The byte range content can be large — for a 100 MB PDF, the stream is approximately 100 MB. If your byte array buffer is smaller, you get a truncated hash and a broken signature that validates in your tests (small PDFs) but fails in production (large PDFs).

32.4 Visible Signature Appearances

An invisible signature is fine for server-side signing workflows, but many use cases — contracts, approvals, notarizations — require a visible signature block on a specific page. This section shows how to create visible appearances with PDFBox.

32.4.1 The Form XObject Approach

A signature appearance is a Form XObject — a self-contained mini-page with its own coordinate system, resources, and content stream. You build it, attach it to the signature field's widget annotation, and PDFBox includes it in the signed content.

```
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.PDResources;
import org.apache.pdfbox.pdmodel.common.PDRectangle;
import org.apache.pdfbox.pdmodel.font.PDType1Font;
import org.apache.pdfbox.pdmodel.font.Standard14Fonts;
import org.apache.pdfbox.pdmodel.graphics.form.PDFormXObject;
import org.apache.pdfbox.pdmodel.interactive.form.PDSignatureField;

import java.io.OutputStream;
import java.nio.charset.StandardCharsets;

/**
 * Builds a visible text-based signature appearance as a Form XObject.
 */
public class SignatureAppearanceBuilder {

    /**
     * Creates and attaches a visible text appearance to a signature field.
     *
     * @param doc      the document being signed
     * @param sigField the signature field to attach the appearance to
     * @param name     signer name to display
     * @param reason   signing reason to display
     * @param width    appearance width in points
     * @param height   appearance height in points
     */
    public static void buildTextAppearance(PDDocument doc,
                                           PDSignatureField sigField,
                                           String name, String reason,
                                           float width, float height)
            throws Exception {

        PDResources resources = new PDResources();
```

```
    // Use a Standard 14 font -- no embedding needed
    PDType1Font font = new PDType1Font(
            Standard14Fonts.FontName.HELVETICA);
    org.apache.pdfbox.cos.COSName fontName = resources.add(font);

    // Build the content stream
    float fontSize = 10f;
    float lineSpacing = fontSize * 1.4f;
    float margin = 4f;
    float y = height - margin - fontSize;

    StringBuilder ops = new StringBuilder();
    ops.append("BT\n");
    ops.append("/").append(fontName.getName())
            .append(" ").append(fontSize).append(" Tf\n");
    ops.append("0 0 0 rg\n"); // black text

    // Line 1: "Digitally signed by:"
    ops.append(margin).append(" ").append(y).append(" Td\n");
    ops.append("(Digitally signed by:) Tj\n");

    // Line 2: signer name
    y -= lineSpacing;
    ops.append(margin).append(" ").append(y).append(" Td\n");
    ops.append("(").append(escapePs(name)).append(") Tj\n");

    // Line 3: reason
    if (reason != null && !reason.isEmpty()) {
        y -= lineSpacing;
        ops.append(margin).append(" ").append(y).append(" Td\n");
        ops.append("(Reason: ").append(escapePs(reason)).append(") Tj\n");
    }

    ops.append("ET\n");

    byte[] streamBytes = ops.toString()
            .getBytes(StandardCharsets.US_ASCII);

    // Create the Form XObject
    PDFormXObject form = new PDFormXObject(doc);
    form.setBBox(new PDRectangle(width, height));
    form.setResources(resources);

    try (OutputStream os = form.getStream().createOutputStream()) {
        os.write(streamBytes);
    }

    // Attach to the widget annotation
    org.apache.pdfbox.cos.COSDictionary apDict =
            new org.apache.pdfbox.cos.COSDictionary();
    apDict.setItem(org.apache.pdfbox.cos.COSName.N, form);
    sigField.getWidgets().get(0).getCOSObject()
            .setItem(org.apache.pdfbox.cos.COSName.AP, apDict);
}
```

```
    /** Escape parentheses and backslashes for a PDF string literal. */
    private static String escapePs(String text) {
        return text.replace("\\", "\\\\")
                   .replace("(", "\\(")
                   .replace(")", "\\)");
    }
}
```

- In PDFBox 3.x, `PDType1Font.HELVETICA` does not exist. You must use `new PDType1Font(Standard14Fonts.`
  This is one of the most common migration errors. See the migration section at the end of this chapter.
- `form.getStream().createOutputStream()` returns an output stream that writes directly into the Form XObject's content stream. This is the correct way to write appearance content in PDFBox 3.x. You cannot use `PDPageContentStream` with a Form XObject.

32.4.2 Image-Based Appearances

For handwritten signature images (e.g., from a drawing canvas or a scanned signature):

```
import org.apache.pdfbox.pdmodel.graphics.image.PDImageXObject;
import java.util.Base64;

/**
 * Creates a signature appearance from a Base64-encoded PNG image.
 */
public static void buildImageAppearance(PDDocument doc,
                                        PDSignatureField sigField,
                                        String base64Png,
                                        float width, float height)
        throws Exception {

    // Decode the image
    String data = base64Png;
    if (data.contains(",")) {
        data = data.substring(data.indexOf(',') + 1);
    }
    byte[] imageBytes = Base64.getDecoder().decode(data);

    PDImageXObject pdImage = PDImageXObject.createFromByteArray(
            doc, imageBytes, "signature-image");

    PDResources resources = new PDResources();
    org.apache.pdfbox.cos.COSName imgName = resources.add(pdImage);

    // Scale the image to fit the appearance rectangle while maintaining
    // aspect ratio
    float imgW = pdImage.getWidth();
    float imgH = pdImage.getHeight();
    float scale = Math.min(width / imgW, height / imgH);
    float drawW = imgW * scale;
    float drawH = imgH * scale;
    float xOffset = (width - drawW) / 2;
    float yOffset = (height - drawH) / 2;
```

```
        // PDF content stream: push graphics state, apply transformation
        // matrix, draw image, restore graphics state
        String ops = "q\n"
                + drawW + " 0 0 " + drawH + " "
                + xOffset + " " + yOffset + " cm\n"
                + "/" + imgName.getName() + " Do\n"
                + "Q\n";

        PDFormXObject form = new PDFormXObject(doc);
        form.setBBox(new PDRectangle(width, height));
        form.setResources(resources);

        try (OutputStream os = form.getStream().createOutputStream()) {
            os.write(ops.getBytes(java.nio.charset.StandardCharsets.US_ASCII));
        }

        org.apache.pdfbox.cos.COSDictionary apDict =
                new org.apache.pdfbox.cos.COSDictionary();
        apDict.setItem(org.apache.pdfbox.cos.COSName.N, form);
        sigField.getWidgets().get(0).getCOSObject()
                .setItem(org.apache.pdfbox.cos.COSName.AP, apDict);
    }
```

The content stream uses the standard PDF imaging operators: `q` (save graphics state), `cm` (concatenate transformation matrix), `Do` (paint XObject), `Q` (restore graphics state). The transformation matrix `[drawW 0 0 drawH xOffset yOffset]` scales and positions the image within the appearance rectangle.

32.5 Multi-Signature Support

A PDF can contain multiple signatures. Each signature is an independent signature field with its own `/Sig` dictionary and its own byte range. The key insight: each subsequent signature covers the *entire* file as it existed after the previous signature, including all previous signatures.

32.5.1 How Multiple Signatures Work

Consider a document signed by Alice, then by Bob:

1. Alice signs the original document. Her byte range covers the entire file (minus her `/Contents` gap). PDFBox writes this as an incremental update — the original file plus a new cross-reference section and trailer.
2. Bob signs the document *after* Alice. His byte range covers the entire file as it now exists — including Alice's incremental update and her signature. Bob's signature is written as a *second* incremental update appended after Alice's.

Each signature's byte range covers everything that existed when that signature was created. Alice's signature does not cover Bob's changes (because Bob had not signed yet), but Bob's signature covers Alice's.

32.5.2 Code: Adding a Second Signature

```
/**
 * Adds a second signature to an already-signed PDF.
 * The key is that we load the already-signed bytes and use saveIncremental()
 * again -- PDFBox appends another revision.
 */
public byte[] addSecondSignature(byte[] alreadySignedPdf,
                                 PrivateKey privateKey,
                                 X509Certificate certificate,
                                 List<X509Certificate> chain,
                                 String fieldName)
        throws Exception {

    try (PDDocument doc = Loader.loadPDF(alreadySignedPdf)) {
        // Find or create the second signature field
        PDAcroForm acroForm = doc.getDocumentCatalog().getAcroForm();
        PDSignatureField sigField = null;

        // Look for an existing empty signature field
        for (PDField field : acroForm.getFieldTree()) {
            if (field instanceof PDSignatureField sf
                    && fieldName.equals(sf.getFullyQualifiedName())
                    && sf.getSignature() == null) {
                sigField = sf;
                break;
            }
        }

        if (sigField == null) {
            throw new IllegalStateException(
                    "No empty signature field found: " + fieldName);
        }

        // Build the second signature
        PDSignature signature = new PDSignature();
        signature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
        signature.setSubFilter(
                COSName.getPDFName("ETSI.CAdES.detached"));
        signature.setSignDate(Calendar.getInstance());
        signature.setName("Bob Jones");

        sigField.setValue(signature);

        SignatureOptions options = new SignatureOptions();
        options.setPreferredSignatureSize(32768);

        SignatureInterface sigInterface =
                new SimpleCmsSignatureInterface(
                        privateKey, certificate, chain);
        doc.addSignature(signature, sigInterface, options);

        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        doc.saveIncremental(baos);
        return baos.toByteArray();
```

```
    }
}
```

- Each call to `saveIncremental()` appends a new revision. The file grows with each signature. A document with three signatures has four revisions (original + three incremental updates).

The order in which signatures are applied matters for validation. Adobe Acrobat validates each signature against its revision — it checks that the bytes covered by Alice's byte range have not been modified. If someone modified Alice's revision after she signed (other than by appending a new incremental update), Acrobat flags Alice's signature as invalid, even if Bob's is fine.

32.6 External Signing: HSM and Remote Key Flows

In many production environments, the private key never leaves a Hardware Security Module (HSM), a cloud KMS, or a remote signing service. The key cannot be exported, so you cannot pass it to a `JcaContentSignerBuilder`. Instead, you need to split the signing process: compute the hash locally, send the hash to the remote system, receive the raw signature value, and assemble the CMS structure yourself.

32.6.1 The Hash-Signing Pattern

The flow has three steps:

1. **Prepare**: Create the PDF with a placeholder signature, compute the byte range, and extract the bytes to be signed.
2. **Hash**: Compute the SHA-256 digest of the byte range content.
3. **Sign remotely**: Send the hash (not the document) to the HSM/cloud service, receive the raw signature bytes, and embed them into the CMS structure.

PDFBox supports this through a two-phase API using `ExternalSigningSupport`:

```
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.ExternalSigningSupport;

/**
 * Signs a PDF using an external key source (HSM, cloud KMS, smart card).
 * The private key never leaves the external system -- only a hash is sent.
 */
public byte[] signWithExternalKey(byte[] pdfBytes,
                                  ExternalKeyService externalKeyService,
                                  X509Certificate certificate,
                                  List<X509Certificate> chain)
        throws Exception {

    ByteArrayOutputStream result = new ByteArrayOutputStream();

    try (PDDocument doc = Loader.loadPDF(pdfBytes)) {
        PDSignature signature = new PDSignature();
        signature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
        signature.setSubFilter(
                COSName.getPDFName("ETSI.CAdES.detached"));
```

```
        signature.setSignDate(Calendar.getInstance());

        SignatureOptions options = new SignatureOptions();
        options.setPreferredSignatureSize(32768);

        // Register signature WITHOUT a SignatureInterface
        doc.addSignature(signature, options);

        // saveIncrementalForExternalSigning writes the PDF with a
        // placeholder and returns an ExternalSigningSupport object
        ExternalSigningSupport externalSigning =
                doc.saveIncrementalForExternalSigning(result);

        // Get the byte range content
        InputStream content = externalSigning.getContent();
        byte[] contentBytes = content.readAllBytes();

        // Compute the hash locally
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(contentBytes);

        // Send hash to external system, receive raw signature value
        byte[] rawSignature =
                externalKeyService.signHash(hash, "SHA256withRSA");

        // Build CMS structure around the raw signature
        byte[] cmsBytes = buildCmsWithExternalSignature(
                rawSignature, contentBytes, certificate, chain);

        // Write the CMS bytes into the placeholder
        externalSigning.setSignature(cmsBytes);
    }

    return result.toByteArray();
}
```

- `saveIncrementalForExternalSigning()` writes the PDF to the output stream with a placeholder `/Contents` and returns an `ExternalSigningSupport` handle.
- `getContent()` returns the byte range content as an `InputStream` — the same data that would be passed to `sign()` in the callback-based approach.
- This is the only operation that touches the external key. You send a 32-byte SHA-256 hash and receive a raw signature value (typically 256 bytes for RSA-2048 or 64 bytes for ECDSA P-256).
- `setSignature()` hex-encodes the CMS bytes and patches them into the `/Contents` placeholder.

Building the CMS structure with a pre-computed raw signature is more involved than using `CMSSignedDataGenerator`. You need to construct the `SignerInfo` manually with the external signature value. Chapter 30 shows the complete code for this.

32.7 Certification Signatures with DocMDP

A certification signature (also called a ”certifying signature” or ”MDP signature”) is a special first signature that declares permission levels for future modifications to the document. It uses the `/DocMDP` transform method (ISO 32000-2:2020 S12.8.2.2).

32.7.1 The Three DocMDP Levels

Table 32.3: DocMDP Permission Levels — ISO 32000-2:2020 S12.8.2.2, Table 257

Level	Name	What Is Allowed After Certification
1	No changes	No modifications whatsoever. Even filling in form fields invalidates the signature.
2	Form fill + sign	Form fields may be filled, signature fields may be signed, and pages may be instantiated from templates. No other changes.
3	Form fill + sign + annotate	Everything in level 2, plus annotations may be added, modified, or deleted.

Level 2 is by far the most common in practice — it allows a document to be certified by the author and then signed by additional parties.

32.7.2 Code: Applying a Certification Signature

```
import org.apache.pdfbox.cos.COSArray;
import org.apache.pdfbox.cos.COSDictionary;
import org.apache.pdfbox.cos.COSInteger;
import org.apache.pdfbox.cos.COSName;

/**
 * Applies a certification (DocMDP) signature to a PDF.
 * This must be the FIRST signature in the document.
 */
public void applyCertificationSignature(PDDocument doc,
                                        PDSignature signature,
                                        PDSignatureField sigField,
                                        int docMdpLevel) {

    // Clamp level to valid range
    int level = (docMdpLevel >= 1 && docMdpLevel <= 3)
            ? docMdpLevel : 2;

    // Build the /Reference array with a DocMDP transform
    COSDictionary sigRef = new COSDictionary();
    sigRef.setItem(COSName.TYPE, COSName.getPDFName("SigRef"));
    sigRef.setItem(COSName.getPDFName("TransformMethod"),
            COSName.getPDFName("DocMDP"));

    COSDictionary transformParams = new COSDictionary();
```

```
        transformParams.setItem(COSName.TYPE,
                COSName.getPDFName("TransformParams"));
        transformParams.setItem(COSName.getPDFName("P"),
                COSInteger.get(level));
        transformParams.setItem(COSName.getPDFName("V"),
                COSName.getPDFName("1.2"));
        sigRef.setItem(COSName.getPDFName("TransformParams"),
                transformParams);

        COSArray refArray = new COSArray();
        refArray.add(sigRef);
        signature.getCOSObject()
                .setItem(COSName.getPDFName("Reference"), refArray);

        // Set /Perms /DocMDP in the document catalog -- this is what
        // tells viewers "this document has a certification signature"
        COSDictionary perms = new COSDictionary();
        perms.setItem(COSName.getPDFName("DocMDP"),
                signature.getCOSObject());
        doc.getDocumentCatalog().getCOSObject()
                .setItem(COSName.getPDFName("Perms"), perms);
    }
```

- The `/P` value is the permission level: 1, 2, or 3.
- The `/Perms` dictionary in the document catalog must reference the certification signature's `/Sig` dictionary. This is how viewers discover that the document has been certified.

A document can have at most *one* certification signature, and it must be the first signature applied. If you try to add a certification signature to a document that already has one, the existing certification is invalidated. If you apply it as a second signature (after an approval signature), the result is undefined — some viewers accept it, some reject it, and Adobe Acrobat shows a confusing error.

32.8 PDFBox 2.x to 3.x Migration

PDFBox 3.0.0 introduced breaking API changes that affect every piece of signing code. If you are migrating from PDFBox 2.x (which many projects still use), these are the changes you need to know.

32.8.1 Document Loading

PDFBox 2.x (removed)

```
// PDFBox 2.x -- these methods no longer exist
PDDocument doc = PDDocument.load(new File("doc.pdf"));
PDDocument doc = PDDocument.load(inputStream);
PDDocument doc = PDDocument.load(byteArray);
```

PDFBox 3.x (current)

```
// PDFBox 3.x -- use the Loader class
PDDocument doc = Loader.loadPDF(new File("doc.pdf"));
```

```
PDDocument doc = Loader.loadPDF(byteArray);
// No direct InputStream loading -- read to byte[] first
byte[] bytes = inputStream.readAllBytes();
PDDocument doc = Loader.loadPDF(bytes);
```

The `Loader` class is in `org.apache.pdfbox.Loader`. The old `PDDocument.load()` static methods were removed entirely — not deprecated, *removed*.

32.8.2 Standard 14 Fonts

This is the single most common compilation error in PDFBox 2.x-to-3.x migration.

PDFBox 2.x (removed)

```
// PDFBox 2.x -- static font constants
PDFont font = PDType1Font.HELVETICA;
PDFont fontBold = PDType1Font.HELVETICA_BOLD;
PDFont fontMono = PDType1Font.COURIER;
```

PDFBox 3.x (current)

```
// PDFBox 3.x -- constructor with FontName enum
PDFont font = new PDType1Font(Standard14Fonts.FontName.HELVETICA);
PDFont fontBold = new PDType1Font(Standard14Fonts.FontName.HELVETICA_BOLD);
PDFont fontMono = new PDType1Font(Standard14Fonts.FontName.COURIER);
```

The `Standard14Fonts.FontName` enum lives in `org.apache.pdfbox.pdmodel.font.Standard14Fonts`.

32.8.3 AcroForm API Changes

PDFBox 2.x

```
boolean needsAppearances = acroForm.isNeedAppearances(); // gone
```

PDFBox 3.x

```
boolean needsAppearances = acroForm.getNeedAppearances();
```

- The method was renamed from `isNeedAppearances()` to `getNeedAppearances()`. A small change, but it breaks at compile time.

32.8.4 Content Stream Writing

PDFBox 2.x — could use PDPageContentStream for Form XObjects

```
// This approach no longer works for Form XObjects in 3.x
```

PDFBox 3.x — use the stream's output stream directly

```
PDFormXObject form = new PDFormXObject(doc);
try (OutputStream os = form.getStream().createOutputStream()) {
    os.write(contentStreamBytes);
}
```

32.8.5 Complete Migration Checklist

Table 32.4: PDFBox 2.x to 3.x Breaking Changes Relevant to Signing

PDFBox 2.x	PDFBox 3.x	Impact
`PDDocument.load(File)`	`Loader.loadPDF(File)`	Compile error
`PDDocument.load(byte[])`	`Loader.loadPDF(byte[])`	Compile error
`PDDocument.load(InputStream)`	`Loader.loadPDF(bytes)` (read first)	Compile error
`PDType1Font.HELVETICA`	`new PDType1Font(Standard14Fonts.FontName.HELVETICA)`	Compile error
`acroForm.isNeedAppearances()`	`acroForm.getNeedAppearances()`	Compile error
`new PDPageContentStream(doc, formXObject)`	`form.getStream().createOutputStream()`	Compile error
`PDFont.getEncoding()`	Not available — use COS dictionary	Compile error
`PDFont.getToUnicodeCMap()`	Not visible — check COS key	Compile error

When migrating a large codebase, search for every occurrence of `PDDocument.load`, `PDType1Font.HELVETICA` (and all other font constants), and `isNeedAppearances`. These three account for about 90% of compilation failures in signing code.

32.9 Common Pitfalls

We have collected these from production incidents, Stack Overflow questions, and our own debugging sessions. Each one has cost someone hours.

32.9.1 Pitfall 1: Not Registering the BouncyCastle Provider

```
// This line MUST execute before any signing operation
java.security.Security.addProvider(
        new org.bouncycastle.jce.provider.BouncyCastleProvider());
```

Without this, you get a `NoSuchAlgorithmException` or a `NoSuchProviderException` at runtime. The error message varies depending on which BouncyCastle class triggers first, making it confusing to diagnose. In Spring Boot applications, put this in a `@PostConstruct` method of a `@Configuration` class:

```
import jakarta.annotation.PostConstruct;
import org.bouncycastle.jce.provider.BouncyCastleProvider;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BouncyCastleConfig {

    @PostConstruct
    public void registerProvider() {
        if (java.security.Security.getProvider("BC") == null) {
            java.security.Security.addProvider(
```

```
                    new BouncyCastleProvider());
        }
    }
}
```

32.9.2 Pitfall 2: Using save() Instead of saveIncremental()

```
// WRONG -- destroys all existing signatures
doc.save(outputStream);

// CORRECT -- preserves existing signatures
doc.saveIncremental(outputStream);
```

A regular `save()` rewrites the entire PDF from scratch. This changes every byte offset in the file, which means every existing signature's byte range is now wrong. Every signature becomes invalid. Use `saveIncremental()`. Always.

32.9.3 Pitfall 3: /Contents Container Too Small

If the CMS blob returned by your `sign()` method exceeds the space reserved by `preferredSignatureSize`, you get:

```
java.io.IOException: Can't write signature, not enough space
```

The fix is straightforward — increase the container size. But be aware that the container size is in *bytes of hex encoding*, not bytes of CMS data. A 32768-byte container holds approximately 16 KB of CMS data (since each byte becomes two hex characters).

When in doubt, use 65536 (64 KB). The extra space is filled with hex zeros and adds about 64 KB to the file size — negligible for most PDFs.

32.9.4 Pitfall 4: Not Closing the PDDocument

```
// WRONG -- resource leak, and on some JVMs the temp files
// created by PDFBox are never cleaned up
PDDocument doc = Loader.loadPDF(bytes);
doc.addSignature(sig, iface, opts);
doc.saveIncremental(baos);
// doc is never closed!

// CORRECT -- use try-with-resources
try (PDDocument doc = Loader.loadPDF(bytes)) {
    doc.addSignature(sig, iface, opts);
    doc.saveIncremental(baos);
}
```

`PDDocument` implements `Closeable`. If you do not close it, temporary files and memory-mapped buffers may not be released. In a server application that signs thousands of documents, this leads to `OutOfMemoryError` or file handle exhaustion.

32.9.5 Pitfall 5: Signing After Closing the Document

```
// WRONG -- the document is closed before saveIncremental()
PDDocument doc = Loader.loadPDF(bytes);
```

```
doc.addSignature(sig, iface, opts);
doc.close();
doc.saveIncremental(baos); // throws IllegalStateException
```

`addSignature()` only registers the signature. The actual signing happens inside `saveIncremental()`. If you close the document before calling `saveIncremental()`, the document's internal structures are released, and the save fails.

32.9.6 Pitfall 6: Reusing a PDSignature Object

```
// WRONG -- reusing the same PDSignature for two documents
PDSignature sig = new PDSignature();
sig.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);

doc1.addSignature(sig, iface, opts);
doc1.saveIncremental(out1);

doc2.addSignature(sig, iface, opts); // <-- BROKEN
doc2.saveIncremental(out2);
```

A `PDSignature` is a COS dictionary tied to a specific document. After it is added to `doc1` and saved, it belongs to `doc1`. Create a new `PDSignature` for each document.

32.10 Timestamp Embedding

A timestamp proves when a signature was created. Without it, you have a PAdES-B-B signature. With it, you have PAdES-B-T.

32.10.1 Embedding a TSA Timestamp as an Unsigned Attribute

The timestamp token is an RFC 3161 `TimeStampToken` embedded as an unsigned attribute (`id-aa-signatureTimeStampToken`, OID 1.2.840.113549.1.9.16.2.14) in the CMS `SignerInfo`.

The flow is:

1. Generate the CMS `SignedData` (without the timestamp)
2. Extract the signer's raw signature value from the `SignerInfo`
3. Hash the signature value with SHA-256
4. Send the hash to the TSA, receive a `TimeStampToken`
5. Attach the token as an unsigned attribute on the `SignerInfo`
6. Return the modified CMS `SignedData`

```
import org.bouncycastle.asn1.ASN1EncodableVector;
import org.bouncycastle.asn1.ASN1Primitive;
import org.bouncycastle.asn1.DERSet;
import org.bouncycastle.asn1.cms.Attribute;
import org.bouncycastle.asn1.cms.AttributeTable;
import org.bouncycastle.asn1.pkcs.PKCSObjectIdentifiers;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.SignerInformation;
import org.bouncycastle.cms.SignerInformationStore;

import java.util.Collections;
```

```
/**
 * Embeds an RFC 3161 timestamp token as an unsigned attribute
 * in a CMS SignedData structure. This upgrades a PAdES-B-B
 * signature to PAdES-B-T.
 */
public CMSSignedData embedTimestamp(CMSSignedData signedData,
                                    TsaClient tsaClient,
                                    String tsaUrl)
        throws Exception {

    // Get the first (and typically only) signer
    SignerInformation signerInfo = signedData.getSignerInfos()
            .getSigners().iterator().next();

    // The signature value is what gets timestamped
    byte[] signatureBytes = signerInfo.getSignature();

    // Request timestamp from TSA (see Chapter 18 for TSA details)
    byte[] tsTokenBytes = tsaClient.getTimestampToken(
            tsaUrl, signatureBytes);

    // Parse the timestamp token as ASN.1
    ASN1Primitive tsToken = ASN1Primitive.fromByteArray(tsTokenBytes);

    // Build the unsigned attribute
    ASN1EncodableVector unsignedAttrsVec = new ASN1EncodableVector();
    unsignedAttrsVec.add(new Attribute(
            PKCSObjectIdentifiers.id_aa_signatureTimeStampToken,
            new DERSet(tsToken)));
    AttributeTable unsignedAttrs = new AttributeTable(unsignedAttrsVec);

    // Replace the signer's unsigned attributes
    SignerInformation newSignerInfo =
            SignerInformation.replaceUnsignedAttributes(
                    signerInfo, unsignedAttrs);

    SignerInformationStore newSignerInfos =
            new SignerInformationStore(
                    Collections.singletonList(newSignerInfo));

    return CMSSignedData.replaceSigners(signedData, newSignerInfos);
}
```

The timestamp is embedded *after* the CMS signature is generated. This is important: the timestamp covers the signature value, not the document content. The document content is covered by the CMS signature itself. The timestamp proves when that signature was created. This two-layer structure is fundamental to PAdES-B-T.

32.11 Complete Production-Ready Signing Class

Here is a complete signing class that handles all the scenarios discussed in this chapter: invisible and visible signatures, PAdES-B-B and PAdES-B-T profiles, certification signatures,

and multi-signature workflows.

```
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.PrivateKey;
import java.security.cert.X509Certificate;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.Collections;
import java.util.List;

import org.apache.pdfbox.Loader;
import org.apache.pdfbox.cos.COSArray;
import org.apache.pdfbox.cos.COSDictionary;
import org.apache.pdfbox.cos.COSInteger;
import org.apache.pdfbox.cos.COSName;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.PDResources;
import org.apache.pdfbox.pdmodel.common.PDRectangle;
import org.apache.pdfbox.pdmodel.font.PDType1Font;
import org.apache.pdfbox.pdmodel.font.Standard14Fonts;
import org.apache.pdfbox.pdmodel.graphics.form.PDFormXObject;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.SignatureInterface;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.SignatureOptions;
import org.apache.pdfbox.pdmodel.interactive.form.PDAcroForm;
import org.apache.pdfbox.pdmodel.interactive.form.PDField;
import org.apache.pdfbox.pdmodel.interactive.form.PDSignatureField;
import org.bouncycastle.asn1.ASN1EncodableVector;
import org.bouncycastle.asn1.ASN1Primitive;
import org.bouncycastle.asn1.DERSet;
import org.bouncycastle.asn1.cms.Attribute;
import org.bouncycastle.asn1.cms.AttributeTable;
import org.bouncycastle.asn1.ess.ESSCertIDv2;
import org.bouncycastle.asn1.ess.SigningCertificateV2;
import org.bouncycastle.asn1.nist.NISTObjectIdentifiers;
import org.bouncycastle.asn1.pkcs.PKCSObjectIdentifiers;
import org.bouncycastle.asn1.x509.AlgorithmIdentifier;
import org.bouncycastle.cert.jcajce.JcaCertStore;
import org.bouncycastle.cms.CMSProcessableByteArray;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.CMSSignedDataGenerator;
import org.bouncycastle.cms.DefaultSignedAttributeTableGenerator;
import org.bouncycastle.cms.SignerInfoGenerator;
import org.bouncycastle.cms.SignerInformation;
import org.bouncycastle.cms.SignerInformationStore;
import org.bouncycastle.cms.jcajce.JcaSignerInfoGeneratorBuilder;
import org.bouncycastle.operator.ContentSigner;
import org.bouncycastle.operator.DigestCalculatorProvider;
import org.bouncycastle.operator.jcajce.JcaContentSignerBuilder;
import org.bouncycastle.operator.jcajce.JcaDigestCalculatorProviderBuilder;
```

```
/**
 * Production-ready PDF signing service using PDFBox {pdfbox-version}
 * and BouncyCastle {bc-version}.
 *
 * Supports:
 * - PAdES-B-B (basic) and PAdES-B-T (timestamped) profiles
 * - Visible and invisible signatures
 * - Approval and certification (DocMDP) signatures
 * - Multi-signature workflows
 */
public class ProductionPdfSigner {

    /** Container size for signatures without timestamps. */
    private static final int CONTAINER_SIZE_BASIC = 32768;

    /** Container size for signatures with TSA timestamps. */
    private static final int CONTAINER_SIZE_WITH_TSA = 65536;

    /**
     * Signs a PDF document.
     *
     * @param pdfBytes    the original PDF bytes
     * @param privateKey  the signer's private key
     * @param certificate the signer's X.509 certificate
     * @param chain       the full certificate chain (signer + intermediates)
     * @param config      signing configuration
     * @return the signed PDF bytes (incremental save)
     */
    public byte[] sign(byte[] pdfBytes,
                       PrivateKey privateKey,
                       X509Certificate certificate,
                       List<X509Certificate> chain,
                       SigningConfig config) throws Exception {

        try (PDDocument doc = Loader.loadPDF(pdfBytes)) {

            // Resolve the target signature field
            PDAcroForm acroForm = doc.getDocumentCatalog().getAcroForm();
            if (acroForm == null) {
                throw new IllegalStateException("PDF has no AcroForm");
            }

            PDSignatureField sigField = findEmptySignatureField(
                    acroForm, config.fieldName());
            if (sigField == null) {
                throw new IllegalStateException(
                         "No empty signature field: " + config.fieldName());
            }

            // Build the /Sig dictionary
            PDSignature signature = new PDSignature();
            signature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
            signature.setSubFilter(
                    COSName.getPDFName("ETSI.CAdES.detached"));
```

```
        signature.setSignDate(Calendar.getInstance());
        signature.setName(extractCN(certificate));

        if (config.reason() != null) {
            signature.setReason(config.reason());
        }
        if (config.location() != null) {
            signature.setLocation(config.location());
        }

        // Certification signature (DocMDP)
        if (config.certification()) {
            applyCertification(doc, signature, config.docMdpLevel());
        }

        sigField.setValue(signature);

        // Determine container size
        boolean useTsa = config.tsaUrl() != null
                && !config.tsaUrl().isBlank();
        int containerSize = useTsa
                ? CONTAINER_SIZE_WITH_TSA : CONTAINER_SIZE_BASIC;

        SignatureOptions options = new SignatureOptions();
        options.setPreferredSignatureSize(containerSize);

        // Visible appearance
        if (config.visible()) {
            buildTextAppearance(doc, sigField,
                    extractCN(certificate), config.reason());
        }

        // CMS signing callback
        PadesCmsCallback callback = new PadesCmsCallback(
                privateKey, certificate, chain,
                useTsa ? config.tsaUrl() : null);

        doc.addSignature(signature, callback, options);

        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        doc.saveIncremental(baos);
        return baos.toByteArray();
    }
}

// ── Configuration record ──────────────────────────────────────────

/**
 * Immutable configuration for a signing operation.
 */
public record SigningConfig(
        String fieldName,
        String reason,
        String location,
        boolean visible,
```

```
        boolean certification,
        int docMdpLevel,
        String tsaUrl) {

    public SigningConfig {
        if (fieldName == null || fieldName.isBlank()) {
            throw new IllegalArgumentException(
                    "fieldName is required");
        }
    }
}

// ── CMS callback ──────────────────────────────────────────────

private static class PadesCmsCallback implements SignatureInterface {

    private final PrivateKey privateKey;
    private final X509Certificate certificate;
    private final List<X509Certificate> chain;
    private final String tsaUrl;

    PadesCmsCallback(PrivateKey privateKey,
                     X509Certificate certificate,
                     List<X509Certificate> chain,
                     String tsaUrl) {
        this.privateKey = privateKey;
        this.certificate = certificate;
        this.chain = chain;
        this.tsaUrl = tsaUrl;
    }

    @Override
    public byte[] sign(InputStream content) throws IOException {
        try {
            byte[] contentBytes = content.readAllBytes();

            // Determine signature algorithm from key type
            String keyAlgo = privateKey.getAlgorithm();
            String sigAlgo = "EC".equals(keyAlgo)
                    ? "SHA256withECDSA" : "SHA256withRSA";

            // Build certificate store
            JcaCertStore certStore = new JcaCertStore(chain);

            // Content signer
            ContentSigner signer = new JcaContentSignerBuilder(sigAlgo)
                    .setProvider("BC")
                    .build(privateKey);

            // Digest calculator
            DigestCalculatorProvider dcProvider =
                    new JcaDigestCalculatorProviderBuilder()
                            .setProvider("BC").build();

            // Build signer info with ESS-signing-certificate-v2
```

```
        JcaSignerInfoGeneratorBuilder signerBuilder =
                new JcaSignerInfoGeneratorBuilder(dcProvider);

        AttributeTable signedAttrs =
                buildEssSigningCertAttribute(certificate);
        signerBuilder.setSignedAttributeGenerator(
                new DefaultSignedAttributeTableGenerator(
                        signedAttrs));

        SignerInfoGenerator signerInfoGen =
                signerBuilder.build(signer, certificate);

        // Assemble the CMS SignedData
        CMSSignedDataGenerator gen =
                new CMSSignedDataGenerator();
        gen.addSignerInfoGenerator(signerInfoGen);
        gen.addCertificates(certStore);

        CMSProcessableByteArray cmsContent =
                new CMSProcessableByteArray(contentBytes);
        CMSSignedData signedData =
                gen.generate(cmsContent, false);

        // PAdES-B-T: add timestamp
        if (tsaUrl != null && !tsaUrl.isBlank()) {
            signedData = addTimestamp(signedData, tsaUrl);
        }

        return signedData.getEncoded();

    } catch (Exception e) {
        throw new IOException(
                "CMS signing failed: " + e.getMessage(), e);
    }
}

private AttributeTable buildEssSigningCertAttribute(
        X509Certificate cert) throws Exception {
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    byte[] certHash = md.digest(cert.getEncoded());

    AlgorithmIdentifier hashAlgo = new AlgorithmIdentifier(
            NISTObjectIdentifiers.id_sha256);
    ESSCertIDv2 essCertId =
            new ESSCertIDv2(hashAlgo, certHash);
    SigningCertificateV2 sigCertV2 =
            new SigningCertificateV2(
                    new ESSCertIDv2[]{essCertId});

    ASN1EncodableVector v = new ASN1EncodableVector();
    v.add(new Attribute(
            PKCSObjectIdentifiers.id_aa_signingCertificateV2,
            new DERSet(sigCertV2)));

    return new AttributeTable(v);
```

```
    }

    private CMSSignedData addTimestamp(CMSSignedData signedData,
                                       String tsaUrl)
            throws Exception {
        SignerInformation signerInfo = signedData
                .getSignerInfos().getSigners().iterator().next();
        byte[] sigBytes = signerInfo.getSignature();

        // Request timestamp (see TsaClient class below)
        byte[] tsToken = TsaClient.getTimestamp(tsaUrl, sigBytes);

        ASN1Primitive tsAsn1 =
                ASN1Primitive.fromByteArray(tsToken);

        ASN1EncodableVector vec = new ASN1EncodableVector();
        vec.add(new Attribute(
                PKCSObjectIdentifiers.id_aa_signatureTimeStampToken,
                new DERSet(tsAsn1)));
        AttributeTable unsignedAttrs = new AttributeTable(vec);

        SignerInformation newSigner =
                SignerInformation.replaceUnsignedAttributes(
                        signerInfo, unsignedAttrs);

        return CMSSignedData.replaceSigners(signedData,
                new SignerInformationStore(
                        Collections.singletonList(newSigner)));
    }
}

// ── TSA client ──────────────────────────────────────────────────────

/**
 * Minimal RFC 3161 TSA client.
 * See Chapter 18 for a full discussion of the timestamp protocol.
 */
static class TsaClient {

    static byte[] getTimestamp(String tsaUrl, byte[] signatureBytes)
            throws Exception {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] hash = md.digest(signatureBytes);

        org.bouncycastle.tsp.TimeStampRequestGenerator reqGen =
                new org.bouncycastle.tsp.TimeStampRequestGenerator();
        reqGen.setCertReq(true);
        org.bouncycastle.tsp.TimeStampRequest tsReq =
                reqGen.generate(NISTObjectIdentifiers.id_sha256, hash);

        java.net.http.HttpClient client =
                java.net.http.HttpClient.newHttpClient();
        java.net.http.HttpRequest request =
                java.net.http.HttpRequest.newBuilder()
                        .uri(java.net.URI.create(tsaUrl))
```

```
                        .header("Content-Type",
                                "application/timestamp-query")
                        .POST(java.net.http.HttpRequest.BodyPublishers
                                .ofByteArray(tsReq.getEncoded()))
                        .build();

        java.net.http.HttpResponse<byte[]> response = client.send(
                request,
                java.net.http.HttpResponse.BodyHandlers.ofByteArray());

        if (response.statusCode() != 200) {
            throw new IOException("TSA returned HTTP "
                    + response.statusCode());
        }

        org.bouncycastle.tsp.TimeStampResponse tsResp =
                new org.bouncycastle.tsp.TimeStampResponse(
                        response.body());
        tsResp.validate(tsReq);

        return tsResp.getTimeStampToken().getEncoded();
    }
}

// ── Appearance builder ──────────────────────────────────────────

private void buildTextAppearance(PDDocument doc,
                                 PDSignatureField sigField,
                                 String name, String reason)
        throws Exception {

    PDRectangle rect = sigField.getWidgets().isEmpty() ? null
            : sigField.getWidgets().get(0).getRectangle();
    if (rect == null || rect.getWidth() < 1
            || rect.getHeight() < 1) {
        return;
    }

    float w = rect.getWidth();
    float h = rect.getHeight();
    PDResources resources = new PDResources();
    PDType1Font font = new PDType1Font(
            Standard14Fonts.FontName.HELVETICA);
    COSName fontName = resources.add(font);

    float fontSize = 9f;
    float y = h - fontSize - 3;
    StringBuilder ops = new StringBuilder();
    ops.append("BT\n");
    ops.append("/").append(fontName.getName())
            .append(" ").append(fontSize).append(" Tf\n");
    ops.append("0 0 0 rg\n");
    ops.append("3 ").append(y).append(" Td\n");
    ops.append("(Digitally signed by: ")
            .append(escapePs(name)).append(") Tj\n");
```

```
    if (reason != null) {
        ops.append("0 -").append(fontSize * 1.3f).append(" Td\n");
        ops.append("(Reason: ").append(escapePs(reason))
                .append(") Tj\n");
    }
    ops.append("ET\n");

    PDFormXObject form = new PDFormXObject(doc);
    form.setBBox(new PDRectangle(w, h));
    form.setResources(resources);
    try (OutputStream os = form.getStream().createOutputStream()) {
        os.write(ops.toString()
                .getBytes(StandardCharsets.US_ASCII));
    }

    COSDictionary apDict = new COSDictionary();
    apDict.setItem(COSName.N, form);
    sigField.getWidgets().get(0).getCOSObject()
            .setItem(COSName.AP, apDict);
}

// ── Certification ──────────────────────────────────────────────

private void applyCertification(PDDocument doc,
                                PDSignature signature,
                                int level) {
    int p = (level >= 1 && level <= 3) ? level : 2;

    COSDictionary sigRef = new COSDictionary();
    sigRef.setItem(COSName.TYPE, COSName.getPDFName("SigRef"));
    sigRef.setItem(COSName.getPDFName("TransformMethod"),
            COSName.getPDFName("DocMDP"));

    COSDictionary params = new COSDictionary();
    params.setItem(COSName.TYPE,
            COSName.getPDFName("TransformParams"));
    params.setItem(COSName.getPDFName("P"),
            COSInteger.get(p));
    params.setItem(COSName.getPDFName("V"),
            COSName.getPDFName("1.2"));
    sigRef.setItem(COSName.getPDFName("TransformParams"), params);

    COSArray refArray = new COSArray();
    refArray.add(sigRef);
    signature.getCOSObject()
            .setItem(COSName.getPDFName("Reference"), refArray);

    COSDictionary perms = new COSDictionary();
    perms.setItem(COSName.getPDFName("DocMDP"),
            signature.getCOSObject());
    doc.getDocumentCatalog().getCOSObject()
            .setItem(COSName.getPDFName("Perms"), perms);
}

// ── Helpers ────────────────────────────────────────────────────
```

```
    private PDSignatureField findEmptySignatureField(PDAcroForm acroForm,
                                                     String fieldName) {
        for (PDField field : acroForm.getFieldTree()) {
            if (field instanceof PDSignatureField sf) {
                String fqn = sf.getFullyQualifiedName();
                String partial = sf.getPartialName();
                if ((fieldName.equals(fqn) || fieldName.equals(partial))
                        && sf.getSignature() == null) {
                    return sf;
                }
            }
        }
        return null;
    }

    private String extractCN(X509Certificate cert) {
        String dn = cert.getSubjectX500Principal().getName();
        java.util.regex.Matcher m = java.util.regex.Pattern
                .compile("CN=([^,]+)", java.util.regex.Pattern.CASE_INSENSITIVE)
                .matcher(dn);
        return m.find() ? m.group(1).trim() : "Unknown";
    }

    private static String escapePs(String text) {
        return text.replace("\\", "\\\\")
                   .replace("(", "\\(")
                   .replace(")", "\\)");
    }
}
```

This class covers the full production workflow. It produces PAdES-B-B compliant signatures by default (with `ESS-signing-certificate-v2` and `ETSI.CAdES.detached`), upgrades to PAdES-B-T when a TSA URL is provided, supports visible and invisible appearances, and handles certification signatures with DocMDP.

In the next chapter, we go deep into the BouncyCastle CMS construction that powers the `PadesCmsCallback` — every signed attribute, every ASN.1 structure, every decision point that determines whether your signature is a basic CMS blob or a fully PAdES-compliant `SignedData`.

Chapter 33

BouncyCastle CMS Construction

Chapter 29 showed how PDFBox orchestrates the signing process — creating the signature dictionary, computing the byte range, calling your `sign()` method, and writing the result into `/Contents`. This chapter goes inside that `sign()` method. We are now in BouncyCastle territory, building the CMS `SignedData` structure that is the actual cryptographic heart of every PDF signature.

If you have ever looked at a CMS signature in an ASN.1 dump and wondered what each field means, why certain attributes are mandatory for PAdES, or how a timestamp token gets attached — this chapter answers all of those questions with complete, compilable code.

We cover: `CMSSignedDataGenerator` and its components, provider registration, building the `ContentSigner`, the four mandatory signed attributes for PAdES-B-B, the `ESS-signing-certificate-v2` attribute in detail, algorithm negotiation between RSA and ECDSA, timestamp tokens as unsigned attributes, debugging CMS with ASN1Dump and openssl, and complete builder classes for PAdES-B-B and PAdES-B-T.

33.1 The CMS SignedData Structure: A Quick Refresher

Before we write code, let us recall what we are building. A CMS `SignedData` structure (RFC 5652 — Cryptographic Message Syntax (CMS) S5) contains:

```
SignedData ::= SEQUENCE {
    version            CMSVersion,
    digestAlgorithms   SET OF AlgorithmIdentifier,
    encapContentInfo   EncapsulatedContentInfo,
    certificates    [0] IMPLICIT CertificateSet OPTIONAL,
    crls            [1] IMPLICIT RevocationInfoChoices OPTIONAL,
    signerInfos        SET OF SignerInfo
}
```

For a PDF signature:

- `version` is 1 (one signer, detached content)
- `digestAlgorithms` contains `SHA-256` (or SHA-384, SHA-512)
- `encapContentInfo` has `contentType = id-data` and *no content* (detached signature — the content lives in the PDF)
- `certificates` contains the signer's certificate and the chain (up to but not including the root)
- `crls` is typically empty (revocation data goes in the DSS for PAdES-B-LT)
- `signerInfos` contains exactly one `SignerInfo` with signed attributes, the signature value, and optionally unsigned attributes (the timestamp)

The `SignerInfo` is where most of the action is:

```
SignerInfo ::= SEQUENCE {
    version              CMSVersion,
    sid                  SignerIdentifier,
    digestAlgorithm      AlgorithmIdentifier,
    signedAttrs      [0] IMPLICIT SignedAttributes OPTIONAL,
    signatureAlgorithm   AlgorithmIdentifier,
    signature            SignatureValue,
    unsignedAttrs    [1] IMPLICIT UnsignedAttributes OPTIONAL
}
```

The `signedAttrs` (signed attributes) are hashed and signed — they form the binding between the document content and the signer's identity. The `unsignedAttrs` (unsigned attributes) are *not* covered by the signature — this is where the timestamp goes, because the timestamp is created *after* the signature.

33.2 BouncyCastle Provider Registration

Every BouncyCastle crypto operation requires the provider to be registered with the JCA (Java Cryptography Architecture). Without it, calls to `JcaContentSignerBuilder`, `JcaDigestCalculatorProviderBuilder`, and `JcaCertStore` fail with `NoSuchProviderException`.

```
import org.bouncycastle.jce.provider.BouncyCastleProvider;

// Register once, at application startup
if (java.security.Security.getProvider("BC") == null) {
    java.security.Security.addProvider(new BouncyCastleProvider());
}
```

In a Spring Boot application, the cleanest approach is a configuration class:

```
import jakarta.annotation.PostConstruct;
import org.bouncycastle.jce.provider.BouncyCastleProvider;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BouncyCastleConfig {

    @PostConstruct
    public void init() {
        if (java.security.Security.getProvider("BC") == null) {
            java.security.Security.addProvider(new BouncyCastleProvider());
        }
    }
}
```

If you register the provider in a `static` block, it races with other static initializers. In complex applications with multiple classloaders (application servers, OSGi), the provider may not be visible where you need it. The `@PostConstruct` approach avoids both problems.

You can verify the registration by calling `java.security.Security.getProvider("BC")`. If it returns `null`, BouncyCastle is not registered. If it returns a `Provider` object, check its version: `provider.getVersion()` should return `1.80` (or whatever version you have on the classpath).

33.3 Building a Basic CMS Signature

Let us build a CMS `SignedData` from scratch, step by step. We will start with the simplest possible version and then add PAdES requirements.

33.3.1 Step 1: The ContentSigner

The `ContentSigner` is BouncyCastle's abstraction over the private key and signature algorithm. It provides an `OutputStream` that you write data into, and when you are done, it produces the raw signature bytes.

```
import org.bouncycastle.operator.ContentSigner;
import org.bouncycastle.operator.jcajce.JcaContentSignerBuilder;

// For RSA keys:
ContentSigner signer = new JcaContentSignerBuilder("SHA256withRSA")
        .setProvider("BC")
        .build(privateKey);

// For EC keys:
ContentSigner signer = new JcaContentSignerBuilder("SHA256withECDSA")
        .setProvider("BC")
        .build(privateKey);
```

The `JcaContentSignerBuilder` takes a JCA algorithm string and a private key. The `.setProvider("BC")` ensures BouncyCastle handles the actual signing, not the JDK's default provider (which may not support all algorithms or may behave differently for edge cases like PSS padding).

33.3.2 Step 2: The DigestCalculatorProvider

BouncyCastle needs a way to compute message digests. The `DigestCalculatorProvider` is the factory for digest calculators:

```
import org.bouncycastle.operator.DigestCalculatorProvider;
import org.bouncycastle.operator.jcajce.JcaDigestCalculatorProviderBuilder;

DigestCalculatorProvider dcProvider =
        new JcaDigestCalculatorProviderBuilder()
                .setProvider("BC")
                .build();
```

This provider is used internally by the signer info generator to compute the message digest of the signed content and the signed attributes.

33.3.3 Step 3: The SignerInfoGenerator

The `SignerInfoGenerator` combines the `ContentSigner`, the `DigestCalculatorProvider`, and the signer's certificate into a component that can produce a `SignerInfo`:

```
import org.bouncycastle.cms.SignerInfoGenerator;
import org.bouncycastle.cms.jcajce.JcaSignerInfoGeneratorBuilder;

SignerInfoGenerator signerInfoGen =
```

```
        new JcaSignerInfoGeneratorBuilder(dcProvider)
                .build(signer, certificate);
```

At this point, the `signerInfoGen` knows which key to use, which algorithm, which certificate identifies the signer, and how to compute digests. It does *not* yet know about signed attributes — we will add those next.

33.3.4 Step 4: The CMSSignedDataGenerator

Now we wire everything together:

```
import org.bouncycastle.cert.jcajce.JcaCertStore;
import org.bouncycastle.cms.CMSProcessableByteArray;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.CMSSignedDataGenerator;

// Certificate store -- signer cert + chain
List<X509Certificate> certList = new ArrayList<>(chain);
JcaCertStore certStore = new JcaCertStore(certList);

CMSSignedDataGenerator gen = new CMSSignedDataGenerator();
gen.addSignerInfoGenerator(signerInfoGen);
gen.addCertificates(certStore);

// The content to be signed -- this is the byte range content from PDFBox
CMSProcessableByteArray cmsContent =
        new CMSProcessableByteArray(contentBytes);

// Generate the SignedData -- false = detached (content not embedded)
CMSSignedData signedData = gen.generate(cmsContent, false);

// Get the DER-encoded result
byte[] cmsBytes = signedData.getEncoded();
```

- The certificates are embedded in the CMS `certificates` field. Include the signer's certificate and all intermediate certificates. Do *not* include the root CA certificate — validators are expected to have the root in their trust store already, and including it wastes space without adding trust.
- The second parameter `false` means detached signature. For PDF signatures, this must always be `false`. If you pass `true`, the content is embedded inside the CMS structure, which doubles the size and does not match what PDF validators expect for `ETSI.CAdES.detached` or `adbe.pkcs7.detached`.

33.4 Signed Attributes: What PAdES Requires

A basic CMS signature (without signed attributes) signs the content directly — the hash of the content is what the signature covers. When signed attributes are present, the signature covers the hash of the *signed attributes* instead, and the signed attributes contain the hash of the content. This indirection is how CMS allows metadata (signing time, content type, certificate binding) to be included in the signed data.

For PAdES-B-B compliance (ETSI EN 319 142-1 V1.1.1 (2016-04)), four signed attributes are

required.

Table 33.1: Mandatory Signed Attributes for PAdES-B-B

Attribute	OID	Purpose
`content-type`	1.2.840.113549.1.9.3	Identifies the type of content being signed. Must be `id-data` (OID 1.2.840.113549.1.7.1). BouncyCastle adds this automatically.
`message-digest`	1.2.840.113549.1.9.4	The hash of the signed content (the byte range bytes). BouncyCastle computes and adds this automatically.
`signing-time`	1.2.840.113549.1.9.5	The claimed signing time. BouncyCastle adds this automatically when using `DefaultSignedAttributeTableGenerator`. Note: PAdES validators prefer the TSA timestamp over this value.
`ESS-signing-certificate-v2`	1.2.840.113549.1.9.16.2.47	Binds the signature to a specific certificate by its hash. This is the one you must add manually. Without it, you have a valid CMS signature but *not* a PAdES-B-B signature.

BouncyCastle's `DefaultSignedAttributeTableGenerator` automatically adds `content-type`, `message-digest`, and `signing-time`. You only need to explicitly construct `ESS-signing-certificate-v2` and pass it in. The generator merges your custom attributes with the automatic ones.

33.5 Creating ESS-signing-certificate-v2

The `ESS-signing-certificate-v2` attribute (RFC 5652 — Cryptographic Message Syntax (CMS) and RFC 5035) prevents certificate substitution attacks. It contains a hash of the signer's certificate, so a validator can verify that the certificate embedded in the CMS `certificates` field is the one the signer intended to use.

The ASN.1 structure:

```
SigningCertificateV2 ::= SEQUENCE {
    certs    SEQUENCE OF ESSCertIDv2,
    policies SEQUENCE OF PolicyInformation OPTIONAL
}

ESSCertIDv2 ::= SEQUENCE {
    hashAlgorithm  AlgorithmIdentifier DEFAULT {sha-256},
    certHash       OCTET STRING,
    issuerSerial   IssuerSerial OPTIONAL
}
```

Here is the complete code to build this attribute:

```
import java.security.MessageDigest;
import java.security.cert.X509Certificate;

import org.bouncycastle.asn1.ASN1EncodableVector;
import org.bouncycastle.asn1.DERSet;
import org.bouncycastle.asn1.cms.Attribute;
import org.bouncycastle.asn1.cms.AttributeTable;
import org.bouncycastle.asn1.ess.ESSCertIDv2;
import org.bouncycastle.asn1.ess.SigningCertificateV2;
import org.bouncycastle.asn1.nist.NISTObjectIdentifiers;
import org.bouncycastle.asn1.pkcs.PKCSObjectIdentifiers;
import org.bouncycastle.asn1.x509.AlgorithmIdentifier;

/**
 * Builds the ESS-signing-certificate-v2 signed attribute
 * required for PAdES-B-B compliance.
 */
public class EssCertAttributeBuilder {

    /**
     * Creates an AttributeTable containing the
     * ESS-signing-certificate-v2 attribute.
     *
     * @param certificate the signer's X.509 certificate
     * @return an AttributeTable that can be passed to
     *         DefaultSignedAttributeTableGenerator
     */
    public static AttributeTable build(X509Certificate certificate)
            throws Exception {

        // 1. Hash the DER-encoded certificate with SHA-256
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] certHash = md.digest(certificate.getEncoded());

        // 2. Create the AlgorithmIdentifier for SHA-256
        AlgorithmIdentifier hashAlgo = new AlgorithmIdentifier(
                NISTObjectIdentifiers.id_sha256);

        // 3. Create the ESSCertIDv2 (hash + algorithm)
        ESSCertIDv2 essCertId = new ESSCertIDv2(hashAlgo, certHash);

        // 4. Wrap in SigningCertificateV2 (array of ESSCertIDv2)
        SigningCertificateV2 sigCertV2 = new SigningCertificateV2(
                new ESSCertIDv2[]{essCertId});

        // 5. Create the CMS Attribute
        ASN1EncodableVector v = new ASN1EncodableVector();
        v.add(new Attribute(
                PKCSObjectIdentifiers.id_aa_signingCertificateV2,
                new DERSet(sigCertV2)));

        return new AttributeTable(v);
    }
```

```
}
```

- `certificate.getEncoded()` returns the DER-encoded X.509 certificate. We hash this entire byte sequence, not just the public key or the subject name. This ensures that any change to the certificate (even adding an extension) changes the hash.
- The hash algorithm identifier must match the algorithm used to compute the hash. SHA-256 is the standard choice. If you use SHA-384 or SHA-512 for the signature algorithm, you may use the matching digest here, but SHA-256 is always acceptable.
- The `SigningCertificateV2` wraps an array of `ESSCertIDv2`. For a single signer, there is one entry. If your signing certificate was issued by an intermediate CA, you may optionally include the intermediate's hash as well, but this is not required by PAdES.
- The OID `1.2.840.113549.1.9.16.2.47` identifies this as the `ESS-signing-certificate-v2` attribute. Do not confuse it with the older `ESS-signing-certificate` (v1, OID 1.2.840.113549.1.9.16.2.12), which uses SHA-1. PAdES requires v2.

Using `ESS-signing-certificate` v1 (with SHA-1) instead of v2 (with SHA-256) is a PAdES compliance failure. Some older code examples on the internet still use v1. SHA-1 is considered cryptographically broken for collision resistance since the SHAttered attack (2017). Always use v2 with SHA-256 or stronger.

33.5.1 Wiring ESS-signing-certificate-v2 into the Generator

Now we pass the attribute table to the `SignerInfoGenerator`:

```
import org.bouncycastle.cms.DefaultSignedAttributeTableGenerator;
import org.bouncycastle.cms.jcajce.JcaSignerInfoGeneratorBuilder;

// Build the ESS attribute table
AttributeTable essAttrs = EssCertAttributeBuilder.build(certificate);

// Create a generator that merges ESS attribute with the automatic ones
DefaultSignedAttributeTableGenerator attrGen =
        new DefaultSignedAttributeTableGenerator(essAttrs);

JcaSignerInfoGeneratorBuilder signerBuilder =
        new JcaSignerInfoGeneratorBuilder(dcProvider);
signerBuilder.setSignedAttributeGenerator(attrGen);

SignerInfoGenerator signerInfoGen =
        signerBuilder.build(signer, certificate);
```

- `DefaultSignedAttributeTableGenerator` accepts an initial `AttributeTable` and *merges* it with the attributes it generates automatically (`content-type`, `message-digest`, `signing-time`). Your `ESS-signing-certificate-v2` is included alongside the automatic attributes. You do not need to manually construct the other three.

33.6 Algorithm Negotiation: RSA vs. ECDSA

The signature algorithm depends on the private key type. You must match the algorithm string to the key, or the `ContentSigner` construction fails.

```
/**
 * Determines the JCA signature algorithm string for a given private key.
 * Supports RSA, RSA-PSS, ECDSA, Ed25519, and Ed448.
 */
public static String resolveSignatureAlgorithm(PrivateKey privateKey) {
    String keyAlgo = privateKey.getAlgorithm();

    return switch (keyAlgo) {
        case "RSA" -> "SHA256withRSA";
        case "RSASSA-PSS" -> "SHA256withRSAandMGF1";
        case "EC", "ECDSA" -> "SHA256withECDSA";
        case "Ed25519" -> "Ed25519";
        case "Ed448" -> "Ed448";
        default -> throw new IllegalArgumentException(
                "Unsupported key algorithm: " + keyAlgo);
    };
}
```

- RSA-PSS (Probabilistic Signature Scheme) is increasingly preferred over PKCS#1 v1.5 padding (`SHA256withRSA`). Some EU validation services flag PKCS#1 v1.5 as deprecated. If your key was generated as RSASSA-PSS, you must use the PSS algorithm string.
- Ed25519 and Ed448 (Edwards curves) use the algorithm name directly — there is no separate hash algorithm because the hash is built into the signature scheme. These are not yet widely supported by PDF validators (Adobe Acrobat does not validate Ed25519 signatures as of 2026), but they are valid per ISO 32000-2:2020 and will gain support over time.

Table 33.2: Signature Algorithm Matrix

Key Type	Algorithm String	Hash	Notes
RSA-2048+	`SHA256withRSA`	SHA-256	Most compatible. Works everywhere.
RSA-2048+	`SHA256withRSAandMGF1`	SHA-256	RSA-PSS. Better security properties. Some legacy validators do not support it.
RSA-3072+	`SHA384withRSA`	SHA-384	For higher security requirements.
EC P-256	`SHA256withECDSA`	SHA-256	Compact signatures (64 bytes vs. 256 for RSA-2048). Widely supported.
EC P-384	`SHA384withECDSA`	SHA-384	Higher security level.
EC P-521	`SHA512withECDSA`	SHA-512	Maximum EC security. Rarely needed.
Ed25519	`Ed25519`	Built-in	Modern curve. Limited validator support (as of 2026).

Key Type	Algorithm String	Hash	Notes
Ed448	`Ed448`	Built-in	Higher security Ed curve. Very limited support.

For maximum compatibility in 2026, use `SHA256withRSA` (RSA-2048 key) or `SHA256withECDSA` (P-256 key). These work with every PDF validator we have tested, including Adobe Acrobat, Foxit Reader, and the EU DSS validation service.

33.7 Timestamp Token as Unsigned Attribute

A PAdES-B-T signature contains a timestamp token as an unsigned attribute on the `SignerInfo`. The timestamp proves when the signature was created.

The attribute is identified by OID `1.2.840.113549.1.9.16.2.14` (`id-aa-signatureTimeStampToken`), and its value is a complete CMS `SignedData` structure — the timestamp is itself a CMS signature, created by the TSA.

33.7.1 The Timestamping Flow

1. Generate the CMS `SignedData` for the document signature (PAdES-B-B)
2. Extract the raw signature value (`SignerInfo.signature`)
3. Hash the signature value with SHA-256
4. Send the hash to the TSA in an RFC 3161 `TimeStampRequest`
5. Receive the `TimeStampResponse`, extract the `TimeStampToken`
6. Encode the token as an ASN.1 structure
7. Attach it as an unsigned attribute on the `SignerInfo`
8. Replace the `SignerInfo` in the `CMSSignedData`
9. Return the modified `CMSSignedData`

33.7.2 Complete Implementation

```
import java.security.MessageDigest;
import java.util.Collections;

import org.bouncycastle.asn1.ASN1EncodableVector;
import org.bouncycastle.asn1.ASN1Primitive;
import org.bouncycastle.asn1.DERSet;
import org.bouncycastle.asn1.cms.Attribute;
import org.bouncycastle.asn1.cms.AttributeTable;
import org.bouncycastle.asn1.nist.NISTObjectIdentifiers;
import org.bouncycastle.asn1.pkcs.PKCSObjectIdentifiers;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.SignerInformation;
import org.bouncycastle.cms.SignerInformationStore;
import org.bouncycastle.tsp.TimeStampRequest;
import org.bouncycastle.tsp.TimeStampRequestGenerator;
import org.bouncycastle.tsp.TimeStampResponse;
```

```
import org.bouncycastle.tsp.TimeStampToken;

/**
 * Adds an RFC 3161 timestamp to a CMS SignedData, upgrading it
 * from PAdES-B-B to PAdES-B-T.
 */
public class TimestampEmbedder {

    /**
     * Embeds a timestamp token as an unsigned attribute.
     *
     * @param signedData the original CMS SignedData (PAdES-B-B)
     * @param tsaUrl     the RFC 3161 TSA endpoint URL
     * @return the modified CMS SignedData (PAdES-B-T)
     */
    public static CMSSignedData addTimestamp(CMSSignedData signedData,
                                             String tsaUrl)
            throws Exception {

        // Get the first (only) signer info
        SignerInformation signerInfo = signedData.getSignerInfos()
                .getSigners().iterator().next();

        // The timestamp covers the signature value, not the content
        byte[] signatureBytes = signerInfo.getSignature();

        // Hash the signature for the TSA request
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] hash = md.digest(signatureBytes);

        // Build and send the RFC 3161 request
        TimeStampRequestGenerator reqGen =
                new TimeStampRequestGenerator();
        reqGen.setCertReq(true);
        TimeStampRequest tsReq = reqGen.generate(
                NISTObjectIdentifiers.id_sha256, hash);

        byte[] tsRespBytes = sendTsaRequest(tsaUrl, tsReq.getEncoded());

        // Parse and validate the response
        TimeStampResponse tsResponse =
                new TimeStampResponse(tsRespBytes);
        tsResponse.validate(tsReq);

        TimeStampToken tsToken = tsResponse.getTimeStampToken();
        if (tsToken == null) {
            throw new IllegalStateException(
                    "TSA response contains no timestamp token");
        }

        // Convert to ASN.1 for embedding
        ASN1Primitive tsAsn1 = ASN1Primitive.fromByteArray(
                tsToken.getEncoded());

        // Build the unsigned attribute
```

```
    ASN1EncodableVector unsignedVec = new ASN1EncodableVector();
    unsignedVec.add(new Attribute(
            PKCSObjectIdentifiers.id_aa_signatureTimeStampToken,
            new DERSet(tsAsn1)));
    AttributeTable unsignedAttrs = new AttributeTable(unsignedVec);

    // Replace the signer's unsigned attributes
    SignerInformation newSignerInfo =
            SignerInformation.replaceUnsignedAttributes(
                    signerInfo, unsignedAttrs);

    SignerInformationStore newSignerInfos =
            new SignerInformationStore(
                    Collections.singletonList(newSignerInfo));

    return CMSSignedData.replaceSigners(
            signedData, newSignerInfos);
}

/** Sends an RFC 3161 request to a TSA endpoint via HTTP POST. */
private static byte[] sendTsaRequest(String tsaUrl, byte[] reqBytes)
        throws Exception {

    java.net.http.HttpClient client =
            java.net.http.HttpClient.newBuilder()
                    .connectTimeout(java.time.Duration.ofSeconds(15))
                    .build();

    java.net.http.HttpRequest request =
            java.net.http.HttpRequest.newBuilder()
                    .uri(java.net.URI.create(tsaUrl))
                    .header("Content-Type",
                            "application/timestamp-query")
                    .timeout(java.time.Duration.ofSeconds(30))
                    .POST(java.net.http.HttpRequest.BodyPublishers
                            .ofByteArray(reqBytes))
                    .build();

    java.net.http.HttpResponse<byte[]> response = client.send(
            request,
            java.net.http.HttpResponse.BodyHandlers.ofByteArray());

    if (response.statusCode() != 200) {
        throw new java.io.IOException(
                "TSA returned HTTP " + response.statusCode()
                        + " from " + tsaUrl);
    }

    byte[] body = response.body();
    if (body == null || body.length == 0) {
        throw new java.io.IOException(
                "TSA returned empty response from " + tsaUrl);
    }

    return body;
```

```
    }
}
```

- `getSignature()` returns the raw signature bytes from the `SignerInfo` — this is the value that was produced by the `ContentSigner`. For RSA-2048, this is 256 bytes. For ECDSA P-256, it is approximately 72 bytes (DER-encoded).
- `setCertReq(true)` tells the TSA to include its signing certificate in the response. This is important: without the TSA's certificate, a validator cannot verify the timestamp without separately fetching the TSA's certificate.
- `validate()` checks that the response corresponds to our request (matching nonce, hash algorithm, and hash value). If the TSA returned a response for a different request, this throws a `TSPValidationException`.
- The timestamp token is wrapped in a `DERSet` and assigned OID `1.2.840.113549.1.9.16.2.14`. This is the standard way to embed a timestamp in CMS (RFC 5652 — Cryptographic Message Syntax (CMS)).
- `replaceUnsignedAttributes()` creates a new `SignerInformation` with the same signed attributes and signature value, but with the specified unsigned attributes. The original `SignerInformation` is immutable.

33.8 Debugging CMS Structures

When a signature fails validation, you need to look inside the CMS blob. There are two indispensable tools: BouncyCastle's `ASN1Dump` (Java) and OpenSSL (command line).

33.8.1 ASN1Dump: Java-Side Inspection

```
import org.bouncycastle.asn1.ASN1InputStream;
import org.bouncycastle.asn1.ASN1Primitive;
import org.bouncycastle.asn1.util.ASN1Dump;

/**
 * Dumps the ASN.1 structure of a CMS blob to stdout.
 * Invaluable for debugging signature construction.
 */
public static void dumpCms(byte[] cmsBytes) throws Exception {
    try (ASN1InputStream asn1In = new ASN1InputStream(cmsBytes)) {
        ASN1Primitive obj = asn1In.readObject();
        String dump = ASN1Dump.dumpAsString(obj, true);
        System.out.println(dump);
    }
}
```

- The `true` parameter enables verbose mode, which shows the full content of OCTET STRING and BIT STRING values. Without it, large values are truncated.

The output looks like this (truncated for readability):

```
SEQUENCE
  OBJECT IDENTIFIER 1.2.840.113549.1.7.2    (signedData)
  [0]
    SEQUENCE
```

```
INTEGER 1                                (version)
SET
  SEQUENCE
    OBJECT IDENTIFIER 2.16.840.1.101.3.4.2.1 (SHA-256)
SEQUENCE
  OBJECT IDENTIFIER 1.2.840.113549.1.7.1     (data)
[0]                                      (certificates)
  SEQUENCE
    ...                                  (X.509 certificate)
SET
  SEQUENCE                               (SignerInfo)
    INTEGER 1                            (version)
    SEQUENCE                             (sid - issuerAndSerialNumber)
      ...
    SEQUENCE                             (digestAlgorithm)
      OBJECT IDENTIFIER 2.16.840.1.101.3.4.2.1 (SHA-256)
    [0]                                  (signedAttrs)
      SEQUENCE
        OBJECT IDENTIFIER 1.2.840.113549.1.9.3  (content-type)
        ...
      SEQUENCE
        OBJECT IDENTIFIER 1.2.840.113549.1.9.4  (message-digest)
        ...
      SEQUENCE
        OBJECT IDENTIFIER 1.2.840.113549.1.9.5  (signing-time)
        ...
      SEQUENCE
        OBJECT IDENTIFIER 1.2.840.113549.1.9.16.2.47 (ESS-signing-cert-v2)
        ...
    SEQUENCE                             (signatureAlgorithm)
      OBJECT IDENTIFIER 1.2.840.113549.1.1.1 (RSA)
    OCTET STRING                         (signature value)
      ...
    [1]                                  (unsignedAttrs)
      SEQUENCE
        OBJECT IDENTIFIER 1.2.840.113549.1.9.16.2.14 (timestamp)
        ...
```

When debugging, check for:

1. Is `ESS-signing-certificate-v2` (OID ...16.2.47) present in the signed attributes? If not, the signature is not PAdES-B-B compliant.
2. Is `message-digest` present? If not, the signature structure is malformed.
3. Is the timestamp (OID ...16.2.14) in the unsigned attributes? If not, it is not PAdES-B-T.
4. Are certificates present in the `[0]` IMPLICIT set? If not, validators cannot build the chain.

33.8.2 OpenSSL: Command-Line Inspection

Extract the CMS blob from the PDF (it is the hex-encoded content of `/Contents`), decode it, and use OpenSSL:

```
# Decode the hex string from /Contents to binary
xxd -r -p signature-hex.txt > signature.der

# Parse the CMS structure
openssl cms -inform DER -in signature.der -cmsout -print

# Verify the CMS signature (requires the signed content)
openssl cms -verify -inform DER -in signature.der \
    -content signed-bytes.bin -noverify -out /dev/null

# Extract and examine the signing certificate
openssl cms -inform DER -in signature.der -cmsout -print \
    | grep -A 50 "cert:"

# View the timestamp token (if present)
openssl ts -reply -in timestamp-token.der -text
```

To extract the `/Contents` hex string from a PDF, search for `/Contents <` in the raw file. The hex string starts after the `<` and ends at `>`. It is zero-padded — strip trailing zeros before decoding. You can also use PDFBox's `PDSignature.getContents(byte[])` to extract the raw bytes programmatically.

33.9 Complete PAdES-B-B Builder

Here is a self-contained class that builds a PAdES-B-B compliant CMS `SignedData`:

```
import java.io.IOException;
import java.io.InputStream;
import java.security.MessageDigest;
import java.security.PrivateKey;
import java.security.cert.X509Certificate;
import java.util.ArrayList;
import java.util.List;

import org.bouncycastle.asn1.ASN1EncodableVector;
import org.bouncycastle.asn1.DERSet;
import org.bouncycastle.asn1.cms.Attribute;
import org.bouncycastle.asn1.cms.AttributeTable;
import org.bouncycastle.asn1.ess.ESSCertIDv2;
import org.bouncycastle.asn1.ess.SigningCertificateV2;
import org.bouncycastle.asn1.nist.NISTObjectIdentifiers;
import org.bouncycastle.asn1.pkcs.PKCSObjectIdentifiers;
import org.bouncycastle.asn1.x509.AlgorithmIdentifier;
import org.bouncycastle.cert.jcajce.JcaCertStore;
import org.bouncycastle.cms.CMSProcessableByteArray;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.CMSSignedDataGenerator;
import org.bouncycastle.cms.DefaultSignedAttributeTableGenerator;
import org.bouncycastle.cms.SignerInfoGenerator;
```

```
import org.bouncycastle.cms.jcajce.JcaSignerInfoGeneratorBuilder;
import org.bouncycastle.operator.ContentSigner;
import org.bouncycastle.operator.DigestCalculatorProvider;
import org.bouncycastle.operator.jcajce.JcaContentSignerBuilder;
import org.bouncycastle.operator.jcajce.JcaDigestCalculatorProviderBuilder;

/**
 * Builds a PAdES-B-B compliant CMS SignedData structure.
 *
 * Implements SignatureInterface for use with PDFBox's
 * PDDocument.addSignature().
 *
 * Produces:
 * - Detached CMS SignedData (SubFilter: ETSI.CAdES.detached)
 * - Signed attributes: content-type, message-digest, signing-time,
 *   ESS-signing-certificate-v2
 * - Certificate chain embedded in CMS certificates field
 */
public class PadesBBBuilder
        implements org.apache.pdfbox.pdmodel.interactive
                .digitalsignature.SignatureInterface {

    private final PrivateKey privateKey;
    private final X509Certificate certificate;
    private final List<X509Certificate> chain;

    public PadesBBBuilder(PrivateKey privateKey,
                          X509Certificate certificate,
                          List<X509Certificate> chain) {
        this.privateKey = privateKey;
        this.certificate = certificate;
        this.chain = chain;
    }

    @Override
    public byte[] sign(InputStream content) throws IOException {
        try {
            byte[] contentBytes = content.readAllBytes();
            return buildSignedData(contentBytes);
        } catch (IOException e) {
            throw e;
        } catch (Exception e) {
            throw new IOException("PAdES-B-B signing failed: "
                    + e.getMessage(), e);
        }
    }

    /**
     * Builds the CMS SignedData from the byte range content.
     * This method can also be called directly for testing.
     */
    public byte[] buildSignedData(byte[] contentBytes) throws Exception {

        // 1. Determine signature algorithm
        String sigAlgo = resolveAlgorithm(privateKey);
```

```
    // 2. Content signer
    ContentSigner signer = new JcaContentSignerBuilder(sigAlgo)
            .setProvider("BC")
            .build(privateKey);

    // 3. Digest calculator provider
    DigestCalculatorProvider dcProvider =
            new JcaDigestCalculatorProviderBuilder()
                    .setProvider("BC").build();

    // 4. ESS-signing-certificate-v2 attribute
    AttributeTable essAttrs = buildEssAttribute(certificate);

    // 5. Signer info generator with custom signed attributes
    JcaSignerInfoGeneratorBuilder signerBuilder =
            new JcaSignerInfoGeneratorBuilder(dcProvider);
    signerBuilder.setSignedAttributeGenerator(
            new DefaultSignedAttributeTableGenerator(essAttrs));

    SignerInfoGenerator signerInfoGen =
            signerBuilder.build(signer, certificate);

    // 6. Certificate store
    List<X509Certificate> certs = new ArrayList<>(chain);
    JcaCertStore certStore = new JcaCertStore(certs);

    // 7. Assemble and generate
    CMSSignedDataGenerator gen = new CMSSignedDataGenerator();
    gen.addSignerInfoGenerator(signerInfoGen);
    gen.addCertificates(certStore);

    CMSProcessableByteArray cmsContent =
            new CMSProcessableByteArray(contentBytes);
    CMSSignedData signedData = gen.generate(cmsContent, false);

    return signedData.getEncoded();
}

private static AttributeTable buildEssAttribute(
        X509Certificate cert) throws Exception {
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    byte[] certHash = md.digest(cert.getEncoded());

    AlgorithmIdentifier hashAlgo = new AlgorithmIdentifier(
            NISTObjectIdentifiers.id_sha256);
    ESSCertIDv2 essCertId = new ESSCertIDv2(hashAlgo, certHash);
    SigningCertificateV2 sigCertV2 = new SigningCertificateV2(
            new ESSCertIDv2[]{essCertId});

    ASN1EncodableVector v = new ASN1EncodableVector();
    v.add(new Attribute(
            PKCSObjectIdentifiers.id_aa_signingCertificateV2,
            new DERSet(sigCertV2)));
    return new AttributeTable(v);
```

```
    }

    private static String resolveAlgorithm(PrivateKey key) {
        String algo = key.getAlgorithm();
        return switch (algo) {
            case "RSA" -> "SHA256withRSA";
            case "RSASSA-PSS" -> "SHA256withRSAandMGF1";
            case "EC", "ECDSA" -> "SHA256withECDSA";
            case "Ed25519" -> "Ed25519";
            case "Ed448" -> "Ed448";
            default -> throw new IllegalArgumentException(
                    "Unsupported key algorithm: " + algo);
        };
    }
}
```

33.10 Complete PAdES-B-T Builder

The PAdES-B-T builder extends B-B by adding a timestamp after the signature is generated:

```
import java.io.IOException;
import java.io.InputStream;
import java.security.MessageDigest;
import java.security.PrivateKey;
import java.security.cert.X509Certificate;
import java.util.Collections;
import java.util.List;

import org.bouncycastle.asn1.ASN1EncodableVector;
import org.bouncycastle.asn1.ASN1Primitive;
import org.bouncycastle.asn1.DERSet;
import org.bouncycastle.asn1.cms.Attribute;
import org.bouncycastle.asn1.cms.AttributeTable;
import org.bouncycastle.asn1.nist.NISTObjectIdentifiers;
import org.bouncycastle.asn1.pkcs.PKCSObjectIdentifiers;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.SignerInformation;
import org.bouncycastle.cms.SignerInformationStore;
import org.bouncycastle.tsp.TimeStampRequest;
import org.bouncycastle.tsp.TimeStampRequestGenerator;
import org.bouncycastle.tsp.TimeStampResponse;
import org.bouncycastle.tsp.TimeStampToken;

/**
 * Builds a PAdES-B-T compliant CMS SignedData structure.
 *
 * Extends PAdES-B-B by adding an RFC 3161 timestamp token
 * as an unsigned attribute on the SignerInfo.
 *
 * Usage with PDFBox:
 * <pre>
 * PadesBTBuilder builder = new PadesBTBuilder(
 *     privateKey, certificate, chain, "http://tsa.example.com");
 * doc.addSignature(signature, builder, options);
```

```
 * doc.saveIncremental(outputStream);
 * </pre>
 */
public class PadesBTBuilder
        implements org.apache.pdfbox.pdmodel.interactive
                .digitalsignature.SignatureInterface {

    private final PadesBBBuilder bbBuilder;
    private final String tsaUrl;

    public PadesBTBuilder(PrivateKey privateKey,
                          X509Certificate certificate,
                          List<X509Certificate> chain,
                          String tsaUrl) {
        this.bbBuilder = new PadesBBBuilder(
                privateKey, certificate, chain);
        this.tsaUrl = tsaUrl;
    }

    @Override
    public byte[] sign(InputStream content) throws IOException {
        try {
            byte[] contentBytes = content.readAllBytes();

            // Step 1: Build PAdES-B-B SignedData
            byte[] bbBytes = bbBuilder.buildSignedData(contentBytes);
            CMSSignedData signedData = new CMSSignedData(bbBytes);

            // Step 2: Add timestamp
            CMSSignedData timestamped =
                    embedTimestamp(signedData, tsaUrl);

            return timestamped.getEncoded();
        } catch (IOException e) {
            throw e;
        } catch (Exception e) {
            throw new IOException("PAdES-B-T signing failed: "
                    + e.getMessage(), e);
        }
    }

    private static CMSSignedData embedTimestamp(
            CMSSignedData signedData, String tsaUrl)
            throws Exception {

        // Extract signature value
        SignerInformation signerInfo = signedData.getSignerInfos()
                .getSigners().iterator().next();
        byte[] sigBytes = signerInfo.getSignature();

        // Hash for TSA request
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] hash = md.digest(sigBytes);

        // Build RFC 3161 request
```

```
        TimeStampRequestGenerator reqGen =
                new TimeStampRequestGenerator();
        reqGen.setCertReq(true);
        TimeStampRequest tsReq = reqGen.generate(
                NISTObjectIdentifiers.id_sha256, hash);

        // Send to TSA
        byte[] tsRespBytes = sendTsaRequest(tsaUrl, tsReq.getEncoded());

        // Parse and validate
        TimeStampResponse tsResp = new TimeStampResponse(tsRespBytes);
        tsResp.validate(tsReq);

        int status = tsResp.getStatus();
        if (status != 0 && status != 1) {
            String msg = tsResp.getStatusString();
            throw new IOException("TSA error: "
                    + (msg != null ? msg : "status=" + status));
        }

        TimeStampToken tsToken = tsResp.getTimeStampToken();
        if (tsToken == null) {
            throw new IOException("TSA returned no timestamp token");
        }

        // Embed as unsigned attribute
        ASN1Primitive tsAsn1 = ASN1Primitive.fromByteArray(
                tsToken.getEncoded());

        ASN1EncodableVector vec = new ASN1EncodableVector();
        vec.add(new Attribute(
                PKCSObjectIdentifiers.id_aa_signatureTimeStampToken,
                new DERSet(tsAsn1)));
        AttributeTable unsignedAttrs = new AttributeTable(vec);

        SignerInformation newSignerInfo =
                SignerInformation.replaceUnsignedAttributes(
                        signerInfo, unsignedAttrs);

        return CMSSignedData.replaceSigners(signedData,
                new SignerInformationStore(
                        Collections.singletonList(newSignerInfo)));
    }

    private static byte[] sendTsaRequest(String tsaUrl, byte[] reqBytes)
            throws Exception {

        java.net.http.HttpClient client =
                java.net.http.HttpClient.newBuilder()
                        .connectTimeout(java.time.Duration.ofSeconds(15))
                        .followRedirects(
                                java.net.http.HttpClient.Redirect.NORMAL)
                        .build();

        java.net.http.HttpRequest request =
```

```
                java.net.http.HttpRequest.newBuilder()
                        .uri(java.net.URI.create(tsaUrl))
                        .header("Content-Type",
                                "application/timestamp-query")
                        .timeout(java.time.Duration.ofSeconds(30))
                        .POST(java.net.http.HttpRequest.BodyPublishers
                                .ofByteArray(reqBytes))
                        .build();

        java.net.http.HttpResponse<byte[]> response = client.send(
                request,
                java.net.http.HttpResponse.BodyHandlers.ofByteArray());

        if (response.statusCode() != 200) {
            throw new IOException("TSA HTTP " + response.statusCode()
                    + " from " + tsaUrl);
        }

        byte[] body = response.body();
        if (body == null || body.length == 0) {
            throw new IOException("TSA empty response from " + tsaUrl);
        }

        return body;
    }
}
```

- TSA status 0 = `granted`, 1 = `grantedWithMods`. Any other status (2 = rejection, 3 = waiting, 4 = revocationWarning, 5 = revocationNotification) means the TSA refused to issue a timestamp. Handle this gracefully — some TSAs reject requests during maintenance windows.

33.11 Putting It All Together

Here is how you use the PAdES-B-B and PAdES-B-T builders with PDFBox:

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.cos.COSName;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.SignatureInterface;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.SignatureOptions;

import java.io.ByteArrayOutputStream;
import java.util.Calendar;

// --- PAdES-B-B (no timestamp) ---
SignatureInterface bbSigner = new PadesBBBuilder(
        privateKey, certificate, chain);

try (PDDocument doc = Loader.loadPDF(pdfBytes)) {
    PDSignature sig = new PDSignature();
    sig.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
    sig.setSubFilter(COSName.getPDFName("ETSI.CAdES.detached"));
```

```
    sig.setSignDate(Calendar.getInstance());

    SignatureOptions opts = new SignatureOptions();
    opts.setPreferredSignatureSize(32768);

    doc.addSignature(sig, bbSigner, opts);

    ByteArrayOutputStream out = new ByteArrayOutputStream();
    doc.saveIncremental(out);
    // out.toByteArray() is the PAdES-B-B signed PDF
}

// --- PAdES-B-T (with timestamp) ---
SignatureInterface btSigner = new PadesBTBuilder(
        privateKey, certificate, chain,
        "http://timestamp.example.com/tsa");

try (PDDocument doc = Loader.loadPDF(pdfBytes)) {
    PDSignature sig = new PDSignature();
    sig.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
    sig.setSubFilter(COSName.getPDFName("ETSI.CAdES.detached"));
    sig.setSignDate(Calendar.getInstance());

    SignatureOptions opts = new SignatureOptions();
    opts.setPreferredSignatureSize(65536); // larger for timestamp

    doc.addSignature(sig, btSigner, opts);

    ByteArrayOutputStream out = new ByteArrayOutputStream();
    doc.saveIncremental(out);
    // out.toByteArray() is the PAdES-B-T signed PDF
}
```

The `PadesBBBuilder` and `PadesBTBuilder` classes are production-ready. They handle algorithm negotiation, ESS attribute construction, certificate embedding, and timestamp integration. Drop them into your project alongside the BouncyCastle and PDFBox dependencies from Chapter 28, register the BouncyCastle provider, and you have a complete signing pipeline.

In Chapter 31, we reverse the process: given a signed PDF, we verify the CMS signature, validate the certificate chain, check the timestamp, and extract every piece of information that these structures contain.

Chapter 34

Verification & Analysis with PDFBox

Signing a PDF is a matter of following a recipe. Verifying a signed PDF is where things get interesting — and where most implementations get things wrong. PDFBox gives you every primitive you need: signature extraction, byte range access, CMS parsing, content stream reading. But it deliberately does not give you a ”verify this signature” method. You build the pipeline yourself, step by step, and you understand every failure mode because you wrote the code that handles it.

This chapter walks through a complete verification implementation using Apache PDFBox 3.0.0 and BouncyCastle 1.80. We start with extracting signatures from a PDF document, proceed through each verification step from Chapter 16, and end with a production-ready five-step pipeline that handles the edge cases real-world PDFs throw at you. If Chapter 16 described *what* to verify, this chapter shows you *how* — with code you can compile and run.

34.1 Extracting Signatures from a PDF Document

34.1.1 The Extraction Path: AcroForm to PDSignature

Every PDF signature lives inside the AcroForm field tree. The extraction path follows the object hierarchy we dissected in Chapter 12: the document’s `/AcroForm` dictionary contains a `/Fields` array, which includes `PDSignatureField` entries, each of which references a `PDSignature` dictionary.

PDFBox models this hierarchy directly. The key classes are:

- `PDAcroForm` — the form dictionary, accessed via `PDDocument.getDocumentCatalog().getAcroForm()`
- `PDSignatureField` — a form field of type `/Sig`, extends `PDTerminalField`
- `PDSignature` — the actual `/Type /Sig` dictionary containing `/Contents`, `/ByteRange`, `/SubFilter`, etc.

There are two ways to access signatures in PDFBox. The first walks the field tree:

```
/**
 * Extract all signature fields from the AcroForm field tree.
 * This approach gives you access to the field (widget, appearance)
 * in addition to the signature dictionary.
 */
public List<PDSignatureField> extractSignatureFields(PDDocument document) {
    PDAcroForm acroForm = document.getDocumentCatalog().getAcroForm();
    if (acroForm == null) {
        log.info("Document has no AcroForm — no signatures possible");
        return Collections.emptyList();
    }
```

```
    List<PDSignatureField> signatureFields = new ArrayList<>();
    for (PDField field : acroForm.getFields()) {
        collectSignatureFields(field, signatureFields);
    }

    log.info("Found {} signature field(s) in AcroForm", signatureFields.size());
    return signatureFields;
}

private void collectSignatureFields(PDField field,
        List<PDSignatureField> result) {
    if (field instanceof PDSignatureField sigField) {
        result.add(sigField);
    } else if (field instanceof PDNonTerminalField nonTerminal) {
        for (PDField child : nonTerminal.getChildren()) {
            collectSignatureFields(child, result);
        }
    }
}
```

The second approach bypasses the field tree and goes directly to the signature dictionaries:

```
/**
 * Extract all PDSignature dictionaries directly.
 * Faster, but you lose access to the field and widget objects.
 * Useful when you only need the cryptographic data.
 */
public List<PDSignature> extractSignatures(PDDocument document) {
    List<PDSignature> signatures = document.getSignatureDictionaries();
    log.info("Found {} signature dictionaries", signatures.size());
    return signatures;
}
```

Use the first approach (field tree walk) when you need the full picture — field name, widget annotation, visual appearance, page location. Use the second approach when you only need the cryptographic payload — for pure verification, it is faster and simpler.

34.1.2 Reading Signature Metadata

Once you have a `PDSignature`, you can extract the metadata that drives the verification pipeline:

```
public void logSignatureMetadata(PDSignature sig) {
    log.info("SubFilter: {}", sig.getSubFilter());
    log.info("Filter: {}", sig.getFilter());
    log.info("Name: {}", sig.getName());
    log.info("Location: {}", sig.getLocation());
    log.info("Reason: {}", sig.getReason());
    log.info("Sign date: {}", sig.getSignDate());
    log.info("Contact info: {}", sig.getContactInfo());

    int[] byteRange = sig.getByteRange();
    if (byteRange != null && byteRange.length == 4) {
        log.info("ByteRange: [{} {} {} {}]",
                byteRange[0], byteRange[1], byteRange[2], byteRange[3]);
```

```
        log.info("Signed bytes: {} + {} = {} total",
                byteRange[1], byteRange[3],
                byteRange[1] + byteRange[3]);
        log.info("Gap (Contents): offset {} to {}, size {} bytes",
                byteRange[1], byteRange[2],
                byteRange[2] - byteRange[1]);
    }
}
```

The `/SubFilter` value tells you what kind of signature you are dealing with:

- `adbe.pkcs7.detached` — Classic PKCS#7 (CMS) detached signature
- `ETSI.CAdES.detached` — PAdES CAdES-based signature (the modern standard)
- `adbe.pkcs7.sha1` — Legacy: SHA-1 hash embedded in PKCS#7 (deprecated, avoid)
- `adbe.x509.rsa_sha1` — Legacy: bare RSA signature with X.509 cert (deprecated, avoid)

For any new code, you should handle `adbe.pkcs7.detached` and `ETSI.CAdES.detached`. The legacy formats exist in old documents, and you may encounter them, but you should never produce them.

34.2 Parsing the CMS SignedData from /Contents

34.2.1 Extracting the Raw CMS Bytes

The `/Contents` value in the signature dictionary is a hex-encoded byte string containing the DER-encoded CMS `SignedData` structure. PDFBox provides this as a byte array:

```
byte[] cmsBytes = signature.getContents();
```

This single line hides a meaningful amount of work. PDFBox reads the hex string from the PDF (which can be 16KB to 64KB or more of hex characters), decodes it from hex to binary, and strips any trailing zero padding. The result is the raw DER-encoded CMS `SignedData` that BouncyCastle can parse.

34.2.2 Parsing with BouncyCastle

To parse the CMS structure, you also need the signed bytes — the byte range segments from the PDF file. For a detached signature (which is what `adbe.pkcs7.detached` and `ETSI.CAdES.detached` are), the signed content is not inside the CMS; it is the external byte range data.

```
/**
 * Parse a CMS SignedData from the /Contents of a PDF signature.
 *
 * @param pdfBytes      the complete PDF file as a byte array
 * @param signature     the PDSignature whose CMS we want to parse
 * @return the parsed CMSSignedData, ready for verification
 */
public CMSSignedData parseCmsSignedData(byte[] pdfBytes,
        PDSignature signature) throws CMSException {

    // Extract the signed byte ranges
    int[] byteRange = signature.getByteRange();
```

```
    byte[] signedContent = new byte[byteRange[1] + byteRange[3]];
    System.arraycopy(pdfBytes, byteRange[0], signedContent,
            0, byteRange[1]);
    System.arraycopy(pdfBytes, byteRange[2], signedContent,
            byteRange[1], byteRange[3]);

    // Parse the CMS with the signed content as the detached data
    CMSProcessableByteArray processable =
            new CMSProcessableByteArray(signedContent);
    return new CMSSignedData(processable, signature.getContents());
}
```

A common mistake is to construct `CMSSignedData` with only the `/Contents` bytes (without the signed content). This will parse the CMS structure, but any subsequent verification call will fail because BouncyCastle cannot compute the message digest without the signed data. Always pass both the signed content and the CMS bytes.

34.2.3 Navigating the CMS Structure

Once parsed, the `CMSSignedData` object exposes the full CMS tree from RFC 5652 — Cryptographic Message Syntax (CMS):

```
public void inspectCmsStructure(CMSSignedData cmsData) {
    // Signer information (usually exactly one for PDF signatures)
    SignerInformationStore signerStore = cmsData.getSignerInfos();
    log.info("Number of signers: {}", signerStore.size());

    for (SignerInformation signer : signerStore.getSigners()) {
        log.info("Digest algorithm OID: {}", signer.getDigestAlgOID());
        log.info("Encryption algorithm OID: {}",
                signer.getEncryptionAlgOID());

        // Signed attributes (authenticated)
        AttributeTable signedAttrs = signer.getSignedAttributes();
        if (signedAttrs != null) {
            log.info("Signed attributes count: {}",
                    signedAttrs.size());
            logAttribute(signedAttrs, CMSAttributes.contentType,
                    "contentType");
            logAttribute(signedAttrs, CMSAttributes.messageDigest,
                    "messageDigest");
            logAttribute(signedAttrs, CMSAttributes.signingTime,
                    "signingTime");
            logAttribute(signedAttrs,
                    PKCSObjectIdentifiers
                            .id_aa_signingCertificateV2,
                    "ESS-signing-certificate-v2");
        }

        // Unsigned attributes (unauthenticated — timestamps live here)
        AttributeTable unsignedAttrs = signer.getUnsignedAttributes();
        if (unsignedAttrs != null) {
            log.info("Unsigned attributes count: {}",
                    unsignedAttrs.size());
```

```
            logAttribute(unsignedAttrs,
                    PKCSObjectIdentifiers
                            .id_aa_signatureTimeStampToken,
                    "signatureTimeStampToken");
        }
    }

    // Embedded certificates
    Store<X509CertificateHolder> certStore = cmsData.getCertificates();
    Collection<X509CertificateHolder> certs =
            certStore.getMatches(null);
    log.info("Embedded certificates: {}", certs.size());
    for (X509CertificateHolder cert : certs) {
        log.info("  Subject: {}", cert.getSubject());
        log.info("  Issuer: {}", cert.getIssuer());
        log.info("  Serial: {}", cert.getSerialNumber());
        log.info("  Valid: {} to {}",
                cert.getNotBefore(), cert.getNotAfter());
    }
}

private void logAttribute(AttributeTable table,
        ASN1ObjectIdentifier oid, String name) {
    Attribute attr = table.get(oid);
    if (attr != null) {
        log.info("  {} present, values: {}",
                name, attr.getAttrValues().size());
    }
}
```

34.3 Byte Range Verification in Code

34.3.1 The Three Checks

Byte range verification (Step 1 from Chapter 16) requires three checks:

1. The `/ByteRange` array has exactly four elements
2. The first offset is zero (the signed region starts at the beginning of the file)
3. The second segment extends to the end of the file (or the end of the revision for multi-signature documents)

```
/**
 * Verify the structural integrity of the byte range.
 *
 * @param byteRange   the four-element ByteRange array
 * @param fileSize    the total size of the PDF file in bytes
 * @return a ValidationResult indicating success or the specific failure
 */
public ValidationResult verifyByteRange(int[] byteRange, long fileSize) {
    if (byteRange == null || byteRange.length != 4) {
        log.error("ByteRange is null or does not have exactly 4 elements");
        return ValidationResult.BYTE_RANGE_MALFORMED;
    }
```

```
        int offset1 = byteRange[0];
        int length1 = byteRange[1];
        int offset2 = byteRange[2];
        int length2 = byteRange[3];

        // Check 1: First segment starts at byte 0
        if (offset1 != 0) {
            log.error("ByteRange offset1 is {} — expected 0", offset1);
            return ValidationResult.BYTE_RANGE_NOT_AT_START;
        }

        // Check 2: Segments don't overlap
        if (offset2 < length1) {
            log.error("ByteRange segments overlap: " +
                    "segment1 ends at {}, segment2 starts at {}",
                    length1, offset2);
            return ValidationResult.BYTE_RANGE_OVERLAP;
        }

        // Check 3: Signed region extends to end of file
        long signedEnd = (long) offset2 + length2;
        if (signedEnd != fileSize) {
            long trailingBytes = fileSize - signedEnd;
            log.warn("ByteRange covers {} bytes but file is {} bytes — " +
                    "{} trailing unsigned bytes detected",
                    signedEnd, fileSize, trailingBytes);
            return ValidationResult.BYTE_RANGE_TRAILING_BYTES;
        }

        log.info("ByteRange verified: [{} {} {} {}], " +
                "file size {}, no trailing bytes",
                offset1, length1, offset2, length2, fileSize);
        return ValidationResult.OK;
    }
```

Trailing bytes are not always an attack. In a multi-signature document, each signature covers only up to the end of its own incremental revision. Subsequent revisions (containing later signatures) appear as ”trailing bytes” from the perspective of the earlier signature. Chapter 8 covers how incremental updates work, and Chapter 13 covers the byte range geometry in multi-signature scenarios.

However, trailing bytes on the *last* signature in a document are suspicious and may indicate a Shadow Attack (Chapter 24). Your verification code should distinguish between ”trailing bytes that are a legitimate later revision” and ”trailing bytes that are unexplained.”

34.3.2 Validating the Contents Gap

The gap between the two segments should contain exactly the `/Contents` hex string. You can validate this by checking that the gap bytes, when decoded from hex, produce valid DER-encoded data:

```
    /**
     * Validate that the gap between byte range segments contains
     * a properly formatted hex string with valid CMS data.
```

```
 */
public boolean validateContentsGap(byte[] pdfBytes, int[] byteRange) {
    int gapStart = byteRange[1];
    int gapEnd = byteRange[2];
    int gapSize = gapEnd - gapStart;

    // The gap should start with '<' and end with '>'
    if (pdfBytes[gapStart] != '<' || pdfBytes[gapEnd - 1] != '>') {
        log.error("Gap does not start with '<' or end with '>'");
        return false;
    }

    // Extract hex content (skip the delimiters)
    String hexContent = new String(pdfBytes, gapStart + 1,
            gapSize - 2, StandardCharsets.US_ASCII).trim();

    // Verify it's valid hex
    if (!hexContent.matches("[0-9a-fA-F]+")) {
        log.error("Gap contains non-hex characters");
        return false;
    }

    log.info("Contents gap validated: {} hex chars = {} bytes CMS",
            hexContent.length(), hexContent.length() / 2);
    return true;
}
```

34.4 CMS Cryptographic Verification

34.4.1 Building the SignerInformationVerifier

This is Step 3 from Chapter 16 — verifying that the CMS `SignedData` was actually created by the claimed signer's private key. BouncyCastle's `SignerInformation.verify()` method performs this check, but you need to provide the correct `SignerInformationVerifier`:

```
/**
 * Verify the CMS signature cryptographically.
 *
 * @param cmsData  the parsed CMS SignedData (with signed content)
 * @return the signer's certificate if verification succeeds, null otherwise
 */
public X509CertificateHolder verifyCmsSignature(CMSSignedData cmsData)
        throws Exception {

    SignerInformationStore signerStore = cmsData.getSignerInfos();
    if (signerStore.size() == 0) {
        log.error("CMS contains no signers");
        return null;
    }

    SignerInformation signerInfo =
            signerStore.getSigners().iterator().next();

    // Find the signer's certificate by matching the SignerIdentifier
```

```
        Store<X509CertificateHolder> certStore = cmsData.getCertificates();
        @SuppressWarnings("unchecked")
        Collection<X509CertificateHolder> matches =
                certStore.getMatches(signerInfo.getSID());

        if (matches.isEmpty()) {
            log.error("No certificate matches the SignerIdentifier — " +
                    "the signer's cert may not be embedded in the CMS");
            return null;
        }

        X509CertificateHolder signerCert = matches.iterator().next();
        log.info("Signer certificate: subject={}, issuer={}",
                signerCert.getSubject(), signerCert.getIssuer());

        // Build the verifier and verify
        SignerInformationVerifier verifier =
                new JcaSimpleSignerInfoVerifierBuilder()
                        .setProvider("BC")
                        .build(signerCert);

        boolean valid = signerInfo.verify(verifier);
        if (valid) {
            log.info("CMS signature verification PASSED");
            return signerCert;
        } else {
            log.error("CMS signature verification FAILED — " +
                    "signature was not created by the claimed certificate");
            return null;
        }
    }
```

34.4.2 What verify() Actually Does

When you call `signerInfo.verify(verifier)`, BouncyCastle performs the following internally:

1. Computes the digest of the signed content (the byte range data you passed when constructing `CMSSignedData`) using the algorithm from `SignerInfo.digestAlgorithm`
2. Checks that this computed digest matches the `messageDigest` attribute in the signed attributes
3. DER-encodes the signed attributes
4. Verifies the RSA/ECDSA/DSA signature over the DER-encoded signed attributes using the public key from the signer's certificate

If signed attributes are absent (which is technically allowed by CMS but rare in PDF signatures), BouncyCastle instead verifies the signature directly over the content digest.

The `verify()` method performs only the cryptographic verification — Steps 2 and 3 combined. It does not check certificate validity, revocation status, or trust. Those are separate steps you must implement yourself.

34.5 Certificate Chain Reconstruction and Validation

34.5.1 The Java PKIX CertPath API

Java's `java.security.cert` package provides a complete PKIX certificate path validation engine via `CertPathBuilder` and `CertPathValidator`. This is the standard way to validate certificate chains in Java, and it handles the complexity of chain building, validity checking, basic constraints verification, and key usage checking.

```
/**
 * Build and validate the certificate chain from the signer's certificate
 * to a trusted root.
 *
 * @param signerCert    the signer's certificate (from CMS verification)
 * @param cmsData       the CMS containing intermediate certificates
 * @param trustAnchors  the set of trusted root certificates
 * @param signingTime   the claimed signing time (for validity checks)
 * @return the validated CertPath, or null if validation fails
 */
public CertPath validateCertificateChain(
        X509CertificateHolder signerCert,
        CMSSignedData cmsData,
        Set<TrustAnchor> trustAnchors,
        Date signingTime) throws Exception {

    // Convert BouncyCastle cert holders to Java X509Certificates
    CertificateFactory cf = CertificateFactory.getInstance("X.509");
    JcaX509CertificateConverter converter =
            new JcaX509CertificateConverter().setProvider("BC");

    // Collect all certificates from CMS into a CertStore
    List<X509Certificate> allCerts = new ArrayList<>();
    Store<X509CertificateHolder> bcStore = cmsData.getCertificates();
    for (X509CertificateHolder holder : bcStore.getMatches(null)) {
        allCerts.add(converter.getCertificate(holder));
    }

    CertStore certStore = CertStore.getInstance("Collection",
            new CollectionCertStoreParameters(allCerts));

    // Configure the target certificate selector
    X509Certificate targetCert = converter.getCertificate(signerCert);
    X509CertSelector targetSelector = new X509CertSelector();
    targetSelector.setCertificate(targetCert);

    // Build PKIX parameters
    PKIXBuilderParameters params =
            new PKIXBuilderParameters(trustAnchors, targetSelector);
    params.addCertStore(certStore);
    params.setRevocationEnabled(false); // We handle revocation separately
    params.setDate(signingTime);        // Validate at signing time

    // Build the chain
    CertPathBuilder builder =
```

```
            CertPathBuilder.getInstance("PKIX", "BC");
    PKIXCertPathBuilderResult result =
            (PKIXCertPathBuilderResult) builder.build(params);

    CertPath certPath = result.getCertPath();
    log.info("Certificate chain built successfully — {} certificates",
            certPath.getCertificates().size());

    for (int i = 0; i < certPath.getCertificates().size(); i++) {
        X509Certificate cert =
                (X509Certificate) certPath.getCertificates().get(i);
        log.info("  [{}] Subject: {}, Issuer: {}",
                i, cert.getSubjectX500Principal(),
                cert.getIssuerX500Principal());
    }

    log.info("  [root] Trust anchor: {}",
            result.getTrustAnchor().getTrustedCert()
                    .getSubjectX500Principal());

    return certPath;
}
```

34.5.2 Building the Trust Anchor Set

The `TrustAnchor` set represents your trust store — the root certificates you are willing to trust. In a real application, this might come from the Java default trust store (`cacerts`), a custom keystore, or the AATL/EUTL (see Chapter 16, Step 5).

```
/**
 * Load trust anchors from a Java KeyStore (JKS or PKCS12).
 */
public Set<TrustAnchor> loadTrustAnchors(Path keystorePath,
        char[] password) throws Exception {

    KeyStore trustStore = KeyStore.getInstance(
            keystorePath.toString().endsWith(".p12")
                    ? "PKCS12" : "JKS");
    try (InputStream is = Files.newInputStream(keystorePath)) {
        trustStore.load(is, password);
    }

    Set<TrustAnchor> anchors = new HashSet<>();
    Enumeration<String> aliases = trustStore.aliases();
    while (aliases.hasMoreElements()) {
        String alias = aliases.nextElement();
        if (trustStore.isCertificateEntry(alias)) {
            X509Certificate cert =
                    (X509Certificate) trustStore.getCertificate(alias);
            anchors.add(new TrustAnchor(cert, null));
            log.info("Loaded trust anchor: {}",
                    cert.getSubjectX500Principal());
        }
    }
```

```
    log.info("Loaded {} trust anchors from {}", anchors.size(),
            keystorePath);
    return anchors;
}
```

34.5.3 Key Usage Validation

The PKIX `CertPathValidator` checks basic constraints and key usage automatically. But if you want to verify specific requirements — for example, that the signer's certificate has the `digitalSignature` or `nonRepudiation` key usage — you can check explicitly:

```
/**
 * Check that the signer's certificate permits digital signing.
 */
public boolean hasSigningKeyUsage(X509Certificate cert) {
    boolean[] keyUsage = cert.getKeyUsage();
    if (keyUsage == null) {
        log.warn("Certificate has no KeyUsage extension — " +
                "signing permitted by default");
        return true;
    }

    // bit 0 = digitalSignature, bit 1 = nonRepudiation (contentCommitment)
    boolean digitalSignature = keyUsage[0];
    boolean nonRepudiation = keyUsage[1];

    if (!digitalSignature && !nonRepudiation) {
        log.error("Certificate KeyUsage does not include " +
                "digitalSignature or nonRepudiation");
        return false;
    }

    log.info("Certificate KeyUsage: digitalSignature={}, " +
            "nonRepudiation={}", digitalSignature, nonRepudiation);
    return true;
}
```

34.6 Timestamp Verification

34.6.1 Extracting the Timestamp Token

A signature timestamp is an unsigned CMS attribute with OID `1.2.840.113549.1.9.16.2.14` (`id-aa-signatureTimeStampToken`). The attribute value is itself a CMS `SignedData` structure — a timestamp is a signature on a signature:

```
/**
 * Extract and verify the signature timestamp, if present.
 *
 * @param signerInfo  the CMS SignerInformation
 * @return the verified signing time, or null if no timestamp or invalid
 */
public Date verifyTimestamp(SignerInformation signerInfo) throws Exception {
```

```
AttributeTable unsignedAttrs = signerInfo.getUnsignedAttributes();
if (unsignedAttrs == null) {
    log.info("No unsigned attributes - no timestamp present");
    return null;
}

Attribute tsAttr = unsignedAttrs.get(
        PKCSObjectIdentifiers.id_aa_signatureTimeStampToken);
if (tsAttr == null) {
    log.info("No signatureTimeStampToken attribute found");
    return null;
}

// Parse the timestamp token
byte[] tsBytes = tsAttr.getAttrValues()
        .getObjectAt(0).toASN1Primitive().getEncoded();
CMSSignedData tsSignedData = new CMSSignedData(tsBytes);
TimeStampToken tsToken = new TimeStampToken(tsSignedData);

// Verify the timestamp's CMS signature
SignerInformation tsSigner = tsSignedData.getSignerInfos()
        .getSigners().iterator().next();
@SuppressWarnings("unchecked")
Collection<X509CertificateHolder> tsMatches =
        tsSignedData.getCertificates()
                .getMatches(tsSigner.getSID());

if (tsMatches.isEmpty()) {
    log.error("Timestamp token does not contain " +
            "the TSA's certificate");
    return null;
}

X509CertificateHolder tsCert = tsMatches.iterator().next();

// Verify the TSA certificate has timeStamping EKU
boolean hasTsEku = false;
if (tsCert.getExtensions() != null) {
    Extension ekuExt = tsCert.getExtensions().getExtension(
            Extension.extendedKeyUsage);
    if (ekuExt != null) {
        ExtendedKeyUsage eku =
                ExtendedKeyUsage.getInstance(ekuExt.getParsedValue());
        hasTsEku = eku.hasKeyPurposeId(
                KeyPurposeId.id_kp_timeStamping);
    }
}
if (!hasTsEku) {
    log.warn("TSA certificate does not have " +
            "id-kp-timeStamping EKU - " +
            "timestamp may not be trusted");
}

// Verify the timestamp's CMS cryptographic signature
SignerInformationVerifier tsVerifier =
```

```
            new JcaSimpleSignerInfoVerifierBuilder()
                    .setProvider("BC")
                    .build(tsCert);
    boolean tsValid = tsSigner.verify(tsVerifier);

    if (!tsValid) {
        log.error("Timestamp CMS signature verification FAILED");
        return null;
    }

    Date signingTime = tsToken.getTimeStampInfo().getGenTime();
    log.info("Timestamp verified — signing time: {}", signingTime);
    log.info("TSA: {}", tsCert.getSubject());
    log.info("Timestamp serial: {}",
            tsToken.getTimeStampInfo().getSerialNumber());
    log.info("Timestamp hash algorithm: {}",
            tsToken.getTimeStampInfo().getHashAlgorithm()
                    .getAlgorithm().getId());

    return signingTime;
}
```

34.6.2 Verifying the Timestamp Covers the Signature

A valid timestamp is not just a signed time — it must also prove that the *signature value* existed at that time. The timestamp's message imprint digest must match the hash of the signature value from the `SignerInfo`:

```
/**
 * Verify that the timestamp's message imprint matches the signature value.
 */
public boolean verifyTimestampCoversSignature(
        TimeStampToken tsToken,
        SignerInformation signerInfo) throws Exception {

    // The timestamp's imprint should be the hash of the signature value
    byte[] signatureBytes = signerInfo.getSignature();
    String tsHashAlg = tsToken.getTimeStampInfo()
            .getHashAlgorithm().getAlgorithm().getId();
    String algName = mapOidToAlgorithm(tsHashAlg);

    MessageDigest digest = MessageDigest.getInstance(algName);
    byte[] computedImprint = digest.digest(signatureBytes);

    byte[] claimedImprint =
            tsToken.getTimeStampInfo().getMessageImprintDigest();

    boolean match = MessageDigest.isEqual(computedImprint, claimedImprint);
    if (match) {
        log.info("Timestamp message imprint matches signature value");
    } else {
        log.error("Timestamp message imprint does NOT match " +
                "signature value — timestamp may cover different data");
    }
```

```
    return match;
}
```

34.7 The Complete Five-Step Verification Pipeline

34.7.1 Putting It All Together

Here is the complete verification pipeline that combines every step from this chapter. Each step is independent — a failure at one step does not prevent the other steps from running. The pipeline produces a comprehensive `VerificationReport` with per-step results:

```
/**
 * Complete signature verification pipeline.
 * Runs all five verification steps and produces a comprehensive report.
 */
public VerificationReport verifySignature(byte[] pdfBytes,
        PDSignature signature,
        Set<TrustAnchor> trustAnchors) {

    VerificationReport report = new VerificationReport();

    try {
        // === Step 1: Byte Range Structural Integrity ===
        log.info("Step 1: Verifying byte range...");
        int[] byteRange = signature.getByteRange();
        ValidationResult byteRangeResult =
                verifyByteRange(byteRange, pdfBytes.length);
        report.setByteRangeResult(byteRangeResult);

        if (byteRangeResult != ValidationResult.OK
                && byteRangeResult
                    != ValidationResult.BYTE_RANGE_TRAILING_BYTES) {
            log.error("Step 1 FAILED: {}", byteRangeResult);
            return report; // Cannot proceed without valid byte range
        }

        // === Step 2: Hash Verification ===
        log.info("Step 2: Verifying message digest...");
        CMSSignedData cmsData = parseCmsSignedData(pdfBytes, signature);
        SignerInformation signerInfo = cmsData.getSignerInfos()
                .getSigners().iterator().next();

        // Compute the hash of the signed bytes
        String digestAlgOid = signerInfo.getDigestAlgOID();
        String algName = mapOidToAlgorithm(digestAlgOid);
        MessageDigest md = MessageDigest.getInstance(algName);
        md.update(pdfBytes, byteRange[0], byteRange[1]);
        md.update(pdfBytes, byteRange[2], byteRange[3]);
        byte[] computedHash = md.digest();

        // Compare to the CMS messageDigest attribute
        Attribute digestAttr = signerInfo.getSignedAttributes()
                .get(CMSAttributes.messageDigest);
        byte[] claimedHash = ASN1OctetString.getInstance(
```

```
        digestAttr.getAttrValues().getObjectAt(0)).getOctets();

boolean hashMatch =
        MessageDigest.isEqual(computedHash, claimedHash);
report.setHashResult(hashMatch
        ? ValidationResult.OK
        : ValidationResult.HASH_MISMATCH);

if (!hashMatch) {
    log.error("Step 2 FAILED: computed hash does not match " +
            "CMS messageDigest — document was modified");
} else {
    log.info("Step 2 PASSED: hash matches");
}

// === Step 3: CMS Cryptographic Verification ===
log.info("Step 3: Verifying CMS signature...");
X509CertificateHolder signerCert =
        verifyCmsSignature(cmsData);
report.setCmsResult(signerCert != null
        ? ValidationResult.OK
        : ValidationResult.CMS_INVALID);

if (signerCert == null) {
    log.error("Step 3 FAILED: CMS verification failed");
    return report;
}
log.info("Step 3 PASSED: CMS signature valid");

// === Step 4: Certificate Chain Validation ===
log.info("Step 4: Validating certificate chain...");
Date validationDate = new Date(); // Default: now
try {
    // Check for timestamp first — use timestamp time if available
    Date tsTime = verifyTimestamp(signerInfo);
    if (tsTime != null) {
        validationDate = tsTime;
        report.setTimestampTime(tsTime);
        report.setTimestampResult(ValidationResult.OK);
        log.info("Using timestamp time for chain validation: {}",
                tsTime);
    }

    CertPath certPath = validateCertificateChain(
            signerCert, cmsData, trustAnchors, validationDate);
    report.setChainResult(certPath != null
            ? ValidationResult.OK
            : ValidationResult.CHAIN_INVALID);
    report.setCertPath(certPath);
} catch (CertPathBuilderException e) {
    log.error("Step 4 FAILED: {}", e.getMessage());
    report.setChainResult(ValidationResult.CHAIN_INVALID);
    report.setChainError(e.getMessage());
}
```

```
        // === Step 5: Trust Evaluation ===
        log.info("Step 5: Evaluating trust...");
        if (report.getChainResult() == ValidationResult.OK) {
            report.setTrustResult(ValidationResult.OK);
            log.info("Step 5 PASSED: certificate chains to " +
                    "a trusted root");
        } else {
            report.setTrustResult(ValidationResult.NOT_TRUSTED);
            log.warn("Step 5: trust could not be established");
        }

    } catch (Exception e) {
        log.error("Verification pipeline failed with exception", e);
        report.setException(e);
    }

    log.info("Verification complete: {}", report.getSummary());
    return report;
}
```

34.7.2 The VerificationReport Model

```
@Data
@Builder
public class VerificationReport {
    private ValidationResult byteRangeResult;
    private ValidationResult hashResult;
    private ValidationResult cmsResult;
    private ValidationResult chainResult;
    private ValidationResult trustResult;
    private ValidationResult timestampResult;
    private Date timestampTime;
    private CertPath certPath;
    private String chainError;
    private Exception exception;

    public String getSummary() {
        if (exception != null) {
            return "ERROR: " + exception.getMessage();
        }
        boolean allPassed =
                byteRangeResult == ValidationResult.OK
                && hashResult == ValidationResult.OK
                && cmsResult == ValidationResult.OK
                && chainResult == ValidationResult.OK
                && trustResult == ValidationResult.OK;
        return allPassed ? "ALL STEPS PASSED" : "ONE OR MORE STEPS FAILED";
    }

    public boolean isValid() {
        return exception == null
                && byteRangeResult == ValidationResult.OK
                && hashResult == ValidationResult.OK
                && cmsResult == ValidationResult.OK;
```

```
    }

    public boolean isTrusted() {
        return isValid()
                && chainResult == ValidationResult.OK
                && trustResult == ValidationResult.OK;
    }
}
```

34.7.3 The ValidationResult Enum

```
public enum ValidationResult {
    OK("Passed"),
    BYTE_RANGE_MALFORMED("ByteRange array is malformed"),
    BYTE_RANGE_NOT_AT_START("ByteRange does not start at offset 0"),
    BYTE_RANGE_OVERLAP("ByteRange segments overlap"),
    BYTE_RANGE_TRAILING_BYTES("Unsigned trailing bytes detected"),
    HASH_MISMATCH("Computed hash does not match CMS messageDigest"),
    CMS_INVALID("CMS cryptographic verification failed"),
    CHAIN_INVALID("Certificate chain validation failed"),
    NOT_TRUSTED("Signer certificate does not chain to a trusted root"),
    TIMESTAMP_INVALID("Timestamp verification failed"),
    REVOKED("A certificate in the chain has been revoked");

    private final String description;

    ValidationResult(String description) {
        this.description = description;
    }

    public String getDescription() {
        return description;
    }
}
```

34.8 Modification Detection Between Revisions

34.8.1 Why Multi-Signature Documents Need Revision Comparison

When a PDF has multiple signatures, each signature covers a specific revision of the document — a snapshot of the file at the point when that signature was applied. Later signatures add incremental updates (Chapter 8) that extend the file. From the perspective of the first signature, the second signature's bytes are "trailing bytes" beyond its byte range.

The critical question is: **did the incremental update between revisions only add the new signature, or did it modify existing content?**

If someone added a new page, changed text, or altered form field values between signing events, the earlier signature's byte range still verifies (the signed bytes haven't changed), but the document's visible content has changed. This is the basis of the Shadow Attacks described in Chapter 24.

34.8.2 Extracting Individual Revisions

PDFBox allows you to extract the bytes of each revision by using the byte range information:

```
/**
 * Extract the signed revision for a specific signature.
 * Returns the PDF bytes up to the end of the signature's coverage.
 */
public byte[] extractSignedRevision(byte[] pdfBytes,
        PDSignature signature) {
    int[] byteRange = signature.getByteRange();
    int revisionEnd = byteRange[2] + byteRange[3];

    byte[] revision = new byte[revisionEnd];
    System.arraycopy(pdfBytes, 0, revision, 0, revisionEnd);

    log.info("Extracted revision: {} bytes (full file: {} bytes)",
            revisionEnd, pdfBytes.length);
    return revision;
}

/**
 * Compare two revisions to detect modifications beyond signature additions.
 * Returns a list of differences found.
 */
public List<RevisionDifference> compareRevisions(
        byte[] earlierRevision, byte[] laterRevision) throws IOException {

    List<RevisionDifference> differences = new ArrayList<>();

    try (PDDocument earlier = Loader.loadPDF(earlierRevision);
         PDDocument later = Loader.loadPDF(laterRevision)) {

        // Check page count changes
        int earlyPages = earlier.getNumberOfPages();
        int laterPages = later.getNumberOfPages();
        if (earlyPages != laterPages) {
            differences.add(new RevisionDifference(
                    DifferenceType.PAGE_COUNT_CHANGED,
                    "Page count changed from " + earlyPages +
                    " to " + laterPages));
        }

        // Check AcroForm field value changes
        PDAcroForm earlyForm =
                earlier.getDocumentCatalog().getAcroForm();
        PDAcroForm laterForm =
                later.getDocumentCatalog().getAcroForm();

        if (earlyForm != null && laterForm != null) {
            compareFormFields(earlyForm, laterForm, differences);
        }

        // Check for new annotations
        for (int i = 0;
```

```
                i < Math.min(earlyPages, laterPages); i++) {
            int earlyAnnots = earlier.getPage(i)
                    .getAnnotations().size();
            int laterAnnots = later.getPage(i)
                    .getAnnotations().size();
            if (earlyAnnots != laterAnnots) {
                differences.add(new RevisionDifference(
                        DifferenceType.ANNOTATIONS_CHANGED,
                        "Page " + (i + 1) + ": annotations changed " +
                        "from " + earlyAnnots + " to " + laterAnnots));
            }
        }
    }

    log.info("Revision comparison found {} difference(s)",
            differences.size());
    return differences;
}
```

Revision comparison is inherently incomplete. A sophisticated attacker can modify content streams in ways that are difficult to detect by comparing PDF objects. The comparison above catches obvious changes (page count, form fields, annotations), but it is not a security guarantee. For high-security applications, consider using the EU DSS framework, which implements the full ETSI modification detection specification.

34.9 Edge Cases: Self-Signed, Expired, and Missing Chain

34.9.1 Self-Signed Certificates

A self-signed certificate is one where the issuer and subject are the same entity — the certificate signs itself. Self-signed signatures are cryptographically valid (the CMS verifies), but the certificate chain is trivially short: one certificate, no intermediates, no root CA.

```
/**
 * Check if a certificate is self-signed.
 */
public boolean isSelfSigned(X509CertificateHolder cert) {
    boolean selfSigned = cert.getSubject().equals(cert.getIssuer());
    if (selfSigned) {
        log.info("Certificate is self-signed: subject and issuer " +
                "are both {}", cert.getSubject());
    }
    return selfSigned;
}
```

For self-signed certificates, the chain validation in Step 4 will fail unless you explicitly add the self-signed certificate as a trust anchor. Most applications handle self-signed signatures by reporting: ”The signature is cryptographically valid, but the identity of the signer could not be verified because the certificate is self-signed.”

This is the correct behavior. A self-signed signature proves that someone *possessing that private key* signed the document. It does not prove *who* possesses that key, because no trusted third party (CA) has vouched for the identity.

34.9.2 Expired Certificates

Certificate expiration is time-dependent — the same signature can be ”valid” today and ”expired” tomorrow. The handling depends on whether a timestamp is present:

```
/**
 * Determine if a signature's certificate is expired,
 * accounting for timestamps.
 */
public CertExpirationStatus checkCertExpiration(
        X509CertificateHolder signerCert,
        Date timestampTime) {

    Date now = new Date();
    Date notBefore = signerCert.getNotBefore();
    Date notAfter = signerCert.getNotAfter();

    if (now.before(notAfter)) {
        return CertExpirationStatus.VALID;
    }

    // Certificate has expired — check if we have a timestamp
    if (timestampTime != null) {
        if (timestampTime.before(notAfter)
                && timestampTime.after(notBefore)) {
            log.info("Certificate expired on {}, but timestamp {} " +
                    "proves signature was created during validity period",
                    notAfter, timestampTime);
            return CertExpirationStatus.EXPIRED_BUT_TIMESTAMPED;
        } else {
            log.error("Certificate expired on {}, and timestamp {} " +
                    "is outside the validity period [{} to {}]",
                    notAfter, timestampTime, notBefore, notAfter);
            return CertExpirationStatus.EXPIRED_AND_TIMESTAMP_INVALID;
        }
    }

    log.warn("Certificate expired on {} and no timestamp is present",
            notAfter);
    return CertExpirationStatus.EXPIRED_NO_TIMESTAMP;
}
```

34.9.3 Missing Intermediate Certificates

The most common real-world chain validation failure is a missing intermediate certificate. The CMS `SignedData` should include all intermediate certificates between the signer’s certificate and the root, but some signing implementations omit them.

When intermediates are missing, you have two options:

1. **AIA chasing**: Follow the Authority Information Access (AIA) extension in the signer’s certificate to download the missing intermediate from the CA’s server
2. **Local certificate cache**: Maintain a local cache of known intermediate certificates

```
/**
```

```
 * Attempt to fetch missing intermediate certificates via AIA.
 *
 * @param cert  the certificate whose issuer we need
 * @return the issuer's certificate, or null if not fetchable
 */
public X509Certificate fetchIssuerViaAia(X509Certificate cert)
        throws Exception {

    byte[] aiaExtBytes = cert.getExtensionValue(
            Extension.authorityInfoAccess.getId());
    if (aiaExtBytes == null) {
        log.info("No AIA extension — cannot fetch issuer");
        return null;
    }

    ASN1OctetString aiaOctet = ASN1OctetString.getInstance(aiaExtBytes);
    AuthorityInformationAccess aia =
            AuthorityInformationAccess.getInstance(
                    ASN1Sequence.getInstance(aiaOctet.getOctets()));

    for (AccessDescription desc : aia.getAccessDescriptions()) {
        if (desc.getAccessMethod().equals(
                AccessDescription.id_ad_caIssuers)) {
            GeneralName gn = desc.getAccessLocation();
            if (gn.getTagNo() == GeneralName.uniformResourceIdentifier) {
                String url = gn.getName().toString();
                log.info("Fetching issuer certificate from AIA: {}", url);

                try (InputStream is = new URL(url).openStream()) {
                    CertificateFactory cf =
                            CertificateFactory.getInstance("X.509");
                    return (X509Certificate) cf.generateCertificate(is);
                }
            }
        }
    }

    log.warn("AIA extension present but no CA Issuers URL found");
    return null;
}
```

In production, always cache downloaded certificates. AIA chasing adds network latency and introduces a dependency on external servers — exactly the kind of dependency that LTV (Chapter 18) is designed to eliminate. If you are building a signing service, embed all intermediate certificates in the CMS at signing time, so validators never need to chase AIA.

34.10 Algorithm OID Mapping Utility

Throughout this chapter, we reference a `mapOidToAlgorithm()` helper. Here is a complete implementation that covers the algorithm OIDs you will encounter in real PDF signatures:

```
/**
 * Map a digest or signature algorithm OID to a Java algorithm name.
 * Covers the algorithms used in real-world PDF signatures.
```

```
 */
public static String mapOidToAlgorithm(String oid) {
    return switch (oid) {
        // Digest algorithms
        case "1.3.14.3.2.26"         -> "SHA-1";
        case "2.16.840.1.101.3.4.2.1" -> "SHA-256";
        case "2.16.840.1.101.3.4.2.2" -> "SHA-384";
        case "2.16.840.1.101.3.4.2.3" -> "SHA-512";
        // RSA signature algorithms
        case "1.2.840.113549.1.1.5"  -> "SHA1withRSA";
        case "1.2.840.113549.1.1.11" -> "SHA256withRSA";
        case "1.2.840.113549.1.1.12" -> "SHA384withRSA";
        case "1.2.840.113549.1.1.13" -> "SHA512withRSA";
        case "1.2.840.113549.1.1.10" -> "RSASSA-PSS";
        // ECDSA signature algorithms
        case "1.2.840.10045.4.3.2"   -> "SHA256withECDSA";
        case "1.2.840.10045.4.3.3"   -> "SHA384withECDSA";
        case "1.2.840.10045.4.3.4"   -> "SHA512withECDSA";
        default -> throw new IllegalArgumentException(
                "Unknown algorithm OID: " + oid);
    };
}
```

34.11 Key Takeaways

- PDFBox provides signature extraction via two paths: the AcroForm field tree (full metadata) or `getSignatureDictionaries()` (fast, crypto-only)
- CMS parsing requires both the `/Contents` bytes *and* the signed content (byte range data) — always pass both to `CMSSignedData`
- The five-step pipeline is: byte range check, hash verification, CMS cryptographic verification, certificate chain validation, trust evaluation
- `SignerInformation.verify()` performs Steps 2 and 3 combined — but not chain validation or trust evaluation
- Java's PKIX `CertPathBuilder` handles chain construction, validity checking, and basic constraints automatically
- Timestamps shift the validation time from "now" to "when the signature was created" — critical for expired certificates
- Revision comparison detects modifications between signatures but is not a complete security guarantee
- Self-signed signatures are cryptographically valid but untrusted — always report the distinction clearly
- Missing intermediates are the most common chain failure — implement AIA chasing or embed all intermediates at signing time
- The `VerificationReport` model gives callers granular per-step results rather than a binary pass/fail

Chapter 35

Test Cases & Reference PDFs

Testing PDF digital signatures is one of the hardest testing problems in software engineering. A signature that is valid today may be invalid tomorrow because a certificate expired. A test that passes on your machine may fail on CI because the TSA server is unreachable. A verification test that ran perfectly for three years suddenly breaks because a CRL distribution point returns HTTP 404. Time, network, and third-party infrastructure are all embedded in your test surface — and they are all moving targets.

This chapter covers how to build a test infrastructure that is deterministic, fast, and comprehensive. We generate our own keystores, mock external services, freeze time, and construct a test matrix that exercises every meaningful signature scenario. By the end, you will have a test suite that runs in seconds, needs no network access, and covers the full signing-to-verification round trip including PAdES compliance.

35.1 Why Testing PDF Signatures Is Hard

35.1.1 The Three Enemies of Deterministic Signature Tests

Enemy 1: Time. Certificate validation is time-dependent. A certificate that is valid right now has a `notBefore` and `notAfter` date range. If your test generates a certificate valid for one year and your test suite runs 13 months later, every test that validates that certificate will fail. Timestamps make this worse — a timestamp proves the signature was created at a specific time, and the TSA certificate has its own validity period.

Enemy 2: Network. Real signing workflows depend on external services: TSA servers for timestamps, OCSP responders for revocation checks, CRL distribution points for certificate revocation lists. Any of these can be slow, unreachable, or returning unexpected responses. A test that calls a real TSA is a test that depends on the internet — and the internet is not deterministic.

Enemy 3: State. PDF files are stateful. A signed PDF is a specific sequence of bytes, and any change — even adding a single trailing newline — invalidates the signature. Tests that generate PDFs must produce identical byte sequences across runs, or the test assertions become meaningless. Different versions of PDFBox may serialize objects in different orders, produce different padding sizes, or use different default values — any of which changes the output bytes.

35.1.2 The Testing Strategy

Our approach addresses all three enemies:

1. **Generate test keystores programmatically** with explicit, long-lived validity periods (100 years — we are not testing certificate expiration unless we want to)
2. **Mock all external services** (TSA, OCSP) using WireMock or BouncyCastle's in-process TSA
3. **Freeze time** using a `Clock` abstraction so certificate and timestamp validation happens at a controlled instant
4. **Generate reference PDFs as part of the test setup** rather than shipping binary PDF files in the repository

35.2 Test Infrastructure: JUnit 5 and Test Keystore Generation

35.2.1 The Test Keystore Generator

Every test needs cryptographic key material: a signing certificate, an intermediate CA, a root CA, and a TSA certificate. We generate all of these programmatically using BouncyCastle:

```
/**
 * Generates a complete PKI hierarchy for testing:
 * Root CA -> Intermediate CA -> Signer certificate
 * Plus a separate TSA certificate signed by the Root CA.
 */
@Slf4j
public class TestKeystoreGenerator {

    private static final int KEY_SIZE = 2048;
    private static final String SIGNATURE_ALGORITHM = "SHA256withRSA";

    /**
     * Generate a complete test PKI and return it as an in-memory KeyStore.
     */
    public static KeyStore generateTestKeystore(char[] password)
            throws Exception {
        // Root CA — self-signed, 100-year validity
        KeyPair rootKeyPair = generateKeyPair();
        X509Certificate rootCert = generateRootCaCertificate(
                rootKeyPair,
                "CN=Test Root CA, O=PDFalyzer Test, C=DE",
                100);

        // Intermediate CA — signed by Root, 50-year validity
        KeyPair intermediateKeyPair = generateKeyPair();
        X509Certificate intermediateCert =
                generateIntermediateCaCertificate(
                        intermediateKeyPair,
                        "CN=Test Intermediate CA, O=PDFalyzer Test, C=DE",
                        rootKeyPair.getPrivate(),
                        rootCert,
                        50);

        // Signer — signed by Intermediate, 10-year validity
        KeyPair signerKeyPair = generateKeyPair();
        X509Certificate signerCert = generateSignerCertificate(
                signerKeyPair,
```

```
            "CN=Test Signer, O=PDFalyzer Test, C=DE",
            intermediateKeyPair.getPrivate(),
            intermediateCert,
            10);

    // TSA — signed by Root, 50-year validity, with timeStamping EKU
    KeyPair tsaKeyPair = generateKeyPair();
    X509Certificate tsaCert = generateTsaCertificate(
            tsaKeyPair,
            "CN=Test TSA, O=PDFalyzer Test, C=DE",
            rootKeyPair.getPrivate(),
            rootCert,
            50);

    // Assemble into a PKCS12 KeyStore
    KeyStore ks = KeyStore.getInstance("PKCS12");
    ks.load(null, password);

    // Store the signer's key + full chain
    X509Certificate[] signerChain = {
            signerCert, intermediateCert, rootCert
    };
    ks.setKeyEntry("signer", signerKeyPair.getPrivate(),
            password, signerChain);

    // Store the TSA's key + chain
    X509Certificate[] tsaChain = { tsaCert, rootCert };
    ks.setKeyEntry("tsa", tsaKeyPair.getPrivate(),
            password, tsaChain);

    // Store the root as a trusted cert entry
    ks.setCertificateEntry("root", rootCert);

    log.info("Generated test keystore with signer, TSA, and root");
    return ks;
}

private static KeyPair generateKeyPair() throws Exception {
    KeyPairGenerator gen = KeyPairGenerator.getInstance("RSA");
    gen.initialize(KEY_SIZE);
    return gen.generateKeyPair();
}

private static X509Certificate generateRootCaCertificate(
        KeyPair keyPair, String dn, int validityYears)
        throws Exception {

    X500Name issuer = new X500Name(dn);
    BigInteger serial = BigInteger.valueOf(
            System.currentTimeMillis());
    Date notBefore = new Date();
    Date notAfter = Date.from(Instant.now()
            .plus(Duration.ofDays(365L * validityYears)));

    JcaX509v3CertificateBuilder builder =
```

```
                new JcaX509v3CertificateBuilder(
                        issuer, serial, notBefore, notAfter,
                        issuer, keyPair.getPublic());

        // CA extensions
        builder.addExtension(Extension.basicConstraints, true,
                new BasicConstraints(true));
        builder.addExtension(Extension.keyUsage, true,
                new KeyUsage(KeyUsage.keyCertSign | KeyUsage.cRLSign));

        ContentSigner signer = new JcaContentSignerBuilder(
                SIGNATURE_ALGORITHM).setProvider("BC")
                .build(keyPair.getPrivate());

        return new JcaX509CertificateConverter().setProvider("BC")
                .getCertificate(builder.build(signer));
    }

    // Similar methods for intermediate, signer, and TSA certificates
    // omitted for brevity — full listings in Appendix A
}
```

35.2.2 JUnit 5 Test Base Class

We use a shared base class that sets up the test PKI and provides utility methods:

```
@Slf4j
abstract class SignatureTestBase {

    protected static KeyStore testKeystore;
    protected static PrivateKey signerKey;
    protected static X509Certificate[] signerChain;
    protected static PrivateKey tsaKey;
    protected static X509Certificate[] tsaChain;
    protected static Set<TrustAnchor> trustAnchors;
    protected static final char[] PASSWORD = "test".toCharArray();

    @BeforeAll
    static void setupPki() throws Exception {
        Security.addProvider(new BouncyCastleProvider());

        testKeystore = TestKeystoreGenerator
                .generateTestKeystore(PASSWORD);

        signerKey = (PrivateKey) testKeystore.getKey("signer", PASSWORD);
        signerChain = Arrays.stream(
                testKeystore.getCertificateChain("signer"))
                .map(c -> (X509Certificate) c)
                .toArray(X509Certificate[]::new);

        tsaKey = (PrivateKey) testKeystore.getKey("tsa", PASSWORD);
        tsaChain = Arrays.stream(
                testKeystore.getCertificateChain("tsa"))
                .map(c -> (X509Certificate) c)
                .toArray(X509Certificate[]::new);
```

```
        X509Certificate rootCert = (X509Certificate)
                testKeystore.getCertificate("root");
        trustAnchors = Set.of(new TrustAnchor(rootCert, null));

        log.info("Test PKI initialized — signer: {}, root: {}",
                signerChain[0].getSubjectX500Principal(),
                rootCert.getSubjectX500Principal());
    }

    /**
     * Generate a minimal unsigned PDF for testing.
     */
    protected byte[] generateTestPdf(String content) throws IOException {
        try (PDDocument doc = new PDDocument()) {
            PDPage page = new PDPage();
            doc.addPage(page);
            try (PDPageContentStream cs =
                    new PDPageContentStream(doc, page)) {
                cs.beginText();
                cs.setFont(new PDType1Font(
                        Standard14Fonts.FontName.HELVETICA), 12);
                cs.newLineAtOffset(72, 700);
                cs.showText(content);
                cs.endText();
            }
            ByteArrayOutputStream baos = new ByteArrayOutputStream();
            doc.save(baos);
            return baos.toByteArray();
        }
    }
}
```

35.3 Test PDF Generation Scenarios

35.3.1 Generating Signed PDFs Programmatically

Rather than shipping pre-built signed PDFs (which become stale and opaque), we generate them as part of each test. This ensures every test controls its own input and can vary specific parameters:

```
/**
 * Sign a PDF with the test keystore and return the signed bytes.
 */
protected byte[] signPdf(byte[] pdfBytes, boolean addTimestamp,
        String subFilter) throws Exception {

    try (PDDocument doc = Loader.loadPDF(pdfBytes)) {
        PDSignature pdSig = new PDSignature();
        pdSig.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
        pdSig.setSubFilter(COSName.getPDFName(subFilter));
        pdSig.setName("Test Signer");
        pdSig.setReason("Integration test");
        pdSig.setSignDate(Calendar.getInstance());
```

```
        SignatureInterface sigInterface =
                new TestSignatureInterface(
                        signerKey, signerChain,
                        addTimestamp ? tsaKey : null,
                        addTimestamp ? tsaChain : null);

        doc.addSignature(pdSig, sigInterface);

        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        doc.saveIncremental(baos);
        return baos.toByteArray();
    }
}
```

35.3.2 Scenario Generators

Each test scenario generates a specific type of signed PDF:

```
/**
 * Generate a PDF signed with different configurations.
 */
protected byte[] generateScenario(TestScenario scenario)
        throws Exception {

    byte[] pdf = generateTestPdf("Scenario: " + scenario.name());

    return switch (scenario) {
        case BASIC_PKCS7 ->
            signPdf(pdf, false, "adbe.pkcs7.detached");
        case BASIC_PADES ->
            signPdf(pdf, false, "ETSI.CAdES.detached");
        case PADES_WITH_TIMESTAMP ->
            signPdf(pdf, true, "ETSI.CAdES.detached");
        case SELF_SIGNED ->
            signPdfWithSelfSigned(pdf);
        case EXPIRED_CERT ->
            signPdfWithExpiredCert(pdf);
        case MULTI_SIGNATURE ->
            signPdfTwice(pdf);
        case TAMPERED_AFTER_SIGNING ->
            tamperSignedPdf(signPdf(pdf, false,
                    "adbe.pkcs7.detached"));
        case MISSING_INTERMEDIATE ->
            signPdfWithMissingIntermediate(pdf);
    };
}
```

35.4 Test Matrix: Scenario to Expected Result

The following table defines every test scenario, what it tests, and the expected outcome at each verification step. This is the specification for your test suite — every row should be a test method:

Table 35.1: Signature test matrix

Scenario	ByteRange	Hash	CMS	Chain	Trust	Notes
Basic PKCS#7 (adbe.pkcs7.detached)	OK	OK	OK	OK	OK	Baseline — everything should pass
Basic PAdES (ETSI.CAdES.detached)	OK	OK	OK	OK	OK	Same as PKCS#7 but with PAdES SubFilter
PAdES with timestamp	OK	OK	OK	OK	OK	Timestamp attribute present and valid
Self-signed certificate	OK	OK	OK	FAIL	FAIL	CMS is valid but chain has no trusted root
Expired certificate (no timestamp)	OK	OK	OK	FAIL	FAIL	CertPathValidator rejects expired cert
Expired certificate (with timestamp)	OK	OK	OK	OK	OK	Timestamp proves signing was within validity
Multi-signature document	WARN	OK	OK	OK	OK	First sig has trailing bytes (second revision)
Tampered after signing	OK	FAIL	—	—	—	Hash mismatch — document was modified
Missing intermediate certificate	OK	OK	OK	FAIL	FAIL	Chain cannot be built without intermediate
Revoked certificate	OK	OK	OK	FAIL	FAIL	Requires OCSP/CRL mock returning ”revoked”
Wrong SubFilter for content	OK	OK	OK	OK	OK	SubFilter says PAdES but ESS attr is missing
Empty /Contents (zero-length CMS)	OK	FAIL	FAIL	—	—	CMSException during parsing
Invisible signature (zero-size widget)	OK	OK	OK	OK	OK	Cryptographically identical to visible sig

Each row in this matrix translates directly to a test method. The ”--” entries mean that step is not reached because an earlier step failed fatally.

35.5 Mocking TSA and OCSP

35.5.1 In-Process TSA with BouncyCastle

For timestamp tests, we need a TSA that returns deterministic timestamp tokens without hitting the network. BouncyCastle provides everything you need to build one in-process:

```
/**
 * In-process TSA for testing.
 * Produces valid RFC 3161 timestamp tokens using the test TSA key.
 */
@Slf4j
public class InProcessTsa {

    private final PrivateKey tsaKey;
    private final X509Certificate tsaCert;
    private final AtomicLong serialNumber = new AtomicLong(1);

    public InProcessTsa(PrivateKey tsaKey, X509Certificate tsaCert) {
        this.tsaKey = tsaKey;
        this.tsaCert = tsaCert;
    }

    /**
     * Generate a timestamp token for the given data hash.
     */
    public TimeStampToken timestamp(byte[] dataToTimestamp)
            throws Exception {

        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(dataToTimestamp);

        TimeStampRequestGenerator reqGen =
                new TimeStampRequestGenerator();
        reqGen.setCertReq(true);
        TimeStampRequest request =
                reqGen.generate(TSPAlgorithms.SHA256, hash);

        // Build the timestamp token
        JcaSignerInfoGeneratorBuilder sigGenBuilder =
                new JcaSignerInfoGeneratorBuilder(
                        new JcaDigestCalculatorProviderBuilder()
                                .setProvider("BC").build());

        ContentSigner contentSigner = new JcaContentSignerBuilder(
                "SHA256withRSA").setProvider("BC").build(tsaKey);
        SignerInfoGenerator sigGen = sigGenBuilder.build(
                contentSigner,
                new JcaX509CertificateHolder(tsaCert));

        DigestCalculatorProvider dcProv =
                new JcaDigestCalculatorProviderBuilder()
                        .setProvider("BC").build();
        DigestCalculator dgCalc =
```

```
                dcProv.get(new AlgorithmIdentifier(NISTObjectIdentifiers.id_sha256));

        TimeStampTokenGenerator tsTokenGen =
                new TimeStampTokenGenerator(sigGen, dgCalc,
                        new ASN1ObjectIdentifier("1.2.3.4.5")); // Test policy
        tsTokenGen.addCertificates(
                new JcaCertStore(List.of(tsaCert)));

        TimeStampResponseGenerator tsRespGen =
                new TimeStampResponseGenerator(tsTokenGen,
                        TSPAlgorithms.ALLOWED);

        TimeStampResponse response = tsRespGen.generate(
                request, serialNumber.getAndIncrement(), new Date());

        log.info("Generated test timestamp — serial: {}, time: {}",
                serialNumber.get() - 1, new Date());
        return response.getTimeStampToken();
    }
}
```

35.5.2 Mocking OCSP with WireMock

For revocation checking tests, we use WireMock to simulate an OCSP responder. This lets us return ”good,” ”revoked,” or ”unknown” responses without any real PKI:

```
/**
 * Configure a WireMock OCSP responder stub.
 * Returns a pre-built OCSP response for any request.
 */
public static void stubOcspResponder(WireMockServer wireMock,
        OCSPResp response) throws Exception {

    wireMock.stubFor(post(urlEqualTo("/ocsp"))
            .withHeader("Content-Type",
                    equalTo("application/ocsp-request"))
            .willReturn(aResponse()
                    .withStatus(200)
                    .withHeader("Content-Type",
                            "application/ocsp-response")
                    .withBody(response.getEncoded())));
}

/**
 * Build a "good" OCSP response for testing.
 */
public static OCSPResp buildGoodOcspResponse(
        X509Certificate targetCert,
        X509Certificate issuerCert,
        PrivateKey responderKey,
        X509Certificate responderCert) throws Exception {

    BasicOCSPRespBuilder respBuilder =
            new JcaBasicOCSPRespBuilder(
                    responderCert.getPublicKey(),
```

```
                    new JcaDigestCalculatorProviderBuilder()
                            .setProvider("BC").build()
                            .get(CertificateID.HASH_SHA1));

    CertificateID certId = new JcaCertificateID(
            new JcaDigestCalculatorProviderBuilder()
                    .setProvider("BC").build()
                    .get(CertificateID.HASH_SHA1),
            new JcaX509CertificateHolder(issuerCert),
            targetCert.getSerialNumber());

    respBuilder.addResponse(certId, CertificateStatus.GOOD);

    ContentSigner signer = new JcaContentSignerBuilder("SHA256withRSA")
            .setProvider("BC").build(responderKey);
    BasicOCSPResp basicResp = respBuilder.build(signer,
            new X509CertificateHolder[]{ new JcaX509CertificateHolder(
                    responderCert) },
            new Date());

    return new OCSPRespBuilder().build(
            OCSPRespBuilder.SUCCESSFUL, basicResp);
}
```

35.6 Time Travel in Tests

35.6.1 The Clock Abstraction

The most reliable way to handle time-dependent tests is to inject a `Clock` into any component that checks certificate validity or timestamp times:

```
/**
 * Verification service that accepts an injectable Clock
 * for deterministic testing.
 */
@Slf4j
public class VerificationService {

    private final Clock clock;

    /**
     * Production constructor — uses system clock.
     */
    public VerificationService() {
        this.clock = Clock.systemUTC();
    }

    /**
     * Test constructor — uses injected clock.
     */
    public VerificationService(Clock clock) {
        this.clock = clock;
    }
```

```
    public VerificationReport verify(byte[] pdfBytes,
            PDSignature signature,
            Set<TrustAnchor> trustAnchors) {
        // ... all validation uses this.clock instead of new Date()
        Date validationTime = Date.from(clock.instant());
        // ...
    }
}
```

35.6.2 Testing Expired Certificates

With the `Clock` abstraction, testing expired certificate scenarios becomes straightforward:

```
@Test
void expiredCertificateWithoutTimestampShouldFail() throws Exception {
    // Generate a certificate valid for only 1 second
    KeyPair keyPair = generateKeyPair();
    X509Certificate shortLivedCert =
            generateCertWithValidity(keyPair, Duration.ofSeconds(1));

    // Sign the PDF
    byte[] pdf = generateTestPdf("Short-lived cert test");
    byte[] signed = signPdfWith(pdf, keyPair.getPrivate(),
            new X509Certificate[]{ shortLivedCert });

    // Travel forward in time — certificate is now expired
    Clock futureClock = Clock.fixed(
            Instant.now().plus(Duration.ofDays(1)),
            ZoneOffset.UTC);
    VerificationService verifier = new VerificationService(futureClock);

    VerificationReport report = verifier.verify(
            signed, extractSignature(signed), trustAnchors);

    assertEquals(ValidationResult.OK, report.getHashResult());
    assertEquals(ValidationResult.OK, report.getCmsResult());
    assertEquals(ValidationResult.CHAIN_INVALID,
            report.getChainResult());
}

@Test
void expiredCertificateWithTimestampShouldPass() throws Exception {
    // Same setup, but this time we add a timestamp
    byte[] signed = signPdfWith(pdf, keyPair.getPrivate(),
            new X509Certificate[]{ shortLivedCert },
            true /* addTimestamp */);

    // The timestamp proves the signature was created
    // before the cert expired
    Clock futureClock = Clock.fixed(
            Instant.now().plus(Duration.ofDays(1)),
            ZoneOffset.UTC);
    VerificationService verifier = new VerificationService(futureClock);

    VerificationReport report = verifier.verify(
```

```
            signed, extractSignature(signed), trustAnchors);

    assertEquals(ValidationResult.OK, report.getChainResult());
    assertNotNull(report.getTimestampTime());
}
```

35.7 Complete Signing-to-Verification Round-Trip Test

This is the most important test in the suite — it exercises the entire pipeline end-to-end:

```
@Test
void completeRoundTripSignAndVerify() throws Exception {
    // 1. Generate a test PDF
    byte[] unsignedPdf = generateTestPdf(
            "Round-trip test — " + Instant.now());
    log.info("Generated unsigned PDF: {} bytes", unsignedPdf.length);

    // 2. Sign with the test keystore (PAdES + timestamp)
    byte[] signedPdf = signPdf(unsignedPdf, true,
            "ETSI.CAdES.detached");
    log.info("Generated signed PDF: {} bytes", signedPdf.length);

    // 3. Extract the signature
    try (PDDocument doc = Loader.loadPDF(signedPdf)) {
        List<PDSignature> sigs = doc.getSignatureDictionaries();
        assertEquals(1, sigs.size(), "Expected exactly one signature");

        PDSignature sig = sigs.get(0);
        assertEquals("ETSI.CAdES.detached",
                sig.getSubFilter());

        // 4. Run the complete verification pipeline
        VerificationService verifier = new VerificationService();
        VerificationReport report =
                verifier.verify(signedPdf, sig, trustAnchors);

        // 5. Assert every step passed
        assertEquals(ValidationResult.OK,
                report.getByteRangeResult(),
                "ByteRange check should pass");
        assertEquals(ValidationResult.OK,
                report.getHashResult(),
                "Hash verification should pass");
        assertEquals(ValidationResult.OK,
                report.getCmsResult(),
                "CMS verification should pass");
        assertEquals(ValidationResult.OK,
                report.getChainResult(),
                "Certificate chain should validate");
        assertEquals(ValidationResult.OK,
                report.getTrustResult(),
                "Trust evaluation should pass");

        // 6. Verify timestamp is present and valid
```

```
        assertNotNull(report.getTimestampTime(),
                "Timestamp should be present");
        assertEquals(ValidationResult.OK,
                report.getTimestampResult(),
                "Timestamp should be valid");

        assertTrue(report.isValid(), "Signature should be valid");
        assertTrue(report.isTrusted(), "Signature should be trusted");

        log.info("Round-trip test PASSED — all steps verified");
    }
}
```

35.8 PAdES Compliance Test

A PAdES-compliant signature must include specific signed attributes beyond what plain CMS requires. We test for these explicitly:

```
@Test
void padesSignatureShouldHaveRequiredAttributes() throws Exception {
    byte[] pdf = generateTestPdf("PAdES compliance test");
    byte[] signed = signPdf(pdf, true, "ETSI.CAdES.detached");

    try (PDDocument doc = Loader.loadPDF(signed)) {
        PDSignature sig = doc.getSignatureDictionaries().get(0);
        CMSSignedData cmsData = parseCmsSignedData(signed, sig);
        SignerInformation signerInfo = cmsData.getSignerInfos()
                .getSigners().iterator().next();

        AttributeTable signedAttrs =
                signerInfo.getSignedAttributes();

        // PAdES-B-B: Required signed attributes
        assertNotNull(
                signedAttrs.get(CMSAttributes.contentType),
                "contentType attribute is required");
        assertNotNull(
                signedAttrs.get(CMSAttributes.messageDigest),
                "messageDigest attribute is required");
        assertNotNull(
                signedAttrs.get(
                    PKCSObjectIdentifiers.id_aa_signingCertificateV2),
                "ESS-signing-certificate-v2 is required for PAdES");

        // PAdES-B-B: signingTime SHOULD NOT be present
        // (PAdES relies on timestamps, not claimed signing time)
        Attribute signingTime =
                signedAttrs.get(CMSAttributes.signingTime);
        if (signingTime != null) {
            log.warn("signingTime attribute is present — " +
                    "PAdES recommends omitting it in favor " +
                    "of a timestamp");
        }
```

```
        // PAdES-B-T: Timestamp unsigned attribute
        AttributeTable unsignedAttrs =
                signerInfo.getUnsignedAttributes();
        assertNotNull(unsignedAttrs,
                "PAdES-B-T requires unsigned attributes");
        assertNotNull(
                unsignedAttrs.get(
                    PKCSObjectIdentifiers
                        .id_aa_signatureTimeStampToken),
                "PAdES-B-T requires a signature timestamp");

        // Verify the digest algorithm is SHA-256 or stronger
        String digestAlg = signerInfo.getDigestAlgOID();
        assertNotEquals("1.3.14.3.2.26", digestAlg,
                "SHA-1 is not allowed for PAdES — " +
                "must use SHA-256 or stronger");

        log.info("PAdES compliance test PASSED");
    }
}
```

35.9 Reference PDF Corpus: Naming Convention

For reference PDFs that you ship with your test suite (for regression testing or interoperability testing), use a consistent naming convention that encodes the signature properties in the filename:

```
Naming pattern:
  <scenario>_<subfilter>_<timestamp>_<cert-type>_<extras>.pdf

Examples:
  basic_pkcs7-detached_no-ts_ca-signed_v1.pdf
  basic_cades-detached_rfc3161_ca-signed_v1.pdf
  pades-b-t_cades-detached_rfc3161_ca-signed_v1.pdf
  pades-b-lt_cades-detached_rfc3161_ca-signed_dss_v1.pdf
  selfsigned_pkcs7-detached_no-ts_self-signed_v1.pdf
  multi-sig_pkcs7-detached_no-ts_ca-signed_2sigs_v1.pdf
  tampered_pkcs7-detached_no-ts_ca-signed_modified_v1.pdf
  invisible_cades-detached_rfc3161_ca-signed_zero-widget_v1.pdf
```

The `_v1` suffix is a version number. When you regenerate reference PDFs (because you changed the test keystore or signing code), increment the version number. This ensures old and new reference files can coexist temporarily during migration.

Store reference PDFs in `src/test/resources/reference-pdfs/` and regenerate them with a dedicated test utility class. Never edit reference PDFs by hand — they should always be produced by code. Commit the generated PDFs to your repository so tests can run without a generation step, but include the generator so anyone can rebuild them.

35.10 Key Takeaways

- PDF signature testing is hard because of three enemies: time, network, and state — address all three explicitly
- Generate test keystores programmatically with long validity periods (100 years) to avoid time-rot
- Use an in-process TSA built from BouncyCastle instead of hitting real TSA servers
- Mock OCSP responders with WireMock for deterministic revocation testing
- Inject a `Clock` abstraction into verification services so you can freeze or advance time in tests
- Build a test matrix that maps every scenario to expected per-step results
- The round-trip test (generate PDF, sign, extract, verify) is the single most important test in the suite
- Test PAdES compliance explicitly: check for `ESS-signing-certificate-v2`, proper SubFilter, and SHA-256+
- Use a consistent naming convention for reference PDFs that encodes all signature properties in the filename
- Generate reference PDFs from code, not by hand — commit both the generator and the generated files

35.11 Complete Code Listings

This appendix collects the complete, compilable source code for the major classes developed throughout Parts 5 and 6. Each listing is self-contained — you can copy it into your project, adjust the package name, and compile. All code targets Java 21, Apache PDFBox 3.0.0, and BouncyCastle 1.80.

35.12 SigningService: Complete PDF Signing Implementation

This is the full signing service from Chapters 29 and 30, consolidated into a single class. It handles both `adbe.pkcs7.detached` and `ETSI.CAdES.detached` signatures, with optional RFC 3161 timestamps.

```
package io.pdfalyzer.signing;

import java.io.ByteArrayOutputStream;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.security.MessageDigest;
import java.security.PrivateKey;
import java.security.cert.Certificate;
import java.security.cert.X509Certificate;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.List;

import lombok.RequiredArgsConstructor;
```

```
import lombok.extern.slf4j.Slf4j;
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.cos.COSName;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.SignatureInterface;
import org.bouncycastle.asn1.ASN1ObjectIdentifier;
import org.bouncycastle.asn1.DERSet;
import org.bouncycastle.asn1.cms.Attribute;
import org.bouncycastle.asn1.cms.AttributeTable;
import org.bouncycastle.asn1.cms.CMSAttributes;
import org.bouncycastle.asn1.ess.ESSCertIDv2;
import org.bouncycastle.asn1.ess.SigningCertificateV2;
import org.bouncycastle.asn1.nist.NISTObjectIdentifiers;
import org.bouncycastle.asn1.pkcs.PKCSObjectIdentifiers;
import org.bouncycastle.asn1.x509.AlgorithmIdentifier;
import org.bouncycastle.cert.jcajce.JcaCertStore;
import org.bouncycastle.cert.jcajce.JcaX509CertificateHolder;
import org.bouncycastle.cms.CMSProcessableByteArray;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.CMSSignedDataGenerator;
import org.bouncycastle.cms.DefaultSignedAttributeTableGenerator;
import org.bouncycastle.cms.SignerInfoGenerator;
import org.bouncycastle.cms.SignerInfoGeneratorBuilder;
import org.bouncycastle.cms.jcajce.JcaSignerInfoGeneratorBuilder;
import org.bouncycastle.operator.ContentSigner;
import org.bouncycastle.operator.DigestCalculatorProvider;
import org.bouncycastle.operator.jcajce.JcaContentSignerBuilder;
import org.bouncycastle.operator.jcajce.JcaDigestCalculatorProviderBuilder;
import org.bouncycastle.tsp.TSPAlgorithms;
import org.bouncycastle.tsp.TimeStampRequest;
import org.bouncycastle.tsp.TimeStampRequestGenerator;
import org.bouncycastle.tsp.TimeStampResponse;
import org.bouncycastle.tsp.TimeStampToken;

/**
 * Complete PDF signing service.
 * Supports PKCS#7 detached and PAdES CAdES-detached signatures
 * with optional RFC 3161 timestamps.
 */
@Slf4j
@RequiredArgsConstructor
public class SigningService {

    private final PrivateKey privateKey;
    private final X509Certificate[] certificateChain;
    private final String tsaUrl; // null if no timestamp desired

    /**
     * Sign a PDF document and return the signed bytes.
     *
     * @param pdfBytes    the unsigned PDF
     * @param subFilter   "adbe.pkcs7.detached" or "ETSI.CAdES.detached"
     * @param name        signer name (optional)
     * @param reason      signing reason (optional)
```

```
 * @param location    signing location (optional)
 * @return the signed PDF bytes
 */
public byte[] sign(byte[] pdfBytes, String subFilter,
        String name, String reason, String location)
        throws Exception {

    try (PDDocument doc = Loader.loadPDF(pdfBytes)) {
        PDSignature pdSig = new PDSignature();
        pdSig.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
        pdSig.setSubFilter(COSName.getPDFName(subFilter));
        if (name != null) pdSig.setName(name);
        if (reason != null) pdSig.setReason(reason);
        if (location != null) pdSig.setLocation(location);
        pdSig.setSignDate(Calendar.getInstance());

        SignatureInterface sigInterface = content -> {
            byte[] data = content.readAllBytes();
            return buildCmsSignature(data, subFilter);
        };

        doc.addSignature(pdSig, sigInterface);

        ByteArrayOutputStream result = new ByteArrayOutputStream();
        doc.saveIncremental(result);

        log.info("PDF signed successfully — {} bytes, SubFilter={}",
                result.size(), subFilter);
        return result.toByteArray();
    }
}

/**
 * Build the CMS SignedData for the given content bytes.
 */
private byte[] buildCmsSignature(byte[] content, String subFilter)
        throws Exception {

    // Digest the content
    MessageDigest digest = MessageDigest.getInstance("SHA-256");
    byte[] hash = digest.digest(content);

    // Build the CMS generator
    CMSSignedDataGenerator gen = new CMSSignedDataGenerator();

    DigestCalculatorProvider dcProv =
            new JcaDigestCalculatorProviderBuilder()
                    .setProvider("BC").build();

    ContentSigner signer = new JcaContentSignerBuilder(
            "SHA256withRSA").setProvider("BC").build(privateKey);

    JcaSignerInfoGeneratorBuilder signerBuilder =
            new JcaSignerInfoGeneratorBuilder(dcProv);
```

```
        // For PAdES: add ESS-signing-certificate-v2
        if ("ETSI.CAdES.detached".equals(subFilter)) {
            byte[] certHash = MessageDigest.getInstance("SHA-256")
                    .digest(certificateChain[0].getEncoded());
            ESSCertIDv2 certId = new ESSCertIDv2(
                    new AlgorithmIdentifier(
                            NISTObjectIdentifiers.id_sha256),
                    certHash);
            SigningCertificateV2 sigCertV2 =
                    new SigningCertificateV2(
                            new ESSCertIDv2[]{ certId });

            Attribute sigCertAttr = new Attribute(
                    PKCSObjectIdentifiers.id_aa_signingCertificateV2,
                    new DERSet(sigCertV2));

            AttributeTable signedAttrs = new AttributeTable(
                    new Attribute[]{ sigCertAttr });
            signerBuilder.setSignedAttributeGenerator(
                    new DefaultSignedAttributeTableGenerator(signedAttrs));
        }

        gen.addSignerInfoGenerator(
                signerBuilder.build(signer, certificateChain[0]));

        // Add all certificates in the chain
        List<Certificate> certList = new ArrayList<>();
        for (X509Certificate cert : certificateChain) {
            certList.add(cert);
        }
        gen.addCertificates(new JcaCertStore(certList));

        // Generate the CMS SignedData
        CMSProcessableByteArray processable =
                new CMSProcessableByteArray(content);
        CMSSignedData signedData = gen.generate(processable, false);

        // Add timestamp if TSA URL is configured
        if (tsaUrl != null) {
            signedData = addTimestamp(signedData);
        }

        byte[] encoded = signedData.getEncoded();
        log.info("CMS signature built: {} bytes, timestamp={}",
                encoded.length, tsaUrl != null);
        return encoded;
    }

    /**
     * Request a timestamp from the TSA and add it as an unsigned attribute.
     */
    private CMSSignedData addTimestamp(CMSSignedData signedData)
            throws Exception {

        // Hash the signature value
```

```
        byte[] signatureBytes = signedData.getSignerInfos()
                .getSigners().iterator().next().getSignature();
        MessageDigest tsDigest = MessageDigest.getInstance("SHA-256");
        byte[] tsHash = tsDigest.digest(signatureBytes);

        // Build and send the timestamp request
        TimeStampRequestGenerator reqGen =
                new TimeStampRequestGenerator();
        reqGen.setCertReq(true);
        TimeStampRequest tsReq =
                reqGen.generate(TSPAlgorithms.SHA256, tsHash);

        HttpURLConnection conn = (HttpURLConnection)
                new URL(tsaUrl).openConnection();
        conn.setDoOutput(true);
        conn.setRequestMethod("POST");
        conn.setRequestProperty("Content-Type",
                "application/timestamp-query");
        conn.getOutputStream().write(tsReq.getEncoded());

        TimeStampResponse tsResp =
                new TimeStampResponse(conn.getInputStream());
        tsResp.validate(tsReq);
        TimeStampToken tsToken = tsResp.getTimeStampToken();

        log.info("Timestamp obtained — TSA time: {}",
                tsToken.getTimeStampInfo().getGenTime());

        // Embed as unsigned attribute
        Attribute tsAttr = new Attribute(
                PKCSObjectIdentifiers.id_aa_signatureTimeStampToken,
                new DERSet(tsToken.toCMSSignedData().toASN1Structure()));

        return CMSSignedData.replaceUnsignedAttributes(
                signedData,
                new AttributeTable(new Attribute[]{ tsAttr }));
    }
}
```

35.13 VerificationService: Complete Signature Verification

The full verification pipeline from Chapter 31, as a single compilable class.

```
package io.pdfalyzer.verification;

import java.security.MessageDigest;
import java.security.cert.CertPath;
import java.security.cert.CertPathBuilder;
import java.security.cert.CertStore;
import java.security.cert.CertificateFactory;
import java.security.cert.CollectionCertStoreParameters;
import java.security.cert.PKIXBuilderParameters;
import java.security.cert.PKIXCertPathBuilderResult;
import java.security.cert.TrustAnchor;
```

```
import java.security.cert.X509CertSelector;
import java.security.cert.X509Certificate;
import java.time.Clock;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Date;
import java.util.List;
import java.util.Set;

import lombok.extern.slf4j.Slf4j;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;
import org.bouncycastle.asn1.ASN1OctetString;
import org.bouncycastle.asn1.cms.CMSAttributes;
import org.bouncycastle.asn1.cms.Attribute;
import org.bouncycastle.asn1.cms.AttributeTable;
import org.bouncycastle.asn1.pkcs.PKCSObjectIdentifiers;
import org.bouncycastle.cert.X509CertificateHolder;
import org.bouncycastle.cert.jcajce.JcaX509CertificateConverter;
import org.bouncycastle.cms.CMSProcessableByteArray;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.SignerInformation;
import org.bouncycastle.cms.SignerInformationStore;
import org.bouncycastle.cms.jcajce.JcaSimpleSignerInfoVerifierBuilder;
import org.bouncycastle.tsp.TimeStampToken;
import org.bouncycastle.util.Store;

/**
 * Complete five-step signature verification service.
 * Accepts an injectable Clock for deterministic testing.
 */
@Slf4j
public class VerificationService {

    private final Clock clock;

    public VerificationService() {
        this.clock = Clock.systemUTC();
    }

    public VerificationService(Clock clock) {
        this.clock = clock;
    }

    /**
     * Run the full five-step verification pipeline.
     */
    public VerificationReport verify(byte[] pdfBytes,
            PDSignature signature,
            Set<TrustAnchor> trustAnchors) {

        VerificationReport report = new VerificationReport();

        try {
            // Step 1: Byte range
            int[] byteRange = signature.getByteRange();
```

```
        report.setByteRangeResult(
                verifyByteRange(byteRange, pdfBytes.length));

        // Step 2: Hash
        byte[] signedContent = extractSignedContent(
                pdfBytes, byteRange);
        CMSSignedData cmsData = new CMSSignedData(
                new CMSProcessableByteArray(signedContent),
                signature.getContents());
        SignerInformation signerInfo = cmsData.getSignerInfos()
                .getSigners().iterator().next();

        report.setHashResult(verifyHash(
                signedContent, signerInfo));

        // Step 3: CMS
        X509CertificateHolder signerCert =
                verifyCms(cmsData, signerInfo);
        report.setCmsResult(signerCert != null
                ? ValidationResult.OK
                : ValidationResult.CMS_INVALID);

        if (signerCert == null) return report;

        // Check for timestamp
        Date validationTime = Date.from(clock.instant());
        Date tsTime = extractTimestampTime(signerInfo);
        if (tsTime != null) {
            validationTime = tsTime;
            report.setTimestampTime(tsTime);
            report.setTimestampResult(ValidationResult.OK);
        }

        // Step 4: Certificate chain
        report.setChainResult(validateChain(
                signerCert, cmsData, trustAnchors, validationTime));

        // Step 5: Trust
        report.setTrustResult(
                report.getChainResult() == ValidationResult.OK
                        ? ValidationResult.OK
                        : ValidationResult.NOT_TRUSTED);

    } catch (Exception e) {
        log.error("Verification failed", e);
        report.setException(e);
    }

    return report;
}

private byte[] extractSignedContent(byte[] pdfBytes,
        int[] byteRange) {
    byte[] content = new byte[byteRange[1] + byteRange[3]];
    System.arraycopy(pdfBytes, byteRange[0], content,
```

```
            0, byteRange[1]);
    System.arraycopy(pdfBytes, byteRange[2], content,
            byteRange[1], byteRange[3]);
    return content;
}

private ValidationResult verifyByteRange(int[] byteRange,
        long fileSize) {
    if (byteRange == null || byteRange.length != 4) {
        return ValidationResult.BYTE_RANGE_MALFORMED;
    }
    if (byteRange[0] != 0) {
        return ValidationResult.BYTE_RANGE_NOT_AT_START;
    }
    long end = (long) byteRange[2] + byteRange[3];
    if (end != fileSize) {
        return ValidationResult.BYTE_RANGE_TRAILING_BYTES;
    }
    return ValidationResult.OK;
}

private ValidationResult verifyHash(byte[] signedContent,
        SignerInformation signerInfo) throws Exception {
    String algName = mapOidToAlgorithm(
            signerInfo.getDigestAlgOID());
    MessageDigest md = MessageDigest.getInstance(algName);
    byte[] computed = md.digest(signedContent);

    Attribute digestAttr = signerInfo.getSignedAttributes()
            .get(CMSAttributes.messageDigest);
    byte[] claimed = ASN1OctetString.getInstance(
            digestAttr.getAttrValues().getObjectAt(0)).getOctets();

    return MessageDigest.isEqual(computed, claimed)
            ? ValidationResult.OK
            : ValidationResult.HASH_MISMATCH;
}

@SuppressWarnings("unchecked")
private X509CertificateHolder verifyCms(CMSSignedData cmsData,
        SignerInformation signerInfo) throws Exception {
    Store<X509CertificateHolder> certStore =
            cmsData.getCertificates();
    Collection<X509CertificateHolder> matches =
            certStore.getMatches(signerInfo.getSID());
    if (matches.isEmpty()) return null;

    X509CertificateHolder cert = matches.iterator().next();
    boolean valid = signerInfo.verify(
            new JcaSimpleSignerInfoVerifierBuilder()
                    .setProvider("BC").build(cert));
    return valid ? cert : null;
}

private Date extractTimestampTime(SignerInformation signerInfo)
```

```
        throws Exception {
    AttributeTable unsigned = signerInfo.getUnsignedAttributes();
    if (unsigned == null) return null;

    Attribute tsAttr = unsigned.get(
            PKCSObjectIdentifiers.id_aa_signatureTimeStampToken);
    if (tsAttr == null) return null;

    byte[] tsBytes = tsAttr.getAttrValues()
            .getObjectAt(0).toASN1Primitive().getEncoded();
    TimeStampToken tst =
            new TimeStampToken(new CMSSignedData(tsBytes));
    return tst.getTimeStampInfo().getGenTime();
}

private ValidationResult validateChain(
        X509CertificateHolder signerCert,
        CMSSignedData cmsData,
        Set<TrustAnchor> trustAnchors,
        Date validationTime) {
    try {
        JcaX509CertificateConverter converter =
                new JcaX509CertificateConverter().setProvider("BC");
        List<X509Certificate> allCerts = new ArrayList<>();
        for (X509CertificateHolder h :
                cmsData.getCertificates().getMatches(null)) {
            allCerts.add(converter.getCertificate(h));
        }

        X509Certificate target =
                converter.getCertificate(signerCert);
        X509CertSelector selector = new X509CertSelector();
        selector.setCertificate(target);

        PKIXBuilderParameters params =
                new PKIXBuilderParameters(trustAnchors, selector);
        params.addCertStore(CertStore.getInstance("Collection",
                new CollectionCertStoreParameters(allCerts)));
        params.setRevocationEnabled(false);
        params.setDate(validationTime);

        CertPathBuilder.getInstance("PKIX", "BC").build(params);
        return ValidationResult.OK;
    } catch (Exception e) {
        log.error("Chain validation failed: {}", e.getMessage());
        return ValidationResult.CHAIN_INVALID;
    }
}

private static String mapOidToAlgorithm(String oid) {
    return switch (oid) {
        case "1.3.14.3.2.26"          -> "SHA-1";
        case "2.16.840.1.101.3.4.2.1" -> "SHA-256";
        case "2.16.840.1.101.3.4.2.2" -> "SHA-384";
        case "2.16.840.1.101.3.4.2.3" -> "SHA-512";
```

```
            default -> throw new IllegalArgumentException(
                    "Unknown digest OID: " + oid);
        };
    }
}
```

35.14 Signature Appearance Builder

A minimal but complete appearance builder for visible signatures. Generates an appearance stream with signer name, date, and reason inside the signature widget rectangle.

```
package io.pdfalyzer.signing;

import java.io.IOException;
import java.text.SimpleDateFormat;
import java.util.Calendar;

import lombok.extern.slf4j.Slf4j;
import org.apache.pdfbox.cos.COSName;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.PDPage;
import org.apache.pdfbox.pdmodel.PDResources;
import org.apache.pdfbox.pdmodel.common.PDRectangle;
import org.apache.pdfbox.pdmodel.font.PDType1Font;
import org.apache.pdfbox.pdmodel.font.Standard14Fonts;
import org.apache.pdfbox.pdmodel.interactive.annotation.PDAnnotationWidget;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;
import org.apache.pdfbox.pdmodel.interactive.form.PDAcroForm;
import org.apache.pdfbox.pdmodel.interactive.form.PDSignatureField;

/**
 * Builds a visible signature appearance with text content.
 */
@Slf4j
public class AppearanceBuilder {

    /**
     * Create a visible signature field with a text-based appearance.
     *
     * @param doc       the document to add the appearance to
     * @param sig       the PDSignature to associate with the field
     * @param page      the page number (0-based)
     * @param rect      the rectangle for the signature widget
     * @param name      signer's name
     * @param reason    signing reason
     * @param signDate  the signing date
     */
    public static void createVisibleAppearance(
            PDDocument doc, PDSignature sig,
            int page, PDRectangle rect,
            String name, String reason, Calendar signDate)
            throws IOException {

        PDAcroForm acroForm = doc.getDocumentCatalog().getAcroForm();
```

```
if (acroForm == null) {
    acroForm = new PDAcroForm(doc);
    doc.getDocumentCatalog().setAcroForm(acroForm);
}

PDSignatureField sigField = new PDSignatureField(acroForm);
sigField.setValue(sig);
acroForm.getFields().add(sigField);

PDAnnotationWidget widget = sigField.getWidgets().get(0);
widget.setRectangle(rect);
widget.setPage(doc.getPage(page));

// Build appearance content
PDType1Font font = new PDType1Font(
        Standard14Fonts.FontName.HELVETICA);
float fontSize = 8f;
float leading = fontSize * 1.4f;
float margin = 4f;

String dateStr = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss z")
        .format(signDate.getTime());

String[] lines = {
        "Digitally signed by: " + name,
        "Date: " + dateStr,
        "Reason: " + (reason != null ? reason : "-")
};

// Create the appearance stream using form XObject
PDRectangle formRect = new PDRectangle(
        rect.getWidth(), rect.getHeight());

org.apache.pdfbox.pdmodel.graphics.form.PDFormXObject form =
        new org.apache.pdfbox.pdmodel.graphics.form.PDFormXObject(doc);
form.setBBox(formRect);
form.setResources(new PDResources());
form.getResources().put(COSName.getPDFName("F1"), font);

// Write content stream operators directly
StringBuilder cs = new StringBuilder();
cs.append("BT\n");
cs.append("/F1 ").append(fontSize).append(" Tf\n");
cs.append(margin).append(" ")
  .append(rect.getHeight() - margin - fontSize)
  .append(" Td\n");
cs.append(leading).append(" TL\n");

for (String line : lines) {
    cs.append("(").append(escapePdfString(line)).append(") Tj\n");
    cs.append("T*\n");
}
cs.append("ET\n");

form.getStream().createOutputStream().write(
```

```
                cs.toString().getBytes(java.nio.charset.StandardCharsets.US_ASCII));
        form.getStream().createOutputStream().close();

        org.apache.pdfbox.pdmodel.interactive.annotation.PDAppearanceDictionary appearance =
                new org.apache.pdfbox.pdmodel.interactive.annotation.PDAppearanceDictionary();
        appearance.setNormalAppearance(
                new org.apache.pdfbox.pdmodel.interactive.annotation.PDAppearanceEntry(form));
        widget.setAppearance(appearance);

        doc.getPage(page).getAnnotations().add(widget);

        log.info("Created visible signature appearance: {}x{} at ({},{})",
                rect.getWidth(), rect.getHeight(),
                rect.getLowerLeftX(), rect.getLowerLeftY());
    }

    private static String escapePdfString(String input) {
        return input
                .replace("\\", "\\\\")
                .replace("(", "\\(")
                .replace(")", "\\)");
    }
}
```

35.15 Test Keystore Generator: Full Implementation

The complete generator from Chapter 32, including all certificate generation methods.

```
package io.pdfalyzer.test;

import java.math.BigInteger;
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.KeyStore;
import java.security.cert.X509Certificate;
import java.time.Duration;
import java.time.Instant;
import java.util.Arrays;
import java.util.Date;

import lombok.extern.slf4j.Slf4j;
import org.bouncycastle.asn1.ASN1ObjectIdentifier;
import org.bouncycastle.asn1.x500.X500Name;
import org.bouncycastle.asn1.x509.BasicConstraints;
import org.bouncycastle.asn1.x509.ExtendedKeyUsage;
import org.bouncycastle.asn1.x509.Extension;
import org.bouncycastle.asn1.x509.KeyPurposeId;
import org.bouncycastle.asn1.x509.KeyUsage;
import org.bouncycastle.cert.jcajce.JcaX509CertificateConverter;
import org.bouncycastle.cert.jcajce.JcaX509v3CertificateBuilder;
import org.bouncycastle.operator.ContentSigner;
import org.bouncycastle.operator.jcajce.JcaContentSignerBuilder;

/**
```

```
 * Generates a complete PKI hierarchy for testing PDF signatures.
 * Root CA -> Intermediate CA -> Signer
 * Root CA -> TSA
 */
@Slf4j
public class TestKeystoreGenerator {

    private static final int KEY_SIZE = 2048;
    private static final String SIG_ALG = "SHA256withRSA";
    private static long serialCounter = System.currentTimeMillis();

    public static KeyStore generateTestKeystore(char[] password)
            throws Exception {
        // Root CA
        KeyPair rootKp = generateKeyPair();
        X500Name rootDn = new X500Name(
                "CN=Test Root CA, O=PDFalyzer Test, C=DE");
        X509Certificate rootCert = buildCaCert(
                rootKp, rootDn, rootKp, rootDn, 100, true);

        // Intermediate CA
        KeyPair intKp = generateKeyPair();
        X500Name intDn = new X500Name(
                "CN=Test Intermediate CA, O=PDFalyzer Test, C=DE");
        X509Certificate intCert = buildCaCert(
                intKp, intDn, rootKp, rootDn, 50, false);

        // Signer
        KeyPair signerKp = generateKeyPair();
        X500Name signerDn = new X500Name(
                "CN=Test Signer, O=PDFalyzer Test, C=DE");
        X509Certificate signerCert = buildSignerCert(
                signerKp, signerDn, intKp, intDn, 10);

        // TSA
        KeyPair tsaKp = generateKeyPair();
        X500Name tsaDn = new X500Name(
                "CN=Test TSA, O=PDFalyzer Test, C=DE");
        X509Certificate tsaCert = buildTsaCert(
                tsaKp, tsaDn, rootKp, rootDn, 50);

        // Assemble keystore
        KeyStore ks = KeyStore.getInstance("PKCS12");
        ks.load(null, password);
        ks.setKeyEntry("signer", signerKp.getPrivate(), password,
                new X509Certificate[]{ signerCert, intCert, rootCert });
        ks.setKeyEntry("tsa", tsaKp.getPrivate(), password,
                new X509Certificate[]{ tsaCert, rootCert });
        ks.setCertificateEntry("root", rootCert);
        ks.setCertificateEntry("intermediate", intCert);

        log.info("Test keystore generated — 4 entries");
        return ks;
    }
```

```
    private static KeyPair generateKeyPair() throws Exception {
        KeyPairGenerator gen = KeyPairGenerator.getInstance("RSA");
        gen.initialize(KEY_SIZE);
        return gen.generateKeyPair();
    }

    private static X509Certificate buildCaCert(
            KeyPair subjectKp, X500Name subjectDn,
            KeyPair issuerKp, X500Name issuerDn,
            int validityYears, boolean isRoot) throws Exception {

        Date notBefore = new Date();
        Date notAfter = Date.from(Instant.now()
                .plus(Duration.ofDays(365L * validityYears)));

        JcaX509v3CertificateBuilder builder =
                new JcaX509v3CertificateBuilder(
                        issuerDn,
                        BigInteger.valueOf(serialCounter++),
                        notBefore, notAfter,
                        subjectDn,
                        subjectKp.getPublic());

        builder.addExtension(Extension.basicConstraints, true,
                new BasicConstraints(isRoot ? Integer.MAX_VALUE : 0));
        builder.addExtension(Extension.keyUsage, true,
                new KeyUsage(
                        KeyUsage.keyCertSign | KeyUsage.cRLSign));

        ContentSigner signer = new JcaContentSignerBuilder(SIG_ALG)
                .setProvider("BC").build(issuerKp.getPrivate());
        return new JcaX509CertificateConverter().setProvider("BC")
                .getCertificate(builder.build(signer));
    }

    private static X509Certificate buildSignerCert(
            KeyPair subjectKp, X500Name subjectDn,
            KeyPair issuerKp, X500Name issuerDn,
            int validityYears) throws Exception {

        Date notBefore = new Date();
        Date notAfter = Date.from(Instant.now()
                .plus(Duration.ofDays(365L * validityYears)));

        JcaX509v3CertificateBuilder builder =
                new JcaX509v3CertificateBuilder(
                        issuerDn,
                        BigInteger.valueOf(serialCounter++),
                        notBefore, notAfter,
                        subjectDn,
                        subjectKp.getPublic());

        builder.addExtension(Extension.basicConstraints, true,
                new BasicConstraints(false));
        builder.addExtension(Extension.keyUsage, true,
```

```
                new KeyUsage(KeyUsage.digitalSignature
                        | KeyUsage.nonRepudiation));

            ContentSigner signer = new JcaContentSignerBuilder(SIG_ALG)
                    .setProvider("BC").build(issuerKp.getPrivate());
            return new JcaX509CertificateConverter().setProvider("BC")
                    .getCertificate(builder.build(signer));
        }

        private static X509Certificate buildTsaCert(
                KeyPair subjectKp, X500Name subjectDn,
                KeyPair issuerKp, X500Name issuerDn,
                int validityYears) throws Exception {

            Date notBefore = new Date();
            Date notAfter = Date.from(Instant.now()
                    .plus(Duration.ofDays(365L * validityYears)));

            JcaX509v3CertificateBuilder builder =
                    new JcaX509v3CertificateBuilder(
                            issuerDn,
                            BigInteger.valueOf(serialCounter++),
                            notBefore, notAfter,
                            subjectDn,
                            subjectKp.getPublic());

            builder.addExtension(Extension.basicConstraints, true,
                    new BasicConstraints(false));
            builder.addExtension(Extension.keyUsage, true,
                    new KeyUsage(KeyUsage.digitalSignature));
            builder.addExtension(Extension.extendedKeyUsage, true,
                    new ExtendedKeyUsage(KeyPurposeId.id_kp_timeStamping));

            ContentSigner signer = new JcaContentSignerBuilder(SIG_ALG)
                    .setProvider("BC").build(issuerKp.getPrivate());
            return new JcaX509CertificateConverter().setProvider("BC")
                    .getCertificate(builder.build(signer));
        }
    }
```

35.16 ASN.1 Structure Reference

This appendix is a reference card for the ASN.1 structures you will encounter when working with PDF digital signatures. Every CMS `SignedData` blob inside a `/Contents` value, every X.509 certificate in the chain, every OCSP response and timestamp token — they are all ASN.1 structures encoded in DER. When your BouncyCastle code throws a `ParseException` at 2 AM, this is the appendix you reach for.

We present the OID table first (because OIDs are the keys to everything), then the ASN.1 grammar for each major structure, with annotations showing where each field appears in the context of PDF signatures.

35.17 OID Reference Table

Object Identifiers (OIDs) are the universal labels of the ASN.1 world. Every algorithm, every attribute, every extension is identified by an OID. The following table covers the OIDs you will encounter in PDF signature processing:

35.17.1 Digest Algorithms

Table 35.2: Digest algorithm OIDs

OID	Algorithm	Status
1.3.14.3.2.26	SHA-1	Deprecated — do not use for new signatures
2.16.840.1.101.3.4.2.1	SHA-256	Recommended minimum
2.16.840.1.101.3.4.2.2	SHA-384	Recommended for high security
2.16.840.1.101.3.4.2.3	SHA-512	Recommended for high security
2.16.840.1.101.3.4.2.4	SHA-224	Acceptable but uncommon

35.17.2 Signature Algorithms

Table 35.3: Signature algorithm OIDs

OID	Algorithm	Notes
1.2.840.113549.1.1.5	SHA-1 with RSA (sha1WithRSAEncryption)	Deprecated
1.2.840.113549.1.1.11	SHA-256 with RSA (sha256WithRSAEncryption)	Recommended
1.2.840.113549.1.1.12	SHA-384 with RSA (sha384WithRSAEncryption)	Recommended
1.2.840.113549.1.1.13	SHA-512 with RSA (sha512WithRSAEncryption)	Recommended
1.2.840.113549.1.1.10	RSASSA-PSS	Modern RSA padding, preferred over PKCS#1 v1.5
1.2.840.10045.4.3.2	SHA-256 with ECDSA (ecdsa-with-SHA256)	Recommended for ECDSA

OID	Algorithm	Notes
1.2.840.10045.4.3.3	SHA-384 with ECDSA (ecdsa-with-SHA384)	Recommended for ECDSA
1.2.840.10045.4.3.4	SHA-512 with ECDSA (ecdsa-with-SHA512)	Recommended for ECDSA

35.17.3 CMS Attribute OIDs

Table 35.4: CMS signed and unsigned attribute OIDs

OID	Attribute Name	Context
1.2.840.113549.1.9.3	contentType	Signed attr (required)
1.2.840.113549.1.9.4	messageDigest	Signed attr (required)
1.2.840.113549.1.9.5	signingTime	Signed attr (optional, discouraged in PAdES)
1.2.840.113549.1.9.16.2.47	id-aa-signingCertificateV2 (ESS-signing-certificate-v2)	Signed attr (required for PAdES)
1.2.840.113549.1.9.16.2.14	id-aa-signatureTimeStampToken	Unsigned attr (required for PAdES-B-T)
1.2.840.113549.1.9.16.2.25	id-aa-ets-escTimeStamp	Unsigned attr (used in CAdES-X)
1.2.840.113549.1.9.16.2.21	id-aa-ets-certValues	Unsigned attr (used in CAdES-C)
1.2.840.113549.1.9.16.2.23	id-aa-ets-revocationValues	Unsigned attr (used in CAdES-C)

35.17.4 X.509 Extension OIDs

Table 35.5: X.509 v3 extension OIDs relevant to PDF signatures

OID	Extension Name	Critical
2.5.29.15	keyUsage	Yes
2.5.29.37	extendedKeyUsage	Varies
2.5.29.19	basicConstraints	Yes (for CAs)

OID	Extension Name	Critical
2.5.29.35	authorityKeyIdentifier	No
2.5.29.14	subjectKeyIdentifier	No
1.3.6.1.5.5.7.1.1	authorityInfoAccess (AIA)	No
2.5.29.31	cRLDistributionPoints	No
1.3.6.1.5.5.7.3.8	id-kp-timeStamping (EKU value)	N/A (EKU value, not standalone extension)

35.18 CMS SignedData Structure (RFC 5652 — Cryptographic Message Syntax (CMS))

The CMS `SignedData` is the core structure inside every PDF `/Contents` value. This is the complete ASN.1 grammar from RFC 5652 — Cryptographic Message Syntax (CMS) Section 5.1, annotated with PDF-specific notes:

```
-- RFC 5652 Section 5.1
ContentInfo ::= SEQUENCE {
    contentType     ContentType,        -- OID 1.2.840.113549.1.7.2 (signedData)
    content     [0] EXPLICIT ANY        -- The SignedData below
}

SignedData ::= SEQUENCE {
    version             CMSVersion,         -- 1 for most PDF signatures
    digestAlgorithms    DigestAlgorithmIdentifiers,
                                            -- SET OF: usually {sha-256}
    encapContentInfo    EncapsulatedContentInfo,
                                            -- EMPTY for detached signatures (PDF)
    certificates    [0] IMPLICIT CertificateSet OPTIONAL,
                                            -- Signer cert + intermediates
    crls            [1] IMPLICIT RevocationInfoChoices OPTIONAL,
                                            -- Rarely used in PDF CMS
    signerInfos         SignerInfos         -- SET OF: usually exactly one
}

-- In PDF signatures, encapContentInfo.eContent is ABSENT because the
-- signed data (the byte range) is external to the CMS structure.
EncapsulatedContentInfo ::= SEQUENCE {
    eContentType    ContentType,        -- 1.2.840.113549.1.7.1 (id-data)
    eContent    [0] EXPLICIT OCTET STRING OPTIONAL
                                        -- ABSENT for detached signatures
}

SignerInfo ::= SEQUENCE {
    version                 CMSVersion,     -- 1 (issuerAndSerialNumber)
    sid                     SignerIdentifier,
    digestAlgorithm         DigestAlgorithmIdentifier,
    signedAttrs         [0] IMPLICIT SignedAttributes OPTIONAL,
                                            -- MUST be present for PDF sigs
```

```
    signatureAlgorithm      SignatureAlgorithmIdentifier,
    signature               SignatureValue,
    unsignedAttrs       [1] IMPLICIT UnsignedAttributes OPTIONAL
                                            -- Timestamp lives here
}

SignerIdentifier ::= CHOICE {
    issuerAndSerialNumber   IssuerAndSerialNumber,
                                            -- Most common in PDF
    subjectKeyIdentifier [0] SubjectKeyIdentifier
}

IssuerAndSerialNumber ::= SEQUENCE {
    issuer          Name,
    serialNumber    CertificateSerialNumber
}

-- Signed attributes (DER-encoded, then signed)
SignedAttributes ::= SET SIZE (1..MAX) OF Attribute

-- The Attribute structure used for both signed and unsigned attrs
Attribute ::= SEQUENCE {
    attrType    OBJECT IDENTIFIER,      -- The attribute OID
    attrValues  SET OF AttributeValue   -- Usually exactly one value
}
```

For PDF detached signatures, the `eContent` field inside `EncapsulatedContentInfo` is absent. The signed content is the byte range data from the PDF file, which is passed externally to the CMS verifier. This is why you must provide the byte range bytes when constructing a `CMSSignedData` in BouncyCastle — without them, the verifier cannot compute the digest.

35.19 X.509 v3 Certificate Structure (RFC 5280 — Internet X.509 PKI Certificate and CRL Profile)

Every certificate in the CMS `certificates` collection, every CA in the chain, and every TSA certificate follows this structure:

```
-- RFC 5280 Section 4.1
Certificate ::= SEQUENCE {
    tbsCertificate      TBSCertificate,     -- The signed payload
    signatureAlgorithm  AlgorithmIdentifier, -- Must match inner sig alg
    signatureValue      BIT STRING           -- Issuer's signature
}

TBSCertificate ::= SEQUENCE {
    version         [0] EXPLICIT INTEGER DEFAULT v1,    -- v3 (2)
    serialNumber        CertificateSerialNumber,
    signature           AlgorithmIdentifier,    -- Inner copy of sig alg
    issuer              Name,                   -- DN of the issuing CA
    validity            Validity,
    subject             Name,                   -- DN of the certificate owner
    subjectPublicKeyInfo SubjectPublicKeyInfo,
    -- v3 extensions follow:
```

```
    extensions      [3] EXPLICIT Extensions OPTIONAL
}

Validity ::= SEQUENCE {
    notBefore   Time,       -- Certificate valid from this date
    notAfter    Time        -- Certificate valid until this date
}

-- Key extensions for PDF signature certificates:
--
-- keyUsage (2.5.29.15):
--   digitalSignature (bit 0) — required for signing certificates
--   nonRepudiation (bit 1) — alternative to digitalSignature
--   keyCertSign (bit 5) — required for CA certificates
--
-- basicConstraints (2.5.29.19):
--   cA: BOOLEAN — TRUE for CA certs, FALSE (or absent) for end-entity
--   pathLenConstraint: INTEGER — max chain depth below this CA
--
-- extendedKeyUsage (2.5.29.37):
--   id-kp-timeStamping (1.3.6.1.5.5.7.3.8) — required for TSA certs
--
-- authorityInfoAccess (1.3.6.1.5.5.7.1.1):
--   id-ad-caIssuers — URL to fetch the issuer's certificate
--   id-ad-ocsp — URL of the OCSP responder
```

35.20 OCSP Response Structure (RFC 6960 — Online Certificate Status Protocol (OCSP))

OCSP responses are embedded in the DSS (Document Security Store) for PAdES-B-LT and PAdES-B-LTA signatures. The structure you will find in `/DSS /OCSPs`:

```
-- RFC 6960 Section 4.2
OCSPResponse ::= SEQUENCE {
    responseStatus      OCSPResponseStatus,     -- 0 = successful
    responseBytes   [0] EXPLICIT ResponseBytes OPTIONAL
}

ResponseBytes ::= SEQUENCE {
    responseType    OBJECT IDENTIFIER,  -- 1.3.6.1.5.5.7.48.1.1 (basicResponse)
    response        OCTET STRING        -- DER-encoded BasicOCSPResponse
}

BasicOCSPResponse ::= SEQUENCE {
    tbsResponseData     ResponseData,
    signatureAlgorithm  AlgorithmIdentifier,
    signature           BIT STRING,
    certs           [0] EXPLICIT SEQUENCE OF Certificate OPTIONAL
}

ResponseData ::= SEQUENCE {
    version         [0] EXPLICIT INTEGER DEFAULT v1,
    responderID         ResponderID,
```

```
    producedAt          GeneralizedTime,
    responses           SEQUENCE OF SingleResponse,
    responseExtensions  [1] EXPLICIT Extensions OPTIONAL
}

SingleResponse ::= SEQUENCE {
    certID              CertID,
    certStatus          CertStatus,         -- good | revoked | unknown
    thisUpdate          GeneralizedTime,
    nextUpdate      [0] EXPLICIT GeneralizedTime OPTIONAL,
    singleExtensions [1] EXPLICIT Extensions OPTIONAL
}

CertStatus ::= CHOICE {
    good        [0] IMPLICIT NULL,          -- Certificate is not revoked
    revoked     [1] IMPLICIT RevokedInfo,   -- Certificate has been revoked
    unknown     [2] IMPLICIT UnknownInfo    -- Responder has no information
}
```

35.21 Timestamp Token Structure (RFC 3161 — Internet X.509 PKI Time-Stamp Protocol (TSP))

The timestamp token (TST) is the structure inside the `id-aa-signatureTimeStampToken` unsigned attribute. It is itself a CMS `SignedData` containing a `TSTInfo` as the encapsulated content:

```
-- RFC 3161 Section 2.4.2
-- The TimeStampToken is a CMS SignedData where the
-- encapContentInfo contains a TSTInfo:

-- id-ct-TSTInfo OBJECT IDENTIFIER ::= { 1.2.840.113549.1.9.16.1.4 }

TSTInfo ::= SEQUENCE {
    version         INTEGER { v1(1) },
    policy          TSAPolicyId,            -- OID of the TSA's policy
    messageImprint  MessageImprint,         -- Hash of the timestamped data
    serialNumber    INTEGER,                -- Unique per TSA
    genTime         GeneralizedTime,        -- The timestamp time (UTC)
    accuracy        Accuracy OPTIONAL,      -- Time precision
    ordering        BOOLEAN DEFAULT FALSE,
    nonce           INTEGER OPTIONAL,       -- Echoed from request
    tsa         [0] GeneralName OPTIONAL,   -- TSA identity
    extensions  [1] IMPLICIT Extensions OPTIONAL
}

MessageImprint ::= SEQUENCE {
    hashAlgorithm   AlgorithmIdentifier,    -- e.g., SHA-256
    hashedMessage   OCTET STRING            -- Hash of the signature value
}

Accuracy ::= SEQUENCE {
    seconds     INTEGER OPTIONAL,
    millis  [0] INTEGER (1..999) OPTIONAL,
```

```
    micros  [1] INTEGER (1..999) OPTIONAL
}

-- The Timestamp Request (sent to the TSA):
TimeStampReq ::= SEQUENCE {
    version         INTEGER { v1(1) },
    messageImprint  MessageImprint,     -- Hash to timestamp
    reqPolicy       TSAPolicyId OPTIONAL,
    nonce           INTEGER OPTIONAL,   -- Anti-replay
    certReq         BOOLEAN DEFAULT FALSE,
                                        -- Request TSA cert in response
    extensions  [0] IMPLICIT Extensions OPTIONAL
}

-- The Timestamp Response (returned by the TSA):
TimeStampResp ::= SEQUENCE {
    status          PKIStatusInfo,      -- 0 = granted
    timeStampToken  TimeStampToken OPTIONAL
                                        -- CMS SignedData containing TSTInfo
}
```

The `messageImprint` in the TSTInfo is the hash of the *signature value* (the raw bytes of the `SignerInfo.signature` field), not the hash of the signed document. This is a critical distinction. The timestamp proves that a specific signature value existed at a specific time. The signature value, in turn, covers the document bytes (via the signed attributes and byte range). The chain of trust is: timestamp proves signature time, signature proves document integrity.

35.22 Algorithm Identifier Structure

The `AlgorithmIdentifier` appears throughout all the structures above. For most algorithms, the parameters field is either absent or `NULL`:

```
AlgorithmIdentifier ::= SEQUENCE {
    algorithm   OBJECT IDENTIFIER,
    parameters  ANY DEFINED BY algorithm OPTIONAL
}

-- For SHA-256: algorithm = 2.16.840.1.101.3.4.2.1, parameters = NULL (or absent)
-- For RSA:     algorithm = 1.2.840.113549.1.1.11, parameters = NULL
-- For ECDSA:   algorithm = 1.2.840.10045.4.3.2, parameters = absent
-- For RSA-PSS: algorithm = 1.2.840.113549.1.1.10,
--              parameters = RSASSA-PSS-params (complex — see RFC 4055)

RSASSA-PSS-params ::= SEQUENCE {
    hashAlgorithm       [0] HashAlgorithm DEFAULT sha1,
    maskGenAlgorithm    [1] MaskGenAlgorithm DEFAULT mgf1SHA1,
    saltLength          [2] INTEGER DEFAULT 20,
    trailerField        [3] TrailerField DEFAULT trailerFieldBC
}
```

35.23 Glossary & Spec References

This appendix collects every acronym, term, and specification referenced throughout the book into two lookup tables. When you encounter an unfamiliar term in a chapter, this is the place to find a concise definition. When you need the precise specification for a standard we reference, the spec table gives you the document number, title, and where to find it.

35.24 Glossary

Term	Definition
AATL	**Adobe Approved Trust List.** Adobe's curated list of root CA certificates trusted by Acrobat and Reader for signature validation. CAs must apply and meet Adobe's requirements to be listed. If a signer's certificate does not chain to an AATL root, Acrobat shows a warning even if the signature is cryptographically valid.
AcroForm	**Interactive Form Dictionary.** The PDF structure (`/AcroForm` in the document catalog) that contains all form fields, including signature fields. Defined in ISO 32000-2:2020 Section 12.7.
AIA	**Authority Information Access.** An X.509 v3 certificate extension (RFC 5280 — Internet X.509 PKI Certificate and CRL Profile Section 4.2.2.1) that provides URLs for the OCSP responder and the issuer's certificate. Used to fetch missing intermediate certificates during chain building.
ASN.1	**Abstract Syntax Notation One.** A standard for describing data structures used in X.509 certificates, CMS, OCSP, and TSP. Structures are encoded using DER (Distinguished Encoding Rules) for cryptographic applications.
ByteRange	The `/ByteRange` array in a PDF signature dictionary. Specifies which bytes of the PDF file are covered by the signature. Always four integers: `[offset1 length1 offset2 length2]`. See ISO 32000-2:2020 Section 12.8.1.
CA	**Certificate Authority.** An entity that issues X.509 certificates. CAs form a hierarchy: root CAs are self-signed and explicitly trusted; intermediate CAs are signed by a root or another intermediate.
CAdES	**CMS Advanced Electronic Signatures.** ETSI standard (EN 319 122) defining enhanced CMS signature profiles for long-term validity and legal compliance. PAdES builds on CAdES for the CMS portion of PDF signatures.
CMS	**Cryptographic Message Syntax.** The standard (RFC 5652 — Cryptographic Message Syntax (CMS)) for signing and encrypting data. PDF signatures use CMS `SignedData` as the cryptographic payload inside the `/Contents` value. Successor to PKCS#7.

Term	Definition
CRL	**Certificate Revocation List.** A signed list published by a CA containing the serial numbers of revoked certificates. Defined in RFC 5280 — Internet X.509 PKI Certificate and CRL Profile Section 5. Checked during certificate chain validation.
DER	**Distinguished Encoding Rules.** A deterministic binary encoding of ASN.1 structures. All cryptographic ASN.1 (certificates, CMS, OCSP, TSP) uses DER encoding.
DSS	**Document Security Store.** A PDF dictionary (`/DSS` in the document catalog) defined in ISO 32000-2:2020 Section 12.8.4.3 that stores certificates, CRLs, and OCSP responses needed for offline (LTV) validation.
ECDSA	**Elliptic Curve Digital Signature Algorithm.** A signature algorithm based on elliptic curve cryptography. Uses smaller keys than RSA for equivalent security (e.g., 256-bit ECDSA approximates 3072-bit RSA).
eIDAS	**Electronic Identification, Authentication, and Trust Services.** EU Regulation (No 910/2014) that establishes a legal framework for electronic signatures, seals, and timestamps across EU member states. Defines three signature levels: simple, advanced, and qualified.
ESS	**Extended Security Services.** Defined in RFC 2634 and RFC 5035. The `ESS-signing-certificate-v2` attribute is required in PAdES signatures to bind the CMS signature to a specific signer certificate.
ETSI	**European Telecommunications Standards Institute.** The standards body that publishes PAdES (EN 319 142), CAdES (EN 319 122), and the validation standards (TS 119 102) referenced by the eIDAS regulation.
EUTL	**European Union Trusted List.** The official registry of Qualified Trust Service Providers (QTSPs) maintained by EU member states. Analogous to Adobe's AATL but for the EU regulatory framework.
HSM	**Hardware Security Module.** A physical device that stores private keys and performs cryptographic operations. Used in production signing environments to protect key material from extraction.
Incremental Update	A mechanism defined in ISO 32000-2:2020 Section 7.5.6 that appends new or modified objects to the end of a PDF file without rewriting the entire file. Every PDF signature is applied via an incremental update.
LTV	**Long-Term Validation.** The practice of embedding all data needed for signature validation (certificates, CRLs, OCSP responses) within the PDF itself, so the signature can be validated offline, indefinitely, without contacting external servers.

Term	Definition
MDP	**Modification Detection and Prevention.** A PDF mechanism (ISO 32000-2:2020 Section 12.8.2.2) that specifies what changes are permitted after a certification signature is applied. Three permission levels exist (P=1, P=2, P=3).
OCSP	**Online Certificate Status Protocol.** A protocol (RFC 6960 — Online Certificate Status Protocol (OCSP)) for checking certificate revocation status in real time. An OCSP responder returns `good`, `revoked`, or `unknown` for a given certificate serial number.
OID	**Object Identifier.** A globally unique dotted-decimal number (e.g., `1.2.840.113549.1.7.2`) that identifies algorithms, attributes, extensions, and other standardized objects in ASN.1 structures.
PAdES	**PDF Advanced Electronic Signatures.** ETSI standard (EN 319 142) that defines constrained profiles for PDF signatures to meet EU legal requirements. Four baseline profiles: B-B, B-T, B-LT, B-LTA.
PKCS#7	**Public-Key Cryptography Standards #7.** The predecessor to CMS. PDF signatures historically used the term ”PKCS#7” (and the SubFilter `adbe.pkcs7.detached`), though modern implementations use CMS. The terms are often used interchangeably.
PKCS#12	**Public-Key Cryptography Standards #12.** A file format for storing private keys and certificates together, typically with `.p12` or `.pfx` extension. Protected by a password.
PKIX	**Public-Key Infrastructure (X.509).** The set of standards and protocols for managing X.509 certificates and building certificate chains. In Java, `CertPathBuilder` and `CertPathValidator` implement the PKIX algorithm.
QTSP	**Qualified Trust Service Provider.** Under eIDAS, a trust service provider that has been granted qualified status by an EU member state’s supervisory body. Only QTSPs can issue qualified certificates and qualified timestamps.
RSA	**Rivest-Shamir-Adleman.** The most widely used public-key algorithm for digital signatures. Common key sizes for PDF signatures are 2048 and 4096 bits.
SHA-256	**Secure Hash Algorithm, 256-bit.** The recommended minimum hash algorithm for new PDF signatures. Produces a 32-byte (256-bit) digest. Part of the SHA-2 family.
SubFilter	The `/SubFilter` entry in a PDF signature dictionary that specifies the CMS encoding format. Values: `adbe.pkcs7.detached` (classic), `ETSI.CAdES.detached` (PAdES), `adbe.pkcs7.sha1` (deprecated), `adbe.x509.rsa_sha1` (deprecated).
TSA	**Timestamp Authority.** A trusted server that issues RFC 3161 timestamp tokens. The timestamp token proves that specific data existed at a specific point in time.

Term	Definition
TSP	**Time-Stamp Protocol.** The protocol defined in RFC 3161 — Internet X.509 PKI Time-Stamp Protocol (TSP) for requesting and receiving timestamp tokens from a TSA.
TST	**Time-Stamp Token.** The CMS `SignedData` structure returned by a TSA containing a `TSTInfo` with the timestamp time and the hash of the timestamped data.
VRI	**Validation Related Information.** A sub-dictionary of the DSS that groups certificates, CRLs, and OCSP responses for a specific signature. Keyed by the SHA-1 hash of the signature value.
X.509	The ITU-T standard for public-key certificates. Version 3 (X.509 v3) is used universally in PDF signatures. Defined in RFC 5280 — Internet X.509 PKI Certificate and CRL Profile.
XAdES	**XML Advanced Electronic Signatures.** ETSI standard (EN 319 132) for XML-based signatures. Not directly relevant to PDF signatures, but part of the same eIDAS framework as PAdES and CAdES.

35.25 Specification Reference Table

Table 35.6: Standards and specifications referenced in this book

Identifier	Title	Access
ISO 32000-2:2020	Document management — Portable document format — Part 2: PDF 2.0	https://www.iso.org/standard/75839.html (paid) or Adobe's published PDF Reference
RFC 5652	Cryptographic Message Syntax (CMS)	https://www.rfc-editor.org/rfc/rfc5652
RFC 5280	Internet X.509 Public Key Infrastructure Certificate and CRL Profile	https://www.rfc-editor.org/rfc/rfc5280
RFC 3161	Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP)	https://www.rfc-editor.org/rfc/rfc3161
RFC 6960	X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP	https://www.rfc-editor.org/rfc/rfc6960
RFC 2634	Enhanced Security Services (ESS) for S/MIME	https://www.rfc-editor.org/rfc/rfc2634
RFC 5035	Enhanced Security Services (ESS) Update: Adding CertID Algorithm Agility	https://www.rfc-editor.org/rfc/rfc5035

Identifier	Title	Access
RFC 4055	Additional Algorithms and Identifiers for RSA Cryptography with SHA-256, SHA-384, and SHA-512	https://www.rfc-editor.org/rfc/rfc4055
ETSI EN 319 142-1	PAdES: Building blocks and PAdES baseline signatures	https://www.etsi.org/deliver/etsi_en/319100_319199/31914201/
ETSI EN 319 142-2	PAdES: Additional PAdES signatures profiles	https://www.etsi.org/deliver/etsi_en/319100_319199/31914202/
ETSI EN 319 122-1	CAdES: Building blocks and CAdES baseline signatures	https://www.etsi.org/deliver/etsi_en/319100_319199/31912201/
ETSI TS 119 102-1	Electronic Signatures and Infrastructures: Procedures for Creation and Validation	https://www.etsi.org/deliver/etsi_ts/119100_119199/11910201/
ETSI TS 119 102-2	Electronic Signatures and Infrastructures: Signature Validation Report	https://www.etsi.org/deliver/etsi_ts/119100_119199/11910202/
EU Regulation 910/2014	eIDAS: Electronic Identification, Authentication, and Trust Services	https://eur-lex.europa.eu/eli/reg/2014/910/oj
EU Regulation 2024/1183	eIDAS 2.0: Amending Regulation 910/2014 (European Digital Identity Framework)	https://eur-lex.europa.eu/eli/reg/2024/1183/oj
Adobe AATL	Adobe Approved Trust List — Technical Requirements	https://helpx.adobe.com/acrobat/kb/approved-trust-list1.html

The RFCs are freely available and are the definitive reference for CMS, X.509, OCSP, and TSP. The ETSI standards are available free of charge from ETSI's website after registration. ISO 32000-2 is a paid standard, but much of the PDF signature specification is reproduced (with commentary) in this book and in Adobe's freely published PDF Reference documents.